

Open Financial Exchange



Specification 1.6

October 18, 1999

© 1997, 1998, 1999 CheckFree Corp., Intuit Inc., Microsoft Corp. All rights reserved

Open Financial Exchange Specification Legend

Open Financial Exchange Specification ©1996-99 by its publishers: CheckFree Corp., Intuit Inc., and Microsoft Corporation. All rights reserved.

A royalty-free, worldwide, and perpetual license is hereby granted to any party to use the Open Financial Exchange Specification to make, use, and sell products and services that conform to this Specification.

THIS OPEN FINANCIAL EXCHANGE SPECIFICATION IS MADE AVAILABLE “AS IS” WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, MICROSOFT, INTUIT AND CHECKFREE (“PUBLISHERS”) FURTHER DISCLAIM ALL WARRANTIES, INCLUDING WITHOUT LIMITATION ANY IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT, ALL OF WHICH ARE HEREBY DISCLAIMED. THE ENTIRE RISK ARISING OUT OF THE USE OF THIS SPECIFICATION REMAINS WITH RECIPIENT. TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, IN NO EVENT SHALL THE PUBLISHERS OF THIS SPECIFICATION BE LIABLE FOR ANY CONSEQUENTIAL, INCIDENTAL, DIRECT, INDIRECT, SPECIAL, PUNITIVE, OR OTHER DAMAGES WHATSOEVER (INCLUDING, WITHOUT LIMITATION, DAMAGES FOR LOSS OF BUSINESS PROFITS, BUSINESS INTERRUPTION, LOSS OF BUSINESS INFORMATION, OR OTHER PECUNIARY LOSS) ARISING OUT OF ANY USE TO WHICH THIS SPECIFICATION IS PUT, EVEN IF THE PUBLISHERS HEREOF HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

TABLE OF CONTENTS

Chapter 1 Overview	15
1.1 Introduction.....	15
1.1.1 Design Principles	16
1.2 Open Financial Exchange at a Glance	18
1.2.1 Data Transport.....	18
1.2.2 Request and Response Model.....	20
1.3 Definitions	21
1.3.1 User	21
1.3.2 Financial Institution	21
1.3.3 Service Provider	21
1.3.4 Client.....	22
1.3.5 Server	22
1.3.6 Service.....	22
1.3.7 Tag.....	22
1.3.8 Element.....	23
1.3.9 Aggregate.....	23
1.3.10 Request	24
1.3.11 Response.....	24
1.3.12 Message	24
1.3.13 Transaction.....	24
1.3.14 Synchronization.....	24
1.3.15 Message Set	25
1.4 OFX Versions.....	25
1.5 Conventions.....	26
Chapter 2 Structure	29
2.1 HTTP Headers.....	31
2.2 Open Financial Exchange Headers	32
2.2.1 OFXHEADER	33
2.2.2 DATA	33
2.2.3 VERSION	33
2.2.4 SECURITY	34
2.2.5 ENCODING and CHARSET.....	34
2.2.6 COMPRESSION	34
2.2.7 OLDFILEUID and NEWFILEUID	34

2.3 SGML Details.	35
2.3.1 Compliance	35
2.3.2 Valid SGML Characters	35
2.3.3 Comments Not Supported.....	36
2.4 Open Financial Exchange SGML Structure.	36
2.4.1 Overview	36
2.4.2 Case Sensitivity	36
2.4.3 Top Level	37
2.4.4 Messages.....	37
2.4.5 Message Sets and Version Control	39
2.4.6 Transactions.....	43
2.4.7 Synchronization Wrapper	45
2.4.8 Message Set Wrapper	46
2.5 The Signon Message Set.	46
2.5.1 Signon <SONRQ> <SONRS>	46
2.5.2 USERPASS Change <PINCHRQ> <PINCHRS>	54
2.5.3 <CHALLENGERQ> <CHALLENGERS>	56
2.5.4 Signon Message Set Profile Information	57
2.5.5 Examples.....	58
2.6 External Data Support	58
2.7 Extensions to Open Financial Exchange	59
Chapter 3 Common Aggregates, Elements, and Data Types	61
3.1 Common Aggregates	61
3.1.1 Identification of Financial Institutions and Accounts	61
3.1.2 Format of User-Supplied Numbers	61
3.1.3 Echoing in Responses	62
3.1.4 Balance Records <BAL>	63
3.1.5 Error Reporting <STATUS>	64
3.2 Common Elements	65
3.2.1 Client-Assigned Transaction UID <TRNUID>.....	65
3.2.2 Server-Assigned ID <SRVRTID>, <SRVRTID2>	66
3.2.3 Financial Institution Transaction ID <FITID>	67
3.2.4 Token <TOKEN>, <TOKEN2>	68
3.2.5 Transaction Amount <TRNAMT>	68
3.2.6 Memo <MEMO>, <MEMO2>	68
3.2.7 Date Start and Date End <DTSTART> <DTEND>	69
3.2.8 Common Data Types	70
3.2.9 Amounts, Prices, and Quantities	72

3.2.10 Language	73
3.2.11 Other Basic Data Types	73
Chapter 4 OFX Security	75
4.1 Security Concepts in OFX	75
4.1.1 Architecture	75
4.1.2 Security Goals	76
4.1.3 Security Standards	76
4.1.4 FI Responsibilities	77
4.1.5 Security Levels: Channel vs. Application	78
4.2 Security Implementation in OFX	79
4.2.1 Channel-Level Security	79
4.2.2 Application-Level Security	81
Chapter 5 International Support	87
5.1 Language and Encoding	87
5.2 Currency <CURDEF> <CURRENCY> <ORIGCURRENCY>	87
5.3 Country-Specific Element Values.	89
Chapter 6 Data Synchronization	91
6.1 Overview	91
6.2 Background	91
6.3 Data Synchronization Approach	92
6.4 Data Synchronization Specifics	93
6.4.1 Tokens	94
6.4.2 The Synchronization Process	95
6.4.3 Synchronizable Objects	97
6.4.4 <SYNCERROR> Status Codes	98
6.5 Conflict Detection and Resolution	98
6.6 Synchronization Options	99
6.6.1 Synchronization Errors	101
6.7 Typical Server Architecture for Synchronization	101
6.8 Typical Client Processing of Synchronization Results	103
6.9 Simultaneous Connections	104

6.10 Synchronization Alternatives	104
6.10.1 File-Based Error Recovery	105
6.10.2 Lite Synchronization	107
6.10.3 Relating Synchronization and Error Recovery	108
6.11 Examples	109
Chapter 7 FI Profile	113
7.1 Overview	113
7.1.1 Message Sets	113
7.1.2 Version Control	114
7.1.3 Batching and Routing	115
7.1.4 Client Signon for Profile Requests	115
7.1.5 Profile Request <PROFRQ>	116
7.2 Profile Response <PROFRS>	117
7.2.1 Message Set	118
7.2.2 Signon Realms	121
7.2.3 Status Codes	122
7.3 Profile Message Set Profile Information	123
Chapter 8 Activation & Account Information	125
8.1 Overview	125
8.1.1 Sign-Up Message Set V2 and Proxy On-Behalf-Of Actions	125
8.2 Approaches to User Sign-Up with OFX	126
8.2.1 Multiple User Enrollment and Activation	127
8.2.2 Rapid Signup	127
8.3 Users and Accounts	128
8.4 Enrollment and Password Acquisition	128
8.4.1 User IDs	129
8.4.2 Enrollment Request <ENROLLRQ>	129
8.4.3 Enrollment Response <ENROLLRS>	131
8.4.4 Enrollment Status Codes	132
8.4.5 Examples	132
8.5 Account Information	133
8.5.1 Request <ACCTINFORQ>	134
8.5.2 Response <ACCTINFORS>	135
8.5.3 Account Information Aggregate <ACCTINFO>	136
8.5.4 Status Codes	136
8.5.5 Examples	137

8.6 Service Activation.....	138
8.6.1 Activation Request <ACCTRQ>.....	139
8.6.2 Activation Response <ACCTRS>.....	141
8.6.3 Status Codes.....	142
8.6.4 Service Activation Synchronization.....	143
8.6.5 Examples.....	145
8.7 Name and Address Changes.....	146
8.7.1 Change User Information Request <CHGUSERINFORQ>.....	147
8.7.2 Change User Information Response <CHGUSERINFORS>.....	148
8.7.3 Status Codes.....	148
8.7.4 Change User Information Synchronization.....	149
8.8 Signup Message Set Profile Information.....	151
Chapter 9 Customer to FI Communication	155
9.1 The E-Mail Message Set.....	155
9.2 E-Mail Messages.....	155
9.2.1 Regular vs. Specialized E-Mail.....	156
9.2.2 Basic <MAIL> Aggregate.....	156
9.2.3 E-Mail <MAILRQ> <MAILRS>.....	158
9.2.4 E-Mail Synchronization <MAILSYNCRQ> <MAILSYNCRS>.....	160
9.2.5 E-Mail Example.....	161
9.3 Get HTML Page.....	164
9.3.1 MIME Get Request and Response <GETMIMERQ> <GETMIMERS>.....	164
9.3.2 MIME Example.....	165
9.4 E-Mail Message Set Profile Information.....	167
Chapter 10 Recurring Transactions	169
10.1 Creating a Recurring Model.....	169
10.2 Recurring Instructions <RECURRINST>.....	170
10.2.1 Values for <FREQ>.....	170
10.2.2 Examples.....	171
10.3 Retrieving Transactions Generated by a Recurring Model.....	173
10.4 Modifying and Canceling Individual Transactions.....	173
10.5 Modifying and Canceling Recurring Models.....	173
10.5.1 Examples.....	174
10.6 Expired Models.....	176

Chapter 11 Banking	177
11.1 Consumer and Business Banking.....	177
11.2 Credit Card Data.....	177
11.3 Common Banking Aggregates	177
11.3.1 Banking Account <BANKACCTFROM> and <BANKACCTTO>	178
11.3.2 Credit Card Account <CCACCTFROM> and <CCACCTTO>	183
11.3.3 Bank Account Information <BANKACCTINFO>	184
11.3.4 Credit Card Account Information <CCACCTINFO>	185
11.3.5 Transfer Information <XFERINFO>.....	186
11.3.6 Transfer Processing Status <XFERPRCSTS>.....	188
11.4 Downloading Transactions and Balances	189
11.4.1 Bank Statement Download	190
11.4.2 Credit Card Statement Download	192
11.4.3 Statement Transaction <STMTTRN>.....	195
11.5 Statement Closing Information.....	199
11.5.1 Statement Closing Download	199
11.5.2 Non-Credit Card Statement <CLOSING>.....	200
11.5.3 Credit Card Statement Closing Request <CCSTMTENDRQ>.....	202
11.5.4 Credit Card Statement Closing Response <CCSTMTENDRS>	202
11.6 Stop Check.....	205
11.6.1 Stop Check Add.....	206
11.6.2 Status Codes.....	209
11.7 Intrabank Funds Transfer	210
11.7.1 Intrabank Funds Transfer Addition	211
11.7.2 Intrabank Funds Transfer Modification.....	214
11.7.3 Intrabank Funds Transfer Cancellation	217
11.8 Interbank Funds Transfer	219
11.8.1 Interbank Funds Transfer – US.....	219
11.8.2 Interbank Funds Transfer – International Usage	220
11.8.3 Interbank Funds Transfer Modification.....	223
11.8.4 Interbank Funds Transfer Cancellation	226
11.8.5 Multiple Interbank Funds Transfer	228
11.9 Wire Funds Transfer.....	231
11.9.1 Wire Funds Transfer Addition	232
11.9.2 Wire Funds Transfer Cancellation	236

11.10	Recurring Funds Transfer	238
11.10.1	Recurring Intrabank Funds Transfer Addition	238
11.10.2	Recurring Intrabank Funds Transfer Modification	242
11.10.3	Recurring Intrabank Funds Transfer Cancellation	245
11.10.4	Recurring Interbank Funds Transfer Addition	246
11.10.5	Recurring Interbank Funds Transfer Modification	250
11.10.6	Recurring Interbank Funds Transfer Cancellation	253
11.11	E-Mail and Customer Notification.....	255
11.11.1	Banking E-Mail	255
11.11.2	Notifications.....	258
11.11.3	Returned Check and Deposit Notification	259
11.12	Data Synchronization for Banking.....	260
11.12.1	Data Synchronization for Stop Check	261
11.12.2	Data Synchronization for Intrabank Funds Transfers.....	262
11.12.3	Data Synchronization for Interbank Funds Transfers.....	265
11.12.4	Data Synchronization for Wire Funds Transfers	267
11.12.5	Data Synchronization for Recurring Intrabank Funds Transfers	269
11.12.6	Data Synchronization for Recurring Interbank Funds Transfers	271
11.12.7	Data Synchronization for Bank Mail	273
11.13	Message Sets and Profile	275
11.13.1	Message Sets and Messages.....	276
11.13.2	Bank Message Set Profile.....	286
11.13.3	Credit Card Message Set Profile.....	289
11.13.4	Interbank Funds Transfer Message Set Profile.....	290
11.13.5	Wire Transfer Message Set Profile	291
11.14	Examples	292
11.14.1	Statement Download	292
11.14.2	Intrabank Funds Transfer	294
11.14.3	Stop Check	296
11.14.4	Recurring Transfers	299
Chapter 12	Payments.....	311
12.1	Consumer and Business Payments	311
12.2	The Payee Model	311
12.2.1	Payee Identifiers	312
12.2.2	Payee Lists	312
12.2.3	Standard Payee Lists.....	313
12.2.4	Identifying Payees.....	314
12.2.5	Side Effects of Payee Adds and Modifications.....	315

12.3 Identifiers Used in Payment Transactions	315
12.4 The Payment Life Cycle.....	317
12.4.1 Payment Creation	317
12.4.2 Payment Modification	317
12.4.3 Payment Status Inquiry	318
12.4.4 Payment Cancellation.....	318
12.4.5 Delayed Payee Matching	318
12.5 Common Payments Aggregates.....	319
12.5.1 Payments Account Information <BPACCTINFO>	319
12.5.2 Credit Mail Order/Telephone Order Account <CCMOTOACCT>	320
12.5.3 Payment Information <PMTINFO> <PMTINFO2>.....	321
12.6 Payments Functions	334
12.6.1 Payment Creation	336
12.6.2 Payment Modification	339
12.6.3 Payment Cancellation.....	343
12.6.4 Payment Status Inquiry	345
12.7 Recurring Payments.....	346
12.7.1 Creating a Recurring Payment	348
12.7.2 Recurring Payment Modification	351
12.7.3 Recurring Payment Cancellation	355
12.8 Payment Mail	357
12.8.1 Payment Mail Request and Response	357
12.8.2 Payment Mail Synchronization.....	360
12.9 Payee Lists	361
12.9.1 Adding a Payee to the Payee List	363
12.9.2 Payee Modification	365
12.9.3 Payee Deletion	369
12.9.4 Payee List Synchronization	371
12.10 Data Synchronization for Payments	373
12.10.1 Payment Synchronization	374
12.10.2 Recurring Payment Synchronization	376
12.10.3 Discussion.....	378
12.11 Message Sets and Profile	379
12.11.1 Bill Pay Message Sets and Messages	380
12.11.2 Bill Pay Message Set Profile <BILLPAYMSGSET>	384
12.12 Examples	389
12.12.1 Scheduling a Payment	389
12.12.2 Modifying a Payment.....	393

12.12.3 Canceling a Payment	397
12.12.4 Updating Payment Status	398
12.12.5 Scheduling a Recurring Payment	399
12.12.6 Modifying a Recurring Payment	401
12.12.7 Canceling a Recurring Payment	403
12.12.8 Adding a Payee to the Payee List	404
12.12.9 Synchronizing Scheduled Payments	406
Chapter 13 Investments	409
13.1 Types of Response Information	410
13.2 Sub-Accounts	410
13.3 Units, Precision, and Signs	410
13.3.1 Units	410
13.3.2 Precision	411
13.3.3 Signs	411
13.4 Bank and Investment Transactions	412
13.5 Money Market Funds	412
13.5.1 Separate Account at the Financial Institution	412
13.5.2 Sweep Account Within an Investment Account	413
13.5.3 Position Within an Investment Account	413
13.6 Investment Accounts	413
13.6.1 Specifying the Investment Account <INVACCTFROM>	413
13.6.2 Investment Account Information <INVACCTINFO>	414
13.6.3 Brokerage, Mutual Fund, and 401K Accounts	415
13.7 Investment Message Sets and Profile	416
13.7.1 Investment Statement Download	417
13.7.2 Security Information	420
13.8 Investment Securities	423
13.8.1 Security Identification <SECID>	423
13.8.2 Security List Request	423
13.8.3 Security List Response	425
13.8.4 Security List <SECLIST>	426
13.8.5 Securities Information	426
13.9 Investment Statement Download	432
13.9.1 Investment Statement Request	432
13.9.2 Investment Statement Response	434

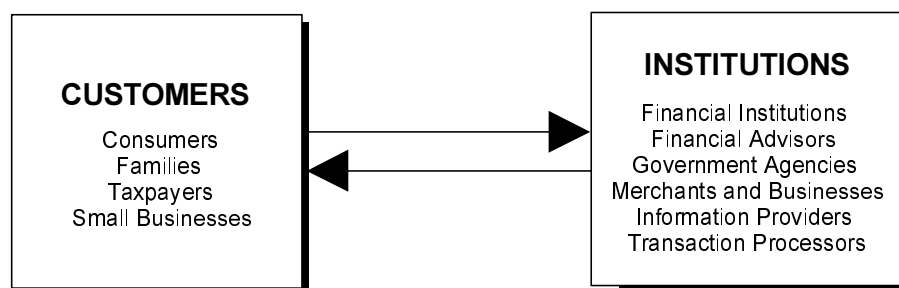
13.10 Investment E-Mail	455
13.10.1 Investment E-Mail Request and Response	455
13.10.2 Investment E-Mail Synchronization.....	457
13.11 Complete Example	459
Chapter 14 Bill Presentment	465
14.1 Overview	465
14.1.1 Bill Presentment Model	465
14.1.2 Servers and Message Sets.....	465
14.2 Biller Directory	466
14.2.1 Client Signon to the Biller Directory Server	466
14.2.2 Search Arguments.....	466
14.2.3 Identification of Bill Publishers.....	467
14.2.4 Find Biller Request <FINDBILLERRQ>.....	467
14.2.5 Find Biller Response <FINDBILLERRS>.....	469
14.2.6 Status Codes <FINDBILLERRS>	471
14.2.7 Account Number Validation	472
14.2.8 Biller Payment Restrictions	473
14.3 Customer Signup	474
14.3.1 Enrollment	475
14.3.2 Account Inquiry.....	475
14.3.3 Service Activation	479
14.3.4 Service Status Update for Groups of Customers	480
14.4 Bill Delivery.....	484
14.4.1 Bill Delivery Process.....	484
14.4.2 Bill List Retrieval.....	484
14.4.3 Bill Detail Retrieval.....	498
14.4.4 Table Structure Definition	503
14.4.5 Delivery Notification	505
14.4.6 Bill Status Modification	508
14.5 Bill Payment.....	509
14.5.1 Remittance Information	509
14.5.2 Payee Identification.....	509
14.6 Bill Presentment E-Mail	510
14.6.1 Bill Presentment Mail Request <PRESMAILRQ>	511
14.6.2 Bill Presentment Mail Response <PRESMAILRS>.....	511
14.6.3 Status Codes <PRESMAILRS>	512

14.6.4 Request <PRESMAILSYNCRQ>	513
14.6.5 Response <PRESMAILSYNCRS>.....	514
14.7 Message Sets and Profile	515
14.7.1 Message Sets and Messages.....	515
14.7.2 Biller Directory Message Set Profile.....	519
14.7.3 Bill Delivery Message Set Profile	519
14.8 Bill Presentment Examples	521
14.8.1 Find Biller Examples.....	521
14.8.2 Enrollment Examples	528
14.8.3 Activation Example	530
14.8.4 Bill Delivery Examples.....	532
Appendix A Status Codes.....	541
Appendix B Change History.....	549
B.1 OFX 1.0 to 1.0.1.....	549
B.1.1 Specification Changes by Chapter	549
B.1.2 General Specification Changes.....	569
B.1.3 DTD Changes	569
B.2 OFX 1.0.1 to 1.0.2	571
B.2.1 Specification Changes by Chapter	571
B.2.2 General Specification Changes.....	577
B.2.3 DTD Changes	577
B.3 OFX 1.0.2 to 1.5.....	579
B.3.1 Specification Changes by Chapter	579
B.4 OFX 1.5 to OFX 1.5.1	593
B.4.1 Specification Changes by Chapter	593
B.4.2 DTD Changes	600
B.5 OFX 1.5.1 to 1.6.....	601
B.5.1 Specification Changes by Chapter	601
B.5.2 DTD Changes	621
Appendix C Recent Changes that Affect Earlier Versions	623
C.1 Changes Made in OFX 1.5	623
C.1.1 Specification Changes by Chapter.....	623
C.2 OFX 1.5 to OFX 1.5.1	630
C.2.1 Specification Changes by Chapter.....	630
C.2.2 DTD Changes	631
C.3 OFX 1.5.1 to 1.6.....	632
C.3.1 Specification Changes by Chapter.....	632
C.3.2 DTD Changes	643

CHAPTER 1 OVERVIEW

1.1 Introduction

Open Financial Exchange is a broad-based framework for exchanging financial data and instructions between customers and their financial institutions. It allows institutions to connect directly to their customers without requiring an intermediary.



Open Financial Exchange is an open specification that anyone can implement: any financial institution, transaction processor, software developer, or other party. It uses widely accepted open standards for data formatting (such as SGML), connectivity (such as TCP/IP and HTTP), and security (such as SSL).

Open Financial Exchange defines the request and response messages used by each financial service as well as the common framework and infrastructure to support the communication of those messages. This specification does not describe any specific product implementation.

1.1.1 Design Principles

The following principles were used in designing Open Financial Exchange:

- ◆ **Broad Range of Financial Activities** – Open Financial Exchange provides support for a **broad** range of financial activities. Open Financial Exchange 1.6 specifies the following services:
 - ◆ Bank statement download
 - ◆ Credit card statement download
 - ◆ Funds transfers including recurring transfers
 - ◆ Consumer payments, including recurring payments
 - ◆ Business payments, including recurring payments
 - ◆ Brokerage and mutual fund statement download, including transaction history, current holdings, and balances.
 - ◆ Bill presentment and payment
- ◆ **Broad Range of Financial Institutions** – Open Financial Exchange supports communication with a **broad** range of financial institutions (FIs), including:
 - ◆ Banks
 - ◆ Brokerage houses
 - ◆ Merchants
 - ◆ Processors
 - ◆ Financial advisors
 - ◆ Government agencies
- ◆ **Broad Range of Front-End Applications** – Open Financial Exchange supports a **broad** range of front-end applications, including Web-based applications, covering all types of financial activities running on all types of platforms.
- ◆ **Extensible** – Open Financial Exchange has been designed to allow the easy addition of new services. Future versions will include support for many new services.
- ◆ **Open** – This specification is publicly available. You can build client and server applications using the Open Financial Exchange protocols independent of any specific technology, product, or company.
- ◆ **Multiple Client Support** – Open Financial Exchange allows a user to use multiple client applications to access the same data at a financial institution. With the popularity of the World Wide Web, customers are increasingly more likely to use multiple applications—either desktop-based or Web-based—to perform financial activities. For example, a customer can track personal finances at home with a desktop application and occasionally pay bills while at work with a Web-based application. The use of data synchronization to support multiple clients is a key innovation in Open Financial Exchange.

- ◆ **Robust** – Open Financial Exchange will be used for executing important financial transactions and for communicating important financial information. Assuring users that transactions are executed and information is correct is crucial. Open Financial Exchange provides robust protocols for error recovery.
- ◆ **Secure** – Open Financial Exchange provides a framework for building secure online financial services. In Open Financial Exchange, security encompasses authentication of the parties involved, as well as secrecy and integrity of the information being exchanged.
- ◆ **Batch & Interactive** – The design of request and response messages in Open Financial Exchange is for use in either batch or interactive style of communication. Open Financial Exchange provides for applying a single authentication context to multiple requests in order to reduce the overhead of user authentication.
- ◆ **International Support** – Open Financial Exchange is designed to supply financial services throughout the world. It supports multiple currencies, country-specific extensions, and different forms of encoding such as UNICODE.
- ◆ **Platform Independent** – Open Financial Exchange can be implemented on a wide variety of front-end client devices, including those running Windows 3.1, Windows 95, Windows NT, Macintosh, or UNIX. It also supports a wide variety of Web-based environments, including those using HTML, Java, JavaScript, or ActiveX. Similarly on the back-end, Open Financial Exchange can be implemented on a wide variety of server systems, including those running UNIX, Windows NT, or OS/2.
- ◆ **Transport Independent** – Open Financial Exchange is independent of the data communication protocol used to transport the messages between the client and server computers. Open Financial Exchange 1.0.2, 1.5.1, and 1.6 use HTTP.

1.2 Open Financial Exchange at a Glance

The design of Open Financial Exchange is as a client and server system. An end-user uses a client application to communicate with a server at a financial institution. The form of communication is requests from the client to the server and responses from the server back to the client.

Open Financial Exchange uses the Internet Protocol (IP) suite to provide the communication channel between a client and a server. IP protocols are the foundation of the public Internet and a private network can also use them.

1.2.1 Data Transport

Clients use the HyperText Transport Protocol (HTTP) to communicate to an Open Financial Exchange server. The World Wide Web throughout uses the same HTTP protocol. In principle, a financial institution can use any off-the-shelf web server to implement its support for Open Financial Exchange.

To communicate by means of Open Financial Exchange over the Internet, the client must establish an Internet connection. This connection can be a dial-up Point-to-Point Protocol (PPP) connection to an Internet Service Provider (ISP) or a connection over a local area network that has a gateway to the Internet.

Clients use the HTTP POST command to send a request to the previously acquired Uniform Resource Locator (URL) for the desired financial institution. The URL presumably identifies a Common Gateway Interface (CGI) or other process on an FI server that can accept Open Financial Exchange requests and produce a response.

The POST identifies the data as being of type application/x-ofx. Use application/x-ofx as the return type as well. Fill in other fields per the HTTP 1.0 specification. Here is a typical request:

```
POST http://www.fi.com/ofx.cgi HTTP/1.0
```

HTTP headers

```
User-Agent:MyApp 5.0
```

```
Content-Type: application/x-ofx
```

```
Content-Length: 1032
```

```
OFXHEADER:100
```

OFX headers

```
DATA:OFXSGML
```

```
VERSION:160
```

```
SECURITY:TYPE1
```

```
ENCODING:USASCII
```

```
CHARSET:NONE
```

```
COMPRESSION:NONE
```

```
NEWFILEUID:NONE
```

```
OLDFILEUID:NONE
```

```
<OFX>
```

OFX request

```
... Open Financial Exchange requests ...
```

```
</OFX>
```

A blank line defines the separation between the HTTP headers and the start of the Open Financial Exchange headers. A blank line also separates the Open Financial Exchange headers and the request. (See Chapter 2, “Structure” for more information about the Open Financial Exchange headers.) A brief note here that a “blank line” means a carriage return and a linefeed pair – CR/LF.

The structure of a response is similar to the request, with the first line containing the standard HTTP result, as shown next. The content length is given in bytes.

```
HTTP 1.0 200 OK
```

HTTP headers

```
Content-Type: application/x-ofx
```

```
Content-Length: 8732
```

```
OFXHEADER:100
```

OFX headers

```
DATA:OFXSGML
```

```
VERSION:160
```

```
SECURITY:TYPE1
```

```
ENCODING:USASCII
```

```
CHARSET:NONE
```

```
COMPRESSION:NONE
```

```
NEWFILEUID:NONE
```

```
OLDFILEUID:NONE
```

```
<OFX>
```

OFX response

```
... Open Financial Exchange responses ...
```

```
</OFX>
```

1.2.2 Request and Response Model

The basis for Open Financial Exchange is the request and response model. One or more requests can be batched in a single file. This file typically includes a signon request and one or more service-specific requests. An FI server will process all of the requests and return a single response file. This batch model lends itself to Internet transport as well as other off-line transports. Both requests and responses are plain text files, formatted using a grammar based on Standard Generalized Markup Language (SGML). Open Financial Exchange is syntactically similar to HyperText Markup Language (HTML), featuring tags to identify and delimit the data. The use of a tagged data format allows Open Financial Exchange to evolve over time while continuing to support older clients and servers.

Here is a simplified example of an Open Financial Exchange request file. (This example does not show the Open Financial Exchange headers and the indentation is only for readability.) For complete details, see the more complete examples throughout this specification.

```
<OFX>                                <!-- Begin request data -->
  <SIGNONMSGSRQV1>
    <SONRQ>                            <!-- Begin signon -->
      <DTCLIENT>19961029101000 <!-- Oct. 29, 1996, 10:10:00 am -->
      <USERID>123-45-6789      <!-- User ID (that is, SSN) -->
      <USERPASS>MyPassword    <!-- Password (SSL encrypts whole) -->
      <LANGUAGE>ENG           <!-- Language used for text -->
      <FI>                     <!-- ID of receiving institution -->
        <ORG>NCH               <!-- Name of ID owner -->
        <FID>1001              <!-- Actual ID -->
      </FI>
      <APPID>MyApp
      <APPVER>0500
    </SONRQ>                      <!-- End of signon -->
  </SIGNONMSGSRQV1>

  <BANKMSGSRQV1>
    <STMTTRNRQ>                    <!-- First request in file -->
      <TRNUID>1001
      <STMTRQ>                     <!-- Begin statement request -->
        <BANKACCTFROM>            <!-- Identify the account -->
          <BANKID>121099999      <!-- Routing transit or other FI ID -->
          <ACCTID>999988         <!-- Account number -->
          <ACCTTYPE>CHECKING     <!-- Account type -->
        </BANKACCTFROM>          <!-- End of account ID -->
        <INCTRAN>                <!-- Begin include transaction -->
          <INCLUDE>Y              <!-- Include transactions -->
```

<code></INCTRAN></code>	<code><!-- End of include transaction --></code>
<code></STMTRQ></code>	<code><!-- End of statement request --></code>
<code></STMTTRNRQ></code>	<code><!-- End of first request --></code>
<code></BANKMSGSRQV1></code>	
<code></OFX></code>	<code><!-- End of request data --></code>

The response format follows a similar structure. Although a response, such as a statement response, contains all of the details of each transaction, each individual detail of the statement is identified using tags.

The key rule of Open Financial Exchange syntax is that each tag is either an element or an aggregate. Data follows its element tag. An aggregate tag begins a compound tag sequence, which must end with a matching tag; for example, `<AGGREGATE> ... </AGGREGATE>`.

The file sent by Open Financial Exchange does not require any white space between tags.

White space following a tag delimiter (`>`), following an element value, or preceding a tag delimiter (`<`) should be ignored. White space within an element value (i.e. not preceding, not following) is significant. If white space is desired preceding or following an element value, this is achieved using the CDATA wrapper or, in version 2 message sets and Bill Presentment, the ` ` macro. If more than one white space element is needed, then multiple ` ` macros should be utilized. See section [2.3.2.1](#).

1.3 Definitions

The following sections detail definitions that hold within the context of OFX.

1.3.1 User

User refers to the person or entity interfacing with the OFX *client* to cause it to generate OFX requests.

1.3.2 Financial Institution

Financial Institution (FI) refers to the institution with which the user has a direct relationship. Generally this means a bank, but in many cases it may be an institution providing non-banking financial services.

1.3.3 Service Provider

Service Provider (SP) refers to an institution with which the user does *not* have a direct relationship. Generally, such an institution is subcontracted by the FI to provide specific services to the customer on behalf of the FI.

1.3.4 Client

An OFX client is the software that generates OFX *requests*, receives *responses* and processes them. This may be a personal finance manager, a web browser running locally interactive code (such as with a Java applet or ActiveX control), a Web server, a proxy, or one of many other possibilities.

1.3.5 Server

An OFX *server* is the software that receives OFX requests, processes them, and generates OFX responses.

1.3.6 Service

A *service* is a collection of related *transactions*. For example, the BANKSVC service encompasses banking transactions such as requesting bank statements, initiating stop checks, initiating wire transfers, etc.

In OFX 1.x, services are used directly only when describing or changing the general options available to a particular customer. Other collections of transactions instead use the concept of Message Sets as described in [section 1.3.15](#).

1.3.7 Tag

Tag is the generic name for either a start tag or an end tag. A *start tag* consists of an *element* or *aggregate* name surrounded by angle brackets. An *end tag* is the same as a start tag, with the addition of a forward slash immediately preceding the name. For example, the start tag for the aggregate named FOO looks like this:

<FOO>

The end tag for the same aggregate looks like this:

</FOO>

1.3.8 Element

An OFX document contains one or more *elements*. An element is some data bounded by a leading start tag and a trailing end tag, end of the containing aggregate or start of the next element. For example, an element named BAZ, containing data “bar,” could look like this:

```
<BAZ>bar</BAZ><!-- An element ended by its own end tag-->
```

or:

```
<BAZ>bar</F00><!-- An element ended by the required end tag of the  
aggregate that contains it -->
```

or:

```
<BAZ>bar<DIFFER>something<!-- An element ended by the start tag of the  
following element -->
```

Note that this definition differs slightly from the SGML definition of element, in that an OFX element must contain data (not just white space) and may *not* contain other elements. Elements in OFX are optional wherever the content might otherwise be missing. An SGML element containing other elements is defined in OFX as an *aggregate*. OFX specifically disallows empty elements and elements with *mixed* content.

With the exception of <MSGBODY> and <EXTDPMTDSC>, end tags are optional for all OFX elements.

1.3.9 Aggregate

An *aggregate* is a collection of elements and /or other aggregates. An aggregate may not contain any data itself, but rather contains elements containing data, and /or recursively contains aggregates.

OFX includes very few empty aggregates and clients and servers should not send an aggregate without content. In general, the entire aggregate should be left out of a request or response file when its (optional) content is missing. The few exceptions to these rules (such as <SECLISTR>, described in section [13.8.3.3](#)) are called out in the relevant sections of this document.

End tags are required for all OFX aggregates.

1.3.10 Request

A *request* is information sent by the client. An OFX *request file* is the entire SGML file sent by the client, including the OFX headers. An *individual request* generally is an aggregate whose name ends in RQ.

1.3.11 Response

A *response* is information sent by the server. An OFX *response file* is the entire SGML file sent by the server, including the OFX headers. An *individual response* generally is an aggregate whose name ends in RS.

When elements and aggregates from the request also appear in the corresponding response they are generally intended to echo the values from a request in the response (this enables client matching with the request, for example). While the server should not modify data in individual elements when echoing, elements not found in a particular request may be added in the response. These situations (such as adding a <PAYEELSTID> when creating a <PMTRQ> response) are described as they arise. OFX also includes a few specific situations requiring different information to be sent and returned in corresponding elements of a request / response pair. Again, these exceptions (such as the <TOKEN> element in a sync request and response) are described as they arise.

1.3.12 Message

A *message* is the unit of work in OFX. It refers to a request and response pair. For example, the message to download a bank statement consists of the request <STMTRQ> and the response <STMTRS>.

1.3.13 Transaction

A *transaction* consists of a message and its associated transaction wrappers. The transaction request wrapper contains a unique transaction identifier used to prevent ambiguity in matching a particular response to its associated request, and the request aggregate. The transaction response wrapper contains a status aggregate, the transaction identifier sent in the request, and (if the transaction was successful) the response aggregate. For details on the use of transaction wrappers, see section [2.4.6](#).

1.3.14 Synchronization

For messages subject to synchronization (see [Chapter 6, "Data Synchronization"](#)), an added layer of aggregates is also part of a message definition: a synchronization request and response. These add a token and, in some cases, other information. Synchronization requests may encapsulate *embedded transactions* that execute only when certain conditions are true at the server (either the

containing synchronization request completed without error or the request had no errors and the client was up to date).

1.3.15 Message Set

Message sets are collections of messages. Generally they form all or part of a *service* (as defined in section [1.3.6](#)). OFX utilizes these smaller groupings when wrapping request or response transactions, profiling server support for the wrappers and describing individual messages. The BANKSVC service, for example, is broken into the BANKMSGSET, CREDITCARDMSGSET, INTERXFERMSGSET and WIREXFERMSGSET message sets.

Please refer to section [2.4.5](#) , "[Message Sets and Version Control](#)" for additional information about message sets.

1.4 OFX Versions

This document specifies three distinct versions of OFX clients and servers.

Version 1.0.2 supports any or all version 1 message sets except Bill Presentment. These message sets are defined by the OFX 1.0.2 Document Type Definition (DTD), which is used for parsing. Applications that conform to this version are referred to as 1.0.2 clients and 1.0.2 servers.

Version 1.5.1 supports all version 2 message sets, Bill Presentment, and all version 1 message sets. Because it supports all message sets, the OFX 1.5.1 DTD can be used to create and support OFX 1.0.2 and /or OFX 1.5.1 clients and servers.

Version 1.6 DTD supports all message sets available in the OFX 1.5.1 DTD. It adds specific enhancements to some of the aggregates. All of those enhancements are optional and should not be used by a client unless the server indicates support in its FI Profile. Applications that conform to this version are referred to as 1.6 clients and 1.6 servers. The OFX 1.6 DTD fully incorporates the OFX 1.0.2 and 1.5.1 message sets, so it can be used to support both 1.0.2 and 1.5.1 applications.

For a complete description of OFX message sets, see section [2.4.5.3](#).

As of the publication of this document, only versions 1.0.2, 1.5.1, and 1.6 of OFX are supported. This document completely describes the supported OFX versions. Other, previous versions and documents (specifications and errata) are obsolete. For a list of changes between versions, see [Appendix B, "Change History."](#)

1.5 Conventions

The conventions used in the element and aggregate descriptions include the following:

- ◆ Required elements and aggregates are in **bold**. Regular face indicates elements and aggregates that are optional. Required means that a client must always include the element or aggregate in a request, and a server must always include the element or aggregate in a response.
- ◆ Required elements and aggregates occur once unless noted as one or more in the description, in which case the specification allows multiple occurrences.
- ◆ Optional elements and aggregates occur once if present unless noted as zero or more in the description, in which case the specification allows multiple occurrences.
- ◆ Character fields are identified with a data type of “A-*n*”, where *n* is the maximum number of allowed Unicode characters.

Note: *n* refers to the number of characters in the resultant string. Each multi-byte or encoded character counts as a single character. UTF-8 encodes “high” Latin-1 characters (decimal 128-255) using two bytes, and double-byte characters using three bytes. In addition, SGML encodes ampersands, less-than symbols, greater-than symbols, and spaces (where required) using multi-character escape strings (see section 2.3.2.1). Therefore, an element of type A-40 may require more than 40 bytes in a UTF-8-encoded SGML stream.

- ◆ N-*n* identifies an element of numeric type where *n* is the maximum number of characters in the value. Values of this type are generally whole numbers, but the data type allows negative numbers. OFX includes a few fixed-position numeric values (such as <APPVER>, see section 2.5.1.1) called out in the text. In all cases, elements of this type may contain only the characters 0 through 9 and - (hyphen, the negative sign indicator). So an element of type “N-6” may take values from -99999 to 999999. The value “0000000” would be illegal for an N-6 element. White space is not allowed within the numeric value. Leading zeroes are allowed, but discouraged except where noted in the text. For example, a <MIN> element containing zero might be sent as “<MIN>0”, “<MIN>00”, “<MIN> 0”, but not “<MIN>0 0”.
- ◆ Common value types, such as a dollar amount, are referenced by name. Chapter 3, “Common Aggregates, Elements, and Data Types” lists value types that are referenced by name.
- ◆ In some aggregates, there are different elements and aggregates used in different message set versions. In those aggregates, there is a third column in the table, called Version. In that column, for any elements and aggregates that differ across message set versions, there is a description of the difference. For example, many elements and aggregates say “V2 only” to indicate that they are in message set version 2 only. Features added with OFX 1.6 are annotated “1.6 add” and should be requested only when the server indicates support for that feature in its FI Profile (see Chapter 7, “FI Profile”). Most “1.6 add” features (all but <REFRESHSUPT>, see section 7.2.1) are also “V2 only” or part of the new <PRESDLVMSGSETV1>.

- ◆ Explanatory information is in *italics*

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<REQUIRED>		Required element or aggregate (1 or more)
<REQUIRED2>		Required element or aggregate that occurs only once
<OPTIONAL>		Optional element or aggregate; this element or aggregate can occur multiple times (0 or more)
<SPECIFIC>		Values are A, B, and C
<ALPHAVALUE>		Takes a value up to 32 characters in length, A-32
<NEW>	V2 only	Element or aggregate used only in version 2 of this message set
<NEW2>	1.6 add	Element or aggregate should be used only when the server indicates support for this option in its FI Profile (see Chapter 7, "FI Profile"). Most "1.6 add" features (all but <REFRESHSUPT>, see section 7.2.1) are also "V2 only" or part of the new <PRESDLVMSGSETV1>.
<i>Explanatory text</i>		Hopefully useful information.

CHAPTER 2 STRUCTURE

This chapter describes the basic structure of an Open Financial Exchange request and response. Structure includes headers, basic syntax, and the Signon request and response. This chapter also describes how Open Financial Exchange encodes external data, such as bit maps.

Open Financial Exchange data consists of some headers plus one Open Financial Exchange data block. This block consists of a signon message and zero or more additional messages. When sent over the Internet using HTTP, standard HTTP and (optionally) multipart MIME headers and formats surround the Open Financial Exchange data. A simple file that contained only Open Financial Exchange data would have the following form:

HTTP headers

MIME type application/x-ofx

Open Financial Exchange headers

Open Financial Exchange SGML block

A more complex file that contained additional Open Financial Exchange data would have this form:

HTTP headers

MIME type multipart/x-mixed-replace; boundary =XYZZY24x7

--XYZZY24x7

MIME type application/x-ofx

Open Financial Exchange headers

Open Financial Exchange SGML block

--XYZZY24x7

MIME type image/jpeg

FI logo

--XYZZY24x7--

Version 1.0.2 of the Open Financial Exchange specification did not specify how to properly separate the various components of an OFX request. In particular, separation of the HTTP headers, the MIME attachments, the OFX headers, the OFX header elements, and the OFX SGML block.

OFX 1.0.2 clients used a mix of LF and CRLF constructs and OFX 1.0.2 servers handled either linefeed (LF) or carriage return /line feed (CRLF), but not often both. In the future, it is expected that 1.0.2 servers will be upgraded to handle both CRLF and LF.

In version 1.5 and later of the Open Financial Exchange specification, the behavior for both an OFX client and an OFX server has been specified to encourage uniform usage of the specification.

OFX 1.6 Client

A proper client should separate the components of an OFX request using a single CRLF between each component. A proper request thus has the form:

HTTP headers
CRLF(s)
MIME type information
CRLF(s)
OFX header element 1
CRLF
OFX header element 2
CRLF
OFX header element <i>n</i>
CRLF(s)
OFX SGML Block

OFX 1.6 Server

An OFX 1.6 specification server should expect OFX request components and elements to be separated by the appropriate number of CRLF characters. However, as per W3C recommendations, an OFX 1.6 server should also accept just a LF as a separator. This behavior is as per the recommendation of the World Wide Web Consortium (W3C). The W3C is the worldwide standards body for web technology.

<http://www.w3.org>

(W3C home page)

<http://www.w3.org/Protocols/HTTP/OldClients.html>

(W3C recommendations)

The text has been included below for ease of reference:

Note: Server tolerance of bad clients.

Whilst it is seen appropriate for testing parsers to check full conformance to this specification, it is recommended that operational parsers be tolerant of deviations.

In particular, lines should be regarded as terminated by the Line Feed, and the preceding Carriage Return character ignored.

Any HTTP Header Field Name which is not recognized should be ignored in operational parsers.

It is recommended that servers use URIs free of “variant” characters whose representation differs in some of the national variant character sets, punctuation characters, and spaces. This will make URIs easier to handle by humans when the need (such as debugging, or transmission through non-hypertext systems) arises.

Copyright © 1992, W3C.

2.1 HTTP Headers

Data delivered by way of HTTP places the standard HTTP result code on the first line. HTTP defines a number of status codes. Servers can return any standard HTTP result. However, FIs should expect clients to collapse these codes into the following three cases:

<i>Code</i>	<i>Meaning</i>	<i>Action</i>
200	OK	The request was processed and a valid Open Financial Exchange result is returned.
400s	Bad request	The request was invalid and was not processed. Clients will report an internal error to the user. Invalid requests include: general HTTP transport errors, SGML formatting errors, invalid OFX syntax, and invalid data values. This error should not appear for request files the server is able to parse.
500s	Server error	The server is unavailable. Clients should advise the user to retry shortly.

Note: The server must return a code in the 400s for any problem that prevents it from processing the request file. Processing problems include failures relating to security, communication, parsing, or the Open Financial Exchange headers (for example, the client requested an unsupported language). For content errors such as wrong USERPASS or invalid account, the server must return a valid Open Financial Exchange response along with code 200. If a communication time-out error occurs while an OFX server and a back-end server are communicating to fill a request, then the server **MUST** return a code in the 500s.

Open Financial Exchange requires the following HTTP standard headers:

<i>Code</i>	<i>Value</i>	<i>Explanation</i>
Content-type	application / x-ofx	The MIME type for Open Financial Exchange
Content-length	length	Length of the data after removing HTTP headers

When responding with multipart MIME (likely only if the request included a <GETMIMERQ> request), the main type will be multipart/x-mixed-replace; one of the parts will use application/x-ofx.

2.2 Open Financial Exchange Headers

The contents of an Open Financial Exchange file consist of a simple set of headers followed by contents defined by that header

The Open Financial Exchange headers are in a simple *element:value* syntax and terminated by a blank line. Open Financial Exchange always sends headers unencrypted, even if application-level encryption is used for the remaining contents. The language and character set used for the headers is the same as the preceding MIME headers.

The first entry will always be OFXHEADER with a version number. This entry identifies the contents as an Open Financial Exchange file and provides the version number of the Open Financial Exchange headers that follow (not the version number of the contents). For example:

```
OFXHEADER:100
```

Open Financial Exchange headers can contain the following elements.

```
DATA:OFXSGML
```

```
VERSION:160
```

```
SECURITY:
```

```
ENCODING:
```

```
CHARSET:
```

```
COMPRESSION:
```

```
OLDFILEUID:
```

```
NEWFILEUID:
```

All OFX headers are required. NONE should be returned if client or server does not make use of an individual element, e.g., COMPRESSION:NONE, OLDFILEUID:NONE

A blank line follows the last header. Then (for type OFXSGML), the SGML-readable data begins with the <OFX> tag.

For information about each of the OFX headers, refer to the following sections.

2.2.1 OFXHEADER

OFXHEADER specifies the version number of the Open Financial Exchange headers.

The OFXHEADER value change its major number only if an existing client is unable to process the new header. This can occur because of a complete syntax change in a header, or a significant change in the semantics of an existing header element.

The current version of the Open Financial Exchange headers is version 1.0 (OFXHEADER:100).

2.2.2 DATA

DATA specifies the content type, in this case OFXSGML.

The value of the DATA element changes only when an entirely new syntax is introduced. In the case of OFXSGML, a new syntax would have to be non-SGML compliant to warrant a new DATA value. It is possible that there will be more than one syntax in use at the same time to meet different needs.

2.2.3 VERSION

VERSION specifies the version number of the Document Type Definition (DTD) used for parsing. There are currently three supported DTDs:

- ◆ The OFX 1.0.2 DTD supports all version 1 message sets, except Bill Presentment. It can be used to create and support OFX 1.0.2 clients and servers.
- ◆ The OFX 1.5.1 DTD supports all version 2 message sets, Bill Presentment, and all version 1 message sets. Because it supports all message sets, the OFX 1.5.1 DTD can be used to create and support OFX 1.0.2 and / or OFX 1.5.1 clients and servers.
- ◆ The OFX 1.6 DTD supports all message sets available in the OFX 1.5.1 DTD. It adds specific enhancements to some of the aggregates. All of those enhancements are optional and should not be used by a client unless the server indicates support in its FI Profile. The OFX 1.6 DTD can be used to create and support OFX 1.0.2 through OFX 1.6 clients and servers.

The current accepted values for VERSION are 102, 151, and 160.

Note: VERSION provides the version number of the DTD. The message set aggregates within the <OFX> block describes the version numbers of specific message sets. See section [2.4.5.3](#).

2.2.4 SECURITY

SECURITY defines the type of application-level security, if any, that is used for the <OFX> block. The values for SECURITY can be NONE or TYPE1.

For more information about security, refer to [Chapter 4, "OFX Security."](#)

2.2.5 ENCODING and CHARSET

ENCODING defines the text encoding used for character data. The values for ENCODING can be USASCII or UTF-8.

CHARSET defines the character set used for character data. The values for CHARSET may be ISO-8859-1 (Latin-1), 1252 (Windows Latin-1), or NONE. Any value specified here is likely to be ignored by an OFX client or server.

For more information about ENCODING and CHARSET, refer to [Chapter 5, "International Support."](#)

2.2.6 COMPRESSION

A future version of the specification will define compression.

2.2.7 OLDFILEUID and NEWFILEUID

NEWFILEUID uniquely identifies this request file. The NEWFILEUID, which clients must send with every request file and which servers must echo in the response, serves two purposes:

- ◆ Servers can use the NEWFILEUID to quickly identify duplicate request files.
- ◆ Clients and servers can use NEWFILEUID in conjunction with OLDFILEUID for file-based error recovery. For more information about using file-based error recovery or *lite synchronization*, see [Chapter 6, "Data Synchronization."](#)

OLDFILEUID is used together with NEWFILEUID only when the client and server support file-based error recovery. OLDFILEUID identifies the last request and response that was received and processed by the client.

2.3 SGML Details

2.3.1 Compliance

SGML is the basis for Open Financial Exchange. A DTD formally defines the SGML wire format for Open Financial Exchange. However, Open Financial Exchange is not completely SGML-*compliant* because the specification allows unrecognized tags to be present. Clients and servers must skip over the unrecognized tags. In this context, an “unrecognized tag” includes existing (well known) tags used in a new location. Clients and servers must ignore all such tags in order to allow future revisions to the specification and private tags which don’t require profile control. That is, if a client or server does not recognize <XYZ>, it must ignore the tag and its enclosed data. A fully compliant SGML parser would not *validate* a document that contains tags that the DTD does not define.

Although SGML is the basis for the specification, and the specification is largely compliant with SGML, do not assume Open Financial Exchange supports any SGML features not documented in this specification.

2.3.2 Valid SGML Characters

Open Financial Exchange elements can be set to any sequence of SGML characters. To be valid, a value must contain at least one character that is not a blank character. In other words, a value cannot contain only white space.

2.3.2.1 Special Characters

For the purposes of Open Financial Exchange, a few characters must be handled as special characters. To represent a special character, use the corresponding escape sequence.

<i>Character</i>	<i>Version</i>	<i>Escape sequence</i>
< (less than)		<
> (greater than)		>
& (ampersand)		&
‘ ‘ (space)	V2 only	

Note: The space macro was not added until OFX 1.5. So when using a version 1 message set (excluding Bill Presentment), there may be both clients and servers that can not process the syntax. In this case, you must prefix strings containing space characters as the first or last characters with “<![CDATA[” and suffix them with “]]>.” See [Chapter 9, "Customer to FI Communication,"](#) for an example.

Note: Space characters in the middle of a value do not require use of the special character macro. If a string value needs to contain space characters as the first or last characters, that's when this macro is needed.

For example, the string "AT&T" encodes "AT&T."

Note: Escape sequences are not required when these special characters are used in element values that accept HTML-formatted strings (for instance, e-mail records). These elements accept SGML-marked section syntax that hides the HTML from the Open Financial Exchange parser. You must prefix the HTML-formatted strings with "<![CDATA[" and suffix them with "]]>." Within these bounds, treat the above characters literally without an escape. See [Chapter 9, "Customer to FI Communication,"](#) for an example.

2.3.3 Comments Not Supported

For explanatory purposes, the examples in this specification contain comments. However, Open Financial Exchange files *cannot* contain comments.

2.4 Open Financial Exchange SGML Structure

2.4.1 Overview

Open Financial Exchange hierarchically organizes request and response blocks:

Top Level <OFX>

Message Set and Version <xxxMSGSVn>

Synchronization Wrappers <xxxSYNCRQ>, <xxxSYNCRS>

Transaction Wrappers <xxxTRNRQ>, <xxxTRNRS>

Specific requests and responses

The following sections describe these levels.

2.4.2 Case Sensitivity

OFX requires upper case letters for tag names and enumerated values. In the example below, <SEVERITY> is an element with an enumerated value and <MESSAGE> is an element with a value that is not enumerated.

<STATUS>

<CODE> 2000

<SEVERITY> ERROR

<MESSAGE> General Error

</STATUS>

2.4.3 Top Level

An Open Financial Exchange request or response has the following top-level form:

<i>Tag</i>	<i>Description</i>
<OFX>	Opening tag
<SONRQ> or <SONRS>	Required signon request or response. See section 2.5.1 .
... Open Financial Exchange requests or responses ...	0 or more transaction requests and responses inside appropriate message set aggregates
</OFX>	Closing tag for the Open Financial Exchange record

This chapter specifies the order of requests and responses.

A single file **MUST** contain only one OFX block.

2.4.4 Messages

A message is the unit of work in Open Financial Exchange. It refers to a request and response pair, and the status codes associated with that response. For example, the message to download a bank statement consists of the request <STMTRQ> and the response <STMTRS>.

OFX uses several common message types to perform specific functions. Within OFX, the following naming conventions are used, where the general xxx messages may be:

- ◆ Basic (or Add) request <xxxRQ> and response <xxxRS>
- ◆ Modify request <xxxMODRQ> and response <xxxMODRS>
- ◆ Delete request <xxxDELRQ> and response <xxxDELRS>
- ◆ Cancel request <xxxCANRQ> and response <xxxCANRS> (these pairs may also be named <xxxCANCRRQ> and <xxxCANCRRS>)

2.4.4.1 Basic and Add Messages

The *basic* OFX message has a name structure of <xxxRQ> / <xxxRS>. It is used for read actions of a specific object (such as a bank statement using <STMTENDRQ>). It is encapsulated in a transaction wrapper <xxxTRNRQ> or <xxxTRNRS> (therefore, <STMTENDTRNRQ> and <STMTENDTRNRS> in the example above).

The *add* OFX message, like the Basic message, has a name structure of <xxxRQ> / <xxxRS>. It is used to create a new instance of object xxx (such as creating a new payment using <PMTRQ>). It is encapsulated in a transaction wrapper <xxxTRNRQ> or <xxxTRNRS> (therefore <PMTRNRQ> and <PMTRNRS> in the example above).

2.4.4.2 Modify Message

The *modify* OFX message has a name structure of <xxxMODRQ> / <xxxMODRS>. It is used to modify an existing instance of object xxx (such as modifying an existing payment using <PMTMODRQ>). It is encapsulated in a transaction wrapper <xxxTRNRQ> or <xxxTRNRS> (therefore, <PMTTRNRQ> and <PMTTRNRS> in the example above).

The <xxxMODRQ> request contains the **complete replacement** data for an existing object xxx. Therefore, **both changed and unchanged elements** must be included in the request.

2.4.4.3 Delete and Cancel Messages

The *delete* and *cancel* OFX messages have a name structure of <xxxDELQR> / <xxxDELRS> and <xxxCANRQ> / <xxxCANRS> or <xxxCANCQR> / <xxxCANCRS>, respectively. They are used to delete an existing instance of object xxx (such as deleting a payee from a payee list using <PAYEEDQR>), or to cancel an existing scheduled object (such as canceling a pending payment using <PMTCANCQR>). They are encapsulated in a transaction wrapper <xxxTRNRQ> or <xxxTRNRS> (therefore, <PAYEETRNRQ> and <PMTTRNRQ> in the examples above).

2.4.4.4 Inquiry Message

The *inquiry* OFX message sometimes has a name structure of <xxxINQRQ> / <xxxINQRS>. It is used to search for and / or gain information about (an) existing object(s) xxx (such as finding one or more existing payments using <PMTINQRQ>). It is encapsulated in a transaction wrapper <xxxINQTRNRQ> or <xxxINQTRNRS> (therefore, <PMTINQTRNRQ> and <PMTINQTRNRS> in the example above).

Inquiry messages limit the response set to records matching the *selection criteria* used in the request. Selection criterion elements in the request are generally repeating elements. Where more than one value is given for a particular element, the query ORs those values. Where multiple different elements (matches for different fields of the objects) are provided, the query ANDs those values. Where an element is absent from the request, the query is not filtering on that

element. If an element has a history associated with it, only the most recent value is intended by the inquiry.

Note: A server is not obligated to support filtering on all selection criterion elements. If a server chooses not to support a particular element as a selection criterion, it **must** treat that element as if it were not present. That is, the server must return the appropriate record set for the elements on which it does support filtering. As a result, clients should be prepared to receive records outside the scope of the selection criteria submitted in the request.

Note: Many inquiry messages do not presently follow the naming conventions detailed above. They may be named <xxxINFORQ>/<xxxINFORS> (<ACCTINFORQ> and <ACCTINFORS> for example) or without reference to an obvious convention (<PRESLISTRQ> and <PRESLISTRS> for example).

2.4.5 Message Sets and Version Control

Message sets are collections of messages. Generally they form all or part of what a user would consider a *service*, something for which they might have signed up, such as “banking.” Message sets are the basis of version control, routing, and security. They are also the basis for the required ordering in Open Financial Exchange files.

Within the OFX block, OFX organizes messages by message set. Message sets follow these rules:

- ◆ With the exception of <SIGNUPMSGSRQV2>, a request file may include at most one message set wrapper of each type.

The one exception to this rule is <SIGNUPMSGSRQV2>. The <SIGNUPMSGSRQV2> message sets may repeat as often as required. Each repetition of the <SIGNUPMSGSRQV2> message set wrappers must have differing top-level elements (for example, different <USERID> values in OFX 1.6).

In addition, repetitions of <SIGNUPMSGSRQV2> must be ordered to match the <SIGNUPMSGSRQV2> wrappers in the request file (each <USERID> must be returned in the same order).

- ◆ For a particular service, the V1 and V2 message sets may not be combined in a single request file.

For example, the <SIGNUPMSGSRQV1> and <SIGNUPMSGSRQV2> message set wrappers must not appear together in a request file.

- ◆ All messages within any message set must be from the same version of that message set.
- ◆ Servers must respond using the same message sets and versions as sent in the request file. For example, if <SIGNUPMSGSRQV1> appears in the request file, <SIGNUPMSGSRQV1> must appear in the response file. And, if <BANKMSGSRQV2> was not in the request file, <BANKMSGSRQV2> must not appear in the response.

There are two exceptions to this rule:

- ◆ Servers may ignore unsupported message set wrappers.
- ◆ Servers may return the <SECLISTMSGSRSVn> wrapper (see 13.7.2 and 13.8.4) in response to an investment statement download (with or without a <SECLISTMSGSRQVn> in the request file).

2.4.5.1 Message Set Aggregates

For each message set of xxx and version n, there are two aggregates, one for requests <xxxMSGSRQVn> and one for responses <xxxMSGSRSVn>. All of the messages from that message set must be enclosed in the appropriate message set aggregate. In the following example, the Open Financial Exchange block contains a signon request inside the signon message set, and two statement requests and a transfer request inside the bank message set.

```
<OFX>
  <SIGNONMSGSRQV1>                                <!-- Signon message set -->
    <SONRQ>                                         <!-- Signon message -->
    ...
    </SONRQ>
  </SIGNONMSGSRQV1>

  <BANKMSGSRQV1>                                    <!-- Banking message set -->
    <STMTTRNRQ>                                    <!-- Statement request -->
    ...
    </STMTTRNRQ>
    <STMTTRNRQ>                                    <!-- Another stmt request -->
    ...
    </STMTTRNRQ>
    <INTRATRNRQ>                                    <!-- Intrabank transfer request -->
    ...
    </INTRATRNRQ>
  </BANKMSGSRQV1>
</OFX>
```


2.4.5.2 Message Set Ordering

Message sets must appear in the following order:

- ◆ Signon
- ◆ Signup
- ◆ Banking
- ◆ Credit card statements
- ◆ Investment statements
- ◆ Interbank funds transfers
- ◆ Wire funds transfers
- ◆ Payments
- ◆ General e-mail
- ◆ Investment security list
- ◆ Biller Directory
- ◆ Bill Delivery
- ◆ FI Profile

The definition of each message set can further prescribe an order of its messages within that message set.

For ordering purposes, versions of a message set are grouped with the message set. For instance, Signon V2 would precede Banking V1, using the “natural” ordering above.

2.4.5.3 Message Set Version Numbers

Message sets have their own version numbers, which are distinct from the version numbers of the Open Financial Exchange headers and the Document Type Definition (DTD) files.

Note: OFX versions 1.5.1 and 1.6 are intended to interoperate without problems. The additions made in OFX 1.6 are optional and may be ignored by 1.5.1 clients and servers without complications. In particular, a server must indicate support for a new feature before a client may use that feature. The version numbers of the Open Financial Exchange headers and the Document Type Definition (DTD) files appear in the Open Financial Exchange headers, before the <OFX> data block. For more information about the Open Financial Exchange headers, see section 2.2. The current version number of the headers is OFXHEADER: 100. The current version numbers of the DTDs are 102, 151, and 160.

The following table lists each message set, along with its aggregate name and the DTD versions that support it.

<i>Message Set</i>	<i>Message Set Aggregate</i>	<i>DTD Support</i>
Signon	<SIGNONMSGSETV1>	1.0.2, 1.5.1, 1.6
Signon	<SIGNONMSGSETV2>	1.5.1, 1.6
Signup	<SIGNUPMSGSETV1>	1.0.2, 1.5.1, 1.6
Signup	<SIGNUPMSGSETV2>	1.5.1, 1.6
Banking	<BANKMSGSETV1>	1.0.2, 1.5.1, 1.6
Banking	<BANKMSGSETV2>	1.5.1, 1.6
Credit Card Statements	<CREDITCARDMSGSETV1>	1.0.2, 1.5.1, 1.6
Credit Card Statements	<CREDITCARDMSGSETV2>	1.5.1, 1.6
Investment Statements	<INVSTMTMSGSETV1>	1.0.2, 1.5.1, 1.6
Investment Statements	<INVSTMTMSGSETV2>	1.5.1, 1.6
Interbank Funds Transfers	<INTERXFERMSGSETV1>	1.0.2, 1.5.1, 1.6
Interbank Funds Transfers	<INTERXFERMSGSETV2>	1.5.1, 1.6
Wire Funds Transfers	<WIREXFERMSGSETV1>	1.0.2, 1.5.1, 1.6
Wire Funds Transfers	<WIREXFERMSGSETV2>	1.5.1, 1.6
Payments	<BILLPAYMSGSETV1>	1.0.2, 1.5.1, 1.6
Payments	<BILLPAYMSGSETV2>	1.5.1, 1.6
General e-mail	<EMAILMSGSETV1>	1.0.2, 1.5.1, 1.6
General e-mail	<EMAILMSGSETV2>	1.5.1, 1.6
Investment security list	<SECLISTMSGSETV1>	1.0.2, 1.5.1, 1.6
Investment security list	<SECLISTMSGSETV2>	1.5.1, 1.6
Biller directory	<PRESDIRMSGSETV1>	1.5.1, 1.6
Bill delivery	<PRESDLVMSGSETV1>	1.5.1, 1.6
FI Profile	<PROFMSGSETV1>	1.0.2, 1.5.1, 1.6
FI Profile	<PROFMSGSETV2>	1.5.1, 1.6

Note: For each message set that it is supporting, a financial institution must indicate which version numbers of that message set it supports. The financial institution includes the message set version number in the <MSGSETCORE> aggregate of the FI profile. For more information about the FI profile, refer to [Chapter 7, "FI Profile."](#)

2.4.6 Transactions

Other than the signon message, each request is made as a transaction. Transactions contain a client-assigned globally-unique ID, optional client-supplied pass-back data, and the request aggregate. A transaction similarly wraps each response. The response transaction returns the client ID sent in the request, along with a status message, the pass-back data if present, and the response aggregate. This technique allows a client to track responses against requests. Section [3.1.2](#) provides more information about the format of information exchanged by the client and server.

The <STATUS> aggregate, defined in [Chapter 3, "Common Aggregates, Elements, and Data Types."](#) provides feedback on the processing of the request. If the <SEVERITY> of the status is ERROR, the server provides the transaction response without the nested response aggregate. Otherwise, the response must be complete even though some warning might have occurred.

Clients can send additional information in <CLTCookie> that servers will return in the response. This allows clients that do not maintain state, and thus do not save <TRNID>s, to cause some additional descriptive information to be present in the response. For example, a client might identify a request as relating to a user or a spouse.

Starting with the message sets new to OFX 1.5 and later (e.g., the V2 message sets and the bill presentation message sets), <CLTCookie> must only be returned by the server in the initial response to the client (and any crash recovery from that response). The <CLTCookie> should not be present in a sync response, except for those transactions whose requests were wrapped in the sync request. This restriction is new for message set version 2 of all the message sets.

In some countries, some banks may require that a customer-supplied authorization number be included to authenticate certain kinds of individual transactions such as payment requests. For those banks, the <TAN> element passes this information to servers.

Note that if a <CLTCookie> is given to an OFX server in a request, the OFX server is required to return it. This return of the <CLTCookie> will necessitate server-side storage of <CLTCookie> data. In the case of an OFX client getting a <CLTCookie> that it didn't send in a request, the default behavior is to ignore it.

2.4.6.1 Transaction Wrapper

With the exception of the <SONRQ> / <SONRS> message, each message has a corresponding *transaction wrapper*. For requests, the transaction wrapper adds a transaction unique ID <TRNUID>. For responses, the transaction wrapper adds the same transaction unique ID <TRNUID> (an echo of that found in the request), plus a <STATUS> aggregate.

The *transaction wrapper* has a name structure of <xxxTRNRQ> / <xxxTRNRS>. A transaction wrapper pair encapsulates a single message (<xxxRQ> / <xxxRS>, <xxxMODRQ> / <xxxMODRS>, etc.).

While the same name may be used for addition, modification and deletion messages, a single transaction wrapper may contain at most one request or response. The request transaction wrapper must contain a single request. The response transaction wrapper must contain a single response unless the contained <STATUS> aggregate indicates an error. The <MULTIINTERNRQ> / <MULTIINTERNRS> pair (section 11.8.5) is an exception to these rules.

Note: Some requests and responses (generally, Add, Modify, and Delete / Cancel types) share a transaction wrapper and synchronization wrapper. In these cases, the names of the transaction and synchronization wrappers reflect the Add message.

A typical request is as follows:

Tag	Description
<xxxTRNRQ>	Transaction-request aggregate
<TRNUID>	Client-assigned globally-unique ID for this transaction, <i>trnuid</i>
<CLTCOOKIE>	Data to be echoed in the transaction response, A-32
<TAN>	Transaction authorization number; used in some countries with some types of transactions. The FI Profile defines messages that require a <TAN>, A-80
Request aggregate	Aggregate for the request
</xxxTRNRQ>	

A typical response is as follows:

Tag	Version	Description
<xxxTRNRS>		Transaction-response aggregate
<TRNUID>		Client-assigned globally-unique ID for this transaction, <i>trnuid</i>
<STATUS>		Status aggregate
</STATUS>		
<CLTCookie>	V2 change	Client-provided data, REQUIRED if provided in request, A-32 See note above regarding usage inside sync response.
Response aggregate		Aggregate for the response
</xxxTRNRS>		

List of status code values for the <CODE> element of <STATUS>:

Value	Meaning
0	Success (INFO)
2000	General error (ERROR)
2022	Invalid TAN (ERROR)

2.4.7 Synchronization Wrapper

The *synchronization wrapper* has a name structure of <xxxSYNCRQ> / <xxxSYNCRS>. It contains synchronization parameters and optionally encapsulates one or more transaction wrappers. For details on the use of synchronization wrappers, see Chapter 6.

When embedded transactions are not present, the synchronization request contains no transaction wrappers. If the client is up to date when the server processes such a request, the synchronization response also contains no transaction wrappers.

Note: If a request / response is a sync request / response only, the transaction wrapper and request that it wraps are omitted.

2.4.8 Message Set Wrapper

The profile *message set wrappers* have a name structure of <xxxMSGSET>, <xxxMSGSETV1> and <xxxMSGSETV2>. The “V1” and “V2” variants always appear inside a <xxxMSGSET> wrapper. They correspond to the supported version of the message set under consideration.

The request and response *message set wrappers* have a name structure of <xxxMSGSRQVn> and <xxxMSGSRSVn> respectively. For OFX 1.6 and earlier, “n” must be either “1” or “2”. This number indicates the version of the message set used by the contained messages.

2.5 The Signon Message Set

The Signon message set includes the signon message, USERPASS change message, and challenge message, which must appear in that order. The <SIGNONMSGSRQV1> and <SIGNONMSGSRSV1> (also V2) aggregates wrap the message.

2.5.1 Signon <SONRQ> <SONRS>

The signon record identifies and authenticates a user to an FI. It also includes information about the application making the request, because some services might be appropriate only for certain clients. Every Open Financial Exchange block contains exactly one <SONRQ>. Every response must contain exactly one <SONRS> record. Use of Open Financial Exchange presumes that FIs authenticate each customer and then give the customer access to one or more accounts or services. Authentication of a <SONRQ> is required, even when in Error Recovery. If passwords are specific to individual services or accounts, a separate Open Financial Exchange request must be made for each user ID or password required. This will not necessarily be in a manner visible to the user. Note that some situations, such as joint accounts or business accounts, will have multiple user IDs and multiple passwords that can access the same account.

FIs assign user IDs for the customer. Although the user ID may be the customer’s social security number, the client must not make any assumptions about the syntax of the ID, add check-digits, or do similar processing.

To improve server efficiency in handling a series of Open Financial Exchange request files sent over a short period of time, clients can request that a server return a <USERKEY> in the signon response. If the server provides a user key, clients will send the <USERKEY> instead of the user ID and password in subsequent sessions, until the <USERKEY> expires. This allows servers to authenticate subsequent requests more quickly. Servers must accept a <GENUSERKEY> element in a <SONRQ>. However, a server may decide <USERKEY> does not afford sufficient security and may optionally not return a <USERKEY> in the <SONRS>.

The client returns <SESSCOOKIE> if the server sent one in a previous <SONRS>. Servers can use the value of <SESSCOOKIE> to track client usage but cannot assume that all requests come from a single client, nor can they deny service if they did not expect the returned cookie. Use of a

backup file, for example, could lead to an unexpected <SESSCOOKIE> value that nevertheless should not stop a user from connecting.

A client may use an anonymous form of <USERID> and <USERPASS> on those rare occasions when a server need not authenticate the <SONRQ>. The only present situations in this class are first-time <PROFRQ>, <FINDBILLERRQ>, and all <ENROLLRQ> transactions. Any request sent by the client after a successful <ENROLLRQ> response (or out of band enrollment) for the service must provide the user's <USERID> and <USERPASS>. The anonymous <USERID> or <USERPASS> value is left aligned and padded with 0 to a length of 32 characters:
anonymous000000000000000000000000

Note: This anonymous password length may exceed the <MAX> value for the profile server (in the corresponding <SIGNONINFO> aggregate). Nonetheless, servers supporting anonymous signon must not reject this password due to its length.

Servers can request that a consumer change his or her password by returning status code 15000. Servers should keep in mind that only one status code can be returned. If the current signon response status should be 15500 (invalid ID or password), the request to change the password must wait until an otherwise successful signon is achieved.

If the server returns any signon error, it must respond to all other requests in the same <OFX> block with status code 15500. For example, if the server returns status code 15502 to the signon request, it must return status code 15500 to all other requests in the same <OFX> block. The server must return status code 15500 to all requests; it cannot simply ignore the requests.

An OFX 1.5.1 or greater server has the option of allowing or disallowing “empty” signon transactions. In the context of signon, “empty” means a simple signon without any other transaction (a sync, statement download, etc.). If the OFX 1.5.1 or greater server does not support empty signon, it should return error 15506. If the OFX 1.5.1 or greater server does support empty signon, it should process the signon and return the appropriate error or success code.

For OFX 1.0.2 servers, it is optional whether or not to support error 15506. Since additional error codes can be added to an OFX 1.0.2 server, it is suggested that logic to support error 15506 be added to OFX 1.0.2 servers that wish to disallow empty signon transactions. Older clients should remap unknown error codes to a general error.

If the server returns any signon error, it must respond to all other requests in the same <OFX> block with status code 15500. For example, if the server returns status code 15502 to the <SONRQ> request, it must return status code 15500 to all other requests in the same <OFX> block. The server must return status code 15500 for all requests; it cannot simply ignore the requests. In addition, any sync responses must indicate an error with <TOKEN>–1, <LOSTSYNC>N (<LOSTSYNC> is an optional element) and (in version 2 responses) an appropriate <SYNCERROR>. Responses for any transactions embedded in the sync request should contain the same <STATUS><CODE>15500. Otherwise, they must be omitted from the sync response wrapper. (See section 6.2 for data synchronization specifics.)

2.5.1.1 Signon Request <SONRQ>

Unlike other requests, the signon request <SONRQ> does not appear within a transaction wrapper.

Tag	Version	Description
<SONRQ>		Signon-request aggregate
<DTCLIENT> <i>User identification. Either <USERID> and <USERPASS> or <USERKEY>, but not both.</i>		Date and time of the request from the client computer, <i>datetime</i> This value should reflect the time (according to the client machine) when the request file is sent to the server, not the (original) creation time of the request file. While not required for existing software, OFX 1.6 clients must comply with this rule. This clarification is particularly important in error recovery situations in which the request file may be sent to the server after its initial creation.
<USERID> <USERPASS> <ONETIMEPASS> -or- <USERKEY>	V2 only	User identification string, A-32 User password on server, A-171 Note: The maximum clear text length of USERPASS is 32 characters; a client must not send a longer password. However, when using Type 1 security, the encrypted value may extend to 171 characters. A special type of password that is used only once (in addition to USERPASS) to authenticate a single OFX session, A-32 Note: The client can send this element only if the <PWTYPE> value returned by the server is ONETIME or HWTOKEN (See section 7.2.2).
<GENUSERKEY> <LANGUAGE> <COUNTRY>	V2 only	Log in using previously authenticated context, A-64 Request server to return a USERKEY for future use, <i>Boolean</i> Requested language for text responses, <i>language</i> Specific country system used for the requests in this <OFX> block: 3-letter country code from ISO / DIS-3166. If this element is not present, the country system is USA. The following countries are currently supported in OFX:

Tag	Version	Description
		<u>COUNTRY</u> <u>Country Name</u> BEL Belgium CAN Canada CHE Switzerland DEU Germany ESP Spain FRA France GBR Great Britain ITA Italy NLD Netherlands USA United States of America
<FI>		Financial-Institution-identification aggregate
</FI>		Note: The client will determine out-of-band whether a FI aggregate should be used and if so, the appropriate values for it. If the FI aggregate is to be used, then the client should send it in every request, and the server should return it in every response.
<SESSCOOKIE>		Session cookie value received in previous <SONRS>, not sent if first login or if none sent by FI, A-1000
<APPID>		ID of client application, A-5
<APPVER>		Version of client application, (6.00 encoded as 0600), N-4
</SONRQ>		

2.5.1.2 Signon Response <SONRS>

Unlike other responses, the signon response <SONRS> does not appear within a transaction wrapper.

Note: In several V2 message sets, the specific definition of an element is dependent upon the <COUNTRY> element in <SONRS>. For example, when using the V2 version of the Banking message set, BANKID of the BANKACCTFROM aggregate is interpreted depending on the value of COUNTRY (see section [11.3.1](#)). If a server wants to prevent potential client ambiguity regarding the country system being used, the server should require Signon Message Set V2.

Note: A client should use <DTPROFUP> and <DTACCTUP> only when the service provider that originated <SONRS> is the same provider that is specified by <SPNAME> in the profile message set. A client can determine if the service provider is the same by comparing the value of <SPNAME> in the appropriate message set with the value for <SPNAME> in the profile message set.

Tag	Version	Description
<SONRS>		Record-response aggregate
<STATUS>		Status aggregate, see section 3.1.5 . See list of possible code values in section 2.5.1.3
</STATUS>		
<DTSERVER>		Date and time of the server response, <i>datetime</i> This value should reflect the time (according to the server) when the response file was originally created. While not required for existing software, OFX 1.6 servers must comply with this rule. This clarification is particularly important in error recovery situations: The server should (must for OFX 1.6 servers) return the time the request was first processed. If the previous attempt failed after transactions were processed, <DTSERVER> in the response file would reflect that processing time.
<USERKEY>		Use user key instead of USERID and USERPASS for subsequent requests. TSKEYEXPIRE can limit lifetime. A-64
<TSKEYEXPIRE>		Date and time that USERKEY expires, <i>datetime</i>
<LANGUAGE>		Language used in text responses, <i>language</i>

Tag	Version	Description
<COUNTRY>	V2 only	Specific country system used for the requests: 3-letter country code from ISO / DIS-3166. If this element is not present, the country system is USA. Note: Where element behaviors are listed within this specification as dependent upon the value of <COUNTRY>, the reference is to this one in the <SONRS> aggregate. It is not to be confused with the element <COUNTRY> that appears elsewhere in OFX, associated with addresses.
<DTPROFUP>		Date and time of last update to profile information for any service supported by this FI (see Chapter 7, "FI Profile"), <i>datetime</i>
<DTACCTUP>		Date and time of last update to account information (see Chapter 8, "Activation & Account Information"), <i>datetime</i>
<FI>		Financial-Institution-identification aggregate Note: The client will determine out-of-band whether an FI aggregate should be used and, if so, the appropriate values for it. If the FI aggregate is to be used, then the client should send it in every request, and the server should return it in every response.
</FI>		
<SESSCOOKIE>		Session cookie that the client should return on the next <SONRQ>,A-1000
</SONRS>		

2.5.1.3 Status Codes

List of status code values for the <CODE> element of <STATUS>:

<i>Value</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
13504	<FI> Missing or Invalid in <SONRQ> (ERROR)
15000	Must change USERPASS (INFO)
15500	Signon invalid (see section 2.5.1) (ERROR)
15501	Customer account already in use (ERROR)
15502	USERPASS Lockout (ERROR)
15505	Country system not supported by server (ERROR)
15506	Empty signon transaction not supported (ERROR)
15507	Signon invalid without supporting pin change request (ERROR)

2.5.1.4 Financial Institution ID <FI>

Some service providers support multiple FIs, and assign each FI an ID. The signon allows clients to pass this information along, so that providers know to which FI the user is signing on.

If a server does not require an FI aggregate in a request but receives one anyway, it should echo the FI aggregate back. This is compliant with the general rule that the server should echo elements and aggregates in the response if they are received and understood in the request.

If a server requires the <FI> aggregate in <SONRQ> requests and it contains incorrect information there are several different specification compliant ways to respond. These are given in the order of preference:

- ◆ Return a 2000 error with appropriate text message – since the FI aggregate information is incorrect the user's information (<USERID> and <USERPASS>) cannot be verified. Returning a 15500 might cause clients to display messages to the user that the attempt to communicate with the server failed. A client would probably suggest that the user verify their <USERID> and <USERPASS> values.
- ◆ Return a 15500 error – since the FI aggregate information is incorrect or unknown the server cannot verify the <USERID>, <USERPASS>, etc.
- ◆ Return an http 400 error – this is the least desirable option since it will provide no useful feedback to the client communicating with the server, however it is legal.

<i>Tag</i>	<i>Description</i>
<FI>	FI-record aggregate
<ORG>	Organization defining this FI name space, A-32
<FID>	Financial Institution ID (unique within <ORG>), A-32
</FI>	

2.5.2 USERPASS Change <PINCHRQ> <PINCHRS>

The client sends a request to change the customer password as a separate request from the signon. The transaction request <PINCHTRNRQ> aggregate contains <PINCHRQ>. Responses are placed inside the transaction response <PINCHTRNRS>.

Password changes pose a special problem for error recovery. If the client does not receive a response, it cannot know whether or not the password change was successful. OFX recommends that servers accept either the old password or the new password on the connection following the one containing a password change. When file-based error recovery is in use, the server must reject the old password except when received with NEWFILEUID/OLDFILEUID headers indicating an error recovery attempt.

Servers that do not support file-based error recovery (or, when interacting with a client that does not utilize file-based error recovery) must not complete a <PINCHRQ> until after the next request file arrives. If that request file uses the new password, the new password must be permanently associated with the <USERID>. Otherwise, the old password may authenticate the user. (For security, servers may return a signon error if the next request file uses the old password but does not include a <PINCHRQ>.) Conforming clients should re-send request files (unchanged beyond the <SONRQ>) after a failure whether or not file-based error recovery is in use.

2.5.2.1 <PINCHRQ>

A USERPASS change request changes the customer's password for the specific realm associated with the messages contained in the OFX block. Based on the properties of an OFX profile, defined in [Chapter 7, "FI Profile,"](#) a single OFX block contains instructions related to a single realm. The USERPASS change request thus changes the USERPASS for all message sets associated with one realm. For more information about signon realms, see section [7.2.2](#).

Tag	Description
<PINCHRQ>	USERPASS-change-request aggregate
<USERID>	User identification string. Often a social security number, but if so, does not include any check digits, A-32 Note: The maximum clear text length of USERPASS is 32 characters: a client must not send a longer password. However, when using Type 1 security, the encrypted value may extend to 171 characters.
<NEWUSERPASS>	New user password, A-171 Note: The effective size of NEWUSERPASS is A-32. However, if Type 1 security is used, then the actual field length is A-171.
</PINCHRQ>	

2.5.2.2 <PINCHRS>

Tag	Description
<PINCHRS>	USERPASS-change-response aggregate
<USERID>	User identification string. Often a social security number, but if so, does not include any check digits, A-32
<DTCHANGED>	Date and time the password was changed, <i>datetime</i>
</PINCHRS>	

2.5.2.3 Status Codes

Note: If On Behalf Of is not allowed for the signed-on user, status code 15508 may result from a <USERID> mismatch between the <SONRQ> and <PINCHRQ>. For information about On Behalf Of actions, see section [8.1.1](#).

Value	Meaning
0	Success (INFO)
2000	General error (ERROR)
15503	Could not change USERPASS (ERROR)
15508	Transaction not authorized (ERROR)

2.5.3 <CHALLENGERQ> <CHALLENGERS>

A challenge request is the first step in Type 1 application-level security. Essentially, it asks for some random data from the server. The challenge response provides that server-generated random data and is the second step in Type 1 security.

The challenge message is part of the signon message set and is not subject to data synchronization.

2.5.3.1 <CHALLENGERQ>

A <CHALLENGERQ> is part of a <CHALLENGETRNQR> transaction, a <CHALLENGERS> part of a <CHALLENGETRNR>.

The client includes <FICERTID> in the request if it already has the server's certificate. If it's included and matches the server's current certificate, the server may omit the actual certificate from the response.

<i>Tag</i>	<i>Description</i>
<CHALLENGERQ>	Opening tag for the challenge request.
<USERID>	User identification string, A-32
<FICERTID>	Optional server certificate ID. A-64
</CHALLENGERQ>	Closing tag for challenge request.

2.5.3.2 <CHALLENGERS>

<i>Tag</i>	<i>Description</i>
<CHALLENGERS>	Opening tag for the challenge response.
<USERID>	User identification string, A-32
<NONCE>	Server-generated random data. A-16
<FICERTID>	ID of server certificate used to encrypt. A-64
</CHALLENGERS>	Closing tag for challenge response.

When generating the <NONCE>, make sure the data is as unpredictable as possible. See RFC 1750 for recommendations.

The server includes <FICERTID> in the response to identify the certificate in a separate MIME part. Even if the certificate itself is not attached, <FICERTID> is still included in the response.

2.5.3.3 Status Codes

Status code values for the <CODE> element of <STATUS>:

<i>Value</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
15504	Could not provide random data (ERROR)
15508	Transaction not authorized (ERROR)

2.5.4 Signon Message Set Profile Information

A server must include the signon message set <SIGNONMSGSET> as part of the <MSGSETLIST> aggregate in the FI profile, since every server must support signon requests.

The information that is part of the <MSGSETCORE> aggregate (for example, the URL and security level) is used only when no other message sets are used. Otherwise, the other message sets override the signon message set for the purposes of batching and routing. For example, if bill payments are sent to a URL that is different from the one used for signon, the client uses the URL specified in the bill payment message set <BILLPAYMSGSET>. For more information about how clients batch and route messages, refer to section [7.1.3](#).

<i>Tag</i>	<i>Description</i>
<SIGNONMSGSET>	Signon-message-set-profile-information aggregate
<SIGNONMSGSETV1>	Opening tag for V1 of the message set profile information
<MSGSETCORE>	Common message set information, defined in Chapter 7, "FI Profile"
</MSGSETCORE>	
</SIGNONMSGSETV1>	
<SIGNONMSGSETV2>	Opening tag for V2 of the message set profile information
<MSGSETCORE>	Common message set information, defined in Chapter 7, "FI Profile"
</MSGSETCORE>	
</SIGNONMSGSETV2>	
</SIGNONMSGSET>	

2.5.5 Examples

User requests a password change:

```
<PINCHTRNRQ>
  <TRNUID>888
  <PINCHRQ>
    <USERID>123456789
    <NEWUSERPASS>5321
  </PINCHRQ>
</PINCHTRNRQ>
```

The server responds with:

```
<PINCHTRNRS>
  <TRNUID>888
  <STATUS>
    <CODE>0
    <SEVERITY>INFO
  </STATUS>
  <PINCHRS>
    <USERID>123456789
  </PINCHRS>
</PINCHTRNRS>
```

2.6 External Data Support

Some data, such as binary data, cannot easily be sent within SGML. For these situations, the specification defines an element that references some external data. The way that clients pick up the external data depends on the transport used. For the HTTP-based transport described in this document, servers can send the data in one of two ways:

- ◆ Send the same response, using multipart MIME types to separate the response into the Open Financial Exchange file and one or more external data files
- ◆ Client can make a separate HTTP get against the supplied URL, if it really needs the data

For example, to retrieve a logo, a <GETMIMERS> might answer a <GETMIMERQ> as follows:

```
<GETMIMERS>
  <URL>https://www.fi.com/xxx/yyy/zzz.jpg
</GETMIMERS>
```

If the file includes the same response using multipart MIME, clients must have the local file, zzz.jpg.

2.7 Extensions to Open Financial Exchange

An organization that provides a customized client and server that communicate by means of Open Financial Exchange might wish to add new requests and responses or even specific elements to existing requests and responses. To ensure that each organization can extend the specification without the risk of conflict, Open Financial Exchange defines a style of tag naming that lets each organization have its own name space.

Organizations can register a specific tag name prefix. (The specific procedure or organization to manage this registration will be detailed at a later time.) If an organization registers “ABC,” then they can safely add new elements and aggregates named <ABC.SOMETHING> without

- ◆ Colliding with another party wishing to extend the specification
- ◆ Confusing a client or server that does not support the extension

The extensions are not considered proprietary. An organization is free to publish their extensions and encourage client and server implementors to support them.

All tag names that do not contain a period (.) are reserved for use in future versions of the Open Financial Exchange specification.

CHAPTER 3 COMMON AGGREGATES, ELEMENTS, AND DATA TYPES

3.1 Common Aggregates

This section describes aggregates used in more than one service of Open Financial Exchange (for example, investments and payments).

3.1.1 Identification of Financial Institutions and Accounts

Open Financial Exchange does not provide a universal space for identifying financial institutions, accounts, or types of accounts. The way to identify an FI and an account at that FI depends on the service. For information about service-specific ID aggregates, see [Chapter 11, "Banking,"](#) [Chapter 12, "Payments,"](#) and [Chapter 13, "Investments."](#)

3.1.2 Format of User-Supplied Numbers

Clients and proxies (or, other intermediate OFX servers) should not attempt to add or remove punctuation from user-supplied values. This information should be transmitted without change to the OFX server. Examples of elements that may contain punctuation significant to the server are <ADDR1>, <EMAIL>, and any of the <xxxPHONE> elements found in the specification.

In some cases, clients may use additional information (not available to the customer) to enhance user-supplied text. An <ACCTFORMAT> or <ACCTEDITMASK> (including required punctuation characters, for example) may be used to normalize an entered <ACCTID> (say, containing only numbers) into the format needed by the server. Or, the <POSTALCODE> in a US address may be extended to include the full ZIP+4 value. These cases of client manipulation of user-supplied values are not common in the OFX specification. For example, a client that removes all punctuation from a user-supplied phone number is compliant with the requirements but not the intent of the OFX specification.

When matching user-supplied text against stored information, servers are free to ignore all supplied punctuation characters. For example, a server might remove all punctuation from an <ACCTID> before performing validation. This temporary modification affects neither how the data would be returned nor its storage format. Such transformations should not occur with values provided for security reasons such as <USERID> and <USERPASS> elements. These transformations (relaxed matches between selection criteria and saved data) are *required* only for <TAXID> in an enrollment request and the <BANKID> and <ACCTID> elements of the <xxxACCTTO> aggregate. Otherwise, exact matches of selection criteria and saved data are all that is required by this specification.

Servers are permitted to add or remove punctuation or otherwise modify client-supplied information, while storing the data after processing a (successful) <xxxRQ> or <xxxMODRQ> request. For example, a server might store only the first five digits of a US <POSTALCODE> value, abbreviate common address components (storing “St.” when the request specified “Street”), or use a special address for well-known payees. If a server does make such modifications, it must return the client-supplied values verbatim in the initial response, and treat the modification as a server-initiated action. Therefore, a subsequent synchronization should include a <xxxMODRS> with the server-stored values and <TRNUID>0 (zero) to indicate that the server modified the client-supplied values.

This last requirement does not distinguish between insignificant changes (case or abbreviations) and semantic differences (use of a completely different address for well-known payees). Although it is recommended that clients be notified of all insignificant storage discrepancies and modifications, it is required that clients be informed of all other such modifications.

In some cases, for security reasons, a server will not accept a value that is punctuated differently than expected. In this case, punctuation must conform to an expected format.

3.1.3 Echoing in Responses

A server should echo back unedited element values in the immediate response, but may store values in edited form. In the cases where the stored value is changed, it is recommended that the server respond with an out-of-band modification synchronization response whenever possible. For example, if a client sends a payee name of “Sears” but the server stores it as “SEARS”, the server should send a <PAYEEMODRS> in the next sync response. (See [Chapter 12, “Payments”](#) for clarification of payee issues.) However, if the server simply edits punctuation in or out of client-supplied numbers such as account numbers and will match both forms in future requests, it is not required to notify the client.

Any intermediate software should avoid any modifications to these values, thus avoiding the need to resolve this issue out-of-band.

3.1.4 Balance Records <BAL>

Several responses allow FIs to send an arbitrary set of balance information as part of a response, for example a bank statement download. FIs might want to send information on outstanding balances, payment dates, interest rates, and so forth. Balances can report the date the given balance reflects in <DTASOF>.

<i>Tag</i>	<i>Description</i>
<BAL>	Balance-response aggregate
<NAME>	Balance name, A-32
<DESC>	Balance description, A-80
<BALTYPE>	Balance type. DOLLAR = dollar (value formatted DDDD.cc) PERCENT = percentage (value formatted XXXX.YYYY) NUMBER = number (value formatted as is)
<VALUE>	Balance value. Interpretation depends on <BALTYPE> field, <i>amount</i>
<DTASOF>	Effective date of the given balance, <i>datetime</i>
<CURRENCY>	If dollar formatting, can optionally include currency
</CURRENCY>	
</BAL>	

3.1.5 Error Reporting <STATUS>

To provide as much feedback as possible to clients and their users, Open Financial Exchange defines a <STATUS> aggregate. The most important element is the code that identifies the error. Each response defines the codes it uses. Codes 0 through 2999 have common meanings in all Open Financial Exchange transactions. Codes from 3000 and up have meanings specific to each transaction.

Clients should assume the burden of checking the profile and not sending a transaction which the server does not support. If the client goes ahead and sends such a transaction, the server must ignore unrecognized and unsupported elements and aggregates within a request. Assuming no other problems occur in processing that request, servers may return warning code 2028 (Request element unknown). In either case (an unrecognized transaction or unknown elements or aggregates within a request), the response file should not contain the unrecognized tags or the contents of unrecognized aggregates.

The last 200 error codes in each assigned range of 1000 are reserved for server-specific status codes. For example, of the general status codes, 2800-2999 are reserved for status codes defined by the server. Of the banking status codes, codes 10800-10999 are reserved for the server. If a client receives a server-specific status code of <SEVERITY> ERROR that it does not know, it must handle it as a general error 2000.

Tag	Version	Description
<STATUS>		Error-reporting aggregate.
<CODE>		Error code, N-6
<SEVERITY>		Severity of the error: INFO = Informational only WARN = Some problem with the request occurred but a valid response still present ERROR = A problem severe enough that response could not be made
<MESSAGE>	V1 only	A textual explanation from the FI. Note that clients will generally have messages of their own for each error ID. Use this element only to provide more details or for the general errors. A-255
<MESSAGE2>	V2 only	A textual explanation from the FI. Note that clients will generally have messages of their own for each error ID. Use this element only to provide more details or for the general errors. A-2000
</STATUS>		

For general errors, the server can respond with one of the following <CODE> values. However, not all codes are possible in a specific context.

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR) Note: Servers should provide a more specific error whenever possible. Error 2000 should be reserved for cases in which a more specific code is not available.
2021	Unsupported version (ERROR)
2028	Requested element unknown (WARNING)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
15500	Signon invalid (See section 2.5.1) (ERROR)

Note: Clients will generally have error messages that are based on <CODE>. Therefore, do not use <MESSAGE> to replace that text. Use <MESSAGE> only to explain an error not well described by one of the defined codes, or to provide some additional information. <MESSAGE> should be returned whenever the <CODE> can be refined. For example, <CODE>2000 should always be accompanied with a <MESSAGE> explaining the problem.

3.2 Common Elements

This section defines elements used in several services of Open Financial Exchange. The format of the value is either character (A-*n*) or numeric (N-*n*) with a maximum length *n*; or as a named type. Section [3.2.8](#) describes the named types.

3.2.1 Client-Assigned Transaction UID <TRNUID>

Format: A-36

Open Financial Exchange uses <TRNUID>s to identify transactions within transaction wrappers (<xxxTRNRQ>, </xxxTRNRQ>).

In most cases, clients originate <TRNUID>s. When a client originates a <TRNUID>, the value of the <TRNUID> is always set to a unique identifier. The server must return the same <TRNUID> in the corresponding response and any later synchronization responses that include this response. Clients may use this <TRNUID> to match up requests and responses or to recognize synchronized responses for transactions they did not initiate. Servers can use <TRNUID>s to reject duplicate requests. Because multiple clients might be generating requests to the same server, transaction IDs must be unique across clients. Thus, <TRNUID> must be a globally unique ID.

In some cases, servers can originate a transaction that was not specifically requested by a client. For instance, a client might set up a recurring payment model. Although the client originates the payment model, the server originates the individual payments. Whenever the server originates a transaction, the value of the <TRNUIID> must be set to zero. Lite synchronization servers (see Chapter 6, "Data Synchronization") must respond to synchronization requests with information about all changes of this type.

The Open Software Foundation Distributed Computing Environment standards specify a 36-character hexadecimal encoding of a 128-bit number and an algorithm to generate it. Clients are free to use their own algorithm, to use smaller <TRNUIID>s, or to relax the uniqueness requirements. However, it is **RECOMMENDED** that clients allow for the full 36 characters in responses to work better with other clients.

For example: A client creates a new recurring payment using <RECPMTRQ> in a <RECPMTTRNRQ> with <TRNUIID>123. Later, the same client might cancel the model using <RECPMTCANRQ> in a <RECPMTTRNRQ> with <TRNUIID>456. The server would inform the client of any spawned payments using <PMTRS> responses with <TRNUIID>0 in later payment synchronization responses (<PMTSYNCRS>).

Usage: All services

3.2.2 Server-Assigned ID <SRVRTID>, <SRVRTID2>

Format: A-10 for <SRVRTID>, used in V1 message sets; A-36 for <SRVRTID2>, used in V2 message sets

A <SRVRTID> is a server-assigned ID for an object that is stored on the server. It should remain constant throughout the lifetime of the object on the server. The client will consider the SRVRTID as its "receipt" or confirmation and will use this ID in any subsequent requests to change, delete, or inquire about this object.

A <SRVRTID> is not unique across FI's or Service Providers, and clients might need to use FI + <SPNAME> + <SRVRTID> when a unique key is necessary.

Where the context allows, a server may use the same *value* for a given server object for both <SRVRTID> and <FITID>, but the client will not know this. In this case, the server must assign <SRVRTID> and <FITID> values that are more unique than otherwise required. Because of the differing uniqueness constraints on the individual elements, such a reused value must be unique throughout the FI.

For example: The server creates the new recurring model from the example in section [3.2.1](#) with <RECSRVRTID>1234:5687:1234. The server uses this identifier in the initial <RECPMTRS> and any synchronization responses that reference this model. The client references the same <RECSRVRTID> in the later <RECPMTCANRQ>.

If any payments are spawned from this model before it is cancelled, they would each have their own <SRVRTID> value (for example, <SRVRTID>8765:4321:8765 and <SRVRTID>1234:5678:1234). The <SRVRTID> value for one of the spawned payments may match the <RECSRVRTID> of the model. Such a match is not required for any spawned payment. To guarantee uniqueness of the payment identifiers, no more than one spawned payment may use the <RECSRVRTID> value of its model.

Usage: Payments, Banking

Elements of this type: PAYEEID2, PAYEELSTID2, RECSRVRTID, RECSRVRTID2, SRVRTID, and SRVRTID2

3.2.3 Financial Institution Transaction ID <FITID>

Format: A-255

An FI (or its Service Provider) assigns an <FITID> to uniquely identify a financial transaction that can appear in an account statement. Its primary purpose is to allow a client to detect duplicate responses. Open Financial Exchange intends <FITID> for use in statement download applications, where every transaction (not just those that are client-originated or server-originated) requires a unique ID.

An <FITID> also uniquely identifies the closing statement in <CLOSINGRS> and <CCCLOSINGRS>. Again, the OFX client should detect repeated closing statements (duplicate downloads) using these identifiers.

FITIDs must be unique within the scope of the requested transactions (that is, within an account) but need not be sequential or even increasing. Clients should be aware that FITIDs are not unique across FIs. If a client performs the same type of request within the same scope at two different FIs, clients will need to use FI + <ACCTID> + <FITID> as a globally unique key in a client database. That is, the <FITID> value must be unique within the account and Financial Institution (independent of the service provider).

Note: Although the specification allows FITIDs of up to 255 characters, client performance may significantly improve if servers use fewer characters. It is recommended that servers use 32 characters or fewer.

For example: The two spawned payments mentioned in section 3.2.1 are processed and later downloaded in a <STMTRS>. The first payment's <STMTRN> would list <SRVRTID>8765:4321:8765, <RECSRVRTID>1234:5678:1234, and <FITID>9999:8888:7777. The second payment would be described in a <STMTRN> containing <SRVRTID>1234:5678:1234, <RECSRVRTID>1234:5687:1234, and <FITID>6666:5555:4444.

Usage: Bank statement download, investment statement download

Elements of this type: <CORRECTFITID>, <FITID>, <RELFITID>, and <REVERSALFITID>

3.2.4 Token <TOKEN>, <TOKEN2>

Format: A-10 for <TOKEN>, used in V1 message sets; A-36 for <TOKEN2>, used in V2 message sets

Open Financial Exchange uses <TOKEN> as part of data synchronization requests to identify the point in history that the client has already received data, and in responses to identify the server's current end of history. See Chapter 6, "Data Synchronization," for more information.

<TOKEN> is unique within an FI and the scope of the synchronization request. For example, if the synchronization request includes an account ID, the <TOKEN> needs to be unique only within an account. Servers are free to use a <TOKEN> that is unique across the entire FI. Clients must save separate <TOKEN>s for each account, FI, and type of synchronization request.

Usage: All synchronization requests and responses

3.2.5 Transaction Amount <TRNAMT>

Format: Amount

Open Financial Exchange uses <TRNAMT> in any request or response that reports the total amount of an individual transaction.

Usage: Bank statement download, investment statement download, payments

3.2.6 Memo <MEMO>, <MEMO2>

Format: A-255 for <MEMO>, used in V1 message sets; A-390 for <MEMO2>, used in V2 message sets

A <MEMO> provides additional information about a transaction.

Usage: Bank statement download, investment statement download, payments, transfers

3.2.7 Date Start and Date End <DTSTART> <DTEND>

Format: *Datetime*

Clients use these elements in requests to indicate the range of response that is desired. Servers use these elements in responses to let clients know what the FI was able to produce.

In requests, the following rules apply:

- ◆ If <DTSTART> is absent, the client is requesting all available history (up to the <DTEND>, if specified). Otherwise, it indicates the *inclusive* date and time in history where the client expects servers to start sending information.
- ◆ If <DTEND> is absent, the client is requesting all available history (starting from <DTSTART>, if specified). Otherwise, it indicates the *exclusive* date and time in history where the client expects servers to stop sending information.

In responses, the following rules apply:

- ◆ <DTSTART> is the date and time where the server began *looking* for information, not necessarily the date of the earliest returned information. If the response <DTSTART> is later than the requested <DTSTART>, clients can infer that the user has not signed on frequently enough to ensure that the client has retrieved all information. If the user has been calling frequently enough, <DTSTART> in the response will match <DTSTART> in the request.
- ◆ <DTEND> is the date and time that, if used by the client as the next requested <DTSTART>, it would pick up exactly where the current response left off. It is the *exclusive* date and time in history where the server stopped *looking* for information, based on the request <DTEND> rules.

Because the system add date for a transaction is not necessarily the post date for the transaction (the latter occurring when the account is actually debited or credited), a server should consider the <DTSTART> and <DTEND> dates in a <(CC)STMTRQ> as a request for any transactions that were posted or added to the FI system at that time. In addition, the transactions returned should probably span a greater window of time than that included in the <DTSTART> / <DTEND> dates since a transaction might be added to the system after a statement download request was made for that time period. (Clients should be able to filter out the unnecessary transactions.) If a client is always requesting a download sequentially, through time, is never requesting an end date using <DTEND> and is always substituting <DTEND> in the response for <DTSTART> in the next request, it is safe for a server to return only those transactions that had a system add date on or after <DTSTART> in the request. In all cases, servers are minimally **required** to use a “system add datetime” as the basis for deciding which details match the requested date range. For example, if an FI posts a transaction dated Jan 3 to a user’s account on Jan 5, and a client connects on Jan 4 and again on Jan 6, the server is **required** to return that Jan 3-dated transaction when the client calls on Jan 6.

Usage: Bank statement download, investment statement download

3.2.8 Common Data Types

3.2.8.1 Dates, Times, and Time Zones

There is one format for representing dates, times, and time zones. The complete form is:

YYYYMMDDHHMMSS.XXX [*gmt offset:tz name*]

3.2.8.2 Date and Datetime

Elements specified as type *date* or *datetime* and generally starting with the letters “DT” accept a fully formatted date-time-timezone string. For example, “19961005132200.124[-5:EST]” represents October 5, 1996, at 1:22 and 124 milliseconds p.m., in Eastern Standard Time. This is the same as 6:22 p.m. Greenwich Mean Time (GMT).

Date and *datetime* also accept values with fields omitted from the right. They assume the following defaults if a field is missing:

<i>Specified date or datetime</i>	<i>Assumed defaults</i>
YYYYMMDD	12:00 AM (the start of the day), GMT
YYYYMMDDHHMMSS	GMT
YYYYMMDDHHMMSS.XXX	GMT

Note that times zones are specified by an offset and optionally, a time zone name. The offset defines the time zone. Valid offset values are in the range from -12 to +12 for whole number offsets. Formatting is +12.00 to -12.00 for fractional offsets, plus sign may be omitted.

Take care when specifying an ending date without a time. If the last transaction returned for a bank statement download was Jan 5 1996 10:46 a.m. and if the <DTEND> was given as just Jan 5, the transactions on Jan 5 would be resent. If results are available only daily, then just using dates and not times will work correctly.

Note: Open Financial Exchange does not require servers or clients to use the full precision specified. However, they are **REQUIRED** to accept any of these forms without complaint.

Some services extend the general notion of a *date* by adding special values, such as “TODAY.” These special values are called “smart dates.” Specific requests indicate when to use these extra values, and list the element as having a special data type.

3.2.8.3 Time

Elements specified as type *time* and generally ending with the letters “TM” accept times in the following format:

HHMMSS.XXX[*gmt offset:tz name*]

The milliseconds and time zone are still optional, and default to GMT.

3.2.8.4 Time Zone Issues

Several issues arise when a customer and FI are not in the same time zone, or when a customer moves a computer into new time zones. In addition, it is generally unsafe to assume that computer users have correctly set their time or time zone.

Although most transactions are not sensitive to the exact time, they often are sensitive to the date. In some cases, time zone errors lead to actions occurring on a different date than intended by the customer. For this reason, servers should always use a complete local time plus GMT offset in any datetime values in a response. If a customer’s request is for 5 p.m. EST, and a server in Europe responds with 1 a.m. MET the next day, a smart client can choose to warn the customer about the date shift.

Clients that maintain local state, especially of long-lived server objects, should be careful how they store datetime values. If a customer initiates a repeating transaction for 5 p.m. EST, then moves to a new time zone, the customer might have intended that the transaction remain 5 p.m. in the new local time, requiring a change request to be sent to the server. If, however, the customer intended it to remain fixed in server time, this would require a change in the local time stored in the client.

Client software that doesn’t know the current local time zone for the user, or client proxies that don’t know the current local time zone of their end users, should maintain and display the datetime value in the time zone indicated by the originator of the value and explicitly marked with that time zone. As an example, consider <DTPMTDUE> in section 11.5.4.2. If the biller gave a due date of 23:59pm EST on Dec 29, 1997, this is best displayed as 23:59pm EST rather than rendered in local time if there is any doubt at all as to the current local time zone of the end user looking at the due date.

When considering timezone conversions, remember the following differences between the *date* and *datetime* datatypes:

- ◆ *Date* = A date without time; this date is explicit. Clients and servers will not convert the value in any way. Examples include birth date and billing date.
- ◆ *Datetime* = A date and time format; clients and servers may convert this date to their local timezone. Examples include last account update date and bill summary fetch date.

Note: Developers should consider the possibility of a date change due to timezone conversion. A *datetime* value in the GMT timezone with a time of 12:00:00 (noon) would be converted to another time on the same date in every timezone. For example, 199812251200 remains Christmas Day in every timezone.

3.2.9 Amounts, Prices, and Quantities

3.2.9.1 Basic Format

Format: A-32

This section describes the format of numerical values used for amounts, prices, and quantities. In all cases, a numerical value that does not contain a decimal point has an implied decimal point at the end of the value. For example, a numerical value of “550” is equivalent to “550.” Trailing and leading spaces should be stripped. Number format uses a leading sign. Negative number format uses a minus sign (-). Positive number format uses a plus sign (+). The plus sign is implied for all amounts and can be omitted.

The following types are defined to have a maximum of 32 characters, including alphabetic characters, digits and punctuation. However, clients and servers may have specific limits for the maximum number of digits to the left or right of a decimal point. If a server cannot support a client request due to the size or precision of a number, the server should return status code 2012.

Amount: Amounts that do not represent whole numbers (for example, 540.32), must include a decimal point or comma to indicate the start of the fractional amount. Amounts should not include any punctuation separating thousands, millions, and so forth. The maximum value accepted depends on the client.

Quantity: Use decimal notation.

Unitprice: Use decimal notation. Unless specifically noted, prices should always be positive.

Rate: Use decimal notation, with the rate specified out of 100%. For example, 5.2 is 5.2%.

Some services define special values, such as INFLATION, which you can use instead of a designated value. Open Financial Exchange refers to these as “smart types,” and identifies them in the specification.

3.2.9.2 Positive and Negative Signs

Most OFX transaction aggregates describe the flow of funds. Amounts in transactions which clearly describe the flow of funds should normally be positive. For example, investment buys and sells, bank statement credits and debits should be positive.

Servers should sign amounts from the perspective of the user in cases where the flow of funds cannot be determined from the transaction aggregate alone. For example, interest amounts can be either positive or negative, depending on whether the interest is earned or paid. Servers should also sign amounts in cases of corrections to transaction. For example, a correction to an Investment Buy Mutual Fund transaction, BUYMF, would contain negatively signed UNITS.

3.2.10 Language

Language identifies the human-readable language used for such things as status messages and e-mail. *Language* is specified as a three-letter code based on ISO-639.

3.2.11 Other Basic Data Types

Boolean: Y = yes or true, N = no or false.

currsymbol: A three-letter code that identifies the currency used for a request or response. The currency codes are based on ISO-4217. For more information about currencies, refer to section 5.2.

URL: String form of a World Wide Web Uniform Resource Location. It should be fully qualified including protocol, host, and path. A-255.

URL2: URL2 is used in V2 message sets to handle URLs that might contain longer server names, security tokens and context data. String form of a World Wide Web Uniform Resource Location. It should be fully qualified including protocol, host, and path. A-1024.

CHAPTER 4 OFX SECURITY

OFX provides several options for ensuring the security of customer transactions. This chapter describes the OFX security framework, security goals, types of security, and financial institution (FI) responsibilities.

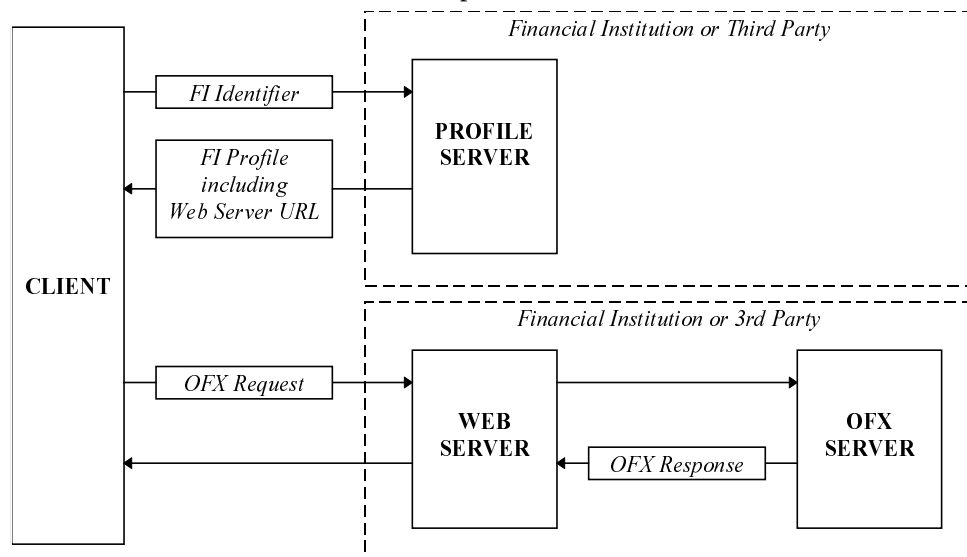
4.1 Security Concepts in OFX

4.1.1 Architecture

OFX security applies to the communication paths between a client and the profile server, a client and the Web server, and, when the OFX server is separate from the Web server, a client and the OFX server. The diagram below illustrates the initial order in which these communications occur, assuming that the client already has the URL for the FI profile server.

The bootstrap process for a client is:

- ◆ From the FI Profile Server, the client gets the URL of the FI Web server, so that it can retrieve a particular message set.
- ◆ The client sends an OFX request to the FI Web Server URL, from which it is forwarded to the OFX Server.
- ◆ The OFX Server sends back a response to the client via the Web Server.



4.1.2 Security Goals

The main goals of OFX security are:

- ◆ **Privacy:** Only the intended recipient can read a message. *Encryption* is a technique often used to ensure privacy.
- ◆ **Authentication:** The recipient of a message can verify the identity of the sender. In OFX, *passwords* allow an FI to authenticate a client, and *certificates* allow a client to authenticate a server.
- ◆ **Integrity:** A message cannot be altered after it is created. A cryptographic *hash* is often used to assist integrity verification.

OFX specifies the minimum security required for Internet transactions and provides several security options, based on existing standards. Through its choice of security techniques and related options, an FI can achieve privacy, authentication, and integrity with varying degrees of assurance. For example, there are many kinds of encryption algorithms, most of which can be strengthened or weakened by changing the key size.

4.1.3 Security Standards

Several standards underlie Type 1 security:

- ◆ Certificates (X.509 v3) are used to identify and authenticate servers, and to convey their public keys.
- ◆ PKCS #1 block type 2 is the encryption format specified by the recipe (See section [4.2.2.4.3](#)).
- ◆ RSA is the encryption algorithm.

4.1.3.1 Certificates and Certification Authorities

A certificate is a digitally signed document that binds a public key to an identity. It contains a public key that identifies information such as the name of the person or organization to whom the key belongs, an expiration date, a unique serial number, and additional descriptive information.

A certificate is useful for authentication because it is signed by a trusted third-party. This assures the verifier that the certificate has not been changed since it was signed. The entity which signs certificates is called a *certification authority*, or CA. A CA acts somewhat like a notary public: the reader of a document stamped by a notary public knows that the notary has checked the identity of the person who originated the document. By digitally signing someone's identity and public key, the CA affirms that the two go together.

If the client and server do not share a common CA, the client cannot validate the server's certificate. For this reason, OFX specifies a number of trusted CAs that all clients must accept and all servers must use.

Certificates are used in Type 1 security, as well as channel-level security through SSL. The format for these is defined by X.509 version 3. For more information, refer to ITU-T Rec. X.509, ISO / IEC 9594-8.

4.1.3.2 PKCS #1

The acronym, PKCS, stands for “Public Key Cryptography Standards,” a set of standards developed by a consortium and hosted by RSA. PKCS #1 is the RSA Encryption Standard, the rules for using RSA public key encryption. For the complete syntax of the PKCS #1 standard, refer to “Public-Key Cryptography Standards (PKCS)” published by RSA Data Security, Inc. at <http://www.rsa.com/>.

4.1.4 FI Responsibilities

OFX is designed with the understanding that there must be a security policy in place at each supporting financial institution. That policy must clearly delineate how customer data is secured, and how transactions are managed such that all parties to the transaction are protected according to accepted and recognized best common practices.

The decision regarding which users may perform a given operation on a given account must be determined by the financial institution. For example, is the specified user authorized to perform a transfer from the specified account? The financial institution must also determine whether the user has exceeded allowed limits on withdrawals, whether the activity on this account is unusual given past history, and other context-sensitive issues.

Although OFX provides many security options, an FI must support a minimal level of security. To ensure the proper security configuration, an FI must follow the steps outlined below.

1. Obtain one certificate for the profile server. This certificate must be rooted in one of the approved Certification Authorities (CAs). Establish appropriate safeguards for this certificate and its private key.
2. Obtain a certificate, rooted in an acceptable CA, for each OFX server, whether it is operated by the FI or by a third party.
3. Decide whether to use Type 1 application-level security for any message sets. For each message set to be secured by Type 1, obtain a certificate.

Type 1 security can be used on any message set, except for the Profile message set.

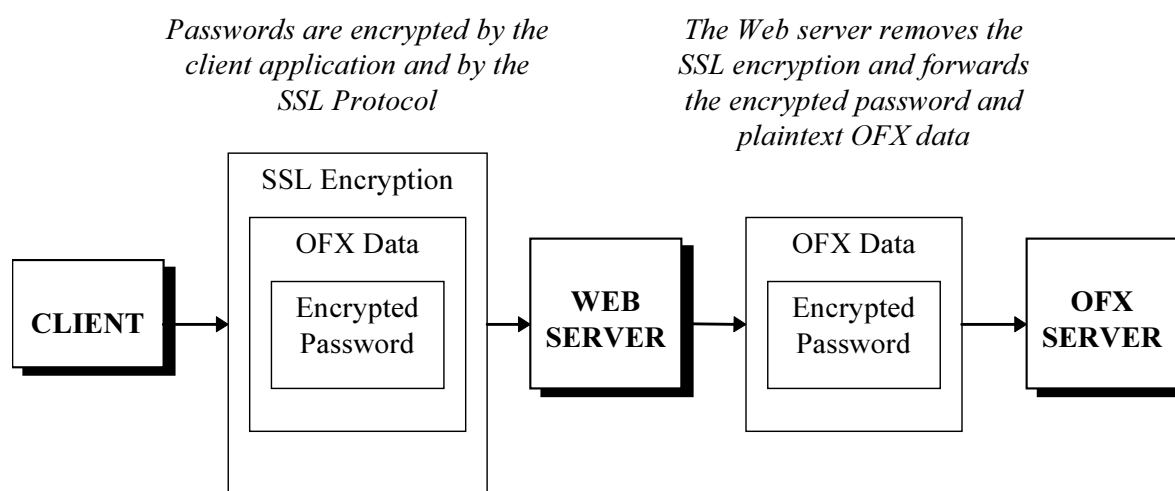
There are a number of other security issues beyond OFX proper, especially those relating to the Internet and network engineering. These issues are beyond the scope of this document. FIs are advised to conduct a complete security review of all servers associated with OFX.

4.1.5 Security Levels: Channel vs. Application

With OFX, security can be applied at two different levels in the message exchange process.

- ◆ **Channel level:** Generally transparent to a client or server, channel-level security is built into the communication process, protecting messages between two ends of the “pipe.” To secure messages during HTTP transport, client and server applications use the Secure Sockets Layer (SSL) protocol. SSL transparently protects messages exchanged between the client and the destination Web server. SSL authenticates the destination Web server using the Web server’s certificate. Additionally, it provides privacy via encryption, and SSL-record integrity, i.e. the block of data sent in each transmission cannot be altered without detection.
- ◆ **Application level:** Transparent to and independent of the transport process, application-level security protects the user password sent from the client application all the way to the server application that handles the OFX messages. The server application typically resides beyond the destination Web server, secured behind an Internet firewall. Application-level security requires channel-level security.

The following diagram illustrates how channel-level and application-level security relate. The diagram shows the path of a request from the client to the server when application-level encryption is used.



Channel-level security is sufficient for most message sets, provided that the network architecture at the destination is adequately secure; however, application-level password encryption can allow a more flexible back-end architecture with a high level of security.

4.2 Security Implementation in OFX

4.2.1 Channel-Level Security

4.2.1.1 Specification in FI Profile

For each message set listed in the FI profile response, the <MSGSETCORE> aggregate describes the channel-level security required for that message set.

The <TRANSPSEC> element defines whether or not channel-level security is required. It can have one of the following values:

<i>Tag</i>	<i>Description</i>
N	Do not use any channel-level security
Y	Use channel-level security

All currently defined message sets require channel-level security.

4.2.1.2 SSL Protocol

Secure Sockets Layer (SSL) is a cryptographic protocol commonly used for channel-level security on the Internet. Central to the security of SSL is the *server certificate*. This certificate assures clients that the server is who it claims to be. It contains the public key of the server, which the client uses to encrypt the session keys it generates as part of each connection.

All of this function is available without significant software development on either the client or server side; however, the client and server must be configured to use appropriate encryption algorithms (CipherSuites). In addition, clients and servers must share a trusted root certificate, or the client will not be able to validate the server's certificate.

Note: Although SSL supports client-side certificates to allow a server to authenticate a client, OFX does not require them at this time. To identify and authenticate a customer, servers should use the information provided in the signon request <SONRQ>.

Setting the <TRANSPSEC> element to Y means that the client must use SSL v3 or higher.

4.2.1.3 Trusted Certificate Authorities

Both channel-level and application-level security rely on clients and servers having at least one trusted certification authority (CA) in common. To ensure that clients can test the validity of a certificate, servers must have their certificates signed by an approved OFX CA. Clients are assumed to have access to this trusted CA.

4.2.1.4 CipherSuites

The following SSL CipherSuites are approved for use with OFX:

- ◆ SSL_RSA_WITH_RC4_128_SHA
- ◆ SSL_RSA_WITH_IDEA_CBC_SHA
- ◆ SSL_RSA_WITH_DES_CBC_SHA
- ◆ SSL_RSA_WITH_3DES_EDE_CBC_SHA
- ◆ SSL_DH_DSS_WITH_DES_CBC_SHA
- ◆ SSL_DH_DSS_WITH_3DES_EDE_CBC_SHA
- ◆ SSL_DH_RSA_WITH_DES_CBC_SHA
- ◆ SSL_DH_RSA_WITH_3DES_EDE_CBC_SHA
- ◆ SSL_DHE_DSS_WITH_DES_CBC_SHA
- ◆ SSL_DHE_DSS_WITH_3DES_EDE_CBC_SHA
- ◆ SSL_DHE_RSA_WITH_DES_CBC_SHA
- ◆ SSL_DHE_RSA_WITH_3DES_EDE_CBC_SHA

Other CipherSuites are not approved.

4.2.1.5 Key Size

Signing keys must be either RSA with a minimum 1024-bit modulus, or DSS with a 1024-bit modulus.

Server RSA keys and Diffie-Hellman keys must both have a minimum 1024-bit modulus. The Diffie-Hellman base must be primitive.

4.2.2 Application-Level Security

4.2.2.1 Specification in FI Profile

For each message set listed in the FI profile response, the <MSGSETCORE> aggregate describes the security required for that message set.

The <OFXSEC> element defines the type of application-level security required for the message set. <OFXSEC> can have one of the following values, which also are used in the SECURITY element of the OFX headers:

Tag	Description
NONE	Do not use any application-level security
TYPE1	Use Type 1 application-level security

Application-level security requires channel-level security.

4.2.2.2 Type 1 Protocol Overview

The goal of the Type 1 protocol is to protect the user password all the way to the destination OFX server. In the absence of client certificates, this password is the primary vehicle for client authentication and is therefore worthy of special consideration.

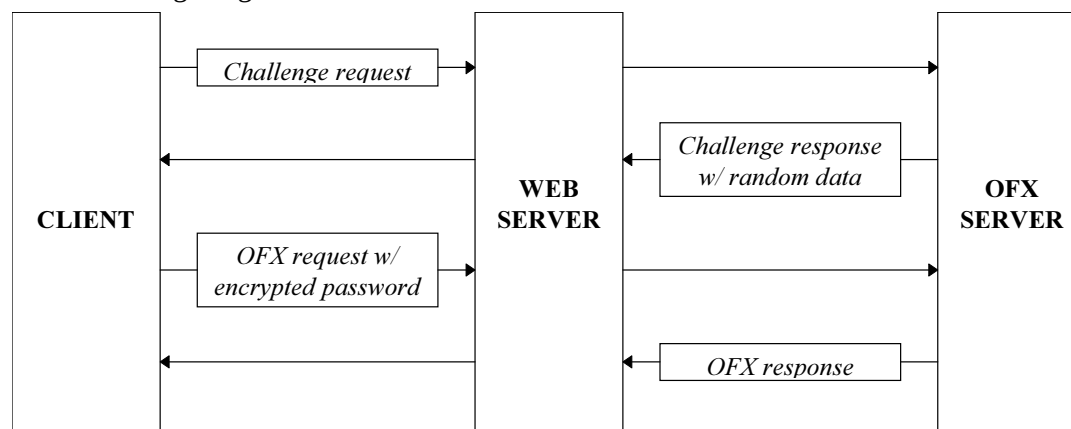
Type 1 requires channel-level security, *i.e.* SSL. Though the password is well protected by SSL alone in the client to Web server connection, the server-side network architecture may render the password less secure while it is in transit between the Web and OFX servers. With Type 1, the user password is not decrypted until the request reaches the OFX server.

Type 1 applies only to the request part of a message; the server response is unaffected.

A simple approach would be to deliver the server's Type 1 certificate in the profile and use it to encrypt the password, but that would permit a *replay attack*. An attacker could capture a transaction, including encrypted password, and replay it to the server. It wouldn't matter that the password remained unknown.

To prevent the *replay attack*, the server introduces some random data to the process, data which is unpredictably different for each transmission. The client asks for the random data with a challenge request. The server sends it, along with its Type 1 certificate, in the challenge response. The client then uses that random data in the encryption process, thereby assuring the server that the client response is associated with this and only this interaction.

The following diagram illustrates:



4.2.2.3 Type 1 Protocol Notation

In this section, the expression, $C = E_A(M)$, means that plain text M is encrypted either symmetrically or asymmetrically with key A into ciphertext C . The expression, $M = D_A(C)$ signifies the inverse operation (decryption), in which ciphertext C is decrypted into plain text M using key A . If C was encrypted asymmetrically, then A in the latter case is understood to be the private component of the key. The expression, $A || B$, indicates that B is concatenated to A .

4.2.2.4 Type 1 Protocol Implementation

Type 1 application-level security provides additional password secrecy. These are the steps for conducting a Type 1 transaction (unless otherwise noted, the term “Server” in this section refers to the Financial Institution Server):

1. Client obtains the Server’s profile from the Profile Server (see [Chapter 7, "FI Profile"](#))
2. Client establishes an SSL connection with the Server (see section [4.2.1](#))
3. Client sends <CHALLENGERQ> to Server (see section [4.2.2.4.1](#))
4. Server sends <CHALLENGERS> which contains a nonce and the Server’s Type 1 certificate (see section [4.2.2.4.2](#))
5. Client builds a transaction request and sends it to the Server (see section [4.2.2.4.3](#))
6. Server parses the request, verifying the user password, and either rejects or processes the transaction (see section [4.2.2.4.4](#))

The following table lists data elements used in the Type 1 protocol:

<i>Field</i>	<i>Type</i>	<i>Description</i>
BT	octet, length 1	Block Type byte. BT = 0x02
CT1	octet string, length 128	Ciphertext: the PKCS #1 RSA encryption of EB with KS. $CT1 = E_{KS}(EB)$
CT2	printable ASCII, length 171	Encoded Ciphertext: the RADIX-64 encoding of CT1 (see RFC 1113, §4.3.2.4 and §4.3.2.5). $CT2 = RADIX64(CT1)$
D	octet string, length 68	Data: the user data to be encrypted. $D = NC P T$
EB	octet string, length 128	Encryption Block: the formatted plain text block, ready for encryption. $EB = 0x00 BT PS 0x00 D$
KS	RSA key, modulus length 1,024 bits	Server's Type 1 RSA key
NC	octet string, length 16	Client Nonce: string of random octets generated by the Client
NS	octet string, length 16	Server Nonce: string of random octets generated by the Server
P	printable ASCII, null-padded, length 32	Password: shared by the Client and Financial Institution, null-padded on the right
PS	octet string, length 57	Padding String: each octet is pseudo-random and non-zero
T	octet string, length 20	Authentication Token. $T = SHA1(NS P NC)$

```

struct {
    unsigned char nc[16];
    unsigned char p[32];
    unsigned char t[20];
} D;
struct {
    unsigned char null1 = 0x00;
    unsigned char bt = 0x02;
    unsigned char ps[57];
    unsigned char null2 = 0x00;
    struct D d;
} EB;

```

4.2.2.4.1 Challenge request

Client sends a <CHALLENGERQ> to the Server.

4.2.2.4.2 Challenge response

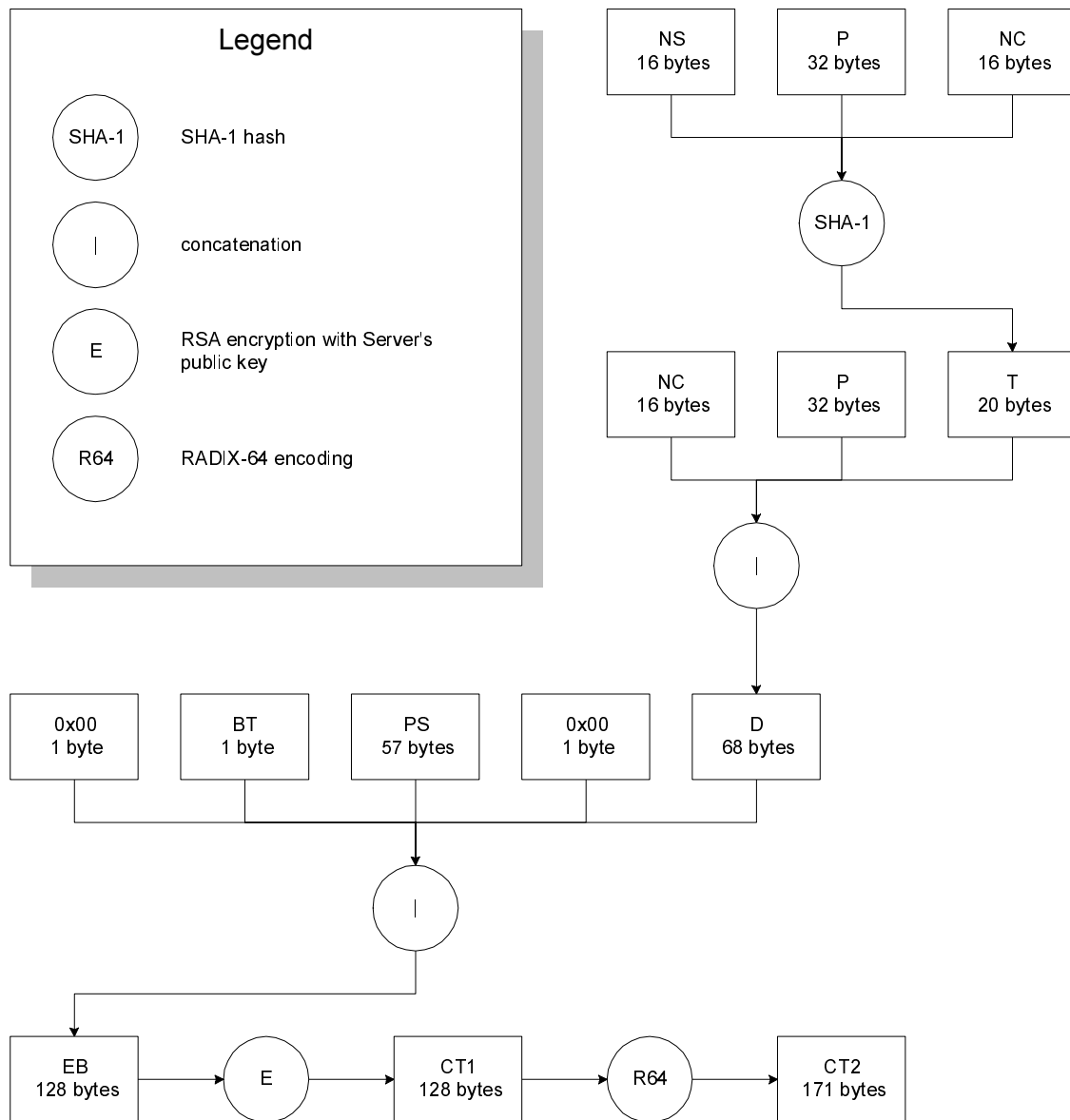
Server sends a <CHALLENGERS> to the client. This response contains the Server's Type 1 certificate and NS.

4.2.2.4.3 Building the OFX Request

1. Client generates 16 random octets and places them in NC (see RFC 1750 for recommendations on entropy generation)
2. Client obtains the User's password (P)
3. Client computes $T = \text{SHA1}(\text{NS} || \text{P} || \text{NC})$
4. Client generates 57 pseudo-random, non-zero octets and places them in PS (NC may be used to seed the pseudo-random number generator)
5. Client sets $D = \text{NC} || \text{P} || T$
6. Client sets $\text{EB} = 0x00 || \text{BT} || \text{PS} || 0x00 || D$
7. Client RSA-encrypts EB using the Server's Type 1 public key (obtained from the Server's Type 1 certificate): $\text{CT1} = E_{KS}(\text{EB})$ (see PKCS #1, §§8.2-8.4)
8. Client encodes the ciphertext for transport: $\text{CT2} = \text{RADIX64}(\text{CT1})$. See RFC 1113, §4.3.2.4 and §4.3.2.5. This is a standard encoding method supported by RSA's Bsafe library and others.
9. Client constructs the body of its OFX request
10. Client copies CT2 to the <USERPASS> field of the OFX <SONRQ>
11. Client sends the complete OFX request to the Server

In <PINCHRQ>, the steps are identical, except that in step 2, P is set to <NEWUSERPASS> and in step 10, CT2 is copied to the <NEWUSERPASS> field of the <PINCHRQ>.

The diagram below illustrates the creation of CT2.



4.2.2.4.4 Parsing the OFX Request

1. Server reads the OFX SECURITY header in the request file to ascertain whether Type 1 processing should be used on this message. If Type 1 is not used, skip to step 6.
2. Server extracts CT2 from the <USERPASS> field of the OFX <SONRQ> and removes the encoding to obtain CT1 (see RFC 1113, §4.3.2.4 and §4.3.2.5)
3. Server decrypts CT1 to obtain EB: $EB = D_{KS}(CT1)$ (see PKCS #1, §9)
4. Server extracts D from EB, then extracts NC, P, and T from D
5. Server looks up the Client's password in its database, and computes $SHA1(NS || P || NC)$. If the result does not match T, Server terminates the session and reports the error to the client
6. Server processes the request and returns confirmation to the Client

In <PINCHRQ>, the steps are identical except that in step 2, CT2 is obtained from the <NEWUSERPASS> field of the <PINCHRQ> and in step 5, the server does not look up the extracted new password in a database.

CHAPTER 5 INTERNATIONAL SUPPORT

5.1 Language and Encoding

Most of the content in OFX is language-neutral. However, some error messages, balance descriptions, and similar elements contain text meant to appear to the financial institution customers. There are also cases, such as e-mail records, where customers need to send text in other languages. To support worldwide languages, OFX must identify the basic text encoding, character set, and language.

The OFX headers specify the encoding and character set, as described in [Chapter 2, "Structure."](#) Current encoding values are USASCII and UTF-8. For USASCII, character set values are code pages. UTF-8 ignores the character set *per se* although it still requires the syntax. UTF-8 is a form of UNICODE defined in International ISO / IEC 10646-1, amendment 2. Servers must respond with the encoding and character set requested by the client.

CHARSET defines the character set used for character data. The values for CHARSET may be ISO-8859-1 (Latin-1), 1252 (Windows Latin-1), or NONE. Any value specified here is likely to be ignored by an OFX client or server.

Clients identify the language in the signon request. OFX specifies languages by three-letter codes as defined in ISO-639. Servers report their supported languages in the profile (see [Chapter 7, "FI Profile"](#)). If a server cannot support the language requested by the client, it must return an error and not process the rest of the transactions.

5.2 Currency <CURDEF> <CURRENCY> <ORIGCURRENCY>

In each transaction involving amounts, responses include a default currency identification, <CURDEF>. The values are based on the ISO-4217 three-letter currency identifiers.

Within each transaction, specific parts of the response might need to report a different currency. Where appropriate, aggregates include an optional <CURRENCY> aggregate. The scope of a <CURRENCY> aggregate is everything within the same aggregate that the <CURRENCY> aggregate appears in, including nested aggregates, unless overridden by a nested <CURRENCY> aggregate. For example, specifying a <CURRENCY> aggregate in an investment statement detail means that the unit price, transaction total, commission, and all other amounts are in terms of the given currency, not the default currency.

Note that there is no way for two or more individual elements that represent amounts—and are directly part of the same aggregate—to have different currencies. For example, there is no way in a statement download to have a different currency for the <LEDGERBAL> and the <AVAILBAL>, because they are both directly members of <STMTRS>. In most cases, you can use

the optional <BAL> aggregates to overcome this limitation, since <BAL> aggregates accept individual <CURRENCY> aggregates.

The default currency for a request is the currency of the source account. For example, the currency for <BANKACCTFROM>.

The <CURRATE> should be the one in effect throughout the scope of the <CURRENCY> aggregate. It is not necessarily the current rate. Note that the <CURRATE> needs to take into account the choice of the FI for formatting of amounts (that is, where the decimal is) in both default and overriding currency, so that a client can do math. This can mean that the rate is adjusted by orders of magnitude (up or down) from what is commonly reported in newspapers.

Tag	Description
<CURRENCY> or <ORIGCURRENCY>	Currency aggregate
<CURRATE>	Ratio of <CURDEF> currency to <CURSYM> currency, in decimal form, <i>rate</i>
<CURSYM>	ISO-4217 3-letter currency identifier, <i>currsymbol</i>
</CURRENCY> or </ORIGCURRENCY>	

In some cases, OFX defines transaction responses so that amounts have been converted to the home currency. However, OFX allows FIs to optionally report the original amount and the original (foreign) currency. In these cases, transactions include a specific aggregate for the original amount, and then an <ORIGCURRENCY> aggregate to report the details of the foreign currency.

Again, <CURRENCY> means that OFX *has not* converted amounts. Whereas, <ORIGCURRENCY> means that OFX *has* already converted amounts.

5.3 Country-Specific Element Values

Some of the elements in OFX have values that are country-specific. For example, <USPRODUCTTYPE> is useful only within the United States. OFX will extend in each country as needed to provide elements that accept values useful to that country. Clients in other countries that do not know about these elements must simply skip them.

In some cases, an element value represents a fundamental way of identifying something, yet there does not exist a world-wide standard for such identification. Examples include bank accounts and securities. In these cases, OFX must define a single, extensible approach for identification. For example, CUSIPs are used within the U.S., but not in other countries. However, CUSIPs are fundamental to relating investment securities, holdings, and transactions. Thus, a security ID consists of a two-part aggregate: one to identify the naming scheme, and one to provide a value. OFX will define valid naming schemes as necessary for each country.

CHAPTER 6 DATA SYNCHRONIZATION

6.1 Overview

Currently, some systems provide only limited support for error recovery and no support for backup files or multiple clients. This chapter defines OFX's powerful means of data synchronization between clients and servers.

OFX data synchronization addresses the following problems:

- ◆ Error recovery
- ◆ Use of multiple data files, including multiple client applications
- ◆ Restoring from an outdated backup file

This chapter first provides a brief introduction to synchronization problems and then presents the strategy used in OFX to ensure data integrity. Additional details about synchronization requests and responses may be found in the relevant sections of this document. The final section in this chapter discusses alternatives to full synchronization and summarizes the options for each.

6.2 Background

When a connection between the client and the server does not successfully complete, there are two main areas of concern:

- ◆ Unconfirmed requests
If a client does not receive a response to work it initiates, it has no way of knowing whether the server processed the request. It also does not have any server-supplied information about the request, such as a server ID number.
- ◆ Unsolicited data
Some message sets allow a server to send data to the client without first receiving a request. OFX assumes that the first client to connect after the unsolicited data is available receives it. If the connection fails, this information could be forever lost to the client. Examples of unsolicited data include updates to the status of a bill payment and e-mail messages.

Unsolicited data presents problems beyond error recovery. Because the first client that connects to a server is the only one to receive unsolicited data, this situation precludes use of multiple clients without a data synchronization method. For example, if a user has a computer at work and one at home, and wants to perform online banking from both computers, a bank server could send unsolicited data to one but not the other.

An even greater problem occurs when a user resorts to an outdated backup copy of the client data file. This backup file may be missing recent unsolicited data with no way to retrieve it from the server again.

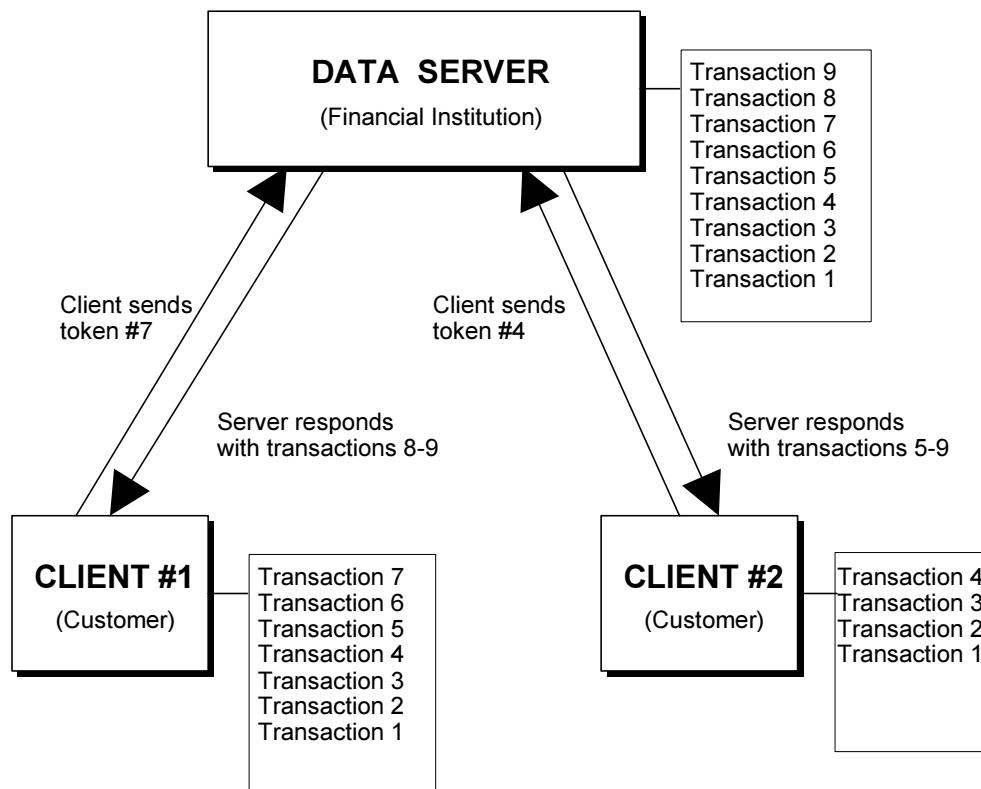
6.3 Data Synchronization Approach

A simple solution is to make sure that clients can always obtain information from the server for a reasonable length of time after it is initially sent. Clients can request recent responses—whether due to client-initiated work or other status changes on the server—by supplying the previous endpoint in the response history. Servers should always supply a new endpoint whenever they supply responses. These endpoints are described by the <TOKEN> element.

To ensure a consistent state after a failure (for example, dropped client connections or a client crash before updating its database), the client must store all data returned in a sync response before updating the saved token for that account and object type. After a failure, the next sync attempt using the old token might download information already reflected in the client database. But, re-integration of that data is much preferred over losing all changes between the old and new token values.

If a user switches to an outdated backup file, then the most recent endpoint known to the client will be older than the most recent endpoint known to the server.

If multiple clients are in use, each will send requests based on its own current endpoint, so that both clients will obtain complete information from the server. This is one reason why OFX responses carry enough information from the request to enable them to be processed independent from the requests. The diagram below shows the interaction between clients and servers.



OFX relieves the server from maintaining any special error-recovery state information. However, OFX requires the server to maintain a history of individual responses and a <TOKEN> to identify a position in the history. This token is commonly a time stamp, but it need not be. Because of the freedom a server has in choosing values for its <TOKEN>s, a client must not assume any sequential relationship between <TOKEN>s based on the <TOKEN> values.

Note: OFX does not require servers to store responses based on individual connections. Also, not all requests are subject to synchronization. For example, OFX does not require servers to store statement-download responses separately for data synchronization.

6.4 Data Synchronization Specifics

OFX performs synchronization separately for each type of response. In addition, a synchronization request might include further identifying information, such as a specific account number. This specification defines the additional information for each synchronization request.

Each OFX service identifies the objects that are subject to data synchronization. For example, a bank-statement download is a read-only operation from the server. A client can request it again; consequently, there is no data synchronization for this type of response.

6.4.1 Tokens

The basis for synchronization is a *token* as defined by the <TOKEN> element. The server can create a token in any way it wishes. The client simply holds the token for possible use in a future synchronization request.

The server can derive a token from one of the following:

- ◆ Time stamp
- ◆ Sequential number
- ◆ Unique nonsequential number
- ◆ Other convenient values for a server

OFX reserves the following tokens:

- ◆ <TOKEN>0 (zero) requests all available history for the referenced account (if specified) and object type. Servers should send all relevant transactions that are accessible, allowing a new client to know about work done by other clients. If a user's account has never been used with OFX, the server returns no history.
- ◆ Servers should return <TOKEN>-1 (negative one) in the event they must respond with an error. Since there is no <xxxTRNRS> wrapper inside the <xxxSYNCRS> wrapper, the <SYNCERROR> element was added in version 2 to allow the server to return the value that would normally be in the <CODE> field of the <STATUS> aggregate. For more information, see section [6.4.4](#).

In all other cases, the server can use different types of tokens for different types of responses, if suitable for the server.

Clients must send either a <REFRESH>Y request (if supported by the server) or <TOKEN>0 in their initial synchronization request for each account (if necessary) and object type. As described in section [6.6](#), a server's response to either request should bring the client up-to-date. The <REFRESH>Y response would not detail how or when an object reached its current state. But, the <TOKEN>0 response might not list every relevant object (for example, some early history that the server has already purged, which might include a payment that was scheduled far in the past, but not yet due.) Should the client require full history information initially, OFX recommends a <REFRESH>Y request together with a <TOKEN>0 request.

Tokens can contain up to 10 characters in V1 message sets, and 36 in V2 message sets; see [Chapter 3, "Common Aggregates, Elements, and Data Types."](#) Tokens must be unique only with respect to the type of synchronization request and the additional information in that request. For example, a bill payment synchronization request takes an account number; therefore, a token needs to be unique only within payments for the account. In sync requests which do not include an account number, token values are scoped to the current user. For example, a token in a payee synchronization request needs to be unique only within payees for the signed on user.

The server can use different types of tokens for different types of responses, if suitable for the server.

Servers will not have infinite history available, so synchronization responses can optionally include a <LOSTSYNC>Y (yes) if the old token in the synchronization request was older than the earliest available history. This element allows clients to alert users that some responses have been lost.

Note: Tokens are unrelated to <TRNUID>s, <SRVRTID>s, and <FITID>s, each of which serves a specific purpose and has its own scope and lifetime.

A <SRVRTID> is not appropriate as a <TOKEN> for bill payment. A single payment has a single <SRVRTID>, but it can undergo several state changes over its life and thus have several entries in the token history.

6.4.2 The Synchronization Process

There are three different ways a client and a server can conduct their requests and responses:

- ◆ Explicit synchronization—A client can request synchronization without sending any other OFX requests. The client sends a synchronization request, including the current token for that type of request. The response includes responses more recent than the given token, along with the current token.
- ◆ Synchronization with new requests—A client can request synchronization as part of any new request. The client gives the latest token it has. The response includes responses to the new requests plus any others that became available since the time of the token in the request, along with the current token. An aggregate contains the requests so that the server can process the new requests and update the token as a single action.
- ◆ New requests without synchronization—A client can make new requests without providing a token. In this case, it expects only responses to the new requests. A subsequent request for synchronization will cause the server to send this response again, because the client did not receive the current token.

SYNC responses should return a new <TOKEN> only if new activity was generated for a set of transactions (e.g. payee, payment, intrabank transfers, payment email, etc.). Alternatively, if a server always returns a new <TOKEN> even if no new activity was generated, the server should remember that the old and new <TOKEN> values are both up-to-date with respect to <REJECTIFMISSING>Y

Each request and response that requires data synchronization will define a synchronization aggregate. The aggregate tells the server which kind of data it should synchronize. By convention, these aggregates always have SYNC as part of their names, for example, <PMTSYNCRQ>. These aggregates can be used on their own to perform explicit synchronization, or as wrappers around one or more new transactions. For example, <PMTSYNCRQ> aggregates request synchronization and may include new work.

Some clients can choose to perform an explicit synchronization before sending any new requests. This practice allows clients to be up-to-date before sending any new requests. Other clients can simply send new requests as part of the synchronization request.

If a client synchronizes in one file, then sends new work inside a synchronization request in a second file, there is a small chance that additional responses became available between the two connections. There is an even smaller chance that these would be conflicting requests, such as modifications to the same object. However, some clients and some requests might require absolute control, so that the user can be certain that they are changing known data. To support this, synchronization requests can optionally specify <REJECTIFMISSING> element. The element tells a server that it should reject all enclosed requests if the supplied <TOKEN> is out of date before considering the new requests. That is, if any new responses became available, whether related to the incoming requests or not (but in scope of the synchronization request), the server should immediately reject the requests. It should still return the new responses. A client can then try again until it finds a stable window to submit the work. See section [6.5](#) for more information about conflict detection and resolution.

Note: If <REJECTIFMISSING>Y causes enclosed requests to be rejected, this rejection can be done in one of two ways:

- ◆ Embedded requests are completely ignored – they are not included in the response.
- ◆ Embedded requests are returned with a 2000 (or 6502 for recent servers) error. This is the preferred approach.

The password change request and response present a special problem. See section [2.5.2](#) for further information.

6.4.3 Synchronizable Objects

OFX allows synchronization of email (in all message sets), service activations, changes to user information, stop checks, banking notifications, transfers (both types), recurring transfers (both types), wire transfers, payees, payments, and recurring payments. OFX includes the following synchronization request/response pairs.

<i>Section</i>	<i>Request</i>	<i>Response</i>
<u>8.6.4</u>	<ACCTSYNCRQ>	<ACCTSYNCRS>
<u>8.7</u>	<CHGUSERINFOSYNCRQ>	<CHGUSERINFOSYNCRS>
<u>9.2.4</u>	<MAILSYNCRQ>	<MAILSYNCRS>
<u>11.12.1</u>	<STPCHKSYNCRQ>	<STPCHKSYNCRS>
<u>11.12.2</u>	<INTRASYNCRQ>	<INTRASYNCRS>
<u>11.12.3</u>	<INTERSYNCRQ>	<INTERSYNCRS>
<u>11.12.4</u>	<WIRESYNCRQ>	<WIRESYNCRS>
<u>11.12.5</u>	<RECINTRASYNCRQ>	<RECINTRASYNCRS>
<u>11.12.6</u>	<RECINTERSYNCRQ>	<RECINTERSYNCRS>
<u>11.12.7</u>	<BANKMAILSYNCRQ>	<BANKMAILSYNCRS>
<u>12.8.2</u>	<PMTMAILSYNCRQ>	<PMTMAILSYNCRS>
<u>12.9.4</u>	<PAYEESYNCRQ>	<PAYEESYNCRS>
<u>12.10.1</u>	<PMTSYNCRQ>	<PMTSYNCRS>
<u>12.10.2</u>	<RECPMTSYNCRQ>	<RECPMTSYNCRS>
<u>13.10.2</u>	<INVMAILSYNCRQ>	<INVMAILSYNCRS>
<u>14.6</u>	<PRESMAILSYNCRQ>	<PRESMAILSYNCRS>

6.4.4 <SYNCERROR> Status Codes

The <SYNCERROR> element within synchronization aggregates, allows the server to return the value that would normally be in the <CODE> field of the <STATUS> aggregate. Values for <SYNCERROR> can include, but are not limited to, the following.

Code	Meaning
0	Success (INFO)
2000	General error (ERROR) The server may return <SYNCERROR>2000 when the history query itself failed. For example, this error would be appropriate when the server is unable to reach its data store.
2002	General account error (ERROR)
2003	Account not found (ERROR)
2004	Account closed (ERROR)
2005	Account not authorized (ERROR)
6500	<REJECTIFMISSING>Y invalid without <TOKEN>(ERROR).
6501	Embedded transactions in request failed to process: Out of date (WARNING)
15500	Signon invalid (ERROR)

<SYNCERROR> codes 2002, 2003, 2004, and 2005 must not be used with the <MAILSYNCRS>, <ACCTSYNCRS>, <CHGUSERINFOSYNCRS>, <PMTMAILSYNCRS>, or <PAYEESYNCRS> messages since those message sets do not include fields to identify a specific account.

When <SYNCERROR> is any value other than 0, any requests embedded within the synchronization aggregate should return a transaction response <xxxTRNRS> aggregate containing the same error code except for using <STATUS><CODE>6502 when <SYNCERROR>6501 appears in the surrounding wrapper. Such requests should not result in a response <xxxRS> aggregate.

6.5 Conflict Detection and Resolution

Conflicts arise whenever two or more clients or servers modify the same data. This can happen to any object that has a <SRVRTID> that supports change or delete requests. For example, two spouses might independently modify the same recurring bill payment model. From a server perspective, there is usually no way to distinguish between the same user making two intended changes and two separate users making perhaps unintended changes. Therefore, OFX provides enough tools to allow clients to detect and resolve conflicts.

A careful client always synchronizes before sending any new requests. If any responses come back that could affect a user's pending requests, the client can ask the user whether it should still

send those pending requests. Because there is a small chance for additional server actions to occur between the initial synchronization request and sending the user's pending requests, extremely careful clients can use the <REJECTIFMISSING> element. Clients can iterate sending pending requests inside a synchronization request with <REJECTIFMISSING> and testing the responses to see if they conflict with pending requests. A client can continue to do this until a window of time exists wherein the client is the only agent trying to modify the server. In reality, this will almost always succeed on the first try.

6.6 Synchronization Options

There are some situations and some types of clients for which it is preferable that the client ask the server to send—by way of a refresh—everything it knows, rather than just a set of changes by way of a synch response. For example, a client that has not connected often enough may have lost synchronization, a user may create a new data file, or the user might be using a completely stateless client, such as a Web browser.

Note: OFX does not require a client to refresh just because it has lost synchronization.

Clients will mainly want to refresh lists of long-lived objects on the server; generally objects with a <SRVRTID>. A brand new client, or a client that lost synchronization, might want to learn about in-progress payments by doing a synchronization refresh of the payment requests. It would almost certainly want to do a synchronization refresh of the recurring payment models, because those often live for months or years.

A client may request a refresh by using <REFRESH>Y instead of the <TOKEN> element. Servers must send responses that emulate a client creating or adding each of the objects governed by the particular synchronization request.

When responding to a <REFRESH>Y sync request, servers must send <TRNUID>0 in each contained transaction wrapper, the standard value for server-generated responses (except responses for embedded transactions).

There is no need to recreate a stream of responses that emulate the entire history of the object. An add response that reflects the current state is sufficient. For example, if you create a model and then modify it several times, even if this history would have been available for a regular synchronization, servers should only send a single add that reflects the current state.

Due to the large volume of data which might be included in the response, clients should not perform <MAILSYNCRQ> (or, one of the service-specific equivalents such as <BANKMAILSYNCRQ>) with <REFRESH>Y.

A client that wants only the current token, without refresh or synchronization, makes requests with <TOKENONLY>Y.

In all cases, servers should send the current ending <TOKEN> for the synchronization request in refresh responses. This allows a client to perform regular synchronization requests in the future.

The following table summarizes the options in a client synchronization request:

Tag	Version	Description
Client synchronization option; <TOKEN>, <TOKEN2>, <TOKENONLY>, or <REFRESH>		
<TOKEN>	V1 only	Previous value of <TOKEN> received for this type of synchronization request from server; 0 for first-time requests; <i>token</i>
<TOKEN2>	V2 only	Previous value of <TOKEN2> received for this type of synchronization request from server; 0 for first-time requests; <i>token2</i>
<TOKENONLY>		Request for just the current <TOKEN> without the history, <i>Boolean</i>
<REFRESH>		Request for refresh of current state, <i>Boolean</i>
<REJECTIFMISSING>		If Y, do not process requests if client <TOKEN> is out of date, <i>Boolean</i>

Note: Compliant clients should not send synchronization requests matching those listed below. Nonetheless, servers should handle such requests and respond as described.

- ◆ <TOKENONLY>N has the same meaning as <TOKENONLY>Y and should be treated identically.
- ◆ <REFRESH>N has the same meaning as <REFRESH>Y and should be treated identically.
- ◆ If a client embeds transaction requests in a <REFRESH> or <TOKENONLY> sync request, the server should respond in such a way that the <REFRESH> data or returned <TOKEN> reflects a specific state, after the transactions have processed. Since servers are not required to reduce the data about any particular object to a single addition response, embedded transactions may be processed before or after the <REFRESH> data is retrieved. As with all synchronization responses, the returned <TOKEN> must reflect the actions of all embedded transactions.
- ◆ <REJECTIFMISSING>Y is illegal unless accompanied by <TOKEN> or <TOKEN2> (depending on the message set version). If received in the same wrapper as <TOKENONLY> or <REFRESH>, the server should fail that synchronization request (as described in section 6.6.1). In V2 message sets and <PRESDLVMSGSRV1>, <SYNCERROR>6500 is appropriate for this case.

6.6.1 Synchronization Errors

When a client sends an unrecognized or “bad” token, the server response should be one of the following. (Note that <LOSTSYNC> is an optional element):

- ◆ Return <TOKEN>-1, <LOSTSYNC>N, with no history
- ◆ Return <TOKEN>X (the current token), <LOSTSYNC>N, with no history
- ◆ Return <TOKEN>X (the current token), <LOSTSYNC>N, with full history (i.e. treat as if it were a <TOKEN>0)

If the synchronization request included a bad account number or BANKID, or signon failed, or an account was closed, etc. the response should include <TOKEN>-1, optionally <LOSTSYNC>N, and no history.

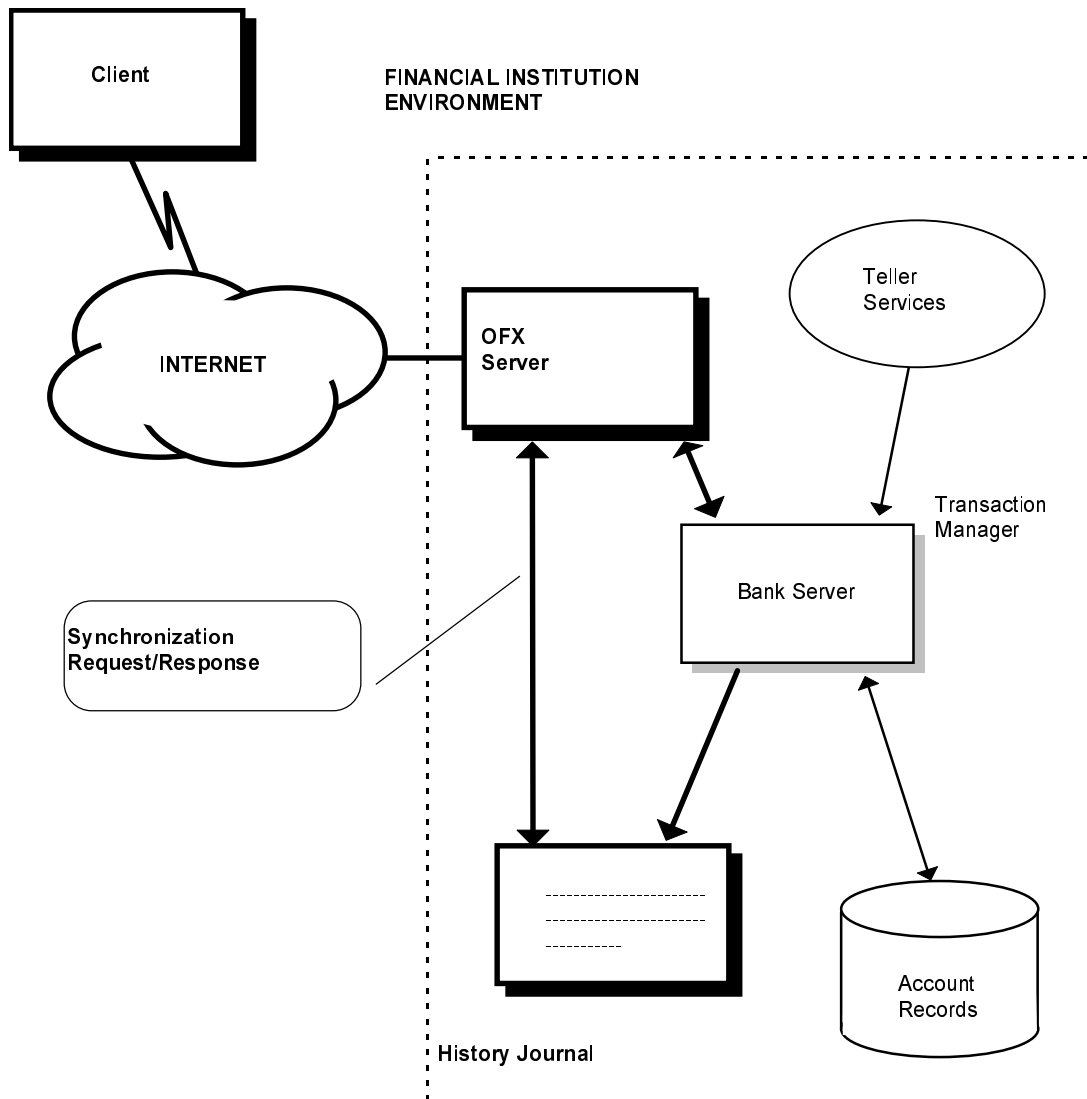
In all cases, the response should include an appropriate <SYNCERROR> if available within the context. That is, the optional <SYNCERROR> element should be used in all error cases that occur in the version 2 or Bill Presentment message sets.

6.7 Typical Server Architecture for Synchronization

This section describes how an FI can approach supporting synchronization based on the assumption that modifications to an existing financial server will be kept to a minimum.

The simplest approach is to create a history database separate from the existing server. This history could consist of the actual OFX transaction responses (<xxxTRNRS> aggregates) that are available to a synchronization request, or simply the information required to re-create the responses upon request from the client. The history database could index records by token, response type, and any other identifying information for that type, such as account number. Clearly, this database must include all <TRNUID>s for all transactions it contains. OFX recommends that <TRNUID>s be stored for as long as possible so that they may be used to detect duplicate client requests even after the original requests have been purged from the synch database.

The diagram below shows a high-level model of the OFX architecture for a financial institution. Notice that the diagram shows the presence of a history journal.



The server adds responses to the history journal for any action that takes place on the existing server. This is true whether the OFX requests initiate the action or, in the case of recurring payments, it happens automatically on the server. Once added to the history journal, the server can forget them.

The areas of the OFX server that process synchronization requests need only search this history database for matching responses that are more recent than the incoming token.

For a refresh request, an OFX server would access the actual bank server to obtain the current state rather than recent history.

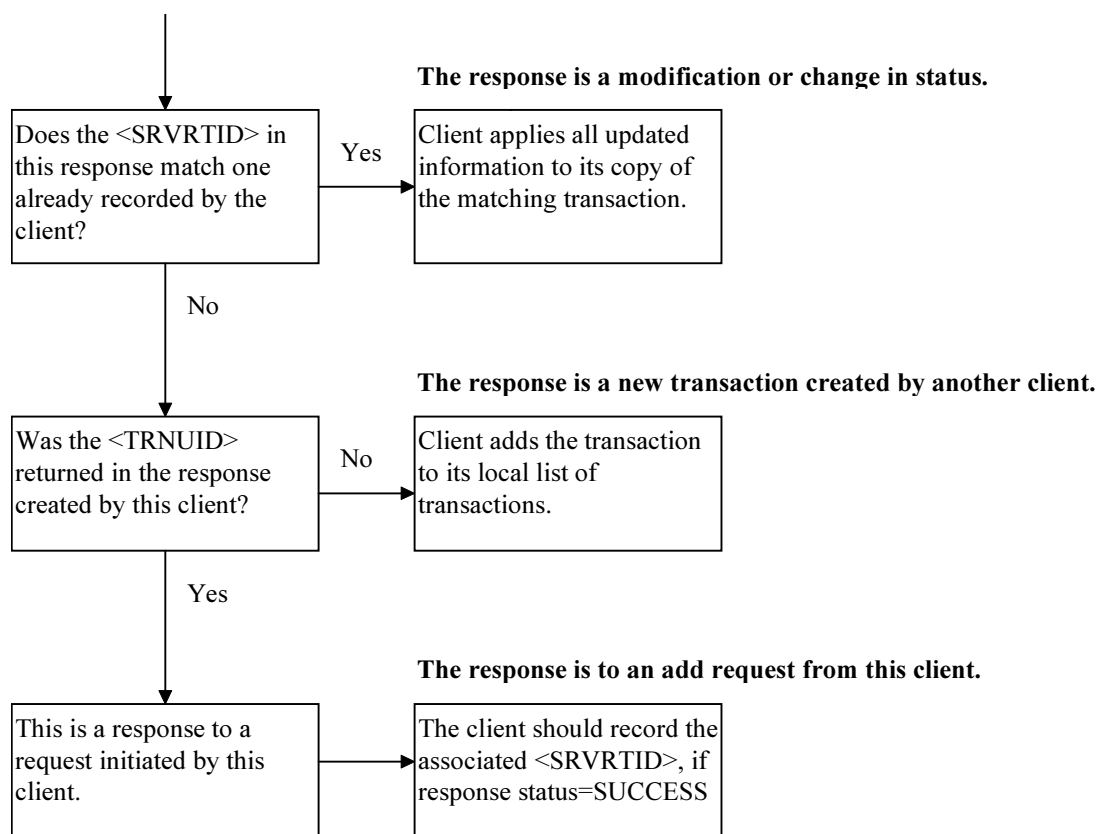
Periodically the bank server would purge the history server of older entries.

Only requests that are subject to synchronization need to have entries in the history database. Statement downloads do not involve synchronization; therefore, the FI server should not add these responses to the history database. Since statement downloads are usually the largest in space and the most frequent, eliminating these saves much of the space a response history might otherwise require.

More sophisticated implementations can save even more space. The history database could save responses in a coded binary form that is more compact than the full OFX response format. Some FIs might have much or all of the necessary data already in their servers; consequently, they would not require new data. An FI could regenerate synchronization responses rather than recall them from a database.

6.8 Typical Client Processing of Synchronization Results

The diagram below shows a general flowchart of what an OFX client would do with the results of a synchronization request. Most requests and responses subject to data synchronization contain both <TRNUID> and <SRVRTID>.



6.9 Simultaneous Connections

It is increasingly common for a server to get simultaneous or overlapping requests from the same user from two different front ends. OFX requires a server to process each set of requests sent in a file as an atomic action. Servers can deal with the problems that arise with simultaneous use in two ways:

- ◆ Allow simultaneous connections, ensure each is processed atomically, and use the data synchronization mechanism to bring the two clients up to date. This is the preferred method.
- ◆ Lock out all but one user at a time, returning the error code 15501 for multiple users.

6.10 Synchronization Alternatives

Although it is **RECOMMENDED** that OFX servers implement full synchronization as described in this chapter, an alternate approach, “lite synchronization,” could be easier for some servers to support. This approach focuses only on error recovery and does not provide any support for multiple clients, multiple data files, or use of backup files. The approach is to preserve the message sets while simplifying the implementation.

In addition, some clients might prefer to use file-based error recovery with all servers, even if the client and some servers support full synchronization. This section first describes file-based error recovery and lite synchronization, and then explains the rules that clients and servers use to decide how to communicate.

Lite synchronizing servers may support both file-based error recovery *and* <REFRESH>Y. This type of server is called a Refresh-capable Lite Synchronizing Server.

For information on how these types of synchronization are profiled, see section [7.2.1](#).

6.10.1 File-Based Error Recovery

Because only full synchronization supports error recovery, an alternative is needed for lite synchronization. Servers using lite synchronization keep a copy of the entire response file they last sent. This is the basis for what is often called “file-based error recovery.” Clients requesting that servers prepare for error recovery generate a globally unique ID for each file they send. Two OFX headers are associated with error recovery:

- ◆ OLDFILEUID—UID of the last request and response that was successfully received and processed by the client
- ◆ NEWFILEUID—UID of the current file

The format of these is the same as used with <TRNUID> as documented in section [2.4.6](#).

Servers use the following rules:

- ◆ If NEWFILEUID is set to NONE, the client is not requesting file-based error recovery for this session. The server does not need to save the response file. If NEWFILEUID is set to NONE and OLDFILEUID matches a previous request file (see below), the client may be ending use of file-based error recovery.
- ◆ If NEWFILEUID matches a previous request file, the client is requesting error recovery. The server should send the matching saved response file.

Note: If NEWFILEUID matches a previous request file then the request file identified by the NEWFILEUID must contain exactly the same set of transactions as the previous request file. Servers can reject the file if it contains new or modified transactions. In particular, clients should disallow new <PINCHRQ> transactions during error recovery. For more information about <PINCHRQ> and synchronization, see section [2.5.2](#).

- ◆ If NEWFILEUID is not set to NONE and does not match a previous request file, the client is preparing for error recovery. The server should save the response file in case the data does not reach the client.
- ◆ If OLDFILEUID is set to NONE, the server may ignore the presence of this header. The server should not search for a response file to delete. Clients should initiate file-based error recovery by sending OLDFILEUID set to NONE and NEWFILEUID set to a unique value.
- ◆ If OLDFILEUID matches a file saved on the server, then OLDFILEUID is a file that the client has successfully processed and the server can delete it.
- ◆ If OLDFILEUID is not set to NONE and does not match a previous request file, the server should ignore the presence of this header. Either the server has purged the associated request file without explicit request from the client or the client is requesting error recovery with identical headers to the initial request attempt (NEWFILEUID should match a previous request file in this case).

Note: While it may indicate a client error for OLDFILEUID and NEWFILEUID to hold identical values other than NONE, the server should ignore this OLDFILEUID header. Earlier rules in this list detail how the server should handle the request file (based solely upon the NEWFILEUID value).

A server should not save more than one file per client data file thread (history of FILEUID values). Servers should purge response files in response to an explicit client request (reference in the OLDFILEUID header) or after some long period (at least 2 months). Clients must not abuse this storage requirement by (for example) setting OLDFILEUID to the header used three request files previously. The server should preserve response files on a per-thread basis. This approach would support multiple clients or data files per user. But, the server has the option to ignore these needs and purge response files as soon as another valid request arrives for the same <USERID>. In either case, if an error recovery attempt comes after the corresponding error recovery file is purged, the server will not recognize the request as an attempt at error recovery. The server would simply process it as a new request. In this case, the server should recognize duplicate transaction UIDs for client-initiated work, such as payments, and then reject them individually. Server-generated responses would be lost to the client.

A server should not save a response file when it is useless to do so. Specifically, the server should not save a response file when the request fails parsing or when the request was rejected due to a <SONRQ> problem (e.g. invalid <USERID>).

If all accounts are shared between two (or more) users (for example, husband and wife have separate online access to the same list of joint accounts and none others), some identifiers may differ and should be maintained separately by the client. Thus, clients should initiate error recovery and maintain / generate xxxFILEUID values on a per-user basis.

6.10.1.1 File-Based Error Recovery and Authentication

There are two aspects of error recovery authentication which must be considered, request validation and password validation.

6.10.1.1.1 Request Validation

When error recovery is being attempted the server should first perform signon authentication on the request file. Once this is done, it should validate that the rest of the transactions in the request file received match those of the request file that was archived for the corresponding response file which was also archived. Recommended matching is defined at two levels:

- ◆ Minimal—Verify that the transactions correspond to the archived file
- ◆ Recommended—Verify the current request and archived request files exactly match. It is recommended that checksums for all characters after the </SONRQ> be used to verify an exact match. (The signon request itself may change between attempts.)

6.10.1.1.2 Password Validation

In all cases, the server must not store response files for the purposes of file-based error recovery when the <SONRQ> has failed. A saved response file matching the OLDFILEUID header (if any) must not be deleted when this occurs. In error recovery situations, the possibility exists that the user will have entered the correct password when a request was originally sent, but will mistype the password when prompted for it again during the recovery attempt. The server should respond as it would whenever sign on fails: It should return 15500 errors in all transaction response aggregates. The server should return synchronization wrappers with <TOKEN>-1 and any embedded transaction response aggregates with the same 15500 error. (The response file should contain no <xxxRS> aggregates apart from the <SONRS>.) This particular situation (sign on failure during an error recovery attempt) merits careful attention to the rules described in the previous paragraph.

6.10.2 Lite Synchronization

Lite synchronization requires servers to accept all synchronization messages, but does not require them to keep any history or tokens. Responses need to be sent only once and then the server can forget them. Responses to client requests, whether or not they are made inside a synchronization request, are processed normally. Responses that represent server-initiated work, such as payment responses that arise from recurring payments, are sent only in response to synchronization requests. A server does not have to hold responses in case a second client makes a synchronization request.

Basic lite synchronization servers do not support <REFRESH>Y. These servers may implement <TOKEN>0 responses as a pseudo-refresh (as described in section [6.10.2.1](#)). Refresh-capable lite synchronizing servers, however, do support <REFRESH>Y. That, in fact, is the only difference in function between a Basic Lite Synch server and a Refresh-capable Lite Synch Server. The purpose of the distinction is to allow a server to provide refresh capability without the burden of supporting full synchronization.

For a server accustomed to sending unsolicited responses, lite synchronization should closely match the current implementation of file-based error recovery. The only difference is that a server should hold the unsolicited responses until the client makes the first appropriate synchronization request; rather than automatically adding them to any response file. Once added, the server can mark them as delivered, relying on error recovery to ensure actual delivery.

Note: OFX requires a server to authenticate a client in Error Recovery.

6.10.2.1 Lite Synchronization and <REFRESH>

Basic lite synchronization servers do not support <REFRESH>Y. These servers may implement <TOKEN>0 responses as a pseudo-refresh. If a server does not support <REFRESH>Y requests, they may still choose to respond to a <TOKEN>0 request as if <REFRESH>Y were requested. In

this case, the response should be returned with a <TOKEN>1. This token never again increments and would be handled as described in section [6.10.2](#) (returning only unsolicited responses). This has the advantage of allowing all unsolicited responses to be discarded immediately after they have been included in a <xxxSYNCRS> (with <TOKEN>1) response.

Refresh-capable Lite Synchronization servers may support both file-based error recovery and <REFRESH>Y. OFX 1.0.2, 1.5.1, and 1.6 support Refresh-capable Lite Synchronization. (Existing clients may ignore this new feature.)

For more information on profiling synchronization support, see section [7.2.1](#).

6.10.3 Relating Synchronization and Error Recovery

Client and server developers should first decide whether or not they will support full synchronization. If they can, then they can support file-based error recovery as well, or they can rely on synchronization to perform error recovery. If they adopt only lite synchronization, OFX requires file-based error recovery. A server describes each of these choices in its server profile records. The following combinations are valid:

- ◆ Full synchronization with file-based error recovery
- ◆ Full synchronization without separate file-based error recovery
- ◆ Lite synchronization with file-based error recovery (with or without <REFRESH>Y support)

Clients request file-based error recovery by including the old and new session UIDs in the header. If these are absent, servers need not save the response file for error recovery. Clients request synchronization by using those synchronization requests defined throughout this specification.

6.11 Examples

Here is an example of full synchronization using bill payment as the service. OFX Payments provides two different synchronization requests and responses, each with their own token; one for payment requests and one for repeating payment model requests. Note that these simplified examples do not include the outer <OFX> layer, <SONRQ>, and so forth.

Client A requests synchronization:

```
<PMTSYNCRQ>
  <TOKEN>123
  <REJECTIFMISSING>N
  <BANKACCTFROM>
    <BANKID>121000248
    <ACCTID>123456789
    <ACCTTYPE>CHECKING
  </BANKACCTFROM>
</PMTSYNCRQ>
```

The server sends in response:

```
<PMTSYNCRS>
  <TOKEN>125
  <LOSTSYNC>N
  <BANKACCTFROM>
    <BANKID>121000248
    <ACCTID>123456789
    <ACCTTYPE>CHECKING
  </BANKACCTFROM>
  <PMTTRNRS>
    <TRUID>123
    <STATUS>
      ... status details
    </STATUS>
    <PMTRS>
      ... details on a payment response
    </PMTRS>
  </PMTTRNRS>
  <PMTTRNRS>
    <TRUID>546
    <STATUS>
      ... status details
    </STATUS>
```

```

    <PMTRS>
      ... details on another payment response
    </PMTRS>
  </PMTTRNRS>
</PMTSYNCRS>

```

Client A was missing two payment responses, which the server provides. At this point, client A is synchronized with the server. Client A now makes a new payment request, and includes a synchronization update as part of the request. This update avoids having to re-synchronize the expected response at a later time.

```

<PMTSYNCRQ>
  <TOKEN>125
  <REJECTIFMISSING>N
  <BANKACCTFROM>
    <BANKID>121000248
    <ACCTID>123456789
    <ACCTTYPE>CHECKING
  </BANKACCTFROM>
  <PMTTRNRQ>
    <TRUID>12345
    <PMTRQ>
      ... details of a new payment request
    </PMTRQ>
  </PMTTRNRQ>
</PMTSYNCRQ>

```

The response to this new request:

```

<PMTSYNCRS>
  <TOKEN>126
  <LOSTSYNC>N
  <BANKACCTFROM>
    <BANKID>121000248
    <ACCTID>123456789
    <ACCTTYPE>CHECKING
  </BANKACCTFROM>
  <PMTTRNRS>
    ... details on a payment response to the new request
  </PMTTRNRS>
</PMTSYNCRS>

```

The client now knows that the server has processed the payment request it just made, and that nothing else has happened on the server since it last synchronized with the server.

Assume client B was synchronized with respect to payments for this account up through token 125. If it called in now and synchronized—with or without making additional requests—it would pick up the payment response associated with token 126. It records the same information that was in client A, which would give both clients a complete picture of payment status.

CHAPTER 7 FI PROFILE

7.1 Overview

OFX clients use the profile to learn the capabilities of an OFX server. This information includes general properties such as account types supported, user password requirements, specific messages supported, and how the client should batch requests and where to send the requests. A client obtains a portion of the profile when a user first selects an FI. The client obtains the remaining information prior to sending any actual requests to that FI. The server uses a time stamp to indicate whether the server has updated the profile, and the client checks periodically to see if it should obtain a new profile.

In more detail, a profile response contains the following sections, which a client can request independently:

- ◆ Message Sets – list of services and any general attributes of those services. Message sets are collections of messages that are related functionally. They are generally subsets of what users see as a service.
- ◆ Signon realms – FIs can require different signons (user ID and/or password) for different message sets. Because there can only be one signon per <OFX> block, a client needs to know which signon the server requires and then provide the right signon for the right batch of messages.

The profile message is itself a message set. In files, OFX uses the <PROFMSGSETV1> and <PROFMSGSETV2> aggregates to identify this profile message set.

The following sections describe the general use of profile information.

7.1.1 Message Sets

A message set may be thought of as representing an available financial service. A message set itself is a collection of related messages. For example, [Chapter 11, "Banking,"](#) defines several message sets: statement download, credit card statement download, intrabank transfers, and so forth. A server may route all of the messages in a message set to a single URL and merge their versions together.

Clients and servers generally use message sets as the granularity to decide what functionality they will support. A “banking” server can choose to support the statement download and intrabank transfer message sets, but not the wire transfer message set. Attributes are available in many cases to further define how OFX supports a message set.

The profile applies only to the requests a client might expect the server to honor. That is, clients should not send requests to servers unless support is indicated. However, the server may send

unsupported responses in a sync response as information is entered out of band. A client is required to at least parse such a file.

Clients should assume the burden of checking the profile and not sending a transaction which the server does not support. If the client goes ahead and sends such a transaction, the server must ignore unrecognized and unsupported elements and aggregates within a request. Assuming no other problems occur in processing that request, servers may return warning code 2028 (Request element unknown). In either case (an unrecognized transaction or unknown elements or aggregates within a request), the response file should not contain the unrecognized tags or the contents of unrecognized aggregates.

Each portion of the OFX specification that defines messages also defines the message set to which those messages belong. This includes what additional attributes are available for those messages and whether OFX requires the message set or it is optional.

7.1.2 Version Control

Message sets are the basis of version control. Over time there will be new versions of the message sets, and at any given time servers will likely want to support more than one version of a message set. Clients should also be capable of supporting as many versions as possible. Through the profile, clients discover which versions are supported for each message set. Clients and servers exchange messages at the highest common level for each message set.

The DTD used to parse the OFX document determines which message sets are supported. There are currently three supported DTDs. OFX 1.5.1 and OFX 1.6 support all currently defined message sets, and OFX 1.0.2 supports only version 1 message sets (except Bill Presentment). For more information about DTD versions, see section [1.4](#) and [2.2.3](#).

There are two versions of the profile message set. Version 2 of the profile message set allows additional options specific to the corresponding request and response wrappers. The <MSGSETCORE> aggregate may include a longer <URL2> element and the <COUNTRY> element when used in version 2 of the profile message set.

If banking version 1 is at one URL (A) and billpay version 1 is at another URL (B), both may need version 1 of signon to be used. In that case, <MSGSETCORE> inside <BANKMSGSETV1> would refer to <URL>A and <MSGSETCORE> inside <BILLPAYMSGSETV1> would refer to <URL>B, but <MSGSETCORE> inside <SIGNONMSGSETV1> may refer to either URL or to some other. As mentioned in Section [2.5.4](#), the <URL> included in <SIGNONMSGSETV1> does not restrict where the <SIGNONMSGSRQV1> wrapper may be sent.

7.1.3 Batching and Routing

To allow FIs to set up different servers for different message sets, different versions, or to directly route some messages to third party processors, message sets define the URL to which a server sends messages in that message set. Each version of a message set can have a different URL. In the common case where many or all message sets are sent to a single URL, clients will consolidate messages across compatible message sets. Clients may consolidate when all of the following are true:

- ◆ Message sets have the same URL;
- ◆ Message sets have a common security level; and
- ◆ Message sets have the same signon realm.

Note: Signon messages can be sent with all other message sets even if the <SIGNONMSGSET> contains incompatible settings for the URL, security level, or signon realm. The message set information for signon messages is used only if the signon message is sent by itself. Otherwise, the settings are inherited from the accompanying service message set.

The same message set may be supported by multiple servers. In this case, each server that supports a particular message set must have a unique URL.

7.1.4 Client Signon for Profile Requests

Clients must include a signon request <SONRQ> with every message, including profile requests. The first time that a client requests the FI profile, the signon request will be present, but the user ID and password will not be valid and will be ignored by the server.

Note: Since elements cannot be set to a blank value, <USERID> and / or <USERPASS> may be set to lower case “anonymous” followed by 23 zeroes.

Once the user has enrolled and received his or her user ID and password, the client must request the profile again, even if the profile is not yet out-of-date. Once it has received a successful <PROFRS> (with or without a profile download) while signed on as the user, the client must not log in anonymously when sending any later <PROFRQ> to this server.

At this point, the server can respond with a profile response that indicates that the profile is up-to-date or return a new FI profile in response. If the FI wants to return a customer-specific profile, the FI must use the second approach. Servers must handle <PROFRQ> without an error whether or not a request arrives with an anonymous <SONRQ>.

Note: OFX 1.0.2 clients and servers may violate these restrictions, which were added in later versions. Clients interacting with 1.0.2 servers should gracefully handle <PROFRS> errors in their first per-user attempt, reverting to anonymous requests for subsequent requests (until the next response with <STATUS><CODE>0, when they should once again make a per-user attempt to retrieve the profile). Servers interacting

with 1.0.2 clients should not require support for customer-specific profiles. Servers correcting problems with per-user <PROFRQ> requests (which previously caused error responses) must update the FI Profile to tell compliant clients to retry.

For more information about signon requests, refer to section [2.5](#).

7.1.5 Profile Request <PROFRQ>

A profile request indicates which profile components a client desires. It also indicates what the client's routing capability is. Profiles returned by the FI must be compatible with the requested routing style, or the server returns an error.

Profile requests are not subject to synchronization.

Profile requests must appear within a <PROFTRNRQ> transaction wrapper.

Tag	Description
<PROFRQ>	Profile-request aggregate
<CLIENTROUTING>	Identifies client routing capabilities, see table below
<DTPROFUP>	Date and time client last received a profile update, <i>datetime</i>
</PROFRQ>	

Tag	Description
NONE	Client cannot perform any routing. All URLs must be the same. All message sets share a single signon realm.
SERVICE	Client can perform limited routing. See details below.
MSGSET	Client can route at the message-set level. Each message set can have a different URL and / or signon realm.

The SERVICE option supports clients that can route bill payment messages to a separate URL from the rest of the messages. Because the exact mapping of message sets to the general concept of bill payment can vary by client and by locale, this specification does not provide precise rules for the SERVICE option. Each client will define its requirements.

7.2 Profile Response <PROFRS>

To determine whether the client has the latest version of the FI profile, the server checks the date and time passed by the client in <DTPROFUP>.

If the client has the latest version of the FI's profile, the server returns status code 1 in the <STATUS> aggregate of the profile-transaction aggregate <PROFTRNRS>. The server does not return a profile-response aggregate <PROFRS>.

Note: Not sending a response aggregate in this case is an exception to rules outlined in sections [2.4.6](#) and [3.1.5](#).

If the client does not have the latest version of the FI profile, the server responds with the profile-response aggregate <PROFRS> in the profile-transaction aggregate <PROFTRNRS>. The response includes message set descriptions, signon information, and general contact information.

Tag	Version	Description
<PROFRS>		Profile-response aggregate
<MSGSETLIST>		Beginning list of message set information
<xxxMSGSET>		One or more message set aggregates
</xxxMSGSET>		
</MSGSETLIST>		
<SIGNONINFOLIST>		Beginning of signon information
<SIGNONINFO>		Zero or more signon information aggregates.
		Though the DTD allows an empty <SIGNONINFOLIST>, servers should profile at list one signon realm (include a minimum of one <SIGNONINFO> aggregate in the <PROFRS> response).
</SIGNONINFO>		
</SIGNONINFOLIST>		
<DTPROFUP>		Time this was updated on server, <i>datetime</i>
<FINAME>		Name of institution, A-32
<ADDR1>		FI address, line 1, A-32
<ADDR2>		FI address, line 2, A-32
<ADDR3>		FI address, line 3. Use of <ADDR3> requires the presence of <ADDR2>, A-32
<CITY>		FI address city, A-32
<STATE>		FI address state, A-5

Tag	Version	Description
<POSTALCODE>		FI address postal code, A-11
<COUNTRY>		FI address country; 3-letter country code from ISO / DIS-3166, A-3
<CSPHONE>		Customer service telephone number, A-32
<TSPHONE>		Technical support telephone number, A-32
<FAXPHONE>		Fax number, A-32
<URL>	V1 only	URL for general information about FI (not for sending data), <i>URL</i>
<URL2>	V2 only	URL for general information about FI (not for sending data), <i>URL2</i>
<URLGETREDIRECT>	V2 only	Whether or not the returned URL is a redirection URL. <i>Boolean</i>
<EMAIL>		E-mail address for FI, A-80
</PROFRS>		

7.2.1 Message Set

An aggregate describes each message set supported by an FI. Message sets in turn contain an aggregate for each version of the message set that is supported. For a message set named *xxx*, the convention is to name the outer aggregate **<xxxMSGSET>** and the tag for each version **<xxxMSGSETVn>**. The reason for message set-specific aggregates is that the set of attributes depends on the message set. These can change from version to version, so there are version-specific aggregates as well.

The general form of the response is:

Tag	Description
<xxxMSGSET>	Service aggregate
<xxxMSGSETVn>	Version-of-message-set aggregate, <xxxMSGSETV1> is required. <xxxMSGSETV2> may appear zero or more times. As mentioned in Sections 14.7.2 and 14.7.3 , <PRESDIRMSGSETV1> and <PRESDLVMSGSETV1> may appear one or more times.
</xxxMSGSETVn>	
</xxxMSGSET>	

The **<xxxMSGSETVn>** aggregate has the following form:

Tag	Description
<xxxMSGSETVn>	Message-set-version aggregate
<MSGSETCORE>	Common message set information aggregate.
</MSGSETCORE>	
Message-set specific	Zero or more attributes specific to this version of this message set, as defined by each message set
</xxxMSGSETVn>	

The common message set information **<MSGSETCORE>** is as follows:

Tag	Version	Description
<MSGSETCORE>		Common-message-set-information aggregate
<VER>		Version number of the message set, (for example, <VER>1 for version 1 of the message set), <i>N-5</i> Because this information is already provided by the surrounding <xxxMSGSETVn> wrapper, <VER> should be ignored by OFX clients. Nonetheless, servers should use one of the supported values (<VER>1 or <VER>2) consistent with that wrapper.
<URL>	V1 only	URL where messages in this set are to be sent, <i>URL</i>
<URL2>	V2 only	URL where messages in this set are to be sent, <i>URL2</i>
<OFXSEC>		Security level required for this message set; see Chapter 4, "OFX Security." <i>NONE</i> or <i>TYPE 1</i> .
<TRANSPSEC>		Y if transport-level security must be used, N if not used; see Chapter 4, "OFX Security." <i>Boolean</i>
<SIGNONREALM>		Signon realm to use with this message set, <i>A-32</i>
<LANGUAGE>		1 or more. Language supported, <i>language</i> . If more than one language is supported, multiple <LANGUAGE> elements can be sent.

Tag	Version	Description																						
<COUNTRY>	V2 only	0 or more. Prefix of country-specific elements supported and field interpretations: 3-letter country code from ISO / DIS-3166. If multiple <COUNTRY> elements are present, then multiple countries are supported. If this field is missing, then it is assumed that only the USA version of OFX is supported. For example, if the values “FRA” and “ITA” are present, then the server supports the French and Italian “versions” of OFX. If a country is supported for any of the main message sets, then the server should accept a signon request for that country. Otherwise, the server should reject such a signon request. The following countries are currently supported in OFX: <table><tr><td><u>COUNTRY</u></td><td><u>Country Name</u></td></tr><tr><td>BEL</td><td>Belgium</td></tr><tr><td>CAN</td><td>Canada</td></tr><tr><td>CHE</td><td>Switzerland</td></tr><tr><td>DEU</td><td>Germany</td></tr><tr><td>ESP</td><td>Spain</td></tr><tr><td>FRA</td><td>France</td></tr><tr><td>GBR</td><td>Great Britain</td></tr><tr><td>ITA</td><td>Italy</td></tr><tr><td>NLD</td><td>Netherlands</td></tr><tr><td>USA</td><td>United States of America</td></tr></table>	<u>COUNTRY</u>	<u>Country Name</u>	BEL	Belgium	CAN	Canada	CHE	Switzerland	DEU	Germany	ESP	Spain	FRA	France	GBR	Great Britain	ITA	Italy	NLD	Netherlands	USA	United States of America
<u>COUNTRY</u>	<u>Country Name</u>																							
BEL	Belgium																							
CAN	Canada																							
CHE	Switzerland																							
DEU	Germany																							
ESP	Spain																							
FRA	France																							
GBR	Great Britain																							
ITA	Italy																							
NLD	Netherlands																							
USA	United States of America																							
<SYNCMODE>		FULL for full synchronization capability LITE for lite synchronization capability See Chapter 6, "Data Synchronization," for more information.																						
<REFRESHSUPT>	1.6 add	Y if server supports <REFRESH>Y within synchronizations. This option is irrelevant for full synchronization servers. Clients must ignore <REFRESHSUPT> (or its absence) if the profile also specifies <SYNCMODE>FULL. For lite synchronization, the default is N. Without <REFRESHSUPT>Y, lite synchronization servers are not required to support <REFRESH>Y requests, <i>Boolean</i>																						
<RESPFILEER>		Y if server supports file-based error recovery, <i>Boolean</i> See Chapter 6, "Data Synchronization," for more information.																						
<SPNAME>		Service provider name, A-32 Some financial institutions out-source their OFX servers to a service provider. In such cases, the SPNAME element should be included in the MSGSETCORE.																						
</MSGSETCORE>																								

Note: For all message sets currently defined in OFX, <TRANSPSEC>Y must be specified.

Note: Within a <MSGSETCORE> aggregate, the <VER> element defines the version number of that message set. It does not refer to the version number of the OFX specification or the DTD files. For more information about message sets and version numbers, refer to section 2.4.5.

Note: Within a message set, there can be more than one <MSGSETCORE> aggregate with the same value for <VER>, or the same value for <URL>, but not the same value for both. The pair must be unique for each instance of <MSGSETCORE> within a message set. Multiple <MSGSETCORE>s with the same value for <VER> are used in instances such as signon or registration, which may have the same version sent to multiple URLs for different services.

7.2.2 Signon Realms

A signon realm identifies a set of messages that can be accessed using the same password. Realms are used to disassociate signons from specific services, allowing FIs to require different signons for different message sets. In practice, FIs will want to use the absolute minimum number of realms possible to reduce the user's workload.

Tag	Version	Description
<SIGNONINFO>		Signon-information aggregate
<SIGNONREALM>		Identifies this realm, A-32
<MIN>		Minimum number of password characters, N-2
<MAX>		Maximum number of password characters, N-2
<CHARTYPE>		Type of characters allowed in password:
	ALPHAONLY	Password may not contain numeric characters. The server would allow "abbc", but not "1223" or "a122".
	NUMERICONLY	Password may not contain alphabetic characters. The server would allow "1223", but not "abbc" or "a122".
	ALPHAORNUMERIC	Password may contain alphabetic or numeric characters (or both). The server would allow "abbc", "1223", or "a122".
	ALPHAANDNUMERIC	Password must contain both alphabetic and numeric characters. The server would allow "a122", but not "abbc" or "1223".

Tag	Version	Description							
<CASESEN>	V2 only	Y if password is case-sensitive, <i>Boolean</i>							
<SPECIAL>		Y if special characters are allowed over and above those characters allowed by <CHARTYPE> and <SPACES>, <i>Boolean</i>							
<SPACES>		Y if spaces are allowed over and above those characters allowed by <CHARTYPE> and <SPECIAL>, <i>Boolean</i>							
<PINCH>		Y if server supports <PINCHRQ> (PIN change requests), <i>Boolean</i>							
<CHGPINFIRST>		Y if server requires clients to execute <PINCHRQ> as part of first signon. Clients must ignore <CHGPINFIRST> if the profile also specifies <PINCH>N. <i>Boolean</i>							
<PWTYPE>		The type of password(s) supplied by the client in the signon request to authenticate the user (see section 2.5.1.1). The following password types are currently supported in OFX:							
</SIGNONINFO>		<table><thead><tr><th><u>PWTYPE</u></th><th><u>Description</u></th></tr></thead><tbody><tr><td>FIXED</td><td>The client supplies (in USERPASS) a normal password that remains fixed until explicitly changed by the user or the FI. This is the default password type. Clients should assume FIXED password type when <PWTYPE> is not present in <SIGNONINFO>.</td></tr><tr><td>ONETIME</td><td>In addition to USERPASS, the client supplies (in ONETIMEPASS) an additional password that was taken from a list of one-time passwords sent from the FI to the user.</td></tr><tr><td>HWTOKEN</td><td>In addition to USERPASS, the client supplies (in ONETIMEPASS) an additional password that was generated by a hardware token such as a crypto-calculator, typically on a time-varying basis.</td></tr></tbody></table>	<u>PWTYPE</u>	<u>Description</u>	FIXED	The client supplies (in USERPASS) a normal password that remains fixed until explicitly changed by the user or the FI. This is the default password type. Clients should assume FIXED password type when <PWTYPE> is not present in <SIGNONINFO>.	ONETIME	In addition to USERPASS, the client supplies (in ONETIMEPASS) an additional password that was taken from a list of one-time passwords sent from the FI to the user.	HWTOKEN
<u>PWTYPE</u>	<u>Description</u>								
FIXED	The client supplies (in USERPASS) a normal password that remains fixed until explicitly changed by the user or the FI. This is the default password type. Clients should assume FIXED password type when <PWTYPE> is not present in <SIGNONINFO>.								
ONETIME	In addition to USERPASS, the client supplies (in ONETIMEPASS) an additional password that was taken from a list of one-time passwords sent from the FI to the user.								
HWTOKEN	In addition to USERPASS, the client supplies (in ONETIMEPASS) an additional password that was generated by a hardware token such as a crypto-calculator, typically on a time-varying basis.								

7.2.3 Status Codes

Value	Meaning
0	Success (INFO)
1	Client is up-to-date (INFO)
2000	General error (ERROR)

7.3 Profile Message Set Profile Information

The profile message set functions the same way as all other message sets; therefore, it contains a profile description for that message set. Because <PROFMSGSET> is always part of a message set response, it is described here. Servers must include the <PROFMSGSET> as part of the profile response <MSGSETLIST>. There are no attributes, but the aggregate must be present to indicate support for the message set.

<i>Tag</i>	<i>Description</i>
<PROFMSGSET>	Message-set-profile-information aggregate
<PROFMSGSETV1>	Opening tag for V1 of the message set profile information
<MSGSETCORE>	Common message set information
</MSGSETCORE>	
</PROFMSGSETV1>	
<PROFMSGSETV2>	Opening tag for V2 of the message set profile information
<MSGSETCORE>	Common message set information
</MSGSETCORE>	
</PROFMSGSETV2>	
</PROFMSGSET>	

CHAPTER 8 ACTIVATION & ACCOUNT INFORMATION

8.1 Overview

The Signup message set defines three messages to help users get setup with their FI:

- ◆ Enrollment – informs FI that a user wants to use OFX and requests that a password be returned
- ◆ Accounts – asks the FI to return a list of accounts and the services supported for each account
- ◆ Activation – allows a client to tell the FI which services a user wants on each account

There is also a message to request name and address changes.

Clients use the account information request on a regular basis to look for changes in a user's account information. A time stamp is part of the request so that a server has to report only new changes. Account activation requests are subject to data synchronization, and will allow multiple clients to learn how the other clients have been enabled.

In OFX request files, the <SIGNUPMSGSRQV1> and <SIGNUPMSGSRQV2> aggregates identify the Signup messages.

8.1.1 Sign-Up Message Set V2 and Proxy On-Behalf-Of Actions

In OFX 1.6, the Sign-Up Message Set V2 can act on behalf of a user other than the one specified in <SONRQ> by specifying an optional <USERID> in <SIGNUPMSGSRQV2> before the <xxxTRNRQ> and / or <xxxSYNCRQ>. It is the server's responsibility to ensure that the user specified in <SONRQ> has the authority to act on behalf of the user specified in <SIGNUPMSGSRQV2>.

Note: The server must indicate support for <USERID> by including <CANSUPPORTUSERID>Y in the <SIGNUPMSGSETV2> message. See section [8.8](#)

If a <USERID> is specified in <SIGNUPMSGSRQV2>, then **all** transactions within that message set wrapper are to be performed on behalf of the user identified by that <USERID>. Therefore, if a single OFX block is to contain transactions on behalf of several users, each user will require its own <SIGNUPMSGSRQV2> (for example, multiple user activations as mentioned in section [8.2.1](#)). The response message set wrapper <SIGNUPMSGSRSV2> echoes the <USERID> if it is supplied by the corresponding request message set wrapper.

8.1.1.1 Sign-Up Message Set Wrapper Request

Tag	Version	Description
<SIGNUPMSGSRQV2>	1.6 add	Sign-Up Message Set Wrapper Request
<USERID>		On-Behalf-Of User, A-32
<xxxxyyRQ>		Zero or more transaction and / or synchronization requests
</xxxxyyRQ>		
</SIGNUPMSGSRQV2>		

8.1.1.2 Sign-Up Message Set Wrapper Response

Tag	Version	Description
<SIGNUPMSGSRSV2>	1.6 add	Sign-Up Message Set Wrapper Response
<USERID>		On-Behalf-Of User, A-32
<xxxxyyRS>		Zero or more transaction and / or synchronization responses
</xxxxyyRS>		
</SIGNUPMSGSRSV2>		

8.2 Approaches to User Sign-Up with OFX

The message sets in this chapter are designed to allow both FIs and clients to support a variety of sign-up procedures. There are four basic steps a user needs to go through to complete the sign-up:

- 1. Select the FI.** OFX does not define this step or provide message sets to support it. Client developers and FIs can let a user browse or search this information on a web site, or might define additional message sets to do this within the client. At the conclusion of this step, the client will have some minimal profile information about the FI, including the set of services supported and the URL to use for the next step.
- 2. Enrollment and password acquisition.** In this step, the user identifies and authenticates itself to the FI *without a password*. In return, the user obtains a password (possibly temporary) to use with OFX. FIs can perform this entire step over the telephone, through a combination of telephone requests and a mailed response, or at the FI web site. FIs can also use the OFX enrollment message to do this by means of the client. The response can contain a temporary password or users can wait for a mailed welcome letter containing the password.
- 3. Account Information.** In this step, the user obtains a list of accounts available for use with OFX, and which specific services are available for each account. Even if users have enrolled over the telephone, clients will still use this message set to help users properly set up the accounts within the client. Clients periodically check back with the FI for updates.

- 4. Service Activation.** The last step is to activate specific services on specific accounts. The activation messages support this step. Synchronization is applied to these messages to ensure that other clients are aware of activated services.

The combination of media-interface through which an FI accomplishes these steps can vary. FIs might wish to do steps two through four over the telephone. Clients will still use OFX messages in steps 3 and 4 to automatically set up the client based on the choices made by the user over the phone. Other FIs might wish to have the entire user experience occur within the client. Either way, the OFX sign-up messages support the process.

8.2.1 Multiple User Enrollment and Activation

Clients may enroll multiple users in a single <SIGNUPMSGSRQV2> (but not V1). This eliminates the need for a separate round-trip or message set wrapper for each user to be enrolled. In such a case, the client sends an <ENROLLRQ> for each user. Note that this cannot be used in conjunction with rapid signup (see section [8.2.2](#)), so no other transactions may be included with the series of <ENROLLRQ>s.

Clients that want to send multiple account activations for multiple users, can send multiple <SIGNUPMSGSRQV2>s in a single OFX block, with <USERID> specified in each message set wrapper.

In multiple enrollments and multiple activations, the server can process the individual requests in any order.

8.2.2 Rapid Signup

Clients may include multiple <ACCTRQ>s in a <SIGNUPMSGSRQV2> (but not V1) that contains an <ENROLLRQ>. This allows the client to combine steps two through four, as outlined above, in a single round trip. In such a case, the <ENROLLRQ> must be the first request in the message set wrapper, and the other transactions apply to that newly enrolled user. When rapid signup is used, the server must process the transactions in the same order as they appear in the message set wrapper. If the enrollment fails, the other transactions need not be processed; however, they must return a status code of 13500.

8.3 Users and Accounts

To support the widest possible set of FIs, OFX assumes that individual users and accounts are in a many-to-many relationship. Consider a household with three accounts:

- ◆ Checking 1 – held individually by one spouse
- ◆ Checking 2 – held jointly by both
- ◆ Checking 3 – held individually by the other spouse

Checking 2 should be available to either spouse, and the spouse holding Checking 1 should be able to see both Checking 1 and 2.

OFX expects FIs to give each user their own user ID and password. Each user will go through the enrollment step separately. A given account need only be activated once for a service; not once for each user. Clients will use the account information and activation messages to combine information about jointly held accounts.

If an FI prefers to have a single user ID and password per household or per master account, it will have to make this clear to users through the enrollment process. It is up to the FI to assign a single user ID and password that can access all three of the checking accounts described above.

8.4 Enrollment and Password Acquisition

The main purpose of the enrollment message is to communicate a user's intent to access the FI by way of OFX and to acquire a password for future use with OFX. Some FIs might return a user ID and an initial password in the enrollment response, while others will send them by way of regular mail.

Note: The client may not know the user ID and password when it sends the enrollment request, in such a case the <USERID> and /or <USERPASS> may be set to lower case "anonymous" followed by 23 zeroes.

Enrollment requests are not subject to synchronization. If the client does not receive a response, it will simply re-request the enrollment. If a user successfully enrolls from another client before the first client obtains a response, the server should respond to subsequent requests from the first client with status code:

13501 - user already enrolled.

8.4.1 User IDs

The OFX <SONRQ> requires a user ID to uniquely identify a user to an FI. The server must accept the user ID with or without punctuation.

Many FIs in the United States use social security numbers (SSNs) as the ID. Others create IDs that are unrelated to the users' SSNs. Some FIs have existing user IDs that they use for other online activities that they want to use for OFX as well. FIs might also create new IDs specifically for OFX. Finally, some FIs might assign IDs while others might allow users to create them. Because users do not usually know either their OFX sign-on user ID or their password at time of enrollment, the enrollment response is designed to return both. The enrollment request allows users to optionally provide a user ID, which an FI can interpret as their existing online ID or a suggestion for what their new user ID should be. Ideally, the enrollment process should explain ID syntax to users.

8.4.2 Enrollment Request <ENROLLRQ>

The enrollment request captures enough information to identify and authenticate a user as being legitimate and that it has a relationship with the FI.

FIs might require that an account number be entered as part of the identification process. However, this is discouraged since the account information request is designed to automatically obtain all account information, avoiding the effort and potential mistakes of a user-supplied account number.

It is **RECOMMENDED** that FIs provide detailed specifications for user IDs and passwords along with information about the services available when a user is choosing an FI.

The enrollment request must appear within an <ENROLLTRNRQ> transaction wrapper.

Tag	Version	Description
<ENROLLRQ>		Enrollment-request aggregate
<FIRSTNAME>		First name of user, A-32
<MIDDLENAME>		Middle name of user, A-32
<LASTNAME>		Last name of user, A-32
<ADDR1>		Address line 1, A-32
<ADDR2>		Address line 2, A-32
<ADDR3>		Address line 3. Use of <ADDR3> requires the presence of <ADDR2>, A-32
<CITY>		City, A-32
<STATE>		State or province, A-5
<POSTALCODE>		Postal code, A-11
<COUNTRY>		3-letter country code from ISO / DIS-3166, A-3
<DAYPHONE>		Daytime telephone number, A-32
<EVEPHONE>		Evening telephone number, A-32
<EMAIL>		Electronic e-mail address, A-80
<USERID>		Actual user ID if already known, or preferred user ID if user can choose, A-32
<TAXID>		ID used for tax purposes (such as SSN), may be same as user ID, A-32
<SECURITYNAME>		Mother's maiden name or equivalent, A-32
<DATEBIRTH>		Date of birth, <i>date</i>
<xxxACCTFROM>		An account description aggregate for an existing account at the FI, for identification purposes only. For example, <BANKACCTFROM> or <INVACCTFROM>.
</xxxACCTFROM>		
</ENROLLRQ>		

This enrollment request is intended for use only by individuals. Business enrollment will be defined in a later release.

8.4.3 Enrollment Response <ENROLLRS>

The main purpose of the enrollment response is to acknowledge the request. In those cases where FIs permit delivery of an ID and a temporary password, the response also provides for this. Otherwise the server will send the real response to the user by way of regular mail, electronic mail, or over the telephone. If enrollment is successful, but the server does not return the ID and password in the response, a server is REQUIRED to use status code 13000 and provide some information to the user by means of the <MESSAGE> element in the <STATUS> aggregate about what to expect next.

The enrollment response must appear within an <ENROLLTRNRS> transaction wrapper.

<i>Tag</i>	<i>Description</i>
<ENROLLRS>	Enrollment-response aggregate
<TEMPPASS>	Temporary password, A-32
<USERID>	User ID, A-32
<DTEXPIRE>	Time the temporary password expires (if <TEMPPASS> included), <i>datetime</i>
</ENROLLRS>	

8.4.4 Enrollment Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
13000	User ID & password will be sent out-of-band (INFO)
13500	Unable to enroll user (ERROR)
13501	User already enrolled (ERROR)
15508	Transaction not authorized (ERROR)

8.4.5 Examples

An enrollment request:

```
<ENROLLTRNRQ>
  <TRNUID>12345
  <ENROLLRQ>
    <FIRSTNAME>Joe
    <MIDDLENAME>Lee
    <LASTNAME>Smith
    <ADDR1>21 Main St.
    <CITY>Anytown
    <STATE>TX
    <POSTALCODE>87321
    <COUNTRY>USA
    <DAYPHONE>123-456-7890
    <EVEPHONE>987-654-3210
    <EMAIL>jsmith@isp.com
    <USERID>jls
    <TAXID>123-456-1234
    <SECURITYNAME>jbmam
    <DATEBIRTH>19530202
  </ENROLLRQ>
</ENROLLTRNRQ>
```

And the reply might be:

```
<ENROLLTRNRS>
  <TRNUID>12345
  <STATUS>
    <CODE>0
    <SEVERITY>INFO
  </STATUS>
  <ENROLLRS>
    <TEMPPASS>changeme
    <USERID>jls
    <DTEXPIRE>19970105
  </ENROLLRS>
</ENROLLTRNRS>
```

8.5 Account Information

Account information requests ask a server to identify and describe all of the accounts accessible by the signed-on user. The definition of *all* is up to the FI. At a minimum, it is **RECOMMENDED** that a server include information about all accounts that it can activate for one or more OFX services. To give the user a complete picture of his relationship with an FI, FIs can give information on other accounts, even if those accounts are available only for limited OFX services.

Some service providers do not have prior knowledge of user account information. The profile allows these servers to report this, and clients then know to ask users for account information rather than reading it from the server.

Clients can perform several tasks for users with this account information. First, the information helps a client set up a user for online services by giving it a precise list of its account information and available services for each. Clients can set up their own internal state as well as prepare service activation requests with no further typing by users. This can eliminate data entry mistakes in account numbers, routing transit numbers, and so forth.

Second, FIs can provide limited information on accounts that would not ordinarily be suitable to OFX services. For example, a balance-only statement download would be useful for certificates of deposits even though a customer or an FI might not want or allow CDs to be used for full statement download.

For each account, there is one <ACCTINFO> aggregate returned. The aggregate includes one service-specific account information aggregate for each service available to that account. That, in turn, provides the service-specific account identification. Common to each service-specific account information aggregate is the <SVCSTATUS> element, which indicates the status of this service on this account.

A server should return joint accounts (accounts for which more than one user ID can be used to access the account) for either user.

Requests and responses include a <DTACCTUP> element. Responses contain the last time a server updated the information. Clients *are* **REQUIRED** to send this in a subsequent request, and servers are **REQUIRED** to compare this to the current modification time and only send information if it is more recent. The server sends the entire account information response if the client's time is older; there is no attempt to incrementally update specific account information. <ACCTINFORS> should not be sent when the client is up-to-date.

Note: Not sending a response aggregate in the case of <ACCTINFORS> is an exception to the rules outlined in [2.4.6](#) and [3.1.5](#).

8.5.1 Request <ACCTINFORQ>

The <ACCTINFORQ> request must appear within an <ACCTINFOTRNRQ> transaction wrapper.

Tag	Version	Description
<ACCTINFORQ>	1.6 add	Account-information-request aggregate
<DTACCTUP>		Last <DTACCTUP> received in a response, <i>datetime</i>
<SVC2>		Zero or more. Services to be included in <ACCTINFORS>. The server has the option of ignoring this element and returning information about all services. BANKSVC = Banking service BPSVC = Payment service INVSVC = Investments PRESSVC = Bill Presentment service
</ACCTINFORQ>		

8.5.2 Response <ACCTINFORS>

The <ACCTINFORS> response must appear within an <ACCTINFOTRNR> transaction wrapper.

Tag	Description
<ACCTINFORS>	Account-information-response aggregate
<DTACCTUP>	Date and time of last update to this information on the server, <i>datetime</i>
<ACCTINFO>	Zero or more account information aggregates
	Left out of the response when nothing is found (in the specified <SVC2>, if any) for the current user.
	Note: When <DTACCTUP> indicates the client is up-to-date, server should not return surrounding <ACCTINFORS>.
</ACCTINFO>	
</ACCTINFORS>	End of account information response

8.5.3 Account Information Aggregate <ACCTINFO>

Tag	Version	Description
<ACCTINFO>		Account-information-record aggregate
<DESC>		Description of the account, A-80
<PHONE>		Telephone number for the account, A-32
<xxxACCTINFO>		Service-specific account information, defined in each service chapter. Some services may include additional elements. Refer to service chapters for details.
<xxxACCTFROM>		Service-specific account identification. For a given service xxx, there can be at most one <xxxACCTINFO> returned. For example, you cannot return two <BANKACCTINFO> aggregates.
</xxxACCTFROM>		
<SVCSTATUS>	V1 only	AVAIL = Available, but not yet requested PEND = Requested, but not yet available ACTIVE = In use
<SVCSTATUS2>	V2 only	AVAIL = Available, but not yet requested PEND = Requested, but not yet available ACTIVE = In use REJECTED = Rejected
<REASON>	V2 only	This element is only relevant if <SVCSTATUS2>REJECTED is specified, A-255
</xxxACCTINFO>		
</ACCTINFO>		

Note: A server uses the <DESC> field to convey the FI's preferred name for the account, such as "PowerChecking." It should not include the account number.

8.5.4 Status Codes

Code	Meaning
0	Success (INFO)
1	Client is up-to-date (INFO)
2000	General error (ERROR)

8.5.5 Examples

An account information request:

```
<ACCTINFOTRNRQ>
  <TRNUID>12345
  <ACCTINFOFORQ>
    <DTACCTUP>19960101</ACCTINFOFORQ>
</ACCTINFOTRNRQ>
```

And a response for a user with access to one account, supporting banking:

```
<ACCTINFOTRNRS>
  <TRNUID>12345
  <STATUS>
    <CODE>0
    <SEVERITY>INFO
  </STATUS>
  <ACCTINFORS>
    <DTACCTUP>19960102
    <ACCTINFO>
      <DESC>Power Checking
      <PHONE>8002223333
      <BANKACCTINFO>
        <BANKACCTFROM>
          <BANKID>1234567789
          <ACCTID>12345
          <ACCTTYPE>CHECKING
        </BANKACCTFROM>
        <SUPTXDL>Y
        <XFERSRC>Y
        <XFERDEST>Y
        <SVCSTATUS>ACTIVE
      </BANKACCTINFO>
    </ACCTINFO>
  </ACCTINFORS>
</ACCTINFOTRNRS>
```

8.6 Service Activation

Clients inform FIs that they wish to start, modify, or terminate a service for an account by sending service activation requests. These are subject to data synchronization, and servers should send responses to inform clients of any changes, even if the changes originated on the server.

Clients use these records during the initial user sign-up process. Once a client learns about the available accounts and services (by using the account information request above, or by having a user directly enter the required information), it sends a series of service ADD requests.

If a user changes any of the identifying information about an account, the client sends a service activation request containing both the old and the new account information. Servers should interpret this as a change in the account, not a request to transfer the service between two existing accounts, and all account-based information such as synchronization tokens should continue. If a user or FI is reporting that a service should be moved between two existing accounts, service must be terminated for the old account and started for the new account. The new account will have reset token histories, as with any new service.

Each service to be added, changed, or removed is contained in its own request because the same real-world account might require different <xxxACCTFROM> aggregates depending on the type of service.

8.6.1 Activation Request <ACCTRQ>

The <ACCTRQ> request must appear within an <ACCTTRNRQ> transaction wrapper.

Tag	Version	Description
<ACCTRQ> <i>Action identification. Specify either <SVCADD>, <SVCCHG>, or <SVCDEL></i>		Account-service-request aggregate Action aggregate, either <SVCADD>, <SVCCHG>, or <SVCDEL>
<SVCADD> </SVCCADD> -or- <SVCCHG> </SVCCHG> -or- <SVCDEL> </SVCDEL>		Service-addition aggregate Service-change aggregate Service-deletion aggregate
<SVC>	V1 only	Service to be added / changed / deleted BANKSVC = Banking service BPSVC = Payments service INVSVC = Investments
<SVC2>	V2 only	Service to be added / changed / deleted BANKSVC = Banking service BPSVC = Payments service INVSVC = Investments PRESSVC = Bill Presentment service
</ACCTRQ>		

8.6.1.1 Service Add Aggregate <SVCADD>

When a client sends a <SVCADD> to a financial institution routing particular messages to another service provider, it is up to the financial institution to determine whether or not an <ENROLLRQ> needs to be sent to the service provider along with the <SVCADD>. The FI may choose to always send an <ENROLLRQ> and ignore the 13550 error message responses, though this would only be reliable if <xxxACCTFROM> is included in the <ENROLLRQ>. The FI may also choose to keep a database of enrolled services, so as to send an <ENROLLRQ> only when the client is sending a <SVCADD> for a new service. The FI also has the option of sending

<ENROLLRQ>s to all service providers when the client sends the initial <ENROLLRQ> to the FI.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<SVCADD>		Service-addition aggregate
<xxxACCTTO>		Service-specific-account-identification aggregate (for example, <BANKACCTTO> or <INVACCTTO>)
</xxxACCTTO>		
<PREAUTHTOKEN>	V2 only	A token provided out-of-band by an FI or biller to speed up activation of the account, A-32
</SVCADD>		

8.6.1.2 Service Change Aggregate <SVCCHG>

<i>Tag</i>	<i>Description</i>
<SVCCHG>	Service-change aggregate
<xxxACCTFROM>	Service-specific-account-identification aggregate (for example, <BANKACCTFROM> or <INVACCTFROM>)
</xxxACCTFROM>	
<xxxACCTTO>	Service-specific-account-identification aggregate (for example, <BANKACCTTO> or <INVACCTTO>)
</xxxACCTTO>	
</SVCCHG>	

8.6.1.3 Service Delete Aggregate <SVCDEL>

<i>Tag</i>	<i>Description</i>
<SVCDEL>	Service-deletion aggregate
<xxxACCTFROM>	Service-specific-account-identification aggregate (for example, <BANKACCTFROM> or <INVACCTFROM>)
</xxxACCTFROM>	
</SVCDEL>	

8.6.2 Activation Response <ACCTRS>

The <ACCTRS> response must appear within an <ACCTTRNRS> transaction wrapper.

Tag	Version	Description
<ACCTRS> <i>Action identification. Specify either <SVCADD>, <SVCCHG>, or <SVCDEL></i>		Account-service-response aggregate
<SVCADD> </SVCADD> -or- <SVCCHG> </SVCCHG> -or- <SVCDEL> </SVCDEL>		Service-addition aggregate Service-change aggregate Service-deletion aggregate
<SVC>	V1 only	Service to be added / changed: BANKSVC = Banking service BPSVC = Payments service INVSVC = Investments
<SVC2>	V2 only	Service to be added / changed: BANKSVC = Banking service BPSVC = Payments service INVSVC = Investments PRESSVC = Bill Presentment service
<SVCSTATUS>	V1 only	AVAIL = Available, but not yet requested PEND = Requested, but not yet available ACTIVE = In use
<SVCSTATUS2>	V2 only	AVAIL = Available, but not yet requested PEND = Requested, but not yet available ACTIVE = In use REJECTED = Rejected
<REASON>	V2 only	This element is only relevant if <SVCSTATUS2>REJECTED is specified, A-255
</ACCTRS>		

8.6.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2006	Source account not found (ERROR)
2007	Source account closed (ERROR)
2008	Source account not authorized (ERROR)
2009	Destination account not found (ERROR)
2010	Destination account closed (ERROR)
2011	Destination account not authorized (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
13502	Invalid service (ERROR)
15508	Transaction not authorized (ERROR)

8.6.4 Service Activation Synchronization

Service activation requests are subject to the standard data synchronization protocol. The scope of these requests and the <TOKEN> is the user ID. The request and response tags are <ACCTSYNCRQ> and <ACCTSYNCRS>.

8.6.4.1 Request <ACCTSYNCRQ>

Tag	Version	Description
<ACCTSYNCRQ> <i>Client synchronization option; <TOKEN>, <TOKEN2>, <TOKENONLY>, or <REFRESH></i>		Activation synchronization request aggregate
<TOKEN>	V1 only	Previous value of <TOKEN> received for this type of synchronization request from server; 0 for first-time requests; <i>token</i>
<TOKEN2>	V2 only	Previous value of <TOKEN2> received for this type of synchronization request from server; 0 for first-time requests; <i>token2</i>
<TOKENONLY>		Request for just the current <TOKEN> without the history, <i>Boolean</i>
<REFRESH>		Request for refresh of current state, <i>Boolean</i>
<REJECTIFMISSING>		If Y, do not process requests if client <TOKEN> is out of date, <i>Boolean</i>
<ACCTTRNRQ>		Account-service-request transactions (0 or more)
</ACCTTRNRQ>		
</ACCTSYNCRQ>		

8.6.4.2 Response <ACCTSYNCRS>

Tag	Version	Description
<ACCTSYNCRS>		Payee-list-request aggregate
<TOKEN>	V1 only	New synchronization token, <i>token</i>
<TOKEN2>	V2 only	New synchronization token, <i>token2</i>
<LOSTSYNC>		Y if the token in the synchronization request is older than the earliest entry in the server's history table. In this case, some responses have been lost. N if the token in the synchronization request is newer than or matches a token in the server's history table. <i>Boolean</i>
<SYNCERROR>	V2 only	Optional error code (See section 6.4.4.), <i>N-6</i> .
<ACCTTRNRS>		Account-service-response transactions (0 or more)
</ACCTTRNRS>		
</ACCTSYNCRS>		

8.6.5 Examples

Activating a payment:

```
<ACCTTRNRQ>
  <TRNUID>12345
  <ACCTRQ>
    <SVCADD>
      <BANKACCTTO>
        <BANKID>1234567789
        <ACCTID>12345
        <ACCTTYPE>CHECKING
      </BANKACCTTO>
    </SVCADD>
    <SVC>BPSVC
  </ACCTRQ>
</ACCTTRNRQ>
```

A response:

```
<ACCTTRNRS>
  <TRNUID>12345
  <STATUS>
    <CODE>0
    <SEVERITY>INFO
  </STATUS>
  <ACCTRS>
    <SVCADD>
      <BANKACCTTO>
        <BANKID>1234567789
        <ACCTID>12345
        <ACCTTYPE>CHECKING
      </BANKACCTTO>
    </SVCADD>
    <SVC>BPSVC
    <SVCSTATUS>ACTIVE
  </ACCTRS>
</ACCTTRNRS>
```

8.7 Name and Address Changes

Users may request that an FI update the official name, address, phone, and e-mail information using the <CHGUSERINFORQ>. All modified and unmodified elements are submitted in a change user information request, <CHGUSERINFORQ>. The lack of inclusion of a field in a change user request when that field was previously populated implies its deletion on the server. The response reports all of the current values. An optional USERID has been added to message set version 2 to facilitate the change of user info by proxy. An example of which would be a customer service representative who might be asked by the end user to change their address. If the USERID element is not present in CHGUSERINFO, then the USERID from the SONRQ is assumed to be the identifier for the user in question. For security reasons, some of the fields in the <ENROLLRQ> cannot be changed online, such as tax ID and userID.

The transaction tags are <CHGUSERINFOTRNRQ> and <CHGUSERINFOTRNRS>. These messages are subject to synchronization, <CHGUSERINFOSYNCRQ>, and <CHGUSERINFOSYNCRS>.

8.7.1 Change User Information Request <CHGUSERINFORQ>

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<CHGUSERINFORQ>	V2 only	Change-user-information-request aggregate
<USERID>		User ID, A-32
<FIRSTNAME>		First name of user, A-32
<MIDDLENAME>		Middle name of user, A-32
<LASTNAME>		Last name of user, A-32
<ADDR1>		Address line 1, A-32
<ADDR2>		Address line 2. Use of <ADDR2> requires the presence of <ADDR1>, A-32
<ADDR3>		Address line 3. Use of <ADDR3> requires the presence of <ADDR2>, A-32
<CITY>		City, A-32
<STATE>		State or province, A-5
<POSTALCODE>		Postal code, A-11
<COUNTRY>		3-letter country code from ISO / DIS-3166, A-3
<DAYPHONE>		Daytime telephone number, A-32
<EVEPHONE>		Evening telephone number, A-32
<EMAIL>		Electronic e-mail address, A-80
</CHGUSERINFORQ>		

8.7.2 Change User Information Response <CHGUSERINFORS>

Tag	Version	Description
<CHGUSERINFORS>	V2 only	Change-user-information-request aggregate
<USERID>		User ID, A-32
<FIRSTNAME>		First name of user, A-32
<MIDDLENAME>		Middle name of user, A-32
<LASTNAME>		Last name of user, A-32
<ADDR1>		Address line 1, A-32
<ADDR2>		Address line 2. Use of <ADDR2> requires the presence of <ADDR1>, A-32
<ADDR3>		Address line 3. Use of <ADDR3> requires the presence of <ADDR2>, A-32
<CITY>		City, A-32
<STATE>		State or province, A-5
<POSTALCODE>		Postal code, A-11
<COUNTRY>		3=letter country code from ISO / DIS-3166, A-3
<DAYPHONE>		Daytime telephone number, A-32
<EVEPHONE>		Evening telephone number, A-32
<EMAIL>		Electronic e-mail address, A-80
<DTINFOCHG>		Date and time of update <i>datetime</i>
</CHGUSERINFORS>		

8.7.3 Status Codes

Code	Meaning
0	Success (INFO)
2000	General error (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
13503	Cannot change user information (ERROR)
15508	Transaction not authorized (ERROR)

8.7.4 Change User Information Synchronization

Change user information requests are subject to the standard data synchronization protocol. The scope of these requests and the <TOKEN> is the user ID. The request and response tags are <CHGUSERINFOSYNCRQ> and <CHGUSERINFOSYNCRS>.

8.7.4.1 Request <CHGUSERINFOSYNCRQ>

Tag	Version	Description
<CHGUSERINFOSYNCRQ> <i>Client synchronization option;</i> <TOKEN>, <TOKEN2>, <TOKENONLY>, or <REFRESH>		Activation synchronization request aggregate
<TOKEN>	V1 only	Previous value of <TOKEN> received for this type of synchronization request from server; 0 for first-time requests; <i>token</i>
<TOKEN2>	V2 only	Previous value of <TOKEN2> received for this type of synchronization request from server; 0 for first-time requests; <i>token2</i>
<TOKENONLY>		Request for just the current <TOKEN> without the history, <i>Boolean</i>
<REFRESH>		Request for refresh of current state, <i>Boolean</i>
<REJECTIFMISSING>		If Y, do not process requests if client <TOKEN> is out of date, <i>Boolean</i>
<CHGUSERINFOTRNRQ> </CHGUSERINFOTRNRQ> </CHGUSERINFOSYNCRQ>		Change user information request transactions (0 or more)

8.7.4.2 Response <CHGUSERINFOSYNCRS>

Tag	Version	Description
<CHGUSERINFOSYNCRS>		Payee-list-request aggregate
<TOKEN>	V1 only	New synchronization token, <i>token</i>
<TOKEN2>	V2 only	New synchronization token, <i>token2</i>
<LOSTSYNC>		Y if the token in the synchronization request is older than the earliest entry in the server's history table. In this case, some responses have been lost. N if the token in the synchronization request is newer than or matches a token in the server's history table. <i>Boolean</i>
<SYNCERROR>	V2 only	Optional error code (See section 6.4.4.), <i>N-6</i> .
<CHGUSERINFOTRNRNRS>		Change user information response transactions (0 or more)
<CHGUSERINFOTRNRNRS>		
</CHGUSERINFOSYNCRS>		

8.8 Signup Message Set Profile Information

A server must include the following aggregates as part of the profile <MSGSETLIST> response, since every server must support at least the account information and service activation messages. Servers indicate how enrollment should proceed: via the client, a given web page, or a text message directing users to some other method (such as a phone call).

Tag	Version	Description
<SIGNUPMSGSET>		Signup-message-set-profile-information aggregate
<SIGNUPMSGSETV1>		Opening tag for V1 of the message set profile information
<MSGSETCORE>		Same as V2, other than <PREAUTH> element added to V2
</MSGSETCORE>		Common message set information, defined in Chapter 7, "FI Profile"
Enrollment options. Choose one of <CLIENTENROLL>, <WEBENROLL>, or <OTHERENROLL>.		
<CLIENTENROLL>		Client-based enrollment supported
<ACCTREQUIRED>		Y if account number is required as part of enrollment, <i>Boolean</i>
</CLIENTENROLL>		
-or-		
<WEBENROLL>		Web-based enrollment supported
<URL>		URL to start enrollment process, <i>URL</i>
</WEBENROLL>		
-or-		
<OTHERENROLL>		Some other enrollment process
<MESSAGE>		Message to consumer about what to do next (for example, a phone number), <i>A-80</i>
</OTHERENROLL>		
<CHGUSERINFO>		Y if server supports client-based user information changes, <i>Boolean</i>
<AVAILACCTS>		Y if server can provide information on accounts with SVCSTATUS available, N means client should expect to ask user for specific account information, <i>Boolean</i>

Tag	Version	Description
<CLIENTACTREQ> </SIGNUPMSGSETV1> <SIGNUPMSGSETV2> <MSGSETCORE> </MSGSETCORE> <i>Enrollment options. Choose one of <CLIENTENROLL>, <WEBENROLL>, or <OTHERENROLL>.</i>		Y if server allows clients to make service activation requests (<ACCTRQ>), N if server will only advise clients via synchronization of service additions, changes, or deletions. <i>Boolean</i> Zero or more V2 message sets Same as V1, other than <PREAUTH> element added to V2 Common message set information, defined in Chapter 7, "FI Profile"
<CLIENTENROLL> <ACCTREQUIRED> </CLIENTENROLL> -or- <WEBENROLL> <URL2> </WEBENROLL> -or- <OTHERENROLL> <MESSAGE2> </OTHERENROLL>		Client-based enrollment supported Y if account number is required as part of enrollment, <i>Boolean</i> Web-based enrollment supported URL to start enrollment process, <i>URL2</i> Some other enrollment process Message to consumer about what to do next (for example, a phone number), <i>A-2000</i>
<CHGUSERINFO> <AVAILACCTS> <CLIENTACTREQ>		Y if server supports client-based user information changes, <i>Boolean</i> Y if server can provide information on accounts with SVCSTATUS available, N means client should expect to ask user for specific account information, <i>Boolean</i> Y if server allows clients to make service activation requests (<ACCTRQ>), N if server will only advise clients via synchronization of service additions, changes, or deletions. <i>Boolean</i>

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<PREAUTH> <CANSUPPORTUSERID> </SIGNUPMSGSETV2> </SIGNUPMSGSET>	1.6 add	Y if server supports <PREAUTHTOKEN> for account activation, <i>Boolean</i> . Y if server supports <USERID> field in the <SIGNUPMSGSRQV2> wrapper. These OFX 1.6 additions must be explicitly supported by the server before they are used by a client. The default for this option is N. <i>Boolean</i>

CHAPTER 9 CUSTOMER TO FI COMMUNICATION

9.1 The E-Mail Message Set

The e-mail message set includes two messages: generic e-mail and generic MIME requests by way of URLs. In OFX files, the message set names are EMAILMSGSV1 and EMAILMSGSV2.

9.2 E-Mail Messages

OFX allows consumers and FIs to exchange messages. The message body can be placed in HTML so that FIs can provide some graphic structure to the message. Keep in mind that, as with regular World Wide Web browsing, an OFX client might not support some or all of the HTML formatting, so the text of the message must be clear on its own. Clients can request the server to send graphics (the images referenced in an tag) as part of the response file, or clients can separately request those elements. If a server sends images, it should use the standard procedure for incorporating external data as described in [Chapter 2, "Structure."](#) Servers are not required to support HTML or to send images, even if the client asks.

A user or an FI can originate a message. E-mail messages are subject to data synchronization so that a server can send a response again if it is lost or if multiple clients use it.

Because e-mail messages cannot be replied to immediately, the response should just echo back the original message (so that data synchronization will get this original e-mail message to other clients). When the FI is ready to reply, it should generate an unsolicited response (<TRNUID>0) and the client will pick this up during synchronization.

<i>Client Sends</i>	<i>Server Responds</i>
Account information	
From, To	
Subject	
Message	
	Account information
	From, To
	Subject
	Message
	Type

9.2.1 Regular vs. Specialized E-Mail

Several services with OFX define e-mail requests and responses that contain additional information specific to that service. To simplify implementation for OFX clients and servers, this section defines a <MAIL> aggregate that OFX uses in all e-mail requests and responses. For regular e-mail, the only additional information is an account-from aggregate and whether to include images in the e-mail response or not.

When users want to send messages about service-specific problems, service-specific messages are best. However, when service-specific mail transactions are not available, general mail is acceptable.

9.2.2 Basic <MAIL> Aggregate

Tag	Description
<MAIL>	Core e-mail aggregate
<USERID>	User ID such as SSN, A-32
<DTCREATED>	When message was created, <i>datetime</i>
<FROM>	Who the message is from, A-32
<TO>	Who the message should be delivered to, A-32
<SUBJECT>	Subject of message (plain text, not HTML), A-60
<MSGBODY>	Body of message, HTML-encoded or plain text depending on <USEHTML>, HTML-encoded text - A-10000 Plain text - A-2000
</MSGBODY>	End of message. Note: Unlike most other elements in OFX, <MSGBODY> always requires an end tag.
<INCIMAGES>	Include images in the message body. <i>Boolean</i>
<USEHTML>	Y for HTML-formatted text. N for plain text. See section 9.2.2.2 for more information. <i>Boolean</i>
</MAIL>	

9.2.2.1 <INCIMAGES>

The meaning of the <INCIMAGES> element depends on whether the element appears in a request or response.

When used in a request, <INCIMAGES> indicates whether the client accepts mail that includes images in the message body.

<i>When used in a request...</i>	<i>Description</i>
<INCIMAGES>Y	The client accepts mail that includes images in the message body. In this case, the server can choose whether to send images in the response.
<INCIMAGES>N	The client does not accept mail that includes images in the message body. In this case, the server must not send images in the response.

When used in a response, <INCIMAGES> indicates whether the server included images in the message body.

<i>When used in a response...</i>	<i>Description</i>
<INCIMAGES>Y	The server included images in the message body.
<INCIMAGES>N	The server did not include images in the message body.

9.2.2.2 <USEHTML>

The meaning of the <USEHTML> element depends on whether the element appears in a request or response.

When used in a request, <USEHTML> indicates whether the client sends and accepts HTML-formatted text in the message body. If a server receives a <xxxMAILSYNCRQ> request with <USEHTML>Y set, the server should process the request whether or not it supports HTML mail. If a server does not support HTML mail, it should simply set the <USEHTML> flag to N in any transactions which are returned in the sync response.

<i>When used in a request...</i>	<i>Description</i>
<USEHTML>Y	The client is including HTML-formatted text in the message body. In addition, the client will accept mail responses that include HTML-formatted text in the message body. In this case, a server can choose whether to respond with HTML-formatted text or plain text.
<USEHTML>N	The client is not including HTML-formatted text in the message body. In addition the client will not accept mail responses that include HTML-formatted text in the message body.

When used in a response, <USEHTML> indicates whether the message body includes HTML-formatted text or plain text.

<i>When used in a response...</i>	<i>Description</i>
<USEHTML>Y	The server is including HTML-formatted text in the message body.
<USEHTML>N	The server is including only plain text in the message body.

Note: When using HTML for the message body, clients and servers are REQUIRED to enclose the HTML in an SGML-marked section to protect the HTML markup: <![CDATA[... html ...]]>. For an example, see section [9.2.5](#).

9.2.3 E-Mail <MAILRQ> <MAILRS>

E-mail is subject to synchronization. The transaction aggregate is <MAILTRNRQ> / <MAILTRNRS> and the synchronization aggregate is <MAILSYNCRQ> / <MAILSYNCRS>.

<i>Tag</i>	<i>Description</i>
<MAILRQ>	E-mail-message-request aggregate
<MAIL>	Core e-mail aggregate
</MAIL>	
</MAILRQ>	

In a response, the <TRNUID> is zero if this is an unsolicited message. Otherwise, it should contain the <TRNUID> of the user's original message. It is RECOMMENDED that servers include the <MESSAGE> of the user's message as part of the reply <MESSAGE>. The <MESSAGE> contents can include carriage returns to identify desired line breaks.

<i>Tag</i>	<i>Description</i>
<MAILRS>	E-mail-message-response aggregate
<MAIL>	Core e-mail aggregate
</MAIL>	
</MAILRS>	

9.2.3.1 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
15508	Transaction not authorized (ERROR)
16500	HTML not allowed (ERROR)
16501	Unknown mail To: (ERROR)

9.2.4 E-Mail Synchronization <MAILSYNCRQ> <MAILSYNCRS>

E-mail presents a special case with regards to synchronization. Since FIs will not immediately reply to a user's e-mail, the response to the user's e-mail only echoes the request and confirms that the e-mail was successfully received. The client receives the real response to the e-mail following a synchronization request.

Note that this synchronization action expects only the basic <MAILRS> responses. Specialized e-mail is received by means of their own synchronization requests.

Tag	Version	Description
<MAILSYNCRQ> <i>Client synchronization option; <TOKEN>, <TOKEN2>, <TOKENONLY>, or <REFRESH></i>		E-mail-synchronization-request aggregate
<TOKEN>	V1 only	Previous value of <TOKEN> received for this type of synchronization request from server; 0 for first-time requests; <i>token</i>
<TOKEN2>	V2 only	Previous value of <TOKEN2> received for this type of synchronization request from server; 0 for first-time requests; <i>token2</i>
<TOKENONLY>		Request for just the current <TOKEN> without the history, <i>Boolean</i>
<REFRESH>		Request for refresh of current state, <i>Boolean</i>
<REJECTIFMISSING>		If Y, do not process requests if client <TOKEN> is out of date, <i>Boolean</i>
<INCIMAGES>		Y if the client accepts mail with images in the message body, N if the client does not accept mail with images in the message body, <i>Boolean</i>
<USEHTML>		Y if client wants an HTML response, N if client wants plain text, <i>Boolean</i>
<MAILTRNRQ> </MAILTRNRQ> </MAILSYNCRQ>		Mail-transaction-request aggregate (0 or more)

Tag	Version	Description
<MAILSYNCRS>		E-mail-synchronization-response. aggregate
<TOKEN>	V1 only	Server history marker, <i>token</i>
<TOKEN2>	V2 only	Server history marker, <i>token2</i>
<LOSTSYNC>		Y if the token in the synchronization request is older than the earliest entry in the server's history table. In this case, some responses have been lost. N if the token in the synchronization request is newer than or matches a token in the server's history table. <i>Boolean</i>
<SYNCEERROR>	V2 only	Optional error code (See section 6.4.4), <i>N-6</i>
<MAILTRNRS>		Missing e-mail response transactions (0 or more)
</MAILTRNRS>		
</MAILSYNCRS>		

9.2.5 E-Mail Example

In this example, a consumer requests information about the checking statement just downloaded. Since the financial institution will not immediately answer the inquiry, the immediate response only echoes the consumer's request and confirms that the request was successfully received.

The client receives the real response at a later time following a mail synchronization request. For an example of the mail synchronization request and response, see section 9.2.5.1.

Note: This example omits the <OFX> top level and the signon <SONRQ>. Since this example uses HTML for the message body, it must protect the HTML content in an SGML CDATA-marked section.

The request:

```
<MAILTRNRQ>
  <TRNUID>54321
  <MAILRQ>
    <MAIL>
      <USERID>123456789
      <FROM>James Hackleman
      <TO>Noelani Federal Savings
      <SUBJECT>What do I need to earn interest?
      <DTCREATED>19960305
```

```

    <MSGBODY><![CDATA[<HTML><BODY>I didn't earn any interest this
month. Can you please tell me what I need to do to earn interest on this
account?</BODY></HTML>
]]></MSGBODY>
    <INCIMAGES>N
    <USEHTML>Y
</MAIL>
</MAILRQ>
</MAILTRNRQ>

```

The response from the FI:

```

<MAILTRNRS>
  <TRNUID>54321
  <STATUS>
    <CODE>0
    <SEVERITY>INFO
  </STATUS>
  <MAILRS>
    <MAIL>
      <USERID>123456789
      <FROM>James Hackleman
      <TO>Noelani Federal Savings
      <SUBJECT>What do I need to earn interest?
      <DTCREATED>19960305
      <MSGBODY><![CDATA[<HTML><BODY>I didn't earn any interest this
month. Can you please tell me what I need to do to earn interest on this
account?</BODY></HTML>
]]></MSGBODY>
      <INCIMAGES>N
      <USEHTML>Y
    </MAIL>
  </MAILRS>
</MAILTRNRS>

```

9.2.5.1 E-Mail Synchronization Example

In the following example, the client has not yet received the reply to the e-mail sent in the previous example, so its <TOKEN> is one less than the server's. The server replies by giving the current <TOKEN> and the missed response.

```

<MAILSYNCRQ>
  <TOKEN>101
  <REJECTIFMISSING>N
  <INCIMAGES>N

```

```

    <USEHTML>Y
  </MAILSYNCRQ>

  <MAILSYNCRS>
    <TOKEN>102
    <MAILTRNRS>
      <TRNUID>0
    <STATUS>                                <!-- server initiated response -->
      <CODE>0
      <SEVERITY>INFO
    </STATUS>
    <MAILRS>
      <MAIL>
        <USERID>123456789
        <DTCREATED>19960307
        <FROM>Noelani Federal Savings
        <TO>James Hackleman
        <SUBJECT>Re: What do I need to earn interest?
        <MSGBODY>><![CDATA[<HTML><BODY>You need to maintain $1000 in
this account to earn interest. Because your balance was only $750 this
month, no interest was earned. You could also switch to our new Checking
Extra plan that always pays interest. Call us or check our web page
http://www.fi.com/check-plans.html for more information.
Sincerely,
Customer Service Department

Original message:
I didn't earn any interest this month. Can you please tell me what I
need to do to earn interest on this account?</BODY></HTML>
]]></MSGBODY>
      <INCIMAGES>N
      <USEHTML>Y
    </MAIL>
  </MAILRS>
</MAILTRNRS>
</MAILSYNCRS>

```

9.3 Get HTML Page

Some responses (<PROFRS> and <FINDBILLERRS> for example) contain values that are URLs intended to be separately fetched by clients. Clients can use their own HTTP libraries to perform this fetch outside of the OFX specification. However, to insulate clients against changes in transport technology, and to allow for fetches that require the protection of an authenticated signon by a specific user, OFX defines a transaction roughly equivalent to an HTTP Get. Any MIME type can be retrieved, including images as well as HTML pages.

When a <GETMIMERQ> request appears in a request file and no error occurs in processing, the server must return a response file containing multiple entities (defined in the MIME protocol to include the MIME headers and content for one part of the transmission). Such a response file has content type “multipart/x-mixed-replace”, as discussed in section [2.1](#). One entity contains the OFX response. Other entities contain the content of individual retrievals corresponding to each <GETMIMERS> in the OFX entity.

When multiple <GETMIMERS> responses (corresponding to successful <GETMIMERQ> requests) appear in an OFX response entity, the server must return individual entities in the same order as the corresponding response aggregates. Since the OFX response itself should be the only entity with content type “application/x-ofx” in the response file, the client may find the retrieved information in predictable locations within the multipart response.

9.3.1 MIME Get Request and Response <GETMIMERQ> <GETMIMERS>

The following table lists the components of a request:

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<GETMIMERQ>		Get-MIME-request aggregate
<URL>	V1 only	URL, <i>URL</i>
<URL2>	V2 only	URL, <i>URL2</i>
</GETMIMERQ>		

The response simply echoes the URL. The actual response, whether HTML, an image, or some other type, is always sent as a separate part of the file using multipart MIME.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<GETMIMERS>		Get-MIME-response aggregate
<URL>	V1 only	URL, <i>URL</i>
<URL2>	V2 only	URL, <i>URL2</i>
</GETMIMERS>		

9.3.1.1 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2019	Duplicate request (ERROR)
16502	Invalid URL (ERROR)
16503	Unable to get URL (ERROR)

9.3.2 MIME Example

A request:

```
<GETMIMETRNRQ>
  <TRNUID>54321
  <GETMIMERQ>
    <URL>http://www.fi.com/apage.html
  </GETMIMERQ>
</GETMIMETRNRQ>
```

A response – the full file is shown here to illustrate the use of multipart MIME:

```
HTTP 1.0 200 OK
Content-Type: multipart/x-mixed-replace; boundary =boundary =XYZZY24x7

--XYZZY24x7
Content-Type: application/x-ofx
Content-Length: 8732

OFXHEADER:100
DATA:OFXSGML
```

```
VERSION:160
SECURITY:TYPE1
ENCODING:USASCII
CHARSET:NONE
COMPRESSION:NONE
NEWFILEUID:NONE
OLDFILEUID:NONE
<OFX>
  <!-- signon not shown
  message set wrappers not shown -->
<GETMIMETRNR>
  <TRNUID>54321
  <STATUS>
    <CODE>0
    <SEVERITY>INFO
  </STATUS>
  <GETMIMERS>
    <URL>http://www.fi.com/apage.html
  </GETMIMERS>
</GETMIMETRNR>
</OFX>

--XYZZY24x7
Content-Type: text/html
<HTML>
  <!-- standard HTML page -->
</HTML>

--XYZZY24x7--
```

9.4 E-Mail Message Set Profile Information

If either or both of the messages in the e-mail message set are supported, the following aggregate must be included in the profile <MSGSETLIST> response. If <EMAILMSGSET> is supported by the server, you must also support <MAILSYNCRQ>.

Tag	Description
<EMAILMSGSET>	E-mail-message-set-profile-information aggregate
<EMAILMSGSETV1>	Opening tag for V1 of the message set profile information
<MSGSETCORE>	Common message set information, defined in Chapter 7, "FI Profile"
</MSGSETCORE>	
<MAILSUP>	Y if server supports <MAILRQ> request. N if server supports only the <MAILSYNCRQ> request. <i>Boolean</i>
<GETMIMESUP>	Y if server supports get MIME message, <i>Boolean</i>
</EMAILMSGSETV1>	
<EMAILMSGSETV2>	Zero or more V2 message sets
<MSGSETCORE>	Common message set information, defined in Chapter 7, "FI Profile"
</MSGSETCORE>	
<MAILSUP>	Y if server supports the <MAILRQ> request. If N, the server supports only the <MAILSYNCRQ> request. <i>Boolean</i>
<GETMIMESUP>	Y if server supports get MIME message, <i>Boolean</i>
</EMAILMSGSETV2>	
</EMAILMSGSET>	

CHAPTER 10 RECURRING TRANSACTIONS

OFX enables users to automate transactions that occur on a regular basis. Recurring transactions are useful when a customer has payments or transfers, for example, that repeat at regular intervals. The customer can create a “model” at the server for automatic generation of these instructions. The model in turn creates payments or transfers until it is canceled or expires. After the user creates a recurring model at the server, the server can relieve the user from the burden of creating these transactions; it generates the transactions on its own, based on the operating parameters of the model.

10.1 Creating a Recurring Model

The client must provide the following information to create a model:

- ◆ Type of transaction generated by the model (payment or transfer)
- ◆ Frequency of recurring transaction
- ◆ Total number of recurring transactions to generate
- ◆ Service-specific information, such as transfer date, payment amount, payee address

The model creates each transaction some time before its due date, usually thirty days. This allows the user to retrieve the transactions in advance of posting. This also gives the user the opportunity to modify or cancel individual transactions without changing the recurring model itself.

When a model is created, it can generate several transactions immediately. The model does not automatically return responses for the newly created transactions. It returns a response only to the request that was made to create the model. For this reason, clients should send a synchronization request along with the request to create a model. This allows the server to return the newly created transaction responses, as well as the response to the request to set up a new model.

10.2 Recurring Instructions <RECURRINST>

The Recurring Instructions aggregate is used to specify the schedule for a repeating instruction. It is passed to the server when a recurring transfer or payment model is first created.

Tag	Description
<RECURRINST>	Recurring-Instructions aggregate
<NINSTS>	Number of instructions If this element is absent, the schedule is open-ended, <i>N-3</i>
<FREQ>	Frequency, see section 10.2.1
</RECURRINST>	

10.2.1 Values for <FREQ>

Value	Description
WEEKLY	Weekly
BIWEEKLY	Biweekly
TWICEMONTHLY	Twice a month
MONTHLY	Monthly
FOURWEEKS	Every four weeks
BIMONTHLY	Bimonthly
QUARTERLY	Quarterly
SEMIANNUALLY	Semiannually
ANNUALLY	Annually

Rules for calculating recurring dates of WEEKLY, BIWEEKLY, and TWICEMONTHLY are as follows:

- ◆ WEEKLY = starting date for first transaction, starting date + 7 days for the second
- ◆ TWICEMONTHLY = starting date for first, starting date + 15 days for the second
- ◆ BIWEEKLY = starting date for first, starting date + 14 days for the second

Examples:

Start date of May 2: next transaction date for WEEKLY is May 9; TWICEMONTHLY is May 17; next transfer date for BIWEEKLY is May 16.

Start date of May 20: next date for WEEKLY is May 27; TWICEMONTHLY is June 4; next date for BIWEEKLY is June 3.

TWICEMONTHLY recurring transactions will occur each month on those days adjusting for weekends and holidays. BIWEEKLY will occur every 14 days.

10.2.2 Examples

The following example illustrates the creation of a repeating payment. The payment repeats on a monthly basis for 12 months. All payments are for \$395.

The request:

```
.  
.   
.   
<RECPMTRQ>  
  <RECURRINST>  
    <NINSTS>12  
    <FREQ>MONTHLY  
  </RECURRINST>  
  <PMTINFO>  
    <BANKACCTFROM>  
      <BANKID>555432180  
      <ACCTID>763984  
      <ACCTTYPE>CHECKING  
    </BANKACCTFROM>  
    <TRNAMT>395.00  
    <PAYEEID>77810  
    <PAYACCT>444-78-97572  
    <DTDUE>19971115  
    <MEMO>Auto loan payment  
  </PMTINFO>  
</RECPMTRQ>  
.   
.   
. 
```

The response includes the <RECSRVRTID> that the client can use to cancel or modify the model:

```
.  
.   
.   
<RECPMTRS>  
  <RECSRVRTID>387687138  
  <RECURRINST>  
    <NINSTS>12  
    <FREQ>MONTHLY  
  
  </RECURRINST>  
  <PMTINFO>  
    <BANKACCTFROM>  
      <BANKID>555432180  
      <ACCTID>763984  
      <ACCTTYPE>CHECKING  
    </BANKACCTFROM>  
    <TRNAMT>395.00  
    <PAYEEID>77810  
    <PAYACCT>444-78-97572  
    <DTDUE>19971115  
    <MEMO>Auto loan payment  
  </PMTINFO>  
</RECPMTRS>  
.   
.   
.
```

10.3 Retrieving Transactions Generated by a Recurring Model

Once created, a recurring model independently generates instructions. At the time the instance is generated, its status is pending. At this point, the pending/spawned transaction is treated as a single transaction, and the rules for what happens to this transaction are the same as if it had been generated from an explicit request. Since the client has not directly generated these transactions, the client has no record of their creation. To enable users to modify and/or cancel these transactions, the client must use data synchronization in order to retrieve these transactions. (Some message sets also support an inquiry request, which may be used once the SRVRTID of the transaction is obtained via synchronization.)

The client has two purposes for synchronizing state with the server with respect to recurring models:

- ◆ Retrieve any added, modified, or canceled recurring models
- ◆ Retrieve any added, modified, or canceled transactions generated by any models

The client must be able to synchronize with the state of any models at the server, as well as the state of any transactions generated by the server.

10.4 Modifying and Canceling Individual Transactions

Once created and retrieved by the customer, recurring payments and transfers are almost identical to customer-created payments or transfers. As with ordinary payments or transfers, you can cancel or modify transactions individually. However, because servers generate these transfers, they are different in the following respects:

- ◆ Recurring transactions must be retrieved as part of a synchronization request.
- ◆ Recurring transactions are related to a model. A server can modify or cancel transactions if the model is modified or canceled.

10.5 Modifying and Canceling Recurring Models

A recurring model can be modified or canceled. When a model is modified, all transactions that it generates in the future will change as well. The client can indicate whether transactions that have been generated, but have not been sent, should be modified as well. The actual elements within a transaction that can be modified differ by service. See the recurring sections within [Chapter 11, "Banking,"](#) and [Chapter 12, "Payments,"](#) for details. When a model is cancelled, the server cancels any transactions that it has not yet sent.

If a client indicates that the modification or cancellation of a model should also affect its pending transactions, those individual modifications/cancellations must appear in the appropriate synchronization response the next time a synchronization request is made. For example, a recurring payment cancellation request that affects pending payments should cause payment

cancellation responses to show up in the payment synchronization response for all pending payments belonging to the model.

10.5.1 Examples

Canceling a recurring payment model requires the client to pass the <RECSRVRTID> of the model. The client requests that pending payments also be canceled. The server cancels the model immediately and notifies the client that the model was canceled.

The request:

```
.  
.   
.   
  <RECPMTCANCRQ>  
    <RECSRVRTID>387687138  
    <CANPENDING>Y  
  </RECPMTCANCRQ>  
.   
.   
. 
```

The response:

```
.   
.   
.   
  <RECPMTCANCERS>  
    <RECSRVRTID>387687138  
    <CANPENDING>Y  
  </RECPMTCANCERS>  
.   
.   
. 
```

The server also cancels any payments that have been generated but not executed. In the example shown above, the client would not learn of this immediately. To receive notification that all pending payments were canceled, the client would need to send a synchronization request in the file. The following example illustrates this.

The next request file contains a synchronization request:

```
.  
.   
.   
  <PMTSYNCRQ>  
    <TOKEN>12345  
    <REJECTIFMISSING>N  
    <BANKACCTFROM>  
      <BANKID>123432123  
      <ACCTID>516273  
      <ACCTTYPE>CHECKING  
    </BANKACCTFROM>  
  </PMTSYNCRQ>  
.   
.   
.   

```

The response file contains one response (assuming one payment was pending).

```
.   
.   
.   
  <PMTSYNCRS>  
    <TOKEN>123456  
    <BANKACCTFROM>  
      <BANKID>123432123  
      <ACCTID>516273  
      <ACCTTYPE>CHECKING  
    </BANKACCTFROM>  
    <PMTTRNRS>  
      <TRNUID>0  
      <STATUS>  
        <CODE>0  
        <SEVERITY>INFO  
      </STATUS>  
      <PMTCANCRS>  
        <SRVRTID>1030155  
      </PMTCANCRS>  
    </PMTTRNRS>  
  </PMTSYNCRS>  
.   
.   

```

Note that because requests are not guaranteed to be executed in order, a PMTSYNCRQ in the same file as the RECPMTCANCRQ would not guarantee that the cancelled payments would be returned, since the PMYSYNCRQ might be executed first. This is the reason two OFX files are required in the example above.

10.6 Expired Models

A model should (preferably) expire after the last pending transfer / payment has been executed for that model, rather than when the last transfer / payment has been spawned. This enables the user to change and / or cancel the model (possibly, with the <MODPENDING>Y or <CANPENDING>Y flags) during this period.

Models should show up in synchronization responses even after they have expired (at least for a time), since the RECSRVRTID will be in payment synchronization responses and a client needs to find the corresponding model. Servers may safely remove this information shortly after the final payment or transfer has posted to the source account.

CHAPTER 11 BANKING

OFX enables financial institution (FI) customers to keep their finances up-to-date and to manage their bank accounts conveniently in several ways. Customers can download transactions and update account balances on a daily basis. They can retrieve a closing statement that contains the same information that they are accustomed to seeing on a paper statement. They can transfer funds between accounts at a financial institution, either immediately upon going online or on a regular schedule. Customers can schedule transfers between accounts on a recurring basis and can transfer funds between accounts at different financial institutions. If necessary, customers can request a wire funds transfer. OFX also enables requests to stop payment on pending checks.

Using customer notification, an FI can notify customers of important events regarding their accounts, such as returned checks or deposits.

11.1 Consumer and Business Banking

OFX supports banking for both consumers and businesses. Some customers might use some areas more heavily within OFX Banking (such as credit card download); other areas might be more appropriate for businesses (such as wire transfers). Yet all of the functionality defined for Banking is appropriate to some extent for both consumer and business applications.

11.2 Credit Card Data

Credit card data is available to OFX clients through the statement download facility. Statement download provides a way to download credit card transaction data and balances on an as-needed basis. Statement closing information can be made available to clients as well.

11.3 Common Banking Aggregates

This section describes several aggregates used throughout the Banking portion of OFX.

11.3.1 Banking Account <BANKACCTFROM> and <BANKACCTTO>

OFX uses the Banking Account aggregates to identify an account at an FI. The aggregates contain enough information to uniquely identify an account for the purposes of statement download, bill payment, and funds transfer. <CCACCTFROM> identifies credit card accounts; see section [11.3.2](#).

Tag	Version	Description																						
<BANKACCTFROM>		Bank-account-from aggregate																						
<BANKID>		Bank identifier, A-9 Use of this field by country: <table><tr><th>COUNTRY</th><th>Interpretation</th></tr><tr><td>BEL</td><td>Bank code</td></tr><tr><td>CAN</td><td>Routing and transit number</td></tr><tr><td>CHE</td><td>Clearing number</td></tr><tr><td>DEU</td><td>Bankleitzahl</td></tr><tr><td>ESP</td><td>Entidad</td></tr><tr><td>FRA</td><td>Banque</td></tr><tr><td>GBR</td><td>Sort code</td></tr><tr><td>ITA</td><td>ABI</td></tr><tr><td>NLD</td><td>Not used (field contents ignored)</td></tr><tr><td>USA</td><td>Routing and transit number</td></tr></table>	COUNTRY	Interpretation	BEL	Bank code	CAN	Routing and transit number	CHE	Clearing number	DEU	Bankleitzahl	ESP	Entidad	FRA	Banque	GBR	Sort code	ITA	ABI	NLD	Not used (field contents ignored)	USA	Routing and transit number
COUNTRY	Interpretation																							
BEL	Bank code																							
CAN	Routing and transit number																							
CHE	Clearing number																							
DEU	Bankleitzahl																							
ESP	Entidad																							
FRA	Banque																							
GBR	Sort code																							
ITA	ABI																							
NLD	Not used (field contents ignored)																							
USA	Routing and transit number																							
<BRANCHID>		Branch identifier. May be required for some non-US banks, A-22 Use of this field by country: <table><tr><th>COUNTRY</th><th>Interpretation</th></tr><tr><td>BEL</td><td>Not present</td></tr><tr><td>CAN</td><td>Not present</td></tr><tr><td>CHE</td><td>Not present</td></tr><tr><td>DEU</td><td>Not present</td></tr><tr><td>ESP</td><td>Oficina</td></tr><tr><td>FRA</td><td>Agence</td></tr><tr><td>GBR</td><td>Not present</td></tr><tr><td>ITA</td><td>CAB</td></tr><tr><td>NLD</td><td>Not present</td></tr><tr><td>USA</td><td>Not present</td></tr></table>	COUNTRY	Interpretation	BEL	Not present	CAN	Not present	CHE	Not present	DEU	Not present	ESP	Oficina	FRA	Agence	GBR	Not present	ITA	CAB	NLD	Not present	USA	Not present
COUNTRY	Interpretation																							
BEL	Not present																							
CAN	Not present																							
CHE	Not present																							
DEU	Not present																							
ESP	Oficina																							
FRA	Agence																							
GBR	Not present																							
ITA	CAB																							
NLD	Not present																							
USA	Not present																							
<ACCTID>		Account number, A-22																						

Tag	Version	Description																						
<ACCTTYPE>	V1 only	Type of account, see section 11.3.1.2																						
<ACCTTYPE2>	V2 only	Type of account, see section 11.3.1.2																						
<ACCTKEY>		Checksum, A-22 Use of this field by country: <table><tr><th><u>COUNTRY</u></th><th><u>Interpretation</u></th></tr><tr><td>BEL</td><td>Check digits</td></tr><tr><td>CAN</td><td>Not present</td></tr><tr><td>CHE</td><td>Not present</td></tr><tr><td>DEU</td><td>Not present</td></tr><tr><td>ESP</td><td>D.C.</td></tr><tr><td>FRA</td><td>Clé</td></tr><tr><td>GBR</td><td>Not present</td></tr><tr><td>ITA</td><td>CIN</td></tr><tr><td>NLD</td><td>Not present</td></tr><tr><td>USA</td><td>Not present</td></tr></table>	<u>COUNTRY</u>	<u>Interpretation</u>	BEL	Check digits	CAN	Not present	CHE	Not present	DEU	Not present	ESP	D.C.	FRA	Clé	GBR	Not present	ITA	CIN	NLD	Not present	USA	Not present
<u>COUNTRY</u>	<u>Interpretation</u>																							
BEL	Check digits																							
CAN	Not present																							
CHE	Not present																							
DEU	Not present																							
ESP	D.C.																							
FRA	Clé																							
GBR	Not present																							
ITA	CIN																							
NLD	Not present																							
USA	Not present																							
</BANKACCTFROM>																								

Tag	Version	Description																						
<BANKACCTTO>		Bank-account-to aggregate																						
<BANKID>		Bank identifier, A-9 Use of this field by country: <table><tr><th><u>COUNTRY</u></th><th><u>Interpretation</u></th></tr><tr><td>BEL</td><td>Bank code</td></tr><tr><td>CAN</td><td>Routing and transit number</td></tr><tr><td>CHE</td><td>Clearing number</td></tr><tr><td>DEU</td><td>Bankleitzahl</td></tr><tr><td>ESP</td><td>Entidad</td></tr><tr><td>FRA</td><td>Banque</td></tr><tr><td>GBR</td><td>Sort code</td></tr><tr><td>ITA</td><td>ABI</td></tr><tr><td>NLD</td><td>Not used (field contents ignored)</td></tr><tr><td>USA</td><td>Routing and transit number</td></tr></table>	<u>COUNTRY</u>	<u>Interpretation</u>	BEL	Bank code	CAN	Routing and transit number	CHE	Clearing number	DEU	Bankleitzahl	ESP	Entidad	FRA	Banque	GBR	Sort code	ITA	ABI	NLD	Not used (field contents ignored)	USA	Routing and transit number
<u>COUNTRY</u>	<u>Interpretation</u>																							
BEL	Bank code																							
CAN	Routing and transit number																							
CHE	Clearing number																							
DEU	Bankleitzahl																							
ESP	Entidad																							
FRA	Banque																							
GBR	Sort code																							
ITA	ABI																							
NLD	Not used (field contents ignored)																							
USA	Routing and transit number																							
<BRANCHID>		Branch identifier. May be required for some banks, A-22 Use of this field by country: <table><tr><th><u>COUNTRY</u></th><th><u>Interpretation</u></th></tr><tr><td>BEL</td><td>Not present</td></tr><tr><td>CAN</td><td>Not present</td></tr><tr><td>CHE</td><td>Not present</td></tr><tr><td>DEU</td><td>Not present</td></tr><tr><td>ESP</td><td>Oficina</td></tr><tr><td>FRA</td><td>Agence</td></tr><tr><td>GBR</td><td>Not present</td></tr><tr><td>ITA</td><td>CAB</td></tr><tr><td>NLD</td><td>Not present</td></tr><tr><td>USA</td><td>Not present</td></tr></table>	<u>COUNTRY</u>	<u>Interpretation</u>	BEL	Not present	CAN	Not present	CHE	Not present	DEU	Not present	ESP	Oficina	FRA	Agence	GBR	Not present	ITA	CAB	NLD	Not present	USA	Not present
<u>COUNTRY</u>	<u>Interpretation</u>																							
BEL	Not present																							
CAN	Not present																							
CHE	Not present																							
DEU	Not present																							
ESP	Oficina																							
FRA	Agence																							
GBR	Not present																							
ITA	CAB																							
NLD	Not present																							
USA	Not present																							
<ACCTID>		Account number, A-22																						
<ACCTTYPE>	V1 only	Type of account, see section 11.3.1.2																						
<ACCTTYPE2>	V2 only	Type of account, see section 11.3.1.2																						
<ACCTKEY>		Checksum, A-22 Use of this field by country:																						

Tag	Version	Description
<EXTBANKACCTTO>	V2 only	<u>COUNTRY</u> <u>Interpretation</u>
		BEL Check digits
		CAN Not present
		CHE Not present
		DEU Not present
		ESP D.C.
		FRA Clé
		GBR Not present
		ITA CIN
		NLD Not present
		USA Not present
		Extended bank account-to information aggregate. This aggregate is only present when <BANKACCTTO> appears within the Bill Payment message set. See section <u>11.3.1.1</u> .
Presence of this field by country:		
</EXTBANKACCTTO>		<u>COUNTRY</u> <u>Interpretation</u>
		BEL Not present
		CAN Not present
		CHE Required for payments and payees
		DEU Required for payments and payees
		ESP Not present
		FRA Not present
		GBR Not present
		ITA Required for payments and payees
		NLD Not present
		USA Not present
		</BANKACCTTO>

11.3.1.1 Extended Banking Account Information <EXTBANKACCTTO>

The <EXTBANKACCTTO> aggregate contains additional information used to identify an account being used for a payment transfer and payee requests. It only appears within <BANKACCTTO> aggregates that are used within the Bill Payment message set, and not inside any version 1 message sets. The <EXTBANKACCTTO> aggregate is only used if COUNTRY is CHE, DEU or ITA.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<EXTBANKACCTTO>	V2 only	Extended Bank-account-to aggregate
<BANKNAME>		Bank name, A-32 (mandatory if COUNTRY is CHE, DEU or ITA)
<BANKBRANCH>		Bank branch name, A-32 (mandatory if COUNTRY is ITA)
<BANKCITY>		Bank branch city, A-32 (mandatory if COUNTRY is CHE)
<BANKPOSTALCODE>		Bank branch postal code, A-11 (mandatory if COUNTRY is CHE)
<CHE.PTTACCTID>		PTT account number for indirect payments, A-9 (optional if COUNTRY is CHE)
</EXTBANKACCTTO>		

11.3.1.2 Account Types for <ACCTTYPE> and <ACCTTYPE2> Elements

<i>Type</i>	<i>Version</i>	<i>Description</i>
CHECKING		Checking
SAVINGS		Savings
MONEYMRKT		Money Market
CREDITLINE		Line of credit
CMA	<ACCTTYPE2> only	Cash Management Account

11.3.2 Credit Card Account <CCACCTFROM> and <CCACCTTO>

OFX uses the Credit Card Account aggregate to identify a credit card account at an FI. The aggregate contains enough information to uniquely identify an account for the purposes of statement downloads and funds transfer. It is not necessary to support the Credit Card Message Set in order to use the Credit card account aggregate.

<i>Tag</i>	<i>Description</i>
<CCACCTFROM>	Credit-card-account-from aggregate
<ACCTID>	Account number, A-22
<ACCTKEY>	Checksum for international banks, A-22
</CCACCTFROM>	

The <CCACCTTO> aggregate contains the same elements.

11.3.3 Bank Account Information <BANKACCTINFO>

OFX uses the bank account information aggregate to download account information from an FI. It includes account number specification in <BANKACCTFROM> as well as the status of the service.

Tag	Version	Description
<BANKACCTINFO>		Bank-account-information aggregate
<BANKACCTFROM>		Bank-account-from aggregate
</BANKACCTFROM>		
<SUPTXDL>		Y if account supports transaction detail downloads, N if it is balance-only, <i>Boolean</i>
<XFERSRC>		Y if account is enabled as a source for an intrabank or interbank transfer, <i>Boolean</i>
<XFERDEST>		Y if account is enabled as a destination for an intrabank or interbank transfer, <i>Boolean</i>
<SVCSTATUS>	V1 only	Status of the account AVAIL = Available, but not yet requested PEND = Requested, but not yet available ACTIVE = In use
<SVCSTATUS2>	V2 only	AVAIL = Available, but not yet requested PEND = Requested, but not yet available ACTIVE = In use REJECTED = Rejected
<REASON>	V2 only	This element is only relevant if <SVCSTATUS2>REJECTED is specified, A-255
</BANKACCTINFO>		

11.3.4 Credit Card Account Information <CCACCTINFO>

OFX uses the credit card account information aggregate to download account information from an FI. It includes credit card number specification in <CCACCTFROM> as well as the status of the service.

Tag	Version	Description
<CCACCTINFO>		Credit-card-account-information aggregate
<CCACCTFROM>		Credit-card-account-from aggregate
</CCACCTFROM>		
<SUPTXDL>		Y if account supports transaction detail downloads, N if it is balance-only, <i>Boolean</i>
<XFERSRC>		Y if account is enabled as a source for an intrabank or interbank transfer, <i>Boolean</i>
<XFERDEST>		Y if account is enabled as a destination for an intrabank or interbank transfer, <i>Boolean</i>
<SVCSTATUS>	V1 only	Status of the account AVAIL = Available, but not yet requested PEND = Requested, but not yet available ACTIVE = In use
<SVCSTATUS2>	V2 only	AVAIL = Available, but not yet requested PEND = Requested, but not yet available ACTIVE = In use REJECTED = Rejected
<REASON>	V2 only	This element is only relevant if <SVCSTATUS2>REJECTED is specified, A-255
</CCACCTINFO>		

11.3.5 Transfer Information <XFERINFO>

Many of the transfer requests and responses use an <XFERINFO> aggregate. This aggregate identifies accounts that are part of the transfer, amount of money to be transferred, and the date of the transfer.

The <DTDUE> in a response may have been adjusted by a server. For example, the server may adjust <DTDUE> to comply with non-processing days. If a client sends a request to make a transfer on July 4 and July 4 happens to be a non-processing day, the <DTDUE> in the response may be July 4 (because the server hasn't adjusted it yet), July 5 (because this server rolls dates forward), or some other date. For this reason, a client should pay attention to the <DTDUE> in the response.

Tag	Version	Description
<XFERINFO> Account-from options. Choose either <BANKACCTFROM> or <CCACCTFROM> .		Transfer-information aggregate
<BANKACCTFROM> </BANKACCTFROM> -or- <CCACCTFROM> </CCACCTFROM>		Account-from aggregate, see section 11.3.1 Credit-card-account-from aggregate, see section 11.3.2
Account-to options. Choose either <BANKACCTTO> or <CCACCTTO> .		
<BANKACCTTO> </BANKACCTTO> -or- <CCACCTTO> </CCACCTTO>		Account-to aggregate, see section 11.3.1 Credit-card-account-to aggregate, see section 11.3.2
<TRNAMT> <DTDUE> <DTAVAIL>	V2 only	Amount of the transfer, <i>amount</i> This amount should be specified as a positive number. Date that the transfer is to be sent. If the client does not specify <DTDUE> , the transfer occurs as soon as possible. <DTDUE> is required for scheduled or repeating transfers, <i>datetime</i> The value date when the funds must be available in BANKACCTTO . This can potentially be set independently of DTDUE , <i>datetime</i> Note that for some countries (notably Italy), DTAVAIL can be a date in the past. This field is only supported by the server if the SUPPORTDTAVAIL element of the Intrabank transfer profile is true (see section 11.13.2.2). This element is not used in the US.
<MEMO2> </XFERINFO>	V2 only	Associated text content for the transfer, <i>memo2</i>

11.3.6 Transfer Processing Status <XFERPRCSTS>

The Transfer Processing Status aggregate contains the current processing status for a transfer. This aggregate is intended to describe status changes to the associated transfer after creation. The interpretation of the date value depends on the value of <XFERPRCCODE>.

<i>Tag</i>	<i>Description</i>
<XFERPRCSTS>	Transfer processing status aggregate
<XFERPRCCODE>	See section 11.3.6.1
<DTXFERPRC>	Transfer processing date; value depends on <XFERPRCCODE>
</XFERPRCSTS>	

11.3.6.1 Transfer Processing Status Values <XFERPRCCODE>

<i>Value</i>	<i>Description</i>
WILLPROCESSION	Will be processed on <DTXFERPRC>
POSTEDON	Posted on <DTXFERPRC>
NOFUNDSON	Funds not available to make transfer on <DTXFERPRC>
CANCELEDON	User canceled payment on <DTXFERPRC>
FAILEDON	Unable to make transfer for unspecified reasons on <DTXFERPRC>

11.4 Downloading Transactions and Balances

Statement download allows a customer to receive transactions and balances that are typically part of a regular paper statement. Clients can retrieve transactions and balances on a daily basis if they wish. Coupled with the information returned by statement closing information request (see section 11.5), a client can construct an “electronic statement” that contains all of the information that appears on a regular paper statement.

Clients typically allow customers to view these transactions and guide customers through a process of updating their account registers based on the downloaded transactions. By using transaction IDs supplied by financial institutions, OFX makes it possible for clients to ensure that a server downloads each transaction only once. The request also contains starting and ending dates to limit the amount of downloaded data. Clients can remember the last date they received data and use it as the starting date in the next request.

The messages in this chapter are appropriate for checking, savings, money market, credit card, and line of credit accounts. Investment statement download is a superset of bank statement download. [Chapter 13, "Investments,"](#) describes the messages specific to investment statement download.

Statement download requires the client to designate an account for the download, and to indicate if the server should download transactions and / or balances. If the client wishes to download transactions, it can specify a date range that the transactions fall within.

The server returns transactions that match the date range (if the client specifies one), and balance information for the account.

<i>Client Sends</i>	<i>Server Responds</i>
Account information	
Include transactions?	
Date range	
	Transactions
	Cycle-ending information

11.4.1 Bank Statement Download

A client can request a download of balances separately from transaction detail. The server downloads transactions only if the <INCTRAN> aggregate is present and the <INCLUDE> flag is set to Y. The current ledger balance (and balance date) are always downloaded.

If a statement download request does not contain <DTSTART> or <DTEND> elements but does request transactions and no transactions are found on the server, the response may or may not include a <BANKTRANLIST> without any <STMTRN> aggregates. The server should leave out the useless <BANKTRANLIST>.

You can use the <STMTRQ> ... <STMTRS> request and response pair to download transactions and balances for checking, savings, money market, and line of credit accounts. Section [11.4.2](#) describes download for credit card accounts.

Clients and servers should interpret <DTSTART> and <DTEND> as described in [Chapter 3, "Common Aggregates, Elements, and Data Types."](#)

11.4.1.1 Request <STMTRQ>

The <STMTRQ> request must appear within a <STMTRNRQ> transaction wrapper.

Tag	Description
<STMTRQ>	Statement-request aggregate
<BANKACCTFROM>	Bank-account-from aggregate, see section 11.3.1
</BANKACCTFROM>	
<INCTRAN>	Include-transactions aggregate
<DTSTART>	Start date of statement requested, <i>datetime</i>
<DTEND>	End date of statement requested, <i>datetime</i>
<INCLUDE>	Include transactions flag, <i>Boolean</i>
</INCTRAN>	
</STMTRQ>	

11.4.1.2 Response <STMTRS>

A statement response comprises elements supplying various balances, plus zero or more <STMTRN> aggregates, each describing one statement transaction.

The <STMTRS> response must appear within a <STMTRNRS> transaction wrapper.

See [Chapter 3, "Common Aggregates, Elements, and Data Types,"](#) for size and type information for common elements (such as currency values).

Tag	Description
<STMTRS>	Statement-response aggregate
<CURDEF>	Default currency for the statement, <i>currsymbol</i>
<BANKACCTFROM>	Account-from aggregate, see section 11.3.1
</BANKACCTFROM>	
<BANKTRANLIST>	Statement-transaction-data aggregate
<DTSTART>	Start date for transaction data, <i>date</i>
<DTEND>	Value that client should send in next <DTSTART> request to ensure that it does not miss any transactions, <i>date</i>
<STMTRN>	Opening tag for each statement transaction (0 or more), see section 11.4.3
</STMTRN>	End tag for each statement transaction
</BANKTRANLIST>	
<LEDGERBAL>	Ledger balance aggregate
<BALAMT>	Ledger balance amount, <i>amount</i>
<DTASOF>	Balance date, <i>datetime</i>
</LEDGERBAL>	
<AVAILBAL>	Available balance aggregate
<BALAMT>	Available balance amount, <i>amount</i>
<DTASOF>	Balance date, <i>datetime</i>
</AVAILBAL>	
<MKTGINFO>	Marketing information (at most 1), A-360
</STMTRS>	

11.4.1.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success
2000	General error (ERROR)
2002	General account error (ERROR)
2003	Account not found (ERROR)
2004	Account closed (ERROR)
2005	Account not authorized (ERROR)
2019	Duplicate request (ERROR)
2020	Invalid date (ERROR)
2027	Invalid date range (ERROR)

11.4.2 Credit Card Statement Download

The credit card download request is semantically identical to the bank statement download request. However, the <CCSTMTRQ> aggregate contains the credit card request, not the <STMTRQ> aggregate.

If a statement download request does not contain <DTSTART> or <DTEND> elements but does request transactions and no transactions are found on the server, the response may or may not include a <BANKTRANLIST> without any <STMTRN> aggregates. The server should leave out the useless <BANKTRANLIST>.

11.4.2.1 Request <CCSTMTRQ>

The <CCSTMTRQ> request must appear within a <CCSTMTRNRQ> transaction wrapper.

Tag	Description
<CCSTMTRQ>	Credit-card-download-request aggregate
<CCACCTFROM>	Credit-card-account-from aggregate
<ACCTID>	Account number, A-22
<ACCTKEY>	Checksum for international banks, A-22
</CCACCTFROM>	
<INCTRAN>	Include transactions
<DTSTART>	Start date of statement requested, <i>datetime</i>
<DTEND>	Ending date of statement requested, <i>datetime</i>
<INCLUDE>	Include transactions flag, <i>Boolean</i>
</INCTRAN>	
</CCSTMTRQ>	

11.4.2.2 Response <CCSTMTRS>

The credit card download response is semantically identical to the bank statement download response. However, the <CCSTMTRS> aggregate contains the credit card response, not the <STMTRS> aggregate.

The <CCSTMTRS> response must appear within a <CCSTMTRNRS> transaction wrapper.

Tag	Description
<CCSTMTRS>	Credit-card-download-response aggregate
<CURDEF>	Default currency for the statement, <i>currsymbol</i>
<CCACCTFROM>	Account from aggregate, see section 11.3.2
</CCACCTFROM>	
<BANKTRANLIST>	Opening tag for statement transaction data
<DTSTART>	Start date for transaction data, <i>date</i>
<DTEND>	Value client should send in next <DTSTART> request to ensure that it does not miss any transactions, <i>date</i>
<STMTRN>	Opening tag for each statement transaction (0 or more), see section 11.4.3
</STMTRN>	
</BANKTRANLIST>	
<LEDGERBAL>	Ledger-balance aggregate
<BALAMT>	Ledger balance amount, <i>amount</i>
<DTASOF>	Balance date, <i>datetime</i>
</LEDGERBAL>	
<AVAILBAL>	Available balance aggregate
<BALAMT>	Available balance amount, <i>amount</i>
<DTASOF>	Balance date, <i>datetime</i>
</AVAILBAL>	
<MKTGINFO>	Marketing information (at most 1), A-360
</CCSTMTRS>	

11.4.2.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success
2001	Invalid account (ERROR)
2002	General account error (ERROR)
2003	Account not found (ERROR)
2004	Account closed (ERROR)
2005	Account not authorized (ERROR)
2019	Duplicate request (ERROR)
2020	Invalid date (ERROR)
2027	Invalid date range (ERROR)

11.4.3 Statement Transaction <STMTTRN>

A <STMTTRN> aggregate describes a single transaction. It identifies the type of the transaction and the date it was posted. The aggregate can also provide additional information to help the customer recognize the transaction: check number, payee name, and memo. The transaction can have a Standard Industrial Code that a client can use to categorize the transaction.

Each <STMTTRN> contains an <FITID> that the client uses to detect whether the server has previously downloaded the transaction.

Transaction amounts are signed from the perspective of the customer. For example, a credit card payment is positive while a credit card purchase is negative.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<STMTTRN>		Statement-transaction aggregate
<TRNTYPE>		Transaction type, see section 11.4.3.1 for possible values. This element does not change the effect of the transaction upon the balance (increases and decreases are indicated by the sign of the <TRNAMT>).
<DTPOSTED>		Date transaction was posted to account, <i>datetime</i>
<DTUSER>		Date user initiated transaction, if known, <i>datetime</i>
<DTAVAIL>		Date funds are available (<u>value date</u>), <i>datetime</i>
<TRNAMT>		Amount of transaction, <i>amount</i>

Tag	Version	Description
<FITID>		Transaction ID issued by financial institution. Used to detect duplicate downloads, <i>FITID</i>
<CORRECTFITID>		If present, the FITID of a previously sent transaction that is corrected by this record. This transaction replaces or deletes the transaction that it corrects, based on the value of <CORRECTACTION> below, <i>FITID</i>
<CORRECTACTION>		Actions can be REPLACE or DELETE. REPLACE replaces the transaction referenced by CORRECTFITID; DELETE deletes it.
<SPNAME>	1.6 add	Service provider name. Used to limit the scope of <SRVRTID> or <SRVRTID2> when matching transactions with OFX requests. This value is normally obtained from <MSGSETCORE> in the FI Profile for the relevant service provider. If <SPNAME> is specified, <SRVRTID> or <SRVRTID2> is required. A-32 Note: This optional element has no default. The server should include <SPNAME> whenever a <SRVRTID> or <SRVRTID2> is available in the download. The client cannot assume that a <SRVRTID> or <SRVRTID2> provided without <SPNAME> will match a single OFX request. The user might need to be consulted when a <SRVRTID> or <SRVRTID2> has multiple potential matches.
<SRVRTID>	V1 only	Server assigned transaction ID; used for transactions initiated by client, such as payment or funds transfer. <i>SRVRTID</i>
<SRVRTID2>	V2 only	Server assigned transaction ID; used for transactions initiated by client, such as payment or funds transfer. <i>SRVRTID2</i>
<CHECKNUM>		Check (or other reference) number, A-12
<REFNUM>		Reference number that uniquely identifies the transaction. Can be used in addition to or instead of a <CHECKNUM>, A-32
<SIC>		Standard Industrial Code, N-6
<PAYEEID>	V1 only	Payee identifier if available, A-12
<PAYEEID2>	V2 only	Payee identifier if available, <i>SRVRTID2</i>
Payee options. Choose either <NAME> or <PAYEE>.		
<NAME>		Name of payee or description of transaction, A-32 Note: Provide NAME or PAYEE, not both

Tag	Version	Description
-or- <PAYEE> </PAYEE> <PAYEE2> </PAYEE2>	V1 only V2 only	Payee aggregate, see section 12.5.3.1 Payee aggregate, see section 12.5.3.1
<i>Account-to options. Choose either <BANKACCTTO> or <CCACCTTO>.</i>		
<BANKACCTTO> </BANKACCTTO> -or- <CCACCTTO> </CCACCTTO>		If this was a transfer to an account and the account information is available, see section 11.3.1
<MEMO> <MEMO2>	V1 only V2 only	Extra information (not in <NAME>), <i>MEMO</i> Extra information (not in <NAME>), <i>MEMO2</i>
<i>Currency options. Choose either <CURRENCY> or <ORIGCURRENCY>.</i>		
<CURRENCY> </CURRENCY> -or- <ORIGCURRENCY> </ORIGCURRENCY>		Currency, if different from CURDEF
</STMTTRN>		

11.4.3.1 Transaction types used in <TRNTYPE>

Transfers generated from a model are treated identically to individually requested transfers by OFX. Therefore, they should have the transaction types listed below once they are processed. Transfers initiated out of band with respect to OFX should also be handled in this fashion when they appear in a statement download.

<i>Type</i>	<i>Description</i>
CREDIT	Generic credit
DEBIT	Generic debit
INT	Interest earned or paid Note: Depends on signage of amount
DIV	Dividend
FEE	FI fee
SRVCHG	Service charge
DEP	Deposit
ATM	ATM debit or credit Note: Depends on signage of amount
POS	Point of sale debit or credit Note: Depends on signage of amount
XFER	Transfer
CHECK	Check
PAYMENT	Electronic payment
CASH	Cash withdrawal
DIRECTDEP	Direct deposit
DIRECTDEBIT	Merchant initiated debit
REPEATPMT	Repeating payment/standing order
OTHER	Other

11.5 Statement Closing Information

OFX provides a way for customers to receive closing statement information that typically appears as part of a paper statement. This information includes opening and closing dates and balances for a statement period, as well as a detailed breakdown of debits, credits, fees, and interest that are usually part of a paper statement. In addition to this information, clients receive a date range for transactions that correspond to the closing statement. Clients might wish to use this date range to retrieve transactions through statement download in order to present the user with an “electronic” statement.

To request statement information, the client is **REQUIRED** to designate an account for the download. The client can also specify a date range to limit the number of closing information aggregates that the server returns. If the client does not specify a date range, the server returns as many closing information aggregates as it can.

<i>Client Sends</i>	<i>Server Responds</i>
Account Information Date range	Cycle-ending information (0 or more)

11.5.1 Statement Closing Download

You can use the <STMTENDRQ> ...<STMTENDRS> request and response pair to download statement closing information for checking, savings, money market, and line of credit accounts. Section [11.5.3](#) describes download for credit card accounts.

11.5.1.1 Request <STMTENDRQ>

The <STMTENDRQ> request must appear within a <STMTENDTRNRQ> transaction wrapper.

<i>Tag</i>	<i>Description</i>
<STMTENDRQ>	Closing-statement-request aggregate
<BANKACCTFROM>	Bank-account-from aggregate
</BANKACCTFROM>	
<DTSTART>	Start date for statement closing information, <i>datetime</i>
<DTEND>	End date of statement closing information, <i>datetime</i>
</STMTENDRQ>	

11.5.1.2 Response <STMTENDRS>

The <STMTENDRS> response must appear within a <STMTENDTRNRS> transaction wrapper.

<i>Tag</i>	<i>Description</i>
<STMTENDRS>	Closing-statement-response aggregate
<CURDEF>	Default currency used for closing information, <i>currsymbol</i>
<BANKACCTFROM>	Account from aggregate, see section 11.3.1
</BANKACCTFROM>	
<CLOSING>	Statement information (0 or more), see section 11.5.2
</CLOSING>	
</STMTENDRS>	

11.5.1.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success
2000	General error (ERROR)
2002	General account error (ERROR)
2003	Account not found (ERROR)
2004	Account closed (ERROR)
2005	Account not authorized (ERROR)
2019	Duplicate request (ERROR)
2020	Invalid date (ERROR)
2027	Invalid date range (ERROR)

11.5.2 Non-Credit Card Statement <CLOSING>

A checking, savings, or money market account uses the <CLOSING> aggregate to describe statement closing information.

The <FITID> provides a way for the client to distinguish one closing statement from another.

For each <CLOSING> aggregate returned, clients can retrieve corresponding transactions by using <DTPOSTSTART> and <DTPOSTEND> as <DTSTART> and <DTEND> in a <STMTRQ> request.

Tag	Description
<CLOSING>	Non-credit-card-account-types aggregate
<FITID>	Unique identifier for this statement, <i>FITID</i>
<DTOPEN>	Opening statement date, <i>date</i>
<DTCLOSE>	Closing statement date, <i>date</i>
<DTNEXT>	Closing date of next statement, <i>date</i>
<BALOPEN>	Opening statement balance, <i>amount</i>
<BALCLOSE>	Closing statement balance, <i>amount</i>
<BALMIN>	Minimum balance in statement period, <i>amount</i>
<DEPANDCREDIT>	Total of deposits and credits, including interest, <i>amount</i>
<CHKANDDEB>	Total of checks and debits, including fees, <i>amount</i>
<TOTALFEES>	Total of all fees, <i>amount</i>
<TOTALINT>	Total of all interest, <i>amount</i>
<DTPOSTSTART>	Start date of transaction data for this statement, <i>date</i> A client should be able to use this date in a <STMTRQ> to request transactions that match this statement.
<DTPOSTEND>	End date of transaction data for this statement, <i>date</i> A client should be able to use this date in a <STMTRQ> to request transactions that match this statement.
<MKTGINFO>	Marketing information (at most 1), A-360
<i>Currency options. Choose either <CURRENCY> or <ORIGCURRENCY></i>	
<CURRENCY> </CURRENCY> -or- <ORIGCURRENCY> </ORIGCURRENCY>	Currency, if different from CURDEF
</CLOSING>	

11.5.3 Credit Card Statement Closing Request <CCSTMTENDRQ>

The credit card statement closing request is semantically identical to the bank statement closing request. However, the <CCSTMTENDRQ> aggregate contains the credit card request, not the <STMTENDRQ> aggregate.

The <CCSTMTENDRQ> request must appear within a <CCSTMTENDTRNRQ> transaction wrapper.

Tag	Description
<CCSTMTENDRQ>	Credit-card-closing-statement-request aggregate
<CCACCTFROM>	Credit-card-account-from aggregate
</CCACCTFROM>	
<DTSTART>	Start date for statement closing information, <i>datetime</i>
<DTEND>	End date of statement closing information, <i>datetime</i>
</CCSTMTENDRQ>	

11.5.4 Credit Card Statement Closing Response <CCSTMTENDRS>

The credit card statement closing response is semantically identical to the bank statement closing response. However, the <CCSTMTENDRS> aggregate contains the credit card response, not the <STMTENDRS> aggregate.

The <CCSTMTENDRS> response must appear within a <CCSTMTENDTRNRS> transaction wrapper.

Tag	Description
<CCSTMTENDRS>	Credit-card-closing-statement-response aggregate
<CURDEF>	Default currency for closing information, <i>currsymbol</i>
<CCACCTFROM>	Account from aggregate, see section 11.3.2
</CCACCTFROM>	
<CCCLOSING>	Statement information (0 or more). See section 11.5.4.2
</CCCLOSING>	
</CCSTMTENDRS>	

11.5.4.1 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success
2000	General error (ERROR)
2002	General account error (ERROR)
2003	Account not found (ERROR)
2004	Account closed (ERROR)
2005	Account not authorized (ERROR)
2019	Duplicate request (ERROR)
2020	Invalid date (ERROR)
2027	Invalid date range (ERROR)

11.5.4.2 Credit Card Statement <CCCLOSING>

A credit card account uses the <CCCLOSING> aggregate to describe statement closing information.

The <FITID> provides a way for the client to distinguish one closing statement from another.

For each <CCCLOSING> returned, clients should be able to retrieve corresponding transactions by using <DTPOSTSTART> and <DTPOSTEND> as <DTSTART> and <DTEND> in a <CCSTMTRQ> request.

Tag	Description
<CCCLOSING>	Credit-card-statement-information aggregate
<FITID>	Unique identifier for this statement, <i>FITID</i>
<DTOPEN>	Opening statement date, <i>date</i>
<DTCLOSE>	Closing statement date, <i>date</i>
<DTNEXT>	Closing date of next statement, <i>date</i>
<BALOPEN>	Opening statement balance, <i>amount</i>
<BALCLOSE>	Closing statement balance, <i>amount</i>
<DTPMTDUE>	Payment due date, <i>date</i>
<MINPMTDUE>	Minimum amount due, <i>amount</i>
<FINCHG>	Finance charges, <i>amount</i>
<PAYANDCREDIT>	Total of payments and credits, <i>amount</i>
<PURANDADV>	Total of purchases and cash advances, <i>amount</i>
<DEBADJ>	Debit adjustments, <i>amount</i>
<CREDITLIMIT>	Current credit limit, <i>amount</i>
<DTPOSTSTART>	Start date of transaction data for this statement, <i>date</i> A client should be able to use this date in a <CCSTMTRQ> to request transactions that match this statement.
<DTPOSTEND>	End date of transaction data for this statement, <i>date</i> A client should be able to use this date in a <CCSTMTRQ> to request transactions that match this statement.
<MKTGINFO>	Marketing information (at most 1), A-360
<i>Currency options. Choose either <CURRENCY> or <ORIGCURRENCY>.</i>	
<CURRENCY> </CURRENCY> -or- <ORIGCURRENCY> </ORIGCURRENCY>	Currency, if different from CURDEF
</CCCLOSING>	

11.6 Stop Check

OFX supports a request to issue a stop payment for one or more outstanding checks. The stop request can be for a single check or for a range of checks. There must be one request for each check or range of checks the user wants to stop.

When stopping a single check, the client can provide a payee name and optionally an amount instead of a check number to describe the check to stop. Not all servers can support this behavior.

Examples:

- Stop check 22 – one request
- Stop check to “Acme Lighting” – one request
- Stop checks 200-224 – one request
- Stop checks 275-280, 283 – two requests (first stops 275-280, the next stops 283)

<i>Client Sends</i>	<i>Server Responds</i>
Account information	
Check number(s) to stop	
-or- Check description	
	Status for each check

11.6.1 Stop Check Add

Stop Check Add is subject to synchronization.

11.6.1.1 Request <STPCHKRQ>

The <STPCHKRQ> request must appear within a <STPCHKTRNRQ> transaction wrapper.

Tag	Description
<STPCHKRQ> <BANKACCTFROM> </BANKACCTFROM> <i>Check options. Choose either <CHKRANGE> or <CHKDESC>.</i>	Stop-check-request aggregate Account-from aggregate, see section 11.3.1
<CHKRANGE> </CHKRANGE> -or- <CHKDESC> </CHKDESC>	Check range aggregate, see section 11.6.1.1.1 Check description aggregate, see section 11.6.1.1.2
</STPCHKRQ>	

11.6.1.1.1 Check Range <CHKRANGE>

Tag	Description
<CHKRANGE> <CHKNUMSTART> <CHKNUMEND>	Check-range aggregate Start check number to cancel, A-12 Ending check number to cancel; omit if only one check is to be stopped, A-12
</CHKRANGE>	

11.6.1.1.2 Check Description <CHKDESC>

A check description must include a payee name or description. It can also include a check number, the date the user wrote the check, and a transaction amount.

Tag	Description
<CHKDESC>	Check description aggregate
<NAME>	Payee name or description, A-32
<CHECKNUM>	Check number, A-12
<DTUSER>	Date on check, <i>datetime</i>
<TRNAMT>	Amount, <i>amount</i>
</CHKDESC>	

11.6.1.2 Response <STPCHKRS>

Consistent with all responses, the stop check response contains a global status that describes whether the response could be delivered. If the server provides a response, it returns a <STPCHKNUM> aggregate for each check for which the client requested a stop payment. Status code 10000 should be returned if the stop check request is in process; a subsequent synchronization should obtain an updated response with a final status.

The <STPCHKRS> response must appear within a <STPCHKTRNRS> transaction wrapper.

Tag	Description
<STPCHKRS>	Stop-check-response aggregate
<CURDEF>	Default currency for stop check response, <i>currsymbol</i>
<BANKACCTFROM>	Account-from aggregate, see section 11.3.1
</BANKACCTFROM>	
<STPCHKNUM>	Stopped check aggregate (1 or more), see section 11.6.1.2.1
</STPCHKNUM>	
<FEE>	Fee for stop check, <i>amount</i>
<FEEMSG>	Description of fee, A-80
</STPCHKRS>	

11.6.1.2.1 Stopped Check <STPCHKNUM>

This aggregate contains a status code that indicates whether or not a specific check was canceled.

Tag	Description
<STPCHKNUM>	Stopped-check-item aggregate
<CHECKNUM>	Check number, A-12
<NAME>	Payee name or description, A-32
<DTUSER>	Date on check, <i>datetime</i>
<TRNAMT>	Amount, <i>amount</i>
<CHKSTATUS>	Status code for individual stop check request 0 = OK 1 = rejected 100 = check not found 101 = check already posted
<CHKERROR>	Further textual explanation, A-255
Currency options. Choose either <CURRENCY> or <ORIGCURRENCY>.	
<CURRENCY> </CURRENCY> -or- <ORIGCURRENCY> </ORIGCURRENCY>	Currency, if different from CURDEF
</STPCHKNUM>	

11.6.2 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success
2000	General error (ERROR)
2002	General account error (ERROR)
2003	Account not found (ERROR)
2004	Account closed (ERROR)
2005	Account not authorized (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10000	Stop check in process (INFO)
10500	Too many checks to process (ERROR)

11.7 Intrabank Funds Transfer

OFX supports transferring funds between two accounts at the same financial institution. Funds transfers in OFX can be immediate or scheduled. Scheduled transfers can repeat at specified intervals.

Financial institutions can choose to support:

- ◆ Immediate transfers
- ◆ Immediate and scheduled transfers
- ◆ Immediate, scheduled, and recurring transfers

Recurring transfers require support for scheduled transfers.

In general, an OFX server may not choose which transactions to support unless the profile can be used to indicate to the client that a transaction is not supported. However, immediate intrabank funds transfers usually cannot be modified or canceled, so a server that does not support scheduled transfers may return an error code on any request for cancel or modify. A preferred approach would be to return status code 2016, which means the transfer may not be modified or canceled because it is already committed. (An immediate transfer may not actually commit until the end of the business day. For more information, see the discussion on the support of INTRASYNCRQ in section [11.12.2](#).)

After a transfer has executed, the server can either issue a transfer modification response in the sync or it can do nothing. In the latter case, it would be up to the client to get status from a statement download. If a transfer fails, it is recommended, but not required, that a transfer modification response with the appropriate XFERPRCCODE be sent in the sync.

In general, all Intrabank Funds Transfer requests are subject to synchronization. The only exception occurs when the request is for an immediate transfer and the server is able to successfully perform the transfer in real time. In that case, the server may choose whether or not the transfer affects the sync history. After receiving an immediate response indicating that a transfer took place in real time, the client must not expect the relevant token to change or to receive information about that transfer in a later sync response.

Note: Servers choosing to ignore real time immediate transfers in the sync history force additional clients to wait until the transfer appears in a statement download for information about the transfer.

11.7.1 Intrabank Funds Transfer Addition

The Intrabank Funds Transfer Add request provides a way for a client to set up a single transfer. The request designates source and destination accounts and the amount of the transfer. The client must provide a date if it has scheduled the transfer. Immediate funds transfers cannot be modified or canceled.

<i>Client Sends</i>	<i>Server Responds</i>
Source account	Server ID for the transfer Source account Destination account Amount Expected / actual posting date
Destination account	
Amount	
Date of transfer (optional)	

Intrabank Funds Transfer Add is subject to synchronization.

11.7.1.1 Request <INTRARQ>

The <INTRARQ> request must appear within an <INTRATRNRQ> transaction wrapper.

<i>Tag</i>	<i>Description</i>
<INTRARQ>	Intrabank-transfer-request aggregate
<XFERINFO>	Transfer information aggregate, see section 11.3.5
</XFERINFO>	
</INTRARQ>	

11.7.1.2 Response <INTRARS>

A server cannot, in all cases, provide complete confirmation for the transfer. The server can confirm only that it received the transfer instruction; and possibly whether it validated the accounts, amount, and date specified in the transfer. For any transfer where the client does not know the status at the time of the response, a server should confirm that it accepted the instruction and indicate the expected posting date of the transfer. A client can pick up the

confirmation at a later date through a synchronization request. Servers should inform clients of any errors found while processing this transaction using the <STATUS> aggregate. A response containing <STATUS><CODE>0 and <XFERPRCSTS><XFERPRCCODE>FAILEDON should be avoided for problems such as an invalid account or amount.

If the request is for an immediate transfer and the server can perform the transfer in real time, the server should indicate whether the transfer succeeded and should return the date of the transfer in <DTPOSTED>. In this case, synchronization is not required.

The <INTRARS> response must appear within an <INTRATRNR> transaction wrapper.

Tag	Version	Description
<INTRARS>		Intrabank-transfer-response aggregate
<CURDEF>		Default currency for the intrabank transfer response, <i>currsymbol</i>
<SRVRTID>	V1 only	Server ID for this transfer, <i>SRVRTID</i>
<SRVRTID2>	V2 only	Server ID for this transfer, <i>SRVRTID2</i>
<XFERINFO>		Transfer information aggregate, see section 11.3.5
</XFERINFO>		
Transfer-date options. Choose either <DTXFERPRJ> or <DTPOSTED>		
<DTXFERPRJ>		Projected date of the transfer; response can contain either a <DTXFERPRJ> or a <DTPOSTED> but not both; <i>datetime</i>
-or-		
<DTPOSTED>		Actual date of the transfer, <i>datetime</i>
<RECSRVRTID>	V1 only	If the response is generated by a recurring transfer model, this ID references it, see section 11.10 , <i>SRVRTID</i>
<RECSRVRTID2>	V2 only	If the response is generated by a recurring transfer model, this ID references it, see section 11.10 , <i>SRVRTID2</i>
<XFERPRCSTS>		Transfer-processing status, see section 11.3.6
</XFERPRCSTS>		
</INTRARS>		

Note: The server can deliver this response to a client immediately after the request is made (for an immediate or one-time scheduled transfer). The server should also return this response for any transfers that were generated by a model.

11.7.1.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2006	Source account not found (ERROR)
2007	Source account closed (ERROR)
2008	Source account not authorized (ERROR)
2009	Destination account not found (ERROR)
2010	Destination account closed (ERROR)
2011	Destination account not authorized (ERROR)
2012	Invalid amount (ERROR)
2014	Date too soon (ERROR)
2015	Date too far in future (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10504	Insufficient funds (ERROR)

11.7.2 Intrabank Funds Transfer Modification

The client sends a Transfer Modification request to modify a scheduled transfer. Immediate transfers cannot be modified, so this request should only be used for scheduled transfers. Once created and retrieved by the customer, spawned transfers are almost identical to customer-created transfers. (The exception is when a spawned transfer is modified or cancelled due to a recurring modification or cancellation request.) As with ordinary transfers, you can cancel or modify transactions individually. When modifying a transfer, the client must specify all of the elements and aggregates within the <XFERINFO> aggregate that were specified when the transfer was created, not just the elements and aggregates that the client wants to modify. <SRVRTID> specifies the transfer the user wants to modify. Some servers cannot support the modification of certain values. Servers must indicate this by returning status code 10505 when the client requests an unsupported modification. Clients must not change <BANKACCTFROM> or <CCACCTFROM> in a funds transfer modification.

Intrabank Funds Transfer Modification is subject to synchronization.

11.7.2.1 Request <INTRAMODRQ>

The <INTRAMODRQ> request must appear within an <INTRATRNRQ> transaction wrapper.

Tag	Version	Description
<INTRAMODRQ>		Modification-request aggregate
<SRVRTID>	V1 only	ID assigned by the server to the transfer being modified, <i>SRVRTID</i>
<SRVRTID2>	V2 only	ID assigned by the server to the transfer being modified, <i>SRVRTID2</i>
<XFERINFO>		Transfer information aggregate, see section 11.3.5
</XFERINFO>		
</INTRAMODRQ>		

11.7.2.2 Response <INTRAMODRS>

This response normally just echoes the values passed by the client. However, if the status of a scheduled transfer changes in any way, clients should expect to receive modification responses when they synchronize with the server. For example, when a server completes a transfer, the status of the transfer goes from *pending* to *posted*. Clients should expect servers to notify them of this status change.

The <INTRAMODRS> response must appear within an <INTRATRNRS> transaction wrapper.

Tag	Version	Description
<INTRAMODRS>		Modification-response aggregate
<SRVRTID>	V1 only	ID assigned by the server to the transfer being modified, <i>SRVRTID</i>
<SRVRTID2>	V2 only	ID assigned by the server to the transfer being modified, <i>SRVRTID2</i>
<XFERINFO>		Transfer information aggregate, see section 11.3.5
</XFERINFO>		
<XFERPRCSTS>		Transfer processing status, see section 11.3.6
</XFERPRCSTS>		
</INTRAMODRS>		

11.7.2.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2006	Source account not found (ERROR)
2007	Source account closed (ERROR)
2008	Source account not authorized (ERROR)
2009	Destination account not found (ERROR)
2010	Destination account closed (ERROR)
2011	Destination account not authorized (ERROR)
2012	Invalid amount (ERROR)
2014	Date too soon (ERROR)
2015	Date too far in future (ERROR)
2016	Transaction already committed (ERROR)
2017	Already canceled (ERROR)
2018	Unknown server ID (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10500	Too many checks to process (ERROR)
10505	Cannot modify element (ERROR)
10514	Transaction already processed (ERROR)

11.7.3 Intrabank Funds Transfer Cancellation

The client sends a Transfer Cancellation request to cancel a scheduled transfer, where <SRVRTID> identifies the transfer. Immediate transfers cannot be canceled, so this request should be used only for scheduled transfers.

Intrabank Funds Transfer Cancellation is subject to synchronization.

11.7.3.1 Request <INTRACANRQ>

The <INTRACANRQ> request must appear within an <INTRATRNRQ> transaction wrapper.

Tag	Version	Description
<INTRACANRQ>		Transfer-cancellation-request aggregate
<SRVRTID>	V1 only	ID of the transfer the user wants to cancel. The server must have previously assigned this ID to a transfer. <i>SRVRTID</i>
<SRVRTID2>	V2 only	ID of the transfer the user wants to cancel. The server must have previously assigned this ID to a transfer. <i>SRVRTID2</i>
</INTRACANRQ>		

11.7.3.2 Response <INTRACANRS>

The <INTRACANRS> response must appear within an <INTRATRNRS> transaction wrapper.

Tag	Version	Description
<INTRACANRS>		Transfer-cancellation-response aggregate
<SRVRTID>	V1 only	ID of the transfer the user wants to cancel. The server must have previously assigned this ID to a transfer. <i>SRVRTID</i>
<SRVRTID2>	V2 only	ID of the transfer the user wants to cancel. The server must have previously assigned this ID to a transfer. <i>SRVRTID2</i>
</INTRACANRS>		

11.7.3.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2016	Transaction already committed (ERROR)
2017	Already canceled (ERROR)
2018	Unknown server ID (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10514	Transaction already processed (ERROR)

11.7.3.4 DTDUE, DTPOSTED, and DTXFERPRJ in Immediate Transfers

The following is a list of what might be returned in an immediate mode transfer response. The interpretation of each response is provided. All responses referenced in this section are immediate and describe success conditions. That is, we are not attempting to describe the INTRARS aggregates that may be returned within a INTRASYNCRS response. Further, successful INTRARS aggregates for immediate transfers are not expected to appear in lite synchronization INTRASYNCRS responses.

- ◆ **DTDUE and INTRARQ**
DTDUE should not be present if the client is requesting an immediate mode transfer. If it is, then this is a client error, and will be treated by the server as if the client were attempting to create a scheduled transfer.
- ◆ **DTDUE, DTPOSTED, and DTXFERPRJ NOT returned in INTRARS**
The client should interpret this from the server as indicating that the immediate transfer request was processed in real-time.
- ◆ **DTDUE only returned**
DTDUE should not be present in the response to a request for an immediate transfer. If it were present, the XFERINFO aggregate would not match that found in the request.
- ◆ **DTPOSTED only returned**
If this is not equal to today's date and an earlier time than "now", then this is a server error. Otherwise, the client should interpret this as confirmation from the server that the request was processed in real-time.
- ◆ **DTXFERPRJ only returned**
The client should interpret this as indication that the transfer request will be completed by the specified date. The client may receive an INTRAMODRS or INTRARS with updated information about this transfer when it is actually processed. That future INTRARS or INTRAMODRS will contain the DTPOSTED to reflect when the transfer occurred. This response is not required for successful transfers processed at the originally specified projected date and time.

11.8 Interbank Funds Transfer

The Interbank Funds Transfer Add request provides a way for a client to set up a single transfer between accounts at different financial institutions. Like intrabank funds transfers, the request designates source and destination accounts and the amount of the transfer. Also, as in intrabank funds transfers, the FI must be able to authenticate the source account. However, interbank funds transfers differ from intrabank funds transfers in the following respects:

- ◆ The routing and transit number of the destination account differs from the source account.
- ◆ At the discretion of an FI, the destination account can be subject to pre-notification.
- ◆ Source and destination accounts must be enabled for the Automated Clearing House (ACH).

Use the ACH system to implement the Interbank Funds Transfer, which is subject to the rules and regulations governing the ACH network.

In all other respects, interbank funds transfers function like intrabank funds transfers. The user can schedule them for a future date or request an immediate transfer. The user can modify or cancel scheduled transfers, but not immediate transfers. Scheduled transfers can recur at regular intervals.

11.8.1 Interbank Funds Transfer – US

In the United States, interbank funds transfers usually use only the <XFERINFO> portion of the request and response.

<i>Client Sends</i>	<i>Server Responds</i>
Source account	
Destination account	
Amount	
Date of transfer (optional)	
	Server ID for the transfer
	Source account
	Destination account
	Amount
	Expected / actual posting date

Interbank Funds Transfer Add is subject to synchronization.

11.8.2 Interbank Funds Transfer – International Usage

In countries where the funds transfer is the basis of the payments system, the OFX payments messages allow specifying payees by destination account (see [Chapter 12, "Payments"](#)).

Interbank Funds Transfer Add is subject to synchronization.

11.8.2.1 Interbank Funds Transfer Request <INTERRQ>

The <INTERRQ> request must appear within an <INTERTRNRQ> transaction wrapper.

<i>Tag</i>	<i>Description</i>
<INTERRQ>	Interbank-transfer-request aggregate
<XFERINFO>	Transfer information aggregate, see section 11.3.5
</XFERINFO>	
</INTERRQ>	

11.8.2.2 Interbank Funds Transfer Response <INTERRS>

The server cannot provide complete confirmation for interbank transfer. It can confirm only that the FI received the transfer instruction and possibly validated the source account, amount, and date specified in the transfer. Since the client does not know the status of the transfer at the time of the response, the server should confirm that it accepted the instruction and indicate the expected posting date of the transfer. The client can pick up the confirmation at a later date through a synchronization request. Servers should inform clients of any errors found while processing this transaction using the <STATUS> aggregate. A response containing <STATUS><CODE>0 and <XFERPRCSTS><XFERPRCCODE>FAILEDON should be avoided for problems such as an invalid account or amount.

The <INTERRS> response must appear within an <INTERTRNRS> transaction wrapper.

Tag	Version	Description
<INTERRS>		Interbank-transfer-response aggregate
<CURDEF>		Currency used in transfer, <i>currsymbol</i>
<SRVRTID>	V1 only	Server ID for this transfer, <i>SRVRTID</i>
<SRVRTID2>	V2 only	Server ID for this transfer, <i>SRVRTID2</i>
<XFERINFO>		Transfer information aggregate, see section 11.3.5
</XFERINFO>		
Transfer-date options. Choose either <DTXFERPRJ> or <DTPOSTED>		
<DTXFERPRJ>		Projected date of the transfer; response can contain either a <DTXFERPRJ> or a <DTPOSTED> but not both; <i>datetime</i>
-or-		
<DTPOSTED>		Actual date of the transfer, <i>datetime</i>
<REFNUM>		Server can generate a reference or check for the transfer, A-32
<RECSRVRTID>	V1 only	If server generates the response by a recurring transfer model, this ID references it. <i>SRVRTID</i>
<RECSRVRTID2>	V2 only	If server generates the response by a recurring transfer model, this ID references it. <i>SRVRTID2</i>
<XFERPRCSTS>		Transfer-processing status, see section 11.3.6
</XFERPRCSTS>		
</INTERRS>		

Note: A server can deliver this response to a client immediately after the client makes the request (for an immediate or one-time scheduled transfer). In response to a synchronization request by a client, the server should provide a second response containing complete status regarding the transfer. It should also return any transfers that it generates by a model.

11.8.2.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2006	Source account not found (ERROR)
2007	Source account closed (ERROR)
2008	Source account not authorized (ERROR)
2009	Destination account not found (ERROR)
2010	Destination account closed (ERROR)
2011	Destination account not authorized (ERROR)
2012	Invalid amount (ERROR)
2014	Date too soon (ERROR)
2015	Date too far in future (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10504	Insufficient funds (ERROR)

11.8.3 Interbank Funds Transfer Modification

The client sends a Transfer Modification request to modify a scheduled transfer. Immediate transfers cannot be modified, so this request should only be used for scheduled transfers. Once created and retrieved by the customer, spawned transfers are almost identical to customer-created transfers. (The exception is when a spawned transfer is modified or cancelled due to a recurring modification or cancellation request.) As with ordinary transfers, you can cancel or modify transactions individually. When modifying a transfer, the client must specify all of the elements and aggregates within the <XFERINFO> aggregate that were specified when the transfer was created, not just the elements and aggregates that the client wants to modify. <SRVRTID> specifies which transfer to modify. Some servers cannot support the modification of certain values. Servers must indicate this by returning status code 10505 when the client requests an unsupported modification. Clients must not change <BANKACCTFROM> or <CCACCTFROM> in a funds transfer modification.

Interbank Funds Transfer Modification is subject to synchronization.

11.8.3.1 Request <INTERMODRQ>

The <INTERMODRQ> request must appear within an <INTERTRNRQ> transaction wrapper.

Tag	Version	Description
<INTERMODRQ>		Modification-request aggregate
<SRVRTID>	V1 only	ID assigned by the server to the transfer being modified, <i>SRVRTID</i>
<SRVRTID2>	V2 only	ID assigned by the server to the transfer being modified, <i>SRVRTID2</i>
<XFERINFO>		Transfer information aggregate, see section 11.3.5
</XFERINFO>		
</INTERMODRQ>		

11.8.3.2 Response <INTERMODRS>

The <INTERMODRS> response must appear within an <INTERTRNRS> transaction wrapper.

Tag	Version	Description
<INTERMODRS>		Modification-response aggregate
<SRVRTID>	V1 only	ID assigned by the server to the transfer being modified, <i>SRVRTID</i>
<SRVRTID2>	V2 only	ID assigned by the server to the transfer being modified, <i>SRVRTID2</i>
<XFERINFO>		Transfer information aggregate; server returns if client provided an <XFERINFO> in the request, see section 11.3.5
</XFERINFO>		
<XFERPRCSTS>		Processing status for transfer, see section 11.3.6
</XFERPRCSTS>		
</INTERMODRS>		

11.8.3.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2006	Source account not found (ERROR)
2007	Source account closed (ERROR)
2008	Source account not authorized (ERROR)
2009	Destination account not found (ERROR)
2010	Destination account closed (ERROR)
2011	Destination account not authorized (ERROR)
2012	Invalid amount (ERROR)
2014	Date too soon (ERROR)
2015	Date too far in future (ERROR)
2016	Transaction already committed (ERROR)
2017	Already canceled (ERROR)
2018	Unknown server ID (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10504	Insufficient funds (ERROR)
10505	Cannot modify element (ERROR)
10514	Transaction already processed (ERROR)

11.8.4 Interbank Funds Transfer Cancellation

The client sends a Transfer Cancellation request to cancel a scheduled interbank transfer, where <SRVRTID> identifies the transfer. Immediate transfers cannot be canceled, so this request should only be used for scheduled transfers.

Interbank Funds Transfer Cancellation is subject to synchronization.

11.8.4.1 Request <INTERCANRQ>

The <INTERCANRQ> request must appear within an <INTERTRNRQ> transaction wrapper.

Tag	Version	Description
<INTERCANRQ>		Transfer-cancellation-request aggregate
<SRVRTID>	V1 only	ID of the transfer to cancel. The server must have previously assigned this ID to a transfer. <i>SRVRTID</i>
<SRVRTID2>	V2 only	ID of the transfer to cancel. The server must have previously assigned this ID to a transfer. <i>SRVRTID2</i>
</INTERCANRQ>		

11.8.4.2 Response <INTERCANRS>

The <INTERCANRS> response must appear within an <INTERTRNRS> transaction wrapper.

Tag	Version	Description
<INTERCANRS>		Transfer-cancellation-response aggregate
<SRVRTID>	V1 only	ID of the transfer to cancel. The server must have previously assigned this ID to a transfer. <i>SRVRTID</i>
<SRVRTID2>	V2 only	ID of the transfer to cancel. The server must have previously assigned this ID to a transfer. <i>SRVRTID2</i>
</INTERCANRS>		

11.8.4.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2016	Transaction already committed (ERROR)
2017	Already canceled (ERROR)
2018	Unknown server ID (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10514	Transaction already processed (ERROR)

11.8.5 Multiple Interbank Funds Transfer

Some countries have a special interbank funds transfer transaction. Such a transaction combines multiple interbank transfer instructions into a single unit of work. OFX has a special transaction wrapper, the <MULTIINTERTRNRQ>, that can contain 1 or more interbank transfer aggregates. Only Message set version 2 and above support <MULTIINTERTRNRQ>, and even then only if the FI profile says it's supported.

Multiple Interbank Funds Transfer is subject to synchronization.

11.8.5.1 Multiple Interbank Funds Transfer Request <MULTIINTERTRNRQ>

The <MULTIINTERTRNRQ> wrapper contains 1 or more <INTERRQ> aggregates. The interbank transfers are a single unit of work that share a common <TRNUID>, <CLTCOOKIE> and <TAN>. In other words if one of the requests within the transaction wrapper fails then all the request fail.

OFX does not support interbank transfer modify and interbank transfer cancellation requests in the <MULTIINTERTRNRQ>.

Tag	Version	Description
<MULTIINTERTRNRQ>	V2 only	Multiple interbank-transfer-transaction wrapper.
<TRNUID>		Client-assigned globally-unique ID for this transaction, <i>trnuid</i>
<CLTCOOKIE>		Data to be echoed in the transaction response, A-32
<TAN>		Transaction authorization number; used in some countries with some types of transactions. The FI Profile defines messages that require a <TAN>, A-80
<INTERRQ>		One or more interbank transfer request aggregates, see section 11.8.2.1 .
</INTERRQ>		
</MULTIINTERTRNRQ>		

11.8.5.2 Multiple Interbank Funds Transfer Response <MULTIINTERTRNRS>

The <MULTIINTERTRNRS> wrapper contains zero or more <INTERRS>aggregates. The interbank transfers share a common <TRNUID>, <CLTCOOKIE> and <STATUS>.

Note: Responses should not be sent in error cases (see section [2.4.6](#)).

Tag	Version	Description
<MULTIINTERTRNRS>	V2 only	Multiple interbank-transfer-transaction wrapper.
<TRNUID>		Client-assigned globally-unique ID for this transaction, <i>trnuid</i>
<STATUS>		Status aggregate
</STATUS>		
<CLTCOOKIE>		Client-provided data, REQUIRED if provided in request, A-32
<INTERRS>		Zero or more interbank transfer response aggregates, see section 11.8.2.2 .
</INTERRS>		
</MULTIINTERTRNRS>		

11.8.5.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2006	Source account not found (ERROR)
2007	Source account closed (ERROR)
2008	Source account not authorized (ERROR)
2009	Destination account not found (ERROR)
2010	Destination account closed (ERROR)
2011	Destination account not authorized (ERROR)
2012	Invalid amount (ERROR)
2014	Date too soon (ERROR)
2015	Date too far in future (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10504	Insufficient funds (ERROR)

11.9 Wire Funds Transfer

OFX enables clients to set up wire funds transfers. Wire funds transfers are similar to other types of funds transfers. Clients designate a source account that the FI can authenticate and a destination account at the same or a different institution. Clients also designate an amount and an optional date.

The FI must know the originator of the transfer. The beneficiary of the transfer might be an established customer at the same institution.

OFX implements wire funds transfers using the FedWire system, and is subject to its rules and regulations.

In almost all respects, wire funds transfers work like interbank funds transfers. A user can schedule and cancel them. Unlike interbank funds transfers, a user cannot modify Wire funds transfers once they have been set up. A user cannot set up wire funds transfers to recur at regular intervals.

<i>Client Sends</i>	<i>Server Responds</i>
Source account Originator Receiver Amount Date of transfer (optional)	Server ID for the transfer Originator Receiver Amount Expected / actual posting date

11.9.1 Wire Funds Transfer Addition

Wire Funds Transfer Add is subject to synchronization.

11.9.1.1 Request <WIRERQ>

The client prepares a <BANKACCTFROM> aggregate to describe the source account. The <WIREBENEFICIARY> aggregate specifies the destination account. The <WIREDESTBANK> aggregate describes the beneficiary's bank.

The <WIRERQ> request must appear within a <WIRETRNRQ> transaction wrapper.

Tag	Description
<WIRERQ>	Wire-transfer-request aggregate
<BANKACCTFROM>	Source of funds
</BANKACCTFROM>	
<WIREBENEFICIARY>	Wire transfer beneficiary, see section 11.9.1.1.1
</WIREBENEFICIARY>	
<WIREDESTBANK>	Beneficiary's bank
<EXTBANKDESC>	Extended bank description, see section 11.9.1.1.2
</EXTBANKDESC>	
</WIREDESTBANK>	
<TRNAMT>	Transfer amount, <i>amount</i>
<DTDUE>	Date to occur, <i>datetime</i>
<PAYINSTRUCT>	Payment instructions, A-255
</WIRERQ>	

11.9.1.1.1 Wire Beneficiary Aggregate <WIREBENEFICIARY>

The wire beneficiary aggregate describes the receiver of a wire transfer.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<WIREBENEFICIARY>		Wire-beneficiary aggregate
<NAME>		Name of beneficiary, A-32
<BANKACCTTO>		Bank details for beneficiary
</BANKACCTTO>		
<MEMO>	V1 only	Information for the beneficiary, <i>memo</i>
<MEMO2>	V2 only	Information for the beneficiary, <i>memo2</i>
</WIREBENEFICIARY>		

11.9.1.1.2 Extended Bank Description aggregate <EXTBANKDESC>

<i>Tag</i>	<i>Description</i>
<EXTBANKDESC>	Extended-bank-description aggregate
<NAME>	Abbreviated name of bank, A-32
<BANKID>	Routing: ABA number or S.W.I.F.T. number, A-9
<ADDR1>	Bank's address line 1, A-32
<ADDR2>	Bank's address line 2, A-32
<ADDR3>	Bank's address line 3. Use of <ADDR3> requires the presence of <ADDR2>, A-32
<CITY>	Bank's city, A-32
<STATE>	Bank's state or province, A-5
<POSTALCODE>	Bank's zip code, A-11
<COUNTRY>	Bank's country; 3-letter country code from ISO / DIS-3166, A-3
<PHONE>	Bank's phone number, A-32
</EXTBANKDESC>	

11.9.1.2 Response <WIRERS>

The server cannot provide complete confirmation for the transfer. It can confirm only that the server received the transfer instruction and possibly that it validated the source account, amount, and date specified in the transfer. For any transfer where the client does not know the status at the time of the response, the server should confirm that it accepted the instruction and indicate the expected posting date of the transfer. The client can pick up the confirmation at a later date through a synchronization request.

The server can indicate the fee assessed for the transfer by using the <FEE> element in the response. The server can also include a confirmation message in the response.

The <DTDUE> in a response may have been adjusted by a server. For example, the server may adjust <DTDUE> to comply with non-processing days. If a client sends a request to make a transfer on July 4 and July 4 happens to be a non-processing day, the <DTDUE> in the response may be July 4 (because the server hasn't adjusted it yet), July 5 (because this server rolls dates forward), or some other date. For this reason, a client should pay attention to the <DTDUE> in the response.

The <WIRERS> response must appear within a <WIRETRNRS> transaction wrapper.

Tag	Version	Description
<WIRERS>		Wire-transfer-response aggregate
<CURDEF>		Currency used in transfer, <i>currsymbol</i>
<SRVRTID>	V1 only	Server ID for this transfer, <i>SRVRTID</i>
<SRVRTID2>	V2 only	Server ID for this transfer, <i>SRVRTID2</i>
<BANKACCTFROM>		Source of funds
</BANKACCTFROM>		
<WIREBENEFICIARY>		Wire transfer beneficiary, see section 11.9.1.1.1
</WIREBENEFICIARY>		
<WIREDESTBANK>		Beneficiary's bank
<EXTBANKDESC>		Extended bank description, see section 11.9.1.1.2
</EXTBANKDESC>		
<WIREDESTBANK>		
<TRNAMT>		Transfer amount, <i>amount</i>
<DTDUE>		Date to occur, echoed if provided in request, <i>datetime</i>
<PAYINSTRUCT>		Payment instructions, echoed if provided in request, A-255
<i>Transfer-date options. Choose either <DTXFERPRJ> or <DTPOSTED></i>		
<DTXFERPRJ>		Projected date of the transfer; response can contain either a <DTXFERPRJ> or a <DTPOSTED> but not both; <i>datetime</i>
-or-		
<DTPOSTED>		Actual date of the transfer, <i>datetime</i>
<FEE>		Fee assessed for the transfer, <i>amount</i>
<CONFMSG>		Confirmation message, A-255
</WIRERS>		

11.9.1.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2006	Source account not found (ERROR)
2007	Source account closed (ERROR)
2008	Source account not authorized (ERROR)
2012	Invalid amount (ERROR)
2014	Date too soon (ERROR)
2015	Date too far in future (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10504	Insufficient funds (ERROR)
10516	Wire beneficiary invalid (ERROR)

11.9.2 Wire Funds Transfer Cancellation

The client sends a Wire Funds Transfer Cancellation Request to cancel a scheduled transfer, where <SRVRTID> identifies the transfer.

Wire Funds Transfer Cancellation is subject to synchronization.

11.9.2.1 Request <WIRECANRQ>

The <WIRECANRQ> request must appear within a <WIRETRNRQ> transaction wrapper.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<WIRECANRQ>		Wire-transfer-cancellation-request aggregate
<SRVRTID>	V1 only	ID of the transfer to cancel; server must have previously assigned this ID to a transfer, <i>SRVRTID</i>
<SRVRTID2>	V2 only	ID of the transfer to cancel; server must have previously assigned this ID to a transfer, <i>SRVRTID2</i>
</WIRECANRQ>		

11.9.2.2 Response <WIRECANRS>

The <WIRECANRS> response must appear within a <WIRETRNRS> transaction wrapper.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<WIRECANRS>		Wire-transfer-cancellation-response aggregate
<SRVRTID>	V1 only	ID of the transfer to cancel; server must have previously assigned this ID to a transfer, <i>SRVRTID</i>
<SRVRTID2>	V2 only	ID of the transfer to cancel; server must have previously assigned this ID to a transfer, <i>SRVRTID2</i>
</WIRECANRS>		

11.9.2.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2016	Transaction already committed (ERROR)
2017	Already canceled (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10514	Transaction already processed (ERROR)

11.10 Recurring Funds Transfer

OFX uses a Recurring Funds Transfer Add request to set up a recurring transfer model. The transfer model generates transfers according to its schedule. Transfers created by a model and retrieved by a customer can be modified or canceled without impacting the model.

A user can create recurring funds transfer models to generate two types of scheduled transfers: interbank and intrabank. You cannot set up recurring wire funds transfers.

For more information on recurring transactions, see [Chapter 10, "Recurring Transactions."](#)

11.10.1 Recurring Intrabank Funds Transfer Addition

A Recurring Intrabank Funds Transfer Add request sets up an intrabank funds transfer that repeats at a specified interval for a specified period of time.

Model-created transfers are retrieved by means of a synchronization request.

<i>Client Sends</i>	<i>Server Responds</i>
Source account	
Destination account	
Amount	
Date of first transfer	
Frequency	
Duration	
	Server ID for the model
	Source account
	Destination account
	Amount
	Date of first transfer
	Frequency
	Duration

Recurring Intrabank Funds Transfer Add is subject to synchronization.

11.10.1.1 Request <RECINTRARQ>

The <RECINTRARQ> request must appear within a <RECINTRATRNRQ> transaction wrapper.

Tag	Version	Description
<RECINTRARQ>		Recurring-transfer-request aggregate
<RECURRINST>		Recurring-instructions aggregate, see section
</RECURRINST>		
<INTRARQ>	V1 only	Intrabank-transfer-request aggregate, see section 11.7.1.1
</INTRARQ>		
<XFERINFO>	V2 only	Transfer info aggregate, see section 11.3.5 .
</XFERINFO>		Version 2 uses <XFERINFO> instead of <INTRARQ>
</RECINTRARQ>		

11.10.1.2 Response <RECINTRARS>

The <RECINTRARS> response must appear within a <RECINTRATRNR> transaction wrapper.

For version 1 of the message set, the <SRVRTID> included in the <INTRARS> should be set to the same value as the <RECSRVRTID>.

Note: This is the response to the recurring model only. Servers must still generate an INTRARS for each instance of the recurring transfer.

Tag	Version	Description
<RECINTRARS>		Recurring-transfer-response aggregate
<RECSRVRTID>	V1 only	Server-assigned ID for this model, <i>SRVRTID</i>
<RECSRVRTID2>	V2 only	Server-assigned ID for this model, <i>SRVRTID2</i>
<RECURRINST>		Recurring-instructions aggregate
</RECURRINST>		
<INTRARS>	V1 only	Intrabank-transfer-response aggregate, see section 11.7.1.2
</INTRARS>		
<XFERINFO>	V2 only	Transfer Info aggregate, see section 11.3.5
</XFERINFO>		Version 2 uses <XFERINFO> instead of <INTRARS>
</RECINTRARS>		

11.10.1.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2006	Source account not found (ERROR)
2007	Source account closed (ERROR)
2008	Source account not authorized (ERROR)
2009	Destination account not found (ERROR)
2010	Destination account closed (ERROR)
2011	Destination account not authorized (ERROR)
2014	Date too soon (ERROR)
2015	Date too far in future (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10508	Invalid frequency (ERROR)

11.10.2 Recurring Intrabank Funds Transfer Modification

The client sends a Recurring Intrabank Funds Transfer Modification request to modify a recurring intrabank transfer model.

Recurring Intrabank Funds Transfer Modification is subject to synchronization.

Clients must not change <BANKACCTFROM> in a recurring funds transfer modification.

11.10.2.1 Request <RECINTRAMODRQ>

<RECSRVRTID> identifies the model. The client can indicate whether the changes should apply to pending transfers.

The <RECINTRAMODRQ> request must appear within a <RECINTRATRNRQ> transaction wrapper.

Tag	Version	Description
<RECINTRAMODRQ>		Recurring-modification-request aggregate
<RECSRVRTID>	V1 only	ID assigned by the server to the model being modified, <i>SRVRTID</i>
<RECSRVRTID2>	V2 only	ID assigned by the server to the model being modified, <i>SRVRTID2</i>
<RECURRINST>		Recurring-instructions aggregate
</RECURRINST>		
<INTRARQ>	V1 only	Intrabank-transfer-request aggregate, see section 11.7.1.1
</INTRARQ>		
<XFERINFO>	V2 only	Transfer info aggregate, see section 11.3.5 . Version 2 uses <XFERINFO> instead of <INTRARQ>
</XFERINFO>		
<MODPENDING>		Modify pending flag, <i>Boolean</i> If the client sets this flag, the server must modify pending and future transfers.
</RECINTRAMODRQ>		

11.10.2.2 Response <RECINTRAMODRS>

The <RECINTRAMODRS> response must appear within a <RECINTRATRNRS> transaction wrapper.

Tag	Version	Description
<RECINTRAMODRS>		Recurring-transfer-modification-request aggregate
<RECSRVRTID>	V1 only	ID assigned by the server to the model being modified, <i>SRVRTID</i>
<RECSRVRTID2>	V2 only	ID assigned by the server to the model being modified, <i>SRVRTID2</i>
<RECURRINST>		Recurring-instructions aggregate
</RECURRINST>		
<INTRARS>	V1 only	Intrabank transfer response aggregate, see section 11.7.1.2
</INTRARS>		
<XFERINFO>	V2 only	Transfer Info aggregate, see section 11.3.5 . Version 2 uses <XFERINFO> instead of <INTRARS>
</XFERINFO>		
<MODPENDING>		Y if client requested that the server modify pending and future transfers. N if the client did not request that the server modify pending and future transfers. <i>Boolean</i>
</RECINTRAMODRS>		

11.10.2.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2006	Source account not found (ERROR)
2007	Source account closed (ERROR)
2008	Source account not authorized (ERROR)
2009	Destination account not found (ERROR)
2010	Destination account closed (ERROR)
2011	Destination account not authorized (ERROR)
2012	Invalid amount (ERROR)
2014	Date too soon (ERROR)
2015	Date too far in future (ERROR)
2016	Transaction already committed (ERROR)
2017	Already canceled (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10500	Too many checks to process (ERROR)
10505	Cannot modify element (ERROR)
10508	Invalid frequency (ERROR)
10514	Transaction already processed (ERROR)
10518	Unknown model ID (ERROR)

11.10.3 Recurring Intrabank Funds Transfer Cancellation

The client sends a Recurring Intrabank Funds Transfer Cancellation request to cancel a recurring intrabank transfer model.

Recurring Intrabank Funds Transfer Cancellation is subject to synchronization.

11.10.3.1 Request <RECINTRACANRQ>

<RECSRVRTID> identifies the model the user wants to cancel. The client can indicate whether the cancel should apply to pending transfers.

The <RECINTRACANRQ> request must appear within a <RECINTRATRNRQ> transaction wrapper.

Tag	Version	Description
<RECINTRACANRQ>		Recurring-transfer-cancellation-request aggregate
<RECSRVRTID>	V1 only	ID assigned by the server to the model being canceled, <i>SRVRTID</i>
<RECSRVRTID2>	V2 only	ID assigned by the server to the model being canceled, <i>SRVRTID2</i>
<CANPENDING>		Cancel pending flag, <i>Boolean</i> If Y, server should cancel all pending and unspawned transfers. If N, server should cancel only the model (and unspawned transfers).
</RECINTRACANRQ>		

11.10.3.2 Response <RECINTRACANRS>

The <RECINTRACANRS> response must appear within a <RECINTRATRNRS> transaction wrapper.

Tag	Version	Description
<RECINTRACANRS>		Recurring-transfer-cancellation-response aggregate
<RECSRVRTID>	V1 only	ID assigned by the server to the model being canceled, <i>SRVRTID</i>
<RECSRVRTID2>	V2 only	ID assigned by the server to the model being canceled, <i>SRVRTID2</i>
<CANPENDING>		Cancel pending flag, <i>Boolean</i> Y if the client requested that the server cancel all pending and unspawned transfers. N if the client requested that the server cancel only unspawned transfers.
</RECINTRACANRS>		

11.10.3.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2016	Transaction already committed (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10509	Model already canceled (ERROR)
10514	Transaction already processed (ERROR)
10518	Unknown model ID (ERROR)

11.10.4 Recurring Interbank Funds Transfer Addition

A Recurring Interbank Funds Transfer Add request sets up an interbank funds transfer that repeats at a specified interval for a specified period of time.

The client retrieves model-created transfers by means of a synchronization request.

<i>Client Sends</i>	<i>Server Responds</i>
Source account	
Destination account	
Amount	
Date of first transfer	
Frequency	
Duration	
	Server ID for the model
	Source account
	Destination account
	Amount
	Date of first transfer
	Frequency
	Duration

Recurring Interbank Funds Transfer Add is subject to synchronization

11.10.4.1 Request <RECINTERRQ>

The <RECINTERRQ> request must appear within a <RECINTERTRNRQ> transaction wrapper.

Tag	Version	Description
<RECINTERRQ>		Recurring-transfer-request aggregate
<RECURRINST>		Recurring-instructions aggregate
</RECURRINST>		
<INTERRQ>	V1 only	Interbank-transfer-request aggregate, see section 11.8.2.1
</INTERRQ>		
<XFERINFO>	V2 only	Transfer info aggregate, see section 11.3.5 .
</XFERINFO>		Version 2 uses <XFERINFO> instead of <INTERRQ>
</RECINTERRQ>		

11.10.4.2 Response <RECINTERRS>

The <RECINTERRS> response must appear within a <RECINTERTRNRS> transaction wrapper.

For version 1 of the message set, the <SRVRTID> included in the <INTERRS> should be set to the same value as the <RECSRVRTID>.

Note: This is the response to the recurring model only. Servers must still generate an <INTERRS> for each instance of the recurring transfer.

Tag	Version	Description
<RECINTERRS>		Recurring-transfer-response aggregate
<RECSRVRTID>	V1 only	Server-assigned ID for this model, <i>SRVRTID</i>
<RECSRVRTID2>	V2 only	Server-assigned ID for this model, <i>SRVRTID2</i>
<RECURRINST>		Recurring-instructions aggregate, see section 10.2
</RECURRINST>		
<INTERRS>	V1 only	Interbank funds transfer response, see section 11.8.2.2
</INTERRS>		
<XFERINFO>	V2 only	Transfer Info aggregate, see section 11.3.5 .
</XFERINFO>		Version 2 uses <XFERINFO> instead of <INTERRS>
</RECINTERRS>		

11.10.4.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2006	Source account not found (ERROR)
2007	Source account closed (ERROR)
2008	Source account not authorized (ERROR)
2009	Destination account not found (ERROR)
2010	Destination account closed (ERROR)
2011	Destination account not authorized (ERROR)
2012	Invalid amount (ERROR)
2014	Date too soon (ERROR)
2015	Date too far in future (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10504	Insufficient funds (ERROR)
10508	Invalid frequency (ERROR)

11.10.5 Recurring Interbank Funds Transfer Modification

The client sends a Recurring Interbank Funds Transfer Modification request to modify a recurring interbank transfer model.

Recurring Interbank Funds Transfer Modification is subject to synchronization.

Clients must not change <BANKACCTFROM> in a recurring funds transfer modification.

11.10.5.1 Request <RECINTERMODRQ>

<RECSRVRTID> identifies the model. The client can indicate whether the changes should apply to pending transfers.

The <RECINTERMODRQ> request must appear within a <RECINTERTRNRQ> transaction wrapper.

Tag	Version	Description
<RECINTERMODRQ>		Recurring-modification-request aggregate
<RECSRVRTID>	V1 only	ID assigned by the server to the model being modified, <i>SRVRTID</i>
<RECSRVRTID2>	V2 only	ID assigned by the server to the model being modified, <i>SRVRTID2</i>
<RECURRINST>		Recurring-instructions aggregate
</RECURRINST>		
<INTERRQ>	V1 only	Interbank-funds-transfer-request aggregate, see section 11.8.2.1 .
</INTERRQ>		
<XFERINFO>	V2 only	Transfer info aggregate, see section 11.3.5 .
</XFERINFO>		Version 2 uses <XFERINFO> instead of <INTERRQ>
<MODPENDING>		Modify pending flag
		If the client sets this flag, the server must modify pending and future transfers. <i>Boolean</i>
</RECINTERMODRQ>		

11.10.5.2 Request <RECINTERMODRS>

The <RECINTERMODRS> response must appear within a <RECINTERTRNRS> transaction wrapper.

Tag	Version	Description
<RECINTERMODRS>		Recurring-transfer-modification-response aggregate
<RECSRVRTID>	V1 only	ID assigned by the server to the model being modified, <i>SRVRTID</i>
<RECSRVRTID2>	V2 only	ID assigned by the server to the model being modified, <i>SRVRTID2</i>
<RECURRINST>		Recurring-instructions aggregate
</RECURRINST>		
<INTERRS>	V1 only	Interbank-funds-transfer-response, see section 11.8.2.2
</INTERRS>		
<XFERINFO>	V2 only	Transfer Info aggregate, see section 11.3.5
</XFERINFO>		Version 2 uses <XFERINFO> instead of <INTERRS>
<MODPENDING>		Modify pending flag, <i>Boolean</i>
		Y if the client requested that the server modify pending and future transfers. N if the client did not request that the server modify pending and future transfers.
</RECINTERMODRS>		

11.10.5.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2006	Source account not found (ERROR)
2007	Source account closed (ERROR)
2008	Source account not authorized (ERROR)
2009	Destination account not found (ERROR)
2010	Destination account closed (ERROR)
2011	Destination account not authorized (ERROR)
2012	Invalid amount (ERROR)
2014	Date too soon (ERROR)
2015	Date too far in future (ERROR)
2016	Transaction already committed (ERROR)
2017	Already canceled (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10504	Insufficient funds (ERROR)
10505	Cannot modify element (ERROR)
10508	Invalid frequency (ERROR)
10510	Invalid payee ID (ERROR)
10514	Transaction already processed (ERROR)
10518	Unknown model ID (ERROR)

11.10.6 Recurring Interbank Funds Transfer Cancellation

The client sends a Recurring Transfer Cancellation request to cancel a recurring transfer model.

Recurring Transfer Cancellation is subject to synchronization.

11.10.6.1 Request <RECINTERCANRQ>

<RECSRVRTID> identifies the model the client wants to cancel. The client can indicate whether the cancel should apply to pending transfers.

The <RECINTERCANRQ> request must appear within a <RECINTERTRNRQ> transaction wrapper.

Tag	Version	Description
<RECINTERCANRQ>		Recurring-transfer-cancellation-request aggregate
<RECSRVRTID>	V1 only	ID assigned by the server to the model being canceled, <i>SRVRTID</i>
<RECSRVRTID2>	V2 only	ID assigned by the server to the model being canceled, <i>SRVRTID2</i>
<CANPENDING>		Cancel pending flag, <i>Boolean</i> If Y, server should cancel all pending and unspawned transfers. If N, server should cancel only the model (and unspawned transfers).
</RECINTERCANRQ>		

11.10.6.2 Response <RECINTERCANRS>

The <RECINTERCANRS> response must appear within a <RECINTERTRNRS> transaction wrapper.

Tag	Version	Description
<RECINTERCANRS>		Recurring-transfer-cancellation-response aggregate
<RECSRVRTID>	V1 only	ID assigned by the server to the model being canceled, <i>SRVRTID</i>
<RECSRVRTID2>	V2 only	ID assigned by the server to the model being canceled, <i>SRVRTID2</i>
<CANPENDING>		Cancel pending flag, <i>Boolean</i> Y if the client requested that the server cancel all pending and unspawned transfers. N if the client requested that the server cancel only unspawned transfers.
</RECINTERCANRS>		

11.10.6.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2016	Transaction already committed (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10509	Model already canceled (ERROR)
10514	Transaction already processed (ERROR)
10518	Unknown model ID (ERROR)

11.11 E-Mail and Customer Notification

OFX enables customers to contact their FIs when they have questions regarding their accounts. FIs can also notify their customers of significant events that have occurred regarding their accounts. For example, notification can occur if a customer writes a check that does not clear due to insufficient funds. The server prepares the notification and the client picks it up the next time it synchronizes with the server.

11.11.1 Banking E-Mail

OFX currently defines one banking e-mail message that clients can send to an FI. With this message, the user can prepare a message to the FI regarding one of his accounts. The server acknowledges receipt of the message. The FI prepares the response that the client picks up when it synchronizes with the server.

<i>Client Sends</i>	<i>Server Responds</i>
Addressed message	Acknowledgment
Bank account information	
.	
.	
.	Response to customer
Synchronization request	

11.11.1.1 Request <BANKMAILRQ>

The client must identify to which bank account the customer query is related.

The <BANKMAILRQ> request must appear within a <BANKMAILTRNRQ> transaction wrapper.

Tag	Description
<BANKMAILRQ> <i>Account-from options. Choose either <BANKACCTFROM> or <CCACCTFROM>.</i>	Bank-e-mail-request aggregate
<BANKACCTFROM> </BANKACCTFROM> -or- <CCACCTFROM> </CCACCTFROM>	Account-from aggregate, see section 11.3.1 Credit-card-account-from aggregate, see section 11.3.2
<MAIL> </MAIL> </BANKMAILRQ>	To, from, message information, see Chapter 9, "Customer to FI Communication"

11.11.1.2 Response <BANKMAILRS>

The <BANKMAILRS> response must appear within a <BANKMAILTRNRS> transaction wrapper.

Tag	Description
<BANKMAILRS> <i>Account-from options. Choose either <BANKACCTFROM> or <CCACCTFROM>.</i>	Bank-e-mail-response aggregate
<BANKACCTFROM> </BANKACCTFROM> -or- <CCACCTFROM> </CCACCTFROM>	Account-from aggregate, see section 11.3.1 Credit-card-account-from aggregate, see section 11.3.2
<MAIL> </MAIL> </BANKMAILRS>	To, from, message information, see Chapter 9, "Customer to FI Communication"

11.11.1.3 Status Codes

Code	Meaning
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2003	Account not found (ERROR)
2004	Account closed (ERROR)
2005	Account not authorized (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
15508	Transaction not authorized (ERROR)
16500	HTML not allowed (ERROR)
16501	Unknown mail To: (ERROR)

11.11.2 Notifications

OFX currently defines two banking notifications that an FI can support:

- ◆ Returned check
- ◆ Returned deposit

You can implement banking notifications through e-mail and synchronization. The client provides a <TOKEN> representing its current state with regard to banking notification. (See section [3.2.4](#).) The server can respond by returning a new token and one or more notification e-mail responses.

<i>Client Sends</i>	<i>Server Responds</i>
Synchronization request with current token	New token Bank e-mail Mail for returned check Mail for returned deposit

11.11.3 Returned Check and Deposit Notification

11.11.3.1 Response <CHKMAILRS>

The server returns this response (when a check has been returned), if it receives a banking e-mail synchronization message.

The <CHKMAILRS> response must appear within a <BANKMAILTRNRS> transaction wrapper.

<i>Tag</i>	<i>Description</i>
<CHKMAILRS>	Notification-message-response aggregate
<BANKACCTFROM>	Account-from aggregate, see section 11.3.1
</BANKACCTFROM>	
<MAIL>	To, from, message information, see Chapter 9, "Customer to FI Communication"
</MAIL>	
<CHECKNUM>	Check number, <i>A-12</i>
<TRNAMT>	Amount of check, <i>amount</i>
<DTUSER>	Customer date on check, <i>date</i>
<FEE>	Fee assessed for NSF, <i>amount</i>
</CHKMAILRS>	

11.11.3.2 Response <DEPMAILRS>

The server returns this response (when a deposit has been returned), if it receives a banking e-mail synchronization message.

The <DEPMAILRS> response must appear within a <BANKMAILTRNRS> transaction wrapper.

Tag	Description
<DEPMAILRS>	Notification-message-response aggregate
<BANKACCTFROM>	Account-from aggregate, see section 11.3.1
</BANKACCTFROM>	
<MAIL>	To, from, message information, see Chapter 9, "Customer to FI Communication"
</MAIL>	
<TRNAMT>	Amount of deposit, <i>amount</i>
<DTUSER>	Customer date of deposit, <i>date</i>
<FEE>	Fee assessed for NSF, <i>amount</i>
</DEPMAILRS>	

11.12 Data Synchronization for Banking

Banking customers must be able to obtain the current status of transactions previously sent to the server for processing. For example, once a client schedules a transfer and the transfer date has passed, the customer might wish to verify that the server made the transfer as directed. Also, OFX allows for interactions with the server through multiple clients. This means, for example, that the customer can perform some transactions from a home PC and others from an office computer, with each session seamlessly incorporating the activities performed on the other.

To accomplish these actions, the client uses a synchronization scheme to ensure that it has an accurate copy of the server data that is relevant to the client application.

Banking requires synchronization in the following areas: Stop Check, IntraBank Transfers, InterBank Transfers, Wire Transfers, and Banking Notifications.

11.12.1 Data Synchronization for Stop Check

11.12.1.1 Request <STPCHKSYNCRQ>

Tag	Version	Description
<STPCHKSYNCRQ> <i>Client synchronization option;</i> <TOKEN> , <TOKEN2> , <TOKENONLY> , or <REFRESH>		Synchronization-request aggregate
<TOKEN>	V1 only	Previous value of <TOKEN> received for this type of synchronization request from server; 0 for first-time requests; <i>token</i>
<TOKEN2>	V2 only	Previous value of <TOKEN2> received for this type of synchronization request from server; 0 for first-time requests; <i>token2</i>
<TOKENONLY>		Request for just the current <TOKEN> without the history, <i>Boolean</i>
<REFRESH>		Request for refresh of current state, <i>Boolean</i>
<REJECTIFMISSING>		If Y, do not process requests if client <TOKEN> is out of date, <i>Boolean</i>
<BANKACCTFROM>		Bank account of interest; token must be interpreted in terms of this account
</BANKACCTFROM>		
<STPCHKTRNRQ>		Stop-check transactions (0 or more)
</STPCHKTRNRQ>		
</STPCHKSYNCRQ>		

11.12.1.2 Response <STPCHKSYNCRS>

Tag	Version	Description
<STPCHKSYNCRS>		Synchronization-response aggregate
<TOKEN>	V1 only	New synchronization token, <i>token</i>
<TOKEN2>	V2 only	New synchronization token, <i>token2</i>
<LOSTSYNC>		Y if the token in the synchronization request is older than the earliest entry in the server's history table. In this case, some responses have been lost. N if the token in the synchronization request is newer than or matches a token in the server's history table. <i>Boolean</i>
<SYNCERROR>	V2 only	Optional error code (See section 6.4.4), <i>N-6</i> .
<BANKACCTFROM>		Bank account of interest; token must be interpreted in terms of this account
</BANKACCTFROM>		
<STPCHKTRNRS>		Stop-check transactions (0 or more)
</STPCHKTRNRS>		
</STPCHKSYNCRS>		

11.12.2 Data Synchronization for Intrabank Funds Transfers

<INTRASYNCRQ> must be supported by all servers, even if it will always return <TOKEN>0 without sync history, because a client cannot know whether or not the server would ever return updated transfer information. Specifically, the client cannot know if a transfer will be processed immediately or at the end of the business day until it has performed at least one transfer operation (then DTPOSTED vs. DTXFERPRJ indicates which “mode” the server operates in). As such, the client must always send an <INTRASYNCRQ> in case the server has updated information about a transfer, specifically immediate transfers, which were actually processed at the end of the business day where it may fail due to other account activity.

Transfers into an account do not show up in the sync for the recipient account. Only transfers out of an account show up in the sync for that account.

11.12.2.1 Request <INTRASYNCRQ>

Tag	Version	Description
<INTRASYNCRQ> <i>Client synchronization option;</i> <TOKEN> , <TOKEN2> , <TOKENONLY> , or <REFRESH>		Synchronization-request aggregate
<TOKEN>	V1 only	Previous value of <TOKEN> received for this type of synchronization request from server; 0 for first-time requests; <i>token</i>
<TOKEN2>	V2 only	Previous value of <TOKEN2> received for this type of synchronization request from server; 0 for first-time requests; <i>token2</i>
<TOKENONLY>		Request for just the current <TOKEN> without the history, <i>Boolean</i>
<REFRESH>		Request for refresh of current state, <i>Boolean</i>
<REJECTIFMISSING> <i>Account-from options. Choose either <BANKACCTFROM> or <CCACCTFROM>.</i>		If Y, do not process requests if client <TOKEN> is out of date, <i>Boolean</i>
<BANKACCTFROM> </BANKACCTFROM> -or- <CCACCTFROM> </CCACCTFROM>	V2 only	Account-from aggregate, see section 11.3.1
<INTRATRNRQ> </INTRATRNRQ> </INTRASYNCRQ>		Credit-card-account-from aggregate, see section 11.3.2
		Intrabank-funds-transfer transactions (0 or more)

11.12.2.2 Response <INTRASYNCRS>

Tag	Version	Description
<INTRASYNCRS>		Synchronization-response aggregate
<TOKEN>	V1 only	New synchronization token, <i>token</i>
<TOKEN2>	V2 only	New synchronization token, <i>token2</i>
<LOSTSYNC>		Y if the token in the synchronization request is older than the earliest entry in the server's history table. In this case, some responses have been lost. N if the token in the synchronization request is newer than or matches a token in the server's history table. <i>Boolean</i>
<SYNCERROR> <i>Account-from options. Choose either <BANKACCTFROM> or <CCACCTFROM>.</i>	V2 only	Optional error code (See section 6.4.4), <i>N-6</i> .
<BANKACCTFROM> </BANKACCTFROM> -or- <CCACCTFROM> </CCACCTFROM>	V2 only	Account-from aggregate, see section 11.3.1 Credit-card-account-from aggregate, see section 11.3.2
<INTRATRNRS> </INTRATRNRS> </INTRASYNCRS>		Intrabank-funds-transfer transactions (0 or more)

The <INTRASYNCRS> responses contain only intrabank transfers where the BANKACCTFROM or CCACCTFROM matches that submitted in the sync request.

11.12.3 Data Synchronization for Interbank Funds Transfers

Transfers into an account do not show up in the sync for the recipient account. Only transfers out of an account show up in the sync for that account.

11.12.3.1 Request <INTERSYNCRQ>

Tag	Version	Description
<INTERSYNCRQ> <i>Client synchronization option;</i> <TOKEN> , <TOKEN2> , <TOKENONLY> , or <REFRESH>		Synchronization-request aggregate
<TOKEN>	V1 only	Previous value of <TOKEN> received for this type of synchronization request from server; 0 for first-time requests; <i>token</i>
<TOKEN2>	V2 only	Previous value of <TOKEN2> received for this type of synchronization request from server; 0 for first-time requests; <i>token2</i>
<TOKENONLY>		Request for just the current <TOKEN> without the history, <i>Boolean</i>
<REFRESH>		Request for refresh of current state, <i>Boolean</i>
<REJECTIFMISSING> <i>Account-from options. Choose either <BANKACCTFROM> or <CCACCTFROM>.</i>		If Y, do not process requests if client <TOKEN> is out of date, <i>Boolean</i>
<BANKACCTFROM> </BANKACCTFROM> -or- <CCACCTFROM> </CCACCTFROM>	V2 only	Account-from aggregate, see section 11.3.1
<INTERTRNRQ> </INTERTRNRQ> <MULTIINTERTRNRQ> </MULTIINTERTRNRQ> </INTERSYNCRQ>	V2 only	Interbank-funds-transfer transactions (0 or more) Multiple interbank-funds-transfer transactions (0 or more)

11.12.3.2 Response <INTERSYNCRS>

Tag	Version	Description
<INTERSYNCRS>		Synchronization-response aggregate
<TOKEN>	V1 only	New synchronization token, <i>token</i>
<TOKEN2>	V2 only	New synchronization token, <i>token2</i>
<LOSTSYNC>		Y if the token in the synchronization request is older than the earliest entry in the server's history table. In this case, some responses have been lost. N if the token in the synchronization request is newer than or matches a token in the server's history table. <i>Boolean</i>
<SYNCERROR> <i>Account-from options. Choose either <BANKACCTFROM> or <CCACCTFROM>.</i>	V2 only	Optional error code (See section 6.4.4), <i>N-6</i> .
<BANKACCTFROM> </BANKACCTFROM> -or- <CCACCTFROM> </CCACCTFROM>	V2 only	Account-from aggregate, see section 11.3.1 Credit-card-account-from aggregate, see section 11.3.2
<INTERTRNRS> </INTERTRNRS>		Interbank-funds-transfer transactions (0 or more)
<MULTIINTERTRNRS> </MULTIINTERTRNRS> </INTERSYNCRS>	V2 only	Multiple interbank-funds-transfer transactions (0 or more)

The <INTERSYNCRS> responses contain only interbank transfers where the BANKACCTFROM or CCACCTFROM matches that submitted in the sync request.

11.12.4 Data Synchronization for Wire Funds Transfers

11.12.4.1 Request <WIRESYNCRQ>

Tag	Version	Description
<WIRESYNCRQ> <i>Client synchronization option;</i> <TOKEN> , <TOKEN2> , <TOKENONLY> , or <REFRESH>		Synchronization-request aggregate
<TOKEN>	V1 only	Previous value of <TOKEN> received for this type of synchronization request from server; 0 for first-time requests; <i>token</i>
<TOKEN2>	V2 only	Previous value of <TOKEN2> received for this type of synchronization request from server; 0 for first-time requests; <i>token2</i>
<TOKENONLY>		Request for just the current <TOKEN> without the history, <i>Boolean</i>
<REFRESH>		Request for refresh of current state, <i>Boolean</i>
<REJECTIFMISSING>		If Y, do not process requests if client <TOKEN> is out of date, <i>Boolean</i>
<BANKACCTFROM>		Bank account of interest; token must be interpreted in terms of this account.
</BANKACCTFROM>		
<WIRETRNRQ>		Wire-transfer transactions (0 or more)
<WIRETRNRQ>		
</WIRESYNCRQ>		

11.12.4.2 Response <WIRESYNCRS>

Tag	Version	Description
<WIRESYNCRS>		Synchronization-response aggregate
<TOKEN>	V1 only	New synchronization token, <i>token</i>
<TOKEN2>	V2 only	New synchronization token, <i>token2</i>
<LOSTSYNC>		Y if the token in the synchronization request is older than the earliest entry in the server's history table. In this case, some responses have been lost. N if the token in the synchronization request is newer than or matches a token in the server's history table. <i>Boolean</i>
<SYNCEERROR>	V2 only	Optional error code (See section 6.4.4.), <i>N-6</i> .
<BANKACCTFROM>		Bank account of interest; token must be interpreted in terms of this account
</BANKACCTFROM>		
<WIRETRNRS>		Wire-transfer transactions (0 or more)
<WIRETRNRS>		
<WIRETRNRS>		
</WIRESYNCRS>		

11.12.5 Data Synchronization for Recurring Intrabank Funds Transfers

11.12.5.1 Request <RECINTRASYNCRQ>

This request will synchronize the client with the server in relation to recurring intrabank transfer models. To synchronize individual transfers that were created by the model (and perhaps canceled by another client), the client must also issue an <INTRASYNCRQ>.

Tag	Version	Description
<RECINTRASYNCRQ> <i>Client synchronization option;</i> <TOKEN> , <TOKEN2> , <TOKENONLY> , or <REFRESH>		Synchronization request
<TOKEN>	V1 only	Previous value of <TOKEN> received for this type of synchronization request from server; 0 for first-time requests; <i>token</i>
<TOKEN2>	V2 only	Previous value of <TOKEN2> received for this type of synchronization request from server; 0 for first-time requests; <i>token2</i>
<TOKENONLY>		Request for just the current <TOKEN> without the history, <i>Boolean</i>
<REFRESH>		Request for refresh of current state, <i>Boolean</i>
<REJECTIFMISSING> <i>Account-from options. Choose either <BANKACCTFROM> or <CCACCTFROM>.</i>		If Y, do not process requests if client <TOKEN> is out of date, <i>Boolean</i>
<BANKACCTFROM> </BANKACCTFROM> -or- <CCACCTFROM> </CCACCTFROM>	V2 only	Account-from aggregate, see section 11.3.1
<RECINTRATRNRQ> </RECINTRATRNRQ> </RECINTRASYNCRQ>		Recurring-intrabank-funds-transfer transactions (0 or more)

11.12.5.2 Response <RECINTRASYNCRS>

Tag	Version	Description
<RECINTRASYNCRS>		Synchronization-response aggregate
<TOKEN>	V1 only	New synchronization token, <i>token</i>
<TOKEN2>	V2 only	New synchronization token, <i>token2</i>
<LOSTSYNC>		Y if the token in the synchronization request is older than the earliest entry in the server's history table. In this case, some responses have been lost. N if the token in the synchronization request is newer than or matches a token in the server's history table. <i>Boolean</i>
<SYNCERROR> <i>Account-from options. Choose either <BANKACCTFROM> or <CCACCTFROM>.</i>	V2 only	Optional error code (See section 6.4.4), <i>N-6</i> .
<BANKACCTFROM> </BANKACCTFROM> -or- <CCACCTFROM> </CCACCTFROM>	V2 only	Account-from aggregate, see section 11.3.1 Credit-card-account-from aggregate, see section 11.3.2
<RECINTRATRNRS> </RECINTRATRNRS> </RECINTRASYNCRS>		Recurring-intrabank-funds-transfer transactions (0 or more)

The <RECINTRASYNCRS> responses contain only intrabank transfer models where the BANKACCTFROM or CCACCTFROM matches that submitted in the sync request.

11.12.6 Data Synchronization for Recurring Interbank Funds Transfers

11.12.6.1 Request <RECINTERSYNCRQ>

This request will synchronize the client with the server in relation to recurring interbank transfer models. To synchronize individual funds transfers that were created by the model (and perhaps canceled by another client), the client must also issue an <INTERSYNCRQ>.

Tag	Version	Description
<RECINTERSYNCRQ> <i>Client synchronization option;</i> <TOKEN> , <TOKEN2> , <TOKENONLY> , or <REFRESH>		Synchronization-request aggregate
<TOKEN>	V1 only	Previous value of <TOKEN> received for this type of synchronization request from server; 0 for first-time requests; <i>token</i>
<TOKEN2>	V2 only	Previous value of <TOKEN2> received for this type of synchronization request from server; 0 for first-time requests; <i>token2</i>
<TOKENONLY>		Request for just the current <TOKEN> without the history, <i>Boolean</i>
<REFRESH>		Request for refresh of current state, <i>Boolean</i>
<REJECTIFMISSING> <i>Account-from options. Choose either <BANKACCTFROM> or <CCACCTFROM>.</i>		If Y, do not process requests if client <TOKEN> is out of date, <i>Boolean</i>
<BANKACCTFROM> </BANKACCTFROM> -or- <CCACCTFROM> </CCACCTFROM>	V2 only	Account-from aggregate, see section 11.3.1
<RECINTERTRNRQ> </RECINTERTRNRQ>		Credit-card-account-from aggregate, see section 11.3.2
</RECINTERSYNCRQ>		Recurring-transfer transactions (0 or more)

11.12.6.2 Response <RECINTERSYNCRS>

Tag	Version	Description
<RECINTERSYNCRS>		Synchronization-response aggregate
<TOKEN>	V1 only	New synchronization token, <i>token</i>
<TOKEN2>	V2 only	New synchronization token, <i>token2</i>
<LOSTSYNC>		Y if the token in the synchronization request is older than the earliest entry in the server's history table. In this case, some responses have been lost. N if the token in the synchronization request is newer than or matches a token in the server's history table. <i>Boolean</i>
<SYNCERROR> <i>Account-from options. Choose either <BANKACCTFROM> or <CCACCTFROM>.</i>	V2 only	Optional error code (See section 6.4.4), <i>N-6</i> .
<BANKACCTFROM> </BANKACCTFROM> -or- <CCACCTFROM> </CCACCTFROM>	V2 only	Account-from aggregate, see section 11.3.1 Credit-card-account-from aggregate, see section 11.3.2
<RECINTERTRNRS> </RECINTERTRNRS> </RECINTERSYNCRS>		Recurring-interbank-funds-transfer transactions (0 or more)

11.12.7 Data Synchronization for Bank Mail

11.12.7.1 Request <BANKMAILSYNCRQ>

Tag	Version	Description
<BANKMAILSYNCRQ> <i>Client synchronization option;</i> <TOKEN> , <TOKEN2> , <TOKENONLY> , or <REFRESH>		Synchronization-request aggregate
<TOKEN>	V1 only	Previous value of <TOKEN> received for this type of synchronization request from server; 0 for first-time requests; <i>token</i>
<TOKEN2>	V2 only	Previous value of <TOKEN2> received for this type of synchronization request from server; 0 for first-time requests; <i>token2</i>
<TOKENONLY>		Request for just the current <TOKEN> without the history, <i>Boolean</i>
<REFRESH>		Request for refresh of current state, <i>Boolean</i>
<REJECTIFMISSING>		If Y, do not process requests if client <TOKEN> is out of date, <i>Boolean</i>
<INCIMAGES>		Y if the client accepts mail with images in the message body. N if the client does not accept mail with images in the message body. <i>Boolean</i>
<USEHTML>		Y if client wants an HTML response, N if client wants plain text, <i>Boolean</i>
<i>Account-from options. Choose either <BANKACCTFROM> or <CCACCTFROM>.</i> <BANKACCTFROM> </BANKACCTFROM> -or- <CCACCTFROM> </CCACCTFROM>		Account-from aggregate, see section 11.3.1 Credit-card-account-from aggregate, see section 11.3.2
<BANKMAILTRNRQ> </BANKMAILTRNRQ> </BANKMAILSYNCRQ>		Bank-mail transactions (0 or more)

11.12.7.2 Response <BANKMAILSYNCRS>

Tag	Version	Description
<BANKMAILSYNCRS>		Synchronization-response aggregate
<TOKEN>	V1 only	New synchronization token, <i>token</i>
<TOKEN2>	V2 only	New synchronization token, <i>token2</i>
<LOSTSYNC>		Y if the token in the synchronization request is older than the earliest entry in the server's history table. In this case, some responses have been lost. N if the token in the synchronization request is newer than or matches a token in the server's history table. <i>Boolean</i>
<SYNCERROR> <i>Account-from options. Choose either <BANKACCTFROM> or <CCACCTFROM>.</i>	V2 only	Optional error code (See section 6.4.4), <i>N-6</i> .
<BANKACCTFROM> </BANKACCTFROM> -or- <CCACCTFROM> </CCACCTFROM>		Account-from aggregate, see section 11.3.1 Credit-card-account-from aggregate, see section 11.3.2
<BANKMAILTRNRS> </BANKMAILTRNRS> </BANKMAILSYNCRS>		Bank-mail transactions (0 or more)

11.13 Message Sets and Profile

OFX separates messages that the client and server send into groups called message sets. Each FI defines the message sets that the institution supports. The messages described in this section fall into the following types:

- ◆ Banking – includes statement download, closing statement download, bank e-mail, notification, and intrabank funds transfer
- ◆ Credit Card – credit card statement download and closing statement download
- ◆ Interbank Funds Transfers
- ◆ Wire Funds Transfers

Each message set contains options and attributes that allow an FI to customize its use of OFX. For example, an institution can support the Interbank Funds Transfer Message Set (INTERXFERMSGSETV1), but it can choose not to support the recurring form of these transfers.

The profile defines the options and attributes as part of each message-set definition. Each set of options and attributes appears within an aggregate that is specific to a message set. For example, <WIREXFERMSGSETV1> contains all of the options and attributes that pertain to wire transfers.

11.13.1 Message Sets and Messages

11.13.1.1 Bank Message Set and Messages

Note: BANKMSGSET version revision due to adding CCACCTFROM as a funding account to the INTRASYNCRQ/S and the REINTRASYNCRQ/S

11.13.1.1.1 Bank Message Set Request Messages

Message Set	Message
<BANKMSGSET>	
<BANKMSGSETV1>	
<BANKMSGSRQV1>	STMTTRNRQ
	STMTRQ
	STMTENDTRNRQ
	STMTENDRQ
	STPCHKTRNRQ
	STPCHKRQ
	INTRATRNRQ
	INTRARQ
	INTRAMODRQ
	INTRACANRQ
	RECINTRATRNRQ
	RECINTRARQ
	RECINTRAMODRQ
	RECINTRACANRQ
	BANKMAILTRNRQ
	BANKMAILRQ
	STPCHKSYNCRQ
	INTRASYNCRQ
	RECINTRASYNCRQ
	BANKMAILSYNCRQ
</BANKMSGSRQV1>	
</BANKMSGSETV1>	

<i>Message Set</i>	<i>Message</i>
<BANKMSGSETV2>	
<BANKMSGSRQV2>	STMTTRNRQ STMTRQ STMTENDTRNRQ STMTENDRQ STPCHKTRNRQ STPCHKRQ INTRATRNRQ INTRARQ INTRAMODRQ INTRACANRQ RECINTRATRNRQ RECINTRARQ RECINTRAMODRQ RECINTRACANRQ BANKMAILTRNRQ BANKMAILRQ STPCHKSYNCRQ INTRASYNCRQ RECINTRASYNCRQ BANKMAILSYNCRQ
</BANKMSGSRQV2>	
</BANKMSGSETV2>	
</BANKMSGSET>	

11.13.1.1.2 Bank Message Set Response Messages

<i>Message Set</i>	<i>Message</i>
<BANKMSGSET>	
<BANKMSGSETV1>	
<BANKMSGSRSV1>	STMTTRNRS STMTRS STMTENDTRNRS STMTENDRS STPCHKTRNRS STPCHKRS INTRATRNRNRS INTRARS INTRAMODRS INTRACANRS RECINTRATRNRNRS RECINTRARS RECINTRAMODRS RECINTRACANRS BANKMAILTRNRS BANKMAILRS CHKMAILRS DEPMAILRS STPCHKSYNCRS INTRASYNCRS RECINTRASYNCRS BANKMAILSYNCRS
</BANKMSGSRSV1>	
</BANKMSGSETV1>	
<BANKMSGSETV2>	
<BANKMSGSRSV2>	STMTTRNRS STMTRS

<i>Message Set</i>	<i>Message</i>
	STMTENDTRNRS STMTENDRS STPCHKTRNRS STPCHKRS INTRATRNRNRS INTRARS INTRAMODRS INTRACANRS RECINTRATRNRNRS RECINTRARS RECINTRAMODRS RECINTRACANRS BANKMAILTRNRNRS BANKMAILRS CHKMAILRS DEPMAILRS STPCHKSYNCRS INTRASYNCRS RECINTRASYNCRS BANKMAILSYNCRS
</BANKMSGSRSV2> </BANKMSGSETV2> </BANKMSGSET>	

11.13.1.2 Credit Card Message Set and Messages

11.13.1.2.1 Credit Card Message Set Request Messages

<i>Message Set</i>	<i>Message</i>
<CREDITCARDMSGSET>	
<CREDITCARDMSGSETV1>	
<CREDITCARDMSGSRQV1>	CCSTMTTRNRQ
	CCSTMTRQ
	CCSTMTENDTRNRQ
	CCSTMTENDRQ
</CREDITCARDMSGSRQV1>	
</CREDITCARDMSGSETV1>	
<CREDITCARDMSGSETV2>	
<CREDITCARDMSGSRQV2>	CCSTMTTRNRQ
	CCSTMTRQ
	CCSTMTENDTRNRQ
	CCSTMTENDRQ
</CREDITCARDMSGSRQV2>	
</CREDITCARDMSGSETV2>	
</CREDITCARDMSGSET>	

11.13.1.2.2 Credit Card Message Set Response Messages

<i>Message Set</i>	<i>Message</i>
<CREDITCARDMSGSET> <CREDITCARDMSGSETV1> <CREDITCARDMSGSRSV1> CCSTMTRNRS CCSTMTRS CCSTMTEENDTRNRS CCSTMTEENDRS </CREDITCARDMSGSRSV1> </CREDITCARDMSGSETV1> <CREDITCARDMSGSETV2> <CREDITCARDMSGSRSV2> CCSTMTRNRS CCSTMTRS CCSTMTEENDTRNRS CCSTMTEENDRS </CREDITCARDMSGSRSV2> </CREDITCARDMSGSETV2> </CREDITCARDMSGSET>	

11.13.1.3 Interbank Transfer Message Set and Messages

11.13.1.3.1 Interbank Transfer Message Set Request Messages

<i>Message Set</i>	<i>Message</i>
<INTERXFERMSGSET>	
<INTERXFERMSGSETV1>	
<INTERXFERMSGSRQV1>	INTERTRNRQ INTERRQ INTERMODRQ INTERCANRQ RECINTERTRNRQ RECINTERRQ RECINTERMODRQ RECINTERCANRQ INTERSYNCRQ RECINTERSYNCRQ
</INTERXFERMSGSRQV1>	
</INTERXFERMSGSETV1>	
<INTERXFERMSGSETV2>	
<INTERXFERMSGSRQV2>	INTERTRNRQ INTERRQ INTERMODRQ INTERCANRQ MULTIINTERTRNRQ INTERRQ RECINTERTRNRQ RECINTERRQ RECINTERMODRQ RECINTERCANRQ INTERSYNCRQ RECINTERSYNCRQ

<i>Message Set</i>	<i>Message</i>
</INTERXFERMSGSRSV2> </INTERXFERMSGSETV2> </INTERXFERMSGSET>	RECINTERCANRS INTERSYNCRS RECINTERSYNCRS

11.13.1.4 Wire Transfer Message Set and Messages

11.13.1.4.1 Wire Transfer Message Set Request Messages

<i>Message Set</i>	<i>Message</i>
<WIREXFERMSGSET> <WIREXFERMSGSETV1> <WIREXFERMSGSRQV1> </WIREXFERMSGSRQV1> </WIREXFERMSGSETV1> <WIREXFERMSGSETV2> <WIREXFERMSGSRQV2> </WIREXFERMSGSRQV2> </WIREXFERMSGSETV2> </WIREXFERMSGSET>	WIRETRNRQ WIRERQ WIRECANRQ WIRESYNCRQ WIRETRNRQ WIRERQ WIRECANRQ WIRESYNCRQ

11.13.1.4.2 Wire Transfer Message Set Response Messages

<i>Message Set</i>	<i>Message</i>
<WIREXFERMSGSET>	
<WIREXFERMSGSETV1>	
<WIREXFERMSGSRSV1>	WIRETRNRS
	WIRERS
	WIRECANRS
	WIRESYNCRS
</WIREXFERMSGSRSV1>	
</WIREXFERMSGSETV1>	
<WIREXFERMSGSETV2>	
<WIREXFERMSGSRSV2>	WIRETRNRS
	WIRERS
	WIRECANRS
	WIRESYNCRS
</WIREXFERMSGSRSV2>	
</WIREXFERMSGSETV2>	
</WIREXFERMSGSET>	

11.13.2 Bank Message Set Profile

11.13.2.1 <BANKMSGSET>, <BANKMSGSETV1>, <BANKMSGSETV2>

Tag	Description
<BANKMSGSET>	Message set for banking
<BANKMSGSETV1>	Version 1 of message set
<MSGSETCORE>	Common message-set core
</MSGSETCORE>	
<INVALIDACCTTYPE>	Account type not supported in <BANKACCTFROM>; 0 or more of account types, see section 11.3.1.2 for values
<CLOSINGAVAIL>	Closing statement information available, <i>Boolean</i>
<XFERPROF>	Intrabank transfer profile (if supported), see section 11.13.2.2
</XFERPROF>	
<STPCHKPROF>	Stop check profile (if supported), see section 11.13.2.3
</STPCHKPROF>	
<EMAILPROF>	E-mail profile, see section 11.13.2.4
</EMAILPROF>	
</BANKMSGSETV1>	End of bank message set version 1
<BANKMSGSETV2>	Zero or more version 2 message sets
<MSGSETCORE>	Common message-set core
</MSGSETCORE>	
<INVALIDACCTTYPE2>	Account type not supported in <BANKACCTFROM>; 0 or more of account types, see section 11.3.1.2 for values
<CLOSINGAVAIL>	Closing statement information available, <i>Boolean</i>
<XFERPROF>	Intrabank transfer profile (if supported), see section 11.13.2.2
</XFERPROF>	
<STPCHKPROF>	Stop check profile (if supported), see section 11.13.2.3
</STPCHKPROF>	
<EMAILPROF>	E-mail profile, see section 11.13.2.4
</EMAILPROF>	
</BANKMSGSETV2>	End of bank message set version 2
</BANKMSGSET>	

11.13.2.2 Banking Profile, Funds Transfer <XFERPROF>

Tag	Version	Description
<XFERPROF>		Intrabank transfer profile (if supported)
<PROCDAYSOFF>		Days of week that no processing occurs: MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, or SUNDAY. 0 or more <PROCDAYSOFF> can be sent.
<PROCENDTM>		Time of day that day's processing ends, <i>time</i>
<CANSCHED>		Supports scheduled transfers, <i>Boolean</i>
<CANRECUR>		Supports recurring transfers, <i>Boolean</i> . Requires <CANSCHED>
<CANMODXFERS>		Permit modifications to transfers, i.e. <INTRAMODRQ>, <i>Boolean</i>
<CANMODMDLS>		Permit modifications to models, i.e. <RECINTRAMODRQ>, <i>Boolean</i>
<MODELWND>		Model window; the number of days before a recurring transaction is scheduled to be processed that it is instantiated on the system, <i>N-3</i>
<DAYSWITH>		Number of days before processing date that funds are withdrawn, <i>N-3</i>
<DFLTDAYSTOPAY>		Default number of days to pay, <i>N-3</i>
<NEEDTANTRANSFER>	V2 only	A TAN is required for user authentication in the transaction wrapper of a request to create, modify or cancel an intrabank transfer (see section 2.4.6), <i>Boolean</i> Not used in USA
<SUPPORTDTAVAIL>	V2 only	Support for specifying a transfer value date using the DTAVAIL element in the <XFERINFO> aggregate (see section 11.3.5), <i>Boolean</i> Not used in USA
</XFERPROF>		

11.13.2.3 Banking Profile, Stop Checks <STPCHKPROF>

<i>Tag</i>	<i>Description</i>
<STPCHKPROF>	Stop check profile (if supported)
<PROCDAYSOFF>	Days of week that no processing occurs: MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, or SUNDAY. 0 or more <PROCDAYSOFF> can be sent.
<PROCENDTM>	Time of day that day's processing ends, <i>time</i>
<CANUSERANGE>	Can stop a range of checks, <i>Boolean</i> .
<CANUSEDESC>	Can stop by description, <i>Boolean</i> .
<STPCHKFEE>	Default stop check free <i>Amount</i>
</STPCHKPROF>	

11.13.2.4 Banking Profile, Email <EMAILPROF>

<i>Tag</i>	<i>Description</i>
<EMAILPROF>	E-mail profile
<CANEMAIL>	Supports generalized banking e-mail, <i>Boolean</i>
<CANNOTIFY>	Supports notification (of any kind), <i>Boolean</i>
</EMAILPROF>	

11.13.3 Credit Card Message Set Profile

<i>Tag</i>	<i>Description</i>
<CREDITCARDMSGSET>	Beginning tag for credit card message set
<CREDITCARDMSGSETV1>	Version 1 of message set
<MSGSETCORE>	Common message-set core
</MSGSETCORE>	
<CLOSINGAVAIL>	Closing statement information available, <i>Boolean</i>
</CREDITCARDMSGSETV1>	Ending tag of credit card message set version 1
<CREDITCARDMSGSETV2>	Zero or more version 2 message sets
<MSGSETCORE>	Common message-set core
</MSGSETCORE>	
<CLOSINGAVAIL>	Closing statement information available, <i>Boolean</i>
</CREDITCARDMSGSETV2>	Ending tag of credit card message set version 2
</CREDITCARDMSGSET>	Ending tag of credit card message set

11.13.4 Interbank Funds Transfer Message Set Profile

Tag	Description
<INTERXFERMSGSET>	Beginning tag for interbank transfers message set
<INTERXFERMSGSETV1>	Version 1 of message set
<MSGSETCORE>	Common message-set core
</MSGSETCORE>	
<XFERPROF>	Interbank transfer profile, same as XFERPROF in banking, see section 11.13.2.2
</XFERPROF>	
<CANBILLPAY>	Server is capable of handling bill payment as a form of transfers, <i>Boolean</i>
<CANCELWND>	Number of days after an interbank transfer occurs that it can be canceled, <i>N-3</i>
<DOMXFERFEE>	Standard fee for a domestic interbank transfer, <i>amount</i>
<INTLXFERFEE>	Standard fee for an international interbank transfer, <i>amount</i>
</INTERXFERMSGSETV1>	End of interbank transfer message set version 1
<INTERXFERMSGSETV2>	Zero or more version 2 message sets
<MSGSETCORE>	Common message-set core
</MSGSETCORE>	
<XFERPROF>	Interbank transfer profile, same as XFERPROF in banking, see section 11.13.2.2
</XFERPROF>	
<CANBILLPAY>	Server is capable of handling bill payment as a form of transfers, <i>Boolean</i>
<CANCELWND>	Number of days after an interbank transfer occurs that it can be canceled, <i>N-3</i>
<DOMXFERFEE>	Standard fee for a domestic interbank transfer, <i>amount</i>
<INTLXFERFEE>	Standard fee for an international interbank transfer, <i>amount</i>
<CANMULTI>	Server is capable of handling multiple interbank funds transfer transactions, <MULTIINTERTRNRQ>, <i>Boolean</i>
</INTERXFERMSGSETV2>	End of interbank transfer message set version 2
</INTERXFERMSGSET>	End of interbank transfer message set

11.13.5 Wire Transfer Message Set Profile

Tag	Description
<WIREXFERMSGSET>	Core message set for wire transfers
<WIREXFERMSGSETV1>	Version 1 of message set
<MSGSETCORE>	Common message-set core
</MSGSETCORE>	
<PROCDAYSOFF>	Days of week that no processing occurs: MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, or SUNDAY. 0 or more <PROCDAYSOFF> can be sent.
<PROCENDTM>	Time of day that day's processing ends, <i>time</i>
<CANSCHED>	Supports scheduled transfers, <i>Boolean</i>
<DOMXFERFEE>	Standard fee for a domestic wire transfer, <i>amount</i>
<INTLXFERFEE>	Standard fee for an international wire transfer, <i>amount</i>
</WIREXFERMSGSETV1>	End of wire transfer message set version 1
<WIREXFERMSGSETV2>	Zero or more version 2 message sets
<MSGSETCORE>	Common message-set core
</MSGSETCORE>	
<PROCDAYSOFF>	Days of week that no processing occurs: MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, or SUNDAY. 0 or more <PROCDAYSOFF> can be sent.
<PROCENDTM>	Time of day that day's processing ends, <i>time</i>
<CANSCHED>	Supports scheduled transfers, <i>Boolean</i>
<DOMXFERFEE>	Standard fee for a domestic wire transfer, <i>amount</i>
<INTLXFERFEE>	Standard fee for an international wire transfer, <i>amount</i>
</WIREXFERMSGSETV2>	End of wire transfer message set version 2
</WIREXFERMSGSET>	Ending tag of wire transfer message set

11.14 Examples

11.14.1 Statement Download

This example represents a customer who requests a statement download for a checking account. The request omits <DTSTART> and <DTEND> because the client is interested in getting all available data. The response contains an updated balance for the account and two transactions.

The request file:

```
<OFX>                                <!-- Begin request data -->
  <SIGNONMSGSRQV1>
    <SONRQ>                            <!-- Begin signon -->
      <DTCLIENT>19961029101000        <!-- Oct. 29, 1996, 10:10:00 am -->
      <USERID>123-45-6789              <!-- User ID (that is, SSN) -->
      <USERPASS>MyPassword             <!-- Password (SSL encrypts whole) -->
      <LANGUAGE>ENG                    <!-- Language used for text -->
      <FI>                             <!-- ID of receiving institution -->
        <ORG>NCH                       <!-- Name of ID owner -->
        <FID>1001                     <!-- Actual ID -->
      </FI>
      <APPID>MyApp
      <APPVER>0500
    </SONRQ>                          <!-- End of signon -->
  </SIGNONMSGSRQV1>

  <BANKMSGSRQV1>
    <STMTTRNRQ>                        <!-- Begin request -->
      <TRNUID>1001
      <STMTRQ>                         <!-- Begin statement request -->
        <BANKACCTFROM>
          <BANKID>121099999           <!-- Identify the account -->
          <ACCTID>999988              <!-- Routing transit or other FI ID -->
          <ACCTTYPE>CHECKING          <!-- Account number -->
        </BANKACCTFROM>              <!-- Account type -->
        <INCTRAN>                     <!-- End of account ID -->
          <INCLUDE>Y                  <!-- Begin include transaction -->
        </INCTRAN>                   <!-- Include transactions -->
      </STMTRQ>                       <!-- End of include transaction -->
    </STMTTRNRQ>                     <!-- End of statement request -->
  </BANKMSGSRQV1>                   <!-- End request -->
```

</OFX>	<!-- End of request data -->
The response file:	
<OFX>	<!-- Begin response data -->
<SIGNONMSGSRSV1>	
<SONRS>	<!-- Begin signon -->
<STATUS>	<!-- Begin status aggregate -->
<CODE>0	<!-- OK -->
<SEVERITY>INFO	
</STATUS>	
<DTSERVER>19961029101003	<!-- Oct. 29, 1996, 10:10:03 am -->
<LANGUAGE>ENG	<!-- Language used in response -->
<DTPROFUP>19961029101003	<!-- Last update to profile -->
<DTACCTUP>19961029101003	<!-- Last account update -->
</SONRS>	<!-- End of signon -->
</SIGNONMSGSRSV1>	
<BANKMSGSRSV1>	
<STMTTRNRS>	<!-- Begin response -->
<TRNUID>1001	<!-- Client ID sent in request -->
<STATUS>	<!-- Start status aggregate -->
<CODE>0	<!-- OK -->
<SEVERITY>INFO	
</STATUS>	
<STMTRS>	<!-- Begin statement response -->
<CURDEF>USD	
<BANKACCTFROM>	<!-- Identify the account -->
<BANKID>121099999	<!-- Routing transit or other FI ID -->
<ACCTID>999988	<!-- Account number -->
<ACCTTYPE>CHECKING	<!-- Account type -->
</BANKACCTFROM>	<!-- End of account ID -->
<BANKTRANLIST>	<!-- Begin list of statement trans. -->
<DTSTART>19961001	<!-- Start date: Oct. 1, 1996 -->
<DTEND>19961028	<!-- End date: Oct. 28, 1996 -->
<STMTTRN>	<!-- First statement transaction -->
<TRNTYPE>CHECK	<!-- Check -->
<DTPOSTED>19961004	<!-- Posted on Oct. 4, 1996 -->
<TRNAMT>-200.00	<!-- \$200.00 -->
<FITID>00002	<!-- Unique ID -->
<CHECKNUM>1000	<!-- Check number -->

```

        </STMTRN>                                <!-- End statement transaction -->
        <STMTRN>                                  <!-- Second transaction -->
            <TRNTYPE>ATM                          <!-- ATM transaction -->
            <DTPOSTED>19961020                  <!-- Posted on Oct. 20, 1996 -->
            <DTUSER>19961020                    <!-- User date of Oct. 20, 1996 -->
            <TRNAMT>-300.00                     <!-- $300.00 -->
            <FITID>00003                        <!-- Unique ID -->
        </STMTRN>                                <!-- End statement transaction -->
    </BANKTRANLIST>                             <!-- End list of statement trans. -->
    <LEDGERBAL>                                  <!-- Ledger balance aggregate -->
        <BALAMT>200.29                         <!-- Bal amount: $200.29 -->
        <DTASOF>199610291120                  <!-- Bal date: 10/29/96, 11:20 am -->
    </LEDGERBAL>                                <!-- End ledger balance -->
    <AVAILBAL>                                  <!-- Available balance aggregate -->
        <BALAMT>200.29                         <!-- Bal amount: $200.29 -->
        <DTASOF>199610291120                  <!-- Bal date: 10/29/96, 11:20 am -->
    </AVAILBAL>                                <!-- End available balance -->
</STMTRS>                                      <!-- End statement response -->
</STMTRNRS>                                   <!-- End of transaction -->
</BANKMSGSRV1>
</OFX>                                         <!-- End of response data -->

```

11.14.2 Intrabank Funds Transfer

This example is for a customer who requests an immediate funds transfer of \$200.00 from a checking account to a savings account.

The request file:

```

<OFX>                                           <!-- Begin request data -->
    <SIGNONMSGSRQV1>
        <SONRQ>                                <!-- ...Sign on request.
                                                For a complete example,
                                                see section 11.14.1-->
        </SONRQ>                                <!-- End of signon -->
    </SIGNONMSGSRQV1>

    <BANKMSGSRQV1>
        <INTRATRNRQ>                            <!-- Begin request -->
            <TRNUID>1001                        <!-- Client's ID for this request -->
            <INTRARQ>                          <!-- Begin transfer request -->
                <XFERINFO>                    <!-- Begin transfer aggregate -->
                    <BANKACCTFROM>            <!-- Identify the account -->

```

<BANKID>121099999	<!-- Routing transit or other FI ID -->
<ACCTID>999988	<!-- Account number -->
<ACCTTYPE>CHECKING	<!-- Account type -->
</BANKACCTFROM>	<!-- End of account ID -->
<BANKACCTTO>	<!-- Identify the account -->
<BANKID>121099999	<!-- Routing transit or other FI ID -->
<ACCTID>999977	<!-- Account number -->
<ACCTTYPE>SAVINGS	<!-- Account type -->
</BANKACCTTO>	<!-- End of account ID -->
<TRNAMT>200.00	<!-- Amount of transfer -->
</XFERINFO>	<!-- End of transfer aggregate -->
</INTRARQ>	<!-- End of transfer request -->
</INTRATRNRQ>	<!-- End request -->
</BANKMSGSRQV1>	
</OFX>	<!-- End of request data -->

The response file:

<OFX>	<!-- Begin response data -->
<SIGNONMSGSRSV1>	
<SONRS>	<!-- ...Sign on response. For a complete example, see section 11.14.1-->
</SONRS>	<!-- End of signon -->
</SIGNONMSGSRSV1>	
<BANKMSGSRSV1>	
<INTRATRNR>	<!-- Begin response -->
<TRNUID>1001	<!-- Client ID sent in request -->
<STATUS>	<!-- Start status aggregate -->
<CODE>0	<!-- OK -->
<SEVERITY>INFO	
</STATUS>	
<INTRARS>	<!-- Begin transfer response -->
<CURDEF>USD	
<SRVRTID>1001	<!-- Server assigned ID -->
<XFERINFO>	<!-- Begin transfer aggregate -->
<BANKACCTFROM>	<!-- Identify the account -->
<BANKID>121099999	<!-- Routing transit or other FI ID -->
<ACCTID>999988	<!-- Account number -->

```

        <ACCTTYPE>CHECKING <!-- Account type -->
    </BANKACCTFROM>      <!-- End of account ID -->
    <BANKACCTTO>          <!-- Identify the account -->
        <BANKID>121099999 <!-- Routing transit or other
                           FI ID -->
        <ACCTID>999977    <!-- Account number -->
        <ACCTTYPE>SAVINGS <!-- Account type -->
    </BANKACCTTO>        <!-- End of account ID -->
    <TRNAMT>200.00        <!-- Amount of transfer -->
</XFERINFO>             <!-- End of transfer aggregate -->
    <DTXFERPRJ>19960829100000<!-- Projected posting date -->
</INTRARS>              <!-- End of transfer response -->
</INTRATRNR>           <!-- End response -->
</BANKMSGSRV1>
</OFX>                  <!-- End of response data -->

```

11.14.3 Stop Check

This example represents a customer who requests a stop for checks 200 through 202. The response indicates that the first check (200) has already posted; the server has stopped the rest of the checks in the range.

The request file:

```

<OFX>                    <!-- Begin request data -->
    <SIGNONMSGSRQV1>
        <SONRQ>           <!-- ...Sign on request.
                           For a complete example,
                           see section 11.14.1-->
        </SONRQ>          <!-- End of signon -->
    </SIGNONMSGSRQV1>

    <BANKMSGSRQV1>
        <STPCHKTRNRQ>      <!-- Begin request -->
            <TRNUID>1001    <!-- Client's ID for this request -->
            <STPCHKRQ>      <!-- Begin stop check request -->
                <BANKACCTFROM> <!-- Identify the account -->
                    <BANKID>121099999 <!-- Routing transit or other
                                       FI ID -->
                    <ACCTID>999988    <!-- Account number -->
                    <ACCTTYPE>CHECKING <!-- Account type -->
                </BANKACCTFROM> <!-- End of account ID -->
                <CHKRANGE>      <!-- Cancel a range of checks -->
                    <CHKNUMSTART>200 <!-- Starting check number -->

```


<CHKNUMEND>202	<!-- Ending check number -->
</CHKRANGE>	<!-- End range -->
</STPCHKRQ>	<!-- End of stop check request -->
</STPCHKTRNRQ>	<!-- End request -->
</BANKMSGSRQV1>	
</OFX>	<!-- End of request data -->

The response file:

<OFX>	<!-- Begin response data -->
<SIGNONMSGSRSV1>	
<SONRS>	<!-- ...Sign on response. For a complete example, see section 11.14.1-->
</SONRS>	<!-- End of signon -->
</SIGNONMSGSRSV1>	
<BANKMSGSRSV1>	
<STPCHKTRNRS>	<!-- Begin response -->
<TRNUID>1001	<!-- Client ID sent in request -->
<STATUS>	<!-- Begin status aggregate -->
<CODE>0	<!-- OK -->
<SEVERITY>INFO	
</STATUS>	<!-- End of status aggregate -->
<STPCHKRS>	<!-- Begin stop check response -->
<CURDEF>USD	
<BANKACCTFROM>	<!-- Identify the account -->
<BANKID>121099999	<!-- Routing transit or other FI ID -->
<ACCTID>999988	<!-- Account number -->
<ACCTTYPE>CHECKING	<!-- Account type -->
</BANKACCTFROM>	<!-- End of account ID -->
<STPCHKNUM>	<!-- First stopped check -->
<CHECKNUM>200	<!-- Check 200 -->
<CHKSTATUS>101	<!-- Too late - already posted -->
</STPCHKNUM>	<!-- End of first stopped check -->
<STPCHKNUM>	<!-- Second stopped check -->
<CHECKNUM>201	<!-- Check 201 -->
<CHKSTATUS>0	<!-- OK -->
</STPCHKNUM>	<!-- End of second stopped check -->
<STPCHKNUM>	<!-- Third stopped check -->
<CHECKNUM>202	<!-- Check 202 -->
<CHKSTATUS>0	<!-- OK -->

```
        </STPCHKNUM>                <!-- End of third stopped check -->
        <FEE>10.00
        <FEEMSG>Fee for stop payment.
    </STPCHKRS>                    <!-- End stop check response -->
    </STPCHKTRNRS>                <!-- End of transaction -->
    </BANKMSGSRSV1>
</OFX>                            <!-- End of response data -->
```

11.14.4 Recurring Transfers

This example represents a customer who creates a transfer model and then cancels it. To follow the life of the model (and the transfers it creates), the example includes sessions that occur over a two month period.

The model is added on November 1 and scheduled to start on November 15. The model creates transfers of \$1000 from a checking to a savings account. The schedule is open-ended.

Because requests within a message set are not guaranteed to be executed in order, the client initially sends two request files: one to create the model and another to collect any transfers generated by the model. The second request file contains a simple transfer synchronization request.

The client sends the file to create the model on November 1:

```
<OFX>                                <!-- Begin request data -->
  <SIGNONMSGSRQV1>
    <SONRQ>                           <!-- ...Sign on request.
                                      For a complete example,
                                      see section 11.14.1-->
    </SONRQ>                           <!-- End of signon -->
  </SIGNONMSGSRQV1>

  <BANKMSGSRQV1>
    <RECINTRATRNRQ>                   <!-- Begin request -->
      <TRNUID>1001                     <!-- Client's ID for this request -->
      <RECINTRARQ>                     <!-- Begin request -->
        <RECURRINST>                  <!-- Begin recurring aggregate -->
          <FREQ>MONTHLY                <!-- Monthly schedule -->
        </RECURRINST>                 <!-- End recur aggregate -->
        <INTRARQ>
          <XFERINFO>                  <!-- Begin transfer aggregate -->
            <BANKACCTFROM>             <!-- Identify the account -->
              <BANKID>121099999        <!-- Routing transit or other
                                      FI ID -->
              <ACCTID>999988           <!-- Account number -->
              <ACCTTYPE>CHECKING        <!-- Account type -->
            </BANKACCTFROM>            <!-- End account ID -->
            <BANKACCTTO>               <!-- Identify the account -->
              <BANKID>121099999        <!-- Routing transit or other
                                      FI ID -->
              <ACCTID>999977           <!-- Account number -->
              <ACCTTYPE>SAVINGS         <!-- Account type -->
            </BANKACCTTO>              <!-- End of account ID -->
```

```

        <TRNAMT>1000.00      <!-- Amount of transfer-->
        <DTDUE>19981115    <!-- First transfer - Nov.15 -->
    </XFERINFO>            <!-- End transfer aggregate -->
    </INTRARQ>
    </RECINTRARQ>          <!-- End transfer request -->
    </RECINTRATRNRQ>       <!-- End request -->
    </BANKMSGSRQV1>
</OFX>

```

The response file shows that the model has been successfully created:

```

<OFX>                                <!-- Begin response data -->
    <SIGNONMSGSRSV1>
        <SONRS>                    <!-- ...Sign on response.
                                   For a complete example,
                                   see section 11.14.1-->

        </SONRS>                  <!-- End of signon -->
    </SIGNONMSGSRSV1>

    <BANKMSGSRSV1>
        <RECINTRATRNR>            <!-- Begin response -->
            <TRNUID>1001          <!-- Client ID sent in request -->
            <STATUS>              <!-- Start of status aggregate -->
                <CODE>0           <!-- OK -->
                <SEVERITY>INFO
            </STATUS>
            <RECINTRARS>          <!-- Begin response -->
                <RECSRVRTID>20000 <!-- Server assigned ID -->
                <RECURRINST>      <!-- Begin recurring aggregate -->
                    <FREQ>MONTHLY <!-- Monthly schedule -->
                </RECURRINST>     <!-- End of recurring aggregate -->
                <INTRARS>
                    <CURDEF>USD
                    <SRVRTID>120000
                <XFERINFO>        <!-- Begin transfer aggregate -->
                    <BANKACCTFROM> <!-- Identify the account -->
                    <BANKID>121099999 <!-- Routing transit or other
                                       FI ID -->
                    <ACCTID>999988   <!-- Account number -->
                    <ACCTTYPE>CHECKING <!-- Account type -->
                    </BANKACCTFROM> <!-- End of account ID -->
                    <BANKACCTTO>    <!-- Identify the account -->
                    <BANKID>121099999 <!-- Routing transit or other
                                       FI ID -->

```

```

        <ACCTID>999977      <!-- Account number -->
        <ACCTTYPE>SAVINGS  <!-- Account type -->
        </BANKACCTTO>      <!-- End of account ID -->
        <TRNAMT>1000.00    <!-- Amount of transfer -->
        <DTDUE>19981115    <!-- First transfer - Nov. 15 -->
    </XFERINFO>            <!-- End of transfer aggregate -->
    </INTRARS>
    </RECINTRARS>          <!-- End of response -->
    </RECINTRATRNR>        <!-- End of response -->
    </BANKMSGSRQV1>
</OFX>                    <!-- End of response data -->

```

The client sends the payment synchronization request later on November 1:

```

<OFX>                      <!-- Begin request data -->
    <SIGNONMSGSRQV1>
        <SONRQ>              <!-- ...Sign on request.
                               For a complete example,
                               see section 11.14.1-->
        </SONRQ>              <!-- End of signon -->
    </SIGNONMSGSRQV1>
    <BANKMSGSRQV1>
        <INTRASYNCRQ>        <!-- Sync intrabank transfers -->
            <TOKEN>0          <!-- Token held by client -->
            <REJECTIFMISSING>N
                <BANKACCTFROM> <!-- Identify the account -->
                    <BANKID>121099999
                    <ACCTID>999988
                    <ACCTTYPE>CHECKING
                </BANKACCTFROM>
            </INTRASYNCRQ>    <!-- End of sync request -->
        </BANKMSGSRQV1>
</OFX>                      <!-- End of request data -->

```

Assuming that the server creates transfers 30 days prior to posting, the server returns status for one pending transfer. This response comes back since the first transfer is scheduled to occur on November 15 and this date falls within 30 days of our session. Had the starting date been more than 30 days from our signon date, the response would not have contained any pending transfers since the model would not have generated any yet.

The response file from the server shows one pending transfer:

```
<OFX>                                <!-- Begin response data -->
  <SIGNONMSGSRSV1>
    <SONRQ>                            <!-- ...Sign on request.
                                        For a complete example,
                                        see section 11.14.1-->

    </SONRQ>                            <!-- End of signon -->
  </SIGNONMSGSRSV1>
  <BANKMSGSRSV1>
    <INTRASYNCRS>                        <!-- Sync intrabank transfers -->
      <TOKEN> 22243                      <!-- Token updated -->
      <BANKACCTFROM>                    <!-- Identify the account -->
        <BANKID>121099999
        <ACCTID>999988
        <ACCTTYPE>CHECKING
      </BANKACCTFROM>
      <INTRATRNR>                        <!-- begin response -->
        <TRNUID>0                       <!-- Server generated, so 0-->
        <STATUS>                         <!-- Success -->
          <CODE>0                        <!-- OK -->
          <SEVERITY>INFO
        </STATUS>
      <INTRARS>                          <!-- Begin transfer response -->
        <CURDEF>USD
        <SRVRTID>100100000 <!-- Server assigned ID -->
        <XFERINFO>                    <!-- Begin transfer aggregate -->
          <BANKACCTFROM>                <!-- Identify the account -->
            <BANKID>121099999 <!-- Routing transit or
                                other FI ID -->
            <ACCTID>999988 <!-- Account number -->
            <ACCTTYPE>CHECKING <!-- Account type -->
          </BANKACCTFROM>                <!-- End of account ID -->
          <BANKACCTTO>                  <!-- Identify the account -->
            <BANKID>121099999 <!-- Routing transit or
                                other FI ID -->
            <ACCTID>999977 <!-- Account number -->
            <ACCTTYPE>SAVINGS <!-- Account type -->
          </BANKACCTTO>                  <!-- End of account ID -->
          <TRNAMT>1000.00 <!-- Amount of transfer -->
        </XFERINFO>                    <!-- End transfer aggregate -->
        <DTXFERPRJ>19981115 <!-- Projected date of the
                                transfer -->
        <RECSRVRTID>20000 <!-- Model that created this xfer -->
```

</INTRARS>	<!-- End of transfer response -->
</INTRATRNR>	<!-- End of response -->
</INTRASYNCRS>	<!-- End of sync response -->
</BANKMSGSRV1>	
</OFX>	<!-- End of response data -->

Suppose the customer does not attempt to connect between November 16 and January 1. When the customer does attempt to connect, it is to cancel the recurring transfer model. The client also sets the <CANPENDING> flag, causing any pending transfers to be immediately cancelled as well. In order to get all synchronization information (since requests are not guaranteed to be executed in order), the client sends two request files, the first to cancel the model and the next to retrieve all transfer activity. This time, the recurring request is wrapped in synchronization wrappers. It should be assumed that the token below was received in a previous RECPMTSYNCRS. (The use of synchronization wrappers in requests is entirely up to the client. Both ways are shown here for explanatory purposes.)

The request file:

```
<OFX>                                <!-- Begin request data -->
  <SIGNONMSGSRQV1>
    <SONRQ>                           <!-- ...Sign on request.
                                      For a complete example,
                                      see section 11.14.1-->
    </SONRQ>                           <!-- End of signon -->
  </SIGNONMSGSRQV1>

  <BANKMSGSRQV1>
    <RECINTRASYNCRQ>                  <!-- Sync recurring transfers -->
      <TOKEN>324789987                <!-- Token held by the client -->
      <REJECTIFMISSING>Y              <!-- Cancel only if up to date -->
      <BANKACCTFROM>
        <BANKID>121099999
        <ACCTID>99998
        <ACCTTYPE>CHECKING
      </BANKACCTFROM>
      <RECINTRATRNRQ>                 <!-- Begin request -->
        <TRNUID>1005                  <!-- Client's ID for this request -->
        <RECINTRACANRQ>              <!-- Begin recur transfer cancel -->
          <RECSRVRTID>20000           <!-- ID of the model -->
          <CANPENDING>Y               <!-- Cancel pending transfers -->
        </RECINTRACANRQ>             <!-- End request -->
      </RECINTRATRNRQ>               <!-- End request -->
    </RECINTRASYNCRQ>               <!-- End of sync request -->
  </BANKMSGSRQV1>
</OFX>                                <!-- End of request data -->
```

The response file:

```
<OFX>                                <!-- Begin response data -->
  <SIGNONMSGSRSV1>
    <SONRS>                           <!-- ...Sign on response.
                                      For a complete example,
                                      see section 11.14.1-->
    </SONRS>                           <!-- End of signon -->
  </SIGNONMSGSRSV1>

  <BANKMSGSRSV1>
    <RECINTRASYNCRS>                  <!-- Sync response -->
      <TOKEN>324789988                <!-- New token -->
      <BANKACCTFROM>
```



```

        <BANKID>121099999
        <ACCTID>99998
        <ACCTTYPE>CHECKING
    </BANKACCTFROM>
    <RECINTRATRNRS>                                <!-- Begin response -->
        <TRNUID>1005                                <!-- Client ID sent in request -->
        <STATUS>                                     <!-- Start of status aggregate -->
            <CODE>0                                  <!-- OK -->
            <SEVERITY>INFO
        </STATUS>
        <RECINTRACANRS>                             <!-- Begin cancel model -->
            <RECSRVRTID>20000                       <!-- Model that was canceled -->
            <CANPENDING>Y
        </RECINTRACANRS>                           <!-- End of cancel model -->
    </RECINTRATRNRS>                                <!-- End response -->
    </RECINTRASYNCRS>                              <!-- End sync response -->
</BANKMSGSRQV1>
</OFX>                                             <!-- End response -->

```

Next request file:

```

<OFX>                                             <!-- Begin request data -->
    <SIGNONMSGSRQV1>
        <SONRQ>                                    <!-- ...Sign on request.
                                                    For a complete example,
                                                    see section 11.14.1-->
        </SONRQ>                                    <!-- End of signon -->
    </SIGNONMSGSRQV1>
    <BANKMSGSRQV1>
        <INTRASYNCRQ>                              <!-- Sync intrabank transfers -->
            <TOKEN>22243                            <!-- Token held by the client -->
            <REJECTIFMISSING>N
            <BANKACCTFROM>
                <BANKID>121099999
                <ACCTID>99998
                <ACCTTYPE>CHECKING
            </BANKACCTFROM>
        </INTRASYNCRQ>                              <!-- End of sync request -->
    </BANKMSGSRQV1>
</OFX>                                             <!-- End of request data -->

```

Since the customer last connected, the November 15 transfer has posted, the December 15 transfer has been scheduled, the December 15 transfer has posted and a transfer has been scheduled for January 15. The response file shows these four transfer responses and the

cancellation response for the January 15 transfer. Note that servers are not required to show the post of transfers via a transfer modification response in the sync. Alternatively, a client may need to note that the transfer happened in a subsequent statement download.

The response file:

```
<OFX>                                <!-- Begin response data -->
  <SIGNONMSGSRSV1>
    <SONRS>                            <!-- ...Sign on response.
                                      For a complete example,
                                      see section 11.14.1-->

    </SONRS>                            <!-- End of signon -->
  </SIGNONMSGSRSV1>
  <BANKMSGSRSV1>
    <INTRASYNCRS>
      <TOKEN> 22244                    <!-- New token -->
      <BANKACCTFROM>
        <BANKID>121099999
        <ACCTID>99998
        <ACCTTYPE>CHECKING
      </BANKACCTFROM>
      <INTRATRNRS>                    <!-- 11/15 post -->
        <TRNUID>0                     <!-- Server generated, so 0 -->
        <STATUS>
          <CODE>0                     <!-- OK -->
          <SEVERITY>INFO
        </STATUS>
        <INTRAMODRS>                  <!-- This is the Nov. 15 post -->
          <SRVRTID>100100000           <!-- Server assigned ID -->
          <XFERINFO>                  <!-- Begin transfer aggregate -->
            <BANKACCTFROM>            <!-- Identify the account -->
              <BANKID>121099999       <!-- Routing transit or other
                                      FI ID -->
              <ACCTID>99998           <!-- Account number -->
              <ACCTTYPE>CHECKING      <!-- Account type -->
            </BANKACCTFROM>           <!-- End of account ID -->
            <BANKACCTTO>              <!-- Identify the account -->
              <BANKID>121099999       <!-- Routing transit or other
                                      FI ID -->
              <ACCTID>999977          <!-- Account number -->
              <ACCTTYPE>SAVINGS       <!-- Account type -->
            </BANKACCTTO>             <!-- End of account ID -->
            <TRNAMT>1000.00           <!-- Amount of transfer -->
          </XFERINFO>                <!-- End of transfer aggregate -->
```

```

        <XFERPRCSTS>                <!-- Status of transfer -->
        <XFERPRCCODE>POSTEDON<!-- Status code -->
        <DTXFERPRC>19981115<!-- Date transfer was posted -->
    </XFERPRCSTS>                <!-- End of transfer status -->
</INTRAMODRS>                <!-- End of Nov. 15 post -->
</INTRATRNR>                <!-- End of response -->

<INTRATRNR>                <!--12/15 pending transfer -->
    <TRNUID>0
    <STATUS>                <!-- Success -->
        <CODE>0                <!-- OK -->
        <SEVERITY>INFO
    </STATUS>
    <INTRARS>                <!-- This is the Dec. 15 pending -->
        <CURDEF>USD
        <SRVRTID>112233        <!-- Server assigned ID -->
        <XFERINFO>            <!-- Begin transfer aggregate -->
            <BANKACCTFROM>        <!-- Identify the account -->
                <BANKID>121099999 <!-- Routing transit or other
                                FI ID -->
                <ACCTID>999988    <!-- Account number -->
                <ACCTTYPE>CHECKING <!-- Account type -->
            </BANKACCTFROM>        <!-- End of account ID -->
            <BANKACCTTO>          <!-- Identify the account -->
                <BANKID>121099999 <!-- Routing transit or other
                                FI ID -->
                <ACCTID>999977    <!-- Account number -->
                <ACCTTYPE>SAVINGS <!-- Account type -->
            </BANKACCTTO>          <!-- End of account ID -->
            <TRNAMT>1000.00        <!-- Amount of transfer -->
        </XFERINFO>            <!-- End of transfer aggregate -->
        <DTXFERPRJ>19981215    <!-- Projected date of the
                                transfer -->
        <RECSRVRTID>20000        <!-- Model -->
    </INTRARS>                <!-- End of Dec. 15 pending -->
</INTRATRNR>                <!-- End response -->
<INTRATRNR>                <!-- 12/15 post -->
    <TRNUID>0                <!-- Client ID sent in request -->
    <STATUS>
        <CODE>0                <!-- OK -->
        <SEVERITY>INFO
    </STATUS>

```

```

<INTRAMODRS>                                <!-- This is the Dec. 15 post -->
  <SRVRTID>112233                            <!-- Server assigned ID -->
  <XFERINFO>                                <!-- Begin transfer aggregate -->
    <BANKACCTFROM>                          <!-- Identify the account -->
      <BANKID>121099999                    <!-- Routing transit or other
                                          FI ID -->
      <ACCTID>999988                      <!-- Account number -->
      <ACCTTYPE>CHECKING                  <!-- Account type -->
    </BANKACCTFROM>                        <!-- End of account ID -->
    <BANKACCTTO>                          <!-- Identify the account -->
      <BANKID>121099999                    <!-- Routing transit or other
                                          FI ID -->
      <ACCTID>999977                      <!-- Account number -->
      <ACCTTYPE>SAVINGS                  <!-- Account type -->
    </BANKACCTTO>                        <!-- End of account ID -->
    <TRNAMT>1000.00                      <!-- Amount of transfer -->
  </XFERINFO>                              <!-- End of transfer aggregate -->
  <XFERPRCSTS>                            <!-- Status of transfer -->
    <XFERPRCCODE>POSTEDON                <!-- Status code -->
    <DTXFERPRC>19981215                  <!-- Date transfer was posted -->
  </XFERPRCSTS>                          <!-- End of transfer status -->
</INTRAMODRS>                              <!-- End of Dec. 15 post -->
</INTRATRNRNS>                            <!-- End of response -->

<INTRATRNRNS>                              <!--This is the 1/15 pending -->
  <TRNUID>0                                <!-- Client ID sent in request -->
  <STATUS>
    <CODE>0                                <!-- OK -->
    <SEVERITY>INFO
  </STATUS>
  <INTRARS>                                <!-- This is the Jan. 15 pending -->
    <CURDEF>USD
    <SRVRTID>112255                        <!-- Server assigned ID -->
    <XFERINFO>                            <!-- Begin transfer aggregate -->
      <BANKACCTFROM>                      <!-- Identify the account -->
        <BANKID>121099999                <!-- Routing transit or other
                                          FI ID -->
        <ACCTID>999988                    <!-- Account number -->
        <ACCTTYPE>CHECKING                <!-- Account type -->
      </BANKACCTFROM>                    <!-- End of account ID -->
      <BANKACCTTO>                      <!-- Identify the account -->
        <BANKID>121099999                <!-- Routing transit or other
                                          FI ID -->

```

<ACCTID>999977	<!-- Account number -->
<ACCTTYPE>SAVINGS	<!-- Account type -->
</BANKACCTT0>	<!-- End of account ID -->
<TRNAMT>1000.00	<!-- Amount of transfer -->
</XFERINFO>	<!-- End of transfer aggregate -->
<DTXFERPRJ>19990115	<!-- Projected date of transfer -->
<RECSRVRTID>20000	<!-- Model -->
</INTRARS>	<!-- End of Jan. 15 pending -->
</INTRATRNR>	<!-- Cancellation of 1/15 pending-->
<INTRATRNR>	<!-- response -->
<TRNUID>0	<!-- Client ID sent in this request -->
<STATUS>	
<CODE>0	<!-- OK -->
<SEVERITY>INFO	
</STATUS>	
<INTRACANRS>	<!-- This is the Jan. 15 cancel -->
<SRVRTID>11225	<!-- Server ID for Jan. 15 xfer -->
</INTRACANRS>	<!-- End of Jan. 15 cancel -->
</INTRATRNR>	<!-- End of response -->
</INTRASYNCR>	<!-- End of sync response -->
</BANKMSGSRV1>	
</OFX>	<!-- End of response -->

CHAPTER 12 PAYMENTS

This section describes the Payments portion of OFX. OFX Payments consists of a set of functions for scheduling and maintaining payment transactions, and for synchronizing with the server to obtain an accurate status of all recent and scheduled transactions.

Clients use payment requests to schedule payments and to modify or delete payments if necessary. OFX also supports business payments, as described in section [12.1](#).

The recurring payments function allows the client to schedule automatic generation of a series of recurring payments by means of a single request. As with individual payments, the client can modify or delete these requests.

The payments function incorporates the synchronization features of OFX, allowing multiple client applications to synchronize with the server to obtain the current status of all payment transactions known to the server.

In many international environments, payments are performed using interbank funds transfers. OFX Payments supports this by allowing a payee to be designated as a destination bank account. Servers can implement these messages as transfers where appropriate.

12.1 Consumer and Business Payments

OFX Payments is designed to support both consumer and business payments. Businesses have additional requirements for payments. In particular, there is a need to include itemized instructions that specify how a payment should be disbursed across multiple invoices and / or line items. OFX supports this requirement through the inclusion of the <EXTDPMT> aggregate within payment requests. The Payment Modification Request <PMTMODRQ> also supports changes to <EXTDPMT> data.

12.2 The Payee Model

The payee model in OFX is designed to provide support for both “pay-some” and “pay-any” payment systems. “Pay-some” systems are those that restrict users to only make payments to payees that appear on an approved list. Such payees are often referred to as “standard payees,” or “standard merchants.” These are generally larger corporations that receive high volumes of payments, such as telephone companies or power utilities. In contrast, “pay-any” systems allow payments to any payee for which the user provides accurate billing information. These systems often also include a list of standard payees.

12.2.1 Payee Identifiers

OFX is designed to be flexible in the requirements for payee identifiers. It supports systems where all, some, or no payees are assigned a payee ID. In addition, it enables the server to assign an ID to a payee that was previously being paid by billing address.

You must implement the scope of payee such that the ID is at least global across the user's set of payments-enabled accounts with the payments provider. For example, if the user has both a checking and a money market account enabled for payments with the payments provider, then a payee ID obtained for a payment made from one of these accounts should identify the same payee if used for a payment drawn on the other account. This simplifies client support for allowing a user to choose from which account to make a payment.

OFX requires payee identifiers to have a one-to-one relationship with the corresponding <PAYEE> information. In other words, different payee IDs must also differ in their corresponding payee billing description or payee name <NAME>. Similarly, a payee ID must be independent of a user's account number with the payee. However, the payment system is free to use the user's account number in combination with the payee ID to determine the routing of a payment. These rules are intended to simplify the payee model for the user, insuring that different payee IDs will have discernibly different descriptions associated with them. They also insure that the user will not be required to maintain multiple payee entries for a payee with which the user holds multiple accounts.

OFX includes an element for indicating the scope of a payee ID returned from the server. This allows clients to adapt by expanding or restricting their functionality depending on the scope of payee IDs it encounters.

A payee list for each user, maintained on the server, allows the server to manage the identifiers assigned to a user's payees. This functionality is described in section [12.2.2](#).

12.2.2 Payee Lists

OFX specifies that a server-hosted payee list is maintained for each payments user. This list contains the payees that a user has paid through the payment system, or has set up to pay. Updates to this list are available through the synchronization mechanism. This insures that multiple clients have access to the full list of payees the user has configured. It is only necessary to enter each payee once.

Some payment systems require a first time setup before using a payee. This can occur externally to the client and server software, for example by filling out a paper form or telephoning the bank. In this case, payee list synchronization provides a way for the payee to become accessible to the client software when the FI completes the setup.

The list can contain payees with or without payee IDs. An important function of the payee list is to communicate payee changes from the server to the client. This includes changes in processing date parameters and conversion of a payee to a standard payee.

Once added to the list, the payment system makes payments by the payee list ID. This makes it clear to a client when the user is adding to a payee list, and when he or she modifies an existing payee on the list.

Although the messages make it clear whether a client is trying to add a new payee, a careful server will check for exact matches on payee adds and not create new payee list entries unnecessarily.

“Pay-any” systems can perform background processing that matches billing addresses with standard payees. When this occurs, the server can update the relevant payee lists and update the clients when they synchronize with the modified list data.

Each payee entry in the list can also include a list of the user’s accounts with that payee. This further reduces the data entry required by a user to make a payment, and facilitates the implementation of lightweight clients.

12.2.3 Standard Payee Lists

Many payment systems maintain a list of payees that receive payments from a large number of users. Payments to these payees are usually consolidated into a few electronic funds transfers or are mailed in large batches to the payee. Payees that receive this special processing are generally referred to as standard payees. In a “pay-some” system, all the approved payees can be considered to be standard payees. When a user pays a standard payee, there might be different processing lead-times used to calculate the payment and / or processing date of a payment.

When a payment system includes a standard payee list, it might be desirable to present the list to the user, who can then select payees he or she wants to pay. Unfortunately, it is cumbersome to provide this functionality in the client software due to the potential size of this list, which makes it problematic to keep updated and to present to the user. While the list can contain thousands of payee entries, a user will typically need less than ten or twenty entries from the list. It can also be difficult for a user to choose the correct payee entry when the list contains a number of similarly named payees.

Therefore, OFX does not provide a mechanism for delivering these lists to the client. However, there are several ways that an external presentation of such a list can be integrated into the client or server. For example, a payment provider’s Web site could present a search engine that assists the user to locate the correct payee. Once identified, the payees can either be imported into the client, or inserted into the user’s payee list on the server. In the latter case, synchronizing the payee list will make the newly added payees visible to the client.

12.2.4 Identifying Payees

Payees can be identified in several ways:

- ◆ Name and address, by means of <PAYEE>, must be identified only once for each payee. Thereafter, clients must use the assigned <PAYEELSTID> and, if assigned, <PAYEEID>. If the clients send both <PAYEE> and <PAYEELSTID> in a <PMTRQ>, <PMTMODRQ>, <RECPMTRQ>, or <RECPMTMODRQ>, the client is making an implicit payee modification request.

In <BILLPAYMSGSRV1>, the server must return a <PAYEEMODRS> in a subsequent <PAYEESYNCRS> for all actual changes. This is not necessary (though still allowed) if no change were made. For <BILLPAYMSGSRQV2>, the <PAYEE> aggregate is no longer required. Its presence always indicates a request for an implicit payee addition or modification.

If a client sends just <PAYEE> in a <PMTRQ> or <RECPMTRQ>, the client is making an implicit payee add request. (Clients must include the known <PAYEELSTID> in a <PMTMODRQ> or <RECPMTMODRQ>.) For more information about implicit payee adds and modifications, see section [12.2.5](#).

- ◆ Destination bank account <BANKACCTTO> should be done only once for each payee. Thereafter, clients should use the assigned <PAYEELSTID> or <PAYEEID>, as with name and address payees. The <PAYEE> aggregate is required to provide name and address information as a backup to account transfers.
- ◆ Payee list ID <PAYEELSTID> after a payee has been added to the list. For payees that have not been assigned a standard payee ID <PAYEEID>, the <PAYEELSTID> is enough to identify the payee in version 2 of the message set.

Note: Duplicate payee list entries can occur if clients are not careful to send the payee list ID in subsequent requests.

- ◆ Standard payee ID <PAYEEID> for any payee that has been assigned a standard payee ID. This could happen before a closed system makes any payments, or anytime after the server has notified the client that a payee has a standard payee ID. If a <PAYEELSTID> also exists for the payee, it is required in the request and response, in addition to the <PAYEEID>.

Note: Servers must always assign <PAYEELSTID>s to payees. Once <PAYEELSTID>s have been assigned, clients must always send the <PAYEELSTID>, even if a payee has both a <PAYEEID> and a <PAYEELSTID>.

12.2.5 Side Effects of Payee Adds and Modifications

Payees are added either implicitly or explicitly. Explicit adds occur by executing a <PAYEERQ>. Implicit payee adds occur with the execution of a <PMTRQ> or <RECPMTRQ> where a payee list ID is not sent with the request. (Thus, duplicate payee list entries can occur if clients are not careful to send the payee list ID if it is known.) In the case of an implicit payee add, a server must create and store a <PAYEERS> to be returned to the client in subsequent payee synchronization responses. Since the change was not generated by an explicit request, the <TRNUID> in this response would be zero.

Payees are modified either implicitly or explicitly. Explicit changes occur by executing a <PAYEEMODRQ>. Implicit payee changes occur with the execution of a <PMTRQ>, <PMTMODRQ>, <RECPMTRQ>, or <RECPMTMODRQ>, if the payee list ID is sent along with the payee aggregate. In <BILLPAYMSGSETV1>, a <PAYEE> aggregate must accompany these requests. In such cases, since the <PAYEELSTID> is present, a server may check to see if the payee information has changed, and only if so, process an implicit payee modification. For <BILLPAYMSGSETV2>, if both a <PAYEE> aggregate and <PAYEELSTID2> are present, this indicates an automatic payee modification, since the <PAYEE> aggregate is not required. In both these cases, an implicit payee change must cause the server to create and store a <PAYEEMODRS>, to be returned to the client in subsequent payee synchronization responses. Since the change was not generated by an explicit request, the <TRNUID> in these responses will be zero.

In addition to the above, a payee change (implicit or otherwise) may also affect models and their future (though not pending) payments. Thus, for any model that is affected by an explicit or implicit payee modification, the server must create and store a <RECPMTMODRS>, to be returned to the client in subsequent recurring payment synchronization responses. The <TRNUID> in this response will be zero.

12.3 Identifiers Used in Payment Transactions

Payment transactions use four types of identifiers. It is important to understand the purpose, scope, and life span of these identifiers.

The client-to-request messages assign the Transaction Universal Identifier <TRNUID>. Its purpose is to allow the client to easily match responses from the server to their corresponding requests. A given transaction ID is used only for a client request and the corresponding server response.

The Server Identifier, <SRVRTID> or <RECSRVRTID>, is assigned by the server to a payment “object,” which can either be a payment or a recurring payment model (in which case it is named <RECSRVRTID>). Both the client and server use the ID thereafter to refer to the payment or model in any transactions that operate on them. For example, the <SRVRTID> is used to identify a payment in a request to modify or cancel it. The <SRVRTID> is valid for the life span of the

payment within the payment system. Similarly the <RECSRVRTID> is valid as long as the associated model exists, that is until the model generates all payments, or the model is canceled. Once a server processes a payment or a model generates all its required payments, the associated <SRVRTID> (or <RECSRVRTID>) is no longer known to the server. Note that the payment system might continue to maintain knowledge of a payment <SRVRTID> or model <RECSRVRTID> for some specified period after it completes processing. This allows clients to access the “completed” status of these operations.

A payment system can assign the Payee Identifier <PAYEEID> to a payee. There is no requirement that all or any payees are assigned a <PAYEEID>. The usage of this identifier will vary by payment system. For example, in “pay-some” systems usually every payee has a payee ID with a scope that is known globally, while in “pay-any” systems there might only be <PAYEEID>s assigned to standard payees. When a payee has an assigned <PAYEEID>, the life span of the ID will depend on its scope. If the scope is global, such as for payees in some “pay-some” systems or those with standard payees, then the <PAYEEID> is expected to be valid as long as that payee is identifiable by ID. If the payee ID is user-specific in scope, then the <PAYEEID> is valid as long as the payee appears in the user’s server-hosted payee list.

The Payee List Identifier <PAYEELSTID> is assigned by the server to each entry in a user’s server-hosted payee list. The need for this identifier is to support the variety of payee models employed in various payment systems. As discussed above, some payment systems assign a payee identifier <PAYEEID> to every payee (this is particularly the case with pay-some systems); others assign <PAYEEID>s only to standard payees. There are also systems that cannot map a payee billing address to a <PAYEEID> in real time. Also, there are systems that can convert a payee from a standard payee with an assigned <PAYEEID> to one that is identified only by billing address. Therefore, systems employ the <PAYEELSTID> to insure that, in systems where payees will not always have a <PAYEEID>, there is another identifier that can be used to reference every payee. This insures that a client can correctly link payments to their payees. The <PAYEELSTID> must be valid as long as the user’s payee list includes the payee it identifies, even if the server subsequently assigns a <PAYEEID> to the payee. In order to ensure that <PAYEELSTID>s are unambiguous to the client, <PAYEELSTID> must be unique for all classes of a particular <SPNAME> and <USERID>. Therefore, a given payment provider may use <PAYEELSTID>12345 to refer to ABC Rentals for one <USERID>, and XYZ Cable for a different <USERID>. Likewise, a client cannot assume that <PAYEELSTID>54321 at payment provider 1 will refer to the same payee as <PAYEELSTID>54321 at payment provider 2.

12.4 The Payment Life Cycle

12.4.1 Payment Creation

The client formulates a <PMTRQ> that includes the payee, the date, the amount of the payment, the funding account, and the user's account number with the payee. If supported by the user's payment system, the billing address can specify the payee.

The server will look up the payee in the user's payee list. If it is not already in the table, the server will add it and issue a payee list identifier <PAYEELSTID>. This form of payment request performs an implicit Payee Request <PAYEERQ>, which is equivalent to explicitly adding the payee (by means of a <PAYEERQ>), prior to issuing the <PMTRQ>. It has the advantage of being atomic. If the payment request fails, the payee is not added to the user's payee list. Conversely the payment request will fail if the payee information is invalid.

The server responds to the <PMTRQ> with a Payment Response <PMTRS>. Some servers will not be able to immediately return a payee ID at this point, or might not issue payee IDs for all payees. Therefore the <PAYEELSTID> contained in the response functions as the linkage between the payee and the payment. Payment systems use the <SRVRTID> returned in the <PMTRS> to identify the payment for the length of its instantiation on the payment system.

Note: Servers should generate explicit responses to implicit requests. In other words, implicit payee additions or modifications resulting from a new or changed payment should generate explicit payee add or payee change responses from the server. Such explicit responses are only returned to the client in a SYNC response. If the payment transactions containing implicit payee additions or modifications fail, then the payee actions are not executed, since such a compound payment transaction represents a single unit of work (comprised of both payee and payment actions).

12.4.2 Payment Modification

Between the time the client schedules a payment and the time the server processes the payment, the client can request changes to the parameters of that payment. For example, the amount or date of the payment can be modified. The system uses the Payment Modification Request <PMTMODRQ> for this purpose, where the <SRVRTID> from the <PMTRS> identifies the targeted payment. The user request must specify the full contents of the payment request, including both modified and unmodified data. However, the <PAYEELSTID> is enough to identify the payee in the <PMTMODRQ> with version 2 of the message set.

Full-featured servers will use <PMTMODRS> messages, conveyed to the client during synchronization <PMTSYNCRS>, to inform the client about changes in the state of the client that occur due to server processing. This would include reporting the date the server actually processed a payment, or it failed due to insufficient funds. Servers that are unable to generate <PMTMODRS> responses for this purpose must support the <PMTINQRQ> message described below.

12.4.3 Payment Status Inquiry

As a scheduled payment progresses through its “life-cycle” on the server, the processing status changes accordingly from “scheduled to be processed” to “was processed” or “failed processing.” A processing date is associated with these states. The preferred method for providing updated processing status to a client is by use of server-generated Payment Modification messages <PMTMODRS>, as discussed above. However it is possible that less full-featured servers might have difficulty in implementing this form of notification. In this case, OFX requires such servers to implement the Payment Status Inquiry message <PMTINQRQ>, which provides an interface for the client to explicitly request the processing status of individual payments.

12.4.4 Payment Cancellation

In the interval between successful processing of a <PMTRQ> and the actual processing of the payment, the client can cancel the payment by issuing a Payment Cancellation Request <PMTCANCRQ>. The <SRVRTID> value returned in <PMTRS> identifies the payment.

When a payment system cancels a payment, servers can generate a <PMTCANCRS>. This might occur if the user requests payment cancellation by way of a telephone call to customer support or through an e-mail message. The client will receive this response when performing a payment synchronization <PMTSYNCRQ> / <PMTSYNCRS>.

12.4.5 Delayed Payee Matching

Payment systems that allow payment by payee billing address often perform a matching operation to determine if the payee is a standard payee. If this matching occurs in the processing of a <PMTRQ>, and the server recognizes the payee as a standard one, then the server returns the payee ID and payment parameters in the <EXTDPAYEE> aggregate of the <PMTRS>. However some payment systems will not be able to perform “payee matching” at this point in processing. In this case, the server sends updated payee information to the client by using <PAYEESYNCRS> to synchronize the payee list. The client can link payee information in the <PAYEESYNCRS> messages to payments with matching <PAYEELSTID> identifiers.

12.5 Common Payments Aggregates

This section documents several aggregates used throughout the Payments portion of the OFX specification.

12.5.1 Payments Account Information <BPACCTINFO>

OFX uses the payments account information aggregate to download account information from an FI. It includes account number specification in <BANKACCTFROM> as well as the status of the service. In OFX, Banking and Payments share the <BANKACCTFROM> aggregate to identify a specific account. For more information, see section [11.3.1](#).

Tag	Version	Description
<BPACCTINFO>		Payments-account-information aggregate
<BANKACCTFROM>		Bank-account-from aggregate
</BANKACCTFROM>		
<SVCSTATUS>	V1 only	Status of the account AVAIL = Available, but not yet requested PEND = Requested, but not yet available ACTIVE = In use
<SVCSTATUS2>	V2 only	AVAIL = Available, but not yet requested PEND = Requested, but not yet available ACTIVE = In use REJECTED = Rejected
<REASON>	V2 only	This element is only relevant if <SVCSTATUS2>REJECTED is specified, A-255
</BPACCTINFO>		

12.5.2 Credit Mail Order/Telephone Order Account <CCMOTOACCT>

The Credit Mail Order/Telephone Order Account aggregate provides account information to be used for credit card payment. This aggregate includes credit card account information as well as billing address information to be used with Address Verification Service (AVS).

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<CCMOTOACCT>	1.6 add	
<CCACCTFROM>		Account-from aggregate, see section 11.3.2
</CCACCTFROM>		
<DTEXPIRE>		Expiration date for the credit card, <i>date</i>
<BRAND>		Type of credit card (VISA, AMEX, etc.); used as a check against the information provided by the account number, A-32
<NAMEACCTHELD>		Name on the credit card, A-96
<ADDR1>		Billing address on the card, Line 1, A-32
<ADDR2>		Billing address on the card, Line 2. Use of <ADDR2> requires the presence of <ADDR1>, A-32
<ADDR3>		Billing address on the card, Line 3. Use of <ADDR3> requires the presence of <ADDR2>, A-32
<CITY>		City of billing address, A-32
<STATE>		State of the billing address, A-32
<POSTALCODE>		Postal code of the billing address, A-11
<COUNTRY>		Country of the billing address, 3-letter country code from ISO / DIS-3166, A-3
</CCMOTOACCT>		

12.5.3 Payment Information <PMTINFO> <PMTINFO2>

The Payment Information aggregate is used to specify detailed payment information. It is used for both single payments and recurring payments. Clients must send the <PAYEELSTID> and <PAYEEID> if known. Clients send a <PAYEE> aggregate if this is an implicit payee add or modify. See section [12.2.4](#) on identifying payees, above. The <EXTDPMT> aggregate (see section [12.5.3.2](#)) allows the inclusion of disbursement instructions to be printed with the payment. This aggregate is optional.

In the case of an implicit add, the returned <PMTINFO> aggregate found in <PMTRS> and <RECPMTRS> must include the generated <PAYEELSTID>. This aggregate may also include <EXTDPMT> information.

The <DTDUE> in a response may have been adjusted by a server. For example, the server may adjust <DTDUE> to comply with non-processing days. If a client sends a request to make a transfer on July 4 and July 4 happens to be a non-processing day, the <DTDUE> in the response may be July 4 (because the server hasn't adjusted it yet), July 5 (because this server rolls dates forward), or some other date.

Tag	Version	Description
<PMTINFO> <BANKACCTFROM> </BANKACCTFROM> <TRNAMT> <i>Specify payee; either <PAYEEID> or <PAYEE>.</i>	V1 only	Account-from aggregate, see section 11.3.1 Payment amount, <i>amount</i> This amount should be specified as a positive number
<PAYEEID> <PAYEE> </PAYEE>		Server payee identifier (required if assigned). Either <PAYEEID> or <PAYEE> can be sent, but not both. A-12 Complete payee billing information, see section 12.5.3.1 . Either <PAYEEID> or <PAYEE> can be sent, but not both.
<PAYEELSTID> <BANKACCTTO> </BANKACCTTO>		Payee list ID (required if assigned), A-12 Destination account information, for systems that pay by transfers (<PAYEE> also required)

Tag	Version	Description
<EXTDPMT>		Zero or more extended Payment aggregates, see section 12.5.3.2
</EXTDPMT>		
<PAYACCT>		User account number with the payee, A-32
<DTDUE>		Payment due date or the date by which payment must be received by payee, <i>datetime</i>
<MEMO>		Memo from user to payee, <i>memo</i>
<BILLREFINFO>		Billers-supplied reference information when paying a bill, if available, A-80 Note: If the client user interface has a single field that can contain either free-form memo text or a structured reference number, then the contents of that field should be passed in the <MEMO> element rather than the <BILLREFINFO> element.
</PMTINFO>		

Tag	Version	Description
<PAYEEID2>		Server payee identifier (required if assigned). Either <PAYEEID2> or <PAYEE2> can be sent, but not both. <i>SRVRTID2</i>
<PAYEE2>		Complete payee billing information, see section 12.5.3.1 . Either <PAYEEID2> or <PAYEE2> can be sent, but not both. In version 2 of the message set, if the payee is not a standard payee, then the <PAYEE2> aggregate and <PAYEEID2> may be omitted and just the <PAYEELSTID2> supplied to identify the payee.
</PAYEE2>		
<PAYEELSTID2>		Payee list ID (required if assigned), <i>SRVRTID2</i>
<BANKACCTTO>	V2 Only	Destination account information, for systems that pay by transfers (<PAYEE2> also required)
</BANKACCTTO>		
<EXTDPMT>		Optional, see section 12.5.3.2
</EXTDPMT>		
<PAYACCT>	V2 change	User account number with the payee, A-32 Required in message set version 1. In message set version 2, required if <COUNTRY> USA or <COUNTRY>CAN is specified or if <COUNTRY> is not present in signon. Optional otherwise.
<DTDUE>	V2 change	Payment due date or the date by which payment must be received by payee, <i>datetime</i> In message set version 2, if <DAYSTOPAY> or <XFERDFLTDAYSTOPAY> is -1, this field is interpreted as an execution date (the date that the payer's bank will begin processing the payment) rather than a due date.
<DTAVAIL>	V2 only	Beneficiary value date. When the funds must be available in <BANKACCTTO>, <i>datetime</i> Note that for some countries (notably Italy), <DTAVAIL> can be a date in the past. This field is only supported by the server if the <SUPPORTDTAVAIL> element of the Bill Pay Message Set profile is true (see section 12.11.2). This element is not used if <COUNTRY> USA is specified
<ITA.CAUSALE>	V2 only	<i>Causale</i> code. Reason for the payment, for <COUNTRY> ITA only, N-2

Tag	Version	Description																						
<PMTFOR>	V2 only	Name of the person on whose behalf the payment is being made. If not present, payment is assumed to be on behalf of the owner of <BANKACCTFROM>, A-32 This element is currently used only if <COUNTRY> ITA is specified.																						
<BOOKINGTEXT>	V2 only	Text with which this payment should be identified in the payer's bank statement, A-40 This element is currently used only if <COUNTRY> CHE is specified.																						
<MEMO2>	V2 change	Memo from user to payee, <i>memo2</i>																						
<BILLREFINFO>	V2 change	Billers-supplied reference information when paying a bill, if available, A-80 Note: If the client user interface has a single field that can contain either free-form memo text or a structured reference number, then the contents of that field should be passed in the <MEMO> element rather than the <BILLREFINFO> element. Use of this field by country: <table><tr><th><u>COUNTRY</u></th><th><u>Interpretation</u></th></tr><tr><td>BEL</td><td>Not present</td></tr><tr><td>CAN</td><td>Not present</td></tr><tr><td>CHE</td><td><i>Referenz-Nr</i>, if applicable</td></tr><tr><td>DEU</td><td>Not present</td></tr><tr><td>ESP</td><td>Not present</td></tr><tr><td>FRA</td><td>Not present</td></tr><tr><td>GBR</td><td>Not present</td></tr><tr><td>ITA</td><td><i>Coordinate Azienda</i>, if applicable</td></tr><tr><td>NLD</td><td><i>Betalingskenmerk</i>, if applicable</td></tr><tr><td>USA</td><td>used to link payment to bill presentment</td></tr></table>	<u>COUNTRY</u>	<u>Interpretation</u>	BEL	Not present	CAN	Not present	CHE	<i>Referenz-Nr</i> , if applicable	DEU	Not present	ESP	Not present	FRA	Not present	GBR	Not present	ITA	<i>Coordinate Azienda</i> , if applicable	NLD	<i>Betalingskenmerk</i> , if applicable	USA	used to link payment to bill presentment
<u>COUNTRY</u>	<u>Interpretation</u>																							
BEL	Not present																							
CAN	Not present																							
CHE	<i>Referenz-Nr</i> , if applicable																							
DEU	Not present																							
ESP	Not present																							
FRA	Not present																							
GBR	Not present																							
ITA	<i>Coordinate Azienda</i> , if applicable																							
NLD	<i>Betalingskenmerk</i> , if applicable																							
USA	used to link payment to bill presentment																							
<SPNAME>	1.6 add	Service provider name. Used to limit the scope of <BILLID> when paying an electronically-presented bill. This value is normally obtained from <MSGSETCORE> in the FI Profile for the relevant Bill Publisher. If <SPNAME> is specified, <BILLID> is required. A-32 Note: This matches the value of <BILLPUB> in <PRESLISTRS> when bill was presented.																						

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<BILLID>	1.6 add	Identifier for the paid bill, included only when paying an electronically presented bill. A-32
</PMTINFO2>		Note: <BILLID> is required when <SPNAME> is specified.

12.5.3.1 Payee <PAYEE>, <PAYEE2>

<PAYEE> specifies a complete billing address for a payee.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<PAYEE>	V1 only	
<NAME>		Name of payee. A-32
<ADDR1>		Payee's address line 1, A-32
<ADDR2>		Payee's address line 2, A-32
<ADDR3>		Payee's address line 3. Use of <ADDR3> requires the presence of <ADDR2>, A-32
<CITY>		Payee's city, A-32
<STATE>		Payee's state, A-5
<POSTALCODE>		Payee's zip code, A-11
<COUNTRY>		Payee's country; 3-letter country code from ISO / DIS-3166, A-3
<PHONE>		Payee's telephone number, A-32
</PAYEE>		

Tag	Version	Description
<PAYEE2>	V2 only	
<NAME>		Name of payee. A-32
<ADDR1>		Payee's address line 1, A-32 Required if <COUNTRY> USA or <COUNTRY>CAN is specified or if <COUNTRY> is not present, optional otherwise
<ADDR2>		Payee's address line 2. Use of <ADDR2> requires the presence of <ADDR1>, A-32
<ADDR3>		Payee's address line 3. Use of <ADDR3> requires the presence of <ADDR2>, A-32
<CITY>		Payee's city, A-32 Required if <COUNTRY> USA or <COUNTRY>CAN is specified or if <COUNTRY> is not present, optional otherwise
<STATE>		Payee's state, A-5 Required if <COUNTRY> USA or <COUNTRY>CAN is specified or if <COUNTRY> is not present, optional otherwise
<POSTALCODE>		Payee's zip code, A-11 Required if <COUNTRY> USA or <COUNTRY>CAN is specified or if <COUNTRY> is not present, optional otherwise
<COUNTRY>		Payee's country; 3-letter country code from ISO / DIS-3166, A-3
<PHONE>		Payee's telephone number, A-32 Required if <COUNTRY> USA or <COUNTRY>CAN is specified or if <COUNTRY> is not present, optional otherwise
</PAYEE2>		

12.5.3.2 Extended Payment <EXTDPMT>

The Extended Payment aggregate provides the payee with information for applying a payment across multiple invoices. It is structured to allow for electronic processing of the invoice data, and allows multiple invoices, as well as multiple line items per invoice, to be specified.

In this case, <EXTDPMT> can specify a block of free text to be transmitted with the payment, by using the <EXTDPMTDSC> instead of the <EXTDPMTINV> element.

Tag	Version	Description
<EXTDPMT>		Extended Payment aggregate
<EXTDPMTFOR>		INDIVIDUAL or BUSINESS. Indicates whether the payment is for an individual or business account. This allows the payment processor to remit payments to the appropriate address for consumers or businesses.
<EXTDPMTCHK>	V1 only	Check number to use for this payment. Overrides “next check in range.” N-10
<EXTDPMTCHK2>	V2 only	Check number to use for this payment. Overrides “next check in range.” A-12
<i>Payment description. At least one of the following: <EXTDPMTDSC>, <EXTDPMTDSC2>, or <EXTDPMTINV>.</i>		
<EXTDPMTDSC>	V1 only	Free text to communicate with the payment. A-255
</EXTDPMTDSC>		Unlike most other elements in OFX, <EXTDPMTDSC> always requires the end tag.
<EXTDPMTDSC2>	V2 only	Free text to communicate with the payment. While <EXTDPMTDSC> always requires an end tag, this restriction does not apply to <EXTDPMTDSC2>. The </EXTDPMTDSC2> end tag is optional. A-1280
<EXTDPMTINV>		
<INVOICE>	V1 only	One or more invoice aggregates. See section 12.5.3.3
</INVOICE>		
<INVOICE2>	V2 only	One or more invoice aggregates. See section 12.5.3.3
</INVOICE2>		
</EXTDPMTINV>		
</EXTDPMT>		

12.5.3.3 Invoice Description <INVOICE> and <INVOICE2>

An extended payment invoice aggregate (see 12.5.3.2) contains either an <INVOICE> or an <INVOICE2> aggregate. The <INVOICE2> aggregate was created for OFX 1.5.1 to add the following capabilities:

- ◆ <INVOICE2> may contain more than one <ADJUSTMENT>.
- ◆ The <DISCOUNT2> aggregate corrects the required fields found within the <DISCOUNT> aggregate.
- ◆ The <INVOICE2> aggregate allows the use of the <LITMCODE> field within <LINEITEM> aggregates.

Tag	Version	Description
<INVOICE>	V1 only	Start tag for the invoice aggregate. There can be one or more invoices per payment request.
<INVNO>		Invoice number associated with the payment. A-32
<INVTOTALAMT>		This value represents the total invoice amount, <i>amount</i> This amount should be specified as a positive number
<INVPAIDAMT>		This value represents the amount of the invoice being paid, <i>amount</i> This amount should be specified as a positive number
<INVDATE>		Date to apply the invoice, <i>datetime</i>
<INVDESC>		Invoice description, A-80
<DISCOUNT>		Discount aggregate; only one discount aggregate per invoice
<DSCRATE>		Discount rate, <i>rate</i>
<DSCAMT>		Discount amount, <i>amount</i> This amount should be specified as a positive number
<DSCDATE>		Date to apply the discount, <i>datetime</i>
<DSCDESC>		Discount description, A-80
</DISCOUNT>		
<ADJUSTMENT>		Adjustment aggregate; only one adjustment aggregate per invoice, see 12.5.3.4
</ADJUSTMENT>		
<LINEITEM>		Line item aggregate; there can be multiple line items per invoice, see 12.5.3.5
</LINEITEM>		
</INVOICE>		

Tag	Version	Description
<INVOICE2> <INVNO> <INVTOTALAMT> <INVPAIDAMT> <INVDATE> <INVDESC> <DISCOUNT2> <i>Specify discount, either DSCRATE or DSCAMT but not both</i>	V2 only	Start tag for the invoice aggregate. There can be one or more invoices per payment request. Invoice number associated with the payment. A-32 This value represents the total invoice amount, <i>amount</i> This amount should be specified as a positive number This value represents the amount of the invoice being paid, <i>amount</i> This amount should be specified as a positive number Date to apply the invoice, <i>datetime</i> Invoice description, A-80 Discount aggregate; only one discount aggregate per invoice
<DSCRATE> <DSCAMT>		Discount rate, <i>rate</i> Either <DSCRATE> or <DSCAMT>, but not both. Discount amount, <i>amount</i> This amount should be specified as a positive number Either <DSCRATE> or <DSCAMT>, but not both.
<DSCDATE> <DSCDESC> </DISCOUNT2> <ADJUSTMENT> </ADJUSTMENT> <LINEITEM> </LINEITEM> </INVOICE2>		Date to apply the discount, <i>datetime</i> Discount description, A-80 Adjustment aggregate, zero or more per invoice, see 12.5.3.4 Line item aggregate; there can be multiple line items per invoice, see 12.5.3.5

12.5.3.4 <ADJUSTMENT>

Tag	Version	Description
<ADJUSTMENT>		
<ADJNO>		Adjustment number associated with the payment, A-32
<ADJDESC>		Adjustment description, A-80
<ADJAMT>		Amount of the adjustment, <i>amount</i> This amount should be signed + or -, as appropriate. A positive adjustment means that the payment amount has been reduced.
<ADJDATE>		Date of adjustment, <i>datetime</i>
</ADJUSTMENT>		

12.5.3.5 <LINEITEM>

Tag	Version	Description
<LINEITEM>		Line item aggregate; there can be multiple line items per invoice
<LITMAMT>		Amount of the line item, <i>amount</i> This amount should be signed + or -, as appropriate. A positive line item amount is an addition to the payment amount, and a negative line item is a discount or reduction in the payment amount.
<LITMDESC>		Line item description, A-80
<LITMCODE>	V2 only	Posting Code A-32
</LINEITEM>		

12.5.3.6 Extended Payee <EXTDPAYEE>

The Extended Payee aggregate communicates a payee identifier to the client. It also contains the processing day parameters for a payee. It can be sent to the client for any payee whose processing day parameters are different from the processor's default values, even for payees with no <PAYEEID>.

Tag	Version	Description
<EXTDPAYEE>		Extended-payee aggregate
<PAYEEID>	V1 only	Server-assigned payee ID, A-12 If <PAYEEID> is present, <IDSCOPE> and <NAME> are required. Should not be included unless the payee is a standard payee.
<PAYEEID2>	V2 only	Server-assigned payee ID, <i>SRVRTID2</i> If <PAYEEID2> is present, <IDSCOPE> and <NAME> are required. Should not be included unless the payee is a standard payee.
<IDSCOPE>		Scope of the payee ID; one of (GLOBAL, USER), where GLOBAL = payee ID valid across the entire payment system USER = payee ID valid with all FI accounts set up for the user's payments account Required if <PAYEEID> or <PAYEEID2> are present.
<NAME>		Standard payee name, A-32 Required if <PAYEEID> or <PAYEEID2> are present.
<DAYSTOPAY>	V2 change	Minimum number of business days needed to process, N-3 In version 2 of the message set, If <DAYSTOPAY>-1 is specified, then <DTDUE> in <PMTINFO> is interpreted as an execution date (the date that the payer's bank will begin processing the payment) rather than a due date.
</EXTDPAYEE>		

12.5.3.7 Payment Processing Status <PMTPRCSTS>

The Payment Processing Status aggregate contains the current processing status for a payment. This aggregate is intended to describe status changes to the associated payment after creation. The interpretation of the date value depends on the value of <PMTPRCCODE>.

<i>Tag</i>	<i>Description</i>
<PMTPRCSTS>	
<PMTPRCCODE>	See table 12.6.2.1
<DTPMTPRC>	Payment processing date; interpretation depends on <PMTPRCCODE>, <i>datetime</i>
</PMTPRCSTS>	Ending tag for payment processing status

12.6 Payments Functions

Payments functions allow a client to create a Payment Request to pay a bill on a specified date. The Payment Request identifies the payee and the amount to pay. Because the flow of money is unambiguous, bill payment amounts are usually specified as positive numbers. See tables for details.

<i>Client Sends</i>	<i>Server Responds</i>
Account information Payment date Amount Payee address, list ID, transfer acct, or standard ID	Payment status Check number Server-assigned ID

From the time the client issues a Payment Request until it is paid, the client can modify the transaction through the Payment Modification Request, <PMTMODRQ>; see section [12.4.2](#). This request allows payment parameters such as the payment date and payment amount to be changed.

<i>Client Sends</i>	<i>Server Responds</i>
Account information Server-assigned ID Information to change: Payment date, Amount,...	Acknowledgment or Error

The client can cancel a Payment Request with a Payment Cancellation Request, <PMTCANCRQ>; see section [12.4.4](#).

<i>Client Sends</i>	<i>Server Responds</i>
Account information Server-assigned ID	Acknowledgment or Error

12.6.1 Payment Creation

A Payment Request is used to schedule an electronic payment. The server responds with a Payment Response. Separate transactions are provided for modifying and canceling a Payment Request.

12.6.1.1 Payment Request <PMTRQ>

The <PMTRQ> request must appear within a <PMTTRNRQ> transaction wrapper.

Tag	Version	Description
<PMTRQ>		Payment-request aggregate
<PMTINFO>	V1 only	Payment Information aggregate, see section 12.5.3
</PMTINFO>		
<PMTINFO2>	V2 only	Payment Information aggregate, see section 12.5.3
</PMTINFO2>		
</PMTRQ>		

Note: If the <PMTRQ> created a new payee or modified an existing one, the server must create and store a payee response that would be available for subsequent payee synchronization requests. In addition, the server should be aware of the fact that implicit payee modifications may affect models. Such changes to models must also appear in subsequent recurring synchronization responses. In all cases, the server need only send the <PMTRS> as a response to the <PMTRQ>, but any implicit payee and recurring changes must be made by the server, and be returned in later synchronization responses. See section [12.2.5](#) for further discussion of implicit payee adds and modifications.

12.6.1.2 Payment Response <PMTRS>

The server sends a Payment Response in response to a Payment Request. The processing status code for a new payment is normally WILLPROCESSON, but in the case of synchronization it can return other status codes. Servers should inform clients of any errors found while processing this transaction using the <STATUS> aggregate. A response containing <STATUS><CODE>0 and <PMTPRCSTS><PMTPRCCODE>FAILEDON should be avoided for problems such as an invalid account or amount.

The <PMTRS> response must appear within a <PMTTRNRS> transaction wrapper.

Note: When processing a <PMTRQ> request that does not contain a <PAYEEID> or <PAYEELSTID>, a server may check the payee against the user's current payee list and return the found <PAYEELSTID> and <PAYEEID> (if any). If the server does this, it should only find a match when all <PAYEE> data, including all <PAYACCT>

elements, match exactly. If the <PAYACCT> or any other element is different, the server must perform an implicit payee addition. If a server doesn't check for duplicate payees, a client could show duplicate entries in the payee list.

Note: Servers matching well-known payees against entries in a user's payee list must ignore the <PAYACCT> information in the user's list. No corresponding information appears in the list of well-known payees.

Tag	Version	Description
<PMTRS>		Payment-response aggregate
<SRVRTID>	V1 only	ID assigned by the server to the payment being created, <i>SRVRTID</i>
<SRVRTID2>	V2 only	ID assigned by the server to the payment being created, <i>SRVRTID2</i>
<PAYEELSTID>	V1 only	Server-assigned payee list record ID for this payee, <i>A-12</i> Note: This identifier must match that found (and required) in the returned <PMTINFO>.
<PAYEELSTID2>	V2 only	Server-assigned payee list record ID for this payee, <i>SRVRTID2</i> Note: This identifier must match that found (and required) in the returned <PMTINFO2>.
<CURDEF>		Default currency for the Recurring Payment Response, <i>currsymbol</i>
<PMTINFO>	V1 only	Payment Information aggregate, see section 12.5.3
</PMTINFO>		
<PMTINFO2>	V2 only	Payment Information aggregate, see section 12.5.3
</PMTINFO2>		
<EXTDPAYEE>		Standard payee information if payee is a standard payee, or payee has non-default processing day parameters; see section 12.5.3.6
</EXTDPAYEE>		
<CHECKNUM>		Check number, <i>A-12</i>
<PMTPRCSTS>		Payment processing status
</PMTPRCSTS>		
<RECSRVRTID>	V1 only	References the payment if it was generated by a recurring payment, <i>SRVRTID</i>
<RECSRVRTID2>	V2 only	References the payment if it was generated by a recurring payment, <i>SRVRTID2</i>
</PMTRS>		

12.6.1.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2006	Source account not found (ERROR)
2007	Source account closed (ERROR)
2008	Source account not authorized (ERROR)
2009	Destination account not found (ERROR)
2010	Destination account closed (ERROR)
2011	Destination account not authorized (ERROR)
2012	Invalid amount (ERROR)
2014	Date too soon (ERROR)
2015	Date too far in future (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10501	Invalid payee (ERROR)
10502	Invalid payee address (ERROR)
10503	Invalid payee account number (ERROR)
10510	Invalid payee ID (ERROR)
10511	Invalid payee city (ERROR)
10512	Invalid payee state (ERROR)
10513	Invalid payee postal code (ERROR)
10517	Invalid payee name (ERROR)
10519	Invalid payee list ID (ERROR)
10520	Invalid <PMTTYPE> in the <PMTINFO2> aggregate (ERROR)
10521	Transaction requires <CCMOTOACCT> (ERROR)
10522	Insufficient information included in <CCMOTOACCT> (ERROR)

12.6.1.4 Discussion

Once the server has assigned a payee identifier <PAYEEID> to the payee, it includes the <EXTDPAYEE> in any <PMTRS> for that transaction. If the <EXTDPAYEE> aggregate is present in the Payment Response <PMTRS>, the client records the standard payee information for use in future payments to the same payee.

When a payment is made using the <PAYEE> aggregate, and no <PAYEELSTID> is present, the payee is implicitly added to the payee list. This is therefore equivalent to first transmitting a <PAYEERQ> for the payee. For payment systems that can immediately return payee IDs, it is preferable to use the single <PMTRQ> message to both add the payee and create the payment. If either operation fails, the server will not complete the other.

If <PAYEELSTID> and <PAYEE> are both included, it is equivalent to sending a <PAYEEMODRQ>. In <BILLPAYMSGSRV1>, the server must return a <PAYEEMODRS> in a subsequent <PAYEESYNCRS> for all actual changes. This is not necessary (though still allowed) if no change were made. For <BILLPAYMSGSRQV2>, the <PAYEE> aggregate is no longer required. Its presence always indicates a request for an implicit payee modification.

The <PMTRS> response will include the <EXTDPAYEE> aggregate if the processor has assigned a payee ID to the payee specified in the payment. It will also appear in the response when the payee has no assigned ID, but has processing day parameters that differ from the processor's defaults for these values. This might occur, for instance, if the processor notes that the zip code of the payee indicates a certain proximity to the payer, and therefore wishes to offer a shorter <DAYSTOPAY> value.

12.6.2 Payment Modification

The Payment Modification Request allows a client to modify a previously scheduled payment. Once created and retrieved by the customer, spawned payments are almost identical to customer-created payments. (The exception is when a spawned payment is modified or cancelled due to a recurring modification or cancellation request.) As with ordinary payments, you can cancel or modify transactions individually. When modifying a payment, the client must specify all of the elements and aggregates within the <PMTINFO> aggregate that were specified during the payment creation or previous modification, not just the elements and aggregates that it wants to modify. Some servers cannot support the modifications of certain values. Servers must indicate this by returning status code 10505 when the client requests an unsupported modification.

Note: In version 2 of the message set, the client specifies all of the elements and aggregates within the <PMTINFO> aggregate that were specified during the payment creation or previous modification as outlined above. The exception to this is <PAYEE>, which can be omitted in version 2 if there is no intent to modify the payee, since the <PAYEELSTID> would identify the payee.

12.6.2.1 Payment Processing Status Values <PMTPRCCODE>

<i>Value</i>	<i>Description</i>
WILLPROCESSION	Will be processed on <DTPMTPRC>
PROCESSEDON	Was processed for payment on <DTPMTPRC>
NOFUNDSON	Funds not available to make payment on <DTPMTPRC>
FAILEDON	Unable to make payment for unspecified reasons on <DTPMTPRC>
CANCELEDON	User canceled payment on <DTPMTPRC>

12.6.2.2 Payment Modification Request <PMTMODRQ>

The client sends a Payment Modification Request to request modification of a payment. The client must provide the full <PMTINFO> including both changed and unchanged values.

The client may modify any data in <PMTINFO> except the recipient or funding account. In particular, payee list ID <PAYEELSTID>, payee ID <PAYEEID>, funding bank account <BANKACCTFROM>, and the <NAME> element of <PAYEE> must match that returned in the original <PMTRS>. Implicit payee modifications (changes in address information for example) are allowed.

If the <PMTMODRQ> caused a payee modification to an existing payee, the server must create and store a <PAYEEMODRS> to be returned in subsequent payee synchronization responses. If the <PMTMODRQ> created a new payee (possible only if the payment were originally created outside of the OFX protocol), the server must, similarly, create and store a <PAYEERS> for later payee synchronization responses. In addition, the server should be aware of the fact that implicit payee modifications may affect models. Such changes to models must also appear in subsequent recurring synchronization responses. In all cases, the server need only send the <PMTMODRS> as a response to the <PMTMODRQ>, but any implicit payee and recurring changes must be made by the server, and be returned in later synchronization responses. See section [12.2.5](#) for further discussion of implicit payee adds and modifications.

The <PMTMODRQ> request must appear within a <PMTRNRQ> transaction wrapper.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<PMTMODRQ>		Modification-request this references
<SRVRTID>	V1 only	ID assigned by the server to the payment being modified, <i>SRVRTID</i>
<SRVRTID2>	V2 only	ID assigned by the server to the payment being modified, <i>SRVRTID2</i>
<PMTINFO>	V1 only	Payment Information aggregate, see section 12.5.3
</PMTINFO>		
<PMTINFO2>	V2 only	Payment Information aggregate, see section 12.5.3
</PMTINFO2>		
</PMTMODRQ>		

12.6.2.3 Payment Modification Response <PMTMODRS>

The server sends a Payment Modification Response in response to a Payment Modification Request.

The <PMTMODRS> response must appear within a <PMTRNRS> transaction wrapper.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<PMTMODRS>		Payment-modification-response this references
<SRVRTID>	V1 only	ID assigned by the server to the payment being modified, <i>SRVRTID</i>
<SRVRTID2>	V2 only	ID assigned by the server to the payment being modified, <i>SRVRTID2</i>
<PMTINFO>	V1 only	Payment Information aggregate, see section 12.5.3
</PMTINFO>		
<PMTINFO2>	V2 only	Payment Information aggregate, see section 12.5.3
</PMTINFO2>		
<PMTPRCSTS>		Payment processing status, see section 12.5.3.7
</PMTPRCSTS>		
</PMTMODRS>		

12.6.2.4 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2006	Source account not found (ERROR)
2007	Source account closed (ERROR)
2008	Source account not authorized (ERROR)
2009	Destination account not found (ERROR)
2010	Destination account closed (ERROR)
2011	Destination account not authorized (ERROR)
2012	Invalid amount (ERROR)
2014	Date too soon (ERROR)
2015	Date too far in future (ERROR)
2016	Transaction already committed (ERROR)
2017	Already canceled (ERROR)
2018	Unknown server ID (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10501	Invalid payee (ERROR)
10502	Invalid payee address (ERROR)
10503	Invalid payee account number (ERROR)
10505	Cannot modify element (ERROR)
10510	Invalid payee ID (ERROR)
10511	Invalid payee city (ERROR)
10512	Invalid payee state (ERROR)
10513	Invalid payee postal code (ERROR)
10514	Transaction already processed (ERROR)
10517	Invalid payee name (ERROR)
10519	Invalid payee list ID (ERROR)

<i>Code</i>	<i>Meaning</i>
10520	Invalid <PMTTYPE> in the <PMTINFO2> aggregate (ERROR)
10521	Transaction requires <CCMOTOACCT> (ERROR)
10522	Insufficient information included in <CCMOTOACCT> (ERROR)

12.6.2.5 Discussion

Servers can initiate <PMTMODRS> messages to communicate changes in the processing status of a payment as it moves through the payment system. This mechanism allows a client to capture the updated status of payments every time it synchronizes.

Implicit payee changes contained in a payment modification transaction do not affect any other existing pending payments. The changes are propagated to the server's payee list and affect payments to that payee as subsequently initiated by the client after the change, or as subsequently spawned from a recurring model. Explicit payee changes are not propagated to payments pending for the changed payee at the time of the change.

12.6.3 Payment Cancellation

The Payment Cancellation Request allows a client to cancel a previously scheduled payment created with a Payment Request (<PMTRQ> in section [12.6.1.1](#)).

Servers cannot initiate <PMTCANCRS> when communicating status changes. This response should be used only when a payment was actually cancelled (by an OFX client or at users request via the phone). When conveying information about a failure in payment processing (such as insufficient funds), a <PMTMODRS> (with the updated <PMTPRCSTS>) should be added to the next <PMTSYNCRS> download.

12.6.3.1 Request <PMTCANCRQ>

The client sends a Payment Cancellation to cancel a scheduled payment request.

The <PMTCANCRQ> request must appear within a <PMTTRNRQ> transaction wrapper.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<PMTCANCRQ>		Cancellation-request this references
<SRVRTID>	V1 only	ID assigned by the server to the payment being cancelled, <i>SRVRTID</i>
<SRVRTID2>	V2 only	ID assigned by the server to the payment being cancelled, <i>SRVRTID2</i>
</PMTCANCRQ>		

12.6.3.2 Response <PMTCANCERS>

The server sends a Payment Cancellation Response in response to a Payment Cancellation Request.

The <PMTCANCERS> response must appear within a <PMTTRNRS> transaction wrapper.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<PMTCANCERS>		Cancellation-response this references
<SRVRTID>	V1 only	ID assigned by the server to the payment being canceled, <i>SRVRTID</i>
<SRVRTID2>	V2 only	ID assigned by the server to the payment being canceled, <i>SRVRTID2</i>
</PMTCANCERS>		

12.6.3.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2016	Transaction already committed (ERROR)
2017	Already canceled (ERROR)
2018	Unknown server ID (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10514	Transaction already processed (ERROR)

12.6.4 Payment Status Inquiry

The Payment Status Inquiry Request allows a client to obtain the current processing status of a payment from the server.

12.6.4.1 Request <PMTINQRQ>

The client sends a Payment Status Inquiry Request to obtain the current processing status of a payment.

The <PMTINQRQ> request must appear within a <PMTINQTRNRQ> transaction wrapper.

Tag	Version	Description
<PMTINQRQ>		Payment-status-inquiry-request aggregate
<SRVRTID>	V1 only	ID assigned by the server to the payment being queried, <i>SRVRTID</i>
<SRVRTID2>	V2 only	ID assigned by the server to the payment being queried, <i>SRVRTID2</i>
</PMTINQRQ>		

12.6.4.2 Response <PMTINQRS>

The server sends a Payment Status Inquiry Response in response to a Payment Status Inquiry Request.

The <PMTINQRS> response must appear within a <PMTINQTRNRS> transaction wrapper.

Tag	Version	Description
<PMTINQRS>		Payment-status-inquiry-response aggregate
<SRVRTID>	V1 only	ID assigned by the server to the payment being queried, <i>SRVRTID</i>
<SRVRTID2>	V2 only	ID assigned by the server to the payment being queried, <i>SRVRTID2</i>
<PMTPRCSTS>		Payment processing status
</PMTPRCSTS>		
<CHECKNUM>		Check number assigned by the server to this payment, <i>A-12</i>
</PMTINQRS>		

12.6.4.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2018	Unknown server ID (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)

12.7 Recurring Payments

Recurring payments are used when a payment is to be made repeatedly at some known interval. Setting up a recurring payment is similar to creating an individual payment, but with additional information about the frequency and number of payments. After a recurring payment is created, the server will generate payments transactions when there are a specified number of days remaining until the next projected payment is due (usually 30 days). The client will be made aware of any generated payment transactions through the synchronization process. [Chapter 10, "Recurring Transactions"](#) and [Chapter 11, "Banking,"](#) provide additional details on models and recurring transactions, and define the recurring transaction aggregates.

Note: As with individual payments, if the recurring payment request adds a payee or changes payee information, the server must create and store a payee response, to be returned in subsequent payee synchronization responses. Furthermore, implicit payee modifications may affect other models (but not their pending payments). The server must also create and store recurring modification responses for these models, to be returned in subsequent recurring synchronization responses. See section [12.2.5](#) for further discussion of implicit payee adds and changes.

Note: The <MODELWND> profile value indicates when the server spawns a payment. If <MODELWND>0 is specified, the server only spawns one payment at a time for each model. In other words, there is always one pending payment per model, unless the model has expired. If <MODELWND> is greater than 0, its value is the number of days before a payment is due to be paid that it is spawned from the model. In this case, it is possible to have zero or more pending payments instantiated at a time.

The table below lists the functional elements for creating a recurring payment:

<i>Client Sends</i>	<i>Server Responds</i>
Account information Payment frequency Number of payments Payment date Amount Payee address, list ID or payee ID	 Standard payee information Server-assigned ID

The table below lists the functional elements for modifying a recurring payment:

<i>Client Sends</i>	<i>Server Responds</i>
Account information Server-assigned ID Information to change: Payment frequency, Number of payments, Payment date, Amount,...	 Acknowledgment or Error

The table below lists the functional elements for canceling a recurring payment:

<i>Client Sends</i>	<i>Server Responds</i>
Account information Server-assigned ID	 Acknowledgment or Error

12.7.1 Creating a Recurring Payment

Use a Recurring Payment Request to set up a recurring electronic payment. The user can specify the frequency and duration of the payments using the Recurring Instructions aggregate <RECURRINST>. The <PMTINFO> aggregate (see section [12.5.3](#)) specifies the payment information for the model, as well as the initial and final amounts (if present and where applicable).

12.7.1.1 Request <RECPMTRQ>

The <RECPMTRQ> request must appear within a <RECPMTTRNRQ> transaction wrapper.

Tag	Version	Description
<RECPMTRQ>		Recurring-payment-request aggregate
<RECURRINST>		Recurring Instructions aggregate, see section 10.2 .
</RECURRINST>		
<PMTINFO>	V1 only	Payment-information aggregate, see section 12.5.3
</PMTINFO>		
<PMTINFO2>	V2 only	Payment-information aggregate, see section 12.5.3
</PMTINFO2>		
<INITIALAMT>		Amount of the initial payment, if different than the following payments, <i>amount</i> This amount should be specified as a positive number
<FINALAMT>		Amount of the final payment, if different than the preceding payments, <i>amount</i> This amount should be specified as a positive number
</RECPMTRQ>		

12.7.1.2 Response <RECPMTRS>

The server sends a Recurring Payment Response upon receipt of a Recurring Payment Request.

The <RECPMTRS> response must appear within a <RECPMTTRNRS> transaction wrapper.

Tag	Version	Description
<RECPMTRS>		Recurring-payment-response aggregate
<RECSRVRTID>	V1 only	Server-assigned ID for this transaction, <i>SRVRTID</i>
<RECSRVRTID2>	V2 only	Server-assigned ID for this transaction, <i>SRVRTID2</i>
<PAYEELSTID>	V1 only	Server-assigned record ID for this payee record, <i>A-12</i> Note: This identifier must match that found (and required) in the returned <PMTINFO>.
<PAYEELSTID2>	V2 only	Server-assigned record ID for this payee record, <i>SRVRTID2</i> Note: This identifier must match that found (and required) in the returned <PMTINFO2>.
<CURDEF>		Default currency for the Recurring Payment Response, <i>currsymbol</i>
<RECURRINST>		Recurring-instructions aggregate, see section 10.2 .
</RECURRINST>		
<PMTINFO>	V1 only	Payment-information aggregate, see section 12.5.3
</PMTINFO>		
<PMTINFO2>	V2 only	Payment-information aggregate, see section 12.5.3
</PMTINFO2>		
<INITIALAMT>		Amount of the initial payment, if different than the following payments, <i>amount</i> This amount should be specified as a positive number
<FINALAMT>		Amount of the final payment, if different than the preceding payments, <i>amount</i> This amount should be specified as a positive number
<EXTDPAYEE>		Extended payee information, see section 12.5.3.3 .
</EXTDPAYEE>		
</RECPMTRS>		

12.7.1.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2006	Source account not found (ERROR)
2007	Source account closed (ERROR)
2008	Source account not authorized (ERROR)
2009	Destination account not found (ERROR)
2010	Destination account closed (ERROR)
2011	Destination account not authorized (ERROR)
2012	Invalid amount (ERROR)
2014	Date too soon (ERROR)
2015	Date too far in future (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10501	Invalid payee (ERROR)
10502	Invalid payee address (ERROR)
10503	Invalid payee account number (ERROR)
10508	Invalid frequency (ERROR)
10510	Invalid payee ID (ERROR)
10511	Invalid payee city (ERROR)
10512	Invalid payee state (ERROR)
10513	Invalid payee postal code (ERROR)
10517	Invalid payee name (ERROR)
10519	Invalid payee list ID (ERROR)
10520	Invalid <PMTTYPE> in the <PMTINFO2> aggregate (ERROR)
10521	Transaction requires <CCMOTOACCT> (ERROR)
10522	Insufficient information included in <CCMOTOACCT> (ERROR)

12.7.1.4 Discussion

The <DTDUE> element of <PMTINFO> specifies payment due date or the date by which the first payment must be received by payee (see section [12.5.3](#)).

The <DTDUE> in a response may have been adjusted by a server. For example, the server may adjust <DTDUE> to comply with non-processing days. If a client sends a request to make a transfer on July 4 and July 4 happens to be a non-processing day, the <DTDUE> in the response may be July 4 (because the server hasn't adjusted it yet), July 5 (because this server rolls dates forward), or some other date. For this reason, a client should pay attention to the <DTDUE> in the response.

12.7.2 Recurring Payment Modification

The client sends a Recurring Payment Modification Request to request modifications to a recurring payment previously created with a Recurring Payment Request. The payment frequency <RECURRINST>, the payment parameters <PMTINFO>, or both, can be changed.

12.7.2.1 Request <RECPMTMODRQ>

The client sends a Recurring Payment Modification Request to request changes to a recurring payment model.

The client may modify any data in <PMTINFO> except the recipient or funding account. In particular, payee list ID <PAYEELSTID>, payee ID <PAYEEID>, funding bank account <BANKACCTFROM>, and the <NAME> element of <PAYEE> must match that returned in the original <RECPMTRS>. Implicit payee modifications (changes in address information for example) are allowed. Clients can modify both elements in the <RECURRINST> aggregate (i.e. <NINSTS> and <FREQ>). Client should send the original number of payments scheduled if there is no change. If there is a change in the number of payments scheduled, clients should send the new number of payments.

A <RECPMTMODRQ> that modifies pending payments via the <MODPENDING> flag is a compound transaction and the server should create and store <PMTMODRS>s, which are returned to the client in subsequent payment synchronization responses. For example, a change to the <TRNAMT> element would cause the server to create and store a <PMTMODRS> for each pending payment, to be returned in a subsequent payment synchronization response. Changes to payment information apply to all future payments.

Note: The <RECPMTMODRQ> element may implicitly modify a payee. A payee modification can, in turn, modify other existing models (though not their pending payments). In such cases, the server must create and store the appropriate responses (<PAYEEMODRS> and, possibly, additional, <RECPMTMODRS>), to be returned to the client in subsequent synchronization responses. See section [12.2.5](#) for further discussion of implicit payee adds and changes. If the <RECPMTMODRQ> created a new payee (this is only possible if the payment were originally created outside of the

OFX protocol), the server must create and store a <PAYEERS> that would be available for a payee synchronization request. In all cases, the server need only send the <RECPMTMODRS> as a response to the <RECPMTMODRQ>, but any implicit payee and recurring changes must be made by the server, and be returned in later synchronization responses.

The <RECPMTMODRQ> request must appear within a <RECPMTTRNRQ> transaction wrapper.

Tag	Version	Description
<RECPMTMODRQ>		Modification-request aggregate
<RECSRVRTID>	V1 only	ID assigned by the server to the payment being modified, <i>SRVRTID</i>
<RECSRVRTID2>	V2 only	ID assigned by the server to the payment being modified, <i>SRVRTID2</i>
<RECURRINST>		Recurring Instructions aggregate, see section 10.2
</RECURRINST>		
<PMTINFO>	V1 only	Payment-Information aggregate, see section 12.5.3
</PMTINFO>		
<PMTINFO2>	V2 only	Payment-Information aggregate, see section 12.5.3
</PMTINFO2>		
<INITIALAMT>		Amount of the initial payment, if different than the following payments, <i>amount</i> This amount should be specified as a positive number
<FINALAMT>		Amount of the final payment, if different than the preceding payments, <i>amount</i> This amount should be specified as a positive number
<MODPENDING>		Modify pending flag If the client sets this flag, the server must modify pending and future payments, <i>Boolean</i>
</RECPMTMODRQ>		

12.7.2.2 Response <RECPMTMODRS>

The server sends a Recurring Payment Modification Response in response to a Recurring Payment Modification Request.

The <RECPMTMODRS> response must appear within a <RECPMTTRNRS> transaction wrapper.

Tag	Version	Description
<RECPMTMODRS>		Modification-response aggregate
<RECSRVRTID>	V1 only	ID assigned by the server to the payment being modified, <i>SRVRTID</i>
<RECSRVRTID2>	V2 only	ID assigned by the server to the payment being modified, <i>SRVRTID2</i>
<RECURRINST>		Recurring-Instructions aggregate, see section 10.2
</RECURRINST>		
<PMTINFO>	V1 only	Payment-Information aggregate, see section 12.5.3
</PMTINFO>		
<PMTINFO2>	V2 only	Payment-Information aggregate, see section 12.5.3
</PMTINFO2>		
<INITIALAMT>		Amount of the initial payment, if different than the following payments, <i>amount</i> This amount should be specified as a positive number
<FINALAMT>		Amount of the final payment, if different than the preceding payments, <i>amount</i> This amount should be specified as a positive number
<MODPENDING>		Y if the client requested that the server modify pending and future payments. N if the client did not request that the server modify pending and future payments., <i>Boolean</i>
</RECPMTMODRS>		

12.7.2.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2006	Source account not found (ERROR)
2007	Source account closed (ERROR)
2008	Source account not authorized (ERROR)
2009	Destination account not found (ERROR)
2010	Destination account closed (ERROR)
2011	Destination account not authorized (ERROR)
2012	Invalid amount (ERROR)
2014	Date too soon (ERROR)
2015	Date too far in future (ERROR)
2016	Transaction already committed (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10501	Invalid payee (ERROR)
10502	Invalid payee address (ERROR)
10503	Invalid payee account number (ERROR)
10505	Cannot modify element (ERROR)
10508	Invalid frequency (ERROR)
10511	Invalid payee city (ERROR)
10512	Invalid payee state (ERROR)
10513	Invalid payee postal code (ERROR)
10514	Transaction already processed (ERROR)
10517	Invalid payee name (ERROR)
10518	Unknown model ID (ERROR)
10519	Invalid payee list ID (ERROR)

<i>Code</i>	<i>Meaning</i>
10520	Invalid <PMTTYPE> in the <PMTINFO2> aggregate (ERROR)
10521	Transaction requires <CCMOTOACCT> (ERROR)
10522	Insufficient information included in <CCMOTOACCT> (ERROR)

12.7.3 Recurring Payment Cancellation

The client sends a Recurring Payment Cancellation Request to cancel a recurring payment previously created with a Recurring Payment Request.

12.7.3.1 Request <RECPMTCANCRQ>

The <RECPMTCANCRQ> request must appear within a <RECPMTTRNRQ> transaction wrapper.

Note: A <RECPMTCANCRQ> that cancels pending payments via the <CANPENDING> flag is a compound transaction, and generates the appropriate explicit payment responses that reflect such cancellations, which are returned to the client via synchronization.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<RECPMTCANCRQ>		Cancellation-request aggregate
<RECSRVRTID>	V1 only	ID assigned by the server to the payment being canceled, <i>SRVRTID</i>
<RECSRVRTID2>	V2 only	ID assigned by the server to the payment being canceled, <i>SRVRTID2</i>
<CANPENDING>		Cancel pending flag, <i>Boolean</i> If Y, server should cancel all pending and unspawned payments. If N, server should cancel only the model (and unspawned payments).
</RECPMTCANCRQ>		

12.7.3.2 Response <RECPMTCANCRS>

The server sends a Recurring Payment Cancellation Response in response to a Recurring Payment Cancellation Request.

The <RECPMTCANCRS> response must appear within a <RECPMTTRNRS> transaction wrapper.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<RECPMTCANCRS>		Modification-request aggregate
<RECSRVRTID>	V1 only	ID assigned by the server to the payment being modified, <i>SRVRTID</i>
<RECSRVRTID2>	V2 only	ID assigned by the server to the payment being modified, <i>SRVRTID2</i>
<CANPENDING>		Cancel pending flag, <i>Boolean</i> Y if the client requested that the server cancel all pending and unspawned payments. N if the client requested that the server cancel only unspawned payments.
</RECPMTCANCRS>		

12.7.3.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2016	Transaction already committed (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10514	Transaction already processed (ERROR)
10518	Unknown model ID (ERROR)

12.8 Payment Mail

Users can correspond by way of e-mail to resolve problems or ask questions about their payments accounts. This function makes use of the general OFX e-mail facility, which is described in [Chapter 9, "Customer to FI Communication."](#)

Note: There is no way to indicate non-support of payment e-mail in the profile. A server that doesn't support <PMTMAILSYNCRQ> should return an "empty" payment mail sync response rather than just ignore the request. This empty sync response includes <TOKEN>0 and no history.

12.8.1 Payment Mail Request and Response

12.8.1.1 Request <PMTMAILRQ>

The <PMTMAILRQ> allows a client to issue an e-mail to the payments processor. If the message refers to a specific payment, then both <SRVRTID> and <PMTINFO> are required to identify the payment to the processor.

The <PMTMAILRQ> request must appear within a <PMTMAILTRNRQ> transaction wrapper.

Tag	Version	Description
<PMTMAILRQ>		Payment e-mail-request aggregate
<MAIL>		General e-mail aggregate
</MAIL>		
<SRVRTID>	V1 only	Transaction ID of the payment that is the subject of the correspondence, <i>SRVRTID</i>
<SRVRTID2>	V2 only	Transaction ID of the payment that is the subject of the correspondence, <i>SRVRTID2</i>
<PMTINFO>	V1 only	Payment Information aggregate, see section 12.5.3
</PMTINFO>		
<PMTINFO2>	V2 only	Payment Information aggregate, see section 12.5.3
</PMTINFO2>		
</PMTMAILRQ>		

12.8.1.2 Response <PMTMAILRS>

The server sends <PMTMAILRS> in response to a Payment E-mail request.

The <PMTMAILRS> response must appear within a <PMTMAILTRNRS> transaction wrapper.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<PMTMAILRS>		Payment e-mail-response aggregate
<MAIL>		General e-mail aggregate, see Chapter 9, "Customer to FI Communication."
</MAIL>		
<SRVRTID>	V1 only	Transaction ID of the payment that is the subject of the correspondence, <i>SRVRTID</i>
<SRVRTID2>	V2 only	Transaction ID of the payment that is the subject of the correspondence, <i>SRVRTID2</i>
<PMTINFO>	V1 only	Payment Information aggregate, see section 12.5.3
</PMTINFO>		
<PMTINFO2>	V2 only	Payment Information aggregate, see section 12.5.3
</PMTINFO2>		
</PMTMAILRS>		

12.8.1.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2003	Account not found (ERROR)
2004	Account closed (ERROR)
2005	Account not authorized (ERROR)
2018	Unknown server ID (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
15508	Transaction not authorized (ERROR)
16500	HTML not allowed (ERROR)
16501	Unknown mail To: (ERROR)

12.8.2 Payment Mail Synchronization

Payment mail is subject to synchronization. The scope of the synchronization request is all of the accounts for which the user might have sent mail, not a specific account.

12.8.2.1 Request <PMTMAILSYNCRQ>

Tag	Version	Description
<PMTMAILSYNCRQ> <i>Client synchronization option; <TOKEN>, <TOKEN2>, <TOKENONLY>, or <REFRESH></i>		Synchronization-request aggregate
<TOKEN>	V1 only	Previous value of <TOKEN> received for this type of synchronization request from server; 0 for first-time requests; <i>token</i>
<TOKEN2>	V2 only	Previous value of <TOKEN2> received for this type of synchronization request from server; 0 for first-time requests; <i>token2</i>
<TOKENONLY>		Request for just the current <TOKEN> without the history, <i>Boolean</i>
<REFRESH>		Request for refresh of current state, <i>Boolean</i>
<REJECTIFMISSING>		If Y, do not process requests if client <TOKEN> is out of date, <i>Boolean</i>
<INCIMAGES>		Y if the client accepts mail with images in the message body. N if the client does not accept mail with images in the message body. <i>Boolean</i>
<USEHTML>		Y if client wants an HTML response, N if client wants plain text, <i>Boolean</i>
<PMTMAILTRNRQ> </PMTMAILTRNRQ> </PMTMAILSYNCRQ>		Payment-mail transactions (0 or more)

12.8.2.2 Response <PMTMAILSYNCRS>

Tag	Version	Description
<PMTMAILSYNCRS>		Synchronization-response aggregate
<TOKEN>	V1 only	New synchronization token, <i>token</i>
<TOKEN2>	V2 only	New synchronization token, <i>token2</i>
<LOSTSYNC>		Y if the token in the synchronization request is older than the earliest entry in the server's history table. In this case, some responses have been lost. N if the token in the synchronization request is newer than or matches a token in the server's history table. <i>Boolean</i>
<SYNCEERROR>	V2 only	Optional error code (See section 6.4.4.), <i>N-6</i> .
<PMTMAILTRNRS>		Payment-mail transactions (0 or more)
</PMTMAILTRNRS>		
</PMTMAILSYNCRS>		

12.9 Payee Lists

Payments-system servers store lists of payees set up for payment by each user. Some systems require this before the user can issue a payment to a payee. In other payment systems, this feature enables the sharing of payee entry among multiple clients, and simplifies server payee maintenance.

A server-assigned payee list-entry ID identifies entries in the payee list. The following set of messages allows clients to obtain this list of payees. Users can add, modify, and delete individual entries in the list. The user-defined Payee list is subject to synchronization, so that multiple clients can use the list.

Creating a payee:

Client Sends	Server Responds
Server-assigned payee identifier, or payee billing address User's account number with the payee	 Payee address Standard payee information

Modifying a payee:

<i>Client Sends</i>	<i>Server Responds</i>
Server-assigned payee identifier User's account number with the payee Information to change: payee name address city state postal code phone number new payee account #	Extended payee information if payee is a standard payee or has non-default processing lead times Acknowledgment or Error

Deleting a payee:

<i>Client Sends</i>	<i>Server Responds</i>
Server-assigned payee identifier User's account number with the payee	Acknowledgment or Error

12.9.1 Adding a Payee to the Payee List

The user can use the Payee Request to add a payee to the server payee list. The server responds with a Payee Response, which can contain a complete billing address for the payee, or if the payee is a standard payee, the lead-time and payee name values.

12.9.1.1 Payee Request <PAYEERQ>

The <PAYEERQ> requests the addition of a payee entry to the server's payee list. Note that the user can use a <PMTRQ> to simultaneously set up a payee. OFX does not require the client to send a <PAYEERQ> before making an initial <PMTRQ> to a payee.

The <PAYEERQ> request must appear within a <PAYEETRNRQ> transaction wrapper.

Tag	Version	Description
<PAYEERQ> <i>Specify payee; either <PAYEEID> or <PAYEE>.</i>		Payee-request aggregate
<PAYEEID>	V1 only	Server-assigned payee identifier, A-12
<PAYEEID2>	V2 only	Server-assigned payee identifier.
<PAYEE>	V1 only	Complete payee billing information, see section 12.5.3.1 .
</PAYEE>		
<PAYEE2>	V2 only	Complete payee billing information, see section 12.5.3.1 .
</PAYEE2>		
<BANKACCTTO>		The destination bank account, specified in countries that pay using transfers. The <PAYEE> or <PAYEE2> (above) must also be specified.
</BANKACCTTO>		
<PAYACCT>		User's account number(s) with the payee (0 or more), A-32
</PAYEERQ>		

12.9.1.2 Payee Response <PAYEERS>

The server sends the Payee Response in response to a Payee Request. It contains the full billing information for the payee if it is not a standard payee. Otherwise, it contains the standard payee information, including lead time and payee name. If the server identifies the payee as having an assigned payee ID, then the server will include the <EXTDPAYEE> aggregate in the response.

If the response indicates that the payee does not have an assigned <PAYEEID>, the client should specify the full billing address <PAYEE> information in subsequent payment requests <PMTRQ> to the payee when the payee is being modified. Otherwise the <PAYEELSTID> is used in lieu of the <PAYEE> aggregate.

If the response indicates that the payee does have a <PAYEEID>, then the client should use the <PAYEEID> for making payments to that payee.

The <PAYEERS> response must appear within a <PAYEETRNRS> transaction wrapper.

Tag	Version	Description
<PAYEERS>		Payee-response aggregate
<PAYEELSTID>	V1 only	Server-assigned record ID for this payee record, A-12
<PAYEELSTID2>	V2 only	Server-assigned record ID for this payee record, <i>SRVRTID2</i>
<PAYEE>	V1 only	Complete payee billing information, see section 12.5.3.1 .
</PAYEE>		
<PAYEE2>	V2 only	Complete payee billing information, see section 12.5.3.1 .
</PAYEE2>		
<BANKACCTTO>		The destination bank account, specified in countries that pay using transfers. The <PAYEE> or <PAYEE2> (above) must also be specified.
</BANKACCTTO>		
<EXTDPAYEE>		Extended payee information, see section 12.5.3.3
</EXTDPAYEE>		
<PAYACCT>		User's account number(s) with the payee (0 or more), A-32
</PAYEERS>		

12.9.1.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2001	Invalid account (ERROR)
2002	General account error (ERROR)
2009	Destination account not found (ERROR)
2010	Destination account closed (ERROR)
2011	Destination account not authorized (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10501	Invalid payee (ERROR)
10502	Invalid payee address (ERROR)
10503	Invalid payee account number (ERROR)
10511	Invalid payee city (ERROR)
10512	Invalid payee state (ERROR)
10513	Invalid payee postal code (ERROR)

12.9.2 Payee Modification

The Payee Modification Request allows the client to make changes to payee entries in the server's payee list. Changes in the server's payee list are not propagated to already created / spawned pending payments. Payments spawned from a model after the payee modification will use the updated information from the server's payee list as modified by the Payee Modification request.

12.9.2.1 Request <PAYEEMODRQ>

The client sends the Payee Modification Request to request changes to an existing payee list entry. The <PAYEE> aggregate must specify the changed and unchanged payee information. Absence of a <PAYACCT> in a <PAYEEMODRQ> could be interpreted as an implicit disassociation of the <PAYACCT> with the payee. Presence or absence of a <PAYACCT> does not imply selective <PAYEE> aggregate changes for the same <PAYEELSTID> as referenced by more than one <PAYACCT>. The <PAYEEMODRQ> request must appear within a <PAYEETRNRQ> transaction wrapper.

A payee modification may also affect models (though not their pending payments). In this case, a server must create and store <RECPMTMODRS> responses, to be returned to the client in subsequent recurring synchronization responses. See section 12.2.5 for further discussion of implicit payee changes.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<PAYEEMODRQ>		Modification-request aggregate
<PAYEELSTID>	V1 only	Server-assigned record ID for this payee record, <i>A-12</i>
<PAYEELSTID2>	V2 only	Server-assigned record ID for this payee record, <i>SRVRTID2</i>
<PAYEE>	V1 only	Payee information to modify
</PAYEE>		
<PAYEE2>	V2 only	Payee information to modify
</PAYEE2>		
<BANKACCTTO>		Destination account for countries that pay using transfers (<PAYEE> or <PAYEE2> required)
</BANKACCTTO>		
<PAYACCT>		Payer account number(s) with the payee (0 or more), <i>A-32</i>
<MODPENDING>	1.6 add	Indicates whether to propagate the payee change to pending single payments; changes to payees are always propagated to payment models, <i>Boolean</i> . Y = changes to payees are propagated to pending payments N (or omitted) = Changes are not propagated. This element is ignored when <PAYEEMODPENDING>IFREQUESTED is not specified in the FI Profile (see section 12.11.2).
</PAYEEMODRQ>		

12.9.2.2 Response <PAYEEMODRS>

The server returns a Payee Modification Response in reply to a Payee Modification Request.

When a server-initiated change occurs to the extended payee information for a payee (for example a change in the payee's lead-time), the server can include this information in the <EXTDPAYEE> of the response.

If a server-initiated response indicates either that a payee now has a payee ID, or no longer has one, then the client should use the appropriate form of designating the payee in any future payment requests <PMTRQ> to that payee.

The <PAYEEMODRS> response must appear within a <PAYEETRNR> transaction wrapper.

Tag	Version	Description
<PAYEEMODRS>		Modification-response aggregate
<PAYEELSTID>	V1 only	Server-assigned record ID for this payee record, <i>A-12</i>
<PAYEELSTID2>	V2 only	Server-assigned record ID for this payee record, <i>SRVRTID2</i>
<PAYEE>	V1 only	Payee information that was modified, see section 12.5.3.1
</PAYEE>		
<PAYEE2>	V2 only	Payee information that was modified, see section 12.5.3.1
</PAYEE2>		
<BANKACCTTO>		Destination account for countries that pay bills using transfers (<PAYEE> or <PAYEE2>required as well)
</BANKACCTTO>		
<PAYACCT>		Payer's account number(s) with the payee (0 or more), <i>A-32</i>
<EXTDPAYEE>		Extended payee information, see section 12.5.3.3
</EXTDPAYEE>		
<MODPENDING>	1.6 add	Y if the client requested that the server modify pending and future payments. N if the client did not request that the server modify pending and future payments., <i>Boolean</i>
</PAYEEMODRS>		

12.9.2.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2001	Invalid account (ERROR)
2002	General account error (ERROR)
2009	Destination account not found (ERROR)
2010	Destination account closed (ERROR)
2011	Destination account not authorized (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10501	Invalid payee (ERROR)
10502	Invalid payee address (ERROR)
10503	Invalid payee account number (ERROR)
10510	Invalid payee ID (ERROR)
10511	Invalid payee city (ERROR)
10512	Invalid payee state (ERROR)
10513	Invalid payee postal code (ERROR)
10515	Payee not modifiable by client (ERROR)

12.9.3 Payee Deletion

The Payee Deletion Request allows a client to delete a payee entry from the server's list of the user's payees. To delete specific <PAYACCT> associations with a payee, clients should use the <PAYEELSTID> combined with absent <PAYACCT>s via a <PAYEEMODRQ>.

The Payee delete request does not cancel payments that are pending at the time of the payee's deletion. References to pending payments subsequent to a payee's deletion pose issues regarding <PAYEELSTID> assignment at both the client and server levels. Therefore, it is suggested that the client disallow payee deletes if there are pending payments / models.

12.9.3.1 Request <PAYEEDELREQ>

The <PAYEEDELREQ> requests the deletion of a payee entry.

The <PAYEEDELREQ> request must appear within a <PAYEETRNRQ> transaction wrapper.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<PAYEEDELREQ>		Deletion-request aggregate
<PAYEELSTID>	V1 only	Server-assigned record ID for this payee record, <i>A-12</i>
<PAYEELSTID2>	V2 only	Server-assigned record ID for this payee record, <i>SRVRTID2</i>
</PAYEEDELREQ>		

12.9.3.2 Response <PAYEEDELRS>

The server sends the Payee Deletion Response <PAYEEDELRS> in response to a Payee Deletion Request <PAYEEDELREQ>.

The <PAYEEDELRS> response must appear within a <PAYEETRNR> transaction wrapper.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<PAYEEDELRS>		Deletion-response aggregate
<PAYEELSTID>	V1 only	Server-assigned record ID for this payee record, A-12
<PAYEELSTID2>	V2 only	Server-assigned record ID for this payee record, SRVRTID2
</PAYEEDELRS>		

12.9.3.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
10519	Invalid payee list ID (ERROR)

12.9.4 Payee List Synchronization

This message allows clients to obtain a list of payees stored on the server that it has configured for use in payments. In a “pay some” system, users are sometimes required to explicitly configure the payees to whom the system will make payments. This can be done by means of a telephone call to the payments provider or through some other interface. The client can then use this message to obtain the user’s list of configured payees. In other systems, the payments provider can elect to store a list of all payees that the user has paid. This is a convenience to the client. It allows payees set up on one client to be accessible from a user’s other clients and ensures each client has the latest version of this list.

12.9.4.1 Request <PAYEESYNCRQ>

Tag	Version	Description
<PAYEESYNCRQ> <i>Client synchronization option; <TOKEN>, <TOKEN2>, <TOKENONLY>, or <REFRESH></i>		Payee-list-request aggregate
<TOKEN>	V1 only	Previous value of <TOKEN> received for this type of synchronization request from server; 0 for first-time requests; <i>token</i>
<TOKEN2>	V2 only	Previous value of <TOKEN2> received for this type of synchronization request from server; 0 for first-time requests; <i>token2</i>
<TOKENONLY>		Request for just the current <TOKEN> without the history, <i>Boolean</i>
<REFRESH>		Request for refresh of current state, <i>Boolean</i>
<REJECTIFMISSING>		If Y, do not process requests if client <TOKEN> is out of date, <i>Boolean</i>
<PAYEETRNRQ>		Payee transactions (0 or more)
</PAYEETRNRQ>		
</PAYEESYNCRQ>		

12.9.4.2 Response <PAYEESYNCRS>

Tag	Version	Description
<PAYEESYNCRS>		Payee-list-request aggregate
<TOKEN>	V1 only	New synchronization token, <i>token</i>
<TOKEN2>	V2 only	New synchronization token, <i>token2</i>
<LOSTSYNC>		Y if the token in the synchronization request is older than the earliest entry in the server's history table. In this case, some responses have been lost. N if the token in the synchronization request is newer than or matches a token in the server's history table. <i>Boolean</i>
<SYNCERROR>	V2 only	Optional error code (See section 6.4.4.), <i>N-6</i> .
<PAYEETRNR>		Payee transactions (0 or more)
</PAYEETRNR>		
</PAYEESYNCRS>		

12.10 Data Synchronization for Payments

Users of OFX Payments need to be able to obtain the current status of transactions previously sent to the server for processing. For example, once the user schedules a payment and the payment date has passed, the user might want to verify that the server made the payment as directed. Additionally, OFX allows for interactions with the server through multiple clients. This means, for example, that the user can perform some transactions from a home PC and others from an office computer with each session incorporating the activities performed on the other.

In order to accomplish these tasks, the client uses a synchronization scheme to insure that it has an accurate copy of the server data that is relevant to the client application. The intent of this scheme is to address three scenarios in which the client might lose synchronization with the server:

- ◆ A transaction has changed its state based on processing actions on the server. For example, a scheduled payment has passed its due date and has been paid or rejected.
- ◆ Transactions relevant to the client's application state have been added, deleted, or modified by a second client. For example, a user might enter or change transactions from more than one PC or application.
- ◆ A communications session between the client and server was interrupted or completed abnormally. As a result the client does not have responses from the server indicating that all the transactions were received and processed.

Note: Except for the <REFRESH>Y sync response, no payee information in any particular response in a sync should have changed from that in the response when it was originally sent. In other words, if a <PMTMODRS> caused a change to that payment's payee address, the original <PMTRS> in the sync should have the old address in it. The <PMTMODRS>, appearing later in the sync, would cause the client to update the payment appropriately.

12.10.1 Payment Synchronization

12.10.1.1 Request <PMTSYNCRQ>

Tag	Version	Description
<PMTSYNCRQ> <i>Client synchronization option; <TOKEN>, <TOKEN2>, <TOKENONLY>, or <REFRESH></i>		Synchronization-request aggregate
<TOKEN>	V1 only	Previous value of <TOKEN> received for this type of synchronization request from server; 0 for first-time requests; <i>token</i>
<TOKEN2>	V2 only	Previous value of <TOKEN2> received for this type of synchronization request from server; 0 for first-time requests; <i>token2</i>
<TOKENONLY>		Request for just the current <TOKEN> without the history, <i>Boolean</i>
<REFRESH>		Request for refresh of current state, <i>Boolean</i>
<REJECTIFMISSING>		If Y, do not process requests if client <TOKEN> is out of date, <i>Boolean</i>
<BANKACCTFROM>		Opening tag for account from aggregate, see section 11.3.1
</BANKACCTFROM>		
<PMTRNRQ>		Payment transactions (0 or more)
</PMTRNRQ>		
</PMTSYNCRQ>		

12.10.1.2 Response <PMTSYNCRS>

Tag	Version	Description
<PMTSYNCRS>		Synchronization-response aggregate
<TOKEN>	V1 only	New synchronization token, <i>token</i>
<TOKEN2>	V2 only	New synchronization token, <i>token2</i>
<LOSTSYNC>		Y if the token in the synchronization request is older than the earliest entry in the server's history table. In this case, some responses have been lost. N if the token in the synchronization request is newer than or matches a token in the server's history table. <i>Boolean</i>
<SYNCERROR>	V2 only	Optional error code (See section 6.4.4.), <i>N-6</i> .
<BANKACCTFROM>		Opening tag for account from aggregate, see section 11.3.1
</BANKACCTFROM>		
<PMTRNRS>		Payment transactions (0 or more)
</PMTRNRS>		
</PMTSYNCRS>		

12.10.2 Recurring Payment Synchronization

12.10.2.1 Request <RECPMTSYNCRQ>

Tag	Version	Description
<RECPMTSYNCRQ> <i>Client synchronization option; <TOKEN>, <TOKEN2>, <TOKENONLY>, or <REFRESH></i>		Synchronization-request aggregate
<TOKEN>	V1 only	Previous value of <TOKEN> received for this type of synchronization request from server; 0 for first-time requests; <i>token</i>
<TOKEN2>	V2 only	Previous value of <TOKEN2> received for this type of synchronization request from server; 0 for first-time requests; <i>token2</i>
<TOKENONLY>		Request for just the current <TOKEN> without the history, <i>Boolean</i>
<REFRESH>		Request for refresh of current state, <i>Boolean</i>
<REJECTIFMISSING>		If Y, do not process requests if client <TOKEN> is out of date, <i>Boolean</i>
<BANKACCTFROM>		Opening tag for account from aggregate, see section 11.3.1
</BANKACCTFROM>		
<RECPMTTRNRQ>		Recurring-payment transactions (0 or more)
</RECPMTTRNRQ>		
</RECPMTSYNCRQ>		

12.10.2.2 Response <RECPMTSYNCRS>

Tag	Version	Description
<RECPMTSYNCRS>		Synchronization-response aggregate
<TOKEN>	V1 only	New synchronization token, <i>token</i>
<TOKEN2>	V2 only	New synchronization token, <i>token2</i>
<LOSTSYNC>		Y if the token in the synchronization request is older than the earliest entry in the server's history table. In this case, some responses have been lost. N if the token in the synchronization request is newer than or matches a token in the server's history table. <i>Boolean</i>
<SYNCERROR>	V2 only	Optional error code (See section 6.4.4.), <i>N-6</i> .
<BANKACCTFROM>		Opening tag for account from aggregate, see section 11.3.1
</BANKACCTFROM>		
<RECPMTTRNRS>		Recurring-payment transactions (0 or more)
</RECPMTTRNRS>		
</RECPMTSYNCRS>		

12.10.3 Discussion

This section describes specific synchronization processing for the OFX Payments functions. [Chapter 6, "Data Synchronization,"](#) provides a more extensive discussion of the OFX synchronization mechanism.

The client follows the steps below to synchronize:

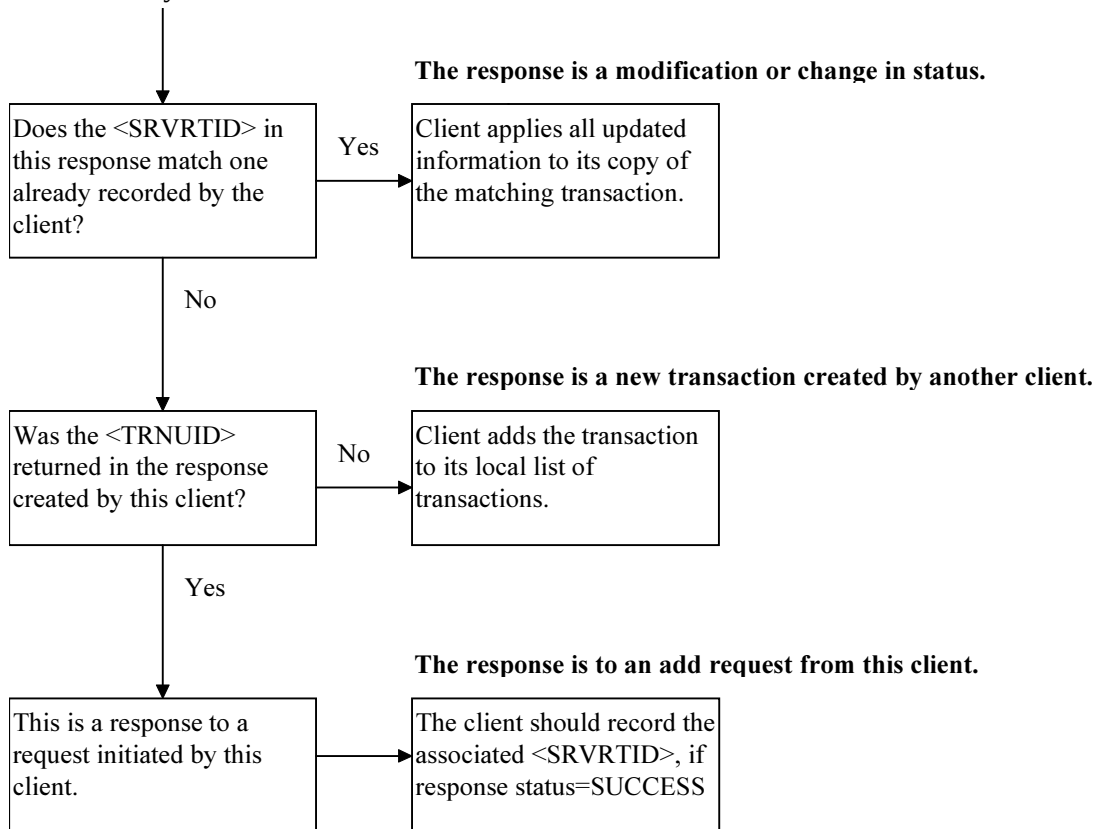
1. The client sends a <PMTSYNCRQ> and /or <RECPMTSYNCRQ> containing the token it has stored from its last successful synchronization (or the special initial token value).
2. The client processes the <PMTSYNCRS> and /or <RECPMTSYNCRS> response from the server.

When the client has requested the server to add a transaction, a response that contains a <TRNUID> matching a transaction originally sent by the client—for which the client has not recorded an associated <SRVRTID>—is the normal scenario. This scenario could also occur if the server response did not reach the client in the previous session. In either case, the client should add these server IDs to their associated transactions at this point.

If the client previously recorded the <SRVRTID>, this response is a change in status or in the contents of the transaction. The request might have originated from this client, another client, or might be the result of server processing.

If the <TRNUID> does not match any transaction known to the client, a second client initiated this transaction. In rarer cases the response might be a transaction initially requested by this client, for which the client has lost its record; for example, the client has reverted to a backup.

The diagram below describes the processing and interpretation of <SRVRTID> and <TRNUID> identifiers by the client:



After receiving the synchronization responses from the server, the client scans its database of transactions to verify that they have all been assigned a <SRVRTID>. Any transactions missing this identifier were never received by the server and should be resent (using the originally assigned <TRNUID> to avoid duplicate requests). Additionally, the client should record the <TOKEN> received in the response.

12.11 Message Sets and Profile

OFX separates messages that the client and server send into groups called message sets. Each financial institution defines the message sets that a particular institution will support. Currently, all the payment messages described in this chapter fall into a single message set.

The message set contains options and attributes that allow a financial institution to customize its use of OFX. The options and attributes are defined in the profile as part of the message set definition. Each set of options and attributes appears within an aggregate that is specific to a message set. Specifically, all of the options and attributes that pertain to payments are contained within <BILLPAYMSGSETV1> and <BILLPAYMSGSETV2>.

12.11.1 Bill Pay Message Sets and Messages

12.11.1.1 Bill Pay Message Set Request Messages

Clients should not send an empty <BILLPAYMSGSRQV1> or <BILLPAYMSGSRQV2> (although allowed by the DTD, such a message is meaningless). Although the DTD imposes a certain transaction order (payee transactions and sync requests go first), the transactions in <BILLPAYMSGSRQV1> and <BILLPAYMSGSRQV2> may be executed in any order.

Message Set	Messages
<BILLPAYMSGSET>	
<BILLPAYMSGSETV1>	
<BILLPAYMSGSRQV1>	PMTTRNRQ
	PMTRQ
	PMTMODRQ
	PMTCANCRRQ
	RECPMTTRNRQ
	RECPMTRQ
	RECPMTMODRQ
	RECPMTCANCRRQ
	PAYEETRNRQ
	PAYEERQ
	PAYEEMODRQ
	PAYEEDELRQ
	PMTINQTRNRQ
	PMTINQRQ
	PMTMAILTRNRQ
	PMTMAILRQ
	PMTSYNCRQ
	RECPMTSYNCRQ
	PAYEESYNCRQ
	PMTMAILSYNCRQ
</BILLPAYMSGSRQV1>	
</BILLPAYMSGSETV1>	

<i>Message Set</i>	<i>Messages</i>
<BILLPAYMSGSETV2> <BILLPAYMSGSRQV2>	PMTTRNRQ PMTREQ PMTMODRQ PMTCANCERQ RECPMTTRNRQ RECPMTRQ RECPMTMODRQ RECPMTCANCERQ PAYEETRNRQ PAYEERQ PAYEEMODRQ PAYEEDELRQ PMTINQTRNRQ PMTINQRQ PMTMAILTRNRQ PMTMAILRQ PMTSYNCRQ RECPMTSYNCRQ PAYEESYNCRQ PMTMAILSYNCRQ
</BILLPAYMSGSRQV2> </BILLPAYMSGSETV2> </BILLPAYMSGSET>	

12.11.1.2 Bill Pay Message Set Response Messages

<i>Message Set</i>	<i>Messages</i>
<BILLPAYMSGSET>	
<BILLPAYMSGSETV1>	
<BILLPAYMSGSRV1>	PMTTRNRS PMTRS PMTMODRS PMTCANCRS RECPMTTRNRS RECPMTRS RECPMTMODRS RECPMTCANCRS PAYEETRNR PAYEERS PAYEEMODRS PAYEEDELRS PMTINQTRNRS PMTINQRS PMTMAILTRNRS PMTMAILRS PMTSYNCRS RECPMTSYNCRS PAYEESYNCRS PMTMAILSYNCRS
</BILLPAYMSGSRV1>	
</BILLPAYMSGSETV1>	
<BILLPAYMSGSETV2>	

12.11.2 Bill Pay Message Set Profile <BILLPAYMSGSET>

Tag	Version	Description
<BILLPAYMSGSET>		
<BILLPAYMSGSETV1>		Version 1 of bill pay message set
<MSGSETCORE>		
</MSGSETCORE>		
<DAYSWITH>		Offset to withdrawal date, such that (DTDUE – DAYSTOPAY) + (DAYSWITH) determines the date on which the funds are withdrawn from the user's account. <i>N-3</i>
		Note: If <DAYSWITH>-1 is specified, then the withdrawal date is the same as the payment date (<DTDUE>).
<DFLTDAYSTOPAY>		Default number of days to pay by check (except by transfer), <i>N-3</i>
		Can be overridden for each payee, by <DAYSTOPAY> in the <EXTDPAYEE> aggregate, see section 12.5.3.3
<XFERDAYSWITH>		Number of days before processing date that funds are withdrawn for payment by transfer, <i>N-3</i>
<XFERDFLTDAYSTOPAY>		Default number of days to pay by transfer, <i>N-3</i>
		Can be overridden for each payee, by <DAYSTOPAY> in the <EXTDPAYEE> aggregate, see section 12.5.3.3
<PROCDAYSOFF>		Days of week that no processing occurs; 0 or more of (MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY). <PROCDAYSOFF> indicate days to exclude when calculating dates that utilize other bill payment bits, such as <DAYSWITH> and <DFLTDAYSTOPAY> values.
<PROCENDTM>		Time of day that day's processing ends, <i>time</i>
<MODELWND>		Model window; the number of days before a recurring transaction is scheduled to be processed that it is instantiated on the system; <MODELWND>0 indicates that the server will always maintain a single spawned payment for each model, <i>N-3</i>
<POSTPROCWND>		Number of days after a transaction is processed that it is accessible for status inquiries, <i>N-3</i>
<STSVIAMODS>		If Y, server supports communication of server-initiated payment status changes by means of the PMTMODRS message

Tag	Version	Description
<PMTBYADDR>	V2 only	The payment provider supports payments to payees identified by billing address, that is, the PAYEE aggregate, <i>Boolean</i>
<PMTBYXFER>		The payment provider supports payments to payees identified by destination account, <i>Boolean</i>
<PMTBYPAYEEID>		The payment provider supports payments to payees identified by a user-supplied payee ID, <i>Boolean</i>
<CANADDPAYEE>		User can add payees. if no, the user is restricted to payees added to the user's payee list by the payment system, <i>Boolean</i>
<HASEXTPMT>		Supports the EXTPMT business payment aggregate, <i>Boolean</i>
<CANMODPMTS>		Permits modifications to payments, that is PMTMODRQ, <i>Boolean</i>
<CANMODMDLS>		Permits modifications to models, that is REQPMTMODRQ, <i>Boolean</i>
<DIFFFIRSTPMT>		Support for specifying a different amount for the first payment generated by a model, <i>Boolean</i>
<DIFFLASTPMT>		Support for specifying a different amount for the last payment generated by a model, <i>Boolean</i>
</BILLPAYMSGSETV1>		Zero or more version 2 bill pay message sets
<BILLPAYMSGSETV2>		
<MSGSETCORE>		
</MSGSETCORE>		
<DAYSWITH>		Offset to withdrawal date, such that (DTDUE – DAYSTOPAY) + (DAYSWITH) determines the date on which the funds are withdrawn from the user's account. <i>N-3</i> Note: If <DAYSWITH>-1 is specified, then the withdrawal date is the same as the payment date (<DTDUE>).
<DFLTDAYSTOPAY>		Default number of days to pay by check (except by transfer), <i>N-3</i> Can be overridden for each payee, by <DAYSTOPAY> in the <EXTDPAYEE> aggregate, see section 12.5.3.3
<XFERDAYSWITH>		Number of days before processing date that funds are withdrawn for payment by transfer, <i>N-3</i>

Tag	Version	Description
<XFERDFLTDAYSTOPAY>		Default number of days to pay by transfer, <i>N-3</i> Can be overridden for each payee, by <DAYSTOPAY> in the <EXTDPAYEE> aggregate, see section 12.5.3.3 Note: If <XFERDFLTDAYSTOPAY>-1 is specified, then <DTDUE> is interpreted as an execution date (the date that the payer's bank will begin processing the payment) rather than a due date.
<PROCDAYSOFF>		Days of week that no processing occurs; 0 or more of (MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY). <PROCDAYSOFF> indicate days to exclude when calculating dates that utilize other bill payment bits, such as <DAYSWITH> and <DFLTDAYSTOPAY> values.
<PROCENDTM>		Time of day that day's processing ends, <i>time</i>
<MODELWND>		Model window; the number of days before a recurring transaction is scheduled to be processed that it is instantiated on the system, <MODELWND>0 indicates that the server will always maintain a single spawned payment for each model, <i>N-3</i>
<POSTPROCWND>		Number of days after a transaction is processed that it is accessible for status inquiries, <i>N-3</i>
<STSVIAMODS>		If Y, server supports communication of server-initiated payment status changes by means of the PMTMODRS message
<PMTBYADDR>		The payment provider supports payments to payees identified by billing address, that is, the PAYEE aggregate, <i>Boolean</i>
<PMTBYXFER>		The payment provider supports payments to payees identified by destination account, <i>Boolean</i>
<PMTBYPAYEEID>		The payment provider supports payments to payees identified by a user-supplied payee ID, <i>Boolean</i>
<CANADDPAYEE>		User can add payees. if no, the user is restricted to payees added to the user's payee list by the payment system, <i>Boolean</i>
<HASEXTDPMT>		Supports the EXTDPMT business payment aggregate for nonrecurring payments, <i>Boolean</i>
<HASRECEXTDPMT>		Supports the EXTDPMT business payment aggregate for recurring models, <i>Boolean</i>
<CANMODPMTS>		Permits modifications to payments, that is PMTMODRQ, <i>Boolean</i>

Tag	Version	Description
<CANMODMDLS>		Permits modifications to models, that is REQPMODRQ, <i>Boolean</i>
<DIFFFIRSTPMT>		Support for specifying a different amount for the first payment generated by a model, <i>Boolean</i>
<DIFFLASTPMT>		Support for specifying a different amount for the last payment generated by a model, <i>Boolean</i>
<NEEDTANPMT>		A <TAN> is required for user authentication in the transaction wrapper of a request to create, modify or cancel a payment or model (see section 2.4.6), <i>Boolean</i> When <COUNTRY> USA is specified, this will always be set to N.
<NEEDTANPAYEE>		A <TAN> is required for user authentication in the transaction wrapper of a request to create, modify or delete a payee (see section 2.4.6), <i>Boolean</i> When <COUNTRY> USA is specified, this will always be set to N.
<SUPPORTDTAVAIL>		Support for specifying a payment value date using the <DTAVAIL> element in the <PMTINFO> aggregate (see section 12.5.3), <i>Boolean</i> When <COUNTRY> USA is specified, this will always be set to N.
<CANMOTO>	1.6 add	Y if server supports <CCMOTOACCT> (Section 12.5.2) in the <PMTINFO2> (Section 12.5.3) aggregate. These OFX 1.6 additions must be explicitly supported by the server before they are used by the client. The default for this option is N. <i>Boolean</i>

Tag	Version	Description
<PAYEEMODPENDNG> </BILLPAYMSGSETV2> </BILLPAYMSGSET>	1.6 add	Describes server propagation of the changes to payees to pending payments. Changes to payees are always propagated to payment models. If server supports the use of <MODPENDING> element in the <PAYEEMODRQ> request, <PAYEEMODPENDING>IFREQUESTED must be specified. These OFX 1.6 additions must be explicitly supported by the server before they are used by a client. Valid values: NEVER = Changes to payees are never propagated to pending payments. The server must ignore the <MODPENDING> element in a <PAYEEMODRQ> request (possibly returning the <STATUS><CODE>2028 warning) if <PAYEEMODPENDING>NEVER is specified or if the element is omitted from the profile. This is the default behavior. IFREQUESTED = Changes to payees are propagated to pending payments when <MODPENDING>Y is specified in the Modify Payee request (<PAYEEMODRQ>) and not propagated when <MODPENDING>N is specified or that element is omitted from the request.

12.12 Examples

12.12.1 Scheduling a Payment

Create a payment to “J.C. Counts” for \$123.45 to be paid on October 1,1997 using funds in a checking account:

```
<!-- payment example 1 -->
<OFX>
  <SIGNONMSGSRQV1>
    <SONRQ>
      <!-- ...Sign on request. For a
      complete example, see section
      11.14.1-->

    </SONRQ>
  </SIGNONMSGSRQV1>
  <BILLPAYMSGSRQV1>
    <PMTTRNRQ>
      <TRNUID>1001
      <PMTRQ>
        <PMTINFO>
          <BANKACCTFROM>
            <BANKID>123432123
            <ACCTID>516273
            <ACCTTYPE>CHECKING
          </BANKACCTFROM>
          <TRNAMT>123.45
          <PAYEE>
            <NAME>J. C. Counts
            <ADDR1>100 Main St.
            <CITY>Turlock
            <STATE>CA
            <POSTALCODE>90101
            <PHONE>415.987.6543
          </PAYEE>
          <PAYACCT>10101
          <DTDUE>19971001
          <MEMO>payment #3
        </PMTINFO>
      </PMTRQ>
    </PMTTRNRQ>
  </BILLPAYMSGSRQV1>
</OFX>
```

The server responds, indicating that it will make the payment on the date requested and that the payee is a standard payee:

```
<OFX>
  <SIGNONMSGSRSV1>
    <SONRS>
      <!-- ...Sign on response. For a
      complete example, see section
      11.14.1-->

    </SONRS>
  </SIGNONMSGSRSV1>
  <BILLPAYMSGSRSV1>
    <PMTTRNRS>
      <TRNUID>1001
      <STATUS>
        <CODE>0
        <SEVERITY>INFO
      </STATUS>
      <PMTRS>
        <SRVRTID>1030155
        <PAYEELSTID>123214
        <CURDEF>USD
        <PMTINFO>
          <BANKACCTFROM>
            <BANKID>123432123
            <ACCTID>516273
            <ACCTTYPE>CHECKING
          </BANKACCTFROM>
          <TRNAMT>123.45
          <PAYEE>
            <NAME>J. C. Counts
            <ADDR1>100 Main St.
            <CITY>Turlock
            <STATE>CA
            <POSTALCODE>90101
            <PHONE>415.987.6543
          </PAYEE>
          <PAYEELSTID>123214
          <PAYACCT>10101
          <DTDUE>19971001
          <MEMO>payment #3
        </PMTINFO>
        <EXTDPAYEE>
          <PAYEEID>9076
```

```

        <IDSCOPE>USER
        <NAME>J. C. Counts
        <DAYSTOPAY>3
    </EXTDPAYEE>
    <PMTPRCSTS>
        <PMTPRCCODE>WILLPROCESSION
        <DTPMTPRC>19971001
    </PMTPRCSTS>
</PMTRS>
</PMTTRNRS>
</BILLPAYMSGSRSV1>
</OFX>

```

Create a second payment to the payee, using the payee ID returned in the previous example:

```

<!-- payment example 2 -->

<OFX>
    <SIGNONMSGSRQV1>
        <SONRQ>
            <!-- ...Sign on request. For a
            complete example, see section
            11.14.1-->

        </SONRQ>
    </SIGNONMSGSRQV1>
    <BILLPAYMSGSRQV1>
        <PMTTRNRQ>
            <TRNUID>1002
            <PMTRQ>
                <PMTINFO>
                    <BANKACCTFROM>
                        <BANKID>123432123
                        <ACCTID>516273
                        <ACCTTYPE>CHECKING
                    </BANKACCTFROM>
                    <TRNAMT>123.45
                    <PAYEEID>9076
                    <PAYEELSTID>123214
                    <PAYACCT>10101
                    <DTDUE>19971101
                    <MEMO>Payment #4
                </PMTINFO>
            </PMTRQ>
        </PMTTRNRQ>
    </BILLPAYMSGSRQV1>
</OFX>

```

```
</BILLPAYMSGSRQV1>
</OFX>
```

The server responds, indicating that it will make the payment on the date requested:

```
<OFX>
  <SIGNONMSGSRSV1>
    <SONRS>                                     <!-- ...Sign on response. For a
                                                    complete example, see section
                                                    11.14.1-->

    </SONRS>
  </SIGNONMSGSRSV1>
  <BILLPAYMSGSRSV1>
    <PMTTRNRS>
      <TRNUID>1002
      <STATUS>
        <CODE>0
        <SEVERITY>INFO
      </STATUS>
      <PMTRS>
        <SRVRTID>1068405
        <PAYEELSTID>123214
        <CURDEF>USD
        <PMTINFO>
          <BANKACCTFROM>
            <BANKID>123432123
            <ACCTID>516273
            <ACCTTYPE>CHECKING
          </BANKACCTFROM>
          <TRNAMT>123.45
          <PAYEEID>9076
          <PAYEELSTID>123214
          <PAYACCT>10101
          <DTDUE>19971101
          <MEMO>payment #4
        </PMTINFO>
        <EXTDPAYEE>
          <PAYEEID>9076
          <IDSCOPE>USER
          <NAME>J. C. Counts
          <DAYSTOPAY>3
        </EXTDPAYEE>
        <PMTPRCSTS>
```



```

        <PMTPRCCODE>WILLPROCESSON
        <DTPMTPRC>19971101
    </PMTPRCSTS>
</PMTRS>
</PMTTRNRS>
</BILLPAYMSGSRV1>
</OFX>

```

12.12.2 Modifying a Payment

Change the amount of the first payment to \$125.99

```

<OFX>
  <SIGNONMSGSRQV1>
    <SONRQ>
      <!-- ...Sign on request. For a
      complete example, see section
      11.14.1-->

    </SONRQ>
  </SIGNONMSGSRQV1>
  <BILLPAYMSGSRQV1>
    <PMTTRNRQ>
      <TRNUID>1021
      <PMTMODRQ>
        <SRVRTID>1030155
        <PMTINFO>
          <BANKACCTFROM>
            <BANKID>123432123
            <ACCTID>516273
            <ACCTTYPE>CHECKING
          </BANKACCTFROM>
          <TRNAMT>125.99      <!-- changed amount -->
          <PAYEE>
            <NAME>J. C. Counts
            <ADDR1>100 Main St.
            <CITY>Turlock
            <STATE>CA
            <POSTALCODE>90101
            <PHONE>415.987.6543
          </PAYEE>
          <PAYEELSTID>123214
          <PAYACCT>10101
          <DTDUE>19971001
        
```

```

        <MEMO>payment #3
    </PMTINFO>
</PMTMODRQ>
</PMTTRNRQ>
</BILLPAYMSGSRQV1>
</OFX>

```

The server responds:

```

<OFX>
  <SIGNONMSGSRSV1>
    <SONRS>
      <!-- ...Sign on response. For a
      complete example, see section
      11.14.1-->

    </SONRS>
  </SIGNONMSGSRSV1>
  <BILLPAYMSGSRSV1>
    <PMTTRNRS>
      <TRNUID>1021
      <STATUS>
        <CODE>0
        <SEVERITY>INFO
      </STATUS>
      <PMTMODRS>
        <SRVRTID>1030155
        <PMTINFO>
          <BANKACCTFROM>
            <BANKID>123432123
            <ACCTID>516273
            <ACCTTYPE>CHECKING
          </BANKACCTFROM>
          <TRNAMT>125.99      <!-- changed amount -->
          <PAYEE>
            <NAME>J. C. Counts
            <ADDR1>100 Main St.
            <CITY>Turlock
            <STATE>CA
            <POSTALCODE>90101
            <PHONE>415.987.6543
          </PAYEE>
          <PAYEELSTID>123214
          <PAYACCT>10101
          <DTDUE>19971001

```

```

        <MEMO>payment #3
    </PMTINFO>
</PMTMODRS>
</PMTTRNRS>
</BILLPAYMSGSRSV1>
</OFX>

```

Change the date of the same payment to December 12, 1997.

```

<OFX>
  <SIGNONMSGSRQV1>
    <SONRQ>
      <!-- ...Sign on request. For a
      complete example, see section
      11.14.1-->

    </SONRQ>
  </SIGNONMSGSRQV1>
  <BILLPAYMSGSRQV1>
    <PMTTRNRQ>
      <TRNUID>32456
      <PMTMODRQ>
        <SRVRTID>1030155
        <PMTINFO>
          <BANKACCTFROM>
            <BANKID>123432123
            <ACCTID>516273
            <ACCTTYPE>CHECKING
          </BANKACCTFROM>
          <TRNAMT>125.99
          <PAYEE>
            <NAME>J. C. Counts
            <ADDR1>100 Main St.
            <CITY>Turlock
            <STATE>CA
            <POSTALCODE>90101
            <PHONE>415.987.6543
          </PAYEE>
          <PAYEELSTID>123214
          <PAYACCT>10101
          <DTDUE>19971212      <!-- changed date -->
          <MEMO>payment #3
        </PMTINFO>
      </PMTMODRQ>
    </PMTTRNRQ>

```

```
</BILLPAYMSGSRQV1>
</OFX>
```

The server responds:

```
<OFX>
  <SIGNONMSGSRSV1>
    <SONRS>
      <!-- ...Sign on response. For a
      complete example, see section
      11.14.1-->

    </SONRS>
  </SIGNONMSGSRSV1>
  <BILLPAYMSGSRSV1>
    <PMTTRNRS>
      <TRNUID>32456
      <STATUS>
        <CODE>0
        <SEVERITY>INFO
      </STATUS>
      <PMTMODRS>
        <SRVRTID>1030155
        <PMTINFO>
          <BANKACCTFROM>
            <BANKID>123432123
            <ACCTID>516273
            <ACCTTYPE>CHECKING
          </BANKACCTFROM>
          <TRNAMT>125.99
          <PAYEE>
            <NAME>J. C. Counts
            <ADDR1>100 Main St.
            <CITY>Turlock
            <STATE>CA
            <POSTALCODE>90101
            <PHONE>415.987.6543
          </PAYEE>
          <PAYEELSTID>123214
          <PAYACCT>10101
          <DTDUE>19971212      <!-- changed date -->
          <MEMO>payment #3
        </PMTINFO>
      </PMTMODRS>
    </PMTTRNRS>
```

```
</BILLPAYMSGSRV1>
</OFX>
```

12.12.3 Canceling a Payment

Cancel a payment:

```
<OFX>
  <SIGNONMSGSRQV1>
    <SONRQ>
      <!-- ...Sign on request. For a
      complete example, see section
      11.14.1-->

    </SONRQ>
  </SIGNONMSGSRQV1>
  <BILLPAYMSGSRQV1>
    <PMTTRNRQ>
      <TRNUID>54601
      <PMTCANCRCQ>
        <SRVRTID>1030155
      </PMTCANCRCQ>
    </PMTTRNRQ>
  </BILLPAYMSGSRQV1>
</OFX>
```

The server responds:

```
<OFX>
  <SIGNONMSGSRSV1>
    <SONRS>
      <!-- ...Sign on response. For a
      complete example, see section
      11.14.1-->

    </SONRS>
  </SIGNONMSGSRSV1>
  <BILLPAYMSGSRSV1>
    <PMTTRNRS>
      <TRNUID>54601
      <STATUS>
        <CODE>0
        <SEVERITY>INFO
      </STATUS>
      <PMTCANCRCRS>
        <SRVRTID>1030155
      </PMTCANCRCRS>
    </PMTTRNRS>
```

```
</BILLPAYMSGSRV1>
</OFX>
```

12.12.4 Updating Payment Status

Update payment status:

```
<OFX>
  <SIGNONMSGSRQV1>
    <SONRQ>
      <!-- ...Sign on request. For a
      complete example, see section
      11.14.1-->

    </SONRQ>
  </SIGNONMSGSRQV1>
  <BILLPAYMSGSRQV1>
    <PMTINQTRNRQ>
      <TRNUID>7865
      <PMTINQRQ>
        <SRVRTID>565321
      </PMTINQRQ>
    </PMTINQTRNRQ>
  </BILLPAYMSGSRQV1>
</OFX>
```

The server responds with updated status and check number:

```
<OFX>
  <SIGNONMSGSRV1>
    <SONRS>
      <!-- ...Sign on response. For a
      complete example, see section
      11.14.1-->

    </SONRS>
  </SIGNONMSGSRV1>
  <BILLPAYMSGSRV1>
    <PMTINQTRNRS>
      <TRNUID>7865
      <STATUS>
        <CODE>0
        <SEVERITY>INFO
      </STATUS>
    <PMTINQRS>
      <SRVRTID>565321
      <PMTPRCSTS>
        <PMTPRCCODE>PROCESSEDON
```

```

        <DTPMTPRC>19970201
    </PMTPRCSTS>
    <CHECKNUM>6017
</PMTINQRS>
</PMTINQTRNRS>
</BILLPAYMSGSRV1>
</OFX>

```

12.12.5 Scheduling a Recurring Payment

Create a recurring payment of 36 monthly payments of \$395 to a (previously known) standard payee. The first payment will be on November 15, 1997:

```

<OFX>
  <SIGNONMSGSRQV1>
    <SONRQ>
      <!-- ...Sign on request. For a
      complete example, see section
      11.14.1-->

    </SONRQ>
  </SIGNONMSGSRQV1>
  <BILLPAYMSGSRQV1>
    <RECPMTTRNRQ>
      <TRNUID>12354
      <RECPMTRQ>
        <RECURRINST>
          <NINSTS>36
          <FREQ>MONTHLY
        </RECURRINST>
        <PMTINFO>
          <BANKACCTFROM>
            <BANKID>555432180
            <ACCTID>763984
            <ACCTTYPE>CHECKING
          </BANKACCTFROM>
          <TRNAMT>395.00
          <PAYEEID>77810
          <PAYEELSTID>27983
          <PAYACCT>444-78-97572
          <DTDUE>19971115
          <MEMO>Auto loan payment
        </PMTINFO>
      </RECPMTRQ>
    </RECPMTTRNRQ>

```

```
</BILLPAYMSGSRQV1>
</OFX>
```

The server responds with the assigned server transaction ID:

```
<OFX>
  <SIGNONMSGSRSV1>
    <SONRS>
      <!-- ...Sign on response. For a
      complete example, see section
      11.14.1-->

    </SONRS>
  </SIGNONMSGSRSV1>
  <BILLPAYMSGSRSV1>
    <RECPMTTRNRS>
      <TRNUID>12345
      <STATUS>
        <CODE>0
        <SEVERITY>INFO
      </STATUS>
      <RECPMTRS>
        <RECSRVRTID>387687138
        <PAYEELSTID>27983
        <CURDEF>USD
        <RECURRINST>
          <NINSTS>36
          <FREQ>MONTHLY
        </RECURRINST>
        <PMTINFO>
          <BANKACCTFROM>
            <BANKID>555432180
            <ACCTID>763984
            <ACCTTYPE>CHECKING
          </BANKACCTFROM>
          <TRNAMT>395.00
          <PAYEEID>77810
          <PAYEELSTID>27983
          <PAYACCT>444-78-97572
          <DTDUE>19971115
          <MEMO>Auto loan payment
        </PMTINFO>
        <EXTDPAYEE>
          <PAYEEID>77810
          <IDSCOPE>USER
```



```

        <NAME>Mel's Used Cars
        <DAYSTOPAY>3
    </EXTDPAYEE>
</RECPMTRS>
</RECPMTTRNRS>
</BILLPAYMSGSRV1>
</OFX>

```

12.12.6 Modifying a Recurring Payment

Change the amount of a recurring payment:

```

<OFX>
  <SIGNONMSGSRQV1>
    <SONRQ>                                     <!-- ...Sign on request. For a
                                                    complete example, see section
                                                    11.14.1-->

    </SONRQ>
  </SIGNONMSGSRQV1>
  <BILLPAYMSGSRQV1>
    <RECPMTTRNRQ>
      <TRNUID>98765
      <RECPMTMODRQ>
        <RECSRVRTID>387687138
        <RECURRINST>
          <NINSTS>36
          <FREQ>MONTHLY
        </RECURRINST>
        <PMTINFO>
          <BANKACCTFROM>
            <BANKID>555432180
            <ACCTID>763984
            <ACCTTYPE>CHECKING
          </BANKACCTFROM>
          <TRNAMT>399.95      <!-- changing amount -->
          <PAYEEID>77810
          <PAYEELSTID>27983
          <PAYACCT>444-78-97572
          <DTDUE>19971115
          <MEMO>Auto loan payment
        </PMTINFO>
        <MODPENDING>N
      </RECPMTMODRQ>
    </BILLPAYMSGSRQV1>
  </OFX>

```

```

    </RECPMTTRNRQ>
  </BILLPAYMSGSRQV1>
</OFX>

```

The server responds:

```

<OFX>
  <SIGNONMSGSRSV1>
    <SONRS>                                     <!-- ...Sign on response. For a
                                                    complete example, see section
                                                    11.14.1-->

    </SONRS>
  </SIGNONMSGSRSV1>
  <BILLPAYMSGSRSV1>
    <RECPMTTRNRS>
      <TRNUID>98765
      <STATUS>
        <CODE>0
        <SEVERITY>INFO
      </STATUS>
      <RECPMTMODRS>
        <RECSRVRTID>387687138
        <RECURINST>
          <NINSTS>36
          <FREQ>MONTHLY
        </RECURINST>
        <PMTINFO>
          <BANKACCTFROM>
            <BANKID>555432180
            <ACCTID>763984
            <ACCTTYPE>CHECKING
          </BANKACCTFROM>
          <TRNAMT>399.95      <!-- changing amount -->
          <PAYEEID>77810
          <PAYEELSTID>27983
          <PAYACCT>444-78-97572
          <DTDUE>19971115
          <MEMO>Auto loan payment
        </PMTINFO>
        <MODPENDING>N
      </RECPMTMODRS>
    </BILLPAYMSGSRSV1>
  </OFX>

```

```

    </RECPMTTRNRS>
  </BILLPAYMSGSRV1>
</OFX>

```

12.12.7 Canceling a Recurring Payment

Cancel a recurring payment:

```

<OFX>
  <SIGNONMSGSRQV1>
    <SONRQ>
      <!-- ...Sign on request. For a
      complete example, see section
      11.14.1-->

      </SONRQ>
    </SIGNONMSGSRQV1>
  <BILLPAYMSGSRQV1>
    <RECPMTTRNRQ>
      <TRNUID>11122
      <RECPMTCANCRQ>
        <RECSRVRTID>387687138
        <CANPENDING>Y
      </RECPMTCANCRQ>
    </RECPMTTRNRQ>
  </BILLPAYMSGSRQV1>
</OFX>

```

The server responds:

```

<OFX>
  <SIGNONMSGSRV1>
    <SONRS>
      <!-- ...Sign on response. For a
      complete example, see section
      11.14.1-->

      </SONRS>
    </SIGNONMSGSRV1>
  <BILLPAYMSGSRV1>
    <RECPMTTRNRS>
      <TRNUID>11122
      <STATUS>
        <CODE>0
        <SEVERITY>INFO
      </STATUS>
      <RECPMTCANCRS>
        <RECSRVRTID>387687138

```

```

        <CANPENDING>Y
    </RECPMTCANCRS>
    </RECPMTTRNRS>
</BILLPAYMSGSRV1>
</OFX>

```

12.12.8 Adding a Payee to the Payee List

The user sends a request to add a payee to the user's payee list:

```

<OFX>
  <SIGNONMSGSRQV1>
    <SONRQ>
      <!-- ...Sign on request. For a
      complete example, see section
      11.14.1-->

    </SONRQ>
  </SIGNONMSGSRQV1>
  <BILLPAYMSGSRQV1>
    <PAYEETRNRQ>
      <TRNUID>127677
      <PAYEERQ>
        <PAYEE>
          <NAME>ACME Rocket Works
          <ADDR1>101 Spring St.
          <ADDR2>Suite 503
          <CITY>Watkins Glen
          <STATE>NY
          <POSTALCODE>12345-6789
          <PHONE>888.555.1212
        </PAYEE>
        <PAYACCT>1001-99-8876
      </PAYEERQ>
    </PAYEETRNRQ>
  </BILLPAYMSGSRQV1>
</OFX>

```

The server responds:

```
<OFX>
  <SIGNONMSGSRSV1>
    <SONRS>
      <!-- ...Sign on response. For a
      complete example, see section
      11.14.1-->

    </SONRS>
  </SIGNONMSGSRSV1>
  <BILLPAYMSGSRSV1>
    <PAYEETRNR>
      <TRNUID>127677
      <STATUS>
        <CODE>0
        <SEVERITY>INFO
      </STATUS>
      <PAYEERS>
        <PAYEELSTID>78096786
        <PAYEE>
          <NAME>ACME Rocket Works
          <ADDR1>101 Spring St.
          <ADDR2>Suite 503
          <CITY>Watkins Glen
          <STATE>NY
          <POSTALCODE>12345-6789
          <PHONE>888.555.1212
        </PAYEE>
        <EXTDPAYEE>
          <PAYEEID>88878
          <IDSCOPE>GLOBAL
          <NAME>ACME Rocket Works, Inc.
          <DAYSTOPAY>2
        </EXTDPAYEE>
        <PAYACCT>1001-99-8876
      </PAYEERS>
    </PAYEETRNR>
  </BILLPAYMSGSRSV1>
</OFX>
```

12.12.9 Synchronizing Scheduled Payments

A client wishes to obtain all Payments active on the server for a particular account:

```
<OFX>
  <SIGNONMSGSRQV1>
    <SONRQ>                                     <!-- ...Sign on request. For a
                                                    complete example, see section
                                                    11.14.1-->

    </SONRQ>
  </SIGNONMSGSRQV1>
  <BILLPAYMSGSRQV1>
    <PMTSYNCRQ>
      <REFRESH>Y
      <REJECTIFMISSING>N
      <BANKACCTFROM>
        <BANKID>123432123
        <ACCTID>516273
        <ACCTTYPE>CHECKING
      </BANKACCTFROM>
    </PMTSYNCRQ>
  </BILLPAYMSGSRQV1>
</OFX>
```

Assuming the only activity on this account has been the two payments created above, the server responds with one payment since the other payment was cancelled. The server also includes the current <TOKEN> value.

Note: If the one outstanding payment had a modification to it, the modification should have been integrated into the one <PMTRS> since this is a refresh, not a sync of all history. In that case, <TRNUID>0 must be returned in the response transaction (no client initiated an exact matching transaction).

```
<OFX>
  <SIGNONMSGSRSV1>
    <SONRS>                                     <!-- ...Sign on response. For a
                                                    complete example, see section
                                                    11.14.1-->

    </SONRS>
  </SIGNONMSGSRSV1>
  <BILLPAYMSGSRSV1>
    <PMTSYNCRS>
      <TOKEN>3247989384
      <BANKACCTFROM>
        <BANKID>123432123
```

```

    <ACCTID>516273
    <ACCTTYPE>CHECKING
  </BANKACCTFROM>
  <PMTTRNRS>
    <TRNUID>0
    <STATUS>
      <CODE>0
      <SEVERITY>INFO
    </STATUS>
    <PMTRS>
      <SRVRTID>1068405
      <PAYEELSTID>123214
      <CURDEF>USD
      <PMTINFO>
        <BANKACCTFROM>
          <BANKID>123432123
          <ACCTID>516273
          <ACCTTYPE>CHECKING
        </BANKACCTFROM>
        <TRNAMT>123.45
        <PAYEEID>9076
        <PAYEELSTID>123214
        <PAYACCT>10101
        <DTDUE>19971001
        <MEMO>payment #4
      </PMTINFO>
      <EXTDPAYEE>
        <PAYEEID>9076
        <IDSCOPE>USER
        <NAME>J. C. Counts
        <DAYSTOPAY>3
      </EXTDPAYEE>
      <PMTPRCSTS>
        <PMTPRCCODE>WILLPROCESSION
        <DTPMTPRC>19971001
      </PMTPRCSTS>
    </PMTRS>
  </PMTTRNRS>
</PMTSYNCRS>
</BILLPAYMSGSRV1>
</OFX>

```


CHAPTER 13 INVESTMENTS

OFX supports download of security information and detailed investment account statements including transactions, open orders, balances, and positions.

<i>Client Sends</i>	<i>Server Responds</i>
Account identifier Whether to download open orders Whether to download transactions Date range if transactions should be downloaded Whether to download positions Whether to download balances Additional securities to send information about	 Date and time for statement Default currency for statement Account identifier Investment transactions Banking transactions Open orders Positions Account balances Available Cash Balance Short Balance Margin Balance Buying power Marketing message List of securities

Note: This release of OFX does not support trading or tax lots.

13.1 Types of Response Information

The response consists of five types of information:

- ◆ Transactions – a combination of bank transaction detail records and investment transaction detail records. Transactions only within the specified start and stop dates are sent.
- ◆ Positions – positions a user has at a brokerage. Each statement response must contain a complete set of position records, even if no transactions occurred in the requested statement period for a particular holding.
- ◆ Balances – current balances typically reported on an FI statement, such as cash balance or buying power. They can also convey other numbers of interest, such as current interest rates.
- ◆ Open Orders – current open trading orders that a user has at a brokerage.
- ◆ Securities – any security referenced in either transactions, positions, open orders or explicitly requested.

13.2 Sub-Accounts

Many FIs distinguish between activity and positions in cash, margin, and short accounts, with some FIs having many other types of “sub-accounts.” OFX defines four standard types of sub-accounts: Cash, Margin, Short, Other. Position, Transaction, and Open Order records identify the sub-account.

13.3 Units, Precision, and Signs

This section provides information about numerical values for investment transactions. For more information about common data types used within OFX, refer to [Chapter 3, "Common Aggregates, Elements, and Data Types."](#)

13.3.1 Units

The units for security units and unit price are those commonly used on brokerage statements, and differ for each type of security.

- ◆ Stocks and Other – use number of shares for units and dollar value for unit price.
- ◆ Mutual Funds – in most cases shares are used, but in some cases the dollar value is used. The unit type is specified in cases in which it can be either.
- ◆ Bonds – use face value for units and percentage of par for unit price. For example, a \$25,000 bond trading at \$88 would use 25000 as the units and 88 as the unit price.
- ◆ Options – use number of contracts (not shares) for units, and price per share (not contract) for unit price.

13.3.2 Precision

OFX does not specify the precision of fields since the precision is client-dependent. However, it is recommended that clients and servers follow these rules:

- ◆ Clients and servers should send as much precision as they have
- ◆ Clients and servers should use a precision equal to or better than 1/256 of a share

13.3.3 Signs

Chapter 3, "Common Aggregates, Elements, and Data Types," describes how to use positive and negative numbers. Briefly, quantities and total values should be signed from the perspective of the user. In a stock buy, the total value is negative, the unit price is always positive, and the number of units is positive.

UNITS and TOTALS are signed from the perspective of the user (positive currency amount for SELLS, negative currency amount for BUYS). All other Investment transaction amounts are always positively signed. In other words UNITPRICE, COMMISSION, FEES, TAXES, WITHHOLDING, LOAD, MARKUP and MARKDOWN are always positive numbers.

A positive COMMISSION, TAXES, LOAD, WITHHOLDING, or FEE increases the negatively signed TOTAL on a BUYSTOCK and decreases the positively signed TOTAL on a SELLSTOCK.

MARKUP and MARKDOWN increase or decrease, respectively, the UNITPRICE.

Servers should return corrections to investment buys or sells as the opposite transaction type, e.g., a correction to a buy is returned as a sell. A correction to MARGININTEREST or RETOFCAP is returned as an INVEXPENSE.

13.4 Bank and Investment Transactions

Many FIs provide investment accounts that allow users to write checks and perform other traditional banking transactions, as well as investment transactions. OFX requires FIs to indicate in the download whether check-writing privileges exist for a given account.

FIs need to use the correct transaction record, bank or investment, for each real-world transaction. Use the following guidelines:

- ◆ Checks, electronic funds transfers, and ATM transactions associated with CMA or money market sweep accounts are always represented with a bank transaction record.
- ◆ Investment actions that involve securities (buy, sell, stock split, reinvest, etc.) are always represented with an investment record. Actions that are cash-only but are directly associated with a security are also investment actions (for example, dividends).
- ◆ Other cash-only actions require careful analysis by the FI. Those that affect investment performance analysis should be sent using the appropriate investment action (investment income - miscellaneous, investment expense). Those that are completely unrelated to investment should be sent as a bank record.

13.5 Money Market Funds

Money market funds can be handled in one of three different ways depending on how the fund is modeled at the financial institution

- ◆ Separate account at the financial institution
- ◆ Sweep account within an investment account
- ◆ Position within an investment account

13.5.1 Separate Account at the Financial Institution

In this case, the money market fund is in its own account with its own account number, distinct from the investment account. In OFX, you should model the money market fund as a separate money market bank account; see [Chapter 11, "Banking."](#) The banking <STMTRQ> request aggregate and <STMTRS> response aggregate will be used to download transactions.

13.5.2 Sweep Account Within an Investment Account

OFX uses the money market as a “sweep” account, where cash is “swept” as needed when buying and selling securities. The money market fund does not have its own account number. The customer sees the money market fund as an investment-account cash balance. In OFX, checks, ATMs, electronic fund transfer, deposit, and withdrawal transactions should be downloaded using banking transactions within the investment account. However, the sweep transactions in and out of the money market fund should not be downloaded to the client.

13.5.3 Position Within an Investment Account

The customer purchases the money market fund and is held in the account as a position. The money market fund does not have its own account number. In OFX, the money market fund should be returned as a <POSOTHER> position in the <INVPOSLIST>, with a <UNITPRICE> of 1.00 and <UNITS> as the current value of the position. Purchases and redemptions should be modeled as <BUYOTHER> and <SELLOTHOTHER> transactions with a <UNITPRICE> of 1.00 and <UNITS> as the transaction amount.

13.6 Investment Accounts

Investment account information is downloaded using the account information response aggregate <ACCTINFORS>. For more information, refer to [Chapter 8, "Activation & Account Information."](#) <INVACCTFROM> specifies the account. The <INVACCTINFO> aggregate specifies the investment-specific information.

13.6.1 Specifying the Investment Account <INVACCTFROM>

Tag	Description
<INVACCTFROM>	Account-from aggregate
<BROKERID>	Unique identifier for the FI, A-22
<ACCTID>	Account number at FI, A-22
</INVACCTFROM>	

Brokers should use the domain name of their company’s URL as the BROKERID, e.g.,

If URL=www.broker.com
then BROKERID=broker.com

The <INVACCTTO> aggregate contains the same elements.

13.6.2 Investment Account Information <INVACCTINFO>

The <INVACCTINFO> aggregate should appear in the <ACCTINFO> aggregate for accounts that support investment statement download. For more information about the <ACCTINFO> aggregate, refer to [Chapter 8, "Activation & Account Information."](#)

Tag	Version	Description
<INVACCTINFO>		Investment-account-information-record aggregate
<INVACCTFROM>		Account at FI
</INVACCTFROM>		
<USPRODUCTTYPE>		Classification of account. See section 13.6.2.1 for values
<CHECKING>		Whether the account has check writing privileges, <i>Boolean</i>
<SVCSTATUS>	V1 only	Activation status for investment statement download for the account. ACTIVE (signed up), PEND (in the process of signing up), AVAIL (have not signed up).
<SVCSTATUS2>	V2 only	AVAIL = Available, but not yet requested PEND = Requested, but not yet available ACTIVE = In use REJECTED = Rejected
<REASON>	V2 only	This element is only relevant if <SVCSTATUS2>REJECTED is specified, A-255
<INVACCTTYPE>		Type of account. INDIVIDUAL, JOINT, TRUST, CORPORATE
<OPTIONLEVEL>		Text description of option trading privileges, A-40
</INVACCTINFO>		

If an investment account has payments functionality, the analogous PMTINFO aggregate (see [12.5.3](#)) should also be sent in the ACCTINFO for the account. Payment information will be sent using the message sets described in the [12.5.3 , "Payment Information <PMTINFO> <PMTINFO2>."](#)

13.6.2.1 Values for <USPRODUCTTYPE>

<USPRODUCTTYPE> classifies accounts according to their account type. Valid values are:

<i>Product Type</i>	<i>Description</i>
401K	A 401(K) account
403B	A 403(B) account
IRA	An IRA account
KEOGH	Keogh (Money Purchase / Profit Sharing)
OTHER	Other account type
SARSEP	Salary Reduction Simplified Employer Pension plan
SIMPLE	Savings Incentive Match Plan for employees
NORMAL	Regular account
TDA	Tax Deferred Annuity
TRUST	Trust (including UTMA)
UGMA	Custodial account

13.6.2.2 International Note

The <USPRODUCTTYPE> element is intended for use by FIs in the United States. OFX will be expanded to provide equivalent elements to support the needs for other countries.

13.6.3 Brokerage, Mutual Fund, and 401K Accounts

Investment accounts include brokerage accounts, mutual fund accounts, 401K accounts, and other retirement accounts. OFX supports transactions, positions, balances, and open orders for all of these account types.

13.7 Investment Message Sets and Profile

OFX separates messages that the client and server send into groups called message sets. Each financial institution defines the message sets that the institution supports. The messages described in this chapter fall into two message sets:

- ◆ Investment Statement Download
- ◆ Security Information

Each message set contains options that allow a financial institution to customize its use of OFX. For example, an institution can support the Investment Statement Download Set (INVSTMTMSGSETV1 and / or INVSTMTMSGSETV2), but it can choose not to support the download of open orders.

The options and attributes are defined in the profile as part of each message set definition. Each set of options and attributes appears within an aggregate that is specific to a message set. For example, <INVSTMTMSGSETV1> and / or <INVSTMTMSGSETV2> contains all the options and attributes that pertain to investment statement download.

13.7.1 Investment Statement Download

13.7.1.1 Investment Statement Message Set Profile <INVSTMTMSGSET>

The investment statement message set profile aggregate <INVSTMTMSGSET> is used in the response to a financial institution profile request (see [Chapter 7, "FI Profile"](#)) to specify which activities it supports.

Tag	Description
<INVSTMTMSGSET>	Investment-statement-message-set-profile aggregate
<INVSTMTMSGSETV1>	Version 1 message set
<MSGSETCORE>	Common message set information, see Chapter 7, "FI Profile"
</MSGSETCORE>	
<TRANDNLD>	Whether the FI server downloads investment statement transactions, <i>Boolean</i>
<ODNLD>	Whether the FI server downloads investment open orders, <i>Boolean</i>
<POSDNLD>	Whether the FI server downloads investment statement positions, <i>Boolean</i>
<BALDNLD>	Whether the FI server downloads investment balances, <i>Boolean</i>
<CANEMAIL>	Whether the FI supports investment e-mail. Use generic e-mail profile to specify whether generic e-mail is supported; see Chapter 9, "Customer to FI Communication." <i>Boolean</i>
</INVSTMTMSGSETV1>	
<INVSTMTMSGSETV2>	Zero or more version 2 message sets
<MSGSETCORE>	Common message set information, see Chapter 7, "FI Profile"
</MSGSETCORE>	
<TRANDNLD>	Whether the FI server downloads investment statement transactions, <i>Boolean</i>
<ODNLD>	Whether the FI server downloads investment open orders, <i>Boolean</i>
<POSDNLD>	Whether the FI server downloads investment statement positions, <i>Boolean</i>
<BALDNLD>	Whether the FI server downloads investment balances, <i>Boolean</i>
<CANEMAIL>	Whether the FI supports investment e-mail. Use generic e-mail profile to specify whether generic e-mail is supported; see Chapter 9, "Customer to FI Communication." <i>Boolean</i>
</INVSTMTMSGSETV2>	
</INVSTMTMSGSET>	

13.7.1.2 Investment Statement Message Set and Messages

13.7.1.2.1 Investment Statement Message Set Request Messages

Tag	Description
<INVSTMTMSGSET>	
<INVSTMTMSGSETV1>	
<INVSTMTMSGSRQV1>	INVSTMTTRNRQ
	INVSTMTTRQ
	INVMAILTRNRQ
	INVMAILRQ
	INVMAILSYNCRQ
</INVSTMTMSGSRQV1>	
</INVSTMTMSGSETV1>	
<INVSTMTMSGSETV2>	
<INVSTMTMSGSRQV2>	INVSTMTTRNRQ
	INVSTMTTRQ
	INVMAILTRNRQ
	INVMAILRQ
	INVMAILSYNCRQ
</INVSTMTMSGSRQV2>	
</INVSTMTMSGSETV2>	
</INVSTMTMSGSET>	

13.7.1.2.2 Investment Statement Message Set Response Messages

<i>Tag</i>	<i>Description</i>
<INVSTMTMSGSET>	
<INVSTMTMSGSETV1>	
<INVSTMTMSGSRSV1>	INVSTMTRNRS
	INVSTMTRS
	INVMAILTRNRS
	INVMAILRS
	INVMAILSYNCRS
</INVSTMTMSGSRSV1>	
</INVSTMTMSGSETV1>	
<INVSTMTMSGSETV2>	
<INVSTMTMSGSRSV2>	INVSTMTRNRS
	INVSTMTRS
	INVMAILTRNRS
	INVMAILRS
	INVMAILSYNCRS
</INVSTMTMSGSRSV2>	
</INVSTMTMSGSETV2>	
</INVSTMTMSGSET>	

13.7.2 Security Information

13.7.2.1 Security List Message Set Profile <SECLISTMSGSET>

The security list message set profile aggregate <SECLISTMSGSET> is used in the response to an FI profile request (see Chapter 7, “FI Profile”) to specify which activities it supports.

Tag	Description
<SECLISTMSGSET>	Security-information-message-set-profile aggregate
<SECLISTMSGSETV1>	Version 1 message set
<MSGSETCORE>	Common message set information, see Chapter 7, "FI Profile"
</MSGSETCORE>	
<SECLISTRQDNLD>	Whether the FI server responds to security list requests, <i>Boolean</i>
</SECLISTMSGSETV1>	
<SECLISTMSGSETV2>	Zero or more version 2 message sets
<MSGSETCORE>	Common message set information, Chapter 7, "FI Profile"
</MSGSETCORE>	
<SECLISTRQDNLD>	Whether the FI server responds to security list requests, <i>Boolean</i>
</SECLISTMSGSETV2>	
</SECLISTMSGSET>	

13.7.2.2 Security List Message Set and Messages

13.7.2.2.1 Security List Message Set Request Messages

<i>Tag</i>	<i>Description</i>
<SECLISTMSGSET>	
<SECLISTMSGSETV1>	
<SECLISTMSGSRQV1>	SECLISTTRNRQ
	SECLISTRQ
</SECLISTMSGSRQV1>	
</SECLISTMSGSETV1>	
<SECLISTMSGSETV2>	
<SECLISTMSGSRQV2>	SECLISTTRNRQ
	SECLISTRQ
</SECLISTMSGSRQV2>	
</SECLISTMSGSETV2>	
</SECLISTMSGSET>	

13.7.2.2.2 Security List Message Set Response Messages

<i>Tag</i>	<i>Description</i>
<SECLISTMSGSET>	
<SECLISTMSGSETV1>	
<SECLISTMSGSRSV1>	SECLISTTRNRS
	SECLISTR
	SECLIST
</SECLISTMSGSRSV1>	
</SECLISTMSGSETV1>	
<SECLISTMSGSETV2>	
<SECLISTMSGSRSV2>	SECLISTTRNRS
	SECLISTR
	SECLIST
</SECLISTMSGSRSV2>	
</SECLISTMSGSETV2>	
</SECLISTMSGSET>	

Note: The <SECLISTMSGSRSV1> and <SECLISTMSGSRSV2> aggregates may appear in a response file when no corresponding <SECLISTMSGSRQV1> and <SECLISTMSGSRQV2> aggregates appeared in the request file. Servers should add the message set response wrapper only when downloading a statement and providing the <SECLIST>.

13.8 Investment Securities

13.8.1 Security Identification <SECID>

Securities must be consistently identified to allow client applications to prepare accurate investment reports across all user investment accounts, even at multiple FIs. At this time, neither a security name nor its symbol is standardized. Therefore, OFX uses CUSIP numbers (a unique 9-digit alphanumeric identifier) to identify securities. CUSIP numbers are available for the vast majority of securities traded today, including those without symbols such as bonds. For a security that does not have a CUSIP, a financial institution must follow the standard procedure of assigning a CUSIP by using itself as the issuer to avoid conflict with any other CUSIP.

Tag	Description
<SECID>	Security-identifier aggregate
<UNIQUEID>	Unique identifier for the security. CUSIP for US FIs. A-32
<UNIQUEIDTYPE>	Name of standard used to identify the security i.e., “CUSIP” for FIs in the United States, A-10
</SECID>	

13.8.1.1 International Note

Non-US financial institutions that do not have access to CUSIP numbers must supply a unique identifier for each security in the UNIQUEID field of this aggregate. OFX will be expanded to include other security identifying standards.

13.8.2 Security List Request

The user can use the SECLISTTRNRQ and SECLISTRQ aggregates to request information about specific securities. The SECLISTTRNRQ is the transaction-level aggregate that contains the SECLISTRQ. The SECLISTRQ aggregate specifies for which securities information is being requested.

13.8.2.1 Security List Transaction Request <SECLISTTRNRQ>

Tag	Description
<SECLISTTRNRQ>	Transaction-request aggregate
<TRUID>	Client-assigned globally unique ID for this transaction <i>trnuid</i>
<CLTCOOKIE>	Data to be echoed in the transaction response, A-32
<TAN>	Transaction authorization number; used in some countries with some types of transactions. Country-specific documentation will define messages that require a <TAN>, A-80
<SECLISTRQ>	Aggregate for the security list request (see section 13.8.2.2)
</SECLISTRQ>	
</SECLISTTRNRQ>	

13.8.2.2 Security List Request <SECLISTRQ>

For the security list request, securities must be specified with either a SECID aggregate, a ticker symbol, or an FI assigned identifier.

Tag	Description
<SECLISTRQ>	Security-list-request aggregate
<SECRQ>	Security request (one or more)
Security identification. Specify either <SECID>, <TICKER>, or <FIID>.	
<SECID>	Security identifier aggregate
</SECID>	
-or-	
<TICKER>	Ticker symbol, A-32
-or-	
<FIID>	FI specific ID for the security, A-32
</SECRQ>	
</SECLISTRQ>	

13.8.3 Security List Response

If the client sends a security list request to an FI, then the server must send back a security list response to the client application. The security list response is used primarily to report the status of the security list request. The actual security information should be sent in the security list SECLIST aggregate described in section [13.8.4](#).

13.8.3.1 Security List Transaction Response <SECLISTTRNRS>

Tag	Description
<SECLISTTRNRS>	Transaction-response aggregate
<TRNUID>	Client-assigned globally unique ID for this transaction <i>trnuid</i>
<STATUS>	Status aggregate
</STATUS>	
<CLTCOOKIE>	Client-provided data, REQUIRED if provided in request, A-32
<SECLISTR>	Aggregate for the security list response, see 13.8.3
</SECLISTR>	
</SECLISTTRNRS>	

13.8.3.2 Status Codes

Code	Meaning
0	Success (INFO)
2000	General error (ERROR)
2019	Duplicate request (ERROR)
12500	One or more securities not found (ERROR)

13.8.3.3 Security List Response <SECLISTR>

The security list response aggregate, the only empty aggregate in OFX, is used to respond to the <SECLISTRQ>. It is used to signify that the security list is generated as a result of a security list request. The actual security information should be included in the <SECLIST> aggregate.

Tag	Description
<SECLISTR>	Security-list-response (the only empty aggregate in OFX).
</SECLISTR>	

13.8.4 Security List <SECLIST>

The SECLIST should be sent in the following two cases.

- ◆ In response to a <SECLISTTRNRQ> sent by a client application where the <SECLIST> should contain information for each security specified in the <SECLISTTRNRQ>
- ◆ When the response file contains an investment statement download that has positions, transactions, or open orders. The <SECLIST> should contain information about security referenced in the investment statement download. Clients are completely dependent on the security list to provide descriptive information for the securities referenced in positions, transactions, and open orders.

Note: The <SECLISTMSGRSV1> and <SECLISTMSGRSV2> aggregates may appear in a response file when no corresponding <SECLISTMSGSRQV1> and <SECLISTMSGSRQV2> aggregates appeared in the request file. Servers should add the message set response wrapper only when downloading a statement and providing the <SECLIST>.

Tag	Description
<SECLIST>	Security-list-request aggregate
<xxxINFO>	Security information aggregates (zero or more): <DEBTINFO>, <MFINFO>, <OPTINFO>, <OTHERINFO>, or <STOCKINFO>.
</xxxINFO>	While allowed by the DTD, servers should not send an empty <SECLIST> aggregate.
</SECLIST>	

13.8.5 Securities Information

The <MFINFO>, <STOCKINFO>, <OPTINFO>, <DEBTINFO>, and <OTHERINFO> aggregates provide security information. They define the type of security, and one or more sets of descriptive information. These aggregates relate the <SECID> used in positions, transactions, and open orders to descriptive information about those securities. In this way, the system describes a given security only once, no matter how many times it is referenced.

13.8.5.1 General Securities Information <SECINFO>

The <SECINFO> aggregate contains fields that are common to all security types. This aggregate is used in the security type specific aggregates in the following sections.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<SECINFO>		Security-information aggregate
<SECID>		Security-identifier aggregate
</SECID>		
<SECNAME>		Full name of security, <i>A-120</i>
<TICKER>		Ticker symbol (at most one), <i>A-32</i>
<FIID>		FI ID number for this security (at most one), <i>A-32</i>
<RATING>		Rating, <i>A-10</i>
<UNITPRICE>		Current price of security, <i>unitprice</i>
<DTASOF>		Date as of for the unit price, <i>datetime</i>
<CURRENCY>		Overriding currency aggregate for unit price, see section 5.2
</CURRENCY>		
<MEMO>	V1 only	<i>Memo</i>
<MEMO2>	V2 only	<i>Memo2</i>
</SECINFO>		

13.8.5.2 Debt Information <DEBTINFO>

<i>Tag</i>	<i>Description</i>
<DEBTINFO>	Opening tag for debt information aggregate
<SECINFO>	Security information aggregate
</SECINFO>	
<PARVALUE>	Par value, <i>amount</i>
<DEBTTYPE>	Debt type (at most one) COUPON = coupon ZERO = zero coupon
<DEBTCLASS>	Classification of debt. TREASURY, MUNICIPAL, CORPORATE, OTHER.
<COUPONRT>	Bond coupon rate for next closest call date (at most one), <i>rate</i>
<DTCOUPON>	Maturity date for next coupon, <i>date</i>
<COUPONFREQ>	When coupons mature. One of the following values: MONTHLY, QUARTERLY, SEMIANNUAL, ANNUAL, or OTHER.
<CALLPRICE>	Bond call price (at most one), <i>unitprice</i>
<YIELDTOCALL>	Yield to next call, <i>rate</i>
<DTCALL>	Next call date (at most one), <i>date</i>
<CALLTYPE>	Type of next call. CALL, PUT, PREFUND, MATURITY
<YIELDTOMAT>	Yield to maturity, <i>rate</i>
<DTMAT>	Debt maturity date (at most one), <i>date</i>
<ASSETCLASS>	Asset Class (at most one), DOMESTICBOND, INTLBOND, LARGESTOCK, SMALLSTOCK, INTLSTOCK, MONEYMKT, OTHER
<FIASSETCLASS>	Text string containing an FI defined asset class, A-32
</DEBTINFO>	

13.8.5.3 Mutual Fund Information <MFINFO>

Tag	Description
<MFINFO>	Mutual-fund-information aggregate
<SECINFO>	Security-information aggregate
</SECINFO>	
<MFTYPE>	Mutual fund type. OPENEND, CLOSEEND, OTHER
<YIELD>	Current yield reported as portion of the fund's assets (at most one), <i>rate</i>
<DTYIELDASOF>	As-of date for yield value, <i>datetime</i>
<MFASSETCLASS>	Asset class breakdown for the mutual fund
<PORTION>	Portion of the mutual fund with a specific asset classification (one or more)
<ASSETCLASS>	Asset Class, DOMESTICBOND, INTLBOND, LARGESTOCK, SMALLSTOCK, INTLSTOCK, MONEYMKT, OTHER
<PERCENT>	Percentage of the fund that falls under this asset class, <i>rate</i>
</PORTION>	
</MFASSETCLASS>	
<FIMFASSETCLASS>	FI defined asset class breakdown for the mutual fund
<FIPORTION>	Portion of the mutual fund with a specific asset classification (one or more)
<FIASSETCLASS>	Text string containing an FI defined asset class, A-32
<PERCENT>	Percentage of the fund that falls under this asset class, <i>rate</i>
</FIPORTION>	
</FIMFASSETCLASS>	
</MFINFO>	

13.8.5.4 Option Information <OPTINFO>

<i>Tag</i>	<i>Description</i>
<OPTINFO>	Option-information aggregate
<SECINFO>	Security-information aggregate
</SECINFO>	
<OPTTYPE>	Option type: PUT = put CALL = call
<STRIKEPRICE>	Strike price <i>unitprice</i>
<DTEXPIRE>	Expiration date, <i>date</i>
<SHPERCTRCT>	Shares per contract, <i>N-5</i>
<SECID>	Security ID of the underlying security
</SECID>	
<ASSETCLASS>	Asset Class (at most one), DOMESTICBOND, INTLBOND, LARGESTOCK SMALLSTOCK, INTLSTOCK, MONEYMKT, OTHER
<FIASSETCLASS>	Text string containing an FI defined asset class, A-32
</OPTINFO>	

13.8.5.5 Other Security Type Information <OTHERINFO>

Use this aggregate for security types other than debts, mutual funds, options, and stocks.

<i>Tag</i>	<i>Description</i>
<OTHERINFO>	Other aggregate.
<SECINFO>	Security information aggregate
</SECINFO>	
<TYPEDESC>	Description of security type, A-32
<ASSETCLASS>	Asset Class (at most one), DOMESTICBOND, INTLBOND, LARGESTOCK SMALLSTOCK, INTLSTOCK, MONEYMKT, OTHER
<FIASSETCLASS>	Text string containing an FI defined asset class, A-32
</OTHERINFO>	

13.8.5.6 Stock Information <STOCKINFO>

<i>Tag</i>	<i>Description</i>
<STOCKINFO>	Stock-information aggregate
<SECINFO>	Security-information aggregate
</SECINFO>	
<STOCKTYPE>	Stock type: COMMON, PREFERRED, CONVERTIBLE, OTHER
<YIELD>	Current yield reported as the dividend expressed as a portion of the current stock price (at most one), <i>rate</i>
<DTYIELDASOF>	As-of date for yield value, <i>datetime</i>
<ASSETCLASS>	Asset Class (at most one): DOMESTICBOND, INTLBOND, LARGESTOCK, SMALLSTOCK, INTLSTOCK, MONEYMRKT, OTHER
<FIASSETCLASS>	Text string containing an FI defined asset class, A-32
</STOCKINFO>	

13.8.5.7 Asset Class Descriptions

<i>Asset Class</i>	<i>Description</i>
DOMESTICBOND	The Domestic Bonds asset class consists of government or corporate bonds issued in the United States.
INTLBOND	The International Bonds asset class consists of government or corporate bonds issued in foreign countries or the United States.
LARGESTOCK	The Large Cap Stocks asset class consists of stocks for U.S. companies with market capitalizations of \$2 billion or more.
SMALLSTOCK	The Small Cap Stocks asset class consists of stocks for U.S. companies with market capitalizations of approximately \$100 million to \$2 billion.
INTLSTOCK	The International Stocks asset class consists of publicly-traded stocks for companies based in foreign countries.
MONEYMRKT	The Money Market asset class consists of stable, short-term investments which provide income that rises and falls with short-term interest rates.
OTHER	The Other asset class consists of investments which do not fit in any of the other asset classes.

13.9 Investment Statement Download

Investment statement download allows a customer to receive transactions, positions, open orders, and balances that are typically part of a regular paper statement.

Clients usually allow customers to view investment transactions and guide customers through a process of updating their account registers based on the downloaded transactions. By using <FITID> values supplied by FIs, OFX makes it possible for clients to insure that each transaction is downloaded only once. The request also contains starting and ending dates to limit the amount of downloaded data. Clients can remember the last date they receive a download, and use that date as the starting date in the next request.

Investment statement download requires the client to designate an account for the download, and to indicate what type of data should be downloaded. If the client wishes to download transactions, it can specify a date range that the transactions fall within. The server returns transactions that match the date range, if one is specified. If a date range is not specified, the server returns all available transactions for the account.

13.9.1 Investment Statement Request

Investment statement download can be requested using the INVSTMTRNRQ and INVSTMTRQ aggregates. The INVSTMTRNRQ is the transaction level aggregate that contains the INVSTMTRQ. The INVSTMTRQ aggregate specifies what types of information to include in the statement download and from which account to download the information.

13.9.1.1 Investment Statement Transaction Request <INVSTMTRNRQ>

Tag	Description
<INVSTMTRNRQ>	Transaction-request aggregate
<TRNUID>	Client-assigned globally unique ID for this transaction, <i>trnuid</i>
<CLTCOOKIE>	Data to be echoed in the transaction response, A-32
<TAN>	Transaction authorization number; used in some countries with some types of transactions. Country-specific documentation will define messages that require a <TAN>, A-80
<INVSTMTRQ>	Aggregate for the investment statement download request (see section 13.9.1.2)
</INVSTMTRQ>	
</INVSTMTRNRQ>	

13.9.1.2 Investment Statement Request <INVSTMTRQ>

The following table shows the Investment Statement Request record. It is similar to a bank statement request, except that there are extra elements to indicate which pieces the user desires. Note that because transaction and position requests require date information, they use aggregates, whereas the other requests are elemental of type Boolean.

Clients and servers should interpret <DTSTART> and <DTEND> as described in [Chapter 3, "Common Aggregates, Elements, and Data Types."](#)

If <DTASOF> is not included with the <INCPOS> aggregate, the server should return the most current position information available.

Tag	Description and Type
<INVSTMTRQ>	Investment-request aggregate
<INVACCTFROM>	Account-from aggregate
</INVACCTFROM>	
<INCTRAN>	Include-transactions aggregate (at most one)
<DTSTART>	Start date of request, <i>datetime</i>
<DTEND>	Ending date of request (at most one), <i>datetime</i>
<INCLUDE>	Whether to include transactions in the statement download, <i>Boolean</i>
</INCTRAN>	
<INCOO>	Include investment open orders in response, <i>Boolean</i>
<INCPOS>	Include investment positions in response
<DTASOF>	Date that positions should be sent down for, <i>datetime</i>
<INCLUDE>	Whether to include positions in the statement download, <i>Boolean</i>
</INCPOS>	
<INCBAL>	Include investment balance in response, <i>Boolean</i>
</INVSTMTRQ>	

13.9.2 Investment Statement Response

13.9.2.1 Investment Statement Transaction Response <INVSTMTRNRS>

Tag	Description
<INVSTMTRNRS>	Transaction-response aggregate
<TRNUID>	Client-assigned globally unique ID for this transaction, <i>trnuid</i>
<STATUS>	Status aggregate
</STATUS>	
<CLTCOOKIE>	Client-provided data, REQUIRED if provided in request, A-32
<INVSTMTRS>	Aggregate for the investment statement download response (see section 13.9.2.2)
</INVSTMTRS>	
</INVSTMTRNRS>	

13.9.2.2 Investment Statement Response <INVSTMTRS>

The response can contain transaction, position, open order, and /or balance detail records; each in its own aggregate. The transaction list aggregate can contain a mixture of bank statement records and investment transactions, as specified below.

Tag	Description
<INVSTMTRS>	Investment-response aggregate
<DTASOF>	As of date & time for the statement download, <i>datetime</i>
<CURDEF>	Default currency for the statement, <i>currsymbol</i>
<INVACCTFROM>	Which account at FI
</INVACCTFROM>	
<INVTRANLIST>	Begin transaction list (at most one)
<DTSTART>	Start date for transaction data, <i>datetime</i>
<DTEND>	This is the value that should be sent in the next <DTSTART> request to insure that no transactions are missed, <i>datetime</i>
(investment transaction aggregates)	Investment statement transaction aggregates (zero or more); see section 13.9.2.4.4 .
<INVBANKTRAN>	Banking-related transactions for the investment account (zero or more)
</INVBANKTRAN>	(See section 13.9.2.3)
</INVTRANLIST>	End of investment transaction list
<INVPOSLIST>	Beginning of investment position list
	Though the DTD allows an empty <INVPOSLIST> in the response, servers should instead leave out the optional list aggregate.
<POSxxxxx>	Security type specific position aggregates (zero or more): POSMF, POSSTOCK, POSDEBT, POSOPT, POSOTHER
</POSxxxxx>	
</INVPOSLIST>	End of investment position list
<INVBAL>	Balances aggregate, see section 13.9.2.7
</INVBAL>	
<INVOOLIST>	Beginning of investment open order list
	Though the DTD allows an empty <INVOOLIST> in the response, servers should instead leave out the optional list aggregate.
<OOxxxxx>	Action and security type specific open order aggregates (zero or more): see section 13.9.2.5.2
</OOxxxxx>	

<i>Tag</i>	<i>Description</i>
</INVOOLIST>	End of investment open order list
<MKTGINFO>	Marketing information (at most one), A-360.
</INVSTMTRS>	

The various sections of the investment statement download are returned only if requested.

13.9.2.2.1 Note on Margin Calls

For investment statement download, margin call information should be included in the balances section. Margin call information should be contained in a <BAL> aggregate and included in the balance list <BALLIST>.

13.9.2.3 Bank Transactions <INVBANKTRAN>

Use the INVBANKTRAN aggregate to download bank transactions in an investment statement download.

<i>Tag</i>	<i>Description</i>
<INVBANKTRAN>	Banking related transactions for the investment account
<STMTRN>	Bank (cash) transaction aggregates
</STMTRN>	(See Chapter 11, "Banking")
<SUBACCTFUND>	The sub-account associated with the funds for the transaction; see section 13.9.2.4.2
</INVBANKTRAN>	

13.9.2.4 Investment Transactions

Note that the following types of investment actions found on statements should **not** be sent in OFX:

- ◆ Transaction-specific miscellaneous fees—fees that affect the basis of the transaction should be incorporated into the <COMMISSION>, <FEES>, <LOAD>, or <TAXES> amounts.
- ◆ Settlement actions.
- ◆ Sweeps, unless handled as any other investment position.

For transactions that involve securities, the client can create transactions based on the formula
total = (units * (unitprice +/- markup/markdown)) +/- (commission + fees + load + taxes)
(after adjusting quantity and unitprice to standard units based on the type of security.)

Thus, it is important the FIs incorporate all other transactional fees into the commission field. Clients can account for bond accrued interest and withholding using separate client transactions.

13.9.2.4.1 General Transaction Aggregate <INVTRAN>

The INVTRAN aggregate contains fields common to many of the investment transactions. It is referenced within the transaction aggregates in the following sections.

Each <INVTRAN> contains an <FITID> that the client uses to detect whether the server previously downloaded the transaction.

Tag	Version	Description
<INVTRAN>		Investment-transaction-response aggregate
<FITID>		Unique FI-assigned transaction ID. This ID is used to detect duplicate downloads. <i>FITID</i>
<SRVRTID>	V1 only	Server assigned transaction ID, <i>SRVRTID</i>
<SRVRTID2>	V2 only	Server assigned transaction ID, <i>SRVRTID2</i>
<DTTRADE>		Trade date; for stock splits, day of record, <i>datetime</i>
<DTSETTLE>		Settlement date; for stock splits, execution date, <i>datetime</i>
<MEMO>	V1 only	Other information about transaction (at most one), <i>memo</i>
<MEMO2>	V2 only	Other information about transaction (at most one), <i>memo2</i>
</INVTRAN>		

13.9.2.4.2 Transaction Aggregate Elements

The following elements are referenced within of the following investment transaction aggregates.

Tag	Version	Description
<ACCRDINT>	V2 change	For debt purchases, accrued interest, <i>amount</i>
<AVGCOSTBASIS>		Average cost basis, <i>amount</i>
<BUYTYPE>		Type of purchase: BUY, BUYTOCOVER
<COMMISSION>		Transaction commission. <i>amount</i>
<DENOMINATOR>		For stock splits, split ratio denominator, <i>quantity</i>
<DTPURCHASE>		The security's original purchase date, <i>date</i>
<GAIN>		For sales, total gain, <i>amount</i>
<FEES>		Fees applied to trade, <i>amount</i>
<FRACCASH>		Cash for fractional units., (used for stock splits), <i>amount</i>
<INCOMETYPE>		Type of investment income: CGLONG (capital gains-long term), CGSHORT (capital gains-short term), DIV (dividend), INTEREST, MISC
<LOAD>		Load on the transaction, <i>amount</i>
<MARKDOWN>		Portion of the unit price that is attributed to the dealer markdown, <i>unitprice</i>
<MARKUP>		Portion of the unit price that is attributed to the dealer markup, <i>unitprice</i>
<NEWUNITS>		For stock splits, number of shares after the split, <i>quantity</i>
<NUMERATOR>	V2 change	For stock splits, split ratio numerator, <i>quantity</i>
<OLDUNITS>		For stock splits, number of shares before the split, <i>quantity</i>
<OPTACTION>		For options, action type: EXERCISE, ASSIGN, EXPIRE
<OPTBUYTYPE>		For options, type of purchase: BUYTOOPEN, BUYTOCLOSE
<OPTSELLTYPE>		For options, type of sell: SELLTOCLOSE, SELLTOOPEN
<POSTYPE>		Position type. LONG, SHORT
<RELFITID>	V2 only	ID of related trade, <i>FITID</i>
<RELTYPE>		Related option transaction type: SPREAD, STRADDLE, NONE, OTHER
<REVERSALFEES>	V2 only	For a reversal transaction, the total amount of fees charged to perform the reversal. See section 13.9.2.4.10 . <i>Amount</i> .

Tag	Version	Description
<REVERSALFITID>	V2 only	For a reversal transaction, the FITID of the transaction that is being reversed. For example, the FITID of the original Buy order. See section 13.9.2.4.10. FITID .
<SECURED>		How an option is secured: NAKED, COVERED
<SELLREASON>		Reason the sell of a debt security was generated: CALL (the debt was called), SELL (the debt was sold), MATURITY (the debt reached maturity)
<SELLTYPE>		Type of sell. SELL, SELLSHORT
<SHPERCTRCT>		For options, number of shares per contract, N-5
<SUBACCTFROM>		Sub-account that security or cash is being transferred from: CASH, MARGIN, SHORT, OTHER
<SUBACCTFUND>		Where did the money for the transaction come from or go to? CASH, MARGIN, SHORT, OTHER
<SUBACCTSEC>		Sub-account type for the security: CASH, MARGIN, SHORT, OTHER
<SUBACCTTO>		Sub-account that security or cash is being transferred to: CASH, MARGIN, SHORT, OTHER
<TOTAL>		Transaction total. Buys, sells, etc.:((quan. * (price +/- markup / markdown)) +/- (commission + fees + load + taxes)). Distributions, interest, margin interest, misc. expense, etc.: amount. Return of cap: cost basis; <i>amount</i>
<TAXES>		Taxes on the trade, <i>amount</i>
<TAXEXEMPT>		Tax-exempt transaction, <i>Boolean</i>
<TFERACTION>		Action for transfers: IN, OUT
<UNITPRICE>		Price per commonly-quoted unit. Does not include markup / markdown, <i>unitprice</i> . Share price for stocks, mutual funds, and others Percentage of par for bonds Per share (not contract) for options
<UNITS>		For security-based actions other than stock splits, <i>quantity</i> Shares for stocks, mutual funds, and others. Face value for bonds. Contracts for options.
<UNITTYPE>		Type of the units value: SHARES, CURRENCY
<WITHHOLDING>		Tax withholdings, <i>amount</i>

13.9.2.4.3 Investment Buy/Sell Aggregates <INVBUY>/<INVSELL>

These aggregates are referenced within investment transaction aggregates.

<i>Aggregate Name</i>	<i>Version</i>	<i>Elements</i>	<i>Description</i>
INVBUY		<INVTRAN> aggregate <SECID> aggregate <UNITS> <UNITPRICE> <MARKUP> <COMMISSION> <TAXES> <FEES> <LOAD> <TOTAL>	
	V2 only	<REVERSALFEES>	
	V2 only	<REVERSALFITID> <CURRENCY> aggregate <ORIGCURRENCY> aggregate <SUBACCTSEC> <SUBACCTFUND>	Though the DTD allows <CURRENCY> and <ORIGCURRENCY> together in this aggregate, servers should return neither or one of the two, but not both.

<i>Aggregate Name</i>	<i>Version</i>	<i>Elements</i>	<i>Description</i>
INVSELL		<INVTRAN> aggregate <SECID> aggregate <UNITS> <UNITPRICE> <MARKDOWN> <COMMISSION> <TAXES> <FEES> <LOAD> <WITHHOLDING> <TAXEXEMPT> <TOTAL> V2 only <REVERSALFEES> V2 only <REVERSALFITID> <GAIN> <CURRENCY> aggregate <ORIGCURRENCY> aggregate <SUBACCTSEC> <SUBACCTFUND>	Though the DTD allows <CURRENCY> and <ORIGCURRENCY> together in this aggregate, servers should return neither or one of the two, but not both.

13.9.2.4.4 Investment Transaction Aggregates

<i>Aggregate Name</i>	<i>Elements</i>	<i>Description</i>
<BUYDEBT>	<INVBUY> aggregate <ACCRDINT>	Buy debt security Accrued interest. This amount is not reflected in the <TOTAL> field of a containing aggregate.
<BUYMF>	<INVBUY> aggregate <BUYTYPE> <RELFITID>	Buy mutual fund The BUYTOCOVER buy type used to close short sales. RELFITID used to relate transactions associated with mutual fund exchanges.
<BUYOPT>	<INVBUY> aggregate <OPTBUYTYPE> <SHPERCTRCT>	Buy option The BUYTOOPEN buy type is like “ordinary” buying of option and works like stocks.
<BUYOTHER>	<INVBUY> aggregate <BUYTYPE>	Buy other security type V2 only
<BUYSTOCK>	<INVBUY> aggregate <BUYTYPE>	Buy stock The BUYTOCOVER buy type used to close short sales.
<CLOSUREOPT>	<INVTRAN> aggregate <SECID> aggregate <OPTACTION> <UNITS> <SHPERCTRCT> <SUBACCTSEC> <RELFITID> <GAIN>	Close a position for an option. The EXERCISE action is used to close out an option that is exercised. The ASSIGN action is used when an option writer is assigned. The EXPIRE action is used when the option’s expired date is reached. When the action is EXERCISE or ASSIGN another transaction must be generated by the server to represent the buy or sell of the underlying security. RELFITID refers to the transaction ID of the underlying buy or sell.

Aggregate Name	Elements	Description
<INCOME>	<INVTRAN> aggregate <SECID> aggregate <INCOMETYPE> <TOTAL> <SUBACCTSEC> <SUBACCTFUND> <TAXEXEMPT> <WITHHOLDING> <CURRENCY> aggregate <ORIGCURRENCY> aggregate	Investment income is realized as cash into the investment account. A negative TOTAL is used to denote adjustments to income. Though the DTD allows <CURRENCY> and <ORIGCURRENCY> together in this aggregate, servers should return neither or one of the two, but not both.
<INVEXPENSE>	<INVTRAN> aggregate <SECID> aggregate <TOTAL> <SUBACCTSEC> <SUBACCTFUND> <CURRENCY> aggregate <ORIGCURRENCY> aggregate	Misc. investment expense that is associated with a specific security. If the expense is associated with the account then an INVBANKTRAN - DEBIT should be used. Though the DTD allows <CURRENCY> and <ORIGCURRENCY> together in this aggregate, servers should return neither or one of the two, but not both.
<JRNLFUND>	<INVTRAN> aggregate <SUBACCTTO> <SUBACCTFROM> <TOTAL>	Journaling cash holdings between sub-accounts within the same investment account.
<JRNLSEC>	<INVTRAN> aggregate <SECID> aggregate <SUBACCTTO> <SUBACCTFROM> <UNITS>	Journaling security holdings between sub-accounts within the same investment account.
<MARGININTEREST>	<INVTRAN> aggregate <TOTAL> <SUBACCTFUND> <CURRENCY> aggregate <ORIGCURRENCY> aggregate	Margin interest expense Though the DTD allows <CURRENCY> and <ORIGCURRENCY> together in this aggregate, servers should return neither or one of the two, but not both.

Aggregate Name	Elements	Description
<REINVEST>	<INVTRAN> aggregate <SECID> aggregate <INCOMETYPE> <TOTAL> <SUBACCTSEC> <UNITS> <UNITPRICE> <COMMISSION> <TAXES> <FEES> <LOAD> <TAXEXEMPT> <CURRENCY> aggregate <ORIGCURRENCY> aggregate	Reinvestment of income REINVEST is a single transaction that contains both income and an investment transaction. If servers can't track this as a single transaction they should return an INCOME transaction and an INVTRAN. TOTAL and UNITS are signed as for an investment buy. Corrections to a REINVEST are signed as for an investment sell. Though the DTD allows <CURRENCY> and <ORIGCURRENCY> together in this aggregate, servers should return neither or one of the two, but not both.
<RETOFCAP>	<INVTRAN> <SECID> <TOTAL> <SUBACCTSEC> <SUBACCTFUND> <CURRENCY> aggregate <ORIGCURRENCY> aggregate	Return of capital Though the DTD allows <CURRENCY> and <ORIGCURRENCY> together in this aggregate, servers should return neither or one of the two, but not both.
<SELLDEBT>	<INVSELL> aggregate <SELLREASON> <ACCRDINT>	Sell debt security. Used when debt is sold, called, or reached maturity.
<SELLMF>	<INVSELL> aggregate <SELLTYPE> <AVGCOSTBASIS> <RELFITID>	Sell mutual fund RELFITID used to relate transactions associated with mutual fund exchanges.
<SELLOPT>	<INVSELL> aggregate <OPTSELLTYPE> <SHPERCTRCT> <RELFITID> <RELTYPE> <SECURED>	Sell option The SELLTOCLOSE action is selling a previously bought option. The SELLTOOPEN action is writing an option

<i>Aggregate Name</i>	<i>Elements</i>	<i>Description</i>
<SELLOTHER>	<INVSELL> aggregate <SELLTYPE>	Sell other type of security V2 only
<SELLSTOCK>	<INVSELL> aggregate <SELLTYPE>	Sell stock
<SPLIT>	<INVTRAN> aggregate <SECID> aggregate <SUBACCTSEC> <OLDUNITS> <NEWUNITS> <NUMERATOR> <DENOMINATOR> <CURRENCY> aggregate <ORIGCURRENCY> aggregate <FRACCASH> <SUBACCTFUND>	Stock or Mutual Fund Split Note: the trade date is interpreted as the “day of record” for the split. Though the DTD allows <CURRENCY> and <ORIGCURRENCY> together in this aggregate, servers should return neither or one of the two, but not both.
<TRANSFER>	<INVTRAN> aggregate <SECID> aggregate <SUBACCTSEC> <UNITS> <TFERACTION> <POSTYPE> <INVACCTFROM> aggregate <AVGCOSTBASIS> <UNITPRICE> <DTPURCHASE>	Transfer holdings in and out of the investment account.

13.9.2.4.5 Valid Transactions by Security Type

	<i>Debt</i>	<i>Mutual Fund</i>	<i>Option</i>	<i>Other</i>	<i>Stock</i>
BUYDEBT	×				
BUYMF		×			
BUYOPT			×		
BUYOTHER				×	
BUYSTOCK					×
CLOSUREOPT			×		
INCOME	×	×		×	×
INVEXPENSE	×	×	×	×	×
JRNLFUND					
JRNLSEC	×	×	×	×	×
MARGININTEREST					
REINVEST	×	×		×	×
RETOFCAP	×	×	×	×	×
SELLDEBT	×				
SELLMF		×			
SELLOPT			×		
SELLOTHER				×	
SELLSTOCK					×
SPLIT		×			×
TRANSFER	×	×	×	×	×

Since JRNLFUND and MARGININTEREST do not refer to securities, there are no checks in any of the security columns for these transactions.

13.9.2.4.6 Notes on Mutual Fund Exchanges

In investment statement download, two transactions are needed to reflect mutual fund exchanges. A SELLMF should be generated for the mutual fund being switched from and a BUYMF should be generated for the mutual fund being switched to. You can use the RELFITID element to link these two transactions to each other. You should use the MEMO element of the individual transactions to explain that a mutual fund exchange occurred.

13.9.2.4.7 Notes on Corporate Actions

Since corporate actions can often be very complicated, it is difficult to define a single action aggregate that encompasses all possible scenarios. Instead, you should describe corporate actions using one or more of the provided basic action types. You should use the memo field of the individual transactions to link transactions to an encompassing corporate action.

13.9.2.4.8 Notes on Option Splits

When the underlying security for an option splits, a new security is generated for the option since the strike price changes. In investment statement download, you need two transactions to reflect this activity. There should be a TRANSFER transaction to show that the old option security is removed from the account and another TRANSFER transaction to show that the new option security is moved into the account.

13.9.2.4.9 Notes on Option actions

For options, the overall sequence of actions is as follows:

For an option writer:

Position is opened with Sell to Open.

Position is closed with one of the following:

- ◆ Buy to Close
- ◆ Expire
- ◆ Assigned

For an option buyer:

Position is opened with Buy to Open.

Position is closed with one of the following:

- ◆ Sell to Close
- ◆ Expire
- ◆ Exercise

13.9.2.4.10 Notes on Reversals

The V2 message set introduces an architecture that enables transaction reversals.

To reverse a Buy transaction, an <INVBUY> aggregate is used, with the <REVERSALFITID> containing the <FITID> of the transaction that is being reversed. The number of shares being reversed is contained in <UNITS>; the amount of the fees being refunded is in <FEES>; etc. If a

fee is charged to perform the reversal, it appears in <REVERSALFEES>. The same approach is used with reversing a Sell transaction, except that an <INVSELL> aggregate is used instead.

13.9.2.5 Open Orders

13.9.2.5.1 General Open Order Aggregate <OO>

The <OO> aggregate contains fields common to all open orders. Use this aggregate to define the open order aggregates as show in the following section.

Tag	Version	Description
<OO>		General-open-order aggregate
<FITID>		Unique FI-assigned transaction ID, <i>FITID</i>
<SRVRTID>	V1 only	Unique server-assigned transaction ID, <i>SRVRTID</i>
<SRVRTID2>	V2 only	Unique server-assigned transaction ID, <i>SRVRTID2</i>
<SECID>		Security identified aggregate
</SECID>		
<DTPLACED>		Date-time the order was placed, <i>datetime</i>
<UNITS>		Quantity of the security the open order is for, <i>unitprice</i>
<SUBACCT>		Sub-account type. CASH, MARGIN, SHORT, OTHER
<DURATION>		How long the order is good for: DAY, GOODTILCANCEL, IMMEDIATE
<RESTRICTION>		Special restriction on the order: ALLORNONE, MINUNITS, NONE
<MINUNITS>		Minimum number of units that must be filled for the order, <i>quantity</i>
<LIMITPRICE>		Limit price, <i>unitprice</i>
<STOPPRICE>		Stop price, <i>unitprice</i>
<MEMO>	V1 only	Other information about order (at most one), <i>memo</i>
<MEMO2>	V2 only	Other information about order (at most one), <i>memo2</i>
<CURRENCY>		Overriding currency aggregate
</CURRENCY>		
</OO>		

Note: An open order is assumed to be a market order if no limit price or stop price is specified.

13.9.2.5.2 Investment Open Order Aggregates

<i>Open Order Aggregates</i>	<i>Elements</i>	<i>Description of Elements</i>
<OOBUYDEBT>	<OO> aggregate <AUCTION> <DTAUCTION>	Whether the debt should be purchased at the auction, <i>Boolean</i> Date of the auction, <i>date</i>
<OOBUYMF>	<OO> aggregate <BUYTYPE> <UNITTYPE>	Type of purchase: BUY, BUYTOCOVER. What the units represent: SHARES, CURRENCY
<OOBUYOPT>	<OO> aggregate <OPTBUYTYPE>	Type of purchase: BUYTOOPEN, BUYTOCLOSE
<OOBUYOTHER>	<OO> aggregate <UNITTYPE>	What the units represent: SHARES, CURRENCY
<OOBUYSTOCK>	<OO> aggregate <BUYTYPE>	Type of purchase: BUY, BUYTOCOVER
<OOSELLDEBT>	<OO> aggregate	
<OOSELLMF>	<OO> aggregate <SELLTYPE> <UNITTYPE> <SELLALL>	Type of sale: SELL, SELLSHORT What the units represent: SHARES, CURRENCY. Sell entire holding, <i>Boolean</i>
<OOSELLOPT>	<OO> aggregate <OPTSELLTYPE>	Type of sale: SELLTOOPEN, SELLTOCLOSE
<OOSELLOther>	<OO> aggregate <UNITTYPE>	What the units represent: SHARES, CURRENCY
<OOSELLSTOCK>	<OO> aggregate <SELLTYPE>	Type of sale: SELL, SELLSHORT
<SWITCHMF>	<OO> aggregate <SECID> aggregate <UNITTYPE> <SWITCHALL>	Security ID of the mutual fund to switch to or purchase What the units represent: SHARES, CURRENCY Switch entire holding, <i>Boolean</i>

13.9.2.6 Investment Positions

Position records represent a user's current positions, regardless of the transactional history. Prices and values should be the most recent available, even if different from a transaction price on the same day.

In position records, securities are identified as being either short or long. Because each FI has different rules regarding which sub-accounts can be used for short compared to long activity, FIs must explicitly indicate the type of position in addition to specifying the sub-account where the position takes place.

For options, position type SHORT is equivalent to WRITING an option, and position type LONG is equivalent to HOLDING an option. For security types where there is only one type (for example, bonds), use LONG.

13.9.2.6.1 General Position Information <INVPOS>

The INVPOS aggregate contains fields relevant to all investment position types. It is included in the position aggregates as shown in the following sections.

Tag	Version	Description
<INVPOS>		General-position aggregate
<SECID>		Security identifier
</SECID>		
<HELDINACCT>		Sub-account type
		CASH, MARGIN, SHORT, OTHER
<POSTYPE>		SHORT = Writer for options, Short for all others. LONG = Holder for options, Long for all others.
<UNITS>		For stocks, MFs, other, number of shares held. Bonds = face value. Options = number of contracts quantity
<UNITPRICE>		For stocks, MFs, other, price per share. Bonds = percentage of par Option = premium per share of underlying security unitprice
<MKTVAL>		Market value of this position, <i>amount</i>
<DTPRICEASOF>		Date and time of unit price and market value. Can be 0 if unit price and market value are unknown, <i>datetime</i>
<CURRENCY>		Currency information if different from default currency.
</CURRENCY>		
<MEMO>	V1 only	Comment, <i>memo</i>
<MEMO2>	V2 only	Comment, <i>memo2</i>
</INVPOS>		

13.9.2.6.2 Investment Positions

<i>Investment Position Aggregates</i>	<i>Elements</i>	<i>Description of Elements</i>
<POSDEBT>	<INVPOS> aggregate	
<POSMF>	<INVPOS> aggregate <UNITSSSTREET> <UNITUSER> <REINVDIV> <REINVCG>	Units in the FI's street name, <i>positive quantity</i> Units in the user's name directly, <i>positive quantity</i> Reinvest dividends, <i>Boolean</i> Reinvest capital gains, <i>Boolean</i>
<POSOPT>	<INVPOS> aggregate <SECURED>	How the option is secured. NAKED, COVERED.
<POSOTHER>	<INVPOS> aggregate	
<POSSTOCK>	<INVPOS> aggregate <UNITSSSTREET> <UNITUSER> <REINVDIV>	Units in the FI's street name, <i>positive quantity</i> Units in the user's name directly, <i>positive quantity</i> Reinvest dividends, <i>Boolean</i>

13.9.2.7 Investment Balances <INVBAL>

The <INVBAL> aggregate contains five specified balances. It can also contain a <BALLIST> aggregate that contains one or more <BAL> aggregates. The <BAL> aggregate (see Chapter 3, “Common Aggregates, Elements, and Data Types”) allows an FI to send any number of balances to the user, complete with description and Help text. The intent is to capture the same type of balance information present on the first page of many FI brokerage statements. You can also use the <BAL> aggregate to send margin call information.

Tag	Description
<INVBAL>	Balances aggregate
<AVAILCASH>	Cash balance across all sub-accounts. Should include sweep funds. <i>Amount</i>
<MARGINBALANCE>	Margin balance. A positive balance indicates a positive cash balance, while a negative balance indicates the customer has borrowed funds. <i>Amount</i>
<SHORTBALANCE>	Market value of all short positions. This is a positive balance, <i>Amount</i>
<BUYPOWER>	Buying power, <i>amount</i>
<BALLIST>	Beginning of Investment balance list (at most one)
<BAL>	Balance aggregates (zero or more)
</BAL>	See Chapter 3, "Common Aggregates, Elements, and Data Types"
</BALLIST>	
</INVBAL>	

13.9.2.8 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2003	Account not found (ERROR)
2004	Account closed (ERROR)
2005	Account not authorized (ERROR)
2019	Duplicate request (ERROR)
2020	Invalid date (ERROR)
2027	Invalid date range (ERROR)
12250	Investment transaction download not supported (WARN)
12251	Investment position download not supported (WARN)
12252	Investment positions for specified date not available (WARN)
12253	Investment open order download not supported (WARN)
12254	Investment balances download not supported (WARN)

13.10 Investment E-Mail

OFX currently defines one investment e-mail message that clients can send to an FI. With this message, the user can prepare a message to the FI regarding one of their accounts. The server acknowledges receipt of the message. The FI prepares the response that the client picks up when it synchronizes with the server. E-mail is subject to synchronization, using <INVMailsyncRQ> / <INVMailsyncRS>.

<i>Client Sends</i>	<i>Server Responds</i>
Addressed message Inv. account information . . . Synchronization request	Acknowledgment Response to customer

13.10.1 Investment E-Mail Request and Response

13.10.1.1 Request <INVMAILRQ>

The client must identify to which investment account the customer query is related.

<i>Tag</i>	<i>Description</i>
<INVMAILRQ>	Investment-e-mail-request aggregate
<INVACCTFROM>	Account-from aggregate
<INVACCTFROM>	
<MAIL>	To, from, message information, see section 9.2.2
</MAIL>	
</INVMAILRQ>	

13.10.1.2 Response <INVMAILRS>

<i>Tag</i>	<i>Description</i>
<INVMAILRS>	Investment-e-mail-response aggregate
<INVACCTFROM>	Account-from aggregate
</INVACCTFROM>	
<MAIL>	To, from, message information, see section 9.2.2
</MAIL>	
</INVMAILRS>	

13.10.1.3 Status Codes

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2003	Account not found (ERROR)
2004	Account closed (ERROR)
2005	Account not authorized (ERROR)
2019	Duplicate request (ERROR)
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)
15508	Transaction not authorized (ERROR)
16500	HTML not allowed (ERROR)
16501	Unknown mail To: (ERROR)

13.10.2 Investment E-Mail Synchronization

13.10.2.1 Request <INVMailsyncRQ>

Tag	Version	Description
<INVMailsyncRQ> <i>Client synchronization option; <TOKEN>, <TOKEN2>, <TOKENONLY>, or <REFRESH></i>		Synchronization-request aggregate
<TOKEN>	V1 only	Previous value of <TOKEN> received for this type of synchronization request from server; 0 for first-time requests; <i>token</i>
<TOKEN2>	V2 only	Previous value of <TOKEN2> received for this type of synchronization request from server; 0 for first-time requests; <i>token2</i>
<TOKENONLY>		Request for just the current <TOKEN> without the history, <i>Boolean</i>
<REFRESH>		Request for refresh of current state, <i>Boolean</i>
<REJECTIFMISSING>		If Y, do not process requests if client <TOKEN> is out of date, <i>Boolean</i>
<INCIMAGES>		Y if the client accepts mail with images in the message body. N if the client does not accept mail with images in the message body. <i>Boolean</i>
<USEHTML>		Y if client wants an HTML response, N if client wants plain text, <i>Boolean</i>
<INVACCTFROM>		Investment account of interest; token must be interpreted in terms of this account
</INVACCTFROM>		
<INVMailTRNRQ>		Investment-mail transactions (0 or more)
</INVMailTRNRQ>		
</INVMailsyncRQ>		

13.10.2.2 Response <INVMailsSyncRS>

Tag	Version	Description
<INVMailsSyncRS>		Synchronization-response aggregate
<TOKEN>	V1 only	New synchronization token, <i>token</i>
<TOKEN2>	V2 only	New synchronization token, <i>token2</i>
<LOSTSYNC>		Y if the token in the synchronization request is older than the earliest entry in the server's history table. In this case, some responses have been lost. N if the token in the synchronization request is newer than or matches a token in the server's history table. <i>Boolean</i>
<SYNCERROR>	V2 only	Optional error code (See section 6.4.4.), <i>N-6</i> .
<INVACCTFROM>		Investment account of interest; token must be interpreted in terms of this account
</INVACCTFROM>		
<INVMailTRNRS>		Investment-mail transactions (0 or more)
</INVMailTRNRS>		
</INVMailsSyncRS>		

13.11 Complete Example

This example is for a user who requests an investment statement download for a single account.

The request file:

```
<OFX>                                <!--Beginning of request data-->
  <SIGNONMSGSRQV1>
    <SONRQ>                          <!-- ...Sign on request. For a
                                     complete example, see section
                                     11.14.1-->

    </SONRQ>                          <!--End of signon-->
  </SIGNONMSGSRQV1>
  <INVSTMTMSGSRQV1>
    <INVSTMTTRNRQ>                   <!--First request in file-->
      <TRUID>1001                    <!--Unique ID for this request-->
      <INVSTMTRQ>                    <!--Beginning of statement download-->
        <INVACCTFROM>               <!--Identify the account: -->
          <BROKERID>121099999       <!--FI ID-->
          <ACCTID>999988            <!--Account number-->
        </INVACCTFROM>              <!--End of account ID-->
        <INCTRAN>                   <!--Request transactions-->
          <DTSTART>19960824130105    <!--Send transactions posted after-->
                                     <!--Aug 24, 1996 1:01:05pm-->

          <INCLUDE>Y                 <!--Include transactions -->
        </INCTRAN>
        <INCOO>Y                    <!--Include open orders in response-->
        <INCPOS>                     <!--Request positions -->
          <INCLUDE>Y                 <!--Include current positions -->
        </INCPOS>
        <INCBAL>Y                   <!--Include balances in request-->
      </INVSTMTRQ>
    </INVSTMTTRNRQ>                 <!--End of first request-->
  </INVSTMTMSGSRQV1>
</OFX>                              <!--End of OFX request data-->
```

A typical server response:

This user has one investment transaction, one bank transaction, one open order, two position entries, and one balance entry. The user deposits some money and buys shares in Acme. The user has an open limit order to buy 100 shares of Hackson Unlimited at \$50/share. The holdings show the user already had 100 shares of Acme and now has 200 shares. The user also has one option contract to sell Lucky Airlines shares, bought before this download.

```
<OFX>                                <!--Beginning of request data-->
<SIGNONMSGSRSV1>
  <SONRS>                             <!-- ...Sign on response. For a
                                       complete example, see section
                                       11.14.1-->

  </SONRS>                             <!--End of signon-->
</SIGNONMSGSRSV1>
<INVTMTMSGSRSV1>
  <INVTMTTRNRS>                       <!--First request in file-->
    <TRNUID>1001                      <!--Client ID for this request-->
    <STATUS>
      <CODE>0                          <!--0 = accepted, good data follows-->
      <SEVERITY>INFO
    </STATUS>
    <INVTMTRS>                        <!--Beginning of statement download-->
      <DTASOF>19960827010000          <!--Statement as of Aug 27, 1996 1am-->
      <CURDEF>USD                     <!--Default currency is US Dollar-->
      <INVACCTFROM>                   <!--Beginning of account information-->
        <BROKERID>121099999          <!--FI ID-->
        <ACCTID>999988                <!--Account number-->
      </INVACCTFROM>                  <!--End of account information-->
      <INVTRANLIST>                   <!--Beginning of transactions-->
        <DTSTART>19960824130105      <!--Send transactions posted after-->
                                       <!--Aug 24, 1996 1:01:05pm-->
        <DTEND>19960828101000        <!--End timestamp (now) -->
        <BUYSTOCK>                   <!--Buy stock transaction-->
          <INVBUY>
            <INVTRAN>
              <FITID>23321            <!--FI transaction ID-->
              <DTTRADE>19960825      <!--Trade date Aug 25, 1996-->
              <DTSETTLE>19960828    <!--Settlement date Aug 28, 1996-->
            </INVTRAN>
          <SECID>                     <!--Security ID-->
            <UNIQUEID>123456789      <!--CUSIP for ACME -->
            <UNIQUEIDTYPE>CUSIP
```

```

    </SECID>
    <UNITS>100          <!--100 shares-->
    <UNITPRICE>50.00    <!--$50/share-->
    <COMMISSION>25.00   <!--$25 commission -->
    <TOTAL>5025.00      <!--Total amount $5025.00-->
    <SUBACCTSEC>CASH    <!--Holding resides in cash account-->
    <SUBACCTFUND>CASH   <!--Bought in cash account-->
  </INVBUY>
  <BUYTYPE>BUY         <!--Normal buy-->
</BUYSTOCK>           <!--End of buy stock transaction-->
<INVBANKTRAN>         <!--Investment acct bank transaction-->
  <STMTTRN>           <!--Beginning of a bank transaction-->
    <TRNTYPE>CREDIT    <!--Generic credit-->
    <DTPOSTED>19960825<!--Aug 25, 1996-->
    <DTUSER>19960825   <!--Aug 25, 1996-->
    <TRNAMT>1000.00    <!--$1,000.00-->
    <FITID>12345       <!--FI transaction ID 12345-->
    <NAME>Customer deposit<!--Description of transaction-->
    <MEMO>Your check #1034<!--Optional memo from FI-->
  </STMTTRN>          <!--End of bank transaction-->
  <SUBACCTFUND>CASH    <!--Credited to the cash account -->
</INVBANKTRAN>
</INVTRANLIST>        <!--End of transactions-->
<INVPOSLIST>          <!--Beginning of positions list-->
  <POSSTOCK>          <!--Beginning of position -->
    <INVPOS>
      <SECID>          <!--Security ID-->
        <UNIQUEID>123456789<!--CUSIP for Acme Development,
                               Inc.-->
        <UNIQUEIDTYPE>CUSIP
      </SECID>
      <HELDINACCT>CASH <!--Cash account-->
      <POSTYPE>LONG    <!--Long position-->
      <UNITS>200       <!--200 shares-->
      <UNITPRICE>49.50 <!--Latest price-->
      <MKTVAL>9900.00  <!--Current market value $9900.00-->
      <DTPRICEASOF>19960827010000 <!--Prices as of Aug27,1996
                                   1am-->
      <MEMO>Next dividend payable Sept 1
    </INVPOS>
  </POSSTOCK>         <!--End of position-->
  <POSOPT>            <!--Beginning of position-->

```

```

<INVPOS>
  <SECID>                <!--Security ID-->
    <UNIQUEID>000342222<!--CUSIP for the option -->
    <UNIQUEIDTYPE>CUSIP
  </SECID>
  <HELDINACCT>CASH  <!--Cash account-->
  <POSTYPE>LONG      <!--Long position-->
  <UNITS>1           <!--100 shares-->
  <UNITPRICE>5<!--Latest price-->
  <MKTVAL>500<!--Current market value $500.00-->
  <DTPRICEASOF>19960827010000 <!--Prices as of Aug27,1996
                                1am-->
  <MEMO> Option is in the money
</INVPOS>
</POSOPT>             <!--End of option position -->
</INVPOSLIST>         <!--End of position -->
<INVBAL>
  <AVAILCASH>200.00    <!--$200.00 cash balance-->
  <MARGINBALANCE>-50.00 <!--$50.00 owed on margin balance-->
  <SHORTBALANCE>0      <!--$0 short balance-->

  <BALLIST>            <!--Beginning of FI-defined balances-->
    <BAL>              <!--Beginning of a balance-->
      <NAME>Margin Interest Rate <!--Name of balance entry-->
      <DESC>Current interest rate on margin balances
      <!--Help text for this balance-->
      <BALTYPE>PERCENT <!--Format as percent-->
      <VALUE>7.85      <!--Will be formatted 7.85%-->
      <DTASOF>19960827010000 <!--Rate as of Aug 27, 1996 1am-->
    </BAL>             <!--End of balance entry-->
  </BALLIST>          <!--End of balances-->
</INVBAL>
<INV00LIST>
  <00BUYSTOCK>
    <00>
      <FITID>23321<!--FI transaction ID-->
      <SECID> <!--Security ID-->
        <UNIQUEID>666678578<!--CUSIP for Hackson Unlimited-->
        <UNIQUEIDTYPE>CUSIP
      </SECID>
      <DTPLACED>19960624031505<!--Order placed 6/24/96
                                3:15:05pm-->

```

```

        <UNITS>100          <!--100 shares-->
        <SUBACCT>CASH      <!--Purchase with cash-->
        <DURATION>GOODTILCANCEL<!--GOODTILCANCEL-->
        <RESTRICTION>NONE <!--No special restrictions-->
        <LIMITPRICE>50.00  <!--Limit price $50/share-->
    </00>
    <BUYTYPE>BUY          <!--Normal buy-->
</00BUYSTOCK>
</INV00LIST>
</INVSTMTRS>
</INVSTMTRNRS>          <!--End of first response-->
</INVSTMMSGSRSV1>
<SECLISTMSGSRSV1>
    <SECLIST>             <!--Beginning of securities list-->
        <STOCKINFO>       <!--Beginning of 1st security ID-->
            <SECINFO>
                <SECID>    <!--Security ID-->
                    <UNIQUEID>123456789 <!--CUSIP for the stock -->
                    <UNIQUEIDTYPE>CUSIP
                </SECID>
                <SECNAME>Acme Development, Inc.
                <TICKER>ACME          <!--Ticker symbol-->
                <FIID>1024           <!--FI internal security identifier-->
            </SECINFO>
            <YIELD>10                <!--$10/share/year-->
            <ASSETCLASS>SMALLSTOCK   <!--Small Capital Stock asset class-->
        </STOCKINFO>               <!--End of security ID-->
        <STOCKINFO>
            <SECINFO>
                <SECID>    <!--Security ID-->
                    <UNIQUEID>666678578 <!--CUSIP for the stock -->
                    <UNIQUEIDTYPE>CUSIP
                </SECID>
                <SECNAME> Hackson Unlimited, Inc.
                <TICKER> HACK          <!--Ticker symbol-->
                <FIID>1027           <!--FI internal security identifier-->
            </SECINFO>
            <YIELD>17                <!--$10/share/year-->
            <ASSETCLASS>SMALLSTOCK   <!--Small Capital Stock asset class-->
        </STOCKINFO>
        <OPTINFO>                  <!--End of security ID-->
        <SECINFO>

```

```

    <SECID>                                <!--Security ID-->
      <UNIQUEID>000342222 <!--CUSIP for the option -->
      <UNIQUEIDTYPE>CUSIP
    </SECID>
    <SECNAME>Lucky Airlines Jan 97 Put
    <TICKER>LUAXX                        <!--Ticker symbol-->
    <FIID>0013                          <!--FI internal security identifier-->
  </SECINFO>
  <OPTTYPE>PUT
  <STRIKEPRICE>35.00                    <!--Strike price $35/share-->
  <DTEXPIRE>19970121                  <!--Option expires Jan 21, 1997-->
  <SHPERCTRCT>100                      <!--100 shares per contract-->
  <SECID>                                <!--Security ID-->
    <UNIQUEID>000342200                <!--CUSIP for the underlying stock -->
    <UNIQUEIDTYPE>CUSIP
  </SECID>
  <ASSETCLASS>LARGESTOCK                <!--Large Capital Stock asset class-->
</OPTINFO>                             <!--End of option information-->
</SECLIST>                             <!--End of securities list-->
</SECLISTMSGSRVS1>
</OFX>                                <!--End of OFX request data-->

```


CHAPTER 14 BILL PRESENTMENT

14.1 Overview

Bill Presentment (PRES) is the electronic delivery of a bill from a biller to a customer.

Although some billers may provide Bill Presentment service themselves, many will choose to work with a bill publisher that provides Bill Presentment service on behalf of many billers. For this reason, Bill Presentment focuses on connecting customers to bill publishers.

14.1.1 Bill Presentment Model

This section summarizes the process of receiving bills electronically, starting with the steps required to find a bill publisher and set up Bill Presentment service.

To receive bills electronically, the client:

- ◆ Finds one or more billers by searching a biller directory server.
- ◆ Determines which bill publishers provide Bill Presentment service for the billers.
- ◆ Enrolls with a bill publisher for Bill Presentment service
- ◆ Signs on with the bill publisher and activates Bill Presentment service for one or more accounts with one or more billers.
- ◆ Requests electronic bills from the bill publisher.
- ◆ (Optionally) Pays bills using the OFX Bill Payment service.

14.1.2 Servers and Message Sets

During the billing process, the client typically communicates with two OFX servers:

- ◆ *Biller directory server*: An independent server that stores information about billers and bill publishers. Clients can query this server to find the bill publishers that serve the billers in which the customer is interested.
- ◆ *Bill publisher server*: The server that delivers bills to customers. A single bill publisher can provide Bill Presentment service for many billers. In some cases, a biller might act as its own bill publisher.

Although it is possible for a single server to perform both of the functions listed above, it is more likely that independent directory servers will provide clients with a single source for finding billers. To allow these functions to be routed separately by clients, Bill Presentment defines separate message sets for directory query and bill delivery.

- ◆ Biller Directory message set <PRESDIRMSGSETV1>
- ◆ Bill Delivery message set <PRESDLVMSGSETV1>

Note: Although these message sets are named “V1”, they are used in conjunction with other V2 messages. Specifically, <SIGNONMSGSETV2> should be used, as well as the “V2” versions of account signup messages. <TOKEN2> should be used in the synchronization wrappers, <MESSAGE2> should be used in the <STATUS> aggregates, and <URL2> should be used in the profile response.

For additional information about the message sets defined for Bill Presentment, see section [14.7](#).

14.2 Biller Directory

To find billers, the client sends a <FINDBILLERRQ> request to the biller directory server. The biller directory server returns a <FINDBILLERRS> response.

<FINDBILLERRQ> and <FINDBILLERRS> are part of the Biller Directory message set <PRESDIRMSGSETV1>. The message set tags are <PRESDIRMSGSRQV1> and <PRESDIRMSGSRSV1>.

14.2.1 Client Signon to the Biller Directory Server

Because the client does not enroll with the biller directory server and the directory does not contain private data, the client may perform an anonymous signon (as described in section 2.5.1) when requesting this service. Unlike the FI Profile message set (see Chapter 7, especially section 7.1.4), this message set does not currently support customer-specific directories. The response should be identical whether or not the request arrived with an anonymous <SONRQ>. Compliant servers must support both request forms. For more information about signon requests, refer to [Chapter 2, "Structure."](#)

14.2.2 Search Arguments

If the client omits all elements in the <FINDBILLERRQ>, the client is requesting a complete directory of billers. Otherwise, the client wants to filter results based on the included elements. For each biller that matches the elements in the request, the biller directory server returns the complete name and address of the biller, plus the biller ID and bill publisher name. Servers can return information using case-insensitive matching, but this is not required.

14.2.3 Identification of Bill Publishers

Bill Publishers must be uniquely and consistently identified by name. Clients need some way to relate the bill publisher name given by a directory server to their own databases of known and approved bill publishers. Since the number of bill publishers is relatively small, and the number of directory servers that must be coordinated even smaller, the official corporate name of a bill publisher will serve as an ID for that publisher.

14.2.4 Find Biller Request <FINDBILLERRQ>

The <FINDBILLERRQ> request must appear within a <FINDBILLERTRNRQ> transaction wrapper.

The Biller directory timestamp in the <FINDBILLERRQ> (<DTUPDATE>) selection criterion allows the client to request all directory entries that have been added or changed since a point in time. Clients that want to receive an updated list of billers and bill publishers can use <DTUPDATE> to avoid receiving a response if nothing has changed. <DTUPDATE> is returned by servers in responses to indicate the date and time of the newest or most recently changed entry, whether or not it was included in the response. Clients performing narrow searches cannot use <DTUPDATE> unless they save the value for each query, and send the corresponding value in future requests. Servers can return information using case-insensitive matching, but this is not required.

The name and address fields refer to the biller, except for <CONSUPOSTALCODE> which refers to the customer's address.

Tag	Description
<FINDBILLERRQ>	
<DTUPDATE>	Date and time of last change to any biller entry as reported by the server on previous query, <i>datetime</i> . If present, <FINDBILLERRS> will include only Billers whose information has changed or been added since this time.
<BILLERID>	ID of this biller at this bill publisher, A-32
<NAME>	Biller's name, A-32
<ADDR1>	Biller's address line 1, A-32
<ADDR2>	Biller's address line 2, A-32
<ADDR3>	Biller's address line 3, A-32
<CITY>	Biller's city, A-32
<STATE>	Biller's state, A-5
<POSTALCODE>	Biller's postal code, A-11
<COUNTRY>	ISO / DIS-3166 3-letter country code standard, A-3
<SIC>	Standard Industry Code, N-6
<CONSUPOSTALCODE>	Postal code of customer, to allow server to filter out billers that do not do business in the customer's area, A-11
<INCIMAGES>	Y if the client wants images (logos) returned, <i>Boolean</i>
</FINDBILLERRQ>	

Note: Future versions of OFX will require <ADDR1> if <ADDR2> is specified and <ADDR2> if <ADDR3> is specified.

14.2.5 Find Biller Response <FINDBILLERRS>

<FINDBILLERRS> must appear within a <FINDBILLERTRNRS> transaction wrapper.

The response is a list of <BILLERINFO> aggregates.

Tag	Description
<FINDBILLERRS>	
<DTUPDATE>	Date and time of last addition or modification to the entries in the directory, whether part of this response or not, <i>datetime</i>
<BILLERINFO>	Zero or more <BILLERINFO> aggregates
</BILLERINFO>	
</FINDBILLERRS>	

14.2.5.1 Biller Information <BILLERINFO>

<BILLERINFO> includes information about a single biller.

Besides basic name and address information, <BILLERINFO> includes the <BILLPUB> and <BILLERID> elements. These elements will be used with the customer's account number to identify the customer's account with the biller. For more information about the account-identification aggregates, refer to <PRESACCTFROM> and <PRESACCTTO> in section [14.3.2.2](#).

<BILLERINFO> can optionally include elements that specify the format of valid account numbers. <ACCTFORMAT> and <ACCTEDITMASK> provide information to the client. <HELPMESSAGE> provides a text message that the client can display to the customer.

To avoid the complications caused by invalid account numbers, <BILLERINFO> can also include a <VALIDATE>*URL* element that the client application can use to validate the customer's account number. See section [14.2.7](#) for more detail on this.

Tag	Version	Description
<BILLERINFO>		
<BILLPUB>		Official standard name of the bill publisher, A-32
<BILLERID>		ID of this biller at this bill publisher, A-32
<NAME>		Name of the biller, A-32
<ADDR1>		Biller's address line 1, A-32
<ADDR2>		Biller's address line 2, A-32
<ADDR3>		Biller's address line 3, A-32

Tag	Version	Description
<CITY>		Biller's city, A-32
<STATE>		Biller's state, A-5
<POSTALCODE>		Biller's postal code, A-11
<COUNTRY>		Biller's country; 3-letter country code from ISO / DIS-3166, A-3
<SIC>		Standard Industrial Classification Code, N-6
<PHONE>		Biller's phone number for customer information (if a special number exists for electronic billing information, use that number), A-32
<PAYMENTINSTRUMENTS>		Types of payment that the biller can accept electronically, see section 14.2.8.1
</PAYMENTINSTRUMENTS>		
<ACCTFORMAT>		Regular expression describing the account number format. For example, <code>^[0-9]{8,10}\$</code> means the account number must be numbers only, and the length must be 8 to 10 numbers. A-255
<ACCTEDITMASK>		An alternative string describing the account number format. See below for details. The client can use the edit mask to assist the user in entering the account number. A-255
<HELPMESSAGE>		Human-readable message that the client can display to assist the customer in entering his or her account number. For example: "Enter in the last 10 digits of your account number without any spaces or dashes." This is defined by the biller during the implementation phase. A-255
<RESTRICT>		Human-readable description of any restrictions on who may sign up with this biller. For example: "Please be sure to enter each account number separately if you have more than one account with us. Your mail bill will be turned off only after another paper billing cycle has passed. Please note that this program is initially available for account numbers beginning with X Y and Z only." This is defined by the biller during the implementation phase. A-255
<LOGO>		URL of the biller's logo. If the client requested images, the logo should be included via multipart MIME in this response. <i>URL2</i>
<VALIDATE>		URL for validation. The client application may use this to validate the customer's account number, see section 14.2.7 . <i>URL2</i>

Tag	Version	Description
<BILLERINFOURL> </BILLERINFO>		URL of human-readable description of additional information the biller would like the customer to have with regard to signing up. <i>URL2</i>

Note: Future versions of OFX will require <ADDR1> if <ADDR2> is specified and <ADDR2> if <ADDR3> is specified.

While <ACCTFORMAT> uses Unix-style regular expressions to describe the account number format, <ACCTEDITMASK> provides a simpler, alternative method. It uses two special characters. The character @ matches one letter, upper or lowercase. The character # matches one number. All other characters match themselves (letters are case insensitive).

Usage examples:

The following represents a 16-digit account number:

#####

A 16-digit account number, separated by hyphens into 4-digit chunks. First four characters must be 4128:

4128-####-####-####

4 letters, a hyphen, a number, a hyphen, 5 numbers:

@@@@-#-#####

10-digit account number that must begin with 153AG, and whose final 5 characters are numbers:

153AG#####

14.2.6 Status Codes <FINDBILLERRS>

Code	Meaning
0	Success (INFO)
2000	General error (ERROR)

14.2.7 Account Number Validation

Servers should implement a lightweight CGI (or equivalent) to validate account numbers. The URL provided in the <VALIDATE> can be accessed with an HTTP GET with three arguments: BILLERID, ACCOUNTNUMBER and CUSTOMERPOSTALCODE. The URL should respond with a text file that includes the following values:

1. Status: (Mandatory)

Error: An error condition (wrong number of parameters, Database error, etc.). Clarifying text may accompany the error status.

Passed: The account number is in an acceptable form for this biller (this is not a guarantee that the account will be accepted for the service).

Failed: The account number does not correspond to an acceptable account number for this biller. Clarifying text may accompany the failed status.

2. Account: (Optional) The preferred format or version of the account number presented in the request.

Heading: (Optional) Additional text to help explain problems to end-users.

Example:

<VALIDATE> = http://testit.com/validate.cgi

Client application uses HTTP GET with “http://testit.com/validate.cgi?billerid=5454&accountnumber=123-456-7890&customerpostalcode=12345”

The server would respond with one of these:

1. Error

Content-type: text/plain

<STATUS>error

<HEADING>The server is unable to process your request at this time. Please resubmit.

2. Failure

Content-type: text/plain

<STATUS>failed

<HEADING>123-456-7890 does not appear to be a valid account number

3. Passed

Content-type: text/plain

<STATUS>passed

<ACCOUNT>1234567890

14.2.8 Biller Payment Restrictions

In the <PAYMENTINSTRUMENTS> aggregate of <BILLERINFO> aggregate (see section [14.2.5.1](#)), the biller specifies the type of payment instruments it can accept for electronic payment. Since electronic payment does not have to occur through the OFX Bill Payment message set, the payment instruments can include some that OFX doesn't support—for example, CyberCash.

In some cases, a biller may have arranged for a certain party to act as its payment concentrator. This means that the biller expects to receive good funds and remit advice in a pre-arranged format from these concentrators. Such a biller will want to have customers direct their payments to the payment concentrator.

The <PAYMENTINSTRUMENTS> aggregate supports the specification of a payment concentrator from which the biller wants to receive funds. However, OFX currently does not provide a way for clients to get information about payment concentrators. Knowledge of payment concentrators will have to be “hardwired” into client applications. For example, the client application may know that a certain payment concentrator, BigConcentrator, is capable of receiving DigiCash funds. A biller who has BigConcentrator as its payment concentrator can then accept DigiCash funds from the customer without having to support the DigiCash protocol and infrastructure directly. The application would direct the DigiCash funds to the concentrator, who would in turn transfer funds and remit advice to the biller using the agreed-upon method.

14.2.8.1 Payment Instruments <PAYMENTINSTRUMENTS>

In the <PAYMENTINSTRUMENTS> aggregate, billers list which payment instruments they accept.

<i>Tag</i>	<i>Description</i>
<PAYMENTINSTRUMENTS>	Opening tag for payment instruments
<PAYMENTINSTRUMENT>	One or more payment instrument aggregates, see section 14.2.8.2
</PAYMENTINSTRUMENT>	
</PAYMENTINSTRUMENTS>	Closing tag for payment instruments

14.2.8.2 Payment Type and Brand <PAYMENTINSTRUMENT>

Each payment instrument is described by <PMTINSTRUMENTTYPE> and <BRAND>. If the server does not specify <BRAND>, the client assumes that all brands of the given <PMTINSTRUMENTTYPE> are acceptable.

Tag	Description
<PAYMENTINSTRUMENT>	Opening tag for payment instrument aggregate
<PMTINSTRUMENTTYPE>	Payment type, see section 14.2.8.3
<BRAND>	Accepted brand for given payment type, A-32
</PAYMENTINSTRUMENT>	Closing tag for payment instrument aggregate

14.2.8.3 Payment Instrument Types <PMTINSTRUMENTTYPE>

Type	Description
CONCENTRATOR	Organization that has a business agreement with the biller to send the biller funds and remittance advice.
CHECKINGACCOUNT	Draft on a demand deposit account (US)
CREDITCARD	Payment by Auth/Settle using Credit Card networks
ECoin	Protocol for payment with electronic cash

If the <BILLERINFO> does not list <PAYMENTINSTRUMENTS>, the following single <PAYMENTINSTRUMENT> is implied:

```
<PAYMENTINSTRUMENT>
  <PMTINSTRUMENTTYPE>CHECKINGACCOUNT
</PAYMENTINSTRUMENT>
```

14.3 Customer Signup

Once the customer has located a biller and its associated bill publisher, the customer must enroll with the bill publisher for Bill Presentment service and activate accounts for one or more billers at that bill publisher.

Bill Presentment uses the standard OFX Signup message set. This section discusses only those portions of signup that differ for Bill Presentment. For more information about the Signup message set, refer to [Chapter 8, "Activation & Account Information."](#)

Note: The “V2” versions of the account signup messages should be used with Bill Presentment.

14.3.1 Enrollment

To enroll with a bill publisher, the client uses the standard OFX enrollment aggregate <ENROLLRQ>. The bill publisher server returns an <ENROLLRS> that provides status about the enrollment and optionally returns a user ID and password to be used during subsequent signons.

To allow a client proxy system to enroll multiple clients, Bill Presentment allows multiple <ENROLLRQ> requests in the same OFX file, each wrapped in a <ENROLLTRNRQ>. A given bill publisher may not need some required fields in the <ENROLLRQ>; in such a case, the client proxy system can include dummy data.

14.3.2 Account Inquiry

To receive account information from a bill publisher, the client can use the standard OFX <ACCTINFORQ> aggregate, contained in the <ACCTINFOTRNRQ> wrapper.

The <ACCTINFORS> response returns a <PRESACCTINFO> aggregate for each of the customer's accounts with the billers at that bill publisher. Typically, the response will list only those accounts that have been activated for Bill Presentment service, not all available accounts.

Unlike a financial institution, bill publishers generally won't have information about all the accounts of its supported billers. Billers that also serve as their own bill publishers may be able to return available accounts as well as activated accounts. The bill publisher can use the <AVAILACCTS> element in the profile for the Signup message set to indicate whether the server can return available account information.

If the server cannot return information about all available accounts, the client must ask customers for account information prior to requesting service activation for one or more accounts.

14.3.2.1 Bill Presentment Account Information <PRESACCTINFO>

The <PRESACCTINFO> aggregate appears within the <ACCTINFORS> aggregate.

Tag	Description
<PRESACCTINFO>	Opening tag for bill presentment account information
<PRESACCTFROM>	Bill presentment account identification, see section 14.3.2.2
</PRESACCTFROM>	
<SVCSTATUS2>	Status of the Bill Presentment service for this account– AVAIL, PEND, ACTIVE, or REJECTED
<REASON>	Relevant only if <SVCSTATUS2>REJECTED is specified, A-255
</PRESACCTINFO>	Closing tag for bill presentment account information

14.3.2.2 Account Identification <PRESACCTFROM> <PRESACCTTO>

The <PRESACCTFROM> aggregate uniquely identifies a customer's account with a biller by the combination of bill publisher, biller ID, and account number. Biller IDs must be unique within a bill publisher.

Clients can optionally include a <USERID> in <PRESACCTFROM/TO> that is different from the one used in the <SONRQ>. This <USERID> supports account activation by a third party on behalf of a user. Based on access rights granted to the <SONRQ>, it is up to the server whether to honor such a request. More than one <SVCADD> can be present in a single <OFX> request file on behalf of multiple <USERID>s.

The <USERID> element is disallowed for single-customer OFX request files. When sent by or returned to a client proxy (or, other group context), any <PRESACCTFROM/TO> aggregates must include the USERID element. A client proxy would include (and receive) this element when initiating an OFX session. But, customer-specific clients initiating a session with the same server would never use or see this information.

Note: <USERID> is not intended to identify individual users of joint accounts. If a transaction might include two different USERIDs within otherwise identical <PRESACCTFROM> aggregates, servers should deliver two separate bills (download the same bill twice in <PRESLISTRS>), allow either to update the bill status (in <PRESNOTIFYRQ> or <BILLSTATUSMODRQ>), or deliver two separate <PRESACCTINFO> aggregates in <PRESGRPACCTINFOTRNRs>. It is up to the server to keep track of activity on joint accounts.

The Bill Publisher would not normally know the <PAYEEID2> and <PAYEELSTID2> (or their <SPNAME> context) information relevant to a particular client or user. But, after setting up (or changing) Biller as a payee with some Payment provider, a client may execute a <SVCCHG> <ACCTRQ> request that updates the corresponding <PRESACCTFROM> at the Bill Publisher.

This request could pass payment identifiers to the Bill Publisher. Since this information does not make the <PRESACCTFROM> more unique, clients may omit <SPNAME>, <PAYEEID2> and <PAYEELSTID2> when using <PRESACCTFROM> outside an <ACCTRQ> request. Servers should return this information in all contexts. After a client has supplied payee information, servers must not make server-initiated changes to the information unless the given (or implied) <SPNAME> is controlled by the same provider. Such servers may include (and update) payment identifiers from their Payment provider prior to the client including them in an <ACCTRQ> request.

Tag	Version	Description
<PRESACCTFROM>		
<BILLPUB>		Official standard name of bill publisher, A-32
<BILLERID>		ID of this biller at this bill publisher, A-32
<BILLERNAME>	1.6 add	Name of the biller; matches <NAME> element in <BILLERINFO>. See section 14.2.5.1 . This element may be used only in cases where <PRESACCTFROM> is sent by the server (for example, in <PRESBILLINFO> or <PRESACCTINFO>), A-32
<ACCTID>		Account number, A-22
<PRESNAMEADDRESS>		Customer's name / address with the biller, see section 14.3.2.2.1
</PRESNAMEADDRESS>		
<USERID>		Customer's user ID, A-32
<SPNAME>	1.6 add	Service provider name. Used to scope the <PAYEEID2> and / or <PAYEELSTID2> (if provided) to a particular Payment service. Allowed only when <PAYEEID2> or <PAYEELSTID2> appear in <PRESACCTFROM>. A-32 Note: <SPNAME> must be supplied with <PAYEELSTID2> unless Bill Publisher also provides a payment service.
<PAYEEID2>	1.6 add	Payee identifier. Identifies this Biller at the user's Payment provider. When sent in account activation, it is intended for storage on the Bill Presentment database, such that it can be returned in subsequent inquiries utilizing this aggregate. Used by clients to facilitate accurate Payee add or change requests when the Bill Presentment service (possibly, due to a prior client <ACCTRQ> request) knows the Payee ID at a Payment provider. The client may use this identifier to match the Biller as known by the Bill Presentment service to a Payee as known by the Payment provider. See section 14.5 . <i>SRVRTID2</i>

Tag	Version	Description
<PAYEELSTID2>	1.6 add	Payee list identifier. Identifies this Biller on the user's payee list at their Payment provider. When sent in account activation, it is intended for storage on the Bill Presentment database, such that it can be returned in subsequent inquiries utilizing this aggregate. Used by clients to facilitate accurate Payee add or change requests when the Bill Presentment service (possibly, due to a prior client <ACCTRQ> request) knows the Payee List ID at a Payment provider. The client may use this identifier to match the Biller as known by the Bill Presentment service to a Payee as known by the Payment provider. See section 14.5. SRVRTID2
</PRESACCTFROM>		

<PRESACCTTO> follows the same structure as <PRESACCTFROM>.

14.3.2.2.1 Customer Information with the biller <PRESNAMEADDRESS>

Tag	Version	Description
<PRESNAMEADDRESS>		Customer name and address information
<NAMEACCTHELD>		Customer's name as it appears on the account, A-96
<BUSNAMEACCTHELD>	1.6 add	Optional "Does Business As" name associated with this account, A-96
<ADDR1>		Customer's address line 1, A-32
<ADDR2>		Customer's address line 2, A-32
<ADDR3>		Customer's address line 3, A-32
<CITY>		Customer's city, A-32
<STATE>		Customer's state, A-5
<POSTALCODE>		Customer's postal code, A-11
<COUNTRY>		Customer's country; 3-letter country code from ISO / DIS-3166, A-3
<DAYPHONE>		Customer's telephone number, A-32
<EVEPHONE>		Customer's telephone number, A-32
</PRESNAMEADDRESS>		

Note: Future versions of OFX will require <ADDR1> if <ADDR2> is specified and <ADDR2> if <ADDR3> is specified.

14.3.3 Service Activation

Bill Presentment uses the standard service activation messages defined in Chapter 8, "Activation & Account Information."

The account service request aggregate <ACCTRQ> accepts action-specific aggregates for service additions, changes, and deletions. To add Bill Presentment service to an account, the client sends an <ACCTRQ> with an <SVCADD> for the service <SVC2>PRESSVC.

14.3.3.1 Service Addition <SVCADD>

When requesting service activation using <SVCADD>, <PRESACCTTO> is used to specify the customer's account with a specific biller. <PRESACCTTO> includes the optional <PRESNAMEADDRESS> aggregate to identify the customer's name and address as it is registered at the biller. In most cases however, the address specified in the <ENROLLRQ> or most recent <CHGUSERINFORQ> is used to provide the <PRESNAMEADDRESS> when activating an account. Clients need only include <PRESNAMEADDRESS> if a special billing address is specified for the customer at the given biller. For example, some billers and service providers may require exact matches for remittance addresses. The user's enrollment data may specify the same location described in the biller's database, but not provide an exact match. In that case, <PRESNAMEADDRESS> may be required for successful service activation.

In cases where <PRESACCTTO> is forwarded to other servers but enrollment information was used rather than a client-provided <PRESNAMEADDRESS> aggregate, the <PRESNAMEADDRESS> information is neither stored at the server nor returned to the originating client with the <PRESACCTTO> aggregate. That is, clients which do not include the <PRESNAMEADDRESS> aggregate in their requests should never see it in a later response from the server.

14.3.3.2 Service Change <SVCCHG>

The server's profile indicates support for storing <PRESNAMEADDRESS> information by including <CANUPDATEPRESNAMEADDRESS>Y. In this case, <SVCCHG> requests may be used to update or delete the <PRESNAMEADDRESS> information stored by the server. <CHGUSERINFORQ> requests have no effect upon this information. Furthermore, no server-initiated transactions change the stored address data. Servers must always return <PRESNAMEADDRESS> data exactly as the client sent it. The presentment server may however forward a customer's change of address entered via <SVCCHG> to the proper biller.

For servers that support storing <PRESNAMEADDRESS>, the <PRESNAMEADDRESS> data can also be used to support users who receive bills at multiple locations.

If a server's profile includes <CANUPDATEPRESNAMEADDRESS>N, the <PRESNAMEADDRESS> should not be included in any <SVCCHG> requests. <PRESNAMEADDRESS> data is used only for the initial activation in this case.

14.3.4 Service Status Update for Groups of Customers

The service activation requested with <SVCADD> will often not happen immediately. In this case, a request for account information <ACCTINFORQ> will return an <ACCTINFORS> with a <SVCSTATUS2>PEND. To find out whether the account has been activated, the client can either send <ACCTINFORQ> once per session until it returns <SVCSTATUS2>ACTIVE, or it can include an <ACCTSYNCRQ> in each session to catch an unsolicited <ACCTRS> response to the <SVCADD> message.

This section describes a method of checking for status changes on behalf of a group of customers within the Bill Presentation service. This method is designed to be used by customer service representatives and client proxy systems. This method applies only where <SVC2> is PRESSVC. To check status for a single customer, use the standard signup messages. For more information, see Chapter 8, "Activation & Account Information."

14.3.4.1 Account Information Request <ACCTINFORQ>

The client uses <ACCTINFORQ> to request information about accounts whose status has changed since the last time the request was made. This request is typically used to retrieve a list of accounts whose status has changed from <SVCSTATUS2>PEND to <SVCSTATUS2>ACTIVE.

For use in the Bill Presentation message set, the <ACCTINFORQ> request must appear within a <PRESGRPACCTINFOTRNRQ> transaction wrapper. This transaction wrapper includes the identifier for the requested group. The <ACCTINFORQ> request may also appear in the <ACCTINFOTRNRQ> wrapper described in Chapter 8, "Activation & Account Information."

Tag	Version	Description
<ACCTINFORQ>	1.6 add	Opening tag for billing account information request
<DTACCTUP>		Last <DTACCTUP> received in a response, <i>datetime</i>
<SVC2>		Zero or more. Services to be included in <ACCTINFORS>. If absent, all supported services are being requested. BANKSVC = Banking service BPSVC = Payment service INVSVC = Investments PRESSVC = Bill Presentment service Note: If used in this message set, the value must be PRESSVC.
</ACCTINFORQ>		Closing tag for billing account information request

14.3.4.2 Account Information Response <ACCTINFORS>

The <ACCTINFORS> aggregate contains zero or more <ACCTINFO> aggregates, which provide the updated account information.

For use in this message set, the <ACCTINFORS> response must appear within a <PRESGRPACCTINFOTRNRS> transaction wrapper.

Tag	Description
<ACCTINFORS>	Opening tag for billing account information response
<DTACCTUP>	Date and time of last update to account information on the server, <i>datetime</i>
<ACCTINFO>	Zero or more account information aggregates, see section 8.5.3. Each <ACCTINFO> aggregate contains at most one <PRESACCTINFO> aggregate, consistent with section 8.5.3. Left out of the response when no <SVC2>PRESSVC accounts for the specified <USERID> or <GROUPID> or current user are found. Note: When <DTACCTUP> indicates the client is up-to-date, server should not return surrounding <ACCTINFORS>.
</ACCTINFO>	
</ACCTINFORS>	Closing tag for billing account information response

14.3.4.3 Group Account Information Transaction Request <PRESGRPACCTINFOTRNRQ>

As a special transaction wrapper for <ACCTINFORS>, <PRESGRPACCTINFOTRNRQ> specifies whether the client is requesting account information for a single user or a group of users.

If the client specifies <GROUPID>, the client is requesting updated account information for a group of users. The server returns an <ACCTINFORS> response with a <PRESACCTINFO> aggregate for each account whose status has changed.

Standard signup messages are the preferred method for checking the status of a single customer, however <PRESGRPACCTINFOTRNRQ> may also be used for a single customer. (Standard signup messages are described in Chapter 8, "Activation & Account Information.")

If the client specifies <USERID> in the <PRESGRPACCTINFOTRNRQ> transaction wrapper, the client is requesting updated account information for a single user. If (in contrast) a <USERID> is specified in <SIGNUPMSGSRQV2>, then all transactions (including <ACCTINFORS>) within that message set wrapper are to be performed on behalf of the user identified by that <USERID>. In either case, if the user's account information has changed since the last time the client requested it, the server will return the account information in <ACCTINFORS>

The client should specify either <USERID> or <GROUPID>; if both are absent, the server uses the <USERID> from the signon request <SONRQ>.

Tag	Description
<PRESGRPACCTINFOTRNRQ>	Opening tag for the transaction request
<TRNUID>	Client-assigned globally unique ID for this transaction, <i>trnuid</i>
<CLTCOOKIE>	Data to be echoed in the transaction response, A-32
<TAN>	Transaction authorization number, A-80
Specify either <USERID> or <GROUPID>	
<USERID>	Requests account information for the specified user, A-32
- or -	
<GROUPID>	Requests account information for users in the group, A-32
<ACCTINFORQ>	Account information request aggregate, (See section 8.5.1).
</ACCTINFORQ>	
</PRESGRPACCTINFOTRNRQ>	Closing tag for the transaction request

14.3.4.4 Group Account Information Transaction Response <PRESGRPACCTINFOTRNRS>

As a special transaction wrapper for <ACCTINFORS>, <PRESGRPACCTINFOTRNRS> contains an <ACCTINFORS> aggregate with zero or more <PRESACCTINFO> aggregates. The <ACCTINFORS> aggregate returns one <PRESACCTINFO> aggregate for each account for which there was a change of status since the <DTACCTUP> date specified. <ACCTINFORS> should not be sent when the client is up-to-date.

Note: Not sending a response aggregate in this case is an exception to rules outlined in sections 2.4.6 and 3.1.5. And, sending a partial response (not every <ACCTINFO> aggregate for the user or group, just changed information) differs from the normal processing of <ACCTINFORQ> (see section 8.5). Within the Signup message set, the <ACCTINFORS> contains all account information if any portion is out of date.

The server includes information for only those USERIDs for which the requester has access rights.

Note: <USERID> is not intended to identify individual users of joint accounts. If a transaction might include two different USERIDs within otherwise identical <PRESACCTFROM> aggregates, servers should deliver two separate copies of the account information (download almost the same account information twice). It is up to the server to keep track of activity on joint accounts. This may occur if, for example, joint account holders are associated with the same <GROUPID>.

<i>Tag</i>	<i>Description</i>
<PRESGRPACCTINFOTRNRS>	Opening tag for the transaction response
<TRNUID>	Client-assigned globally unique ID for this transaction, <i>trnuid</i>
<STATUS>	
</STATUS>	
<CLTCookie>	Data to be echoed in the transaction response, A-32
<ACCTINFORS>	Account information response aggregate. See section 8.5.2.
</ACCTINFORS>	
</PRESGRPACCTINFOTRNRS>	Closing tag for the transaction response

14.3.4.5 Status Codes <PRESGRPACCTINFORS>

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
1	The client is up-to-date
2000	General error (ERROR)
2002	General account error (ERROR)
2006	Account not found (ERROR)
2008	Account not authorized (ERROR)
15508	Transaction not authorized (ERROR)

14.4 Bill Delivery

The Bill Delivery message set contains messages to obtain bills. The message set <PRESDLVMSGSETV1> contains the following aggregates: <PRESDLVMSGSRQV1> and <PRESDLVMSGSRSV1>.

14.4.1 Bill Delivery Process

Typically, the client periodically requests a list of bills from the bill publisher. The bill publisher responds with a list of bills, each of which contains summary data such as the due date and amount due. For each bill, the bill publisher might also return a URL to a Web site that contains an HTML-rendered version of the bill. Depending on the client's request, the server might also return structured bill detail for a given bill.

The aggregate for a bill list request is <PRESLISTRQ>. This request must be wrapped inside <PRESLISTTRNRQ>. There is no synchronization wrapper for bill list requests, since clients that require a list of bills can send another <PRESLISTRQ>.

The transaction wrapper <PRESLISTTRNRQ> contains optional elements that allow bills for one or more customers to be accessed by customer service representatives or client proxy systems. It is up to the server to decide who can access bills other than their own; it is recommended that all such access be logged in an audit trail.

14.4.2 Bill List Retrieval

<PRESLISTRQ> retrieves bills from the bill publisher. The bill publisher returns a <PRESLISTRS> response that contains a list of one or more bills.

14.4.2.1 Bill List Request <PRESLISTRQ>

The client requests bills from a bill publisher by date range. To specify the date range, clients use <DTSTART> and <DTEND>, as described in section [3.2.7](#). The date range includes all bills that were added or modified within the date range.

The bill publisher returns information sufficient to identify the biller and provide the amount due, due date, and remittance information so that a payment can be made to the biller. The bill publisher does not provide a viewable form of the bill, but returns a URL to an HTML rendering of the bill. Billing detail, such as individual purchases or transactions, can be included in the original response or obtained from a subsequent <PRESDETAILRQ>.

The <PRESLISTRQ> must be wrapped in the <PRESLISTTRNRQ> transaction wrapper.

Tag	Version	Description
<PRESLISTRQ>		Opening tag for bill list request
<BILLPUB>		Official standard name of bill publisher, A-32
<DTSTART>		If present, indicates earliest date for which to include bills, <i>datetime</i>
<DTEND>		If present, indicates latest date for which to include bills, <i>datetime</i>
<DTDUEBY>	1.6 add	If present, indicates that the customer is requesting bills due on or before the date/ time specified in <DTDUEBY>. <i>datetime</i>
<BILLERID>	1.6 add	Biller Identifier, If present, restricts the response to the given Biller, A-32
<BILLID>		If present, restrict response to given statement identifier, A-32
<BILLTYPE>	1.6 add	0 or more. If present, indicates which types of bills the customer is requesting. Possible values are: BILL = Invoice of an amount due to the biller that is payable. STATEMENT = History of activity on an account with the biller that is not payable. NOTICE = Generic letter from either the biller or the bill publisher that is not payable.
<BILLSTATUSCODE>	1.6 add	0 or more. If present indicates which bill statuses the customer is requesting. Possible values are: NEW = The server has not sent the bill to either the client or client proxy. This is the initial status code of a bill. DELIVERED = The server has sent the bill to either a client or client proxy. VIEWED = The customer has seen the bill. Implies previous status of DELIVERED. RETIRED = The customer no longer wishes to see this bill. Implies previous status of DELIVERED. WITHDRAWN = The biller or publisher no longer wishes this bill to be displayed. UNDELIVERABLE = Attempts to deliver this bill to the consumer in a timely fashion have failed. This status is not generally used when presenting a bill to a consumer. However, notifications using this status cover many useful cases.

Tag	Version	Description
<BILLPMTSTATUSCODE>	1.6 add	0 or more. Bill payment status code. If present, indicates the customer is requesting bills matching the bill payment status code. Possible values are: NONE = There is neither a payment scheduled, nor has one been made against this bill. This may be the initial payment status of a bill. SCHEDULED = A payment has been scheduled, but not yet processed against this bill. PROCESSED = The payment has been processed against this bill, and can no longer be cancelled. POSTED = The biller has posted the payment against this bill. PAIDOUTOFBAND = A Payment has been initiated for this bill via a mechanism that does not report status via OFX. This status is intended to indicate the customer has paid the biller directly with cash or a check or has initiated an electronic payment through a mechanism that does not report payment status through OFX. AUTOPAY = The Biller or Service Provider will initiate the payment based on a pre-authorization by the customer, typically a “good until cancelled” instruction with no defined end date. In the US this is often implemented using a recurring pre-authorized ACH debit, though some Billers offer pre-authorized automatic payment through credit card. Examples include monthly deductions to cover a mortgage, regular payments from a checking account to a credit card, and the Automatic Payment Service (APS) offered by many utilities. Like NONE, this may be the initial payment status of the bill. CANCELLED = The customer cancelled the payment that was previously scheduled. UNPAYABLE = None of the Payment Instruments allowed for this bill are supported by the Payment Provider. This is intended to be used where the bill restricts payment to a subset of the Payment Instruments allowed in the Biller Directory entry. This could occur if the Payment Provider or the Biller changed their supported payment instrument types after enrollment and account activation.
<NOTIFYWILLING>		Flag indicating that client is prepared to send notifications of bill delivery, if desired (see section 14.4.5), <i>Boolean</i>
<INCLUDEDDETAIL>		Flag indicating bill detail should be included too, <i>Boolean</i>
<INCLUDEBILLSTATUS>	1.6 add	Flag indicating bill status should be included too, <i>Boolean</i> Default is N.

Tag	Version	Description
<INCLUDEBILLPMTSTATUS>	1.6 add	Flag indicating bill payment status should be included, <i>Boolean</i> Default is N.
<INCLUDESTATUSHIST>	1.6 add	Flag indicating bill status history and / or bill payment status history should be included too. Only valid if <INCLUDEBILLSTATUS>Y and / or <INCLUDEBILLPMTSTATUS>Y are specified. <i>Boolean</i> Default is N.
<INCLUDECOUNTS>	1.6 add	If Y, indicates that the response should include <PRESCOUNTS> and not <PRESBILLINFO>, <i>Boolean</i> May not be Y if <INCLUDESUMMARY>Y, <INCLUDEDETAIL>Y, <INCLUDEBILLSTATUS>Y, <INCLUDEBILLPMTSTATUS>Y, or <INCLUDESTATUSHIST>Y are specified. Default is N.
<INCLUDESUMMARY>	1.6 add	Include bill summaries (<PRESBILLINFO>), <i>Boolean</i> . May not be N if <INCLUDEDETAIL>Y, <INCLUDEBILLSTATUS>Y, <INCLUDEBILLPMTSTATUS>Y, <INCLUDESTATUSHIST>Y, or <INCLUDECOUNTS>N are specified. Unlike other boolean elements of this request, the default is Y.
</PRESLISTRQ>		Closing tag for bill list request

14.4.2.2 Bill List Response <PRESLISTRS>

The <PRESLISTRS> response must appear within a <PRESLISTTRNRS> transaction wrapper.

The <PRESLISTRS> response can contain zero or more bill summaries, with optional detail. Each bill summary corresponds to a (usually monthly) bill. When a server has no bills to return, it should return <STATUS><CODE>0 and leave out the <PRESLIST> within the <PRESLISTRS>.

When multiple selection criteria are used they are ANDed (as described in section 2.4.4.4). When counts are requested in the <PRESLISTRQ> (<INCLUDECOUNTS>), the server should include counts of each status requested where the bill's <BILLSTATUSCODE> matches one of those specified in the <PRESLISTRQ>, and the bill's <BILLPMTSTATUSCODE> matches one of those specified in the <PRESLISTRQ>. Thus, a request containing <BILLSTATUSCODE>NEW, <BILLSTATUSCODE>DELIVERED, <BILLSTATUSCODE>WITHDRAWN>, <BILLPMTSTATUSCODE>NONE, and <BILLPMTSTATUSCODE>CANCELLED>, will return three <BILLSTATUSCODE> counts and two <BILLPMTSTATUSCODE> counts. The sum of <BILLSTATUSCODE> counts will be equal to the sum of the <BILLPMTSTATUSCODE> counts. No inference can be drawn as to which bills have a combination of a specific <BILLSTATUSCODE> value and a specific <BILLPMTSTATUSCODE> value.

Tag	Version	Description
<PRESLISTRS>		Opening tag for bill list response
<BILLPUB>		Official standard name of bill publisher, A-32
<USERID>		User whose bill data is being returned. Must match <USERID> provided in <PRESLISTTRNRQ> (if specified), "anonymous0000000000000000000000" (if <GROUPID> was specified in the <PRESLISTTRNRQ>), or the <USERID> for the authenticated user (otherwise), A-32
<DTSTART>		Start date of bills returned, <i>datetime</i>
<DTEND>		Date to present as start date for next request, <i>datetime</i>
<PRESLIST>		Bill summary list, see section 14.4.2.2.1
</PRESLIST>		
<PRESCOUNTS>	1.6 add	Bill Counts Aggregate
<BILLSTATUSCOUNTS>		Bill Status Counts, zero or more. The count(s) of all bills matching the given selection criteria, having a particular status(es). If <BILLSTATUSCODE> is not included in the request with <INCLUDECOUNTS>Y, counts are returned for every status with a non-zero count.

The bill identifier <BILLID> must uniquely identify the bill with the bill publisher (not merely with the biller). The <BILLPUB> and <BILLID> combination must be globally unique, not the <FI> and <BILLID> combination.

The bill date <DTBILL> is usually a fixed number of days after the end of the bill period. It is not the date on which the bill publisher received the bill for publication.

Tag	Version	Description
<PRESBILLINFO>		Opening tag for bill information
<BILLID>		Identifier for this bill within the bill publisher, A-32
<PRESACCTFROM>		Biller account information (see section 14.3.2.2)
</PRESACCTFROM>		
<PAYEEID2>		Payee identifier. Specify only if the bill publisher is also provides Bill Payment service. See section 14.5.2 . <i>SRVRTID2</i>
<BILLREFINFO>		Biller-defined text to include with the payment, for the biller's Accounts Receivable reconciliation. Sections 14.5 , 14.5.2 . A-80
<AMTDUE>		Full payment amount due, <i>amount</i>
<MINAMTDUE>		Minimum payment amount due, <i>amount</i>
<DTPMTDUE>		Payment due date, <i>datetime</i>
<DTBILL>		Bill date, <i>datetime</i>
<DTOPEN>		Opening statement date, <i>datetime</i>
<DTCLOSE>		Closing statement date, <i>datetime</i>
<PREVBAL>		Balance of the account as of the previous period, <i>amount</i>
<ACTIVITY>		Net inflows and outflows for the account since the last period, <i>amount</i>
<ACCTBAL>		Balance of the account at the end of the current period, <i>amount</i>
<INVOICE2>		Optional invoice data that the biller would like to receive with a payment (See 12.5.3.3). Client applications should allow the user to edit the amounts before returning this in a payment Note: The <LITMCODE> supplied by the biller should not be modified.
</INVOICE2>		
<NOTIFYDESIRED>		Indicator that a delivery notification (see section 14.4.5) is desired, <i>Boolean</i>

Tag	Version	Description
<BILLTYPE>	1.6 add	Bill Type. Possible values are: BILL = Invoice of an amount due to the biller that is payable. STATEMENT = History of activity on an account with the biller that is not payable. NOTICE = Generic letter from either biller or the bill publisher that is not payable.
<BILLSTATUS>	1.6 add	Zero or more bill status aggregates. See section 14.4.2.2.3 .
</BILLSTATUS>		
<BILLPMTSTATUS>	1.6 add	Zero or more bill payment status aggregates. See section 14.4.2.2.4 .
</BILLPMTSTATUS>		
<STMNTIMAGE>		Statement image aggregate, see section 14.4.2.2.5
</STMNTIMAGE>		
Choose <i>DETAILAVAILABLE</i> or <i>BILLDETAILETABLE</i> , but not both		
<DETAILAVAILABLE>		Indicator that structured detail is available, <i>Boolean</i>
-or-		
<BILLDETAILETABLE>		Bill details, when requested, see section 14.4.3.2.1
</BILLDETAILETABLE>		
</PRESBILLINFO>		Closing tag for bill information

If <PREVBAL>, <ACTIVITY>, and <ACCTBAL> are all present, then <PREVBAL> and <ACTIVITY> must add up to <ACCTBAL>.

Note: This means payments from the consumer received by the biller are counted as negative activity.

14.4.2.2.3 Bill Status <BILLSTATUS>

Tag	Version	Description
<BILLSTATUS>	1.6 add	
<BILLSTATUSCODE>		Bill status code. Possible values are: NEW = The server has not sent the bill to either the client or client proxy. This is the initial status code of a bill. DELIVERED = The server has sent the bill to either a client or client proxy. VIEWED = The customer has seen the bill. Implies previous status of DELIVERED. RETIRED = The customer no longer wishes to see this bill. Implies previous status of DELIVERED. WITHDRAWN = The biller or publisher no longer wishes this bill to be displayed. UNDELIVERABLE = Attempts to deliver this bill to the consumer in a timely fashion have failed. This status is not generally used when presenting a bill to a consumer. However, notifications using this status cover many useful cases
<DTEFF>		Date/Time at which the status became effective (for example, the date and time a bill is created in the initial status description for a bill). <i>datetime</i>
<STATUSMODBY>		Status modified by. Servers are not required to store this information. Possible values are: CUSTOMER = customer. CUSTAGENT = An automated software agent acting on behalf of customer. BILLPUBLISHER = Bill Publisher. BILLPUBLISHERSR = Service representative acting on behalf of the payment provider. PMTPROVIDER = Payment Provider. PMTPROVIDERSR = Service representative acting on behalf of the payment provider. BILLER = biller. BILLERSR = Service representative acting on behalf of the biller.
</BILLSTATUS>		

14.4.2.2.4 Bill Payment Status <BILLPMTSTATUS>

Tag	Version	Description
<BILLPMTSTATUS> <SRVRTID2> <BILLPMTSTATUSCODE>	1.6 add	Zero or more bill payment status aggregates The server transaction ID of the payment against this bill (see section 3.2.2), A-36 Bill payment status code. Possible values are: NONE = There is neither a payment scheduled, nor has one been made against this bill. SCHEDULED = A payment has been scheduled, but not yet processed against this bill. PROCESSED = The payment has been processed against this bill, and can no longer be cancelled. POSTED = The biller has posted the payment against this bill. PAIDOUTOFBAND = A Payment has been initiated for this bill via a mechanism that does not report status via OFX. This status is intended to indicate the customer has paid the biller directly with cash or a check or has initiated an electronic payment through a mechanism that does not report payment status through OFX. AUTOPAY = The Biller or Service Provider will initiate the payment based on a pre-authorization by the customer, typically a “good until cancelled” instruction with no defined end date. In the US this is often implemented using a recurring pre-authorized ACH debit, though some Billers offer pre-authorized automatic payment through credit card. Examples include monthly deductions to cover a mortgage, regular payments from a checking account to a credit card, and the Automatic Payment Service (APS) offered by many utilities. Like NONE, this may be the initial payment status of the bill. CANCELLED = The customer cancelled the payment that was previously scheduled. UNPAYABLE = None of the Payment Instruments allowed for this bill are supported by the Payment Provider. This is intended to be used where the bill restricts payment to a subset of the Payment Instruments allowed in the Biller Directory entry. This could occur if the Payment Provider or the Biller changed their supported payment instrument types after enrollment and account activation.

Tag	Version	Description
<DTEFF>		Date / Time at which the status became effective (for example, the date and time a bill is created in the initial status description for a bill). <i>datetime</i>
<STATUSMODBY>		Status modified by. Servers are not required to store this information. Possible values are: CUSTOMER = customer. CUSTAGENT = An automated software agent acting on behalf of customer. BILLPUBLISHER = Bill Publisher. BILLPUBLISHERSR = Service representative acting on behalf of the payment provider. PMTPROVIDER = Payment Provider. PMTPROVIDERSR = Service representative acting on behalf of the payment provider. BILLER = biller. BILLERSR = Service representative acting on behalf of the biller.
</BILLPMTSTATUS>		

14.4.2.2.5 Statement Image <STMNTIMAGE>

The <STMNTIMAGE> aggregate provides one or more URLs that point to a fully rendered image of the bill, in HTML.

<IMAGEURL> accesses the complete bill image. This URL may contain navigation to other sites, or to other pages of bill images at the same site.

To support off-line viewing of the bill, the server may provide one or more additional URLs. Each <PREFETCHURL> points to a local Web page.

Each URL associated with <IMAGEURL> and <PREFETCHURL> must include an authentication token at the end (for example, *?authtoken=randomString*). These embedded tokens guarantee that only the customer can access the Web page. Accessing the statement image requires SSL. The bill publisher might include an expiration date for the authentication token, and hence for the URLs. The expiration date could be quite short (for example, 1 hour) or quite long (for example, 1 month). After the expiration date, the client can obtain a new authentication token only by sending a new <PRESLISTRQ> request.

<i>Tag</i>	<i>Description</i>
<STMNTIMAGE>	Opening tag for statement image
<IMAGEURL>	URL address for retrieving an image of the complete bill encoded as HTML. This can be cached by the client for later display, or it can be viewed live directly from the Web. <i>URL2</i>
<PREFETCHURL>	Advice in support of off-line viewing. Zero or more. <i>URL2</i>
<DTEXPIRE>	Date after which embedded authentication token expires, <i>datetime</i>
</STMNTIMAGE>	Closing tag for statement image

14.4.2.3 Bill List Transaction Request <PRESLISTTRNRQ>

As the transaction wrapper for <PRESLISTRQ>, this aggregate specifies whether the client is requesting bills for a single user or a group of users. The optional <USERID> and <GROUPID> elements support the following scenarios:

- ◆ **A customer requests his or her own bills from the bill publisher:** In this case, the client can optionally specify the customer's <USERID> in the <PRESLISTTRNRQ>. If the client does not specify <USERID>, the bill publisher uses the <USERID> in the signon request <SONRQ>.
- ◆ **A customer service representative requests a bill on behalf of a user:** The client sends the representative's <USERID> in the signon request <SONRQ>. To specify the user for which the representative is retrieving bills, the client sends the customer's <USERID> in the <PRESLISTTRNRQ>. The bill publisher must ultimately decide whether the customer service representative can access the requested bills. In its response, the bill publisher includes only those bills for which the requester has access privileges.
- ◆ **A client proxy system fetches bills on behalf of a group of users:** Instead of sending a <USERID>, the client sends the <GROUPID> that identifies the group of users. Within the <PRESLISTTRNRQ>, the bill publisher returns a single <PRESLISTRS> containing zero or more <PRESBILLINFO> aggregates for each user in the named group. Individual customers are distinguished using the <USERID> element of each <PRESACCTFROM> in the <PRESBILLINFO> aggregates. Again, the bill publisher decides whether access should be granted. Bill publishers that support usage of <GROUPID> must maintain knowledge of which users are in which named group. The OFX specification does not provide a way to track membership in a named group. Any such management must happen out-of-band.

Note: <USERID> is not intended to identify individual users of joint accounts. If a transaction might include two different USERIDs within otherwise identical <PRESACCTFROM> aggregates, servers should deliver two separate bills (download the same bill twice). It is up to the server to keep track of activity on joint accounts.

Tag	Description
<PRESLISTTRNRQ> <TRNUID> <CLTCOOKIE> <TAN> <i>Specify either <USERID> or <GROUPID></i>	Opening tag for bill list transaction request Client-assigned globally unique ID for this transaction, <i>trnuid</i> Data to be echoed in the transaction response, A-32 Transaction authorization number, A-80
<USERID> - or - <GROUPID>	If present, the bill request is on behalf of this particular user, A-32 If present, the bill request is on behalf of all users in the named group. If both <USERID> and <GROUPID> are absent, the <USERID> in the <SONRQ> is implied. A-32
<PRESLISTRQ> </PRESLISTRQ> </PRESLISTTRNRQ>	Bill List Request Aggregate Closing tag for bill list transaction request

14.4.2.4 Bill List Transaction Response <PRESLISTTRNRS>

Tag	Description
<PRESLISTTRNRS> <TRNUID> <STATUS> </STATUS> <CLTCOOKIE> <PRESLISTRS> </PRESLISTRS> </PRESLISTTRNRS>	Opening tag for bill list transaction response Client-assigned globally unique ID for this transaction <i>trnuid</i> Status aggregate Client provided data. REQUIRED if provided in request A-32 Bill list response aggregate, optional
	Closing tag for bill list transaction response

14.4.2.5 Status Codes <PRESLISTRS>

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2003	Account not found (ERROR)
2004	Account closed (ERROR)
2005	Account not authorized (ERROR)
2020	Invalid date (ERROR)
2027	Invalid date range (ERROR)
15508	Transaction not authorized (ERROR)

14.4.3 Bill Detail Retrieval

If statement detail is available for a bill, the client can retrieve the detail using a bill detail request <PRESDetailRQ>. One example of statement detail is the individual telephone calls from a telephone bill.

14.4.3.1 Bill Detail Request <PRESDetailRQ>

The <PRESDetailRQ> request must appear within a <PRESDetailTRNRQ> transaction wrapper.

<i>Tag</i>	<i>Description</i>
<PRESDetailRQ>	Opening tag for bill detail request
<BILLID>	Statement identifier from <PRESBILLINFO>, A-32
<BILLDETAILTABLETYPE>	If present, filters response to just tables of this type (See table 14.4.3.2.3), A-32.
</PRESDetailRQ>	Closing tag for bill detail request

14.4.3.2 Bill Detail Response <PRESDETAILRS>

The <PRESDETAILRS> request must appear within a <PRESDETAILTRNRS> transaction wrapper.

The bill detail response contains zero or more <BILLDETAILTABLE> aggregates.

<i>Tag</i>	<i>Description</i>
<PRESDETAILRS>	Opening tag for bill detail response
<PRESDETAIL>	Zero or more bill detail aggregates
<BILLID>	Statement identifier from <PRESBILLINFO>, A-32
<PRESACCTFROM>	Identifies biller account. Must be included if in response to an <PRESDETAILRQ>, is redundant inside <PRESBILLINFO>
</PRESACCTFROM>	
<BILLDETAILTABLE>	Zero or more bill detail table aggregates. See section 14.4.3.2.1 .
</BILLDETAILTABLE>	
</PRESDETAIL>	Closing tag for bill detail aggregate
</PRESDETAILRS>	Closing tag for bill detail response

14.4.3.2.1 Bill Detail Table <BILLDETAILTABLE>

The bill detail table allows billers to send tabular data to the customer in a flexible way. The table might contain phone calls from a telephone bill, or electrical meter readings for a utility bill.

A table consists of one or more rows, each having one or more columns. Within a table, all rows must have identical structures. The <BILLDETAILTABLETYPE> determines the “shape” or schema of the table. The <TABLENAME> gives a name to this table, and should be unique within an <PRESDetailRS>

Note: The bill detail table may be redesigned in the future. Please consider this area “under construction.”.

Tag	Description
<BILLDETAILTABLE>	Opening tag for bill detail table
<TABLENAME>	Name of bill detail table, A-32
<BILLDETAILTABLETYPE>	Type of bill detail table (See section 14.4.3.2.3), A-32.
<BILLDETAILROW>	Zero or more bill detail row aggregates, see section 14.4.3.2.2
</BILLDETAILROW>	
</BILLDETAILTABLE>	Closing tag for bill detail table

14.4.3.2.2 Bill Detail Row <BILLDETAILROW>

A <BILLDETAILTABLE> contains zero or more bill detail rows <BILLDETAILROW>.

A <BILLDETAILROW> contains zero or more columns <C>, whose meanings are specific to the type of table <BILLDETAILTABLETYPE> in which they occur. For the purpose of the DTD parser, all columns <C> are considered to be **Format: A-255**.

OFX requires all elements return data. If bill publishers do not use specific columns, they can return null columns, represented by the element <N>. All columns <N> are considered to be **Format: A-1**.

Note: Bill publishers must include one character of data in a null column. Bill publishers can omit blank columns at the end of a <BILLDETAILROW>, tag and all. DTD should not enforce ordering, i.e. it should look like this:

```
<!ELEMENT BILLDETAILROW          - - ( C | N )*>
```

<i>Tag</i>	<i>Description</i>
<BILLDETAILROW>	Opening tag for bill detail row
<C>	Zero or more column data elements, A-255
<N>	Zero or more column data elements, A-1
</BILLDETAILROW>	Closing tag for bill detail row

14.4.3.2.3 Table Types <BILLDETAILTABLETYPE>

OFX defines some common table types. Individual billers can define their own table types, and hence their own table structures, but must honor the custom tag naming convention outlined in section [2.7](#).

<i>Value</i>	<i>Description</i>
TransactionList	Table defined for “payment register”-style line items
CallLog	Table defined for record of telephone calls
ABC.Usage	Table defined by biller, not by OFX

14.4.3.2.3.1 TransactionList Table Type

<BILLDETAILTABLE> aggregates marked with <BILLDETAILTABLETYPE>TransactionLists have rows of 14 columns. The first column contains a unique identifier (like a BILLID), and must be present. Other columns may not always apply and can be left blank.

The TransactionList table type is a subset of the <STMTTRN> aggregate in section [11.4.3](#).

<i>Column</i>	<i>Name</i>	<i>Description</i>
1	BillId	Unique identifier token from server, A-32
2	TrnType	Transaction type (see section 11.4.3.1)
3	DtPosted	Date item was posted, <i>datetime</i>
4	DtUser	Date user initiated transaction, if known, <i>datetime</i>
5	TrnAmount	Amount of transaction, <i>amount</i>
6	CorrectBillId	If present, unique identifier of previously sent transaction that is corrected by this record, A-32
7	CorrectAction	Replace or delete. Specify only if column 6 is present.
8	CheckNum	Check or other reference number, A-12
9	RefNum	Other reference number, A-12
10	SIC	Standard Industrial Code, N-6
11	Name	Name of payee or description of transaction, A-32
12	Memo	Extra Information, <i>memo</i>
13	OrigCurSym	Original Currency Identifier (ISO 42173 3-letter), A-3
14	CurRate	Currency rate, ratio of currency to original currency, <i>rate</i>

14.4.3.2.3.2 CallLog Table Type

<BILLDETAILTABLE> aggregates marked with <BILLDETAILTABLETYPE>CallLog have rows of 10 columns.

Column	Name	Description
1	TNCalledFrom	Telephone number called from, A-32
2	CityStateFrom	City, state (or place, region) called from, A-16
3	TNCalled	Telephone number called, A-32
4	CityState	City, state (or place, region) called, A-16
5	Originated	Date / time call started, <i>datetime</i>
6	Type	Type of call, A-8
7	Rate	Rate (for example, Night, Day, Eve, Wknd), A-5
8	Duration	Duration of call in tenths of seconds, N-6
9	Cost	Cost of call, <i>amount</i>
10	TNChargedTo	Telephone number charged to, A-32

14.4.3.3 Status Codes <PREDETAILRS>

Code	Meaning
0	Success (INFO)
2000	General error (ERROR)
2023	Unknown BILLID (ERROR)
10600	Table type not found (ERROR)

14.4.4 Table Structure Definition

Clients can obtain the definition of a table structure by sending a table structure request <BILLTBLSTRUCTRQ>.

Clients need only request the structure of tables it does not already know about. For instance, the client might request the structure of a biller-specific table that starts with the x- prefix. Knowing the structure of a table allows the client to display the data more clearly or store the data in a more compact form, such as a database table.

14.4.4.1 Table Structure Request <BILLTBLSTRUCTRQ>

To identify the table, the client includes the type of table <BILLDETAILTABLETYPE> and unique identifier <BILLID> for the table. Although <BILLDETAILTABLETYPE> uniquely identifies the table, OFX requires the <BILLID> as well to allow various server implementations.

The <BILLTBLSTRUCTRQ> request must appear within a <BILLTBLSTRUCTTRNRQ> transaction wrapper.

<i>Tag</i>	<i>Description</i>
<BILLTBLSTRUCTRQ>	Opening tag for the table structure request
<BILLID>	Statement Identifier, A-32
<BILLDETAILTABLETYPE>	Table type for which the structure is requested (See table 14.4.3.2.3), A-32.
</BILLTBLSTRUCTRQ>	Closing tag for the table structure request

14.4.4.2 Table Structure Response <BILLTBLSTRUCTRS>

The <BILLTBLSTRUCTRS> response must appear within a <BILLTBLSTRUCTTRNRS> transaction wrapper.

The table structure response contains one or more column type definitions, which correspond positionally with the <C> aggregates in a <BILLDETAILROW> in a <BILLDETAILTABLE> of the corresponding <TABLETYPE>.

<i>Tag</i>	<i>Description</i>
<BILLTBLSTRUCTRS>	Opening tag for table structure response
<BILLID>	Table identifier, A-32
<BILLDETAILTABLETYPE>	Table type (See table 14.4.3.2.3), A-32.
<COLDEF>	Zero or more column definition aggregates (see section 14.4.4.2.1)
</COLDEF>	
</BILLTBLSTRUCTRS>	Closing tag for table structure response

14.4.4.2.1 Column Definition <COLDEF>

A column definition <COLDEF> associates a name and a data type with a column.

Tag	Description
<COLDEF>	Opening tag for column definition
<COLNAME>	Column name, A-32
<COLTYPE>	Column type, valid values are (choose one): (A-255, D, N-6), A-8. Specifying D in this field means the column type is datetime.
</COLDEF>	Closing tag for column definition

14.4.4.3 Status Codes <BILLTBLSTRUCTRS>

Code	Meaning
0	Success (INFO)
2000	General error (ERROR)
2023	Unknown BILLID (ERROR)
10600	Table type not found (ERROR)

14.4.5 Delivery Notification

In OFX 1.6, a new Bill Status Modify transaction was added. This new transaction (see section [14.4.6](#)) is a semantic superset of Delivery Notification. Bill Status Modify should be used instead of Delivery Notification.

The bill publisher can request delivery notification through the <NOTIFYDESIRED> flag in the <PRESBILLINFO> aggregate (see section [14.4.2.2.2](#)). The bill publisher will expect to receive the delivery notification only if the <PRESLISTRQ> had the <NOTIFYWILLING> flag set.

The delivery notification request tells the bill publisher that the client has presented the specified bills to the customer. This is a stronger statement than acknowledging that the bills have been received by the client, specifically when the client software implements the pre-fetching or (push) model. Delivery notification should be sent only once for any given bill, and it should be sent the first time that the Bill Summary is displayed. Receipt of a delivery notification by the bill publisher has no legal significance. OFX does not define the maximum elapsed time between the presentation of the bill and the delivery notification.

14.4.5.1 Delivery Notification Request <PRESNOTIFYRQ>

In OFX 1.6, a new Bill Status Modify request, <BILLSTATUSMODRQ>, was added. This new request (see section [14.4.6.1](#)) should be used instead of <PRESNOTIFYRQ>.

The <PRESNOTIFYRQ> request must appear within a <PRESNOTIFYTRNRQ> transaction wrapper.

Tag	Description
<PRESNOTIFYRQ>	Opening tag for delivery notification request
<PRESDeliveryID>	A bill delivery ID aggregate (see section 14.4.5.1.1)
</PRESDeliveryID>	
</PRESNOTIFYRQ>	Closing tag for delivery notification Request

14.4.5.1.1 Bill Delivery Identification <PRESDeliveryID>

This aggregate identifies a bill delivery instance and suggests when the bill was “seen.”

<DTSEEN> is the date and time at which the client displayed the bill to the customer. There is no legal significance to this bill delivery identification.

Tag	Description
<PRESDeliveryID>	Opening tag for the bill delivery identification
<PRESACCTFROM>	Billers account information
</PRESACCTFROM>	
<BILLID>	Identifies the bill from the given biller, A-32
<DTSEEN>	Date and time at which the bill was made available to the requester’s client, <i>datetime</i>
</PRESDeliveryID>	Closing tag for the bill delivery identification

14.4.5.2 Delivery Notification Response <PRESNOTIFYRS>

In OFX 1.6, a new Bill Status Modify request, <BILLSTATUSMODRS>, was added. This new request (see section [14.4.6.2](#)) should be used instead of <PRESNOTIFYRS>.

The <PRESNOTIFYRS> response must appear within a <PRESNOTIFYTRNRS> transaction wrapper.

The delivery notification response lets the client know that the delivery notification request was received by the bill publisher.

<i>Tag</i>	<i>Description</i>
<PRESNOTIFYRS>	Opening tag for Delivery Notification Response
<PRESDeliveryID>	A bill delivery ID aggregate (See section 14.4.5.1.1)
</PRESDeliveryID>	
</PRESNOTIFYRS>	Closing tag for Delivery Notification Response

14.4.5.3 Status Codes <PRESNOTIFYRS>

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2023	BILLID not found (ERROR)
15508	Transaction not authorized (ERROR)

14.4.6 Bill Status Modification

<BILLSTATUSMODRQ> is a new addition to OFX in version 1.6. This transaction (if the server supports it) should be used in place of <PRESNOTIFYRQ>.

14.4.6.1 Request <BILLSTATUSMODRQ>

The Bill Status Modify Request <BILLSTATUSMODRQ> must appear within a <BILLSTATUSMODTRNRQ> transaction wrapper.

Tag	Version	Description
<BILLSTATUSMODRQ>	1.6 add	
<BILLID>		Identifies the bill from a given biller, A-32
<BILLSTATUS>		Bill status aggregate. See section 14.4.2.2.3 .
</BILLSTATUS>		
<BILLPMTSTATUS>		Bill payment status aggregate. See section 14.4.2.2.4 .
</BILLPMTSTATUS>		
</BILLSTATUSMODRQ>		

14.4.6.2 Response <BILLSTATUSMODRS>

The Bill Status Modify Response <BILLSTATUSMODRS> must appear within a <BILLSTATUSMODTRNRS> transaction wrapper.

Tag	Version	Description
<BILLSTATUSMODRS>	1.6 add	
<BILLID>		Identifies the bill from a given biller, A-32
<BILLSTATUS>		Bill status aggregate. Echoed from the request. See section 14.4.2.2.3 .
</BILLSTATUS>		
<BILLPMTSTATUS>		Bill payment status aggregate. Echoed from the request. See section 14.4.2.2.4 .
</BILLPMTSTATUS>		
</BILLSTATUSMODRS>		

14.4.6.3 Status Codes <BILLSTATUSMODRS>

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2023	BILLID not found (ERROR)
15508	Transaction not authorized (ERROR)

14.5 Bill Payment

To pay a bill received through a <PRESLISTRQ> request, the client can use the Bill Payment message set defined in [Chapter 12, "Payments."](#) To construct the payment information <PMTINFO> (see section [12.5.3](#)), the client can use the bill information from <PRESBILLINFO>.

14.5.1 Remittance Information

The client should include the <BILLREFINFO> from the <PRESBILLINFO> aggregate as the <BILLREFINFO> in the <PMTINFO> aggregate. This token allows the biller to link the payment with the bill.

14.5.2 Payee Identification

Client software can produce <PAYEEID2> or <PAYEE2> in one of two ways.

If the same company provides Bill Presentment and Bill Payment services, the client can use the <PAYEEID2> included in the <PRESBILLINFO> aggregate.

If the Bill Payment provider is a different company, the client must use information from the <PRESACCTINFO> to construct the <PAYEE2> information.

14.6 Bill Presentment E-Mail

OFX currently defines a Bill Presentment e-mail message that clients can send to bill publishers. With this message, a customer can send a message to a bill publisher regarding one of his or her accounts.

The server acknowledges receipt of the message. The bill publisher then prepares a response that the client picks up when it synchronizes with the server. E-mail is subject to synchronization, using <PRESMAILSYNCRQ> (defined in section [14.6.4](#)) and <PRESMAILSYNCRS> (defined in section [14.6.5](#).)

<i>Client Sends</i>	<i>Server Responds</i>
Addressed message	Acknowledgment
PRES account information	
.	
.	Response to customer
.	
Synchronization request	

14.6.1 Bill Presentment Mail Request <PRESMAILRQ>

The client must identify the account to which account the customer query is related.

14.6.1.1 The <PRESMAILRQ> request must appear with a <PRESMAILTRNRQ> transaction wrapper.

<i>Tag</i>	<i>Description</i>
<PRESMAILRQ>	PRES-e-mail-request aggregate
<PRESACCTFROM>	Account-from aggregate
</PRESACCTFROM>	
<MAIL>	To, from, message information, see section 9.2.2
</MAIL>	
</PRESMAILRQ>	

14.6.2 Bill Presentment Mail Response <PRESMAILRS>

The <PRESMAILRS> request must appear with a <PRESMAILTRNRS> transaction wrapper.

<i>Tag</i>	<i>Description</i>
<PRESMAILRS>	PRES-e-mail-response aggregate
<PRESACCTFROM>	Account-from aggregate
</PRESACCTFROM>	
<MAIL>	To, from, message information, see section 9.2.2
</MAIL>	
</PRESMAILRS>	

14.6.3 Status Codes <PREMAILRS>

<i>Code</i>	<i>Meaning</i>
0	Success (INFO)
2000	General error (ERROR)
2002	General account error (ERROR)
2003	Account not found (ERROR)
2004	Account closed (ERROR)
2005	Account not authorized (ERROR)
15508	Transaction not authorized (ERROR)
16500	HTML not allowed (ERROR)
16501	Unknown mail To: (ERROR)

14.6.4 Request <PREMAILSYNCRQ>

Tag	Description
<PREMAILSYNCRQ> <i>Client synchronization option; <TOKEN2>, <TOKENONLY>, or <REFRESH></i>	Synchronization request aggregate
<TOKEN2>	Previous value of <TOKEN2> received for this type of synchronization request from server; 0 for first-time requests; <i>token2</i>
<TOKENONLY>	Request for just the current <TOKEN2> without the history, <i>Boolean</i>
<REFRESH>	Request for refresh of current state, <i>Boolean</i>
<REJECTIFMISSING>	If Y, do not process requests if client <TOKEN2> is out of date, <i>Boolean</i>
<INCIMAGES>	Y if the client accepts mail with images in the message body. N if the client does not accept mail with images in the message body. <i>Boolean</i>
<USEHTML>	Y if client wants an HTML response, N if client wants plain text, <i>Boolean</i>
<PRESACCTFROM>	Account-from aggregate
</PRESACCTFROM>	
<PREMAILTRNRQ>	Bill presentment mail transactions (0 or more)
</PREMAILTRNRQ>	
</PREMAILSYNCRQ>	

14.6.5 Response <PREMAILSYNCRS>.

Tag	Description
<PREMAILSYNCRS>	Synchronization response aggregate
<TOKEN2>	New synchronization token, <i>token2</i>
<LOSTSYNC>	Y if the token in the synchronization request is older than the earliest entry in the server's history table. In this case, some responses have been lost. N if the token in the synchronization request is newer than or matches a token in the server's history table. <i>Boolean</i>
<SYNCERROR>	Optional error code (See section 6.4.4.), <i>N-6</i> .
<PRESACCTFROM>	Account-from aggregate
</PRESACCTFROM>	
<PREMAILTRNRS>	Bill presentment mail transactions (0 or more)
</PREMAILTRNRS>	
</PREMAILSYNCRS>	

14.7 Message Sets and Profile

OFX separates the messages that the client and server send into groups called message sets. In its profile response <PROFRS>, each bill publisher or other server provider defines the message sets that it supports and any options available for those message sets.

This section defines the message sets supported by Bill Presentment. It then describes the corresponding message set profile aggregates that can be provided in the profile response <PROFRS>. The message set profile aggregates for the <PROFRS> allow a bill publisher or other server provider to customize its use of OFX. For example, a server might support the Bill Delivery message set <PRESDLVMSGSET>, but not the Group Account Information message set <PRESDIRMSGSET>.

For general information about profiles, see [Chapter 7, "FI Profile."](#)

14.7.1 Message Sets and Messages

Bill Presentment defines the following message sets:

- ◆ Biller Directory message set <PRESDIRMSGSET>, which includes messages for finding billers and bill publishers
- ◆ Bill Delivery message set <PRESDLVMSGSET>, which includes messages for delivering bills and bill detail to customers, as well as messages for getting account information for a group of users

14.7.1.1 Biller Directory Message Set and Messages

14.7.1.1.1 Biller Directory Request Messages

<i>Message Set</i>	<i>Message</i>
<PRESDIRMSGSET> <PRESDIRMSGSETV1> <PRESDIRMSGSRQV1> </PRESDIRMSGSRQV1> </PRESDIRMSGSETV1> </PRESDIRMSGSET>	FINDBILLERTRNRQ FINDBILLERRQ

14.7.1.1.2 Biller Directory Response Messages

<i>Message Set</i>	<i>Message</i>
<PRESDIRMSGSET> <PRESDIRMSGSETV1> <PRESDIRMSGSRSV1> </PRESDIRMSGSRSV1> </PRESDIRMSGSETV1> </PRESDIRMSGSET>	FINDBILLERTRNRS FINDBILLERRS

14.7.1.2 Bill Delivery Message Set and Messages

14.7.1.2.1 Bill Delivery Request Messages

Message Set	Message
<PRESDLVMSGSET>	
<PRESDLVMSGSETV1>	
<PRESDLVMSGSRQV1>	PRESLISTTRNRQ
	PRESLISTRQ
	PRESDETAILTRNRQ
	PRESDETAILRQ
	BILLTBLSTRUCTTRNRQ
	BILLTBLSTRUCTRQ
	BILLSTATUSMODTRNRQ
	BILLSTATUSMODRQ
	PRESNOTIFYTRNRQ
	PRESNOTIFYRQ
	PRESGRPACCTINFOTRNRQ
	ACCTINFORQ
	PRESMAILTRNRQ
	PRESMAILRQ
	PRESMAILSYNCRQ
</PRESDLVMSGSRQV1>	
</PRESDLVMSGSETV1>	
</PRESDLVMSGSET>	

14.7.1.2.2 Bill Delivery Response Messages

<i>Message Set</i>	<i>Message</i>
<PRESDLVMSGSET> <PRESDLVMSGSETV1> <PRESDLVMSGRSV1>	PRESLISTTRNRS PRESLISTRS PRESDETAILTRNRS PRESDETAILRS BILLTBLSTRUCTTRNRS BILLTBLSTRUCTRS BILLSTATUSMODTRNRS BILLSTATUSMODRS PRESNOTIFYTRNRS PRESNOTIFYRS PRESGRPACCTINFOTRNRS ACCTINFORS PRESMAILTRNRS PRESMAILRS PRESMAILSYNCRS
</PRESDLVMSGRSV1> </PRESDLVMSGSETV1> </PRESDLVMSGSET>	

14.7.2 Biller Directory Message Set Profile

This section defines the profile aggregate for the Biller Directory message set. This profile aggregate should be included in the <PROFRS> response for those servers that support the Biller Directory message set.

<i>Message Set</i>	<i>Message</i>
<PRESDIRMSGSET>	Opening tag for the Biller Directory message set profile
<PRESDIRMSGSETV1>	Version 1 of Biller Directory message set, one or more
<MSGSETCORE>	Common message-set core
</MSGSETCORE>	
<PRESDIRPROF>	Directory profile (if supported)
<PROCDAYSOFF>	Days of week that no processing occurs: MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, or SUNDAY. 0 or more <PROCDAYSOFF> can be sent.
<CANSUPPORTIMAGES>	Supports delivery of images as multipart MIME, <i>Boolean</i>
<PROCENDTM>	Time of day that day's processing ends, <i>time</i>
</PRESDIRPROF>	
</PRESDIRMSGSETV1>	
</PRESDIRMSGSET>	Closing tag for the Biller Directory message set profile

14.7.3 Bill Delivery Message Set Profile

This section defines the profile aggregate for the Bill Delivery message set. This profile aggregate should be included in the <PROFRS> response for those servers that support the Bill Delivery message set.

<i>Tag</i>	<i>Version</i>	<i>Description</i>
<PRESDLVMSGSET>		Opening tag for the Bill Delivery message set profile
<PRESDLVMSGSETV1>		Version 1 of Bill Delivery message set, one or more
<MSGSETCORE>		Common message-set core
</MSGSETCORE>		
<PRESDLVPROF>		Bill Delivery profile (if supported)

Tag	Version	Description
<CANSUPPORTGROUPID>	1.6 add	Supports account information requests for a group of users, that is <PRESGRPACCTINFOTRNRQ> and <GROUPID> in <PRESLISTTRNRQ>, <i>Boolean</i>
<PROCDAYSOFF>		Days of week that no processing occurs: MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, or SUNDAY. 0 or more <PROCDAYSOFF> can be sent.
<CANSUPPORTIMAGES>		Supports delivery of images as multipart MIME, <i>Boolean</i>
<PROCENDTM>		Time of day that day's processing ends, <i>time</i>
<CANUPDATEPRESNAMEADDRESS>		Supports update of the PRESNAMEADDRESS associated with a particular bill. See section 14.3.3.1
<CANMODSTATUS>		Y if server supports the <BILLSTATUSMODRQ>. These OFX 1.6 additions must be explicitly supported by the server before they are used by the client. The default for this option is N. <i>Boolean</i> Note: Servers that support <BILLSTATUSMODRQ> are required to continue support for <PRESNOTIFYRQ>.
</PRESDLVPROF>		
<EMAILPROF>		E-mail profile
<CANEMAIL>		Supports generalized e-mail, <i>Boolean</i>
<CANNOTIFY>		Supports notification (of any kind), <i>Boolean</i>
</EMAILPROF>		
</PRESDLVMSGSETV1>		
</PRESDLVMSGSET>		Closing tag for the Bill Delivery message set profile

14.8 Bill Presentment Examples

14.8.1 Find Biller Examples

14.8.1.1 Get All Billers

The client sends a <FINDBILLERRQ> request, as shown in the following example, to retrieve all available billers.

Note: These examples show the customer signing on anonymously. But, they may have enrolled in the past and choose to use their actual <USERID> and <USERPASS>. Anonymous signon is not required for use of the Biller Directory service (see section 14.2.1).

```
<OFX> <!-- Begin request data -->
  <SIGNONMSGSRQV1>
    <SONRQ>                                <!-- Begin anonymous signon -->
      <DTCLIENT>19990707202000 <!-- Jul. 7, 1999, 8:20:00 PM -->
      <USERID>anonymous000000000000000000000000
      <USERPASS>anonymous000000000000000000000000
      <LANGUAGE>ENG                        <!-- Language used for text -->
      <FI>                                <!-- ID of receiving institution -->
        <ORG>NCH                          <!-- Name of ID owner -->
        <FID>1001                         <!-- Actual ID -->
      </FI>
      <APPID>MyApp
      <APPVER>0500
    </SONRQ>                                <!-- End of signon -->
  </SIGNONMSGSRQV1>
  <PRESDIRMSGSRQV1>
    <FINDBILLERTRNRQ>
      <TRNUID>1231239
      <FINDBILLERRQ>                      <!--Beginning of find biller request-->
        <INCIMAGES>N                     <!--LOGO Images are not requested-->
      </FINDBILLERRQ>                      <!--End of request-->
    </FINDBILLERTRNRQ>
  </PRESDIRMSGSRQV1>
</OFX>
```

To keep the size of the example reasonable, we will assume that there are only four billers. Here is the server reply.

```
<OFX> <!-- Begin response data -->
  <SIGNONMSGSRSV1>
    <SONRS>                                <!-- Begin signon -->
      <STATUS>                              <!-- Begin status aggregate -->
        <CODE>0                            <!-- OK -->
        <SEVERITY>INFO
      </STATUS>
      <DTSERVER>19990707202001 <!-- Jul. 7, 1999, 8:20:01 PM -->
      <LANGUAGE>ENG <!-- Language used in response -->
      <DTPROFUP>19990630000000 <!-- Last update to profile -->
      <DTACCTUP>19990701233045 <!-- Last account update -->
    </SONRS>                                <!-- End of signon -->
  </SIGNONMSGSRSV1>
  <PRESDIRMSGSRSV1>
    <FINDBILLERTRNRS>
      <TRNUID>1231239
      <STATUS>                              <!-- Begin status aggregate -->
        <CODE>0 <!-- OK -->
        <SEVERITY>INFO
      </STATUS>
      <FINDBILLERRS>                        <!--Beginning of response-->
        <DTUPDATE>19960415092000
                                         <!--Date last update 04/15/97 9:20am-->
        <BILLERINFO>
          <BILLPUB>Wepubbills <!--Name of Bill Publisher-->
          <BILLERID>123456789 <!--Biller ID at Wepubbills-->
          <NAME>RealBig Credit Co.
                                         <!--Name of biller-->
          <ADDR1>1324 Whatever St.
                                         <!--Street address of biller-->
          <CITY>MajorMetro <!--City of the Biller-->
          <STATE>OH <!--State of the biller-->
          <POSTALCODE>12345-1234
                                         <!--Postal code of biller-->
          <COUNTRY>USA
          <SIC>23 <!--Standard Industry Code of biller-->
          <PHONE>614-235-2323 <!--Biller's phone number-->
          <PAYMENTINSTRUMENTS> <!--Type of payment accepted-->
            <PAYMENTINSTRUMENT>
              <PMTINSTRUMENTTYPE>CHECKINGACCOUNT
```

```

    </PAYMENTINSTRUMENT>
    <PAYMENTINSTRUMENT>
        <PMTINSTRUMENTTYPE>CONCENTRATOR
        <BRAND>CityBank
    </PAYMENTINSTRUMENT>
</PAYMENTINSTRUMENTS>
<ACCTFORMAT>([0-9]\{3\}-)\{3\}
    <!--Regular expression describing -->
    <!--biller's account number-->
<ACCTEDITMASK>###-###-###-
    <!--Edit mask for account number-->
<LOGO>http://www.realbig.com/logo.gif
    <!--URL to logo of biller-->
</BILLERINFO>
<BILLERINFO>
    <BILLPUB>Wepubbills <!--Name of Bill Publisher-->
    <BILLERID>222334465 <!--Biller ID at Wepubbills-->
    <NAME>Aphone Company <!--Name of biller-->
    <ADDR1>1324 Where Blvd
        <!--Street address of biller-->
    <CITY>Sometown <!--City of the biller-->
    <STATE>CA <!--State of the biller-->
    <POSTALCODE>10992-1234
        <!--Postal code of biller-->
    <COUNTRY>USA
    <SIC>39 <!--Standard Industry Code of biller-->
    <PHONE>345-345-3489 <!--Biller's phone number-->
    <PAYMENTINSTRUMENTS> <!--Type of payment accepted-->
        <PAYMENTINSTRUMENT>
            <PMTINSTRUMENTTYPE>CHECKINGACCOUNT
        </PAYMENTINSTRUMENT>
    </PAYMENTINSTRUMENTS>
    <ACCTFORMAT>([1-9]\{2\}-)\{2\}[0-9]\{3\}
        <!--Regular expression describing-->
        <!--biller's account number-->
    <ACCTEDITMASK>##-##-###
        <!--Edit mask for account number-->
    <LOGO>http://www.webup.com/aphone.gif
        <!--URL to logo of biller-->
</BILLERINFO>
<BILLERINFO>
    <BILLPUB>Wepubbills <!--Name of Bill Publisher-->
    <BILLERID>98765123454<!--Biller ID at Wepubbills-->

```

```

<NAME>Goodol Mortgage<!--Name of biller-->
<ADDR1>8273 Magnolia St.
                                <!--Street address of biller-->
<CITY>Atlanta                  <!--City of the Biller-->
<STATE>GA                      <!--State of the biller-->
<POSTALCODE>34342-6789
                                <!--Postal code of biller-->

<COUNTRY>USA
<SIC>03                        <!--Standard Industry Code of biller-->
<PHONE>864-234-6745 <!--Biller's phone number-->
<PAYMENTINSTRUMENTS> <!--Type of payment accepted-->
    <PAYMENTINSTRUMENT>
        <PMTINSTRUMENTTYPE>CHECKINGACCOUNT
    </PAYMENTINSTRUMENT>
</PAYMENTINSTRUMENTS>
<ACCTFORMAT>[0-1]\{12\}
                                <!--Regular expression describing-->
                                <!--biller's account number-->
<HELPMESSAGE>Enter the first 13 digits of your account
number
                                <!--to help user key account number-->
<RESTRICT> GA residents only.
                                <!--Indicate restricted availability-->
<LOGO>http://www.wepub.com/mort.gif
                                <!--URL to logo of biller-->
</BILLERINFO>
<BILLERINFO>
    <BILLPUB>Wepubbills <!--Name of Bill Publisher-->
    <BILLERID>32812816734<!--Biller ID at Wepubbills-->
    <NAME>Sam's Widgets <!--Name of biller-->
    <ADDR1>Apt B3          <!--Street address of biller-->
    <ADDR2>1267 Tank Rd
    <CITY>Columbus         <!--City of Biller-->
    <STATE>OH              <!--State of the biller-->
    <POSTALCODE>77723-8989
                                <!--Postal code of biller-->

    <COUNTRY>USA
    <SIC>12                 <!--Standard Industry Code of biller-->
    <PHONE>614-657-8934 <!--Biller's phone number-->
    <PAYMENTINSTRUMENTS> <!--Type of payment accepted-->
        <PAYMENTINSTRUMENT>
            <PMTINSTRUMENTTYPE>CHECKINGACCOUNT
        </PAYMENTINSTRUMENT>

```

```

        <PAYMENTINSTRUMENT>
            <PMTINSTRUMENTTYPE>CONCENTRATOR
            <BRAND>BigConcentrator
        </PAYMENTINSTRUMENT>
    </PAYMENTINSTRUMENTS>
    <ACCTEDITMASK>A###-####-####
                                <!--Edit mask for account number-->
    <LOGO>http://www.relbilg.com/logo.gif
                                <!--URL to logo of biller-->
    <VALIDATE>http://www.wepub.com/sam.cgi
                                <!--URL used to validate acct number-->

    </BILLERINFO>
    </FINDBILLERRS>
    </FINDBILLERTRNRS>
    </PRESDIRMSGSRV1>
</OFX>

```

14.8.1.2 Find Selected Billers

In the following example, the client requests only those billers that are located in Ohio.

Note: These examples show the customer signing on anonymously. But, they may have enrolled in the past and choose to use their actual <USERID> and <USERPASS>. Anonymous signon is not required for use of the Biller Directory service (see section 14.2.1).

```

<OFX> <!-- Begin request data -->
    <SIGNONMSGSRQV1>
        <SONRQ>                                <!-- Begin anonymous signon -->
            <DTCLIENT>19990707202003 <!-- Jul. 7, 1999, 8:20:03 PM -->
            <USERID>anonymous000000000000000000000000
            <USERPASS>anonymous000000000000000000000000
            <LANGUAGE>ENG                        <!-- Language used for text -->
            <FI>                                <!-- ID of receiving institution -->
                <ORG>NCH                        <!-- Name of ID owner -->
                <FID>1001                       <!-- Actual ID -->
            </FI>
            <APPID>MyApp
            <APPVER>0500
        </SONRQ>                                <!-- End of signon -->
    </SIGNONMSGSRQV1>
    <PRESDIRMSGSRQV1>
        <FINDBILLERTRNRQ>

```

```

    <TRNUID>1231245
    <FINDBILLERRQ>
      <STATE>OH
      <INCIMAGES>N
    </FINDBILLERRQ>
  </FINDBILLERTRNRQ>
</PRESDIRMSGSRQV1>
</OFX>

```

In the same circumstances as before, the response would be:

```

<OFX> <!-- Begin response data -->
  <SIGNONMSGSRSV1>
    <SONRS>                                <!-- Begin signon -->
      <STATUS>                             <!-- Begin status aggregate -->
        <CODE>0                            <!-- OK -->
        <SEVERITY>INFO
      </STATUS>
      <DTSERVER>19990707202004 <!-- Jul. 7, 1999, 8:20:04 PM
      <LANGUAGE>ENG <!-- Language used in response -->
      <DTPROFUP>19990630000000 <!-- Last update to profile -->
      <DTACCTUP>19990701233045 <!-- Last account update -->
    </SONRS>                                <!-- End of signon -->
  </SIGNONMSGSRSV1>
  <PRESDIRMSGSRSV1>
    <FINDBILLERTRNRS>
      <TRNUID>1231245
      <STATUS>                             <!-- Begin status aggregate -->
        <CODE>0 <!-- OK -->
        <SEVERITY>INFO
      </STATUS>
      <FINDBILLERRS>
        <DTUPDATE>19960415092000
                                          <!--Date last update 04/15/97 9:20am-->
        <BILLERINFO>
          <BILLPUB>Wepubbills <!--Name of Bill Publisher-->
          <BILLERID>123456789 <!--Biller ID at Wepubbills-->
          <NAME>RealBig Credit Co.
                                          <!--Name of biller-->
          <ADDR1>1324 Whatever St.
                                          <!--Street address of biller-->
          <CITY>MajorMetro <!--City of the Biller-->

```

```

<STATE>OH                <!--State of the biller-->
<POSTALCODE>12345-1234
                        <!--Postal code of biller-->
<COUNTRY>USA
<SIC>23                  <!--Standard Industry Code of biller-->
<PHONE>614-235-2323 <!--Biller's phone number-->
<PAYMENTINSTRUMENTS> <!--Type of payment accepted-->
    <PAYMENTINSTRUMENT>
        <PMTINSTRUMENTTYPE>CHECKINGACCOUNT
    </PAYMENTINSTRUMENT>
    <PAYMENTINSTRUMENT>
        <PMTINSTRUMENTTYPE>CONCENTRATOR
        <BRAND>CityBank
    </PAYMENTINSTRUMENT>
</PAYMENTINSTRUMENTS>
<ACCTFORMAT>([0-1]\{3\}-)\{3\}
<!--Regular expression describing-->
                        <!--biller's account number-->
<ACCTEDITMASK>###-###-###-
                        <!--Edit mask for account number-->
<LOGO>http://www.relbig.com/logo.gif
                        <!--URL to logo of biller-->

    </BILLERINFO>
</FINDBILLERRS>
</FINDBILLERTRNRS>
</PRESDIRMSGSRSV1>
</OFX>

```

14.8.2 Enrollment Examples

In this example, the client wants to enroll with a bill publisher.

14.8.2.1 Enrollment Request

```
<OFX>
  <SIGNONMSGSRQV2>                                <!--Signon Request-->
    <SONRQ>
      <DTCLIENT>19970307022243 <!--Timestamp, 3/07/97, 2:22:43am-->
      <USERID>anonymous000000000000000000000000
      <USERPASS>anonymous000000000000000000000000
      <LANGUAGE>ENG
      <APPID>0FXAPP
      <APPVER>0201
    </SONRQ>
  </SIGNONMSGSRQV2>
  <SIGNUPMSGSRQV2>                                <!--Enrollment Request-->
    <ENROLLTRNRQ>
      <TRNUID>10001
      <ENROLLRQ>
        <FIRSTNAME>Cindy
        <MIDDLENAME>P
        <LASTNAME>Williams
        <ADDR1>123 Oak St
        <CITY>San Jose
        <STATE>CA
        <POSTALCODE>94111
        <COUNTRY>USA
        <DAYPHONE>415-555-0123
        <EVEPHONE>408-555-2323
        <EMAIL>cindy@aol.com
        <USERID>cindyid
        <TAXID>111-33-5555
        <SECURITYNAME>wynona
        <DATEBIRTH>19650402
      </ENROLLRQ>
    </ENROLLTRNRQ>
  </SIGNUPMSGSRQV2>
</OFX>
```


14.8.2.2 Enrollment Response

For this example, the server responds with immediate acceptance. In practice, many servers would send an enrollment status code of 13000 and send the user ID and password in a welcome letter.

```
<OFX>
  <SIGNONMSGSRSV2>                                <!--Signon response-->
    <SONRS>
      <STATUS>
        <CODE>0
        <SEVERITY>INFO
      </STATUS>
      <DTSERVER>19970307081437 <!--Timestamp, 3/07/97, 8:14:37am-->
      <LANGUAGE>ENG
      <DTPROFUP>19970301070000 <!--Timestamp, 3/01/97, 7:00:00am-->
      <DTACCTUP>19970301070000 <!--Timestamp, 3/01/97, 7:00:00am-->
    </SONRS>
  </SIGNONMSGSRSV2>
  <SIGNUPMSGSRSV2>                                <!--Enrollment response-->
    <ENROLLTRNRS>
      <TRNUID>10001
      <STATUS>
        <CODE>0
        <SEVERITY>INFO
      </STATUS>
      <ENROLLRS>
        <TEMPPASS>y12345
        <USERID>cindyid
        <DTEXPIRE>19970407    <!--When Temp Password Expires-->
      </ENROLLRS>
    </ENROLLTRNRS>
  </SIGNUPMSGSRSV2>
</OFX>
```

14.8.3 Activation Example

After enrollment, Cindy wants to sign up with a biller, presumably found with the directory services, with biller ID 415-552-9923 of bill publisher Publisher, Inc.

14.8.3.1 Activation Request

```
<OFX>
  <SIGNONMSGSRQV2>                                <!--Signon Request-->
    <SONRQ>                                          <!-- ...Sign on request. For a
                                                    complete example, see section
                                                    11.14.1-->

    </SONRQ>
  </SIGNONMSGSRQV2>
  <SIGNUPMSGSRQV2>                                <!--Activation Request-->
    <ACCTTRNRQ>
      <TRNUID>10002
      <ACCTRQ>                                     <!--Activate biller acct -->
      <SVCADD>
        <PRESACCTO>
          <BILLPUB>Publisher, Inc.
          <BILLERID>415-552-9923
          <ACCTID>4128 9343 2324 2314
        <PRESNAMEADDRESS>
          <NAMEACCTHELD>Cindy P Williams<!--Name as on biller's
                                                    statement-->
          <ADDR1>123 Oak St<!--Address as on statement-->
          <CITY>San Jose
          <STATE>CA
          <POSTALCODE>94111
          <COUNTRY>USA
          <DAYPHONE>408-555-2323
        </PRESNAMEADDRESS>
        <USERID>cindyid
      </PRESACCTO>
    </SVCADD>
    <SVC2>PRESSVC
  </ACCTRQ>
</ACCTTRNRQ>
</SIGNUPMSGSRQV2>
</OFX>
```

14.8.3.2 Activation Response

```
<OFX>
  <SIGNONMSGSRSV2>
    <SONRS>
      <!--Signon Response-->
      <!-- ...Sign on response. For a
      complete example, see section
      11.14.1-->
    </SONRS>
  </SIGNONMSGSRSV2>
  <SIGNUPMSGSRSV2>
    <!--Enrollment Response-->
    <!--Service Activation Response-->
    <ACCTTRNRS>
      <TRNUID>10002
      <STATUS>
        <CODE>0
        <SEVERITY>INFO
      </STATUS>
      <ACCTRS>
        <SVCADD>
          <PRESACCTT0>
            <BILLPUB>Publisher, Inc.
            <BILLERID>415-552-9923
            <ACCTID>4128 9343 2324 2314
          <PRESNAMEADDRESS>
            <NAMEACCTHELD>Cindy P Williams
            <ADDR1>123 Oak St
            <CITY>San Jose
            <STATE>CA
            <POSTALCODE>94111
            <COUNTRY>USA
            <DAYPHONE>408-555-2323
          </PRESNAMEADDRESS>
            <USERID>cindyid
          </PRESACCTT0>
        </SVCADD>
        <SVC2>PRESSVC
      </ACCTRS>
    </ACCTTRNRS>
  </SIGNUPMSGSRSV2>
</OFX>
```

14.8.4 Bill Delivery Examples

14.8.4.1 Customer Bill Delivery

The customer, Dan North, wants to see his bills since 3/1/97, which is the last time he asked to see his bills.

14.8.4.1.1 Customer Bill Delivery Request

```
<OFX>
  <SIGNONMSGSRQV2>
    <SONRQ>                                     <!-- ...Sign on request. For a
                                                    complete example, see section
                                                    11.14.1-->

    </SONRQ>
  </SIGNONMSGSRQV2>
  <PRESDLVMSGSRQV1>
    <PRESLISTTRNRQ>
      <TRNUID>12345
      <PRESLISTRQ>
        <BILLPUB> ABillPublisher
        <DTSTART>19970301000000<!--Get Dan's bills since 3/1/97-->
        <NOTIFYWILLING>Y
        <INCLUDEDETAIL>Y
      </PRESLISTRQ>
    </PRESLISTTRNRQ>
  </PRESDLVMSGSRQV1>
</OFX>
```

14.8.4.1.2 Customer Bill Delivery Response

```
<OFX>
  <SIGNONMSGSRSV2>
    <SONRS>                                     <!-- ...Sign on response. For a
                                                    complete example, see section
                                                    11.14.1-->

    </SONRS>
  </SIGNONMSGSRSV2>
  <PRESDLVMSGSRSV1>
    <PRESLISTTRNRS>
      <TRNUID>12345
      <STATUS>
        <CODE>0
        <SEVERITY>INFO
    </PRESLISTTRNRS>
  </PRESDLVMSGSRSV1>
</OFX>
```

```

</STATUS>
<PRESLISTRS>
  <BILLPUB>ABillPublisher
  <USERID>123-45-6789
  <DTSTART>19970301000000<!--Same as in request: no data loss-->
  <DTEND>19970409090000 <!--Value for DTSTART next time-->
  <PRESLIST>
    <PRESBILLINFO>
      <BILLID>65432
      <PRESACCTFROM>
        <BILLPUB>ABillPublisher
        <BILLERID>1001 <!--Biller id for Power Inc-->
        <ACCTID>1245678GL7<!--Dan North's acct w/Power Inc-->
      </PRESACCTFROM>
      <BILLREFINFO>1234678GL7970501
      <AMTDUE>124.24<!--Dan North to pay $124.24-->
      <DTPMTDUE>19970501<!--by 5/1/97 -->
      <DTBILL>19970401
      <NOTIFYDESIRED>N
      <STMNTIMAGE>
        <IMAGEURL>
https://www.Power.com/bills/apr/dannorth.htm?authtoken=65j3ltfm7
        <DTEXPIRE>199704101200
          <!--Must visit url by 4/10/97 12am-->
      </STMNTIMAGE>
      <BILLDETAILTABLE>
        <TABLENAME>usage
        <BILLDETAILTABLETYPE>x_Power_usage
          <!--Power Inc format for usage-->
        <BILLDETAILROW>
          <C>elec<!--Consumable-->
          <C>19970228
            <!--Date meter reading start of period-->
          <C>65543
            <!--Meter reading at start of period-->
          <C>19970328
            <!--Date meter reading end of period-->
          <C>65643<!--Meter reading at end of period-->
          <C>100<!--Difference in meter readings-->
          <C>KWH<!--Units-->
          <C>.8934<!--Rate (price per unit)-->
          <C>89.34<!--Charge -->

```

```

</BILLDETAILROW>
<BILLDETAILROW>
    <C>gas<!--Consumable -->
    <C>19970226
        <!--Date meter reading start of
        period-->
    <C>509843
        <!--Meter reading at start of period-->
    <C>19970327
        <!--Date meter reading end of period-->
    <C>510843<!--Meter reading at end of period->
    <C>1000<!--Difference in meter readings -->
    <C>Therms<!--Units -->
    <C>.02543<!--Rate (price per unit) -->
    <C>25.43<!--Charge -->
</BILLDETAILROW>
</BILLDETAILTABLE>
</PRESBILLINFO>
<PRESBILLINFO>
    <BILLID>65436
    <PRESACCTFROM>
        <BILLPUB>ABillPublisher
        <BILLERID>2021
            <!--Biller id of FluteRental, Inc. -->
        <ACCTID>8765XY95
            <!--Dan North's account number -->
    </PRESACCTFROM>
    <BILLREFINFO>8765XY95970428
    <AMTDUE>16.21<!--Total to be paid -->
    <DTPMTDUE>19970428<!--by 4/28/97 -->
    <DTBILL>19970408
    <NOTIFYDESIRED>N
    <STMNTIMAGE>
        <IMAGEURL>
            https://www.FluteRental.com/95rs3vlx/bill.asp
        <DTEXPIRE>19970601<!--Must visit url by 6/1/97-->
    </STMNTIMAGE>
    <DETAILAVAILABLE>N<!--No structured detail exists-->
</PRESBILLINFO>
</PRESLIST>
</PRESLISTRS>
</PRESLISTTRNRS>

```

```
</PRESDLVMSGSRV1>
</OFX>
```

14.8.4.2 Bill Delivery for Customer Service Representative

This example assumes that Dan North calls Power Inc with a question about his power bill. Power's customer service representative, Maria Smith, uses a similar application and a similar OFX request to see the same bill that Dan sees.

14.8.4.2.1 Bill Delivery Request Example for a Customer Service Representative

```
<OFX>
  <SIGNONMSGSRQV2>
    <SONRQ>                                <!-- ...Sign on request. For a
                                              complete example, see section
                                              11.14.1-->

    </SONRQ>
  </SIGNONMSGSRQV2>
  <PRESDLVMSGSRQV1>
    <PRESLISTTRNRQ>
      <TRNUID>23456
      <USERID>123-45-6789                  <!--Asks for Dan North's bills -->
      <PRESLISTRQ>
        <BILLPUB> ABillPublisher
        <DTSTART>19970330                  <!--Approximate date -->
        <DTEND>1997040410                  <!--Approximate date -->
        <NOTIFYWILLING>N
        <INCLUDEDDETAIL>Y
      </PRESLISTRQ>
    </PRESLISTTRNRQ>
  </PRESDLVMSGSRQV1>
</OFX>
```

14.8.4.2.2 Bill Delivery Response Example for a Customer Service Representative

The response from the server includes the Power Incorporated bill, but not the FluteRental bill. This is because the server decides that Maria Smith's credentials are good enough to see Dan North's Power Inc bill, but not good enough to see anything else.

```
<OFX>
  <SIGNONMSGSRV2>
    <SONRS>                                <!-- ...Sign on response. For a
                                              complete example, see section
                                              11.14.1-->

    </SONRS>
```

```

</SIGNONMSGSRSV2>
<PRESDLVMSGSRSV1>
  <PRESLISTTRNRS>
    <TRNUID>23456
    <STATUS>
      <CODE>0
      <SEVERITY>INFO
    </STATUS>
    <PRESLISTRS>
      <BILLPUB>ABillPublisher
      <USERID>123-45-6789 <!--Dan North's userid, so Dan's bill-->
      <DTSTART>19970328 <!--Same as in request: no data loss-->
      <DTEND>19970409 <!--Same as in request-->
      <PRESLIST>
        <PRESBILLINFO>
          <BILLID>65432
          <PRESACCTFROM>
            <BILLPUB>ABillPublisher
            <BILLERID>1001<!--Biller id for Power Inc-->
            <ACCTID>1245678GL7
            <!--Dan's account with Power Inc-->
            <USERID>123-45-6789<!--Dan North's userid-->
          </PRESACCTFROM>
          <BILLREFINFO>1234678GL7970501
          <AMTDUE>124.24<!--Dan to pay $124.24-->
          <DTPMTDUE>19970501<!--by 5/1/97 -->
          <DTBILL>19970401
          <NOTIFYDESIRED>N
          <STMNTIMAGE>
            <IMAGEURL>
https://www.Power.com/bills/apr/dannorth.htm?authtoken=987ab6gr8y
            <DTEXPIRE>199704111200
            <!--Must visit url by 4/11/97 12am-->
          </STMNTIMAGE>
          <BILLDETAILTABLE>
            <TABLENAME>usage
            <BILLDETAILTABLETYPE>x_Power_usage
            <!--Power Inc format for usage-->
            <BILLDETAILROW>
              <C>elec<!--Consumable-->
              <C>19970228
              <!--Date meter reading start of period-->

```



```

        <C>65543
            <!--Meter reading at start of period-->
        <C>19970328
            <!--Date meter reading end of period-->
        <C>65643
            <!--Meter reading at end of period-->
        <C>100<!--Difference in meter readings-->
        <C>KWH<!--Units-->
        <C>.8934<!--Rate (price per unit)-->
        <C>89.34<!--Charge-->
    </BILLDETAILROW>
    <BILLDETAILROW>
        <C>gas<!--Consumable-->
        <C>19970226
            <!--Date meter reading start of period-->
        <C>509843
            <!--Meter reading at start of period-->
        <C>19970327
            <!--Date meter reading end of period-->
        <C>510843
            <!--Meter reading at end of period-->
        <C>1000<!--Difference in meter readings-->
        <C>Therms<!--Units-->
        <C>.02543<!--Rate (price per unit)-->
        <C>25.43<!--Charge-->
    </BILLDETAILROW>
</BILLDETAILTABLE>
</PRESBILLINFO>
</PRESLIST>
</PRESLISTRS>
</PRESLISTTRNRS>
</PRESDLVMSGSRV1>
</OFX>

```

14.8.4.3 Bill Delivery for a Group of Users

In this example, Realtors Company downloads the phone bills for the employees' office phones by asking the bill publisher to see the bills for the group RealtorsEmployees. The composition of the group RealtorsEmployees has been agreed upon between Realtors Company and the bill publisher; moreover, the bill publisher has agreed to grant Realtors Company access rights to the RealtorsEmployees group. All this took place outside of OFX.

14.8.4.3.1 Bill Delivery Request Example for a Group of Users

```
<OFX>
  <SIGNONMSGSRQV2>
    <SONRQ>                                     <!-- ...Sign on request. For a
                                                complete example, see section
                                                11.14.1-->

    </SONRQ>
  </SIGNONMSGSRQV2>
  <PRESDLVMSGSRQV1>
    <PRESLISTTRNRQ>
      <TRNUID>34567
      <GROUPID>RealtorsEmployees<!--Asks for Employee's phone bills-->
      <PRESLISTRQ>
        <BILLPUB> ABillPublisher
        <DTSTART>19970430      <!--since 4/30/1997-->
        <NOTIFYWILLING>N
        <INCLUDEDETAIL>Y
      </PRESLISTRQ>
    </PRESLISTTRNRQ>
  </PRESDLVMSGSRQV1>
</OFX>
```

14.8.4.3.2 Bill Delivery Response Example for a Group of Users

The response, not shown here, will include several bills each marked with its own <USERID>. The only bills returned will be the employees' phone bills for their office phones, since those bills are the only ones to which Realtor Company has access rights.

14.8.4.4 Group Account Information

This is an example of a client proxy system that is tracking changes to the accounts of a group of users.

14.8.4.4.1 Group Account Information Request

```
<OFX>
  <SIGNONMSGSRQV2>                                <!--Signon Request-->
    <SONRQ>                                          <!-- ...Sign on request. For a
                                                         complete example, see section
                                                         11.14.1-->

    </SONRQ>
  </SIGNONMSGSRQV2>
  <PRESDLVMSGSRQV1>                                <!--Group Account Info Request-->
    <PRESGRPACCTINFOTRNRQ>
      <TRNUID>10001
      <GROUPID>AClientProxysCustomers
                                                         <!--Predefined group of customers-->

      <ACCTINFORQ>
        <DTACCTUP>19970104    <!--Last DTACCTUP received for group-->
      </ACCTINFORQ>
    </PRESGRPACCTINFOTRNRQ>
  </PRESDLVMSGSRQV1>
</OFX>
```

14.8.4.4.2 Group Account Information Response

```
<OFX>
  <SIGNONMSGSRSV2>                                <!--Signon Response-->
    <SONRS>                                          <!-- ...Sign on response. For a
                                                         complete example, see section
                                                         11.14.1-->

    </SONRS>
  </SIGNONMSGSRSV2>
  <PRESDLVMSGSRSV1>                                <!--Group Account Info Response-->
    <PRESGRPACCTINFOTRNRS>
      <TRNUID>10001
      <STATUS>
        <CODE>0
        <SEVERITY>INFO
      </STATUS>
      <ACCTINFORS>
        <DTACCTUP>19970122092431
        <ACCTINFO>
```

```

    <PRESACCTINFO>
      <PRESACCTFROM>
        <BILLPUB>PUBLISHER, INC.
        <BILLERID>415-552-9923
        <ACCTID>408-555-4342-M132
        <USERID>bygeorge
        <!--User from group with new status-->
      </PRESACCTFROM>
      <SVCSTATUS2>ACTIVE
    </PRESACCTINFO>
  </ACCTINFO>
</ACCTINFORS>
<ACCTINFORS>
  <DTACCTUP>19970123082423
  <ACCTINFO>
    <PRESACCTINFO>
      <PRESACCTFROM>
        <BILLPUB>PUBLISHER, INC.
        <BILLERID>415-552-9923
        <ACCTID>408-555-2341-U421
        <USERID>132-42-5242
        <!--User from group with new status-->
      </PRESACCTFROM>
      <SVCSTATUS2>REJECTED
      <REASON>ACCOUNT NOT FOUND
      <!--User supplied account with biller-->
      <!--didn't match biller's records-->
    </PRESACCTINFO>
  </ACCTINFO>
</ACCTINFORS>
</PRESGRPACCTINFOTRNR>
</PRESDLVMSGSRV1>
</OFX>

```

APPENDIX A STATUS CODES

The following table provides a complete list of the status codes that can be returned by a server. For each status code, the table includes the following information:

- ◆ Number of the status code
- ◆ Meaning of the status code
- ◆ Conditions under which a server must return the status code
- ◆ OFX Version that supports this code. A code added in a later version of OFX (for example, 1.6) should not be sent to a client that supports only an earlier version of OFX (for example, 1.0.2) without (otherwise necessary) text in the <MESSAGE> element describing the error condition.

Note: If a server must send an unsupported message to an earlier client, it should include a <MESSAGE> element describing the general error.

<i>Code</i>	<i>Meaning</i>	<i>Condition</i>	<i>Version</i>
Code	Meaning	Condition	
0	Success (INFO)	The server successfully processed the request.	
1	Client is up-to-date (INFO)	Based on the client timestamp, the client has the latest information. The response does not supply any additional information.	
2000	General error (ERROR)	Error other than those specified by the remaining error codes. Note: Servers should provide a more specific error whenever possible. Error code 2000 should be reserved for cases in which a more specific code is not available.	
2001	Invalid account (ERROR)		
2002	General account error (ERROR)	Account error not specified by the remaining error codes.	
2003	Account not found (ERROR)	The specified account number does not correspond to one of the user's accounts.	
2004	Account closed (ERROR)	The specified account number corresponds to an account that has been closed.	
2005	Account not authorized (ERROR)	The user is not authorized to perform this action on the account, or the server does not allow this type of action to be performed on the account.	

Code	Meaning	Condition	Version
2006	Source account not found (ERROR)	The specified account number does not correspond to one of the user's accounts.	
2007	Source account closed (ERROR)	The specified account number corresponds to an account that has been closed.	
2008	Source account not authorized (ERROR)	The user is not authorized to perform this action on the account, or the server does not allow this type of action to be performed on the account.	
2009	Destination account not found (ERROR)	The specified account number does not correspond to one of the user's accounts.	
2010	Destination account closed (ERROR)	The specified account number corresponds to an account that has been closed.	
2011	Destination account not authorized (ERROR)	The user is not authorized to perform this action on the account, or the server does not allow this type of action to be performed on the account.	
2012	Invalid amount (ERROR)	The specified amount is not valid for this action; for example, the user specified a negative payment amount.	
2014	Date too soon (ERROR)	The server cannot process the requested action by the date specified by the user.	
2015	Date too far in future (ERROR)	The server cannot accept requests for an action that far in the future.	
2016	Transaction already committed (ERROR)	Transaction has entered the processing loop and cannot be modified /cancelled using OFX. The transaction may still be cancelled or modified using other means (for example, a phone call to Customer Service).	
2017	Already canceled (ERROR)	The transaction cannot be canceled or modified because it has already been canceled.	
2018	Unknown server ID (ERROR)	The specified server ID does not exist or no longer exists.	
2019	Duplicate request (ERROR)	A request with this <TRNUID> has already been received and processed.	
2020	Invalid date (ERROR)	The specified datetime stamp cannot be parsed; for instance, the datetime stamp specifies 25:00 hours.	

Code	Meaning	Condition	Version
2021	Unsupported version (ERROR)	The server does not support the requested version. The version of the message set specified by the client is not supported by this server.	
2022	Invalid TAN (ERROR)	The server was unable to validate the TAN sent in the request.	
2023	Unknown FITID (ERROR) [BILLID not found (ERROR) in the billing message sets]	The specified FITID / BILLID does not exist or no longer exists.	1.5 add
2025	Branch ID missing (ERROR)	A <BRANCHID> value must be provided in the <BANKACCTFROM> aggregate for this country system, but this field is missing.	1.5 add
2026	Bank name doesn't match bank ID (ERROR)	The value of <BANKNAME> in the <EXTBANKACCTTO> aggregate is inconsistent with the value of <BANKID> in the <BANKACCTTO> aggregate.	1.5 add
2027	Invalid date range (ERROR)	Response for non-overlapping dates, date ranges in the future, et cetera.	1.6 add
2028	Requested element unknown (WARNING)	One or more elements of the request were not recognized by the server or the server (as noted in the FI Profile) does not support the elements. The server executed the element transactions it understood and supported. For example, the request file included private tags in a <PMTRQ> but the server was able to execute the rest of the request.	1.6 add
6500	<REJECTIFMISSING>Y invalid without <TOKEN> (ERROR)	This error code may appear in the <SYNCERROR> element of an <xxxSYNCRS> wrapper (in <PRESDLVMSGSRV1> and V2 message set responses) or the <CODE> contained in any embedded transaction wrappers within a sync response. The corresponding sync request wrapper included <REJECTIFMISSING>Y with <REFRESH>Y or <TOKENONLY>Y, which is illegal.	1.6 add

Code	Meaning	Condition	Version
6501	Embedded transactions in request failed to process: Out of date (WARNING)	<REJECTIFMISSING>Y and embedded transactions appeared in the request sync wrapper and the provided <TOKEN> was out of date. This code should be used in the <SYNCERROR> of the response sync wrapper.	1.6 add
6502	Unable to process embedded transaction due to out-of-date <TOKEN> (ERROR)	Used in response transaction wrapper for embedded transactions when <SYNCERROR>6501 appears in the surrounding sync wrapper.	1.6 add
10000	Stop check in process (INFO)	Stop check is already in process.	
10500	Too many checks to process (ERROR)	The stop-payment request <STPCHKRQ> specifies too many checks.	
10501	Invalid payee (ERROR)	Payee error not specified by the remaining error codes.	
10502	Invalid payee address (ERROR)	Some portion of the payee's address is incorrect or unknown.	
10503	Invalid payee account number (ERROR)	The account number <PAYACCT> of the requested payee is invalid.	
10504	Insufficient funds (ERROR)	The server cannot process the request because the specified account does not have enough funds.	
10505	Cannot modify element (ERROR)	The server does not allow modifications to one or more values in a modification request.	
10506	Cannot modify source account (ERROR)	Reserved for future use.	
10507	Cannot modify destination account (ERROR)	Reserved for future use.	
10508	Invalid frequency (ERROR)	The specified frequency <FREQ> does not match one of the accepted frequencies for recurring transactions.	
10509	Model already canceled (ERROR)	The server has already canceled the specified recurring model.	
10510	Invalid payee ID (ERROR)	The specified payee ID does not exist or no longer exists.	
10511	Invalid payee city (ERROR)	The specified city is incorrect or unknown.	

Code	Meaning	Condition	Version
10512	Invalid payee state (ERROR)	The specified state is incorrect or unknown.	
10513	Invalid payee postal code (ERROR)	The specified postal code is incorrect or unknown.	
10514	Transaction already processed (ERROR)	Transaction has already been sent or date due is past	
10515	Payee not modifiable by client (ERROR)	The server does not allow clients to change payee information.	
10516	Wire beneficiary invalid (ERROR)	The specified wire beneficiary does not exist or no longer exists.	
10517	Invalid payee name (ERROR)	The server does not recognize the specified payee name.	
10518	Unknown model ID (ERROR)	The specified model ID does not exist or no longer exists.	
10519	Invalid payee list ID (ERROR)	The specified payee list ID does not exist or no longer exists.	
10520	Payment type not supported (ERROR)	The value of <PMTTYPE> in the <PMTINFO2> aggregate is not supported by this server.	V2 only
10521	Transaction requires <CCMOTOACCT> (ERROR)	Specified source account cannot be utilized without additional material provided by the <CCMOTOACCT> aggregate.	1.6 add
10522	Insufficient information included in <CCMOTOACCT> (ERROR)	Provided <CCMOTOACCT> does not include data required to process the payment. <MESSAGE> should describe what is lacking.	1.6 add
10600	Table type not found (ERROR)	The specified table type is not recognized or does not exist.	1.5 add
12250	Investment transaction download not supported (WARN)	The server does not support investment transaction download.	
12251	Investment position download not supported (WARN)	The server does not support investment position download.	
12252	Investment positions for specified date not available (WARN)	The server does not support investment positions for the specified date.	

Code	Meaning	Condition	Version
12253	Investment open order download not supported (WARN)	The server does not support open order download.	1.6 add
12254	Investment balances download not supported (WARN)	The server does not support investment balances download.	
12500	One or more securities not found (ERROR)	The server could not find the requested securities.	
13000	User ID & password will be sent out-of-band (INFO)	The server will send the user ID and password via postal mail, e-mail, or another means. The accompanying message will provide details.	
13500	Unable to enroll user (ERROR)	The server could not enroll the user.	
13501	User already enrolled (ERROR)	The server has already enrolled the user.	
13502	Invalid service (ERROR)	The server does not support the service <SVC> specified in the service-activation request.	
13503	Cannot change user information (ERROR)	The server does not support the <CHGUSERINFORQ> request.	
13504	<FI> Missing or Invalid in <SONRQ> (ERROR)	The FI requires the client to provide the <FI> aggregate in the <SONRQ> request, but either none was provided, or the one provided was invalid.	
15000	Must change USERPASS (INFO)	The user must change his or her <USERPASS> number as part of the next OFX request.	
15500	Signon invalid (ERROR)	The user cannot signon because he or she entered an invalid user ID or password.	
15501	Customer account already in use (ERROR)	The server allows only one connection at a time, and another user is already signed on. Please try again later.	
15502	USERPASS lockout (ERROR)	The server has received too many failed signon attempts for this user. Please call the FI's technical support number.	
15503	Could not change USERPASS (ERROR)	The server does not support the <PINCHRRQ> request.	
15504	Could not provide random data (ERROR)	The server could not generate random data as requested by the <CHALLENGERQ>.	

Code	Meaning	Condition	Version
15505	Country system not supported (ERROR)	The server does not support the country specified in the <COUNTRY> field of the <SONRQ> aggregate.	V2 only
15506	Empty signon not supported (ERROR)	The server does not support signons not accompanied by some other transaction.	1.5 add
15507	Signon invalid without supporting pin change request (ERROR)	The OFX block associated with the signon does not contain a pin change request and should.	1.5 add
15508	Transaction not authorized. (ERROR)	Current user is not authorized to perform this action on behalf of the <USERID>.	1.6 add
16500	HTML not allowed (ERROR)	The server does not accept HTML formatting in the request.	
16501	Unknown mail To: (ERROR)	The server was unable to send mail to the specified Internet address.	
16502	Invalid URL (ERROR)	The server could not parse the URL.	
16503	Unable to get URL (ERROR)	The server was unable to retrieve the information at this URL (e.g., an HTTP 400 or 500 series error).	

APPENDIX B CHANGE HISTORY

This appendix lists changes made to OFX as the specification has been developed.

- ◆ Section [B.1](#) covers changes from [OFX 1.0 to 1.0.1](#)
- ◆ Section [B.2](#) covers changes from [OFX 1.0.1 to 1.0.2](#)
- ◆ Section [B.3](#) covers changes from [OFX 1.0.2 to 1.5](#)
- ◆ Section [B.4](#) covers changes from [OFX 1.5 to OFX 1.5.1](#)
- ◆ Section [B.5](#) covers changes from [OFX 1.5.1 to 1.6](#)

B.1 OFX 1.0 to 1.0.1

This section describes the revisions to Open Financial Exchange that occurred between version 1.0 and version 1.0.1. These revisions are grouped into the following categories:

- ◆ Revisions to the specification text, organized by chapter
- ◆ General revisions to the specification text
- ◆ Changes to the Document Type Definition (DTD) files

B.1.1 Specification Changes by Chapter

B.1.1.1 Chapter 1, “Overview”

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
1.2.1	Header examples, SECURITY tag	Correction	Changed “SECURITY:1” to “SECURITY:Type1”.

B.1.1.2 Chapter 2, “Structure”

B.1.1.2.1 Sections 2.2 to 2.4.4

Section	Subject	Change Type	Change
2.1	HTTP headers, errors	Clarification	The server must return code 400 for any problem that prevents it from processing the request file. Processing problems include failures relating to security, communication, parsing, or the Open Financial Exchange headers (for example, the client requested an unsupported language).
2.2, 2.2.1 to 2.2.7	Headers	Clarification	Defined values for the Open Financial Exchange headers.
2.3.1	SGML compliance	Correction	Changed “complaint” to “compliant.”
2.3.2	SGML values	Clarification	Clarified acceptable SGML values.
2.3.3	Comments	Addition	For explanatory purposes, the examples in the specification contain comments. However, Open Financial Exchange files <i>cannot</i> contain comments.
2.4.3	Messages, transaction wrappers	Clarification	For requests, the transaction wrapper adds a transaction unique ID <TRNUID>. For responses, the transaction wrapper adds the same transaction unique ID <TRNUID>, plus a <STATUS> aggregate.
2.4.4	Message sets	Correction	Added missing “rq” and “rs” for <xxxMSGSRQVn> and <xxxMSGSRSVn>. For each message set of xxx and version n, there are two aggregates – one for requests (<xxxMSGSRQVn>) and one for responses (<xxxMSGSRSVn>).

B.1.1.2.2 Sections 2.5.1 to 2.6

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
2.5.1	<SONRQ>, user IDs in signon requests	Clarification	Servers must accept user IDs, with or without punctuation.
	<SONRS>, server response to signon failure	Clarification	If the server returns any signon error, it must respond to all other requests in the same <OFX> block with status code 15500. The server must return status code 15500 to all requests; it cannot simply ignore the requests.
2.5.1.1	<SONRQ>	Correction	Moved <SESSCOOKIE> after <FI> aggregate.
		Correction	Choose either <USERID> and <USERPASS> or <USERKEY>.
2.5.1.2	<SONRS>	Clarification	Updated element sizes.
2.5.2.1	<PINCHRQ>	Clarification	Clarified scope of the PIN change.
2.5.3	<SIGNONMSGSET>	Addition	Added description of the signon message set.
2.6	<GETMIMERS> example	Correction	Changed zzz.html to zzz.jpg.
2.7	Extensions to Open Financial Exchange	Clarification	End tags are required for all aggregates and elements introduced by an organization. For example, <ABC.SOMETHING> must be followed by </ABC.SOMETHING>.

B.1.1.3 Chapter 3, “Common Aggregates, Elements, and Data Types”

B.1.1.3.3 Sections 3.1.2 to 3.2.3

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
3.1.2	User-supplied numbers	Addition	Clients will not attempt to strip dashes or other punctuation from user-supplied numbers, such as the <TAXID> in an enrollment request or the <xxxACCTTO> in a service-addition request. Servers must be prepared to accept these numbers with or without punctuation.
3.1.3	<BAL>	Correction	Balance <NAME> is of type A-32.
		Correction	Moved <DTASOF> before <CURRENCY> aggregate.
		Correction	Changed format of DOLLAR to DDDD.cc.
3.1.4	<STATUS>	Clarification	<MESSAGE> element is of type A-255.
		Addition	Added status code 15500 to the list of general errors that a server can return.
3.2.3	<TRNUID> element	Clarification	In most cases, clients originate transactions. When a client originates a <TRNUID> for a transaction, the value of the <TRNUID> is always set to a unique identifier. When the server originates a transaction, the value of the <TRNUID> must be set to zero.

B.1.1.3.4 Sections 3.3.1 to 3.3.4

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
3.3.1, 3.3.1.1, 3.3.1.2, 3.3.1.3	Dates and times	Clarification	Clarified format of <i>date</i> , <i>datetime</i> , and <i>time</i> .
3.3.2	Amounts, prices, and quantities	Clarification	Clarified size of types, implied decimal point if none is specified, and server error code for unsupported size or precision.
3.3.2.1	Amounts	Correction	Amounts that do not represent whole numbers (for example, 540.32), must include a decimal point or comma to indicate the start of the fractional amount. Amounts should not include any punctuation separating thousands, millions, and so forth.
3.3.4	<i>currSymbol</i> type	Addition	Added <i>currSymbol</i> type, a three-letter code that identifies the currency being used for a request or response. The currency codes are based on ISO-4217.
	URL	Clarification	Specified size of <i>URL</i> type as A-255.

B.1.1.4 Chapter 5, “International Support”

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
5.1	Language and encoding	Clarification	Servers must respond with the encoding, character set, and language requested by the client.
5.2	Currency aggregate, <CURRATE> element	Clarification	<CURRATE> element is of type <i>rate</i> .

B.1.1.5 Chapter 6, “Data Synchronization”

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
6.4	<LOSTSYNC>	Correction	Synchronization responses can optionally include the <LOSTSYNC> element.
6.6	Client synchronization requests	Clarification	Synchronization requests can contain <TOKEN>, <TOKENONLY>, or <REFRESH>.
6.10.1	Lite synchronization, OLDFILEUID and NEWFILEUID	Clarification	Clarified role of OLDFILEUID and NEWFILEUID in file-based error recovery.
6.11	Synchronization example	Addition	Added <REJECTIFMISSING>N after <TOKEN>.
		Correction	Moved <TRNUID> before <STATUS> aggregate in synchronization response.

B.1.1.6 Chapter 7, “FI Profile”

Section	Subject	Change Type	Change
7.1.3	Batching and routing	Clarification	Signon message set is an exception to the batching and routing rule. See section 7.1.3.
7.1.4	Client signon in profile requests	Clarification	Clarified values for the first signon request (before the user has a valid user ID and password). Described profile to be returned before and after user enrollment.
7.1.5	<PROFRQ>	Clarification	Specified type of <DTPROFUP> element as <i>datetime</i> .
7.2	<PROFRS>	Clarification	Clarified contents of <PROFRS> depending on whether the client has the most up-to-date profile.
		Correction	Moved <COUNTRY> after <POSTALCODE>.
	Element sizes	Clarification	Clarified size of elements
7.2.1	<MSGSETCORE>	Clarification	Clarified size and definition of elements.
7.2.2	<SIGNONINFO>	Clarification	Clarified size and definition of elements.
		Addition and deletion	Replaced <ALPHA> and <NUMERIC> with <CHARTYPE>.
7.2.3	Profile-response status codes	Addition	Added status code 1, “Client is up-to-date.”

B.1.1.7 Chapter 8, “Activation & Account Information”

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
8.4.1	User IDs in signon requests	Clarification	Servers must accept user IDs with or without punctuation.
8.4.2	<ENROLLRQ>	Clarification	Clarified size and definition of elements.
8.4.3	<ENROLLRS>	Clarification	Clarified size and definition of elements.
8.5.1	<ACCTINFORQ>	Clarification	Clarified size and definition of elements.
		Correction	Made <DTACCTUP> required.
		Deletion	Deleted <INCIMAGES> element.
8.5.2	<ACCTINFORS>	Clarification	Clarified size and definition of elements.
8.5.3	<ACCTINFO>	Clarification	Clarified size and definition of elements.
		Deletion	Deleted <LOGO> element.
8.5.4	Account information status codes	Addition	Added status code 1, “Client is up-to-date.”
		Deletion	Deleted status code 13001, “No change since supplied <DTACCTUP>”.
8.5.5	Account information example	Deletion	Deleted <INCIMAGES> and <LOGO> elements from example.
8.6.1.1	<ACCTRQ>	Clarification	Clarified size and definition of elements.
8.6.1.2	<ACCTRS>	Addition	Added <SVCSTATUS> element.
8.6.1.3	<SVCADD>	Correction	Changed <ACCTTO> to <xxxACCTTO>.
8.6.1.4	<SVCCHG>	Correction	Changed <ACCTTO> to <xxxACCTTO>.
		Correction	Changed <ACCTFROM> to <xxxACCTFROM>.
8.6.1.5	<SVCDEL>	Correction	Changed <ACCTTO> to <xxxACCTTO>.
			Changed <ACCTFROM> to <xxxACCTFROM>
8.6.3	Service activation example	Addition	Added <SVCSTATUS> element to example.
8.7.1	<CHGUSERINFORQ>	Clarification	Clarified size and definition of elements.
8.7.2	<CHGUSERINFORS>	Clarification	Clarified size and definition of elements.
		Addition	Added <ADDR3> element.
8.8	<SIGNUPMSGSET>	Clarification	Clarified size and definition of elements.

Section	Subject	Change Type	Change
		Correction	<WEBENROLL> is not required.
		Addition	Added <OTHERENROLL> aggregate.
		Addition	Added <CLIENTACTREQ> element.

B.1.1.8 Chapter 9, “Customer to FI Communication”

Section	Subject	Change Type	Change
9.2	E-mail messages	Correction	The message body can be HTML-encoded text or plain text.
9.2.2	<MAIL>	Clarification	Clarified size and definition of elements.
9.2.2.1	<INCIMAGES>	Addition	Described use of <INCIMAGES> in requests and responses.
9.2.2.2	<USEHTML>	Addition	Described use of <USEHTML> in requests and responses.
9.2.4	<MAILSYNCRQ> and <MAILSYNCRS>	Clarification	Clarified size and definition of elements.
	<MAILSYNCRQ>	Addition	Added <TOKENONLY>, <REFRESH>, <REJECTIFMISSING>, <INCIMAGES>, <USEHTML>, and <MAILTRNRQ>.
9.2.5	E-mail example	Correction	Revised example to show the use of e-mail synchronization
9.2.5.1	Example of synchronization request	Addition	Added <REJECTIFMISSING>N, <INCIMAGES>N, and <USEHTML>Y after <TOKEN>.
9.3.2	Multi-part MIME example	Correction	Changed SECURITY:1 to SECURITY:TYPE1.
9.4	<EMAILMSGSET>	Correction	Changed <EMAIL> to <MAILSUP>.
		Correction	Changed <GETMIME> to <GETMIMESUP>.
		Clarification	Clarified size and definition of elements.

B.1.1.9 Chapter 10, “Recurring Transactions”

Section	Subject	Change Type	Change
10.2	<RECURRINST>	Correction	Moved <NINSTS> element before <FREQ>.
10.2.1	<FREQ> element	Deletion	Deleted TRIANNUALLY value.
10.2.2	Repeating payment example	Correction	Moved <NINSTS> element before <FREQ> in the request and response.
10.5.1	Example of synchronization request	Addition	Added <REJECTIFMISSING>N after <TOKEN>.

B.1.1.10 Chapter 11, “Banking”

B.1.1.10.5 Sections 11.2.1.1 to 11.8.3

Section	Subject	Change Type	Change
11.2.1.1	<STPCHKSYNCRQ>	Addition	Added <TOKENONLY> and <REFRESH>.
11.3.1	<BANKACCTFROM>	Clarification	Clarified size and definition of elements.
11.3.2	<CCACCTFROM>	Clarification	Clarified size and definition of elements.
11.3.3	<BANKACCTINFO>	Clarification	Clarified size and definition of elements.
11.4.1.1	<STMTRQ>	Clarification	Clarified size and definition of elements.
11.4.1.2	<STMTRS>	Clarification	<CURDEF> is of type <i>currsymbol</i> .
11.4.2.1	<CCSTMTRQ>	Clarification	<CURDEF> is of type <i>currsymbol</i> .
11.4.2.3.1	<STMTRN>	Clarification	Clarified size and definition of elements.
11.5.1.2	<STMENDRS>	Clarification	<CURDEF> is of type <i>currsymbol</i> .
11.5.2	<CLOSING>	Clarification	Clarified size and definition of elements.
11.5.4	<CCSTMENDRS>	Clarification	<CURDEF> is of type <i>currsymbol</i> .
11.6.1.1	<STPCHKRQ>	Correction	Changed type of <CHKNUMSTART> and <CHKNUMEND> to A-12.
11.6.1.2	<STPCHKRS>	Clarification	<CURDEF> is of type <i>currsymbol</i> .
11.6.1.2.1	<STPCHKNUM>	Clarification	Clarified size and definition of elements.
11.7.1.2	<INTRARS>	Addition	Added <XFERPRCSTS> aggregate.
		Clarification	Clarified size and definition of elements.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
11.7.2	<INTRAMODRQ>	Correction	When modifying a transfer, the client must specify all of the tags within the <XFERINFO> aggregate that were specified when the transfer was created, not just the tags that the client wants to modify.
11.8.2.2	<INTERRS>	Addition	Added <XFERPRCSTS> aggregate.
		Clarification	Clarified size and definition of elements.
11.8.3	<INTERMODRQ>	Correction	When modifying a transfer, the client must specify all of the tags within the <XFERINFO> aggregate that were specified when the transfer was created, not just the tags that the client wants to modify.

B.1.1.10.6 Sections 11.9.1.1 to 11.10.6.2

Section	Subject	Change Type	Change
11.9.1.1	<WIRERQ>	Clarification Correction	Clarified size and definition of elements. Made <WIREDESTBANK> aggregate optional.
11.9.1.1.1	<WIREBENEFICIARY>	Clarification	Clarified size and definition of elements.
11.9.1.1.2	<EXTBANKDESC>	Clarification	Clarified size and definition of elements.
11.9.1.2	<WIRERS>	Clarification	Clarified size and definition of elements.
11.10.2.1	<MODPENDING>	Clarification	If the client sets this flag, the server must modify pending and future transfers.
11.10.2.2	<MODPENDING>	Clarification	Y if client requested that the server modify pending and future transfers. N if the client did not request that the server modify pending and future transfers. <i>Boolean</i>
11.10.3.1	<CANPENDING>	Clarification	If the client sets this flag, the server must cancel pending and future transfers.
11.10.3.2	<CANPENDING>	Clarification	Y if the client requested that the server cancel pending and future transfers. N if the client did not request that the server cancel pending and future transfers.
11.10.5.1	<MODPENDING>	Clarification	If the client sets this flag, the server must modify pending and future transfers.
11.10.5.2	<MODPENDING>	Clarification	Y if client requested that the server modify pending and future transfers. N if the client did not request that the server modify pending and future transfers. <i>Boolean</i>
11.10.5.1	<RECINTERMODRQ>	Clarification	Clarified size and definition of elements.
11.10.6.1	<CANPENDING>	Clarification	If the client sets this flag, the server must cancel pending and future transfers.
11.10.6.2	<CANPENDING>	Clarification	Y if the client requested that the server cancel pending and future transfers. N if the client did not request that the server cancel pending and future transfers.

B.1.1.10.7 Sections 11.12.1.1 to 11.12.7.2

Section	Subject	Change Type	Change
11.12.1.1	<STPCHKSYNCRQ>	Clarification	Clarified size and definition of elements.
		Addition	Added <TOKENONLY>, <REFRESH>, and <REJECTIFMISSING>.
11.12.1.2	<STPCHKSYNCRS>	Clarification	Clarified size and definition of elements.
11.12.2.1	<INTRASYNCRQ>	Clarification	Clarified size and definition of elements.
		Addition	Added <TOKENONLY>, <REFRESH>, and <REJECTIFMISSING>.
11.12.2.2	<INTRASYNCRS>	Clarification	Clarified size and definition of elements.
11.12.3.1	<INTERSYNCRQ>	Clarification	Clarified size and definition of elements.
		Addition	Added <TOKENONLY>, <REFRESH>, and <REJECTIFMISSING>.
11.12.3.2	<INTERSYNCRS>	Clarification	Clarified size and definition of elements.
11.12.4.1	<WIRESYNCRQ>	Clarification	Clarified size and definition of elements.
		Addition	Added <TOKENONLY>, <REFRESH>, and <REJECTIFMISSING>.
11.12.4.2	<WIRESYNCRS>	Clarification	Clarified size and definition of elements.
11.12.5.1	<RECINTRASYNCRQ>	Clarification	Clarified size and definition of elements.
		Addition	Added <TOKENONLY>, <REFRESH>, and <REJECTIFMISSING>.
11.12.5.2	<RECINTRASYNCRS>	Clarification	Clarified size and definition of elements.
11.12.6.1	<RECINTERSYNCRQ>	Clarification	Clarified size and definition of elements.
		Addition	Added <TOKENONLY>, <REFRESH>, and <REJECTIFMISSING>.
11.12.6.2	<RECINTERSYNCRS>	Clarification	Clarified size and definition of elements.
11.12.7.1	<BANKMAILSYNCRQ>	Clarification	Clarified size and definition of elements.
		Addition	Added <REJECTIFMISSING>, <INCIMAGES>, and <USEHTML> elements.
		Correction	The account-from aggregate can be either <BANKACCTFROM> or <CCACCTFROM>.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
11.12.7.2	<BANKMAILSYNCRS>	Clarification Correction	Clarified size and definition of elements. The account-from aggregate can be either <BANKACCTFROM> or <CCACCTFROM>.

B.1.1.10.8 Sections 11.13.1to 11.14.4

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
11.13.1, 11.13.1.1, 11.13.1.2, 11.13.1.3, 11.13.1.4	Bank message sets and messages	Clarification	Updated tables to identify the message set aggregates and corresponding messages.
11.13.2	<BANKMSGSET>	Correction	Made <INVALIDACCTTYPE>, <PROC DAYSOFF>, <XFERPROF>, and <STPCHKPROF> optional.
11.13.4	<INTERXFERMSGSET>	Correction	Made <PROC DAYSOFF> optional.
11.13.5	<WIREXFERMSGSET>	Correction	Made <PROC DAYSOFF> optional.
11.14.1	Example	Correction	Changed <BALAMT>> to <BALAMT>.
11.14.2	Recurring transfer response	Correction	Changed <CURDEF>ENG to <CURDEF>USD.
11.14.3	Example	Correction	Changed <CHKSTATUS G>0 to <CHKSTATUS>0.
11.14.4	Example of synchronization request	Addition	Added <FEE> and <FEEMSG> elements.
		Addition	Added <REJECTIFMISSING>N after <TOKEN>.

B.1.1.11 Chapter 12, “Payments”

B.1.1.11.9 Sections 12.5.1 to 12.8.2.2

Section	Subject	Change Type	Change
12.5.1	<BPACCTINFO>	Clarification	Clarified values of <SVCSTATUS> element.
12.5.2	<PMTINFO>	Clarification	Clarified size and definition of elements.
12.5.2.1	<PAYEE>	Clarification	Clarified size and definition of elements.
12.5.2.2	<EXTDPMT>	Additions	Added <EXTDPMTFOR>, <EXTDPMTCHK>, and <ADJDATE>.
		Correction	<EXTDPMT> can contain one or both of the following aggregates: <EXTDPMTINV> and <EXTDPMTDSC>.
		Clarification	Clarified size and definition of elements.
12.5.3.6	<EXTDPAYEE>	Clarification	Clarified size and definition of elements.
12.5.3.7	<PMTPRCSTS>	Clarification	Clarified size and definition of elements.
		Correction	Made <PMTPRCCODE> and <DTPMTPRC> required.
12.6.1.2	<PMTRS>	Addition	Added note regarding payee synchronization.
		Clarification	Clarified size and definition of elements.
12.6.2	<PMTMODRQ>	Correction	When modifying a payment, the client must specify all of the tags within the <PMTINFO> aggregate that were specified during the payment creation, not just the tags that it wants to modify.
12.6.2.3	<PMTMODRS>	Addition	If the <PMTMODRQ> created a new payee, the server must create and store a payee response that would be available for a payee synchronization request.
12.6.4.2	<PMTINQRS>	Clarification	Clarified size and definition of elements.
12.7.1.1	<RECPMTRQ>	Clarification	Clarified size and definition of elements.
12.7.1.2	<RECPMTRS>	Clarification	Clarified size and definition of elements.
12.7.2.1	<RECPMTMODRQ>	Clarification	Clarified size and definition of elements.
12.7.2.2	<RECPMTMODRS>	Clarification	Clarified size and definition of elements.
12.7.3.1	<RECPMTCANCRQ>	Clarification	Clarified size and definition of elements.
12.7.3.2	<RECPMTCANCRS>	Clarification	Clarified size and definition of elements.

Section	Subject	Change Type	Change
12.8.1.1	<PMTMAILRQ>	Clarification	Clarified size and definition of elements.
12.8.1.2	<PMTMAILRS>	Clarification	Clarified size and definition of elements.
12.8.2, 12.8.2.1, 12.8.2.2	Payment mail synchronization	Addition	Added description of <PMTMAILSYNCRQ> and <PMTMAILSYNCRS> aggregates.

B.1.1.11.10 Sections 12.9.1.1 to 12.12.9

Section	Subject	Change Type	Change
12.9.1.1	<PAYEERQ>	Clarification	Clarified size and definition of elements.
12.9.1.2	<PAYEERS>	Correction	Moved <PAYACCT> after <EXTDPAYEE> aggregate.
12.9.2.1	<PAYEEMODRQ>	Clarification	Clarified size and definition of elements.
		Correction	Made <PAYACCT> optional.
12.9.2.2	<PAYEEMODRS>	Deletion	Deleted <PAYEEID>
		Correction	Moved <PAYACCT> before <EXTDPAYEE> aggregate.
12.9.3.1	<PAYEEDELQR>	Clarification	Clarified size and definition of elements.
12.9.3.2	<PAYEEDELRS>	Clarification	Clarified size and definition of elements.
12.9.4.1	<PAYEESYNCRQ>	Clarification	Clarified size and definition of elements.
		Addition	Added <TOKENONLY>, <REFRESH>, and <REJECTIFMISSING>.
12.9.4.2	<PAYEESYNCRS>	Clarification	Clarified size and definition of elements.
12.10.1.1	<PMTSYNCRQ>	Clarification	Clarified size and definition of elements.
		Addition	Added <TOKENONLY>, <REFRESH>, and <REJECTIFMISSING>.
12.10.1.2	<PMTSYNCRS>	Clarification	Clarified size and definition of elements.
12.10.2.1	<RECPMTSYNCRQ>	Correction	Changed <PMTTRNRQ> to <RECPMTTRNRQ>.
		Addition	Added <TOKENONLY>, <REFRESH>, and <REJECTIFMISSING>.
		Clarification	Clarified size and definition of elements.
12.10.2.2	<RECPMTSYNCRS>	Correction	Changed <PMTTRNRQ> to <RECPMTTRNRQ>.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
12.11	Message sets and profile	Clarification	Clarified size and definition of elements.
		Correction	Changed <PMTMSGSETV1> to <BILLPAYMSGSETV1>.
12.11.2	<BILLPAYMSGSET> (was <PMTMSGSET>)	Correction	Changed <PMTMSGSET> to <BILLPAYMSGSET>.
		Correction	Made <PROCDAYSOFF> optional.
		Clarification	Clarified size and definition of elements.
12.12.9	Example of synchronization request	Addition	Added <REJECTIFMISSING>N after <TOKEN>.

B.1.1.12 Chapter 13, “Investments”

B.1.1.12.11 Sections 13 to 13.8.5.6

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
13	Client Sends / Server Responds table	Correction	Changed Cash Sub-Account Balance to Available Cash Balance.
		Correction	Changed Short Sub-Account Balance to Short Balance.
		Correction	Changed Margin Sub-Account Balance to Margin Balance.
		Deletion	Removed Other Sub-Account Balance.
13.3	Units, precision, and sign	Clarification	Clarified definition and precision of values.
13.6.1	<INVACCTFROM>	Clarification	Clarified size and definition of elements.
	<INVACCTTO>	Addition	Contains the same elements as <INVACCTFROM>.
	<INVACCTINFO>	Clarification	Clarified size and definition of elements.
		Clarification	Clarified size and definition of elements.
13.7.1.1	<INVSTMTMSGSET>	Clarification	Clarified usage of <CANEMAIL>.
13.8.1	<SECID>	Clarification	Clarified size and definition of elements.
13.8.2.1	<SECLISTRNRQ>	Correction	Made <SECLISTRQ> aggregate required.
		Correction	Added <TAN> element.
13.8.2.2	<SECLISTRQ>	Clarification	Clarified size and definition of elements.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
13.8.3.1	<SECLISTTRNRS>	Correction	For the security identification, specify either <SECID>, <TICKER>, or <FIID>.
13.8.5.1	<SECINFO>	Correction	Moved <CLTCOOKIE> after <STATUS> aggregate.
		Clarification	Clarified size and definition of elements.
		Deletion	Removed paragraph beginning “Descriptive information is not standard...” The paragraph referred to the <SOURCE> tag, which is not used.
13.8.5.2	<DEBTINFO>	Correction	Changed <NAME> element to <SECNAME>.
		Clarification	Clarified size and definition of elements.
		Correction	Made <COUPONFREQ> an enumerated type with the following values: MONTHLY, QUARTERLY, SEMIANNUAL, ANNUAL, or OTHER.
13.8.5.3	<MFINFO>	Clarification	Clarified size and definition of elements.
13.8.5.4	<OPTINFO>	Clarification	Clarified size and definition of elements.
13.8.5.5	<OTHERINFO>	Clarification	Clarified size and definition of elements.
13.8.5.6	<STOCKINFO>	Clarification	Clarified size and definition of elements.

B.1.1.12.12 Sections 13.9.1.2 to 13.9.2.5.2

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
13.9.1.1	<INVSTMTRNRQ>	Addition	Added <TAN> element.
		Correction	Made <INVSTMTRQ> aggregate required.
13.9.1.2	<INVSTMTRQ>	Addition	If <DTASOF> is not included with the <INCPOS> aggregate, the server should return the most current position information available.
		Correction	Made <INCOO> element and <INCPOS> aggregate required.
		Clarification	Clarified size and definition of elements.
13.9.2.1	<INVSTMTRNRS>	Correction	Moved <CLTCOOKIE> after <STATUS> aggregate.
13.9.2.2	<INVSTMTRS>	Clarification	Clarified size and definition of elements.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
13.9.2.3	<INVBANKTRAN>	Correction	All elements are required.
13.9.2.4.1	<INVTRAN>	Clarification	Clarified size and definition of elements.
13.9.2.4.2	Transaction aggregate elements	Addition	Added <DTPURCHASE>.
		Correction	Changed <AVECOSTBASIS> to <AVGCOSTBASIS>
		Clarification	Clarified size and definition of elements.
13.9.2.4.4	Investment transaction aggregates	Clarification	<CLOSUREOPT> aggregate: Defined RELFITID.
		Correction	<SELLOPT> aggregate: Corrected order of aggregates and elements.
		Addition	<TRANSFER> aggregate: Added <AVECOSTBASIS>, <UNITPRICE>, and <DTPURCHASE>.
13.9.2.4.5	Valid transactions by security type	Addition	Since JRNLFUND and MARGININTEREST do not refer to securities, there are no checks in any of the security columns for these transactions.
		Correction	Moved <PAYACCT> before <EXTDPAYEE> aggregate.
13.9.2.5.1	<OO>	Clarification	Clarified size and definition of elements.
13.9.2.5.2	Investment Open Order Aggregates	Clarification	Clarified size and definition of elements.

B.1.1.12.13 Sections 13.9.2.6.1 to 13.11

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
13.9.2.6.1	<INVPOS>	Clarification	Clarified size and definition of elements.
13.9.2.6.2	Investment positions	Clarification	Clarified size and definition of elements.
13.9.2.7	<INVBAL>	Clarification	Clarified aggregate description. Clarified size and definition of elements in table.
13.10.2, 13.10.2.1, 13.10.2.2 13.10.2.1 13.11	Investment e-mail synchronization	Correction	Changed <CASHACCT> to <AVAILCASH>.
			Changed <MARGINACCT> to <MARGINBALANCE>.
			Changed <SHORTACCT> to <SHORTBALANCE>.
	<INVMAILSYNCRQ>	Deletion	Deleted <OTHERACCT>.
		Addition	Added description of requests and responses.
	Example	Addition	Added <TOKENONLY>, <REFRESH>, and <REJECTIFMISSING>.
		Correction	Changed <BALTYPE>P to <BALTYPE>PERCENT.
		Deletion	Deleted <DTEND> element, since the example is requesting transactions up to the current date.
		Correction	Changed <CASHACCT> to <AVAILCASH>.
		Correction	Changed <MARGINACCT> to <MARGINBALANCE>.
		Correction	Changed <SHORTACCT> to <SHORTBALANCE>.
		Deletion	Deleted <OTHERACCT>.
		Correction	Changed <NAME> to <SECNAME> in all <SECINFO> aggregates.

B.1.1.13 Appendix A

Appendix A, “Status Codes,” has been added to the specification. It provides a complete list of the status codes that a server can return.

B.1.2 General Specification Changes

The text associated with status codes has been standardized throughout the specification. The text for a status code should always be the same, regardless of the context in which it is used. For a complete list of the status codes that can be returned by a server, refer to Appendix A, “Status Codes.”

B.1.3 DTD Changes

The corrections to the DTD files ensure that the DTD files match the specification. They do not represent changes to the specification itself.

<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
<ACCTINFO>	Correction	Made <DESC> and <PHONE> optional.
<ACCTRS>	Addition	Added <SVCSTATUS>.
<BANKMSGSETV1>	Correction	Made <XFERPROF> and <STPCHKPROF> optional.
<BANKMSGSRQV1>, <BANKMSGSRSV1>	Addition	Some of the synchronization requests and responses were not listed as being part of <BANKMSGSRQV1> and <BANKMSGSRSV1>.
<CLTCookie>, <TAN>, <PINCH>, <REINVDIV>	Correction	Made elements optional.
<DEBTINFO>	Correction	Made <COUPONFREQ> an enumerated type with the following values: MONTHLY, QUARTERLY, SEMIANNUAL, ANNUAL, or OTHER.
<EXTDPMT>	Correction	<EXTDPMT> can contain one or both of the following aggregates: <EXTDPMTDSC> and <EXTDPMTINV>.
	Correction	Made <DISCOUNT>, <DSCDATE>, <ADJUSTMENT>, and <ADJNO> optional.
	Correction	Added <DSCDESC> and <ADJDATE>.
	Addition	Added <EXTDPMTFOR> and <EXTDPMTCHK>.
Investment statements	Correction	Removed required ordering constraint for transactions, open orders, positions, and security lists.

<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
<INVTMTMSGSET>	Addition	Added <CANEMAIL> element to DTD.
<LOSTSYNC>	Addition	Added <LOSTSYNC> to DTD.
<MAILSYNCRQ>	Addition	Added <INCIMAGES> and <USEHTML>.
<MAILSYNCRS>	Deletion	Removed <INCIMAGES> and <USEHTML>.
<PAYEEMODRQ>	Correction	If <BANKACCTTO> is used, <PAYEE> is required.
<PAYEEMODRS>	Correction	Made <BANKACCTTO> and <PAYEE> optional.
<PROFRS>	Addition	Added <COUNTRY> after <POSTALCODE>.
<SECINFO>	Correction	Changed <NAME> element to <SECNAME>.
<SECRQ>	Correction	Corrected aggregate to match specification.
<SIGNONINFO>	Correction	Replaced <ALPHA> and <NUMERIC> with <CHARTYPE>.
<SIGNUPMSGSRQV1>, <SIGNUPMSGSRV1>,	Addition	Added <CHGUSERINFOSYNCRQ> and <CHGUSERINFOSYNCRS>.
<SVCADD>	Deletion	Deleted <SVCSTATUS>.
<SVCCHG>	Deletion	Deleted <SVCSTATUS>.
<TAXEXEMPT>	Correction	Corrected spelling of tag.
Unused elements	Deletion	Removed elements that were not used in any aggregates. ERROR, DESCRIPTION, AMOUNT, PAYEEACCTTO, GLOBALBUDGET, and LOGO.
<xxxSYNCRQ>	Addition	Added <TOKENONLY>, <REJECTIFMISSING>, <REFRESH>.
<xxxSYNCRS>	Addition	Added <LOSTSYNC> element.
<xxxTRNRS>	Correction	Made <xxxRS> optional.

B.2 OFX 1.0.1 to 1.0.2

This section describes the revisions to Open Financial Exchange that occurred between version 1.0.1 and version 1.0.2. These revisions are grouped into the following categories:

- ◆ Revisions to the specification text, organized by chapter
- ◆ General revisions to the specification text
- ◆ Changes to the Document Type Definition (DTD) files

B.2.1 Specification Changes by Chapter

B.2.1.1 Chapter 1, “Overview”

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
1.2	OFX headers, VERSION	Update	Updated VERSION:101 to VERSION:102 to indicate version 1.0.2 of the DTDs.

B.2.1.2 Chapter 2, “Structure”

B.2.1.2.14 Sections 2 to 2.4.5

Section	Subject	Change Type	Change
2.	Data blocks	Correction	Open Financial Exchange data consists of some headers plus only <i>one</i> Open Financial Exchange data block.
2.1	HTTP headers	Addition	If a communication time-out error occurs while an OFX server and back-end server are communicating to fill a request, then the server MUST return code 500.
2.2, 2.2.3	OFX headers, VERSION	Update	Updated VERSION:101 to VERSION:102 to indicate version 1.0.2 of the DTDs.
2.2.4	Security	Deletion	Reference to Type 2 security has been deleted.
2.4.2	Case sensitivity	Addition	Upper case letters must be used for element names and enumerated values.
2.4.3	Top level	Clarification	A single file must contain only one OFX block.
2.4.5.3	Message Set Version Numbers	Addition	Added section about message set version numbers. This section distinguishes message set version numbers from the version numbers of the OFX headers and the DTD files.
2.4.6	Message sets and version control	Addition	List of status code values for the <CODE> element of <STATUS>: 0 Success (INFO) 2000 General error (ERROR) 2022 Invalid TAN (ERROR)

B.2.1.2.15 Sections 2.5.1.1 to 2.5.2.3

Section	Subject	Change Type	Change
2.5.1.1	<SONRQ>	Clarification	The effective size of <USERPASS> is A-32. However, if Type 1 security is used, then the actual length is A-171.
2.5.1.1	<FI>	Clarification	The client will determine out-of-band whether a FI aggregate should be used and if so, the appropriate values for it. If the FI aggregate is to be used, then the client should send it in every request, and the server should return it in every response.
2.5.1.2	<SONRS>	Clarification	A client should use <DTPROFUP> and <DTACCTUP> only when the service provider that originated <SONRS> is the same provider that is specified by <SPNAME> in the profile message set. A client can determine if the service provider is the same by comparing the value of <SPNAME> in the appropriate message set with the value for <SPNAME> in the profile message set.
2.5.2.1	<PINCHRQ>	Clarification	A <USERPASS> change request changes the customer's password for the specific realm associated with the messages contained in the OFX block.
2.5.2.1	<NEWUSERPASS>	Clarification	The effective size of <NEWUSERPASS> is A-32. However, if Type 1 security is used, then the actual field length is A-171.
2.5.2.3	<CHALLENGERQ> <CHALLENGERS>	Addition	Information on the challenge request and challenge response, the first steps in Type 1 security, has been added.
2.5.2.3	<CHALLENGERQ> <CHALLENGERS>	Addition	Status code 15501, Could not provide random data (ERROR), has been added to the list of Status code values for the <CODE> element of <STATUS>.
2.5.3	<SIGNONMSGSET>	Deletion	Deleted <PINCH> and <CHGPFIRST> from <SIGNONMSGSET>.

B.2.1.3 Chapter 3, “Common Aggregates, Elements, & Data Types”

Section	Subject	Change Type	Change
3.2.8.2	Date and datetime	Addition	Time zones are specified by an offset and optionally, a time zone name. The offset defines the time zone.

B.2.1.4 Chapter 4, “Open Financial Exchange Security”

Section	Subject	Change Type	Change
Entire chapter	Security	Addition	The previous Chapter 4 on Open Financial Exchange Security has been replaced by a new one.

B.2.1.5 Chapter 7, “FI Profile”

Section	Subject	Change Type	Change
7.2	Profile response	Deletion	<SYNCMODE> AND <RESPFILEER> were deleted from <PROFRS> and added to <MSGSETCORE>.
7.2.1	Profile response	Clarification	<SPNAME> was added to <MSGSETCORE> with a note indicating that if a FI out-sources its OFX server to a service provider, then the SPNAME element should be included in the <MSGSETCORE>.
7.2.1	Message set	Addition	<SYNCMODE> AND <RESPFILEER> were added to <MSGSETCORE> and deleted from <PROFRS>.
7.2.1	<TRANSPSEC>	Clarification	For all message sets currently defined in Open Financial Exchange, the value of <TRANSPSEC> must be YES.
7.2.1	<TRANSPSEC>	Deletion	Type 2 has been deleted as an enumerated type for <TRANSPSEC>.
7.2.1	<VER>	Correction	Version number of the message set, (for example, <VER>1 for version 1 of the message set), N-5
7.2.2	Signon realms	Addition	<PINCH> and <CHGPINFIRST> have been added to <SIGNONINFO>.

B.2.1.6 Chapter 8, “Activation & Account Information”

Section	Subject	Change Type	Change
8.5	Account information	Clarification	Requests and responses include a <DTACCTUP> element. Responses contain the last time a server updated the information. Clients are REQUIRED to send this in a subsequent request, and servers are REQUIRED to compare this to the current modification time and only send information if it is more recent.
8.6.1.1	Request <ACCTRQ>	Clarification	The components of <ACCTRQ> are described in more detail.
8.6.1.2	Response <ACCTRS>	Clarification	The components of <ACCTRS> are described in more detail.
8.7	<CHGUSERINFOTRNRQ> and <CHGUSERINFOTRNRS>	Correction	In the first sentence of the second paragraph, the incorrect tag, <CHGUSERINFOTRNRSRQ>, was changed to <CHGUSERINFOTRNRS>.

B.2.1.7 Chapter 9, “Customer to FI Communication”

Section	Subject	Change Type	Change
9.3.2	OFX headers, VERSION	Update	Updated VERSION:101 to VERSION:102 to indicate version 1.0.2 of the DTDs.

B.2.1.8 Chapter 11, “Banking”

Section	Subject	Change Type	Change
11.4.2.3.1	<STMTTRN>	Correction	<FITID> is of type <i>FITID</i> .
11.5.4.2	<CCCLOSING>	Correction	<FITID> is of type <i>FITID</i> .
11.6.1.1	Stop check	Change	<NAME> type changed from A-255 to A-32.
11.6.1.1.2	Stop check	Change	<NAME> type changed from A-255 to A-32.
11.6.1.1.2.1	Stop check	Change	<NAME> type changed from A-255 to A-32.
11.12.5.1	Statement closing download	Change	<RECINTRATRNRQ> has been changed to optional.

B.2.1.9 Chapter 12, “Payments”

Section	Subject	Change Type	Change
12.5.2	<PMTINFO>	Clarification	More detailed information on <PAYEEID> has been added.
12.5.2	<PMTINFO>	Correction	<PAYEELSTID> is of type A-12.
12.5.2	<PMTINFO>	Clarification	< DTDUE > is the payment due date or the date by which payment must be received by payee, <i>datetime</i>
12.5.3.6	<EXTDPAYEE>	Change	<PAYEEID> type changed from N-12 to A-12.
12.6.1.2	<PMTRS>	Change	<PAYEELSTID> type changed from N-12 to A-12.
12.7.1.2	<RECPMTRS>	Change	<PAYEELSTID> type changed from N-12 to A-12.
12.9.1.1	<PAYEERQ>	Change	<PAYEEID> type changed from N-12 to A-12.
12.11.2	<BILLPAYMSGSET>	Clarification	<DAYSWITH> is an offset to withdrawal date, such that (DTDUE - DAYSTOPAY) + (DAYSWITH) determines the date on which the funds are withdrawn from the user’s account. Note: If the value of <DAYSWITH> is -1, then the withdrawal date is the same as the payment date (DTDUE).

B.2.1.10 Chapter 13, “Investments”

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
13.8.3.3	<SECLISTR>	Clarification	The security list response aggregate, <SECLISTR>, is the only empty aggregate in OFX.
13.9.2.4.2	Transaction Aggregate Elements	Correction	Deleted redundant <DEBTACTION> tag.
13.9.2.4.3	<INVBUR> / <INVSELL>	Change	<COUPONFREQ> has been changed from N-12 to enumerated type.
13.9.2.4.3	<INVBUR> / <INVSELL>	Change	<AVGCOSTBASIS>, <UNITPRICE>, and <DTPURCHASE> have been changed to optional.
13.9.2.4.4	<TRANSFER>	Clarification	Clarified <DTPURCHASE>tag name in the <TRANSFER> aggregate.

B.2.1.11 Appendix A

<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
Status code, 2021	Addition	The server does not support the requested version.
Status code, 2022	Addition	The server was unable to validate the TAN sent in the request.
Status code, 15504	Addition	Could not provide random data

B.2.2 General Specification Changes

The text item “PIN” has been replaced with “USERPASS.” This change has not been made to tags containing “PIN.”

B.2.3 DTD Changes

The corrections to the DTD files ensure that the DTD files match the specification. They do not necessarily represent changes to the specification itself.

<i>File</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
OFXBANK.DTD	<RECINTRASYNCRQ>	Correction	Made RECINTRATRNRQ zero or more.
OFXBANK.DTD	<CHKNUMSTART>	Correction	Changed to IDTYPE.
OFXBANK.DTD	<CHKNUMEND>	Correction	Changed to IDTYPE.

<i>File</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
OFXBILL.DTD	<CHECKNUM>	Correction	Changed to IDTYPE.
OFXINV.DTD	<COUPONFREQ>	Correction	Changed to STRTYPE.
OFXINV.DTD	<TRANSFER>	Correction	Changed AVGCOSTBASIS, UNITPRICE, DTPURCHASE to optional.
OFXPROF.DTD	<PROFRS>	Move	Moved SYNCMODE and RESPFILEER from PROFRS to MSGSETCORE.
OFXPROF.DTD	<MSGSETCORE>	Move	Moved SYNCMODE and RESPFILEER from PROFRS to MSGSETCORE.
OFXPROF.DTD	<MSGSETCORE>	Addition	Added SPNAME.
OFXPROF.DTD	<SIGNONINFO>	Move	Moved PINCH and CHGPINFIRST from SIGNONMSGSETV1 to SIGNONINFO.
OFXSIGN.DTD	<SIGNONMSGSETV1>	Move	Moved PINCH and CHGPINFIRST from SIGNONMSGSETV1 to SIGNONINFO.
OFXSIGN.DTD	<SIGNONMSGSRQV1>	Addition	Added CHALLENGETRNRQ.
OFXSIGN.DTD	<SIGNONMSGSRSV1>	Addition	Added CHALLENGETRNRs.
OFXSIGN.DTD	<CHALLENGETRNRQ>	Addition	New request.
OFXSIGN.DTD	<CHALLENGETRNRs>	Addition	New response.
OFXSIGN.DTD	<SONRS>	Change	Made DTPROFUP and DTACCTUP optional to accommodate service providers.
OFXSIGN.DTD	<CHALLENGERQ>	Addition	New request.
OFXSIGN.DTD	<CHALLENGERS>	Addition	New response.
OFXSIGN.DTD	<NONCE>	Addition	Added new element.
OFXSIGN.DTD	<FICERTID>	Addition	Added new element.

B.3 OFX 1.0.2 to 1.5

B.3.1 Specification Changes by Chapter

B.3.1.1 Chapter 1, “Overview”

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
1.2.2	White space following or preceding a tag delimiter	Addition	If white space is desired preceding or following an element value, this is achieved using the CDATA wrapper or, in version 2 message sets, the macro.

B.3.1.2 Chapter 2, “Structure”

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
2	Proper use of end of line characters	Addition	A proper client should separate the components of an OFX request using a single CRLF between each component
2	Server tolerance of bad clients	Addition	Whilst it is seen appropriate for testing parsers to check full conformance to this specification, it is recommended that operational parsers be tolerant of deviations.
2.2	All OFX headers are required	Addition	All OFX headers are required. NONE should be returned if client or server does not make use of an individual tag.
2.2.5	ENCODING	Change	ENCODING now accepts UTF-8 as a valid value, instead of UNICODE.
2.3.2.1	Special Characters	Addition	Support has been added for use of the (space) macro in V2 messages. Clarification on the use of the CDATA.
2.4.5.2	Message Set Ordering	Addition	New Bill Presentment message sets added. Ordering amongst message set versions explained.
2.4.6	Transactions	Clarification	CLTCOOKIE must only be returned by the server in the initial response.
2.5.1	Signon	Clarification	Authentication of the SONRQ is always required, even in error recovery. Servers must accept a GENUSERKEY in a SONRQ. Anonymous logons explained. Discussion of “empty” sign-on transactions.

Section	Subject	Change Type	Change
2.5.1.1	<SONRQ>	Addition	Optional tags ONETIMEPASS and COUNTRY added for V2 messages
2.5.1.2	<SONRS>	Addition	Optional tag COUNTRY added for V2 messages

B.3.1.3 Chapter 3, “Common Aggregates, Elements, and Data Types”

Section	Subject	Change Type	Change
3.1.4	<BAL>	Correction	VALUE should be of type Amount, rather than N-20.
3.1.4	<STATUS>	Global	MESSAGE2 replaces MESSAGE for V2 messages
3.2.2	<SRVRTID>, <SRVRTID2>	Global	SRVRTID2 defined for use in V2 messages. Note: SRVRTID2 replaces SRVRTID for all V2 messages. Due to the large number of instances of this tag, this change document will not note each of these instances. Note: Since RECSRVRTID is based on SRVRTID, it is replaced by RECSRVRTID2 for all V2 messages. Due to the large number of instances of this tag, this change document will not note each of these instances.
3.2.4	<TOKEN>, <TOKEN2>	Global	TOKEN2 defined for use in V2 messages. Note: TOKEN2 replaces TOKEN for all V2 messages. Due to the large number of instances of this tag, this change document will not note each of these instances.
3.2.6	<MEMO>, <MEMO2>	Global	MEMO2 defined for use in V2 messages. Note: MEMO2 replaces MEMO for all V2 messages. cell Due to the large number of instances of this tag, this change document will not note each of these instances.
3.2.8	Dates and Times	Clarification	Valid gmt offset values are in the range from -12 to +12 for whole number offsets. Formatting is +12.00 to -12.00 for fractional offsets, plus sign may be omitted.
3.2.8.2	Date and Datetime	Clarification	Clarification of timezone offset formatting
3.2.8.4	Time Zone Issues	Addition	Suggested behavior for clients not knowing the current local time zone.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
3.2.9	Amount	Clarification	Trailing and leading spaces should be stripped. Number format uses a leading sign. Negative number format uses a minus sign (-).
3.2.9.1	Amount, Prices, and Quantities – Basic Format	Clarification Addition	Leading and trailing spaces should be stripped.
3.2.9.2	Positive and Negative signs	Correction	Except for cases where the flow of funds is unclear, there would be no signage.
3.2.11	<URL>, URL2>	Global	URL2 defined for use in V2 messages. Note: URL2 replaces URL for all V2 messages. Due to the large number of instances of this tag, this change document will not note each of these instances.

B.3.1.4 Chapter 5, “International Support”

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
5.1	ENCODING	Change	ENCODING now accepts UTF-8 as a valid value, instead of UNICODE

B.3.1.5 Chapter 6, “Data Synchronization”

Section	Subject	Change Type	Change
6.4	Data Synchronization	Clarification	Servers should return TOKEN=-1 if they must respond with an error SYNCERROR was added in version 2 to allow the server to return the value that would normally be in the CODE field of the STATUS aggregate
6.6	Synchronization request wrapper	Addition	TOKEN2 defined for use in V2 messages
6.6	Client Synchronization	Clarification	All xxxSYNCRQ are required to contain one, and only one, of <TOKEN>, <TOKENONLY>, or <REFRESH>.
6.10	Lite synchronization and Refresh	Clarification	Lite synchronization servers do not support Refresh. <TOKEN>0 is not the same as Refresh synchronization. Therefore, clients should not send Refresh to a lite synchronization server.
6.10.1	Lite Synchronization	Addition	Suggested behavior when NEWFILEUID matches a previous request file Note: Open Financial Exchange requires a server to authenticate a client in Error Recovery
6.10.1	OLDFILEUID	Change	If OLDFILEUID is set to NONE, the client is not requesting file based error recovery. <i>This change was never implemented by OFX clients or servers. Due to problems with these suggestions, they were removed and/or superseded with the publication of OFX 1.6.</i>
6.10.1	NEWFILEUID	Clarification	If NEWFILEUID matches a previous request file the client is requesting error recovery. The request file identified by the NEWFILEUID must contain exactly the same set of transactions as the previous request file. Servers can reject the file if it contains new or modified transactions. NOTE: Clients should disallow <PINCH> transactions during error recovery. <i>This change was never implemented by OFX clients or servers. Due to problems with these suggestions, they were removed and/or superseded with the publication of OFX 1.6.</i>

B.3.1.6 Chapter 7, “FI Profile”

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
7.1.2	Version Control	Addition	Version 2 of the profile message set is explained.
7.1.3	Batching and Routing	Clarification	Supporting a message set using multiple servers.
7.1.4	Client Signon for Profile Requests	Clarification	Suggested text for anonymous logon.
7.2	<PROFRS>	Correction	Removed bolding from the optional SIGNONINFO aggregate in the PROFRS table.
7.2	<PROFRS>	Addition	Optional tag URLGETREDIRECT added for V2 messages
7.2	Signon Information in the profile response	Clarification	The DTD is correct that zero or more <SIGNONINFO> aggregates can be returned in a <PROFRS>. The specification incorrectly stated 1 or more.
7.2.1	Tag order in <MSGSETCORE>	Clarification	The specification incorrectly ordered the tags within the <MSGSETCORE> aggregate. The DTD correctly ordered the tags with the <SPNAME> element last.
7.2.1	<MSGSETCORE>	Clarification	LANGUAGE can be used 1 or more times
7.2.1	<MSGSETCORE>	Addition	Optional tag COUNTRY added for V2 messages
7.2.1	<MSGSETCORE>	Clarification	The VER / URL pair must be unique for each instance of MSGSETCORE
7.2.2	SIGNONINFO	Addition	Optional tag PWTYPE added for V2 messages

B.3.1.7 Chapter 8, “Activation & Account Information”

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
8.4	Enrollment and Password Acquisition	Clarification	Suggested text for anonymous logon
8.6.1.1	<ACCTRQ>	Addition	SVC2 replaces SVC for V2 messages
8.6.1.2	<ACCTRS>	Addition	SVC2 replaces SVC for V2 messages
8.6.1.3	<SVCADD>	Addition	Explanatory text on the use of SVCADD. Optional tag PREAUTHTOKEN added for V2 messages.
8.7	<CHGUSERINFORQ>, <CHGUSERINFORS>	Addition	All elements must be submitted in a CHGUSERINFORQ. Lack of inclusion implies its deletion on the server.
8.7.1	<CHGUSERINFORQ>	Addition	Optional tag USERID added for V2 messages
8.7.2	<CHGUSERINFORS>	Addition	Optional tag USERID added for V2 messages
8.8	<SIGNUPMSGSET>	Addition	SIGNUPMSGSETV2: same as V1, except use of MESSAGE2 in OTHERENROLL, and the additional required tag PREAUTH

B.3.1.8 Chapter 9, “Customer to FI Communication”

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
9.4	<EMAILMSGSET>	Addition	EMAILMSGSETV2: same as V1

B.3.1.9 Chapter 11, “Banking”

Section	Subject	Change Type	Change
11.3.1	<BANKACCTFROM>	Addition	ACCTTYPE2 replaces ACCTTYPE for V2 messages
11.3.1	<BANKACCTTO>	Addition	ACCTTYPE2 replaces ACCTTYPE for V2 messages. Optional aggregate EXTBANKACCTTO added for V2 messages.
11.3.2	<CCACCTFROM>	Clarification	<CCACCTFROM> can be used even if the credit card message set is not supported.
11.3.5	<XFERINFO>	Addition	Optional tag DTAVAIL added for V2 messages. Optional tag MEMO2 added for V2 messages.
11.4.3	<STMTRN>	Addition	PAYEEID2 replaces PAYEEID for V2 messages
11.7.1.2	<INTRARS>	Clarification	The DTD does not require either <DTXFERPRJ> or <DTPOSTED> in an intrabank, interbank or wire transfer response.
11.8.2.2	<INTERRS>		
11.8.5	<MULTIINTERTRNRQ>, <MULTIINTERTRNRS>	Addition	Some countries have a special interbank funds transfer transaction. Such a transaction combines multiple interbank transfer instructions into a single unit of work. V2 only.
11.9.1.2	<WIRERS>	Clarification	The DTD does not require either <DTXFERPRJ> or <DTPOSTED> in an intrabank, interbank or wire transfer response.
11.10	Recurring Funds Transfer	Global	For all V2 recurring funds transfer messages, XFERINFO replaces INTRARQ and INTRARS. Note: Due to the large number of instances of these aggregates, this change document will not note each of these instances.

Section	Subject	Change Type	Change
11.10.1.2	<RECINTRARS>	Clarification	The <SRVRTID> included in the <INTRARS> should be set to the same value as the <RECSRVRTID>. NOTE this is the response to the recurring model only. Servers must still generate a <INTRARS> for each instance of the recurring transfer.
11.12	Data Synchronization for Funds Transfer	Global	For all V2 funds transfer synchronization wrappers, CCACCTFROM should be used when appropriate. Note: Due to the large number of instances of these wrappers, this change document will not note each of these instances.
11.12.2.2	<INTRASYNCRS>	Clarification	<INTRASYNCRS> Response contain only intrabank transfers where the <BANKACCTFROM> of the Sync response matches the <BANKACCTFROM> of the <INTRARS>.
11.12.3.1	<INTERSYNCRQ>	Addition	Optional aggregate MULTIINTERTRNRQ added for V2 messages
11.12.3.2	<INTERSYNCRS>	Clarification	<INTERSYNCRS> Response contain only interbank transfers where the <BANKACCTFROM> of the Sync response matches the <BANKACCTFROM> of the <INTERRS>.
11.12.3.2	<INTERSYNCRS>	Addition	Optional aggregate MULTIINTERTRNRS added for V2 messages
11.13.2.1	<BANKMSGSET>	Addition	BANKMSGSETV2: same as V1, except INVALIDACCTTYPE2 replaces INVALIDACCTTYPE
11.13.2.2	<XFERPROF>	Addition	Optional tag NEEDTANTRANSFER added for V2 messages. Optional tag SUPPORTDTAVAIL added for V2 messages.
11.13.4	<INTERXFERMSGSET>	Addition	INTERXFERMSGSETV2: same as V1, except additional required CANMULTI tag
11.13.4	<INTERXFERMSGSET>	Correction	DOMXFERFEE and INTLXFERFEE should be of type Amount, rather than N-8.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
11.13.5	<WIREXFERMSGSET>	Correction	DOMXFERFEE and INTLXFERFEE should be of type Amount, rather than N-8.

B.3.1.10 Chapter 12, “Payments”

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
12	<PAYEEID2>	Global	PAYEEID2 replaces PAYEEID for all V2 messages in the Bill Payment message set. Due to the large number of instances of this tag, this change document will not note each of these instances.
12	<PAYEELSTID2>	Global	PAYEELSTID2 replaces PAYEELSTID for all V2 messages in the Bill Payment message set. Due to the large number of instances of this tag, this change document will not note each of these instances.
12.2.4	Identifying Payees	Clarification	Servers must always assign PAYEELSTIDs to payees. Once PAYEELSTIDs have been assigned, clients must always send the <PAYEELSTID>, even if a Payee has both a <PAYEEID> and a <PAYEELSTID>.
12.4.1	Implicit requests	Clarification	Servers should generate explicit responses to implicit requests. In other words, implicit payee additions or modifications resulting from a new or changed payment should generate explicit payee add or payee change responses from the server. Such explicit responses are only returned to the client in a sync response.
12.4.1	Payment Discussion	Clarification	Payment transactions that also contain an implicit payee transaction (change or add) are considered a single unit of work. A failure of either causes a failure in both. This is true for payment creation, modification and deletion transactions.

Section	Subject	Change Type	Change
12.5.2	Payee Info	Clarification	The DTD requires either <PAYEE> or <PAYEEID> inside <PMTINFO>. The specification incorrectly says that both are optional.
12.5.2	<PMTINFO2>	Global	PMTINFO2 aggregate defined for use in V2 messages. Note: PMTINFO2 replaces PMTINFO for all V2 messages. Due to the large number of instances of this aggregate, this change document will not note each of these instances.
12.5.2.1	<PAYEE2>	Global	PAYEE2 aggregate defined for use in V2 messages. Note: PAYEE2 replaces PAYEE for all V2 messages in the Bill Payment message set. Due to the large number of instances of this aggregate, this change document will not note each of these instances.
12.5.2.2	<EXTDPMT>	Addition	EXTDPMTCHK2 replaces EXTDPMTCCHK for V2 messages
12.5.3.3	<INVOICE>	Correction	Both DSCRATE and DSCAMT are required. Previously it was stated that each was optional if the other was included.
12.5.3.3	<INVOICE>	Addition	DISCOUNT2 replaces DISCOUNT for V2 messages. Optional tag LITMCODE added to the LINEITEM aggregate for V2 messages.
12.5.3.6	<EXTDPAYEE>	Correction	Notes added to tag descriptions for PAYEEID, IDSCOPE, and NAME to indicate that if any of these optional tags is used, the others are then required.
12.5.3.6	<EXTDPAYEE>	Addition	In version 2 of the message set, If the value of <DAYSTOPAY> is -1, then <DTDUE> in <PMTINFO> is interpreted as an execution rather than a due date.
12.6	Payment Functions	Clarification	Because the flow of money is unambiguous, bill payment amounts are usually specified as positive numbers.
12.6.2	Payment Modification	Addition	Details on the use of PMTINFO in V2 messages.

Section	Subject	Change Type	Change
12.6.2.5	Payment Modification Discussion	Clarification	Implicit payee changes contained in a payment modification transaction affect the individual payment. The changes are also propagated to the server's payee list and affect subsequent payments to that payee. NOTE: Explicit payee changes are not propagated to pending payments.
12.7.1.4	<PMTINFO>	Addition	The <DTDUE> element of <PMTINFO> specifies payment due date or the date by which the first payment must be received by payee (see section 12.5.2).
12.7.2	Recurring Payment Modification	Clarification	Clients can modify both elements in the <RECURRINST> aggregate, i.e., <NINSTS> and <FREQ>. Clients should send the original number of payments scheduled if there is no change. If there is a change in the number of payments scheduled clients should send the new number of payments. Clients must account for pending payments and the server's ModPending flag. This can result in implicit payment deletions.
12.7.2.1	<RECPMTMODRQ>	Clarification	A <RECPMTMODRQ> that modifies pending payments via the <MODPENDING> flag is a compound transaction, and generates the appropriate explicit payment responses that reflect such modifications, which are returned to the client via synchronization.
12.7.3.1	< RECPMTCANCRQ>	Clarification	A <RECPMTCANCRQ> that cancels pending payments via the <CANPENDING> flag is a compound transaction, and generates the appropriate explicit payment responses that reflect such cancellations, which are returned to the client via synchronization.
12.9.1.1	<PAYEERQ>	Correction	One of either PAYEE or PAYEEID is required

Section	Subject	Change Type	Change
12.9.2	Payee Modification	Clarification	Explicit payee modifications are made in the server's payee list. They are not propagated to already created pending payments. Recurring Payments created after the Payee modification will use the updated information from the server's payee list. Implicit payee modifications contained within a payment creation, modification or deletion request are propagated to the payment and to the server's payee list.
12.9.2.1	<PAYEEMODRQ>	Clarification	Absence of a PAYACCT in PAYEEMODRQ could be interpreted as an implicit disassociation of the PAYACCT.
12.9.3	Payee Deletion	Clarification	Clients send the <PAYEELSTID> in the <PAYEEDELRQ> aggregate to delete an entire Payee. To modify or delete specific payee information, clients should use the <PAYEELSTID> and the <PAYACCT> elements in the <PAYEEMODRQ>.
12.11.2	<BILLPAYMSGSET>	Addition	<PROCDAYSOFF> indicate days to exclude when calculating dates that utilize other bill payment bits, such as <DAYSWITH> and <DFLTDAYSTOPAY> values. BILLPAYMSGSETV2: same as V1, except additional required tags NEEDTANPMT, NEEDTANPAYEE, and SUPPORTDTAVAIL.

B.3.1.11 Chapter 13, “Investments”

Section	Subject	Change Type	Change
13.3.3.	Signs	Clarification	<UNITS> and <TOTAL> are signed from the perspective of the user (positive for SELLS, negative for BUYS). All other Investment transaction amounts are always positively signed. In other words <UNITPRICE>, <COMMISSION>, <FEES>, <TAXES>, <WITHHOLDING>, <LOAD>, <MARKUP> and <MARKDOWN> are always positive numbers. A positive <COMMISSION>, <TAXES>, <LOAD>, <WITHHOLDING>, or <FEE> increased the negatively signed <TOTAL> on a <BUYSTOCK> and decreased the positively signed <TOTAL> on a <SELLSTOCK>. <MARKUP> and <MARKDOWN> increase or decrease, respectively, the <UNITPRICE>. Servers should return corrections to investment buys or sells as the opposite transaction type, e.g., a correction to a buy is returned as a sell. A correction to <MARGININTEREST> or <RETOFCAP> is returned as an <INVEXPENSE>.
13.6.1	Specifying the Investment Account	Clarification	Brokers should use the domain name of their company’s URL as the <BROKERID>, e.g., If URL=www.broker.com then BROKERID=broker.com>
13.8.4	<SECLIST>	Correction	The xxxINFO security information aggregates can occur zero or more times
13.9.2.4.2	<REVERSALFEES>, <REVERSALFITID>	Addition	REVERSALFEES and REVERSALFITID added for use in V2 messages

Section	Subject	Change Type	Change
13.9.2.4.3	Investment Aggregate	Clarification	<REINVEST> is a single transaction that contains both income and an investment transaction. If servers can't track this as a single transaction they should return an <INCOME> transaction and an <INVTRAN>. <TOTAL> and <UNITS> are signed as for an investment buy. Corrections to a <REINVEST> are signed as for an investment sell.
13.9.2.4.3	<INVBUY>	Addition	Optional tag REVERSALFEES added for V2 messages. Optional tag REVERSALFITID added for V2 messages.
13.9.2.4.3	<INVSELL>	Addition	Optional tag REVERSALFEES added for V2 messages. Optional tag REVERSALFITID added for V2 messages.
13.9.2.4.4	<BUYOTHER>	Addition	Optional tag BUYTYPE added for V2 messages.
13.9.2.4.4	<SELLOther>	Addition	Optional tag SELLTYPE added for V2 messages.
13.9.2.6.2	Investment Positions	Clarification	<UNITSSTREET> and <UNITSUSER> are incorrectly specified as <i>quantity</i> . They must be positive whole numbers A-32 (INTTYPE as per the DTD).
13.9.2.7	<BALLIST>	Correction	The BAL aggregate can occur zero or more times

B.3.1.12 Appendix A

Subject	Change Type	Change
Status Code 15507	Addition	Signon invalid without supporting pin change request (ERROR).

B.4 OFX 1.5 to OFX 1.5.1

This section describes the revisions to Open Financial Exchange that occurred between version 1.5 and version 1.5.1. These revisions are grouped into the following categories:

- ◆ Revisions to the specification text, organized by chapter
- ◆ Changes to the Document Type Definition (DTD) files

B.4.1 Specification Changes by Chapter

The following table describes changes implemented in OFX 1.5.1. There are some changes to the DTD because of these corrections. Changes to the DTD are noted in the Change Type column. An update to the DTD will be released concurrent to this errata document. OFX implementors should refer to the DTD in cases of a discrepancy between the specification and the DTD; in other words, “the DTD rules.” The changes in this table have been incorporated in the OFX 1.5.1 version of this specification. If you are viewing this document online, you can click on a section number to go directly to that section.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
<u>1.2.2</u>	White space and macro	Addition	Macro can be used with version 2 message sets and Bill Presentment.
<u>1.4</u>	OFX Versions	Addition	Specifies OFX versions supported by this specification.
<u>2.2.3</u>	<VERSION>	Change	Updated to reflect supported versions: OFX 1.0.2 and 1.5.1.
<u>2.4.5.3</u>	Message Set Version Numbers	Addition	Added column to cross-reference message set versions to DTD versions.
<u>7.1.2</u>	Version Control	Change	Updated to reflect supported versions: OFX 1.0.2 and 1.5.1. Added cross-references to DTD version descriptions and message set version descriptions.
<u>8.6.2</u>	<ACCTRS>	Addition DTD Change	<SVCSTATUS> is for use in V1 messages only. A required <SVCSTATUS2> tag is added for V2 messages only. The <SVCSTATUS2> has an additional enumerated value of “REJECTED”. An optional <REASON> tag, type A-255, is added after <SVCSTATUS2>, for V2 messages only. This tag is only relevant if <SVCSTATUS2>REJECTED is specified.
<u>10.5</u>	Canceling Recurring Models	Correction	Models cannot be cancelled at a future date. Updated description.

Section	Subject	Change Type	Change
<u>11.3.3</u>	<BANKACCTINFO>	Addition DTD Change	<SVCSTATUS> is for use in V1 messages only. A required <SVCSTATUS2> tag is added for V2 messages only. The <SVCSTATUS2> has an additional enumerated value of “REJECTED”. An optional <REASON> tag, type A-255, is added after <SVCSTATUS2>, for V2 messages only. This tag is only relevant if <SVCSTATUS2>REJECTED is specified.
<u>11.3.4</u>	<CCACCTINFO>	Addition DTD Change	<SVCSTATUS> is for use in V1 messages only. A required <SVCSTATUS2> tag is added for V2 messages only. The <SVCSTATUS2> has an additional enumerated value of “REJECTED”. An optional <REASON> tag, type A-255, is added after <SVCSTATUS2>, for V2 messages only. This tag is only relevant if <SVCSTATUS2>REJECTED is specified.
<u>11.12.8</u>	<SYNCERROR> Status Codes	Clarification	Added <SYNCERROR> to title. Added cross-reference from <SYNCERROR> description in <BANKMAILSYNCRS> definition.
<u>11.14.1</u>	Statement Download Example	Correction	Updated <TRNAMT> statements to use negative numbers for check and ATM withdrawal.
<u>12.5.1</u>	<BPACCTINFO>	Addition DTD Change	<SVCSTATUS> is for use in V1 messages only. A required <SVCSTATUS2> tag is added for V2 messages only. The <SVCSTATUS2> has an additional enumerated value of “REJECTED”. An optional <REASON> tag, type A-255, is added after <SVCSTATUS2>, for V2 messages only. This tag is only relevant if <SVCSTATUS2>REJECTED is specified.
<u>12.5.3</u>	<PMTINFO>	Clarification	Although <PMTINFO> allows multiple occurrences of <EXTDPMT>, it is recommended that multiple invoices be expressed using multiple occurrences of the <INVOICE> aggregate. This usage will correspond with the requirements of <PMTINFO2>.

Section	Subject	Change Type	Change
<u>12.5.3</u>	<PMTINFO2>	Change DTD Change	<EXTDPMT> is not repeatable. Multiple invoices may be expressed using multiple occurrences of the <INVOICE> aggregate. The note for <EXTDPMT> should read, “Optional, see section 12.5.2.2.”
<u>12.5.3.2</u>	<EXTDPMT>	Correction	Final paragraph of text before table should read “The <INVPAIDAMT> must be supplied when <INVOICE> is used inside an <EXTDPMT> as shown. It is marked required in accordance with the use of <INVOICE> in section 14.2.2.2”
<u>12.5.3.2</u>	<EXTDPMT>	Change DTD Change	<EXTDPMTDSC> is for use in V1 messages only. An optional <EXTDPMTDSC2> tag is added for V2 messages only. The <EXTDPMTDSC2> tag is longer; it has type “A-1280”.
<u>12.5.3.3</u>	<EXTDPMTINV>	Change DTD Change	<INVOICE> and <INVOICE2> aggregates defined. <ADJUSTMENT> and <LINEITEM> definitions moved to separate tables.
<u>12.9.4.3</u>	<SYNCERROR> Status Codes	Clarification	Added <SYNCERROR> to title. Added cross-reference from <SYNCERROR> description in <PAYEESYNCRS> definition.
<u>12.10.2.3</u>	<SYNCERROR> Status Codes	Clarification	Added <SYNCERROR> to title. Added cross-reference from <SYNCERROR> description in <RECPMTSYNCRS> definition.
<u>12.11.2</u>	<BILLPAYMSGSET>	Clarification	The usage note for <HASEXTDPMT> in <BILLPAYMSGSETV2> should read “Supports the <EXTDPMT> business payment aggregate for nonrecurring payments, Boolean.”
<u>12.11.2</u>	<BILLPAYMSGSET>	Addition DTD Change	Add an additional required Boolean tag, <HASRECEXTDPMT>, after <HASEXTDPMT> in <BILLPAYMSGSETV2>. The usage note should read “Supports the <EXTDPMT> business payment aggregate for recurring models, Boolean.”

Section	Subject	Change Type	Change
<u>13.6.2</u>	<INVACCTINFO>	Addition DTD Change	<SVCSTATUS> is for use in V1 messages only. A required <SVCSTATUS2> tag is added for V2 messages only. The <SVCSTATUS2> has an additional enumerated value of “REJECTED”. An optional <REASON> tag, type A-255, is added after <SVCSTATUS2>, for V2 messages only. This tag is only relevant if <SVCSTATUS2>REJECTED is specified.
<u>14.1.2</u>	Severs and Message Sets	Clarification	Note: Although these message sets are named “V1”, they are used in conjunction with other V2 messages. Specifically, <SIGNONMSGSETV2> should be used, as well as the “V2” versions of account signup messages, and of the <INVOICE> aggregate used in <PRESBILLINFO>.
<u>14.3</u>	Customer Signup	Clarification	Note: The “V2” versions of the account signup messages should be used with Bill Presentment.
<u>14.3.2.1</u>	<PRESACCTINFO>	Correction DTD Change	The <SVCSTATUS> tag should be named <SVCSTATUS2>.
<u>14.3.2.2</u>	<PRESACCTFROM>, <PRESACCTTO>	Correction DTD Change	Clients can optionally include a <USERID> in <PRESACCTFROM/TO> that is different from the one used in the <SONRQ>. This <USERID> supports account activation by a third party on behalf of a user. Based on access rights granted to the <SONRQ>, it is up to the server whether to honor such a request. More than one <SVCADD> can be present in a single <OFX> request file on behalf of multiple <USERID>s. The <USERID> tag is disallowed for single-customer OFX request files. When sent by or returned to a client proxy (or, other group context), any <PRESACCTFROM/TO> aggregates <i>must</i> include the <USERID> tag. A client proxy would include (and receive) this tag when initiating an OFX session. But, customer-specific clients initiating a session with the same server would never use or see this information. This tag is thus <i>optional</i> in the DTD.

Section	Subject	Change Type	Change
<u>14.3.3.1</u>	<SVCADD>	Correction	The first and last paragraphs of this section and the <SVCADD> table were deleted. Since the Bill Presentment message set is in support of V2 messages, the use of the <PREAUTHTOKEN> tag is not a variation of the <SVCADD> service addition aggregate. Information regarding the proper use of <PRESNAMEADDRESS> within <PRESACCTTO> has been revised.
<u>14.3.3.2</u>	<SVCCHG>	Correction	Further detail is given on how the server stores <PRESNAMEADDRESS> information and how this information is used within <SVCCHG>. This explanation includes the use of <CANUPDATEPRESNAMEADDRESS>.
<u>14.3.4</u>	Service Status Update for Groups of Customers	Correction	The tag <SVCSTATUS> mentioned twice in the text should read <SVCSTATUS2>.
<u>14.3.4.1</u>	<ACCTINFORQ>	Correction	The tag <SVCSTATUS> mentioned twice in the text should read <SVCSTATUS2>.
<u>14.4.2.2.2</u>	<PRESBILLINFO>	Correction	Second paragraph should read: “If the client requested bill detail in the <PRESLISTRQ>, the bill publisher provides the detail in zero or more <BILLDETAILTABLE> aggregates. If the client did not request bill detail, the server should use the <DETAILAVAILABLE> flag to indicate whether the client can request bill detail at a later time using the <PREDETAILRQ> request.
<u>14.4.2.2.2</u>	<PRESBILLINFO>	Clarification DTD Change	Use <INVOICE2> instead of <INVOICE>.
<u>14.4.2.2.2</u>	<PRESBILLINFO>	Change	Usage note above <DETAILAVAILABLE> tag should read: “Choose <DETAILAVAILABLE> or <BILLDETAILTABLE>, but not both.”
<u>14.4.2.2.2</u>	<PRESBILLINFO>	Change DTD Change	The <BILLDETAILTABLE> aggregate should be used instead of the <PREDETAIL> aggregate. Referenced section should be 14.4.3.2.1 for more information about this aggregate.

Section	Subject	Change Type	Change
<u>14.4.2.4</u>	<PRESLISTTRNRS>	Correction DTD Change	<PRESLISTRS> is described in the specification as occurring zero or more times. The DTD shows it as required and occurring only once. Both documents should show PRESLISTRS as optional and occurring only once.
<u>14.4.3.2.1</u>	<BILLDETAILTABLE>	Change	The bill detail table may be redesigned in the future. Please consider this area “under construction.”
<u>14.6</u>	<PRESMAILSYNCRQ> <PRESMAILSYNCRS>	Addition DTD Change	Bill Presentment mail synchronization tables added (<u>14.6.4</u> and <u>14.6.5</u>).
<u>14.7.3</u>	<PRESDLVMSGSET>	Addition DTD Change	The required Boolean tag <CANUPDATEPRESNAMEADDRESS> is added to the end of the <PRESDLVPROF> aggregate. The description is “Supports update of the <PRESNAMEADDRESS> associated with a particular bill. See section 14.3.3.1”
<u>14.8.3</u>	Activation Examples	Correction	These corrections apply to both examples in this section. The <BILLER> tag should be <BILLERID>. The <NAME> tag should be <NAMEACCTHELD>. The <PHONE> tag should be <DAYPHONE> or <EVEPHONE>. The <SVC> tag should be <SVC2>.
<u>14.8.3.2</u>	Activation Response Example	Correction	The <COUNTRY> tag was misspelled as “COUTNRY”.
<u>14.8.4.4.1</u>	Customer Bill Delivery Request Example	Correction	The <PRESLISTRQ> aggregate should include a <NOTIFYWILLING> tag.
<u>14.8.4.1.2</u>	Customer Bill Delivery Response Example	Correction	The <SONRS> aggregate should include a <LANGUAGE> aggregate. The <PRESDTAILRS> aggregate and the <BILLID> tag contained within it are removed. In effect, the list of <BILLDETAILTABLE> aggregates moves up one level in this response.
<u>14.8.4.2.2</u>	Customer Bill Delivery Response Example for a Customer Service Representative	Correction	The <SONRS> aggregate should include a <LANGUAGE> aggregate. The <PRESDTAILRS> aggregate and the <BILLID> tag contained within it are removed. In effect, the list of <BILLDETAILTABLE> aggregates moves up one level in this response.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
<u>14.8.4.4.2</u>	Group Account Information Response Examples	Correction	The <PRESACCTINFO> aggregate should be enclosed by an <ACCTINFO> aggregate. The <BILLER> tag should be <BILLERID>. The <PRESACCTINFORS> tag appears three times, once as a begin tag for an aggregate, and twice as an end tag for an aggregate. In all three cases it should be <ACCTINFORS>. The <SVCSTATUS> tag appears twice and should be <SVCSTATUS2> in both cases.

B.4.2 DTD Changes

The corrections to the DTD files ensure that the DTD files match the specification. They do not represent changes to the specification itself.

<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
<PRESDIRPROF>, <PRESDLVPROF>	Correction	Made <PROCDAYSOFF> repeating.
<PRESDLVPROF>	Correction	Changed <GROUPID> to <CANSUPPORTGROUPID>.
<FINDBILLERTRNRS>	Correction	<FINDBILLERS> is optional.
<PRESGRPACCTINFOTRNR>	Correction	<ACCTINFORS> is optional.
<PRESLISTTRNRS>	Correction	<PRESLISTRS> is optional.
<PRESNOTIFYTRNRS>	Correction	<PRESNOTIFYRS> is optional.
<PRESDETAILTRNRS>	Correction	<PRESDETAILRS> is optional.
<BILLTBLSTRUCTTRNRS>	Correction	<BILLTBLSTRUCTRS> is optional.
<PRESMAILTRNRS>	Correction	<PRESMAILRS> is optional.
MSGSETMACRO	Correction	<PROFMSGSET> is required. <i>While this portion of the profile response remains optional in the OFX 1.0.2 DTD, it should be returned by every compliant server.</i>
<WIRESYNCRS>	Correction	Removed the <CCACCTFROM> aggregate from the synchronization wrapper. <i>This change was required to correct a problem introduced in OFX 1.5. It restores compatibility between OFX 1.0.2 and 1.5.1 clients and servers</i>
<XFERPROF>	Correction	Added <NEEDTANTRANSFER> and <SUPPORTDTAVAIL> to the bottom of <XFERPROF>.
Special Characters	Correction	Added macros corresponding to the escape sequences for "<", ">", and "&".

B.5 OFX 1.5.1 to 1.6

B.5.1 Specification Changes by Chapter

Section	Subject	Change Type	Change
Global	“Either or both of” OR construct	Correction	Searched document and changed any instance of “either or both of” to “At least one of”.
Global	XXX to xxx	Correction	Searched the document and changed any instance of <XXXtag> to <xxxtag>
Global	<tag> value	Correction	Searched document and replaced any instance of “<tag>” with a value of “<tag> value”.
Global	tag	Correction	Changed “tag” to element, aggregate, or both
Many	Error codes 2016, 2020, 2027, 2028, 6500, 6501, 6502, 10514, 10520, 10521, and 10522, 13504, 15508	Addition / Clarification	Added errors or clarified description of these errors in the status tables. <u>Appendix A</u> contains a full list of error codes and descriptions.
Many: <u>2.5.4</u> , <u>7.3</u> , <u>8.8</u> , <u>9.4</u> , <u>11.13.2</u> , <u>11.13.3</u> , <u>11.13.4</u> , <u>11.13.5</u> , <u>12.11.2</u> , <u>13.7.1.1</u> , <u>13.7.2.1</u> , <u>14.7.2</u> , <u>14.7.3</u>	Version 1 Message Sets	Addition, DTD Change	All <xxxMSGSETV1> are required, <xxxMSGSETV2> are optional. <xxxMSGSETV2> message sets are now repeatable (with the exception of <u>2.5.4</u> and <u>7.3</u>). <i>To ensure compatibility with OFX 1.0.2 clients, this change applies retroactively to OFX 1.5.1.</i>
Many: <u>7.2</u> , <u>8.4.2</u> , <u>8.7.1</u> , <u>8.7.2</u> , <u>11.9.1.1.2</u> , <u>12.5.2</u> , <u>12.5.3.1</u>	<ADDR2>, <ADDR3>	Addition	Added text stating ADDR3 requires ADDR2. Also added text stating ADDR2 requires ADDR1 for following sections: <u>8.7.1</u> , <u>8.7.2</u> , <u>12.5.2</u> , and <u>12.5.3.1 (PAYEE2 only)</u> .
Many: <u>11.3.6</u> , <u>11.7.1.2</u> , <u>11.8.2.2</u> , <u>12.5.3.7</u> , <u>12.6.1.2</u>	Client errors using the STATUS aggregate	Clarification / Addition	Added text to describe intended use of <STATUS>, <PMTPRCSTS>, and <XFERPRCSTS> aggregates.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
Many: 11.3.5 11.9.1.2 12.5.3 12.7.1.4	<DTDUE>	Clarification	Added words to explain why a server might return a different <DTDUE> than in the request.
Many 11.14.2 11.14.3 11.14.4 12.12.1 12.12.2 12.12.3 12.12.4 12.12.5 12.12.6 12.12.7 12.12.8 12.12.9 13.11 14.8.3.1 14.8.3.2 14.8.4.1.1 14.8.4.1.2 14.8.4.2.1 14.8.4.2.2 14.8.4.3.1 14.8.4.4.1 14.8.4.4.2	<SONRQ> and <SONRS> Examples	Improvement	Replaced full example with reference to example in 11.14.1 . Eliminated needless repetition and reduced overall length of examples.
10.3 11.7.2 11.8.3 12.6.2 12.7.2	Spawned payments	Clarification	Spawned payments behave identically to those created by explicit client requests. Changes to the spawned payment to not affect the model.
11.10.3.1 11.10.3.2 11.10.6.1 11.10.6.2 12.7.3.1 12.7.3.2	<CANPENDING> element	Clarification	Clarified usage note.
1.2.1	Data Transport	Correction / Addition	Changed the version number in the example to 160. Also added CHARSET, COMPRESSION, NEWIFLEUID, and OLDFILEUID to the examples.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
<u>1.2.2</u>	Request and Response Model	Clarification	Added some words to clarify how elements and aggregates are used in responses.
<u>1.3</u>	Definitions	Addition	Added definitions for key concepts.
<u>1.4</u>	OFX Versions	Correction	Clarified which versions are supported and which versions and publications are obsolete.
<u>1.5</u>	Conventions	Correction / Clarification	Added the new text for character and number data types.
<u>1.5</u>	Conventions	Addition	Added text to explain “1.6 add” version type.
<u>2</u>	Structure	Correction	Updated boundary text in example.
<u>2.1</u>	HTTP Headers	Addition	Define “invalid request” to include: general HTTP transport errors, SGML formatting errors, invalid OFX syntax (per the DTDs), and invalid data values (including data type restrictions and enum values per the DTDs). Clarified note about responding with responding with multipart MIME.
<u>2.1</u>	HTTP Headers	Correction	Removed the inference that only HTTP 200, 400, and 500 HTTP transport errors are possible. Changed occurrences of 400 to 400s. Changed occurrences of 500 to 500s.
<u>2.2.3</u>	VERSION	Correction	“151 and 160” are also supported versions. Also added some text explaining how each DTD can be used and what is supported.
<u>2.2.5</u>	ENCODING and CHARSET	Clarification	Added text clarifying use.
<u>2.2.7</u>	OLDFILEUID and NEWFILEUID	Correction	Changed wording slightly. The sentence before the bullets now reads “... serves two purposes.”
<u>2.2.7</u>	OLDFILEUID and NEWFILEUID	Correction	Deleted the reference to Chapter 4 and the third bulleted item.
<u>2.3.1</u>	Compliance	Clarification	Enhanced definition of unrecognized tags.

Section	Subject	Change Type	Change
<u>2.4.3</u>	Top Level	Clarification	Added required <SONRQ> or <SONRS>.
<u>2.4.4</u>	Messages	Clarification	Enhanced description of message types.
<u>2.4.5</u>	Message Sets and Version Control	Clarification, DTD Change	Clarified rules regarding repeating message sets, message set versions within a file, message versions within a message set, message set versions in a RQ / RS file pair, and message set ordering within a file. The only DTD change necessary for these corrections relates to the new ability of the <SIGNUPMSGSRQV2> message set wrapper to repeat. Please see section <u>8.1.1</u> for additional information.
<u>2.4.5.3</u>	Message Set Version Numbers	Addition	Added the following note: OFX versions 1.5.1 and 1.6 are intended to interoperate without problems. The additions made in OFX 1.6 are optional and may be ignored by 1.5.1 clients and servers without complications. In particular, a server must indicate support for a new feature before a client may use that feature.
<u>2.4.5.3</u>	Message Set Version Numbers		Changed DTD Version support statement from “... and the DTD version that support it” to “... and the DTD versions that support it.”
<u>2.4.5.3</u>	Message Set Version Numbers	Addition	Added 160 to list of version numbers supported and to table.
<u>2.4.6</u>	Transactions	Addition	Added cross-reference to section 3.1.2.
<u>2.4.6.1</u>	Transaction Wrapper	Clarification	Enhanced description of transaction wrapper.
<u>2.4.7</u>	Synchronization Wrapper	Addition	Added description of synchronization wrapper.
<u>2.4.8</u>	Message Set Wrapper	Addition	Added description of message set wrapper.

Section	Subject	Change Type	Change
<u>2.5.1</u>	Signon <SONRQ> <SONRS>	Correction	Deleted the line “Servers must accept user IDs, with or without punctuation.”
<u>2.5.1</u>	Signon <SONRQ> <SONRS>	Addition	Added the following text: The only present situation in this class are first time <PROFRQ> <FINDBILLERRQ>, and all <ENROLLRQ> transactions. Any request sent by the client after a successful <ENROLLRQ> response (or, out of band enrollment) for the service must provide the user’s <USERID> and <USERPASS>.
<u>2.5.1</u>	Signon <SONRQ> <SONRS>	Correction	Changed “OFX 1.5.1” to “OFX 1.5.1 or greater”.
<u>2.5.1</u>	Signon <SONRQ> <SONRS>	Addition	Added note concerning length of anonymous password.
<u>2.5.1</u>	Signon <SONRQ> <SONRS>	Clarification	Replaced “validation” with “authentication” in order to avoid inconsistent terminology.
<u>2.5.1</u>	Signon <SONRQ> <SONRS>	Addition	Added paragraph regarding signon errors and the use of code 15500.
<u>2.5.1.1</u>	Signon Request <SONRQ>	Clarification	Added additional usage note for <DTCLIENT>.
<u>2.5.1.1,</u> <u>2.5.2.1</u>	Signon Request <SONRQ>, <PINCHRQ>	Clarification	Reworded note on password length for <USERPASS>.
<u>2.5.1.2</u>	Signon Response <SONRS>	Correction	Removed the repetition of a sentence.
<u>2.5.1.2</u>	Signon Response <SONRS>	Clarification	Added additional usage note for <DTSERVER>.
<u>2.5.1.4</u>	Financial Institution ID <FI>	Clarification	Added information regarding use of <FI>.
<u>2.5.2</u>	USERPASS Change <PINCHRQ> <PINCHRS>	Clarification	Added information on password change and error recovery.
<u>2.5.3,</u> <u>2.5.3.1,</u> <u>2.5.3.2,</u> <u>2.5.3.3</u>	<CHALLENGERQ> <CHALLENGERS>	Correction / Clarification	Moved the <CHALLENGERQ> and <CHALLENGERS> sections under their own headings.
<u>3.1.2</u>	Format of User-Supplied Numbers	Addition / Clarification	Added text to further explain how to handle format of user-supplied text.

Section	Subject	Change Type	Change
<u>3.1.3</u>	Echoing in Responses	Addition	Added section.
<u>3.1.5</u>	Error Reporting <STATUS>	Correction	Update the reserved error codes to last 200 numbers of each 1000 error range.
<u>3.1.5</u>	Error Reporting <STATUS>	Addition	Restricted the 2000 error further. Added the following to the table: General error for which a more specific code is not available (ERROR) NOTE: Provide a more specific error whenever possible. Also added the following text to the note: <MESSAGE> should be returned whenever the <CODE> can be refined. For example, <CODE>2000 should always be accompanied with a <MESSAGE> explaining the problem.
<u>3.1.5</u>	Error Reporting <STATUS>	Addition	Added error code 2028 (requested element unknown) to status table and words explaining how unrecognized elements should be handled.
<u>3.2, 3.2.3</u>	Common Elements, Financial Institution Transaction ID <FITID>	Correction	Changed alphanumeric to character.
<u>3.2.1</u>	Client-Assigned Transaction UID <TRNUID>	Clarification	Added words to better describe <TRNUID> definition.
<u>3.2.2</u>	Server-Assigned ID <SRVRTID>, <SRVRTID2>	Clarification	Added words to better describe definition. Added words to clarify the uniqueness of <SRVRTID>.
<u>3.2.3</u>	Financial Institution Transaction ID <FITID>	Clarification	Added words to better describe definition. Added words to clarify the uniqueness of <FITID>.
<u>3.2.7</u>	Date Start and Date End <DTSTART> <DTEND>	Clarification	Enhanced usage information.
<u>3.2.8.4</u>	Time Zone Issues	Addition	Added clarification of issue dealing with difference between date and datetime when considering timezone conversions.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
<u>4.2.1.3</u>	Trusted Certificate Authorities	Clarification	Removed footnote that said approved OFX CAs would be identified at a later date. Servers should select their own CA.
<u>5.1</u>	Language and Encoding	Clarification.	Added detail on values for CHARSET.
<u>5.2</u>	Currency <CURDEF> <CURRENCY> <ORIGCURRENCY>	Correction	Corrected type for <CURSYM> (currsymbol).
<u>Chapter 6</u>	Data Synchronization	Clarification	Many editorial changes to improve description of synchronization options. Added level 3 headings and clarified text. Added central <SYNCERROR> table with cross-references to it in later chapters. Also added warnings about use of <SYNCERROR> with <MAILSYNCRS>, <PMTMAILSYNCRS>, and <PAYEESYNCRS>. Added section on “Synchronizable Objects.” Included table of SYNCRQ/SYNCRS pairs with cross-references to definitions.
<u>6.4</u>	Data Synchronization Specifics	Clarification	Added level 3 headings and clarified text. Added central <SYNCERROR> table with cross-references to it in later chapters. Also added warnings about use of <SYNCERROR> with <MAILSYNCRS>, <PMTMAILSYNCRS>, and <PAYEESYNCRS>. Added section on “Synchronizable Objects.” Included table of SYNCRQ/SYNCRS pairs with cross-references to definitions.
<u>6.4.1</u>	Tokens	Clarification	Added words to describe the cases in which to use <REFRESH>Y and <TOKEN>0.
<u>6.4.1</u>	Tokens	Clarification	In sync requests which do not include an account number, token values are scoped to the current user.

Section	Subject	Change Type	Change
<u>6.4.4</u>	<SYNCERROR> Status Codes	Addition / Clarification	Added or clarified error codes 6500, 6501, 6502, and 15500.
<u>6.5</u>	Conflict Detection and Resolution	Clarifications	Minor edits for clarification.
<u>6.6</u>	Synchronization Options	Correction	Changed title.
<u>6.6</u>	Synchronization Options	Clarifications	Edits for clarification. Added Section 6.6.1 , "Synchronization Errors" and Section 6.10.1.1 , "File Based Error Recovery and Authentication."
<u>6.6</u>	Synchronization Options	Clarification	Servers must send responses that emulate a client creating or adding each of the objects governed by the particular synchronization request.
<u>6.10</u> <u>6.10.2</u> <u>6.10.3</u>	File-based error recovery	Clarification	Substituted several different, equivalent terms with one, consistent term.
<u>6.10.1</u>	File-Based Error Recovery	Clarification	Edits to improve discussion of when to save files. Also improved discussion of NEWFILEUID and OLDFILEUID. <i>This change supersedes previous changes (for OFX 1.5) to this section.</i>
<u>6.10.1</u>	File-Based Error Recovery	Correction / Addition	Added a new section that explains the difference between basic lite synchronization servers and Refresh-capable lite synchronization servers. This change supersedes previous changes (for OFX 1.5) to this section.
<u>6.10.2.1</u>	Lite Synchronization and <REFRESH>	Addition	Added section.
<u>6.10.3</u>	Relating Synchronization and Error Recovery	Addition	Third bullet now reads: Lite synchronization with file-based error recovery (with or without <REFRESH>Y support)
<u>7.1.1</u>	Message Sets	Clarification	Added text explaining how clients and servers should handle unsupported transactions or unknown elements or aggregates.

Section	Subject	Change Type	Change
<u>7.1.2</u>	Version Control	Clarification	Clarified use of URL in different message set versions.
<u>7.1.3</u>	Batching and Routing	Clarification	Added text stating all conditions listed in bullets must be true.
<u>7.1.3</u>	Batching and Routing	Clarification	Added text to clarify use of the signon message set with other message sets.
<u>7.1.4</u>	Client Signon for Profile Requests	Clarification	Clarification of profile response.
<u>7.2</u>	Profile Response <PROFRS>	Addition	Noted that not sending <PROFRS> for success was an exception to the rules stated in sections <u>2.4.6</u> and <u>3.1.5</u> .
<u>7.2</u>	Profile Response <PROFRS>	Clarification	Added usage note. Added recommendation against empty <SIGNONINFOLIST>.
<u>7.2.1</u>	Message Set	Clarification	Substituted several different, equivalent terms with one, consistent term: file-based error recovery
<u>7.2.1</u>	Message Set	Correction, DTD Change	Improved wording of various usage notes in this section to better correspond to the features supported by the DTD. <i>Part of this update also affects the DTD for OFX 1.5.1 to preserve compatibility between OFX 1.0.2 and all later versions.</i>
<u>7.2.1</u>	Message Set	Addition, DTD Change	Added the <REFRESHSUPT> element to the Message Set Core table. <i>While this element does not appear in earlier versions of the DTD, clients and servers implementing OFX 1.0.2 or 1.5.1 may utilize the feature. Such newly-implemented applications should use the latest DTD. OFX 1.0.2 software would ignore the “V2” message sets in their implementation.</i>
<u>7.2.2</u>	Signon Realms	Correction	Changed <USERPASSPIN> to <PINCHRQ>

Section	Subject	Change Type	Change
<u>7.2.2</u>	Signon Realms	Correction	Changed the word “change” to “execute” in the usage note for <CHGPINFIRST>. Added requirement for clients to ignore when <PINCH>N is specified in the profile.
<u>7.2.2</u>	Signon Realms	Correction	Added line defining default value in the FIXED portion of table for <PWTYPE>. Also corrected minor display issue with the table.
<u>7.2.2</u>	Signon Realms	Addition	Added detail about valid <CHARTYPE> types and relationship between <CHARTYPE>, <SPECIAL> and <SPACES> profile elements.
<u>8.1</u>	Overview	Clarification	Made edits to final paragraph.
<u>8.1.1</u>	Sign-Up Message Set V2 and Proxy On-Behalf-Of Actions	Clarification	Added section subsections defining <SIGNUPMSGSRQV2> and <SIGNUPMSGSRSV2>.
<u>8.1.1.1</u>	Sign-Up Message Set Wrapper Request	Addition, DTD Change	Added <USERID> to <SIGNUPMSGSRQV2>.
<u>8.1.1.2</u>	Sign-Up Message Set Wrapper Response	Addition, DTD Change	Added <USERID> to <SIGNUPMSGSRSV2>.
<u>8.2.1</u>	Multiple User Enrollment and Activation	Addition	Added section
<u>8.2.2</u>	Rapid Signup	Addition	Added section
<u>8.4.2</u>	Enrollment Request <ENROLLRQ>	Correction	Changed the word “pick” to “choose” in <USERID> usage note.
<u>8.4.3</u>	Enrollment Response <ENROLLRS>	Correction	Changed Status code 10 to 13000 in first paragraph.
<u>8.5</u>	Account Information	Addition	Added the following text: <ACCTINFORS> should not be sent when the client is up-to-date. Also added a note stating this is an exception to the rules as stated in <u>2.4.6</u> and <u>3.1.5</u> .
<u>8.5.1</u>	Request <ACCTINFORQ>	Addition, DTD Change	Added <SVC2> to the ACCTINFORQ table.
<u>8.5.2</u>	Response <ACCTINFORS>	Addition	Added usage notes.

Section	Subject	Change Type	Change
<u>8.6.4</u>	Service Activation Synchronization	Addition	Added syntax tables for synchronization request and response.
<u>8.7.4</u>	Change User Information Synchronization	Addition	Added syntax tables for synchronization request and response.
<u>8.8</u>	Signup Message Set Profile Information	Addition, DTD Change	Added <CANSUPPORTUSERID>.
<u>9.2</u>	E-Mail Messages	Addition	Added words stating preference of service-specific messages.
<u>9.2.1</u>	Regular vs. Specialized E-Mail	Clarification	Corrected typographic error, changing “for140 clients” to “for OFX clients.”
<u>9.2.2</u>	Basic <MAIL> Aggregate	Clarification	Add words explaining that <MSGBODY> always needs an end tag.
<u>9.2.2.2</u>	<USEHTML>	Clarification	Added clarification of server processing of <USEHTML>.
<u>9.2.4</u>	E-Mail Synchronization <MAILSYNCRQ> <MAILSYNCRS>	Clarification	Added cross-reference to <SYNCEERROR> Status Codes in 6.4.4.
<u>9.3</u>	Get HTML Page	Clarification	Added paragraphs describing use of multipart MIME files.
<u>9.3.2</u>	MIME Example	Correction / Addition	Changed the version number in the example to 160. Also added CHARSET, COMPRESSION, NEWIFLUID, and OLDFILEUID to the examples.
<u>9.3.1</u>	MIME Get Request and Response <GETMIMERQ> <GETMIMERS>	Clarification	Added “multipart” to: “The actual response, whether HTML, an image, or some other type, is always sent as a separate part of the file using multipart MIME.”
<u>9.3.2</u>	MIME Example	Correction	Updated example.
<u>9.4</u>	E-Mail Message Set Profile Information	Addition	Add words to say that if email message set is supported by the server, they must support <MAILSYNCRQ>.

Section	Subject	Change Type	Change
<u>9.4</u>	E-Mail Message Set Profile Information	Correction	Added clarification concerning <MAILRQ> and <MAILSYNCRQ> support to usage notes.
<u>10.3</u>	Retrieving Transactions Generated by a Recurring Model	Clarification	Clarified status as pending.
<u>10.5</u>	Modifying and Canceling Recurring Models	Clarification	Added information on synchronization requirements.
<u>10.5.1</u>	Examples	Correction	Updated examples to show that requests are not guaranteed to be processed in a specific order. Made other general improvements to examples.
<u>10.6</u>	Expired Models	Addition	Added section.
<u>11</u>	Banking	Change	Removed <SYNCERROR> tables and added reference to central table in Chapter 6.
<u>11.4.1</u>	Bank Statement Download	Addition	Added processing requirements for a statement download request that does not contain <DTSTART> or <DTEND>.
<u>11.4.2</u>	Credit Card Statement Download	Addition	Added processing requirements for a statement download request that does not contain <DTSTART> or <DTEND>.
<u>11.4.3</u>	Statement Transaction <STMTRN>	Addition, DTD Change	Added <SPNAME>.
<u>11.4.3</u>	Statement Transaction <STMTRN>	Clarification	Added words stating that <TRNTYPE> does not effect account balance.
<u>11.4.3.1</u>	Transaction types used in <TRNTYPE>	Addition	Added requirement to use these transaction types for transactions generated by a recurring model and out of band.
<u>11.7</u>	Intrabank Funds Transfer	Clarification	Clarified synchronization rules.
<u>11.7.1.2</u>	Response <INTRARS>	Addition	Added information regarding use of <STATUS> aggregate.

Section	Subject	Change Type	Change
<u>11.7.2</u>	Intrabank Funds Transfer Modification	Clarification	Added some text to clarify the server support of modifications. Also added error 10505 to the status table to support the new text in this section.
<u>11.7.3.4</u>	DTDUE, DTPOSTED, and DTXFERPRJ in Immediate Transfers	Addition	Added section.
<u>11.8.3</u>	Interbank Funds Transfer Modification	Clarification	Added some text to clarify the server support of modifications.
<u>11.8.5.2</u>	Multiple Interbank Funds Transfer Response <MULTIINTERTRNRS>	Addition, DTD Change	Change the words “one or more” to “zero or more” in both text before table and usage note for <INTERRS>. <i>This change applies retroactively to OFX 1.5.1 because it corrects an important bug in that version.</i>
<u>11.8.5.2</u>	Multiple Interbank Funds Transfer Response <MULTIINTERTRNRS>	Clarification	Added a note before the table to indicate that responses are optional in error cases.
<u>11.10.4.2</u>	Response <RECINTERRS>	Addition	For version 1 of the message set, the <SRVRTID> included in the <INTRARS> should be set to the same value as the <RECSRVRTID>.
<u>11.12.2</u>	Data Synchronization for Intrabank Funds Transfers	Addition	Added introductory paragraph explaining requirements.
<u>11.12.3</u>	Data Synchronization for Interbank Funds Transfers	Added	Clarified transfers and sync.
<u>11.14.4</u>	Recurring Transfers	Correction	Updated examples to show that requests are not guaranteed to be processed in a specific order. Made other general improvements to examples.
<u>12</u>	Payments	Change	Updated comments for <DTXFERPRJ> and <DTXFERPRC> to be more descriptive. Removed <SYNCERROR> tables and added reference to central table in Chapter 6.

Section	Subject	Change Type	Change
<u>12.2.4</u>	Identifying Payees	Clarification	Clarified implicit payee behavior, particularly differences between implicit payee behavior in V1 and V2.
<u>12.2.5</u>	Side Effects of Payee Adds and Modifications	Addition	Added section.
<u>12.3</u>	Identifiers Used in Payment Transactions	Addition	Added text to clarify the scoping of <PAYEELSTID>s.
<u>12.5.2</u>	Credit Mail Order / Telephone Order Account <CCMOTOACCT>	Addition, DTD Change	Added <CCMOTOACCT> Table. Only required elements are <CCACCTFROM> and <DTEXPIRE>.
<u>12.5.3</u>	Payment Information <PMTINFO> <PMTINFO2>	Addition, DTD Change	Added <CCMOTOACCT> to <PMTINFO2>.
<u>12.5.3</u>	Payment Information <PMTINFO> <PMTINFO2>	Addition	Added the following paragraph: In the case of an implicit add, the returned <PMTINFO> aggregate found in <PMTRS> and <RECPMTRS> must include the generated <PAYEELSTID>. This aggregate may also include <EXTDPMT> information.
<u>12.5.3</u>	Payment Information <PMTINFO> <PMTINFO2>	Addition, DTD Change	Added <SPNAME> and <BILLID> to <PMTINFO2>.
<u>12.5.3</u>	Payment Information <PMTINFO> <PMTINFO2>	Correction	Changed <PAYEE> to <PAYEE2> in <BANKACCTTO> usage note.
<u>12.5.3.2</u>	Extended Payment <EXTDPMT>	Addition, DTD Change	Changed the payment OR inclusion construct from reading “Either or both of” to “At least one of”. Added the </EXTDPMTDSC> tag to the table and a note explaining that it is required. Also added note stating that the end tag for <EXTDPMTDSC2> is not required. <i>The change to once again make </EXTDPMTDSC> required applies retroactively to OFX 1.5.1 to preserve compatibility between OFX 1.0.2 and all later versions</i>
<u>12.5.3.6</u>	Extended Payee <EXTDPAYEE>	Clarification	Updated usage notes for <PAYEEID> and <PAYEEID2>.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
<u>12.6.1.1</u>	Payment Request <PMTRQ>	Clarification	Added note describing implicit changes and required behavior.
<u>12.6.1.2</u>	Payment Response <PMTRS>	Correction	Added explanatory notes.
<u>12.6.1.4</u>	Discussion	Clarification	Added sentences to clarify differences between implicit payee behavior in V1 and V2.
<u>12.6.2</u>	Payment Modification	Clarification	Added some text to clarify the server support of modifications.
<u>12.6.2.2</u>	Payment Modification Request <PMTMODRQ>	Clarification	Added text to clarify how the client may modify data in <PMTINFO>.
<u>12.6.3</u>	Payment Cancellation	Correction	Clarified in which cases <PMTCANCERS> should be used.
<u>12.7</u>	Recurring Payments	Clarification	Implicit modification requires information returned in subsequent sync request. Clarification of <MODELWND>. Added notes.
<u>12.7.1.1</u>	Request <RECPMTRQ>	Addition	Added information regarding implicit additions or modifications.
<u>12.7.1.2</u>	Response <RECPMTRS>	Addition	Added a note stating that the <PAYEELSTID> must match that returned in <PMTINFO>.
<u>12.7.2.1</u>	Request <RECPMTMODRQ>	Clarification	Added text to clarify how the client may modify data in <PMTINFO>.
<u>12.8</u>	Payment Mail	Addition	Added note regarding non-support of payment e-mail.
<u>12.9</u>	Payee Lists	Clarification	Added words saying the Payee list is subject to synchronization.
<u>12.9.2</u>	Payee Modification	Clarification	Replaced ambiguous “Recurring payments created after” with “Payments spawned from a model after”
<u>12.9.2.1</u>	Request <PAYEEMODRQ>	Addition, DTD Change	Added <MODPENDING>.
<u>12.9.2.1</u>	Request <PAYEEMODRQ>	Clarification	Added information regarding changes to payee information.
<u>12.9.2.2</u>	Response <PAYEEMODRS>	Addition, DTD Change	Added <MODPENDING>.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
<u>12.9.4</u>	Payee List Synchronization	Clarification	Deleted a sentence and added words pertaining to getting the latest version of payee list.
<u>12.10</u>	Data Synchronization for Payments	Clarification	Added note regarding payee information and synchronization.
<u>12.10.3</u>	Discussion	Clarification	Minor text edits.
<u>12.11.1.1</u>	Bill Pay Message Set Request Messages	Addition	<BILLPAYMSGSRQV1> and <BILLPAYMSGSRQV2> should not be empty. Transactions within these messages can be executed in any order.
<u>12.11.2</u>	Bill Pay Message Set Profile <BILLPAYMSGSET>	Addition	Added usage note for <MODELWND>.
<u>12.11.2</u>	Bill Pay Message Set Profile <BILLPAYMSGSET>	Addition, DTD Change	Added <CANMOTO> and <PAYEEMODPENDING>.
<u>12.12</u>	Examples	Correction	Updated examples to show that requests are not guaranteed to be processed in a specific order. Made other general improvements to examples.
<u>13</u>	Investments		Added references to <SYNCERROR> table in Chapter 6.
<u>13.7.2.2.2</u>	<u>Security List Message Set Response Messages</u>	Clarification	Added note discouraging empty security list response wrapper.
<u>13.8.4</u>	<u>Security List <SECLIST></u>	Clarification	Added note discouraging empty security list wrapper.
<u>13.9</u>	Investment Statement Download	Correction	Deleted the sentence referring to how often a client can retrieve statement data. Also changed second sentence in second paragraph to read: “By using <FITID> values supplied by FIs, OFX . . .”
<u>13.9.2.2</u>	Investment Statement Response <INVSTMTRS>	Clarification	Clarified usage notes for <INVPOSLIST> and <INVOOLIST>.
<u>13.9.2.4.2</u>	Transaction Aggregate Elements	Clarification	Updated usage notes for <REVERSALFEES> and <REVERSALFITID> to include reference to section 13.9.2.4.10.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
<u>13.9.2.4.3</u> <u>13.9.2.4.4</u>	<u>Investment Buy / Sell Aggregates</u> <u><INVBUY>/<INVSELL>.</u> <u>Investment Transaction</u> <u>Aggregates</u>	Clarification	Added to description of <CURRENCY> and <ORIGCURRENCY>: Though the DTD allows <CURRENCY> and <ORIGCURRENCY> together in this aggregate, servers should return neither or one of the two, but not both.
<u>13.9.2.7</u>	Investment Balances <INVBAL>	Clarification	Added text saying <SHORTBALANCE> is a positive balance.
<u>14</u>	Bill Presentment	Correction	Changed all <BILLID> data types to A-32. Added references to <SYNCERROR> table in Chapter 6.
<u>14.2.1</u>	Client Signon to the Biller Directory Server	Correction	Updated text.
<u>14.2.2.</u> <u>14.2.4</u>	Search Arguments, Find Biller Request <FINDBILLERRQ>	Addition	Added text to explain use of case-insensitive searches.
<u>14.2.4</u>	Find Biller Request <FINDBILLERRQ>	Clarification	Added clarification of use of <DTUPDATE>. Added note about future use of address elements.
<u>14.2.5.1</u>	Biller Information <BILLERINFO>	Clarification	Added note about future use of address elements.
<u>14.2.7</u>	Account Number Validation	Correction	In item 1, changed Passes to Passed.
<u>14.2.8</u>	Biller Payment Restrictions	Correction	Moved this section from 14.3.5 to 14.2.8.
<u>14.2.8.3</u>	Payment Instrument Types <PMTINSTRUMENTTYPE>	Correction	Changed CONCENTRATOR description.
<u>14.3.2.2</u>	Account Identification <PRESACCTFROM> <PRESACCTTO>	Addition, DTD Change	Added <BILLERNAME>, <SPNAME>, <PAYEEID2>, and <PAYEELSTID2> to <PRESACCTFROM>. Added usage information above table.
<u>14.3.2.2</u>	Account Identification <PRESACCTFROM> <PRESACCTTO>	Clarification	For <PRESACCTFROM>, <USERID> does not identify individual users of joint accounts.
<u>14.3.2.2</u>	Account Identification <PRESACCTFROM> <PRESACCTTO>	Correction	Data type for <ACCTID> is A-22.

Section	Subject	Change Type	Change
<u>14.3.2.2.1</u>	Customer Information with the biller <PRESNAMEADDRESS>	Addition, DTD Change	Added <BUSNAMEACCTHELD>.
<u>14.3.2.2.1</u>	Customer Information with the biller <PRESNAMEADDRESS>	Clarification	Added note about future use of address elements.
<u>14.3.4</u>	Service Status Update for Groups of Customers	Addition	Explained to use signup message set for single users.
<u>14.3.4</u>	Service Status Update for Groups of Customers	Addition	Added the following text to the last paragraph: This method applies only where <SVC2> is PRESSVC.
<u>14.3.4.1</u>	Account Information Request <ACCTINFORQ>	Addition, DTD Change	Added <SVC2>.
<u>14.3.4.2</u>	Account Information Response <ACCTINFORS>	Addition	Added usage note.
<u>14.3.4.3</u>	Group Account Information Transaction Request <PRESGRPACCTINFOTRNRQ>	Addition	Added usage note regarding single customer status checks. Added usage note and reference for <ACCTINFORQ>.
<u>14.3.4.4</u>	Group Account Information Transaction Response <PRESGRPACCTINFOTRNRS>	Addition	Added the following text: <ACCTINFORS> should not be sent when the client is up-to-date. Also added a note stating this is an exception to the rules as stated in <u>2.4.6</u> and <u>3.1.5</u> . <USERID> does not identify individual users of joint accounts. Added reference to usage note for <ACCTINFORS>.
<u>14.4.2.1</u>	Bill List Request <PRESLISTRQ>	Addition, DTD Change	Added <DTDUEBY>, <BILLERID>, <BILLTYPE>, <BILLSTATUSCODE>, <BILLPMTSTATUSCODE>, <INCLUDEBILLSTATUS>, <INCLUDEBILLPMTSTATUS>, <INCLUDESTATUSHIST>, <INCLUDECOUNTS>, <INCLUDESUMMARY>.
<u>14.4.2.1</u>	Bill List Request <PRESLISTRQ>	Addition	Added text to explain the date range.
<u>14.4.2.2</u>	Bill List Response <PRESLISTRS>	Clarification	Added preferred behavior when no bills to return

Section	Subject	Change Type	Change
<u>14.4.2.2</u>	Bill List Response <PRESLISTRS>	Addition	Added information on using multiple selection criteria.
<u>14.4.2.2</u>	Bill List Response <PRESLISTRS>	Clarification	Updated <USERID> usage note.
<u>14.4.2.2</u>	Bill List Response <PRESLISTRS>	Addition, DTD Change	Added <PRESCOUNTS>.
<u>14.4.2.2.1</u>	Bill List <PRESLIST>	Clarification	Added note discouraging use of empty <PRESLIST>.
<u>14.4.2.2.2</u>	Bill Information <PRESBILLINFO>	Clarification	Added words to explain that the <BILLPUB> and <BILLID> combination must be globally unique.
<u>14.4.2.2.2</u>	Bill Information <PRESBILLINFO>	Correction	Changed the usage note for <INVOICE2> from “Optional INVOICE2 tag that” to “Optional invoice data that”.
<u>14.4.2.2.2</u>	Bill Information <PRESBILLINFO>	Addition, DTD Change	Added <BILLTYPE>, <BILLSTATUS>, and <BILLPMTSTATUS>.
<u>14.4.2.2.3</u>	Bill Status <BILLSTATUS>	Addition, DTD Change	Added <BILLSTATUS> aggregate.
<u>14.4.2.2.4</u>	Bill Payment Status <BILLPMTSTATUS>	Addition, DTD Change	Added <BILLPMTSTATUS> aggregate.
<u>14.4.2.2.5</u>	Statement Image <STMNTIMAGE>	Clarification	Added note about use of repeating <PREFETCHURL> element with earlier OFX 1.5.1 clients.
<u>14.4.2.3</u>	Bill List Transaction Request <PRESLISTTRNRQ>	Addition	Individual customers are distinguished using the <USERID> element of each <PRESACCTFROM> in the <PRESBILLINFO> aggregates. <USERID> does not identify individual users of joint accounts.
<u>14.4.5</u>	Delivery Notification	Addition	Added a note on the Bill Status Modify transaction.
<u>14.4.5.2</u>	Delivery Notification Response <PRESNOTIFYRS>	Addition, DTD Change	Made the <PRESDeliveryID> element required and non-repeating. <i>This change applies retroactively to OFX 1.5.1 because it corrects an important bug in that version.</i>

Section	Subject	Change Type	Change
<u>14.4.6</u>	Bill Status Modification	Addition, DTD Change	Added <BILLSTATUSMODRQ> and <BILLSTATUSMODRS> messages.
<u>14.7.2</u>	Biller Directory Message Set Profile	Clarification	Updated <PRESDIRMSGSETV1> usage description to allow one or more.
<u>14.7.3</u>	Bill Delivery Message Set Profile, <PRESDLVPROF>	Clarification	Updated <PRESDIRMSGSETV1> usage description to allow one or more.
<u>14.7.3</u>	Bill Delivery Message Set Profile, <PRESDLVPROF>	Clarification	Clarified use of <CANSUPPORTGROUPID>.
<u>14.7.3</u>	Bill Delivery Message Set Profile, <PRESDLVPROF>	Addition, DTD Change	Added <CANMODSTATUS>.
<u>14.8.1</u>	Find Biller Examples	Correction	Added note stating that enrollment may have already occurred and the transaction does not require anonymous sign on. Updated examples to include wrappers (transactions, message sets, signon and <OFX>). Corrected <ACCTEDITMASK>.
<u>14.8.2.1</u>	Enrollment Request	Correction	Corrected the example to use the correct anonymous password
<u>Appendix A, "Status Codes"</u>	Version	Addition	Added version column to table.
<u>Appendix A, "Status Codes"</u>	Many	Addition	Added text clarifying the (expanded) <MESSAGE> information which should be sent to an older client. Utilizes the new "Version" column in the status code table.
<u>Appendix A, "Status Codes"</u>	Error code 2000	Clarification	Added note spelling out preference to use a more specific error code.
<u>Appendix A, "Status Codes"</u>	Error codes 0, 2016, 2020, 2027, 2028, 6500, 6501, 6502, 10514, 10520, 10521, and 10522, 13504, 15508	Addition / Clarification	Added errors or clarified description of these errors in the status tables. Clarified difference between 2016 and 10514.

B.5.2 DTD Changes

The following table lists DTD changes that are not covered in the text of the specification. Other OFX 1.6 DTD changes are called out in the table in section B.5.1.

<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
<MSGSETLIST>	Correction	Made contents required (once only). %MSGSETMACRO was allowed zero or more times within <MSGSETLIST>. <i>This change applies retroactively to OFX 1.0.2 and to OFX 1.5.1.</i>
<EXTDPMTDSC>	Correction	Made end tag required to ensure compatibility with OFX 1.0.2 clients and servers. <i>This change applies retroactively to OFX 1.5.1.</i>
<PREFETCHURL>	Correction	Made this element of <STMNTIMAGE> optional and repeating. It previously was optional, contradicting the intent of this element and the text in section <u>14.4.2.2.5</u> of the specification. <i>This change applies retroactively to OFX 1.5.1.</i>
<PWTYPE>	Correction	An error in the DTD previously linked this element to <PROFMSGSETV2> instead of <PROFMSGSRSV2>. This element cannot appear within the <PROFMSGSETV2> wrapper. <i>This change applies retroactively to OFX 1.5.1.</i>

APPENDIX C RECENT CHANGES THAT AFFECT EARLIER VERSIONS

This appendix lists feature changes made in OFX 1.5, 1.5.1, and 1.6 that apply to OFX 1.0.2. Servers and clients that support OFX 1.0.2 must support these changes.

This appendix does not list all editorial clarifications that might also be of use to OFX 1.0.2 developers. Editorial corrections are listed in [Appendix B, "Change History."](#)

This appendix also lists DTD changes made in recent versions that affect OFX 1.0.2. These changes allow 1.0.2 developers to use the 1.5.1 or 1.6 DTD's which are each contained in a single file, rather than several distributed files.

C.1 Changes Made in OFX 1.5

C.1.1 Specification Changes by Chapter

Section	Subject	Change Type	Change
1.2.2	White space following or preceding a tag delimiter	Addition	If white space is desired preceding or following an element value, this is achieved using the CDATA wrapper or, in version 2 message sets, the macro.
2	Proper use of end of line characters	Addition	A proper client should separate the components of an OFX request using a single CRLF between each component
2	Server tolerance of bad clients	Addition	Whilst it is seen appropriate for testing parsers to check full conformance to this specification, it is recommended that operational parsers be tolerant of deviations.
2.2	All OFX headers are required	Addition	All OFX headers are required. NONE should be returned if client or server does not make use of an individual tag.
2.4.5.2	Message Set Ordering	Addition	Ordering amongst message set versions explained.
2.5.1	Signon	Clarification	Authentication of the SONRQ is always required, even in error recovery. Servers must accept a GENUSEKEY in a SONRQ. Anonymous logons explained. Discussion of "empty" sign-on transactions.

Section	Subject	Change Type	Change
3.1.4	<BAL>	Correction	VALUE should be of type Amount, rather than N-20.
3.2.8	Dates and Times	Clarification	Valid gmt offset values are in the range from -12 to +12 for whole number offsets. Formatting is +12.00 to -12.00 for fractional offsets, plus sign may be omitted.
3.2.8.2	Date and Datetime	Clarification	Clarification of timezone offset formatting
3.2.8.4	Time Zone Issues	Addition	Suggested behavior for clients not knowing the current local time zone.
3.2.9	Amount	Clarification	Trailing and leading spaces should be stripped. Number format uses a leading sign. Negative number format uses a minus sign (-).
3.2.9.1	Amount, Prices, and Quantities – Basic Format	Clarification Addition	Leading and trailing spaces should be stripped.
3.2.9.2	Positive and Negative signs	Correction	Except for cases where the flow of funds is unclear, there would be no signage.
5.1	ENCODING	Change	ENCODING now accepts UTF-8 as a valid value, instead of UNICODE
6.4	Data Synchronization	Clarification	Servers should return TOKEN=-1 if they must respond with an error
6.6	Client Synchronization	Clarification	All xxxSYNCRQ are required to contain one, and only one, of <TOKEN>, <TOKENONLY>, or <REFRESH>.
6.10	Lite synchronization and Refresh	Clarification	Lite synchronization servers do not support Refresh. <TOKEN>0 is not the same as Refresh synchronization. Therefore, clients should not send Refresh to a lite synchronization server.
6.10.1	Lite Synchronization	Addition	Suggested behavior when NEWFILEUID matches a previous request file Note: Open Financial Exchange requires a server to authenticate a client in Error Recovery
7.1.3	Batching and Routing	Clarification	Supporting a message set using multiple servers.
7.1.4	Client Signon for Profile Requests	Clarification	Suggested text for anonymous logon.

Section	Subject	Change Type	Change
7.2	Signon Information in the profile response	Clarification	The DTD is correct that zero or more <SIGNONINFO> aggregates can be returned in a <PROFRS>. The specification incorrectly stated 1 or more.
7.2.1	Tag order in <MSGSETCORE>	Clarification	The specification incorrectly ordered the tags within the <MSGSETCORE> aggregate. The DTD correctly ordered the tags with the <SPNAME> element last.
7.2.1	<MSGSETCORE>	Clarification	LANGUAGE can be used 1 or more times
7.2.1	<MSGSETCORE>	Clarification	The VER / URL pair must be unique for each instance of MSGSETCORE
8.4	Enrollment and Password Acquisition	Clarification	Suggested text for anonymous logon
8.6.1.3	<SVCADD>	Addition	Explanatory text on the use of SVCADD.
8.7	<CHGUSERINFORQ>, <CHGUSERINFORS>	Addition	All elements must be submitted in a CHGUSERINFORQ. Lack of inclusion implies its deletion on the server.
11.3.2	<CCACCTFROM>	Clarification	<CCACCTFROM> can be used even if the credit card message set is not supported.
11.7.1.2	<INTRARS>	Clarification	The DTD does not require either <DTXFERPRJ> or <DTPOSTED> in an intrabank, interbank or wire transfer response.
11.8.2.2	<INTERRS>		
11.9.1.2	<WIRERS>	Clarification	The DTD does not require either <DTXFERPRJ> or <DTPOSTED> in an intrabank, interbank or wire transfer response.
11.10.1.2	<RECINTRARS>	Clarification	The <SRVRTID> included in the <INTRARS> should be set to the same value as the <RECSRVRTID>. NOTE this is the response to the recurring model only. Servers must still generate a <INTRARS> for each instance of the recurring transfer.
11.12.2.2	<INTRASYNCRS>	Clarification	<INTRASYNCRS> Response contain only intrabank transfers where the <BANKACCTFROM> of the Sync response matches the <BANKACCTFROM> of the <INTRARS>.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
11.12.3.2	<INTERSYNCRS>	Clarification	<INTERSYNCRS> Response contain only interbank transfers where the <BANKACCTFROM> of the Sync response matches the <BANKACCTFROM> of the <INTERRS>.
11.13.4	<INTERXFERMSGSET>	Correction	DOMXFERFEE and INTLXFERFEE should be of type Amount, rather than N-8.
11.13.5	<WIREXFERMSGSET>	Correction	DOMXFERFEE and INTLXFERFEE should be of type Amount, rather than N-8.
12.2.4	Identifying Payees	Clarification	Servers must always assign PAYEELSTIDs to payees. Once PAYEELSTIDs have been assigned, clients must always send the <PAYEELSTID>, even if a Payee has both a <PAYEEID> and a <PAYEELSTID>.
12.4.1	Implicit requests	Clarification	Servers should generate explicit responses to implicit requests. In other words, implicit payee additions or modifications resulting from a new or changed payment should generate explicit payee add or payee change responses from the server. Such explicit responses are only returned to the client in a sync response.
12.4.1	Payment Discussion	Clarification	Payment transactions that also contain an implicit payee transaction (change or add) are considered a single unit of work. A failure of either causes a failure in both. This is true for payment creation, modification and deletion transactions.
12.5.2	Payee Info	Clarification	The DTD requires either <PAYEE> or <PAYEEID> inside <PMTINFO>. The specification incorrectly says that both are optional.
12.5.3.3	<INVOICE>	Correction	Both DSCRATE and DSCAMT are required. Previously it was stated that each was optional if the other was included.
12.5.3.6	<EXTDPAYEE>	Correction	Notes added to tag descriptions for PAYEEID, IDSCOPE, and NAME to indicate that if any of these optional tags is used, the others are then required.
12.6	Payment Functions	Clarification	Because the flow of money is unambiguous, bill payment amounts are usually specified as positive numbers.

Section	Subject	Change Type	Change
12.6.2.5	Payment Modification Discussion	Clarification	Implicit payee changes contained in a payment modification transaction affect the individual payment. The changes are also propagated to the server's payee list and affect subsequent payments to that payee. NOTE: Explicit payee changes are not propagated to pending payments.
12.7.1.4	<PMTINFO>	Addition	The <DTDUE> element of <PMTINFO> specifies payment due date or the date by which the first payment must be received by payee (see section 12.5.2).
12.7.2	Recurring Payment Modification	Clarification	Clients can modify both elements in the <RECURRINST> aggregate, i.e., <NINSTS> and <FREQ>. Clients should send the original number of payments scheduled if there is no change. If there is a change in the number of payments scheduled clients should send the new number of payments. Clients must account for pending payments and the server's ModPending flag. This can result in implicit payment deletions.
12.7.2.1	<RECPMTMODRQ>	Clarification	A <RECPMTMODRQ> that modifies pending payments via the <MODPENDING> flag is a compound transaction, and generates the appropriate explicit payment responses that reflect such modifications, which are returned to the client via synchronization.
12.7.3.1	< RECPMTCANCRQ>	Clarification	A <RECPMTCANCRQ> that cancels pending payments via the <CANPENDING> flag is a compound transaction, and generates the appropriate explicit payment responses that reflect such cancellations, which are returned to the client via synchronization.
12.9.1.1	<PAYEERQ>	Correction	One of either PAYEE or PAYEEID is required
12.9.2	Payee Modification	Clarification	Explicit payee modifications are made in the server's payee list. They are not propagated to already created pending payments. Recurring Payments created after the Payee modification will use the updated information from the server's payee list. Implicit payee modifications contained within a payment creation, modification or deletion request are propagated to the payment and to the server's payee list.

Section	Subject	Change Type	Change
12.9.2.1	<PAYEEMODRQ>	Clarification	Absence of a PAYACCT in PAYEEMODRQ could be interpreted as an implicit disassociation of the PAYACCT.
12.9.3	Payee Deletion	Clarification	Clients send the <PAYEELSTID> in the <PAYEEDELRQ> aggregate to delete an entire Payee. To modify or delete specific payee information, clients should use the <PAYEELSTID> and the <PAYACCT> elements in the <PAYEEMODRQ>.
12.11.2	<BILLPAYMSGSET>	Addition	<PROCDAYSOFF> indicate days to exclude when calculating dates that utilize other bill payment bits, such as <DAYSWITH> and <DFLTDAYSTOPAY> values.
13.3.3.	Signs	Clarification	<UNITS> and <TOTAL> are signed from the perspective of the user (positive for SELLS, negative for BUYS). All other Investment transaction amounts are always positively signed. In other words <UNITPRICE>, <COMMISSION>, <FEES>, <TAXES>, <WITHHOLDING>, <LOAD>, <MARKUP> and <MARKDOWN> are always positive numbers. A positive <COMMISSION>, <TAXES>, <LOAD>, <WITHHOLDING>, or <FEE> increased the negatively signed <TOTAL> on a <BUYSTOCK> and decreased the positively signed <TOTAL> on a <SELLSTOCK>. <MARKUP> and <MARKDOWN> increase or decrease, respectively, the <UNITPRICE>. Servers should return corrections to investment buys or sells as the opposite transaction type, e.g., a correction to a buy is returned as a sell. A correction to <MARGININTEREST> or <RETOFCAP> is returned as an <INVEXPENSE>.
13.6.1	Specifying the Investment Account	Clarification	Brokers should use the domain name of their company's URL as the <BROKERID>, e.g., If URL=www.broker.com then BROKERID=broker.com>
13.8.4	<SECLIST>	Correction	The xxxINFO security information aggregates can occur zero or more times

Section	Subject	Change Type	Change
13.9.2.4.3	Investment Aggregate	Clarification	<REINVEST> is a single transaction that contains both income and an investment transaction. If servers can't track this as a single transaction they should return an <INCOME> transaction and an <INVTRAN>. <TOTAL> and <UNITS> are signed as for an investment buy. Corrections to a <REINVEST> are signed as for an investment sell.
13.9.2.6.2	Investment Positions	Clarification	<UNITSSTREET> and <UNITSUSER> are incorrectly specified as <i>quantity</i> . They must be positive whole numbers A-32 (INTTYPE as per the DTD).
13.9.2.7	<BALLIST>	Correction	The BAL aggregate can occur zero or more times
A	Status Code 15507	Addition	Signon invalid without supporting pin change request (ERROR).

C.2 OFX 1.5 to OFX 1.5.1

C.2.1 Specification Changes by Chapter

The following table describes changes implemented in OFX 1.5.1. There are some changes to the DTD because of these corrections. Changes to the DTD are noted in the Change Type column. OFX implementors should refer to the DTD in cases of a discrepancy between the specification and the DTD; in other words, “the DTD rules.” The changes in this table have been incorporated in the OFX 1.5.1 version of this specification. If you are viewing this document online, you can click on a section number to go directly to that section.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
<u>1.4</u>	OFX Versions	Addition	Specifies OFX versions supported by this specification.
<u>2.2.3</u>	<VERSION>	Change	Updated to reflect supported versions: OFX 1.0.2 and 1.5.1.
<u>10.5</u>	Canceling Recurring Models	Correction	Models cannot be cancelled at a future date. Updated description.
<u>12.5.3</u>	<PMTINFO>	Clarification	Although <PMTINFO> allows multiple occurrences of <EXTDPMT>, it is recommended that multiple invoices be expressed using multiple occurrences of the <INVOICE> aggregate. This usage will correspond with the requirements of <PMTINFO2>.

C.2.2 DTD Changes

The corrections to the DTD files ensure that the DTD files match the specification. They do not represent changes to the specification itself.

<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
<PRESDIRPROF>, <PRESDLVPROF>	Correction	Made <PROCDAYSOFF> repeating.
<PRESDLVPROF>	Correction	Changed <GROUPID> to <CANSUPPORTGROUPID>.
<FINDBILLERTRNRS>	Correction	<FINDBILLERS> is optional.
<PRESGRPACCTINFOTRNRS>	Correction	<ACCTINFORS> is optional.
<PRESLISTTRNRS>	Correction	<PRESLISTRS> is optional.
<PRESNOTIFYTRNRS>	Correction	<PRESNOTIFYRS> is optional.
<PRESDETAILTRNRS>	Correction	<PRESDETAILRS> is optional.
<BILLTBLSTRUCTTRNRS>	Correction	<BILLTBLSTRUCTRS> is optional.
<PRESMAILTRNRS>	Correction	<PRESMAILRS> is optional.
MSGSETMACRO	Correction	<PROFMSGSET> is required.
<WIRESYNCRS>	Correction	Removed the <CCACCTFROM> aggregate from the synchronization wrapper.
<XFERPROF>	Correction	Added <NEEDTANTRANSFER> and <SUPPORTDTAVAIL> to the bottom of <XFERPROF>.
Special Characters	Correction	Added macros corresponding to the escape sequences for "<", ">", and "&".

C.3 OFX 1.5.1 to 1.6

C.3.1 Specification Changes by Chapter

Section	Subject	Change Type	Change
Many: 2.5.4 , 7.3 , 8.8 , 9.4 , 11.13.2 , 11.13.3 , 11.13.4 , 11.13.5 , 12.11.2 , 13.7.1.1 , 13.7.2.1 , 14.7.2 , 14.7.3	Version 1 Message Sets	Addition, DTD Change	All <xxxMSGSETV1> are required, <xxxMSGSETV2> are optional. <xxxMSGSETV2> message sets are now repeatable (with the exception of 2.5.4 and 7.3). <i>To ensure compatibility with OFX 1.0.2 clients, this change applies retroactively to OFX 1.5.1.</i>
Many: 7.2 , 8.4.2 , 8.7.1 , 8.7.2 , 11.9.1.1.2 , 12.5.2 , 12.5.3.1	<ADDR2>, <ADDR3>	Addition	Added text stating ADDR3 requires ADDR2. Also added text stating ADDR2 requires ADDR1 for following sections: 8.7.1 , 8.7.2 , and 12.5.3.1 (PAYEE2 only).
Many: 11.3.6 , 11.7.1.2 , 11.8.2.2 , 12.5.3.7 , 12.6.1.2	Client errors using the STATUS aggregate	Clarification / Addition	Added text to describe intended use of <STATUS>, <PMTPRCSTS>, and <XFERPRCSTS> aggregates.
Many: 11.3.5 11.9.1.2 12.5.3 12.7.1.4	<DTDUE>	Clarification	Added words to explain why a server might return a different <DTDUE> than in the request.
10.3 11.7.2 11.8.3 12.6.2 12.7.2	Spawned payments	Clarification	Spawned payments behave identically to those created by explicit client requests. Changes to the spawned payment to not affect the model.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
<u>11.10.3.1</u> <u>11.10.3.2</u> <u>11.10.6.1</u> <u>11.10.6.2</u> <u>12.7.3.1</u> <u>12.7.3.2</u>	<CANPENDING> element	Clarification	Clarified usage note.
<u>1.3</u>	Definitions	Addition	Added definitions for key concepts.
<u>1.4</u>	OFX Versions	Correction	Clarified which versions are supported and which versions and publications are obsolete.
<u>1.5</u>	Conventions	Addition	Added text to explain “1.6 add” version type.
<u>2.3.1</u>	Compliance	Clarification	Enhanced definition of unrecognized tags.
<u>2.4.3</u>	Top Level	Clarification	Added required <SONRQ> or <SONRS>.
<u>2.4.5</u>	Message Sets and Version Control	Clarification, DTD Change	Clarified rules regarding repeating message sets, message set versions within a file, message versions within a message set, message set versions in a RQ/RS file pair, and message set ordering within a file.
<u>2.5.1</u>	Signon <SONRQ> <SONRS>	Addition	Added the following text: The only present situation in this class are first time <PROFRQ> <FINDBILLERRQ>, and all <ENROLLRQ> transactions. Any request sent by the client after a successful <ENROLLRQ> response (or, out of band enrollment) for the service must provide the user’s <USERID> and <USERPASS>.
<u>2.5.1</u>	Signon <SONRQ> <SONRS>	Addition	Added note concerning length of anonymous password.
<u>2.5.1</u>	Signon <SONRQ> <SONRS>	Addition	Added paragraph regarding signon errors and the use of code 15500.
<u>2.5.1.2</u>	Signon Response <SONRS>	Clarification	Added additional usage note for <DTSERVER>.
<u>2.5.1.4</u>	Financial Institution ID <FI>	Clarification	Added information regarding use of <FI>.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
<u>2.5.2</u>	USERPASS Change <PINCHRQ> <PINCHRS>	Clarification	Added information on password change and error recovery.
<u>3.1.2</u>	Format of User-Supplied Numbers	Addition / Clarification	Added text to further explain how to handle format of user-supplied text.
<u>3.1.3</u>	Echoing in Responses	Addition	Added section.
<u>3.1.5</u>	Error Reporting <STATUS>	Correction	Update the reserved error codes to last 200 numbers of each 1000 error range.
<u>3.1.5</u>	Error Reporting <STATUS>	Addition	Restricted the 2000 error further. Added the following to the table: General error for which a more specific code is not available (ERROR) NOTE: Provide a more specific error whenever possible. Also added the following text to the note: <MESSAGE> should be returned whenever the <CODE> can be refined. For example, <CODE>2000 should always be accompanied with a <MESSAGE> explaining the problem.
<u>3.2, 3.2.3</u>	Common Elements, Financial Institution Transaction ID <FITID>	Correction	Changed alphanumeric to character.
<u>3.2.1</u>	Client-Assigned Transaction UID <TRNUID>	Clarification	Added words to better describe <TRNUID> definition.
<u>3.2.2</u>	Server-Assigned ID <SRVRTID>, <SRVRTID2>	Clarification	Added words to better describe definition. Added words to clarify the uniqueness of <SRVRTID>.
<u>3.2.3</u>	Financial Institution Transaction ID <FITID>	Clarification	Added words to better describe definition. Added words to clarify the uniqueness of <FITID>.
<u>3.2.7</u>	Date Start and Date End <DTSTART> <DTEND>	Clarification	Enhanced usage information.
<u>3.2.8.4</u>	Time Zone Issues	Addition	Added clarification of issue dealing with difference between date and datetime when considering timezone conversions.

Section	Subject	Change Type	Change
<u>4.2.1.3</u>	Trusted Certificate Authorities	Clarification	Removed footnote that said approved OFX CAs would be identified at a later date. Servers should select their own CA.
<u>5.1</u>	Language and Encoding	Clarification.	Added detail on values for CHARSET.
<u>5.2</u>	Currency <CURDEF> <CURRENCY> <ORIGCURRENCY>	Correction	Corrected type for <CURSYM> (currsymbol).
<u>6.4</u>	Data Synchronization Specifics	Clarification	Added section on "Synchronizable Objects." Included table of SYNCRQ / SYNCRS pairs with cross-references to definitions.
<u>6.4.1</u>	Tokens	Clarification	Added words to describe the cases in which to use <REFRESH>Y and <TOKEN>0.
<u>6.4.1</u>	Tokens	Clarification	In sync requests which do not include an account number, token values are scoped to the current user.
<u>6.4.4</u>	<SYNCERROR> Status Codes	Addition / Clarification	Added or clarified error code 15500.
<u>6.6</u>	Synchronization Options	Clarifications	Added Section 6.6.1 , "Synchronization Errors" and Section 6.10.1.1 , "File Based Error Recovery and Authentication."
<u>6.6</u>	Synchronization Options	Clarification	Servers must send responses that emulate a client creating or adding each of the objects governed by the particular synchronization request.
<u>6.10</u> <u>6.10.2</u> <u>6.10.3</u>	File-based error recovery	Clarification	Substituted several different, equivalent terms with one, consistent term.
<u>6.10.1</u>	File-Based Error Recovery	Clarification	Edits to improve discussion of when to save files. Also improved discussion of NEWFILEUID and OLDFILEUID. <i>This change supersedes previous changes (for OFX 1.5) to this section.</i>

Section	Subject	Change Type	Change
<u>6.10.1</u>	File-Based Error Recovery	Correction / Addition	Added a new section that explains the difference between basic lite synchronization servers and Refresh-capable lite synchronization servers. This change supersedes previous changes (for OFX 1.5) to this section.
<u>6.10.2.1</u>	Lite Synchronization and <REFRESH>	Addition	Added section.
<u>6.10.3</u>	Relating Synchronization and Error Recovery	Addition	Third bullet now reads: Lite synchronization with response-file error recovery (with or without <REFRESH>Y support)
<u>7.1.1</u>	Message Sets	Clarification	Added text explaining how clients and servers should handle unsupported transactions or unknown elements or aggregates.
<u>7.1.2</u>	Version Control	Clarification	Clarified use of URL in different message set versions.
<u>7.1.3</u>	Batching and Routing	Clarification	Added text stating all conditions listed in bullets must be true.
<u>7.1.3</u>	Batching and Routing	Clarification	Added text to clarify use of the signon message set with other message sets.
<u>7.1.4</u>	Client Signon for Profile Requests	Clarification	Clarification of profile response.
<u>7.2</u>	Profile Response <PROFRS>	Clarification	Added usage note. Added recommendation against empty <SIGNONINFOLIST>.
<u>7.2.1</u>	Message Set	Correction, DTD Change	Improved wording of various usage notes in this section to better correspond to the features supported by the DTD. <i>Part of this update also affects the DTD for OFX 1.5.1 to preserve compatibility between OFX 1.0.2 and all later versions.</i>

Section	Subject	Change Type	Change
<u>7.2.1</u>	Message Set	Addition, DTD Change	Added the <REFRESHSUPT> element to the Message Set Core table. <i>While this element does not appear in earlier versions of the DTD, clients and servers implementing OFX 1.0.2 or 1.5.1 may utilize the feature. Such newly-implemented applications should use the latest DTD. OFX 1.0.2 software would ignore the “V2” message sets in their implementation.</i>
<u>7.2.2</u>	Signon Realms	Correction	Changed <USERPASSPIN> to <PINCHRQ>
<u>7.2.2</u>	Signon Realms	Correction	Changed the word “change” to “execute” in the usage note for <CHGPINFIRST>. Added requirement for clients to ignore when <PINCH>N is specified in the profile.
<u>7.2.2</u>	Signon Realms	Correction	Added line defining default value in the FIXED portion of table for <PWTYPE>. Also corrected minor display issue with the table.
<u>7.2.2</u>	Signon Realms	Addition	Added detail about valid <CHARTYPE> types.
<u>8.4.3</u>	Enrollment Response <ENROLLRS>	Correction	Changed Status code 10 to 13000 in first paragraph.
<u>8.5</u>	Account Information	Addition	Added the following text: <ACCTINFORS> should not be sent when the client is up-to-date. Also added a note stating this is an exception to the rules as stated in <u>2.4.6</u> and <u>3.1.5</u>.
<u>8.5.2</u>	Response <ACCTINFORS>	Addition	Added usage notes.
<u>8.6.4</u>	Service Activation Synchronization	Addition	Added syntax tables for synchronization request and response.
<u>9.2</u>	E-Mail Messages	Addition	Added words stating preference of service-specific messages.

Section	Subject	Change Type	Change
<u>9.2.2</u>	Basic <MAIL> Aggregate	Clarification	Add words explaining that <MSGBODY> always needs an end tag.
<u>9.2.2.2</u>	<USEHTML>	Clarification	Added clarification of server processing of <USEHTML>.
<u>9.3</u>	Get HTML Page	Clarification	Added paragraphs describing use of multipart MIME files.
<u>9.4</u>	E-Mail Message Set Profile Information	Addition	Add words to say that if email message set is supported by the server, they must support <MAILSYNCRQ>.
<u>9.4</u>	E-Mail Message Set Profile Information	Correction	Added clarification concerning <MAILRQ> and <MAILSYNCRQ> support to usage notes.
<u>10.3</u>	Retrieving Transactions Generated by a Recurring Model	Clarification	Clarified status as pending.
<u>10.5</u>	Modifying and Canceling Recurring Models	Clarification	Added information on synchronization requirements.
<u>10.6</u>	Expired Models	Addition	Added section.
<u>11.4.1</u>	Bank Statement Download	Addition	Added processing requirements for a statement download request that does not contain <DTSTART> or <DTEND>.
<u>11.4.2</u>	Credit Card Statement Download	Addition	Added processing requirements for a statement download request that does not contain <DTSTART> or <DTEND>.
<u>11.4.3</u>	Statement Transaction <STMTRN>	Clarification	Added words stating that <TRNTYPE> does not effect account balance.
<u>11.4.3.1</u>	Transaction types used in <TRNTYPE>	Addition	Added requirement to use these transaction types for transactions generated by a recurring model and out of band.
<u>11.7</u>	Intrabank Funds Transfer	Clarification	Clarified synchronization rules.
<u>11.7.1.2</u>	Response <INTRARS>	Addition	Added information regarding use of <STATUS> aggregate.

Section	Subject	Change Type	Change
<u>11.7.2</u>	Intrabank Funds Transfer Modification	Clarification	Added some text to clarify the server support of modifications. Also added error 10505 to the status table to support the new text in this section.
<u>11.7.3.4</u>	DTDUE, DTPOSTED, and DTXFERPRJ in Immediate Transfers	Addition	Added section.
<u>11.8.3</u>	Interbank Funds Transfer Modification	Clarification	Added some text to clarify the server support of modifications.
<u>11.8.5.2</u>	Multiple Interbank Funds Transfer Response <MULTIINTERTRNRS>	Addition, DTD Change	Change the words “one or more” to “zero or more” in both text before table and usage note for <INTERRS>. <i>This change applies retroactively to OFX 1.5.1 because it corrects an important bug in that version.</i>
<u>11.10.4.2</u>	Response <RECINTERRS>	Addition	For version 1 of the message set, the <SRVRTID> included in the <INTRARS> should be set to the same value as the <RECSRVRTID>.
<u>11.12.2</u>	Data Synchronization for Intrabank Funds Transfers	Addition	Added introductory paragraph explaining requirements.
<u>11.12.3</u>	Data Synchronization for Interbank Funds Transfers	Added	Clarified transfers and sync.
<u>11.14.4</u>	Recurring Transfers	Correction	Updated examples to show that requests are not guaranteed to be processed in a specific order. Updated comments for <DTXFERPRJ> and <DTXFERPRC> to be more descriptive.
<u>12.2.4</u>	Identifying Payees	Clarification	Clarified implicit payee behavior, particularly differences between implicit payee behavior in V1 and V2.
<u>12.2.5</u>	Side Effects of Payee Adds and Modifications	Addition	Added section.
<u>12.3</u>	Identifiers Used in Payment Transactions	Addition	Added text to clarify the scoping of <PAYEELSTID>s.

Section	Subject	Change Type	Change
<u>12.5.3</u>	Payment Information <PMTINFO> <PMTINFO2>	Addition	Added the following paragraph: In the case of an implicit add, the returned <PMTINFO> aggregate found in <PMTRS> and <RECPMTRS> must include the generated <PAYEELSTID>. This aggregate may also include <EXTDPMT> information.
<u>12.5.3.2</u>	Extended Payment <EXTDPMT>	Addition, DTD Change	Added the </EXTDPMTDSC> tag to the table and a note explaining that it is required. <i>The change to once again make </EXTDPMTDSC> required applies retroactively to OFX 1.5.1 to preserve compatibility between OFX 1.0.2 and all later versions</i>
<u>12.5.3.6</u>	Extended Payee <EXTDPAYEE>	Clarification	Updated usage notes for <PAYEEID>.
<u>12.6.1.1</u>	Payment Request <PMTRQ>	Clarification	Added note describing implicit changes and required behavior.
<u>12.6.1.2</u>	Payment Response <PMTRS>	Correction	Added explanatory notes.
<u>12.6.2</u>	Payment Modification	Clarification	Added some text to clarify the server support of modifications.
<u>12.6.2.2</u>	Payment Modification Request <PMTMODRQ>	Clarification	Added text to clarify how the client may modify data in <PMTINFO>.
<u>12.6.3</u>	Payment Cancellation	Correction	Clarified in which cases <PMTCANCRS> should be used.
<u>12.7</u>	Recurring Payments	Clarification	Implicit modification requires information returned in subsequent sync request. Clarification of <MODELWND>. Added notes.
<u>12.7.1.1</u>	Request <RECPMTRQ>	Addition	Added information regarding implicit additions or modifications.
<u>12.7.1.2</u>	Response <RECPMTRS>	Addition	Added a note stating that the <PAYEELSTID> must match that returned in <PMTINFO>.
<u>12.7.2.1</u>	Request <RECPMTMODRQ>	Clarification	Added text to clarify how the client may modify data in <PMTINFO>.
<u>12.8</u>	Payment Mail	Addition	Added note regarding non-support of payment e-mail.

Section	Subject	Change Type	Change
<u>12.9</u>	Payee Lists	Clarification	Added words saying the Payee list is subject to synchronization.
<u>12.9.2</u>	Payee Modification	Clarification	Replaced ambiguous “Recurring payments created after” with “Payments spawned from a model after”
<u>12.9.2.1</u>	Request <PAYEEMODRQ>	Clarification	Added information regarding changes to payee information.
<u>12.9.4</u>	Payee List Synchronization	Clarification	Deleted a sentence and added words pertaining to getting the latest version of payee list.
<u>12.10</u>	Data Synchronization for Payments	Clarification	Added note regarding payee information and synchronization.
<u>12.11.1.1</u>	Bill Pay Message Set Request Messages	Addition	<BILLPAYMSGSRQV1> and <BILLPAYMSGSRQV2> should not be empty. Transactions within these messages can be executed in any order.
<u>12.11.2</u>	Bill Pay Message Set Profile <BILLPAYMSGSET>	Addition	Added usage note for <MODELWND>.
<u>13.7.2.2.2</u>	<u>Security List Message Set Response Messages</u>	Clarification	Added note discouraging empty security list response wrapper.
<u>13.8.4</u>	<u>Security List <SECLIST></u>	Clarification	Added note discouraging empty security list wrapper.
<u>13.9</u>	Investment Statement Download	Correction	Deleted the sentence referring to how often a client can retrieve statement data. Also changed second sentence in second paragraph to read: “By using <FITID> values supplied by FIs, OFX . . .”
<u>13.9.2.2</u>	Investment Statement Response <INVSTMTRS>	Clarification	Clarified usage notes for <INVPOSLIST> and <INVPOOLIST>.
<u>13.9.2.4.3</u> <u>13.9.2.4.4</u>	<u>Investment Buy/Sell Aggregates <INVBUY>/<INVSELL>.</u> <u>Investment Transaction Aggregates</u>	Clarification	Added to description of <CURRENCY> and <ORIGCURRENCY>: Though the DTD allows <CURRENCY> and <ORIGCURRENCY> together in this aggregate, servers should return neither or one of the two, but not both.

<i>Section</i>	<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
<u>13.9.2.7</u>	Investment Balances <INVBAL>	Clarification	Added text saying <SHORTBALANCE> is a positive balance.
<u>Appendix A, "Status Codes"</u>	Many	Addition	Added text clarifying the (expanded) <MESSAGE> information which should be sent to an older client. Utilizes the new "Version" column in the status code table.
<u>Appendix A, "Status Codes"</u>	Error code 2000	Clarification	Added note spelling out preference to use a more specific error code.

C.3.2 DTD Changes

The following table lists DTD changes that are not covered in the text of the specification. Other OFX 1.6 DTD changes are called out in the tables in sections B.5.1 and B.5.2.

<i>Subject</i>	<i>Change Type</i>	<i>Change</i>
MSGSETMACRO	Correction	<PROFMSGSET> is required. <i>While this portion of the profile response remains optional in the OFX 1.0.2 DTD, it should be returned by every compliant server.</i>
<MSGSETLIST>	Correction	Made contents required (once only). Made contents required (once only). %MSGSETMACRO was allowed zero or more times within <MSGSETLIST>. <i>This change applies retroactively to OFX 1.0.2 and to OFX 1.5.1.</i>
<WIRESYNCRS>	Correction	Removed the <CCACCTFROM> aggregate from the synchronization wrapper. <i>This change was required to correct a problem introduced in OFX 1.5. It restores compatibility between OFX 1.0.2 and 1.5.1 clients and servers</i>
<EXTDPMTDSC>	Correction	Made end tag required to ensure compatibility with OFX 1.0.2 clients and servers. <i>This change applies retroactively to OFX 1.5.1.</i>
<PWTYPE>	Correction	An error in the DTD previously linked this element to <PROFMSGSETV2> instead of <PROFMSGSRSV2>. This element cannot appear within the <PROFMSGSETV2> wrapper. <i>This change applies retroactively to OFX 1.5.1.</i>

TAG INDEX

Conventions

This index uses the conventions shown in the following table to identify different entry types.

Entry Type	Text Style
Tag definition	Bold text
Tag shown in example	<i>Italic text</i>
Tag used within body of text	Plain text

A

ACCRDINT **438**, 442, 444
ACCTBAL **490**, 491
ACCTEDITMASK 469, **470**, 471, 523, 525, 527, 620
ACCTFORMAT 469, **470**, 471, 523, 524, 527
ACCTID 20, 61, 109, 110, 137, 145, 171, 172, 175, **178**, 180, 183, 193, 292, 293, 295, 296, 297, 299, 300, 301, 304, 305, 306, 308, 309, 389, 390, 391, 392, 393, 394, 395, 396, 399, 400, 401, 402, 406, 407, 413, 459, 460, 477, 530, 531, 533, 534, 536, 540, 617
ACCTINFO 133, 135, **136**, 137, 414, 481, 539, 540, 556, 569, 599
ACCTINFORQ 134, **134**, 137, 475, 480, **480**, 480, 481, 482, 539, 556, 597, 610, 618
ACCTINFORS 134, 135, **135**, 135, 137, 413, 475, 476, 480, **481**, 481, 482, 483, 539, 540, 556, 599, 600, 610, 618, 631, 637
ACCTINFOTRNRQ 134, 137, 475, 480
ACCTINFOTRNRS 135, 137
ACCTKEY **179**, 180, 183, 193
ACCTREQUIRED 151, **152**
ACCTRQ 127, 139, **139**, 143, 145, 152, 476, 477, 478, 479, 530, 556, 575, 584

ACCTRS 141, **141**, 144, 145, 480, 531, 556, 569, 575, 584, 593
ACCTSYNCRQ 97, 143, **143**, 480
ACCTSYNCRS 97, 98, 143, **144**
ACCTTRNRQ 139, 145, 530
ACCTTRNRS 141, 145, 531
ACCTTYPE 20, 109, 110, 137, 145, 171, 172, 175, 179, 180, **182**, 292, 293, 295, 296, 297, 299, 300, 301, 304, 305, 306, 308, 309, 389, 390, 391, 392, 393, 394, 395, 396, 399, 400, 401, 402, 406, 407
ACCTTYPE2 179, 180, **182**, 182
ACTIVITY **490**, 491
ADDR1 61, 117, 130, 132, 147, 148, 233, 320, 327, 328, 389, 390, 393, 394, 395, 396, 404, 405, 468, 469, 471, 478, 522, 523, 524, 526, 528, 530, 531
ADDR2 117, 130, 147, 148, 233, 320, 327, 328, 404, 405, 468, 469, 471, 478, 524, 601, 632
ADDR3 117, 130, 147, 148, 233, 320, 327, 328, 468, 469, 471, 478, 556, 601, 632
ADJAMT **332**, 332
ADJDATE **332**, 332, 563, 569
ADJDESC **332**, 332
ADJNO **332**, 332, 569
ADJUSTMENT 330, 331, **332**, 332, 569, 595
AMTDUE **490**, 533, 534, 536
APPID 20, **49**, 292, 521, 525, 528
APPVER 20, 26, **49**, 292, 521, 525, 528
ASSETCLASS **428**, **429**, 430, 431, 463, 464
AUCTION **449**
AVAILACCTS **151**, 152, 475
AVAILBAL 87, **191**, 194, 294
AVAILCASH **453**, 462, 568
AVGCOSTBASIS **438**, 444, 445, 567, 577

B

BAL **63**, 63, 88, 436, 453, 462, 552, 580, 624
BALAMT **191**, 191, 194, 294, 562
BALCLOSE **201**, 204
BALDNLD **417**, 417
BALLIST 436, 453, **453**, 462, 592, 629

BALMIN **201**
 BALOPEN **201**, 204
 BALTYPE **63**, 63, 462, 568
 BANKACCTFROM 20, 88, 109, 110, 130, 137, 140, 171, 172, 175, **178**, 178, 184, 187, 190, 191, 199, 200, 206, 207, 214, 223, 232, 235, 242, 250, 256, 257, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 286, 292, 293, 294, 295, 296, 297, 299, 300, 304, 306, 308, 319, 321, 323, 325, 340, 351, 374, 375, 376, 377, 389, 390, 391, 392, 393, 394, 395, 396, 399, 400, 401, 402, 406, 407, 543, 558, 561, 562, 585, 586, 625, 626
 BANKACCTINFO 136, 137, **184**, 184, 558, 594
 BANKACCTTO 140, 145, **178**, 180, 181, 182, 187, 197, 233, 295, 296, 299, 300, 306, 308, 314, 321, 324, 363, 364, 366, 367, 543, 570, 585, 614
 BANKBRANCH **182**
 BANKCITY **182**
 BANKID 20, 61, 109, 110, 137, 145, 171, 172, 175, **178**, 180, 233, 292, 293, 295, 296, 297, 299, 300, 304, 305, 306, 308, 389, 390, 391, 392, 393, 394, 395, 396, 399, 400, 401, 402, 406, 407, 543
 BANKMAILRQ **256**, 256
 BANKMAILRS **257**, 257
 BANKMAILSYNCRQ 97, **273**, 273, 561
 BANKMAILSYNCRS 97, **274**, 274, 562, 594
 BANKMAILTRNRQ 256, **273**
 BANKMAILTRNRS 257, 259, 260, **274**
 BANKMSGSET **276**, 278, **286**, 286, 562, 586
 BANKMSGSETV1 42, 114, 276, 278, **286**, 286, 569
 BANKMSGSETV2 42, 277, 278, **286**, 286
 BANKMSGSRQV1 20, 40, 276, 292, 294, 296, 299, 304, **569**, 569
 BANKMSGSRQV2 39, 277
 BANKMSGSRSV1 278, 293, 295, 297, 300, 302, 304, 569
 BANKMSGSRSV2 39, 278
 BANKNAME **182**, 543
 BANKPOSTALCODE **182**
 BANKTRANLIST **191**, 194, 293
 BILLDETAILROW 500, **501**, 501, 504, 533, 534, 536, 537
 BILLDETAILTABLE 489, 491, 499, **500**, 500, 501, 502, 503, 504, 533, 536, 597, 598
 BILLDETAILTABLETYPE 498, 500, 501, **501**, 502, 503, 504, 533, 536
 BILLERID 468, 469, **469**, 477, 485, 522, 523, 524, 526, 530, 531, 533, 534, 536, 540, 598, 599, 618
 BILLERINFO 469, **469**, 469, 473, 474, 477, 522, 523, 524, 526
 BILLERINFOURL **471**
 BILLERNAME **477**, 617
 BILLID 325, 326, **485**, 490, 498, 499, 504, 506, 508, 533, 534, 536, 598, 614, 617, 619
 BILLPAYMSGSET 57, 380, 382, **384**, 384, 565, 576, 590, 595, 616, 628, 641
 BILLPAYMSGSETV1 42, 114, 315, 379, 380, 382, **384**, 565
 BILLPAYMSGSETV2 42, 315, 379, 381, 382, **385**, 595
 BILLPAYMSGSRQV1 380, 389, 391, 393, 395, 397, 398, 399, 401, 403, 404, 406, 616, 641
 BILLPAYMSGSRQV2 339, 380, 381, 616, 641
 BILLPAYMSGSRSV1 314, 339, 382, 390, 392, 394, 396, 397, 398, 400, 402, 403, 405, 406
 BILLPAYMSGSRSV2 314, 383
 BILLPMTSTATUS **491**, 493, **508**, 508, 619
 BILLPMTSTATUSCODE **486**, 488, 489, 493, 618
 BILLPMTSTATUSCOUNTS **489**
 BILLPUB 325, 469, **469**, 477, 485, 488, 490, 522, 523, 524, 526, 530, 531, 532, 533, 534, 535, 536, 538, 540, 619
 BILLREFINFO **322**, 322, 325, 490, 509, 533, 534, 536
 BILLSTATUS **491**, 492, 508, 619
 BILLSTATUSCODE **485**, 488, 489, 492, 618
 BILLSTATUSCOUNTS **488**
 BILLSTATUSMODRQ 506, **508**, 508, 520, 620
 BILLSTATUSMODRS 507, **508**, 508, 620
 BILLSTATUSMODTRNRQ 508
 BILLTBLSTRUCTRQ 503, **504**, 504
 BILLTBLSTRUCTRS **504**, 504, 505, 600, 631
 BILLTBLSTRUCTTRNRQ 504
 BILLTBLSTRUCTTRNRS 504, 600, 631

BILLTYPE **485**, 491, 618, 619
 BODY 162, 163
 BOOKINGTEXT **325**
 BPACCTINFO **319**, 319, 563, 594
 BRANCHID **178**, 180, 543
 BRAND 320, 474, **474**, 523, 525, 527
 BROKERID **413**, 459, 460, 591, 628
 BUSNAMEACCTHELD **478**, 618
 BUYDEBT **442**
 BUYMF **442**
 BUYOPT **442**
 BUYOTHER 413, **442**, 592
 BUYPOWER **453**
 BUYSTOCK **442**, 460, 591, 628
 BUYTYPE **438**, 442, 449, 461, 463

C

C 501, **501**, 504, 533, 534, 536, 537
 CALLPRICE **428**
 CALLTYPE **428**
 CANADDPAYEE **385**, 386
 CANBILLPAY **290**, 290
 CANCELWND **290**, 290
 CANEMAIL 288, **417**, 417, 520, 565, 570
 CANMODMDLS 287, **385**, 387
 CANMODPMTS **385**, 386
 CANMODSTATUS **520**, 620
 CANMODXFERS **287**
 CANMOTO 323, **387**, 616
 CANMULTI **290**
 CANNOTIFY **288**, 520
 CANPENDING 174, **245**, 245, 253, 304, 305,
 355, 356, 403, 404, 560, 589, 602, 627,
 633
 CANRECUR **287**
 CANSCHED **287**, 287, 291
 CANSUPPORTGROUPID **520**, 600, 620, 631
 CANSUPPORTIMAGES **519**, 520
 CANSUPPORTUSERID **153**, 611
 CANUPDATEPRESNAMEADDRESS 479,
520, 597, 598
 CANUSEDESC **288**
 CANUSERANGE **288**
 CASESEN **122**

CCACCTFROM 178, **183**, 183, 185, 187, 193,
 194, 202, 214, 223, 256, 257, 263, 264,
 265, 266, 269, 270, 271, 272, 273, 274,
 320, 558, 561, 562, 585, 600, 614, 625,
 631, 643
 CCACCTINFO **185**, 594
 CCACCTTO **183**, 183, 187, 197
 CCCLOSING 202, **203**, 203, 204, 575
 CCMOTOACCT **320**, 320, 323, 614
 CCSTMTENDRQ **202**, 202
 CCSTMTENDRS **202**, 202, 558
 CCSTMTENDTRNRQ 202
 CCSTMTENDTRNRS 202
 CCSTMTRQ 192, **193**, 193, 204, 558
 CCSTMTRS **194**, 194
 CCSTMTRNRQ 193
 CCSTMTRNRS 194
 CHALLENGERQ **56**, 56, 82, 84, 546, 573,
 578, 605
 CHALLENGERS **56**, 56, 82, 84, 573, 578, 605
 CHALLENGETRNRRQ 56, 578
 CHALLENGETRNRS 56, 578
 CHARTYPE **121**, 555, 570, 610, 637
 CHE.PTTACCTID **182**
 CHECKING **414**
 CHECKNUM 196, 207, 208, 259, 293, 297,
337, 345, 399, 578
 CHGPINFIRST **122**, 122, 574, 610, 637
 CHGUSERINFO **151**, 152
 CHGUSERINFORQ 146, **147**, 479, 546, 556,
 584, 625
 CHGUSERINFORS **148**, 556, 584
 CHGUSERINFOSYNCRQ 97, 146, **149**, 570
 CHGUSERINFOSYNCRS 97, 98, 146, **150**,
 570
 CHGUSERINFOTRNRRQ 146, 149, 575
 CHGUSERINFOTRNRS 146, 150, 575
 CHKANDDEB **201**
 CHKDESC 206, **207**, 207
 CHKERROR **208**
 CHKMAILRS **259**, 259
 CHKNUMEND **206**, 297, 558, 577
 CHKNUMSTART **206**, 296, 558, 577
 CHKRANGE **206**, 206, 296
 CHKSTATUS **208**, 297, 562

CITY 117, 130, 132, 147, 148, 233, 320, 327,
 328, 389, 390, 393, 394, 395, 396, 404,
 405, 468, 470, 478, 522, 523, 524, 526,
 528, 530, 531
 CLIENTACTREQ **152**, 152, 557
 CLIENTENROLL 151, **151**, 152
 CLIENTROUTING **116**
 CLOSING **200**, 200, 201, 558
 CLOSINGAVAIL **286**, 286, 289
 CLOSUREOPT **442**, 567
 CLTCOOKIE 43, 44, 45, 228, **228**, 229, 424,
 425, 432, 434, 482, 483, 497, 566, 569
 CODE 36, 45, 52, 57, 58, **64**, 65, 94, 98, 133,
 137, 145, 162, 163, 166, 175, 212, 221,
 293, 295, 297, 300, 305, 306, 307, 336,
 390, 392, 394, 396, 397, 398, 400, 402,
 403, 405, 407, 460, 488, 522, 526, 529,
 531, 532, 536, 539, 543, 572, 573, 606,
 634
 COLDEF 504, **505**, 505
 COLNAME **505**
 COLTYPE **505**
 COMMISSION 436, **438**, 440, 441, 444, 461,
 591, 628
 CONFMSG **235**
 CONSUPOSTALCODE 468, **468**
 CORRECTACTION **196**, 196
 CORRECTFITID 67, **196**
 COUNT **489**, 489
 COUNTRY **48**, 50, 51, 114, 118, 120, 130, 132,
 147, 148, 233, 320, 324, 325, 327, 328,
 387, 468, 470, 478, 522, 523, 524, 527,
 528, 530, 531, 547, 555, 570, 598
 COUPONFREQ **428**, 566, 569, 577, 578
 COUPONRT **428**
 CREDITCARDMSGSET **280**, 281, **289**
 CREDITCARDMSGSETV1 42, 280, 281, **289**
 CREDITCARDMSGSETV2 42, 280, 281, **289**
 CREDITCARDMSGSRQV1 280
 CREDITCARDMSGSRQV2 280
 CREDITCARDMSGSRSV1 281
 CREDITCARDMSGSRSV2 281
 CREDITLIMIT **204**
 CSPHONE **118**
 CURDEF **87**, 87, 88, 191, 194, 200, 202, 207,
 212, 221, 235, 293, 295, 297, 300, 308,
337, 349, 390, 392, 400, 407, 435, 460,
 558, 562, 607, 635

CURRATE 88, **88**, 554
 CURRENCY 63, **87**, 87, 88, **88**, 197, 201, 204,
 208, 427, 440, 441, 443, 444, 445, 448,
 451, 552, 607, 617, 635, 641
 CURSYM 88, **88**, 607, 635

D

DATEBIRTH **130**, 132, 528
 DAYPHONE **130**, 132, 147, 148, 478, 528,
 530, 531, 598
 DAYSTOPAY 324, **333**, 333, 339, 384, 385,
 386, 391, 392, 405, 407, 588
 DAYSWITH 287, **384**, 384, 385, 386, 576, 590,
 628
 DEBADJ **204**
 DEBTCLASS **428**
 DEBTINFO 426, **428**, 428, 566, 569
 DEBTTYPE **428**
 DENOMINATOR **438**, 445
 DEPANDCREDIT **201**
 DEPMAILRS **260**, 260
 DESC **63**, 136, 137, 462, 569
 DETAILAVAILABLE 489, **491**, 534, 597
 DFLTDAYSTOPAY 287, **384**, 384, 385, 386,
 590, 628
 DIFFFIRSTPMT **385**, 387
 DIFFLASTPMT **385**, 387
 DISCOUNT 330, **330**, 330, 569
 DISCOUNT2 330, **331**, 331
 DOMXFERFEE **290**, 290, 291
 DSCAMT **330**, 330, 331
 DSCDATE **330**, 330, 331, 569
 DSCDESC **330**, 330, 331, 569
 DSCRATE **330**, 330, 331
 DTACCTUP 50, 51, 134, **134**, 134, 135, 137,
 293, 480, 481, 482, 522, 526, 529, 539,
 540, 556, 573, 575
 DTASOF 63, 191, 194, 294, **427**, 433, 435,
 460, 462, 552, 566
 DTAUCTION **449**
 DTAVAIL **187**, 195, 324, 387
 DTBILL **490**, 490, 533, 534, 536
 DTCALL **428**
 DTCHANGED **55**
 DTCLIENT 20, **48**, 292, 521, 525, 528, 605
 DTCLOSE **201**, 204, 490

DTCOUPON **428**
 DTREATED **156**, 161, 162, 163
 DTDUE 171, 172, **187**, 187, 232, 235, 300,
 301, 322, 324, 333, 351, 384, 385, 386,
 389, 390, 391, 392, 393, 394, 395, 396,
 399, 400, 401, 402, 407, 576, 588, 589,
 602, 627, 632
 DTDUEBY **485**, 485, 618
 DTEFF **492**, 494
 DTEND **69**, 69, 70, 190, 191, 193, 194, 199,
 201, 202, 204, 292, 293, 433, 435, 460,
 484, 485, 488, 533, 535, 536, 568, 606,
 612, 634, 638
 DTEXPIRE 131, 133, 320, **430**, 464, **495**, 529,
 533, 534, 536, 614
 DTINFOCHG **148**
 DTMAT **428**
 DTNEXT **201**, 204
 DTOPEN **201**, 204, 490
 DTPLACED **448**, 462
 DTPMTDUE 71, **204**, 490, 533, 534, 536
 DTPMTPRC **334**, 340, 391, 393, 399, 407, 563
 DTPOSTED **195**, 212, 221, 235, 293, 294, 461,
 585, 625
 DTPOSTEND 201, **201**, 204
 DTPOSTSTART 201, **201**, 204
 DTPRICEASOF **451**, 461, 462
 DTPROFUP 50, 51, **116**, 117, 293, 522, 526,
 529, 555, 573
 DTPURCHASE **438**, 445, 567, 577
 DTSEEN **506**, 506
 DTSERVER **50**, 293, 522, 526, 529, 605, 633
 DTSETTLE **437**, 460
 DTSTART **69**, 69, 190, 191, 193, 194, 199,
 201, 202, 204, 292, 293, 433, 435, 459,
 460, 484, 485, 488, 532, 533, 535, 536,
 538, 606, 612, 634, 638
 DTTRADE **437**, 460
 DTUPDATE 467, **468**, 469, 522, 526, 617
 DTUSER **195**, 207, 208, 259, 260, 294, 461
 DTXFERPRC **188**, 188, 307, 308, 613, 639
 DTXFERPRJ 212, **212**, 212, 221, 235, 296, 309,
 585, 613, 625, 639
 DTYIELDASOF **429**, **431**
 DURATION **448**, 463

E

EMAIL 61, **118**, **130**, 132, 147, 148, 528
 EMAILMSGSET 167, **167**, 557, 584
 EMAILMSGSETV1 42, **167**
 EMAILMSGSETV2 42, **167**
 EMAILPROF 286, **288**, 288, 520
 ENROLLRQ 47, 127, **129**, **130**, 132, 139, 140,
 146, 475, 479, 528, 556, 605, 610, 633
 ENROLLRS **131**, 133, 475, 529, 556, 610, 637
 ENROLLTRNRQ 130, 132, 475, 528
 ENROLLTRNRS 131, 133, 529
 EVEPHONE **130**, 132, 147, 148, 478, 528, 598
 EXTBANKACCTTO **181**, **182**, 182, 543
 EXTBANKDESC 232, **233**, 233, 235, 560
 EXTDPAYEE 318, **333**, 333, 337, 339, 349,
 364, 367, 384, 385, 386, 390, 392, 405,
 407, 563, 564, 567, 576, 588, 614, 626
 EXTDPMT 311, 321, 322, 324, **329**, 329, 563,
 569, 588, 594, 595, 614, 630, 640
 EXTDPMTCHK **329**, 563, 569
 EXTDPMTCHK2 **329**
 EXTDPMTDSC 329, **329**, 329, 563, 569, 595,
 614, 621, 640, 643
 EXTDPMTDSC2 329, **329**, 329, 595, 614
 EXTDPMTFOR **329**, 563, 569
 EXTDPMTINFO 321
 EXTDPMTINV 329, **330**, 330, 332, 563, 569,
 595

F

FAXPHONE **118**
 FEE **207**, 234, **235**, **259**, 260, 298, 562, 591,
 628
 FEEMSG **207**, 298, 562
 FEES 436, **438**, 440, 441, 444, 447, 591, 628
 FI 20, 49, 51, **53**, 53, 292, 490, 521, 525, 551,
 573, 605, 633
 FIASSETCLASS **428**, **429**, 430, 431
 FICERTID 56, **56**, 56, 578
 FID 20, **53**, 292, 521, 525
 FIID 424, **424**, **427**, 463, 464, 566
 FIMFASSETCLASS **429**
 FINALAMT 348, **349**, 352, 353
 FINAME **117**
 FINCHG **204**

FINDBILLERRQ 47, 466, **467**, 467, 468, 521, 521, 526, 605, 617, 633
 FINDBILLERRS 466, **469**, 469, 522, 526, 600, 631
 FINDBILLERTRNRQ 467, 521, 525
 FINDBILLERTRNRS 469, 471, 522, 526, 600, 631
 FIPTION **429**
 FIRSTNAME **130**, 132, 147, 148, 528
 FITID 66, **67**, 67, 67, 95, 195, 196, 200, 201, 203, 204, 293, 294, 432, 437, 447, 448, 460, 461, 462, 575, 606, 616, 634, 641
 FRACCASH **438**, 445
 FREQ **170**, 170, 171, 172, 299, 300, 351, 399, 400, 401, 402, 544, 558, 589, 627
 FROM **156**, 161, 162, 163

G

GAIN **438**, 441, 442
 GENUSEKEY 46, **48**
 GETMIMERQ 58, 164, **164**, 164, 165
 GETMIMERS 58, 58, 164, **164**, 165, 166, 551
 GETMIMESUP **167**, 167, 557
 GETMIMETRNRQ 165
 GETMIMETRNRRS 166
 GROUPIP 481, 482, **482**, 488, 496, 497, **497**, 497, 538, 539, 600, 631

H

HASEXTDPMT **385**, 386, 595
 HASREXTDPMT **386**, 595
 HELDINACCT **451**, 461, 462
 HELPMESSAGE 469, **470**, 524
 HTML 162, 163, 166

I

IDSCOPE 333, **333**, 391, 392, 405, 407
 IMAGEURL 495, **495**, 533, 534, 536
 INCBAL **433**, 459
 INCIMAGES 156, **157**, 157, 160, 162, 163, 273, 360, 457, 468, 513, 521, 526, 556, 557, 561, 570

INCLUDE 20, 190, **190**, 193, 292, 433, 459
 INCLUDEBILLPMTSTATUS **487**, 487, 618
 INCLUDEBILLSTATUS **486**, 487, 618
 INCLUDECOUNTS 487, 488, 618
 INCLUDEDDETAIL **486**, 487, 532, 535, 538
 INCLUDESTATUSHIST **487**, 487, 618
 INCLUDESUMMARY **487**, 487, 618
 INCOME **443**, 592, 629
 INCOMETYPE **438**, 443, 444
 INCOO **433**, 459, 566
 INCPOS 433, **433**, 459, 566
 INCTAN 20, 190, **190**, 193, 292, **433**, 459
 INITIALAMT 348, **349**, 352, 353
 INTERCANRQ **226**, 226
 INTERCANRS **226**, 226
 INTERMODRQ **223**, 223, 559
 INTERMODRS **224**, 224
 INTERRQ **220**, 220, 228, 247, 250
 INTERRS **221**, 221, 229, 248, 251, 559, 585, 586, 613, 625, 626, 639
 INTERSYNCRQ 97, **265**, 265, 271, 561, 586
 INTERSYNCRS 97, **266**, 266, 561, 586, 626
 INTERTRNRQ 220, 223, 226, **265**
 INTERTRNRS 221, 224, 226, **266**
 INTERXFERMSGSET 282, 283, **290**, 562, 586, 626
 INTERXFERMSGSETV1 42, 282, 283, **290**
 INTERXFERMSGSETV2 42, 282, 283, **290**
 INTERXFERMSGSRQV1 282
 INTERXFERMSGSRQV2 282
 INTERXFERMSGSRSV1 283
 INTERXFERMSGSRSV2 283
 INTLXFERFEE **290**, 290, 291
 INTRACANRQ **217**, 217
 INTRACANRS **217**, 217
 INTRAMODRQ **214**, 214, 287, 559
 INTRAMODRS **215**, 215, 306, 308
 INTRARQ **211**, 211, 239, 242, 294, 299
 INTRARS **211**, 212, 240, 243, 295, 300, 308, 558, 585, 586, 612, 613, 625, 638, 639
 INTRASYNCRQ 97, **263**, 263, 269, 561
 INTRASYNCRS 97, **264**, 264, 561, 586, 625
 INTRATRNRQ 40, 211, 214, 217, **263**, 294
 INTRATRNRRS 212, 215, 217, **264**, 295, 306, 307
 INVACCTFROM 130, 140, 413, **413**, 413, 414, 433, 435, 445, 455, 456, 457, 458, 459, 460, 565

INVACCTINFO 413, **414**, 414, 565, 596
 INVACCTTO 140, **413**, 565
 INVACCTTYPE **414**
 INVALIDACCTTYPE **286**, 562
 INVALIDACCTTYPE2 **286**
 INVBAL 435, **453**, 453, 462, 568, 617, 642
 INVBANKTRAN 435, **436**, 436, 461, 567
 INVBUY **440**, 442, 447, 460, 577, 592, 617, 641
 INVDATE **330**, 330, 331
 INVDESC **330**, 330, 331
 INVEXPENSE **443**, 591, 628
 INVMAILRQ **455**, 455, 513
 INVMAILRS 456, **456**, **514**
 INVMAILSYNCRQ 97, 455, **457**, 457, 568
 INVMAILSYNCRS 97, 455, **458**, 458
 INVMAILTRNRQ **457**
 INVMAILTRNRS **458**
 INVNO **330**, 330, 331
 INVOICE 329, 330, **330**, 330, 331, 490, 588, 594, 595, 596, 597, 626, 630
 INVOICE2 329, 330, **331**, 595, 597, 619
 INVOOLIST **435**, 462, 616, 641
 INVPAIDAMT **330**, 330, 331, 595
 INVPOS **451**, 451, 452, 461, 462, 568
 INVPOSLIST 413, **435**, 461, 616, 641
 INVSELL **440**, 444, 445, 448, 577, 592, 617, 641
 INVSTMTMSGSET **417**, 417, 418, 419, 565, 570
 INVSTMTMSGSETV1 42, 416, **417**, 418, 419
 INVSTMTMSGSETV2 42, 416, **417**, 418, 419
 INVSTMTMSGSRQV1 418, 459
 INVSTMTMSGSRQV2 418
 INVSTMTMSGSRSV1 419, 460
 INVSTMTMSGSRSV2 419
 INVSTMTRQ 432, **433**, 433, 459, 566
 INVSTMTRS 434, **435**, 435, 460, 566, 616, 641
 INVSTMTRNRQ **432**, 432, 459, 566
 INVSTMTRNRS **434**, 434, 460, 566
 INVTOTALAMT **330**, 330, 331
 INVTRAN **437**, 437, 440, 441, 442, 443, 444, 445, 460, 567, 592, 629
 INVTRANLIST **435**, 460
 ITA.CAUSALE **324**

J

JRNLFUND **443**
 JRNLSEC **443**

L

LANGUAGE 20, **48**, 50, 119, 292, 293, 521, 522, 525, 526, 528, 529, 598
 LASTNAME **130**, 132, 147, 148, 528
 LEDGERBAL 87, **191**, 194, 294
 LIMITPRICE **448**, 463
 LINEITEM 330, 331, **332**, 332, 595
 LITMAMT **332**, 332
 LITMCODE 330, **332**, 332, 490
 LITMDESC **332**, 332
 LOAD 436, **438**, 440, 441, 444, 591, 628
 LOGO **470**, 523, 524, 525, 527
 LOSTSYNC 95, 101, 109, 110, **144**, 150, 161, 262, 264, 266, 268, 270, 272, 274, 361, 372, 375, 377, 458, 514, 554, 570

M

MAIL 156, **156**, 158, 161, 162, 163, 256, 257, 259, 260, 357, 358, 455, 456, 511, 557, 611, 638
 MAILRQ **158**, 158, 161, 167, 612, 638
 MAILRS **158**, 158, 160, 162, 163
 MAILSUP **167**, 167, 557
 MAILSYNCRQ 97, 158, **160**, 160, 162, 167, 557, 570, 611, 612, 638
 MAILSYNCRS 97, 98, 158, **160**, 161, 163, 557, 570, 607, 611
 MAILTRNRQ 158, **160**, 161, 557
 MAILTRNRS 158, **161**, 162, 163
 MARGINBALANCE **453**, 462, 568
 MARGININTEREST **443**, 591, 628
 MARKDOWN **438**, 441, 591, 628
 MARKUP **438**, 440, 591, 628
 MAX 47, **121**
 MEMO **68**, 68, 171, 172, 197, 233, 322, 325, 389, 390, 391, 392, 394, 395, 396, 399, 400, 401, 402, 407, 427, 437, 448, 451, 461, 462, 580

MEMO2 **68**, 68, 187, 197, 233, 325, 427, 437, 448, 451, 580
 MESSAGE 36, 36, **64**, 65, 131, **151**, 158, 552, 606, 634
 MESSAGE2 **64**, **152**, 466
 MFASSETCLASS **429**
 MFINFO 426, **429**, 429, 566
 MFTYPE **429**
 MIDDLENAME **130**, 132, 147, 148, 528
 MIN 26, **121**
 MINAMTDUE **490**
 MINPMTDUE **204**
 MINUNITS **448**
 MKTGINFO **191**, 194, 201, 204, 436
 MKTVL **451**, 461, 462
 MODELWND 287, 346, **384**, 386, 615, 616, 640, 641
 MODPENDING **242**, **243**, 250, 251, 351, 352, 353, **366**, 401, 402, 560, 589, 615, 627
 MSGBODY **156**, 156, 162, 163, 611, 638
 MSGSETCORE 42, 57, 79, 81, 114, 119, **119**, 121, 123, 151, 152, 167, 286, 289, 290, 291, 325, 384, 385, 417, 420, 519, 555, 574, 578, 583, 625
 MSGSETLIST 57, **117**, 123, 151, 167, 621, 643
 MULTIINTERTRNRQ 228, **228**, 228, 265, 290, 585
 MULTIINTERTRNRS **229**, 229, 266, 585, 613, 639

N

N 501, **501**
 NAME 63, 196, 197, 207, 208, 233, 312, 327, 328, 333, 340, 351, 389, 390, 391, 392, 393, 394, 395, 396, 404, 405, 407, 461, 462, 468, 469, 477, 522, 523, 524, 526, 552, 566, 570, 575, 598
 NAMEACCTHELD 320, **478**, 530, 531, 598
 NEEDTANPAYEE **387**
 NEEDTANPMT **387**
 NEEDTANTRANSFER **287**, 600, 631
 NEWUNITS **438**, 445
 NEWUSERPASS **54**, 58, 84, 86, 573
 NINSTS **170**, 171, 172, 351, 399, 400, 401, 402, 558, 589, 627
 NONCE 56, **56**, 578

NOTIFYDESIRED **490**, 505, 533, 534, 536
 NOTIFYWILLING **486**, 505, 532, 535, 538, 598
 NUMERATOR **438**, 445

O

OFX 19, 20, 33, 34, 36, **37**, 37, 40, 41, 47, 48, 113, 161, 166, 292, 293, 294, 295, 296, 297, 299, 300, 304, 305, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 459, 460, 476, 521, 522, 525, 526, 528, 529, 530, 531, 532, 535, 538, 539, 596
 OFXSEC **81**, 81, **119**
 OLDUNITS **438**, 445
 ONETIMEPASS **48**
 OO **448**, 448, 449, 462, 567
 OOBUYDEBT **449**
 OOBUYMF **449**
 OOBUYOPT **449**
 OOBUYOTHER **449**
 OOBUYSTOCK **449**, 462
 OODNLD **417**, 417
 OOSELLDEBT **449**
 OOSELLMF **449**
 OOSELLOPT **449**
 OOSELLOTHER **449**
 OOSELLSTOCK **449**
 OPTACTION **438**, 442
 OPTBUYTYPE **438**, 442, 449
 OPTINFO 426, **430**, 430, 463, 566
 OPTIONLEVEL **414**
 OPTSELLTYPE **438**, 444, 449
 OPTTYPE **430**, 464
 ORG 20, **53**, 53, 292, 521, 525
 ORIGCURRENCY **87**, 88, **88**, 197, 201, 204, 208, 440, 441, 443, 444, 445, 607, 617, 635, 641
 OTHERENROLL 151, **151**, 152, 557
 OTHERINFO 426, **430**, 430, 566

P

PARVALUE **428**

PAYACCT 171, 172, **322**, **324**, 336, 337, 363, 364, 365, 366, 367, 369, 389, 390, 391, 392, 393, 394, 395, 396, 399, 400, 401, 402, 404, 405, 407, 544, 564, 567, 590, 628
 PAYANDCREDIT **204**
 PAYEE 196, 197, 312, 314, 315, 321, 323, 324, **327**, 327, 336, 339, 340, 351, 363, 364, 365, 366, 367, 389, 390, 393, 394, 395, 396, 404, 405, 563, 570, 588, 614, 626
 PAYEE2 197, 324, **327**, 328, 363, 364, 366, 367, 509, 588, 614
 PAYEEDELQR 38, **369**, 369, 370, 564, 590, 628
 PAYEEDELRS **370**, 370, 564
 PAYEEID 171, 172, 196, 314, **316**, 316, 321, 323, 333, 336, 339, 340, 351, 363, 364, 390, 391, 392, 399, 400, 401, 402, 405, 407, 576, 587, 588, 614, 626, 640
 PAYEEID2 67, 196, **316**, 324, 333, 363, 476, 477, 490, 509, 587, 614, 617
 PAYEELSTID 314, 315, **316**, 316, 317, 318, 321, 336, **337**, 339, 340, 349, 351, 364, 365, 366, 367, 369, 370, 390, 392, 392, 400, 402, 405, 407, 576, 587, 590, 614, 615, 626, 628, 639, 640
 PAYEELSTID2 67, 315, **316**, 324, **337**, 349, 364, 366, 367, 369, 370, 476, 477, 478, 587, 617
 PAYEEMODPENDING 366, 616
 PAYEEMODRQ 315, 339, **365**, 365, 366, 369, 564, 570, 590, 615, 628
 PAYEEMODRS 314, 315, 339, 340, 351, **367**, 367, 564, 570
 PAYEERQ 315, 317, 339, **363**, 363, 404, 564, 576, 589, 627
 PAYEERS 315, 340, 352, **364**, 364, 405, 564
 PAYEESYNCRQ 97, **143**, 143, **149**, 149, 318, **371**, 371, 564
 PAYEESYNCRS 97, 98, **144**, 144, **150**, 150, 314, 339, **372**, 372, 564, 595, 607
 PAYEETRNRQ 38, 363, 365, 369, **371**, 404
 PAYEETRNR 318, 364, 367, 370, **372**, 405
 PAYINSTRUCT **232**, **235**
 PAYMENTINSTRUMENT 473, **474**, 474, 522, 523, 524, 525, 527
 PAYMENTINSTRUMENTS 470, 473, **473**, 473, 474, 522, 523, 524, 527
 PERCENT **429**, 429
 PHONE 136, 137, 233, 327, 328, 389, 390, 393, 394, 395, 396, 404, 405, 470, 522, 523, 524, 527, 569, 598
 PINCH **122**, 122, 569, 574, 582, 610, 637
 PINCHRQ **54**, 54, 55, 58, 84, 86, 105, 122, 546, 551, 573, 605, 609, 637
 PINCHRS **54**, **55**, 55, 58
 PINCHTRNRQ **54**, 58
 PINCHTRNRS **54**, 58
 PMTBYPADDR **385**, 386
 PMTBYPAYEEID **385**, 386
 PMTBYPXFER **385**, 386
 PMTCANCRQ 38, 318, 335, **343**, 343, 397
 PMTCANCRS 175, 318, 343, **344**, 344, 397, 615, 640
 PMTFOR **325**
 PMTINFO 171, 172, **321**, 321, 333, 336, 337, 338, 339, 340, 341, 343, 348, 349, 350, 351, 352, 353, 355, 357, 358, 387, 389, 390, 391, 392, 393, 394, 395, 396, 399, 400, 401, 402, 407, 509, 545, 563, 576, 588, 589, 594, 614, 615, 626, 627, 630, 640
 PMTINFO2 **321**, 323, 336, 337, 341, 348, 349, 352, 353, 357, 358, 588, 594, 595, 614, 630, 640
 PMTINQRQ 38, 317, 318, **345**, 345, 398
 PMTINQRS **345**, 345, 398, 563
 PMTINQTRNRQ 38, 345, 398
 PMTINQTRNRS 345, 398
 PMTINSTRUMENTTYPE 474, **474**, 474, 522, 523, 524, 525, 527, 617
 PMTMAILRQ **357**, 357, 564
 PMTMAILRS **358**, 358, 564
 PMTMAILSYNCRQ 97, 357, **360**, 360, **511**, 564
 PMTMAILSYNCRS 97, 98, **361**, 361, 564, 607
 PMTMAILTRNRQ 357, **360**
 PMTMAILTRNRS 358, **361**
 PMTMODRQ 38, 311, 314, 315, 317, 335, **340**, 340, 341, 393, 395, 563, 615, 640
 PMTMODRS 317, 318, 340, **341**, 341, 343, 351, 373, 394, 396, 563
 PMTPRCCODE 334, 336, **340**, 391, 393, 398, 407, 563
 PMTPRCSTS **334**, 334, 336, **337**, 341, 343, 345, 391, 392, 398, 407, 563, 601, 632

PMTRQ 38, 110, 314, 315, 317, 318, **336**, 336, 339, 343, 363, 364, 367, 389, 391, 615, 640
 PMTRS 109, 110, 317, 318, 321, 336, **336**, 336, **337**, 339, 340, 373, 390, 392, 406, 407, 563, 576, 614, 615, 640
 PMTSYNCRQ 95, 97, 109, 110, 175, 318, **374**, 374, 378, 406, 564
 PMTSYNCRS 97, 109, 110, 175, 317, 318, 343, **375**, 375, 378, 406, 564
 PMTTRNRQ 38, 110, 336, 341, 343, **374**, 389, 391, 393, 395, 397, 564
 PMTTRNRS 109, 110, 175, 336, 341, 344, **375**, 390, 392, 394, 396, 397, 407
 PMTTYPE **323**, 338, 343, 350, 355, 545
 PORTION **429**
 POSDEBT **452**
 POSDNLD **417**, 417
 POSMF **452**
 POSOPT **452**, 461
 POSOTHER 413, **452**
 POSSTOCK **452**, 461
 POSTALCODE 118, 130, 132, 147, 148, 233, 320, 327, 328, 389, 390, 393, 394, 395, 396, 404, 405, 468, 470, 478, 522, 523, 524, 527, 528, 530, 531, 555, 570
 POSTPROCWND **384**, 386
 POSTYPE **438**, 445, **451**, 461, 462
 PREAUTH 151, 152, **153**
 PREAUTHTOKEN **140**, 153, 597
 PREFETCHURL 495, **495**, 619, 621
 PRESACCTFROM 469, **476**, 476, 477, 478, 490, 496, 499, 506, 511, 513, 514, 533, 534, 536, 540, 596, 617, 619
 PRESACCTINFO 475, **476**, 476, 477, 481, 482, 509, 540, 596, 599
 PRESACCTTO 469, **476**, 476, 478, 479, 530, 531, 596, 597, 617
 PRESBILLINFO 477, 487, 489, **489**, 489, 490, 496, 498, 499, 505, 509, 533, 534, 536, 596, 597, 619
 PRESCOUNTS 487, **488**, 619
 PREDELIVERYID **506**, 506, 507, 619
 PREDETAIL **499**, 597
 PREDETAILRQ 484, 489, 498, **498**, 498, 499, 597
 PREDETAILRS **499**, 499, 500, 597, 598, 600, 631
 PREDETAILTRNRQ 498
 PREDETAILTRNRS 499, 503, 600, 631
 PRESDIRMSGSET 515, 516, **519**
 PRESDIRMSGSETV1 42, 466, 515, 516, **519**, 620
 PRESDIRMSGSRQV1 466, 516, 521, 525
 PRESDIRMSGSRSV1 466, 516, 522, 526
 PRESDIRPROF **519**, 600, 631
 PRESDLVMSGSET 515, 517, 518, **519**, 598
 PRESDLVMSGSETV1 26, 27, 42, 466, 484, 517, 518, **519**
 PRESDLVMSGSRQV1 484, 517, 532, 535, 538, 539
 PRESDLVMSGSRSV1 484, 518, 532, 536, 539, 543
 PRESDLVPROF **519**, 598, 600, 620, 631
 PRESGRPACCTINFOTRNRQ 480, **481**, 481, 482, 539, 618
 PRESGRPACCTINFOTRNRS 481, **482**, 482, **483**, 539, 600, 618, 631
 PRESLIST 488, **489**, 489, 533, 536, 619
 PRESLISTRQ 484, **484**, 485, 488, 489, 495, 496, 497, 505, 509, 532, 535, 538, 597, 598, 618
 PRESLISTRS 325, 484, **488**, 488, 496, 497, 533, 536, 598, 600, 618, 631
 PRESLISTTRNRQ 484, 485, 488, **496**, 496, 497, 532, 535, 538, 619
 PRESLISTTRNRS 488, 496, **497**, 497, 498, 532, 536, 598, 600, 631
 PRESMAILRQ **511**, 511
 PRESMAILRS **511**, 511, 600, 631
 PRESMAILSYNCRQ 97, 510, **513**, 513, 598
 PRESMAILSYNCRS 97, 510, **514**, 514, 598
 PRESMAILTRNRQ 511, **513**
 PRESMAILTRNRS 511, 512, **514**, 600, 631
 PRESNAMEADDRESS 477, **478**, 478, 479, 530, 531, 597, 598, 618
 PRESNOTIFYRQ **506**, 506, 508, 520
 PRESNOTIFYRS **507**, 507, 600, 619, 631
 PRESNOTIFYTRNRQ 506
 PRESNOTIFYTRNRS 507, 509, 600, 631
 PREVBAL **490**, 491
 PROCDAYSOFF 287, 288, 291, 384, 386, 519, **520**, 520, 562, 565, 590, 600, 628, 631
 PROCENDTM 287, 288, 291, **384**, 386, **519**, **520**
 PROFMSGSET 123, **123**, 600, 631, 643

PROFMSGSETV1 42, 113, **123**
 PROFMSGSETV2 42, 113, **123**, 621, 643
 PROFMSGSRSV2 621, 643
 PROFRQ 47, **116**, 555, 605, 633
 PROFRS **117**, 117, **117**, 515, 519, 555, 570,
 574, 578, 583, 609, 625
 PROFTRNRQ 116
 PROFTRNRS 117
 PURANDADV **204**
 PWTYPE 48, **122**, 122, 610, 621, 637, 643

R

RATING **427**
 REASON **136**, 185, 476, 540, 593, 594, 596
 RECINTERCANRQ **253**, 253
 RECINTERCANRS **253**, 253
 RECINTERMODRQ **250**, 250, 560
 RECINTERMODRS **251**, 251
 RECINTERRQ **247**, 247
 RECINTERRS **248**, 248, 613, 639
 RECINTERSYNCRQ **271**, 271, 561
 RECINTERSYNCRS 97, **272**, 272, 561
 RECINTERTRNRQ 247, 250, 253, **271**
 RECINTERTRNRS 248, 251, 253, **272**
 RECINTRACANRQ **245**, 245, 304
 RECINTRACANRS **245**, 245, 305
 RECINTRAMODRQ **242**, 242, 287
 RECINTRAMODRS **243**, 243
 RECINTRARQ **239**, 239, 299
 RECINTRARS **240**, 240, 300, 586, 625
 RECINTRASYNCRQ 97, **269**, 269, 304, 561,
 577
 RECINTRASYNCRS 97, **270**, 270, 304, 561
 RECINTRATRNRQ 239, 242, 245, **269**, 299,
 304, 575
 RECINTRATRNRRS 240, 243, 245, **270**, 300,
 305
 RECPMTCANCRQ 174, **355**, 355, 403, 563,
 589, 627
 RECPMTCANCRS 174, **356**, 356, 403, 563
 RECPMTMODRQ 314, 315, **351**, 351, 352,
352, 401, 563, 589, 615, 627, 640
 RECPMTMODRS 315, 351, 352, **353**, 353,
 366, 402, 563
 RECPMTRQ 171, 314, 315, **348**, 348, 399,
 563, 615, 640

RECPMTRS 66, 172, 321, **349**, 349, 351, 400,
 563, 576, 614, 615, 640
 RECPMTSYNCRQ 97, **376**, 376, 378, 564
 RECPMTSYNCRS 97, **377**, 377, 378, 564, 595
 RECPMTTRNRQ 348, 352, 355, **376**, 399,
 401, 403, 564
 RECPMTTRNRS 349, 353, 356, **377**, 400, 402,
 403
 RECSRVRTID 66, 172, 172, 174, 174, 212,
 221, 240, 242, 243, 245, 248, 250, 251,
 253, 300, 304, 305, 309, 315, 316, **337**,
 349, 352, 353, 355, 356, 400, 401, 402,
 403, 586, 613, 625, 639
 RECSRVRTID2 67, 212, 221, 240, 242, 243,
 245, 248, 250, 251, 253, **337**, 349, 352,
 353, 355, 356
 RECURRENST **170**, 170, 171, 172, 239, 240,
 242, 243, 247, 248, 250, 251, 299, 300,
 348, 349, 351, 352, 353, 399, 400, 401,
 402, 558, 589, 627
 REFNUM **196**, 221
 REFRESH 99, 100, **100**, 107, 120, 143, 149,
 160, 261, 263, 265, 267, 269, 271, 273,
 360, 371, 373, 374, 376, 457, 513, 543,
 554, 557, 558, 561, 564, 568, 570, 582,
 607, 608, 624, 635, 636
 REFRESHSUPT 26, 27, **120**, 120, 609, 637
 REINVC **452**
 REINVDIV **452**, 452, 569
 REINVEST **444**, 592, 629
 REJECTIFMISSING 95, 96, 98, 99, **100**, 109,
 110, 143, 149, 160, 162, 175, 261, 263,
 265, 267, 269, 271, 273, 304, 360, 371,
 374, 376, 406, 457, 513, 543, 544, 554,
 557, 558, 561, 562, 564, 565, 568, 570
 RELFITID 67, **438**, 442, 444
 RELTYPE **438**, 444
 RESPFILEER **120**, 574
 RESTRICT **470**, 524
 RESTRICTION **448**, 463
 RETOFCAP **444**, 591, 628
 REVERSALFEES **438**, 440, 441, 448, 591, 616
 REVERSALFITID 67, **439**, 440, 441, 447, 591,
 616

S

- SECID **423**, 423, 424, **424**, 426, 427, 430, 440, 441, 442, 443, 444, 445, 448, 449, 451, 460, 461, 462, 463, 464, 565, 566
- SECINFO **427**, 427, 428, 429, 430, 431, 463, 566, 568, 570
- SECLIST 422, 425, **426**, 426, 463, 591, 616, 628, 641
- SECLISTMSGSET **420**, 420, 421, 422
- SECLISTMSGSETV1 42, **420**, 421, 422
- SECLISTMSGSETV2 42, **420**, 421, 422
- SECLISTMSGSRQV1 40, 421, 422, 426
- SECLISTMSGSRQV2 40, 421, 422, 426
- SECLISTMSGSRSV1 40, 422, 426, 463
- SECLISTMSGSRSV2 40, 422, 426
- SECLISTRQ **424**, 424, 425, 565
- SECLISTRQDNLD **420**, 420
- SECLISTRS **425**, 425, 577
- SECLISTTRNRQ **424**, 424, 426, 565
- SECLISTTRNRS **425**, 425, 566
- SECNAME **427**, 463, 464, 566, 568, 570
- SECRQ **424**, 570
- SECURED **439**, 444, 452
- SECURITYNAME **130**, 132, 528
- SELLALL **449**
- SELLDEBT **444**
- SELLMF **444**
- SELLOPT **444**, 567
- SELLOTHER 413, **445**, 592
- SELLREASON **439**, 444
- SELLSTOCK **445**, 591, 628
- SELLTYPE **439**, 444, 445, 449
- SESSCOOKIE 46, 47, **49**, 51, 551
- SEVERITY 36, 36, 43, 58, 64, **64**, 133, 137, 145, 162, 163, 166, 175, 293, 295, 297, 300, 305, 306, 307, 390, 392, 394, 396, 397, 398, 400, 402, 403, 405, 407, 460, 522, 526, 529, 531, 532, 536, 539
- SHORTBALANCE **453**, 462, 568, 617, 642
- SHPERCTRCT **430**, **439**, 442, 444, 464
- SIC 196, **468**, 470, 522, 523, 524, 527
- SIGNONINFO 47, 117, **121**, 122, 555, 570, 574, 578, 583, 625
- SIGNONINFOLIST **117**, 609, 636
- SIGNONMSGSET 57, **57**, 115, 551, 573
- SIGNONMSGSETV1 42, **57**, 114, 578
- SIGNONMSGSETV2 42, **57**, 466, 596
- SIGNONMSGSRQV1 20, 40, 46, 292, 294, 296, 299, 301, 302, 304, 305, 389, 391, 393, 395, 397, 398, 399, 401, 403, 404, 406, 459, 521, 525, 578
- SIGNONMSGSRQV2 528, 530, 532, 535, 538, 539
- SIGNONMSGSRSV1 46, 293, 295, 297, 300, 304, 306, 390, 392, 394, 396, 397, 398, 400, 402, 403, 405, 406, 460, 522, 526, 578
- SIGNONMSGSRSV2 529, 531, 532, 535, 539
- SIGNONREALM 119, **121**
- SIGNUPMSGSET **151**, 556, 584
- SIGNUPMSGSETV1 42, **151**
- SIGNUPMSGSETV2 42, **152**
- SIGNUPMSGSRQV1 39, 125, 570
- SIGNUPMSGSRQV2 39, 125, **126**, 127, 153, 481, 528, 530, 604, 610
- SIGNUPMSGSRSV1 39, 570
- SIGNUPMSGSRSV2 39, 125, **126**, 529, 531, **610**
- SONRQ 20, 40, **46**, 46, 47, **48**, 48, 51, 54, 55, 79, 84, 86, 106, 107, 109, 115, 125, 129, 161, 292, 294, 296, 299, 301, 302, 304, 305, 389, 391, 393, 395, 397, 398, 399, 401, 403, 404, 405, 406, 459, 460, 466, 476, 482, 496, 497, 521, 525, 528, 530, 532, 535, 538, 539, 547, 551, 573, 580, 596, 602, 604, 605, 633
- SONRS **46**, 46, 49, **50**, 50, 51, 107, 293, 295, 297, 300, 304, 306, 390, 392, 394, 396, 397, 398, 400, 402, 405, 406, 460, 522, 526, 529, 530, 531, 532, 535, 538, 539, 551, 573, 578, 580, 598, 602, 604, 605, 633
- SPACES **122**, 610
- SPECIAL **122**, 610
- SPLIT **445**
- SPNAME 50, **120**, 196, 316, **325**, 325, 326, 476, 477, 573, 574, 583, 612, 614, 617, 625
- SRVRTID **66**, 66, 67, 95, 98, 99, 103, 175, 196, 212, 214, 215, 217, 221, 223, 224, 226, 235, 236, 237, 240, 248, 295, 300, 306, 308, 315, 316, 317, 318, **337**, 341, 343, 344, 345, 357, 358, 378, 379, 390, 392, 393, 394, 395, 396, 397, 398, 407, 437, 448, 580, 586, 606, 613, 625, 634, 639

SRVRTID2 **66**, 66, 67, 196, 212, 214, 215, 217,
 221, 223, 224, 226, 235, 236, 237, **337**,
 341, 343, 344, 345, 357, 358, 437, 448,
 493, 580, 606, 634
 STATE 117, 130, **132**, 147, 148, 233, 320, 327,
 328, 389, 390, 393, 394, 395, 396, **404**,
 405, 468, 470, 478, 522, 523, 524, 526,
 527, 528, 530, 531
 STATUS 36, 43, 44, 45, 50, 52, 57, 58, **64**, 64,
 94, 98, **109**, 117, 131, **133**, **137**, **145**,
 162, 163, 166, 175, 212, 221, 229, 293,
 295, 297, 300, 305, 306, 307, 336, 390,
 392, 394, 396, 397, 398, **400**, **402**, **403**,
 405, 407, 425, 434, 460, 466, 472, 483,
 488, 497, 522, 526, 529, 531, 532, 536,
 539, 550, 552, 554, 566, 572, 573, 580,
 601, 606, 612, 632, 634, 638
 STATUSMODBY **492**, 494
 STMNTIMAGE 491, **495**, 495, **495**, 495, 533,
 534, 536, 621
 STMTENDRQ 38, 199, **199**, 199, 202
 STMTENDRS 199, **200**, 200, 202, 558
 STMTENDTRNRQ 38, 199
 STMTENDTRNRS 200
 STMTRQ 20, 24, 37, 190, **190**, 190, 192, 201,
 292, 412, 558
 STMTRS 24, 37, 87, 190, **191**, 191, 194, 293,
 412, 558
 STMTTRN 191, 194, **195**, 195, 293, 294, 436,
 461, 502, 558, 575, 585, 612
 STMTTRNRQ 20, 40, 190, 292
 STMTTRNRS 191, 293
 STOCKINFO 426, **431**, 431, 463, 566
 STOCKTYPE **431**
 STOPPRICE **448**
 STPCHKFEE **288**
 STPCHKNUM 207, **208**, 208, 297, 558
 STPCHKPROF 286, **288**, 288, 562, 569
 STPCHKRQ **206**, 206, 296, 544, 558
 STPCHKRS **207**, 207, 297, 558
 STPCHKSYNCRQ 97, **261**, 261, 558, 561
 STPCHKSYNCRS 97, **262**, 262, 561
 STPCHKTRNRQ 206, **261**, 296
 STPCHKTRNRS 207, **262**, 297
 STRIKEPRICE **430**, 464
 STSVIAMODS **384**, 386
 SUBACCT **448**, 463
 SUBACCTFROM **439**, 443
 SUBACCTFUND 436, **439**, 440, 441, 443, 444,
 445, 461
 SUBACCTSEC **439**, 440, 441, 442, 443, 444,
 445, 461
 SUBACCTTO **439**, 443
 SUBJECT **156**, 161, 162, 163
 SUPPORTDTAVAIL **287**, 324, 387, 600, 631
 SUPTXDL 137, **184**, 185
 SVC **139**, 141, 145, 546, 598
 SVC2 **134**, 135, **139**, **141**, 479, 480, 481, 530,
 531, 598, 610, 618
 SVCADD **139**, 139, **140**, 141, 145, 476, 479,
479, 479, 480, 530, 531, 556, 570, 584,
 596, 597, 625
 SVCCADD **139**
 SVCCHG 139, **140**, 141, 476, 479, 556, 570,
 597
 SVCDEL 139, **140**, 141, 556
 SVCSTATUS 133, **136**, 137, 141, 145, 184,
 185, 319, 414, 556, 563, 569, 593, 594,
 596, 597, 599
 SVCSTATUS2 **136**, 136, 141, 184, 185, 319,
 414, 476, 480, 540, 593, 594, 596, 597,
 599
 SWITCHALL **449**
 SWITCHMF **449**
 SYNCERROR 94, **98**, 98, 101, **144**, 150, 161,
 262, 264, 266, 268, 270, 272, 274, 361,
 372, 375, 377, 458, 514, 543, 544, 594,
 595, 607, 608, 611, 612, 613, 616, 617,
 635
 SYNCMODE **120**, 120, 574

T

TABLENAME 500, **500**, 533, 536
 TABLETYPE **500**, 504
 TAN 43, 44, 228, **228**, 228, 387, **424**, 424, 432,
 482, 497, 565, 566, 569
 TAXES 436, **439**, 440, 441, 444, 591, 628
 TAXEXEMPT **439**, 441, 443, 444, 570
 TAXID **130**, 132, 528, 552
 TEMPPASS **131**, 131, 133, 529
 TFERACTION **439**, 445
 TICKER 424, **424**, **427**, 463, 464, 566
 TO **156**, 161, 162, 163

TOKEN 65, **68**, 68, 92, 93, 94, 95, 96, 98, 99,
100, 101, 107, *109*, *110*, 142, 143, 144,
148, 149, 150, 159, 160, 161, *162*, 162,
162, *163*, *175*, 209, 213, 216, 218, 222,
225, 227, 230, 236, 237, 241, 244, 246,
249, 252, 254, 257, 258, 261, 262, 263,
264, 265, 266, 267, 268, 269, 270, 271,
272, 273, 274, *304*, 338, 342, 344, 346,
350, 354, 356, 359, 360, 361, 365, 368,
370, 371, 372, 374, 375, 376, 377, 379,
406, 456, 457, 458, 543, 544, 554, 557,
558, 562, 565, 580, 582, 607, 624, 635
TOKEN2 **68**, 68, 100, 143, 144, 149, 150, 160,
161, 261, 262, 263, 264, 265, 266, 267,
268, 269, 270, 271, 272, 273, 274, 360,
361, 371, 372, 374, 375, 376, 377, 457,
458, 466, 513, 514, 580
TOKENONLY 100, **100**, 143, 149, 160, 261,
263, 265, 267, 269, 271, 273, 360, 371,
374, 376, 457, 513, 543, 554, 557, 558,
561, 564, 568, 570, 582, 624
TOTAL **439**, 440, 441, 442, 443, 444, 461, 591,
592, 628, 629
TOTALFEES **201**
TOTALINT **201**
TRANDNLD **417**, 417
TRANSFER **445**, 567, 577, 578
TRANSPSEC **79**, 79, **119**, 121, 574
TRNAMT **68**, 68, *171*, *172*, 187, 195, 207, 208,
232, 235, 259, 260, 293, 294, 295, 296,
300, *301*, *306*, *308*, *309*, 321, 323, 351,
389, 390, 391, 392, 393, 394, 395, 396,
399, 400, *401*, *402*, *407*, 461, 594
TRNTYPE 195, **198**, 293, 294, 461, 612, 638
TRNUID 20, 43, 44, 45, 58, **65**, 65, 66, 95, 99,
101, 103, 105, *109*, *110*, *132*, *133*, *137*,
145, 155, 158, *161*, *162*, *163*, *165*, *166*,
228, 229, 292, 293, 294, 295, 296, 297,
299, *300*, *304*, *305*, *306*, *307*, 315, 378,
379, 389, 390, 391, 392, 393, 394, 395,
396, 397, 398, 399, *400*, *401*, *402*, *403*,
404, *405*, *406*, *407*, 424, 425, 432, 434,
459, 460, 482, 483, 497, *521*, *522*, *526*,
528, 529, 530, 531, 532, 535, 536, 538,
539, 542, 550, 552, 554, 606, 634
TSKEYEXPIRE **50**
TSPHONE **118**
TYPEDESC **430**

U

UNIQUEID **423**, 460, 461, 462, 463, 464
UNIQUEIDTYPE **423**, 460, 461, 462, 463, 464
UNITPRICE 413, **427**, **439**, 440, 441, 444, 445,
451, 461, 462, 567, 577, 591, 628
UNITS 413, **439**, 440, 441, 442, 443, 444, 445,
447, **448**, **451**, 461, 462, 463, 591, 592,
628, 629
UNITSSTREET **452**, 452, 592, 629
UNITSUSER **452**, 452, 592, 629
UNITTYPE **439**, 449
URL 58, 114, **118**, **119**, 121, **151**, 164, 165,
165, 166, 581
URL2 114, **118**, **119**, **152**, 164, 165, 466, 581
URLGETREDIRECT **118**
USEHTML 156, **157**, 157, 158, 160, *162*, *163*,
273, 360, 457, 513, 557, 561, 570, 611,
638
USERID 20, 39, 47, 48, **48**, 54, 55, 56, 58, 115,
125, 126, 127, 128, 130, 131, *132*, *133*,
147, 148, 153, 156, *161*, *162*, *163*, 292,
316, 476, 477, 481, 482, 488, 496, 497,
521, *521*, 525, 525, 528, 529, 530, 531,
533, 535, 536, 538, 540, 547, 551, 596,
605, 610, 617, 618, 619, 633
USERKEY 46, 48, **48**, 50, 551
USERPASS 20, 47, 48, **48**, 84, 86, 115, 128,
292, 521, *521*, 525, 525, 528, 546, 551,
573, 605, 633
USPRODUCTTYPE 89, 414, **415**, 415

V

VALIDATE 469, **470**, 472, 525
VALUE **63**, 462
VER **119**, 119, 121, 574

W

WEBENROLL 151, **151**, 152, 557
WIREBENEFICIARY 232, **233**, 233, 235, 560
WIRECANRQ **236**, 236
WIRECANRS **237**, 237
WIREDSTBANK 232, **232**, 235, 560
WIRERQ **232**, 232, 560

WIRERS **234**, 235, 560, 585, 625
WIRESYNCRQ 97, **267**, 267, 561
WIRESYNCRS 97, **268**, 268, 561, 600, 631,
643
WIRETRNRQ 232, 236, **267**
WIRETRNRS 235, 237, **268**
WIREXFERMSGSET **284**, 285, **291**, 562, 587,
626
WIREXFERMSGSETV1 42, 275, 284, 285, **291**
WIREXFERMSGSETV2 42, 284, 285, **291**
WIREXFERMSGSRQV1 284
WIREXFERMSGSRQV2 284
WIREXFERMSGSRSV1 285
WIREXFERMSGSRSV2 285
WITHHOLDING **439**, 441, 443, 591, 628

X

XFERDAYSWITH **384**, 385
XFERDEST 137, **184**, 185
XFERDFLTDAYSTOPAY 324, **384**, 386
XFERINFO **186**, 186, 187, 211, 212, 214, 215,
219, 220, 221, 223, 224, 239, 240, 242,
243, 247, 248, 250, 251, 287, 294, 295,
299, 300, 306, 308, 559, 585
XFERPRCCODE 188, **188**, 188, 212, 221, 307,
308
XFERPRCSTS **188**, 188, 212, 215, 221, 224,
307, 308, 558, 559, 601, 632
XFERPROF 286, **287**, 287, 290, 562, 569, 586,
600, 631
XFERSRC 137, **184**, 185

Y

YIELD **429**, **431**, 463
YIELDTOCALL **428**
YIELDTOMAT **428**